Python 101: A Function-by-Function Code Walkthrough

Data preparation, demand estimation, and dynamic pricing

November 9, 2026

Contents

Server-only empirical execution **14**

Python 101: learning to read this research code **15**
1. A program is a sequence of instructions . . . 15
2. Values, types, and containers . . . 16
3. Functions: inputs, work, and outputs . . . 16
4. Branches, loops, and scope . . . 17
5. Arrays: many numbers with one operation . . . 17
6. Masks and comparisons . . . 18
7. pandas tables and Polars query expressions . . . 18
8. Strings, regular expressions, and paths . . . 19
9. Numerical optimization and symbolic algebra are different . . . 19
10. Modules, entry points, and tests . . . 19
11. A tiny trace you can do on paper . . . 20

Current workflow: where to return after a long break **21**
The shortest reading path . . . 21
Keep the completed data build . . . 21
Two specifications are two separate analyses . . . 21
What novelty means here . . . 22
What the firm chooses and what is held fixed . . . 22
Fixed beliefs describe decisions; persistent types describe consumers . . . 22
Read failure states as results of checks . . . 23
Recover only the result boundary . . . 23

Worked reading lessons: unpacking the intimidating lines **24**
Lesson 1: a table pipeline is a sequence of ordinary calls . . . 24
Lesson 2: masks express a sentence one cell at a time . . . 25
Lesson 3: from named coefficients to a utility matrix . . . 25
Lesson 4: softmax, log probabilities, and zero counts . . . 25
Lesson 5: one household must keep its latent type across weeks . . . 26
Lesson 6: understanding the gradient's einsum without memorizing it . . . 26
Lesson 7: first-stage least squares does not mean matrix inversion . . . 27
Lesson 8: symbolic expressions are not hidden numerical functions . . . 27
Lesson 9: closures and the expanding price bracket . . . 27
Lesson 10: practice before editing the real scripts . . . 28

Current empirical workflow: separate switching and habit **29**
Lesson 11: two coefficients mean two independently estimated effects . . . 29
Lesson 12: a population distribution is not an estimation confidence interval . . . 29
Lesson 13: recover both household effects, but rank only by switching . . . 30
Lesson 14: focal sellers and demand alternatives are different lists . . . 31
Lesson 15: read backward induction in the order it is solved . . . 31
Lesson 16: reconstruct deleted inputs without estimating demand again . . . 32

Lesson 17: restricting focal sellers does not restrict demand alternatives . . . 32
Lesson 18: one category list controls several aligned arrays . . . 32
Lesson 19: novelty is a score for an alternative before choosing it . . . 33
Lesson 20: decisions and evaluation need not use the same averaging order . . . 33
Lesson 21: an exception can reject one optional component without erasing another . . . 33
Lesson 22: restarting after data is an explicit sequence, not a hidden resume . . . 34

data_code: source walkthrough **35**
data_code/cleaning_merging/master_dat.py . . . 35
Module setup and direct-execution blocks . . . 35
data_code/cleaning_merging/panel_concat.py . . . 44
Module setup and direct-execution blocks . . . 44
data_code/cleaning_merging/panel_convert.py . . . 46
Module setup and direct-execution blocks . . . 46
data_code/cleaning_merging/panel_dat.py . . . 52
Module setup and direct-execution blocks . . . 52
data_code/cleaning_merging/panel_merge.py . . . 52
Module setup and direct-execution blocks . . . 52
data_code/cleaning_merging/panel_week.py . . . 57
Module setup and direct-execution blocks . . . 57
data_code/cleaning_merging/retail_concat.py . . . 59
Module setup and direct-execution blocks . . . 60
data_code/cleaning_merging/retail_convert.py . . . 63
Module setup and direct-execution blocks . . . 63
data_code/cleaning_merging/retail_dat.py . . . 70
Module setup and direct-execution blocks . . . 70
data_code/cleaning_merging/retail_merge.py . . . 70
Module setup and direct-execution blocks . . . 70
data_code/recover_flavors.py . . . 76
Module setup and direct-execution blocks . . . 76
main . . . 77
data_code/summ_stats.py . . . 78
Module setup and direct-execution blocks . . . 78
normalize upcs . . . 80
normalized upc expr . . . 81
valid flavor labels . . . 82
scan purchases . . . 83
single person purchases . . . 86
repeated yogurt households . . . 87
household summary . . . 88
as lazy . . . 91
partition purchases . . . 92
household batches . . . 94
save table . . . 95
demographic tables . . . 96
scan reference . . . 99
recover flavor reference . . . 99
flavor reference . . . 103
yogurt weeks . . . 107
flavor summaries . . . 109
make hhi figures . . . 115
make figures . . . 117
run . . . 120
main . . . 128

est_code: source walkthrough **132**

est_code/bootstrap_se.py . . . 132
Module setup and direct-execution blocks . . . 132
resample households . . . 133
bootstrap standard errors . . . 134
est_code/est.py . . . 138
Module setup and direct-execution blocks . . . 138
normalize upcs . . . 140
normalized upc expr . . . 141
read flavors . . . 142
week dates . . . 148
prepare data . . . 149
add stocks . . . 160
make arrays . . . 162
make draws . . . 165
parameter names . . . 166
mixture components . . . 169
mixture loglike score . . . 170
history utility . . . 172
household loglike . . . 173
demand objective gradient . . . 176
optimize demand . . . 180
optimize demand.objective . . . 184
optimize demand.progress . . . 185
plot gamma distribution . . . 186
plot history distributions . . . 190
fit model . . . 191
fit model.estimate . . . 196
main . . . 197
est_code/est_gamma.py . . . 201
Module setup and direct-execution blocks . . . 201
est_code/est_het_gamma.py . . . 202
Module setup and direct-execution blocks . . . 202
est_code/est_het_history.py . . . 202
Module setup and direct-execution blocks . . . 202
est_code/est_het_naive.py . . . 203
Module setup and direct-execution blocks . . . 203
est_code/est_mixture_gamma.py . . . 203
Module setup and direct-execution blocks . . . 203
est_code/est_naive.py . . . 204
Module setup and direct-execution blocks . . . 204
est_code/est_no_price_control.py . . . 205
Module setup and direct-execution blocks . . . 205
est_code/est_separate_history.py . . . 205
Module setup and direct-execution blocks . . . 205
est_code/hessian_se.py . . . 206
Module setup and direct-execution blocks . . . 206
observed covariance . . . 206
hessian standard errors . . . 209
est_code/history_distributions.py . . . 210
Module setup and direct-execution blocks . . . 210
export history distributions . . . 211
est_code/price_control.py . . . 215
Module setup and direct-execution blocks . . . 216
check dma . . . 216
other dma instruments . . . 217
absorb effects . . . 219

add price controls . . . 221
est_code/regenerate_weekly_inputs.py . . . 224
Module setup and direct-execution blocks . . . 224
regenerate . . . 226
main . . . 232
est_code/wtp.py . . . 233
Module setup and direct-execution blocks . . . 233
implied wtp . . . 233
implied wtp.add . . . 235

supply_code: source walkthrough **237**
supply_code/comparative_statics/___init___.py . . . 237
Module setup and direct-execution blocks . . . 237
supply_code/comparative_statics/alpha_p1.py . . . 237
Module setup and direct-execution blocks . . . 237
supply_code/comparative_statics/alpha_p2.py . . . 237
Module setup and direct-execution blocks . . . 238
supply_code/comparative_statics/beta_p1.py . . . 238
Module setup and direct-execution blocks . . . 238
supply_code/comparative_statics/beta_p1gamma.py . . . 238
Module setup and direct-execution blocks . . . 239
supply_code/comparative_statics/beta_p1p2.py . . . 239
Module setup and direct-execution blocks . . . 239
supply_code/comparative_statics/beta_p2.py . . . 239
Module setup and direct-execution blocks . . . 239
supply_code/comparative_statics/beta_profit.py . . . 240
Module setup and direct-execution blocks . . . 240
supply_code/comparative_statics/canonical.py . . . 240
Module setup and direct-execution blocks . . . 240
price2 derivative . . . 242
value2 derivative . . . 243
price1 derivative . . . 243
total derivative . . . 244
terminal total derivative . . . 245
profit derivative . . . 246
supply_code/comparative_statics/delta_p1.py . . . 247
Module setup and direct-execution blocks . . . 247
supply_code/comparative_statics/delta_p2.py . . . 248
Module setup and direct-execution blocks . . . 248
supply_code/comparative_statics/gamma2_p1.py . . . 248
Module setup and direct-execution blocks . . . 248
supply_code/comparative_statics/gamma2_p2.py . . . 248
Module setup and direct-execution blocks . . . 248
supply_code/comparative_statics/gamma_p1.py . . . 249
Module setup and direct-execution blocks . . . 249
supply_code/comparative_statics/gamma_p1beta.py . . . 249
Module setup and direct-execution blocks . . . 249
supply_code/comparative_statics/gamma_p1p2.py . . . 250
Module setup and direct-execution blocks . . . 250
supply_code/comparative_statics/gamma_p2.py . . . 250
Module setup and direct-execution blocks . . . 250
supply_code/comparative_statics/gamma_p2beta.py . . . 250
Module setup and direct-execution blocks . . . 250
supply_code/comparative_statics/gamma_profit.py . . . 251
Module setup and direct-execution blocks . . . 251
supply_code/comparative_statics_symbolic.py . . . 251

Module setup and direct-execution blocks . . . 251
supply_code/export/___init___.py . . . 252
Module setup and direct-execution blocks . . . 252
supply_code/export/latex_export.py . . . 252
Module setup and direct-execution blocks . . . 252
display equation . . . 252
supply_code/export/pretty_print.py . . . 253
Module setup and direct-execution blocks . . . 253
show . . . 253
supply_code/make_comparative_static_figures.py . . . 254
Module setup and direct-execution blocks . . . 254
state outcomes . . . 255
plot state outcomes . . . 256
plot state shares . . . 258
save . . . 259
main . . . 260
supply_code/make_latex_document.py . . . 267
Module setup and direct-execution blocks . . . 267
main . . . 268
supply_code/model/___init___.py . . . 275
Module setup and direct-execution blocks . . . 275
supply_code/model/assumptions.py . . . 275
Module setup and direct-execution blocks . . . 275
supply_code/model/numerical.py . . . 276
Module setup and direct-execution blocks . . . 276
inside share . . . 277
optimal price . . . 277
evaluate . . . 279
solve . . . 281
supply_code/model/profit.py . . . 282
Module setup and direct-execution blocks . . . 283
supply_code/model/shares.py . . . 283
Module setup and direct-execution blocks . . . 283
supply_code/model/symbols.py . . . 284
Module setup and direct-execution blocks . . . 284
supply_code/model/utility.py . . . 285
Module setup and direct-execution blocks . . . 285
signed state . . . 286
raw utilities . . . 286
supply_code/pricing/___init___.py . . . 287
Module setup and direct-execution blocks . . . 287
supply_code/pricing/period1_foc.py . . . 287
Module setup and direct-execution blocks . . . 287
supply_code/pricing/period1_price.py . . . 288
Module setup and direct-execution blocks . . . 288
supply_code/pricing/period2_foc.py . . . 288
Module setup and direct-execution blocks . . . 288
supply_code/pricing/period2_price.py . . . 289
Module setup and direct-execution blocks . . . 289
supply_code/run_all.py . . . 289
Module setup and direct-execution blocks . . . 289
supply_code/structural_switching_reference.py . . . 290
Module setup and direct-execution blocks . . . 290
expit safe . . . 291
inside share . . . 292
p2 opt . . . 293

evaluate . . . 294
verify . . . 296
plot series . . . 298
main . . . 299

pricing_exercises: source walkthrough **304**
pricing_exercises/run_empirical_targeting.py . . . 304
Module setup and direct-execution blocks . . . 304
expit . . . 305
logsumexp . . . 306
plotting . . . 307
persistent draws . . . 308
posterior . . . 309
compositions . . . 313
forecast context . . . 313
digest . . . 316
targeted coupons . . . 316
run households . . . 324
make targeting figures . . . 332
self test . . . 338
main . . . 346
pricing_exercises/run_pricing_exercises.py . . . 350
Module setup and direct-execution blocks . . . 350
main . . . 351
pricing_exercises/run_split_targeting.py . . . 352
Module setup and direct-execution blocks . . . 352
expit . . . 354
logsumexp . . . 354
plotting . . . 355
persistent draws . . . 356
posterior . . . 357
compositions . . . 361
forecast context . . . 362
digest . . . 364
targeted coupons . . . 365
run households . . . 373
make targeting figures . . . 381
self test . . . 387
main . . . 388
pricing_exercises/run_supply_exercises.py . . . 392
Module setup and direct-execution blocks . . . 392
expit . . . 393
golden maximum . . . 394
scalar maximum . . . 395
static price . . . 397
static price.foc . . . 399
supply solution . . . 399
supply solution.share1 . . . 404
supply solution.adaptive value . . . 405
supply solution.committed profile . . . 405
supply solution.committed profile.objective . . . 406
plotting . . . 407
outcome tables . . . 408
make figures . . . 410
self test . . . 415
main . . . 416

pricing_exercises/unknown_type/plot_supply_information.py . . . 420
Module setup and direct-execution blocks . . . 420
committed policy . . . 422
committed policy.profile . . . 424
committed policy.profile.objective . . . 426
draw figures . . . 426
run information . . . 428
read command line . . . 432
pricing_exercises/unknown_type/run_unknown_type.py . . . 433
Module setup and direct-execution blocks . . . 433
solve . . . 434
solve.maximize . . . 440
solve.terminal policy . . . 441
solve.terminal policy.expected terminal profit . . . 442
solve.expected profit . . . 443
figures . . . 444
self test . . . 446
main . . . 448

Current readable research workflow (final_simple) 453
../final_simple/data/___init___.py . . . 453
Module setup and direct-execution blocks . . . 453
../final_simple/data/build_data.py . . . 453
Module setup and direct-execution blocks . . . 453
convert one file . . . 456
convert panel files . . . 458
convert retail files . . . 459
clean parquet files . . . 460
combine panel files . . . 461
align retail types . . . 462
combine retail files . . . 463
join panel tables . . . 465
merge panel files . . . 466
add saturday week . . . 467
attach panel weeks . . . 468
join retail tables . . . 469
merge retail files . . . 471
yogurt retail quotes . . . 472
match one year . . . 473
match yogurt files . . . 474
audit file . . . 476
read command line . . . 479
run build . . . 481
../final_simple/data/summaries.py . . . 485
Module setup and direct-execution blocks . . . 485
normalize upcs . . . 488
normalized upc expr . . . 489
valid flavor labels . . . 490
scan purchases . . . 491
single person purchases . . . 494
repeated yogurt households . . . 495
household summary . . . 496
as lazy . . . 499
partition purchases . . . 500
summarize household batches . . . 502
save table . . . 503

demographic tables . . . 504
scan reference . . . 507
recover flavor reference . . . 507
flavor reference . . . 512
yogurt weeks . . . 516
flavor summaries . . . 518
make hhi figures . . . 523
make figures . . . 526
prepare summary samples . . . 529
summarize flavor groups . . . 532
save estimation sample . . . 534
run . . . 537
read command line . . . 540
run summaries . . . 542
../final_simple/demand/__init__.py . . . 544
Module setup and direct-execution blocks . . . 544
../final_simple/demand/estimation.py . . . 545
Module setup and direct-execution blocks . . . 545
check dma . . . 549
other dma instruments . . . 550
absorb effects . . . 552
add price controls . . . 554
normalize upcs . . . 558
normalized upc expr . . . 559
read flavors . . . 560
week dates . . . 566
prepare data . . . 567
switch away event . . . 578
add stocks . . . 579
make arrays . . . 581
make draws . . . 584
parameter names . . . 586
mixture components . . . 588
mixture loglike score . . . 589
history utility . . . 591
household loglike . . . 592
demand objective gradient . . . 595
optimize demand . . . 599
optimize demand.objective . . . 604
optimize demand.progress . . . 604
resample households . . . 605
bootstrap standard errors . . . 606
observed covariance . . . 610
hessian standard errors . . . 612
implied wtp . . . 614
implied wtp.add . . . 616
plot gamma distribution . . . 617
export history distributions . . . 621
plot history distributions . . . 625
write run notes . . . 626
fit model . . . 627
fit model.estimate . . . 632
read command line . . . 633
run estimation . . . 635
../final_simple/demand/gamma_calibration.py . . . 638
Module setup and direct-execution blocks . . . 638

known prices . . . 639
maximize . . . 641
uncertainty . . . 642
uncertainty.shares . . . 644
uncertainty.objective . . . 645
CalibrationError. init . . . 645
calibration rank . . . 646
calibrate . . . 647
calibrate.predictions . . . 649
calibrate.residual . . . 650
run calibration . . . 651
../final_simple/demand/history_distributions.py . . . 656
Module setup and direct-execution blocks . . . 656
main . . . 657
../final_simple/demand/plain_berry_pricing.py . . . 659
Module setup and direct-execution blocks . . . 659
read command line . . . 662
run comparison . . . 663
../final_simple/demand/pricing.py . . . 667
Module setup and direct-execution blocks . . . 667
expit . . . 670
logsumexp . . . 670
persistent draws . . . 671
posterior . . . 672
compositions . . . 676
forecast context . . . 677
targeted coupons . . . 679
digest . . . 688
run households . . . 689
plotting . . . 697
make targeting figures . . . 697
self test . . . 703
write run notes . . . 712
read command line . . . 713
run pricing . . . 714
../final_simple/demand/regenerate_weekly_inputs.py . . . 717
Module setup and direct-execution blocks . . . 717
regenerate . . . 719
main . . . 724
../final_simple/demand/split_pricing.py . . . 725
Module setup and direct-execution blocks . . . 726
expit . . . 729
logsumexp . . . 730
plotting . . . 730
persistent draws . . . 731
posterior . . . 732
compositions . . . 736
compositions.generate . . . 737
forecast context . . . 738
digest . . . 740
targeted coupons . . . 741
information frictions . . . 752
run households . . . 755
make targeting figures . . . 767
self test . . . 772
read command line . . . 774

run pricing . . . 776
../final_simple/flavor_specs.py . . . 780
Module setup and direct-execution blocks . . . 780
categories . . . 780
classify descriptor . . . 781
classification expression . . . 782
../final_simple/server/export_uncontrolled_pricing.py . . . 784
Module setup and direct-execution blocks . . . 784
export results . . . 785
export results.digest . . . 791
../final_simple/server/run_after_data.py . . . 791
Module setup and direct-execution blocks . . . 791
run pipeline . . . 793
run pipeline.call . . . 801
../final_simple/supply/___init___.py . . . 801
Module setup and direct-execution blocks . . . 801
../final_simple/supply/pricing_model.py . . . 801
Module setup and direct-execution blocks . . . 802
expit . . . 802
golden maximum . . . 803
scalar maximum . . . 804
static price . . . 806
static price.foc . . . 808
supply solution . . . 808
supply solution.share1 . . . 813
supply solution.adaptive value . . . 814
supply solution.committed profile . . . 814
supply solution.committed profile.objective . . . 815
plotting . . . 816
outcome tables . . . 817
self test . . . 819
../final_simple/supply/supply_information.py . . . 821
Module setup and direct-execution blocks . . . 821
committed policy . . . 823
committed policy.profile . . . 825
committed policy.profile.objective . . . 826
draw figures . . . 827
run information . . . 829
read command line . . . 832
../final_simple/supply/supply_simulation.py . . . 834
Module setup and direct-execution blocks . . . 834
expit . . . 838
golden maximum . . . 839
scalar maximum . . . 840
static price . . . 842
static price.foc . . . 843
supply solution . . . 844
supply solution.share1 . . . 849
supply solution.adaptive value . . . 849
supply solution.committed profile . . . 850
supply solution.committed profile.objective . . . 851
outcome tables . . . 851
plotting . . . 854
make case figures . . . 855
inside share . . . 859
optimal price . . . 860

evaluate . . . 861
solve . . . 863
state outcomes . . . 864
plot state outcomes . . . 865
plot state shares . . . 867
save . . . 869
make derivative figures . . . 870
self test . . . 876
write run notes . . . 878
read command line . . . 878
run supply . . . 880
../final_simple/supply/supply_sympy.py . . . 885
Module setup and direct-execution blocks . . . 885
signed state . . . 891
raw utilities . . . 892
price2 derivative . . . 892
value2 derivative . . . 893
price1 derivative . . . 894
total derivative . . . 896
terminal total derivative . . . 897
profit derivative . . . 898
display equation . . . 900
show . . . 900
make latex appendix . . . 901
../final_simple/supply/unknown_type.py . . . 908
Module setup and direct-execution blocks . . . 908
solve . . . 910
solve.maximize . . . 916
solve.terminal policy . . . 917
solve.terminal policy.expected terminal profit . . . 918
solve.expected profit . . . 919
figures . . . 920
self test . . . 922
read command line . . . 924
run unknown type . . . 925

Tests and verification: worked calling examples **929**
data_code/test_summ_stats_memory.py . . . 929
Module setup and direct-execution blocks . . . 929
MemorySummaryChecks.test single person records precede repeat week rule . . . 930
MemorySummaryChecks.test batches preserve global statistics . . . 931
MemorySummaryChecks.test demographics match previous fieldwise sort . . . 935
MemorySummaryChecks.test recovery preserves existing labels and uses earliest year . . . 937
est_code/test_est.py . . . 938
Module setup and direct-execution blocks . . . 939
EstimationChecks.frame . . . 940
EstimationChecks.test history excludes current counts and accumulates all occasions . . . 940
EstimationChecks.test xi sign and unlimited history . . . 941
EstimationChecks.test parquet pipeline and fit . . . 943
EstimationChecks.test parquet pipeline and fit.append . . . 947
EstimationChecks.test unavailable flavor is excluded from logit denominator . . . 948
EstimationChecks.test persistent household integration . . . 950
EstimationChecks.test separate history nests combined index . . . 952
EstimationChecks.test no control ignores residuals but keeps price . . . 953
EstimationChecks.test analytic gradient . . . 955
EstimationChecks.test mixture combines household likelihoods and nests normal . . . 956

EstimationChecks.test mixture fit outputs . . . 958
EstimationChecks.test four model specifications . . . 960
EstimationChecks.test mixed naive integrates only alpha . . . 961
EstimationChecks.test zero heterogeneity nests homogeneous model . . . 963
est_code/test_hessian_se.py . . . 964
Module setup and direct-execution blocks . . . 964
HessianChecks.test correlated information and household scaling . . . 964
HessianChecks.test boundary and indefinite information withheld . . . 966
est_code/test_price_control.py . . . 967
Module setup and direct-execution blocks . . . 967
synthetic panel . . . 968
PriceControlChecks.test excludes consumer dma even when exposure store elsewhere . . . 970
PriceControlChecks.test absorption matches explicit dummy ols . . . 972
PriceControlChecks.test first stage residuals orthogonal and in utility . . . 973
PriceControlChecks.test first stage uses only covered flavor prices . . . 975
PriceControlChecks.test resampling preserves histories and relabels duplicates . . . 977
PriceControlChecks.test bootstrap refits first stage and reports empirical sd . . . 978
PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate . . . 980
PriceControlChecks.test controlled demand optimizer runs . . . 981
est_code/test_regenerate_weekly_inputs.py . . . 982
Module setup and direct-execution blocks . . . 982
RecoveryTests.check recovery . . . 983
RecoveryTests.test recovery preserves parameters . . . 985
RecoveryTests.test mismatch preserves existing export . . . 985
est_code/test_summary_stats.py . . . 986
Module setup and direct-execution blocks . . . 986
SummaryChecks.test demographics and mixed baskets . . . 987
est_code/test_wtp.py . . . 990
Module setup and direct-execution blocks . . . 990
WTPChecks.table . . . 991
WTPChecks.test fixed alpha and signed history . . . 991
WTPChecks.test mixture moments include between component variation . . . 992
WTPChecks.test separate history wtp has no fixed weight . . . 993
WTPChecks.test beta averages ratios . . . 994
supply_code/checks/___init___.py . . . 995
Module setup and direct-execution blocks . . . 995
supply_code/checks/check_alpha.py . . . 995
Module setup and direct-execution blocks . . . 995
supply_code/checks/check_beta.py . . . 996
Module setup and direct-execution blocks . . . 996
supply_code/checks/check_delta.py . . . 996
Module setup and direct-execution blocks . . . 996
supply_code/checks/check_focs.py . . . 997
Module setup and direct-execution blocks . . . 997
supply_code/checks/check_gamma.py . . . 997
Module setup and direct-execution blocks . . . 997
supply_code/checks/check_numerical.py . . . 998
Module setup and direct-execution blocks . . . 998
supply_code/checks/check_pricing_derivatives.py . . . 998
Module setup and direct-execution blocks . . . 998
supply_code/checks/test_symmetric.py . . . 999
Module setup and direct-execution blocks . . . 999
SymbolicChecks.equal . . . 1000
SymbolicChecks.test raw normalization and both states . . . 1001
SymbolicChecks.test focs from actual profit . . . 1002
SymbolicChecks.test envelope and implicit gamma derivatives . . . 1003

SymbolicChecks.test all parameter implicit derivatives . . . 1004
NumericalChecks.test reference on entire benchmark grid . . . 1005
NumericalChecks.test independent reoptimization and derivatives . . . 1006
NumericalChecks.test independent reoptimization and derivatives.negative profit . . . 1008
NumericalChecks.test all parameter and mixed derivatives numerically . . . 1009
NumericalChecks.test all parameter and mixed derivatives numerically.arguments at . . . 1011
NumericalChecks.test zero switching static case and symmetry . . . 1012
pricing_exercises/test_empirical_timing.py . . . 1013
Module setup and direct-execution blocks . . . 1013
EmpiricalTimingTests.test first observation becomes t0 . . . 1014
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 1017
pricing_exercises/test_split_targeting.py . . . 1020
Module setup and direct-execution blocks . . . 1020
EmpiricalTimingTests.test first observation becomes t0 . . . 1021
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 1024
SplitPosteriorTests.fixture . . . 1028
SplitPosteriorTests.test posterior matches scalar and estimator . . . 1029
SplitPosteriorTests.test all focal exports and integrated types . . . 1033
../final_simple/checks/test_split_pricing.py . . . 1035
Module setup and direct-execution blocks . . . 1035
EmpiricalTimingTests.test first observation becomes t0 . . . 1036
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 1040
SplitPosteriorTests.fixture . . . 1044
SplitPosteriorTests.test posterior matches scalar and estimator . . . 1046
SplitPosteriorTests.test three focal cases both type treatments . . . 1049
PlainBerryComparisonTests.test only focal sellers change and full demand is retained . . . 1050
PlainBerryComparisonTests.test only focal sellers change and full demand is retained.solve . . . 1052
PlainBerryComparisonTests.test only focal sellers change and full demand is retained.capture . . . 1053
../final_simple/checks/test_regenerate_weekly_inputs.py . . . 1053
Module setup and direct-execution blocks . . . 1054
RecoveryTests.check recovery . . . 1054
RecoveryTests.test recovery preserves parameters . . . 1057
RecoveryTests.test mismatch preserves existing export . . . 1057
../final_simple/checks/check_flavor_specifications.py . . . 1057
Module setup and direct-execution blocks . . . 1058
module . . . 1065
mathematical data . . . 1066

Coverage, rebuilding, and next steps **1068**

Server-only empirical execution

The code now enforces a server boundary before empirical data I/O. Earlier local data-pipeline exercises in this book are superseded: do not run them on this computer. Only pure numerical/theoretical exercises and results-only work are local. See `Code/privacy/README.md` for enforcement and the non-data check. The current source listings include the guards; rebuilding this book parses code without executing empirical workflows.

Python 101: learning to read this research code

This is a separate companion to the economic and pipeline guides. Those guides explain the models. This book teaches you to read their implementation, beginning with Python punctuation and ending with each function and script in `data_code`, `est_code`, `supply_code`, and `pricing_exercises`, together with the current readable workflow across `final_simple`. You do not need to understand all the economics before learning what a line of Python does.

The book contains the actual source, source line references, function-specific explanations, statement-by-statement reading notes, alternatives, and small examples. Nested helper functions and test methods are included. Scripts with no functions receive the same treatment at module level. Empty package files are identified explicitly. Source excerpts are a snapshot of the working tree at build time; they are not edited versions of the executable research code.

Current edition, November 9, 2026. Begin with the current-workflow chapter. Lessons 18–22 explain alongside three/five categories, the original novelty interpretation, uncontrolled demand, fixed-belief decisions versus persistent-type revenues, and recovery after the auxiliary calibration failure. The source chapters now include every production Python module in `final_simple`, including data summaries, shared flavor rules and server orchestration. Older combined-gamma and `final_draft` chapters are retained as historical comparisons, not current submission instructions.

Read the foundations first, then the small `make_draws` and `signed_state` functions. After that, try `add_stocks`, `resample_households`, and `optimal_price`. Return to the long preparation functions after you can recognize loops, columns, masks, and returned values. The table of contents and PDF bookmarks let you find any function by file and name. Appendix tests are useful worked examples of how to call the code without the private datasets.

1. A program is a sequence of instructions

Python executes statements in order. An assignment evaluates its right-hand side, then binds its result to the name on its left:

```
packages = 3
price = 1.50
spending = packages * price
```

`spending` is now 4.5. The equals sign is an instruction to store a result; it is not a mathematical claim to solve. `spending == 4.5` instead asks a question and produces the Boolean value `True`.

Names such as `weekly` and `frame` are ordinary variable names chosen by the author. They have no special meaning to Python. A name can later refer to a different object. `frame = frame.copy()` first copies the old table and then rebinds `frame` to the new one. Assigning `other = frame` does not copy the table; it gives the same object another name.

`#` begins a comment outside a string. Python ignores the rest of that line. Triple-quoted text at the start of a module, class, or function is a docstring: it is documentation retrievable with `help`, not a hidden calculation.

2. Values, types, and containers

Example	Type	How this code uses it
`3`	Integer, `int`	Counts, loop limits, seeds
`3.0`, `.7`, `1e-6`	Floating-point number, `float`	Prices, coefficients, tolerances
`"berry"`	String, `str`	Column labels, model names, paths
`True`, `False`	Boolean, `bool`	Settings and validation
`None`	The special absence value	Optional input not supplied
`["berry", "plain"]`	List	Ordered, changeable sequence
`("j", "0")`	Tuple	Ordered collection, often used for fixed choices
`{"price": 1.5}`	Dictionary, `dict`	Named values looked up by key
`{"upc", "flavor"}`	Set	Unordered distinct labels and membership checks

Square brackets have several jobs. In `[1, 2]` they create a list. In `items[0]` they select an item. In `frame["price"]` they select a named table column. Read the object to the left before deciding what the brackets mean.

Python indexes start at zero. For `names = ["other", "berry", "plain"]`, `names[0]` is other and `names[2]` is plain. `names[:2]` selects positions zero and one; slice stops are excluded. `names[1:]` starts at berry and goes to the end. A NumPy slice such as `prices[:, j]` means every row of column j.

`dict(zip(names, theta))` pairs labels with values. If the labels are `["beta", "gamma"]` and values `[2.0, -.5]`, the result is `{"beta": 2.0, "gamma": -.5}`. Ordinary `zip` stops at the shorter input; the surrounding code is responsible for consistent lengths. `p["gamma"]` requires the key; `p.get("gamma", 0)` supplies zero if it is absent.

`a, b = (10, 20)` unpacks two returned values. `_` is a conventional name for an ignored value; Python does not magically discard it. In `fit, _, _ = ...`, the author keeps the fit and does not use the other returned objects.

3. Functions: inputs, work, and outputs

```python
def package_cost(quantity, price=1.0):
    total = quantity * price
    return total

answer = package_cost(3, price=1.5)
```

`def` defines a function. The name is `package_cost`; parentheses contain its parameters. The colon introduces the body. Indentation determines which statements belong to that body. Defining the function does not yet execute its body. Calling it with parentheses does.

In the call, 3 is a positional argument and `price=1.5` is a keyword argument. The `=` inside a call names an argument; it is not assignment to a variable outside the function. `price=1.0` in the definition supplies a default when the caller omits that argument. `return` ends this call and gives its value back to the caller. Printing 4.5 displays text; returning 4.5 lets another calculation use the number. A function that falls off its end returns `None`.

`def solve(*, initial_state="0", **parameters)` contains two more pieces of syntax: bare `*` makes following arguments keyword-only; `**parameters` collects additional named arguments into a dictionary. In a call, `evaluate(gamma, state, **parameters)` expands such a dictionary back into named arguments. `*command` in a list expands a sequence into its elements.

Functions defined inside another function are nested functions. They can read names from the surrounding call. In `optimize_demand`, the nested `objective` remembers `data`, `draws`, and `args`; SciPy only has to pass the changing parameter vector. This is a closure. It avoids global variables, whose contents could accidentally affect another fit.

`lambda p: p * p` is a compact function returning the square of p. It is handy for a one-expression root equation. A named `def` is clearer if the calculation needs several steps, comments, or reuse. A function object can itself be passed as an argument: `brentq(f, lo, hi)` asks a solver to call f repeatedly.

4. Branches, loops, and scope

```
if quantity <= 0:
    raise ValueError("Quantity must be positive")
else:
    print("Usable quantity")
```

`if` evaluates a condition, `elif` tries another condition when the previous one was false, and `else` handles the remaining case. `raise` stops normal execution by creating an exception. Rejecting unusable counts is preferable to quietly turning them into valid-looking zeros.

`for j, name in enumerate(FLAVORS):` repeats a block once for each flavor, providing both its numeric position and its label. It is equivalent to looping over positions and looking up labels, but makes the correspondence explicit. `range(3)` produces 0, 1, 2. A `while` loop instead keeps going while its condition is true; the price solver uses it because it does not know in advance how many bracket expansions are needed.

`["price_" + f for f in FLAVORS]` is a list comprehension. A slower-to-write but equally valid spelling is:

```
columns = []
for flavor in FLAVORS:
    columns.append("price_" + flavor)
```

An optional `if` at the end filters elements. `{key: value for ...}` builds a dictionary. `(value for ...)` is a generator expression: it yields values as needed rather than first making a complete list. `next(generator, None)` takes the first yielded value, or returns `None` if no match exists.

`try` executes code that might fail. A matching `except` handles named errors. The bootstrap records identification failures rather than accepting them as estimates. Catching every possible exception would also hide programming bugs, so it catches specific expected failure types.

5. Arrays: many numbers with one operation

NumPy arrays are rectangular numerical containers. Their `shape` is a tuple of axis lengths. A three-week, three-flavor table has shape `(3, 3)`. Axis zero indexes rows; axis one indexes columns. Consequently, `.sum(axis=1)` adds across flavors for each week, while `.sum(axis=0)` adds across weeks for each flavor.

```
import numpy as np
counts = np.array([[2., 0., 1.], [0., 4., 0.]])
totals = counts.sum(axis=1)             # [3., 4.]
shares = counts / totals[:, None]
```

`totals[:, None]` changes a shape `(2,)` vector into a `(2, 1)` column. NumPy then broadcasts each row total across its three columns. Without that extra axis, `(2,3) / (2,)` cannot broadcast: the trailing dimensions 3 and 2 conflict. Broadcasting is controlled shape matching, not a guess at intended economics.

Similarly, adding a three-element beta vector to a W-by-3 utility matrix adds the flavor intercepts to every row. Array multiplication `*` is elementwise. Matrix multiplication `@` sums products over the matching inner dimension. For one-dimensional vectors it is the dot product, e.g. `[1,2] @ [3,4] = 11`. `**2` squares a value; `^` is not Python's exponentiation operator.

`np.zeros((W,3))` allocates an array of zeros. `np.zeros_like(prices)` matches another array's shape and dtype. Preallocating avoids repeatedly growing an array inside a long loop. `np.column_stack` joins columns; `np.vstack` stacks rows. `np.bincount(hh, weights=row_ll)` sums contributions by nonnegative integer household label, avoiding a slower Python loop over each household.

`np.nan` represents a missing floating-point value; `np.inf` represents positive infinity. NaN is not equal to itself, so use `np.isnan` or table missing-value methods rather than `x == np.nan`. `np.isfinite` rejects both NaN and infinities.

6. Masks and comparisons

```
covered = np.isfinite(prices) & (prices > 0)
clean_prices = np.where(covered, prices, 0.)
```

A mask is an array of True/False values. `&` combines array conditions with elementwise AND; `|` is elementwise OR; `~` reverses Booleans. Put each comparison in parentheses because Python operator precedence otherwise changes the grouping. `and`, `or`, and `not` instead combine scalar truth values and short-circuit; they generally cannot combine an entire pandas Series unambiguously.

`.any()` asks whether at least one entry is true; `.all()` asks whether every entry is true. `available.any(axis=1)` asks separately for each row. A useful translation is: `((counts == 0) | available).all(axis=1)` means "every flavor was either unpurchased or available." It rejects a purchased flavor without a price.

`np.where` selects values element by element. It does not lazily evaluate its two alternatives: both argument expressions are evaluated before the call. Use safe denominators or explicit indexing when the unused branch would raise or perform an invalid calculation. A zero price placeholder is not a free good when the corresponding alternative is subsequently masked out of utility.

7. pandas tables and Polars query expressions

A pandas `DataFrame` is a table with named columns and row labels; a `Series` is a single labeled column. `frame["price"]` selects one Series; `frame[["price", "quantity"]]` selects a two-column DataFrame. `frame.loc[mask, "price"]` selects specified rows and one column. `frame.iloc[0]` selects by numeric row position. `.copy()` makes an independent table before modifications, avoiding accidental changes to the original.

`.groupby(keys)["quantity"].sum()` splits rows by common keys and sums within each group. `.transform("nunique")` broadcasts a group's distinct count back to every original row. In contrast, an ordinary group aggregation returns one row per group. `.unstack` and `.pivot` turn flavor labels into columns; `.reindex(columns=FLAVORS)` makes their ordering explicit for numerical arrays.

A merge is a lookup using matching key columns. `how="left"` keeps left rows, `inner` keeps matches, `semi` keeps only matching left rows without adding right columns, and `anti` keeps unmatched left rows. A duplicate right key can multiply rows. `validate="many_to_one"` checks the right side is unique; it does not force an invalid table to become unique. Deduplication and validation are different decisions.

Polars uses `pl.col("price")` to build a column expression. Think of it as a recipe, not yet a column of numbers. `pl.lit(0)` is a literal-valued expression; `.alias("price")` names an expression's output. In `pl.when(condition).then(a) .otherwise(b)`, each row chooses an outcome. `.with_columns` adds or replaces columns, while `.select` projects only the requested columns/expressions.

`pl.scan_parquet(path)` builds a lazy plan. `.filter`, `.select`, and `.join` extend that plan. `.collect()` executes it and materializes a table in memory; `.sink_parquet(path)` executes it while writing output. This allows early projection/filtering and streaming execution. It does not guarantee tiny memory: joins, sorts, and exact distinct counts still need state. `.to_pandas()` converts to an eager pandas table, appropriate for the later NumPy computations.

Method chaining means "take the result so far, then call the next method":

```
cleaned = frame.dropna().sort_values("week").reset_index(drop=True)
```

The code often places the chain inside parentheses so it can span physical lines. Leading dots on later lines continue the same expression; they are not new statements. Nested function calls evaluate their arguments before the outer call. Starting with the innermost parentheses is often the easiest reading strategy.

8. Strings, regular expressions, and paths

`"price_" + flavor` concatenates text. `f"Start {s + 1}"` inserts an expression into a formatted string. `:.4f` means four digits after the decimal point, `:,` adds thousands separators, and `:.1%` displays a fraction as a percentage. `flush=True` asks printed progress to reach the log promptly instead of waiting in a buffer.

Regexes describe string patterns. In `r"[0-9]+(?:\.0+)?"`, `[0-9]+` means one or more digits, `(?:...)` groups without capturing, `\.` is a literal dot, and `?` makes the decimal-zero suffix optional. The raw-string prefix r keeps Python from processing backslashes first. `^` and `$` anchor start and end. The UPC normalization removes only a numeric `.0` suffix, not every decimal point or arbitrary letters. String handling avoids rounding a large UPC through a floating-point number.

`Path("/scratch") / "Data"` joins filesystem components. The slash is overloaded by `Path`; here it does not divide numbers. `.exists()` checks presence, `.mkdir(parents=True, exist_ok=True)` creates needed directories, and `.resolve()` normalizes a path for comparisons. A `with TemporaryDirectory() as tmp:` block cleans up the temporary directory on exit, including ordinary exception exits. `with path.open("w") as file:` similarly ensures the file is closed.

9. Numerical optimization and symbolic algebra are different

`minimize(objective, initial, ...)` repeatedly calls a numerical function, changing an array of parameter values to lower its returned loss. `jac=True` means this objective supplies both the number and its derivative vector. `brentq(f, lo, hi)` finds a zero of a continuous scalar function with a sign-changing bracket; it does not minimize the function. A root of a profit derivative is a candidate optimum, whose optimality needs the argument in the economic guide.

`sp.symbols("alpha beta")` creates symbolic mathematical placeholders. It does not create estimated numerical values. `sp.diff(expr, alpha)` differentiates algebra; `.subs({alpha: -1})` substitutes; `sp.simplify` tries to reduce an expression. `sp.lambdify(arguments, expression, "numpy")` turns that symbolic formula into a callable numerical function with an explicit argument order.

A crucial trap: SymPy treats distinct symbols as independent unless dependence is encoded. If `s` is a separate symbol, differentiating `s*p` with respect to p gives s, not `s + p*ds/dp`. The supply code's total-derivative helper manually adds that chain-rule contribution for equilibrium shares and prices.

For likelihood calculations, `logsumexp` computes the logarithm of a sum of exponentials stably. `np.log(np.exp(x).sum())` can overflow even when its final mathematical answer is finite. The optimizer would then receive unusable values. The analytic gradient is exact for the fixed simulated draws, while the simulation integral itself still approximates the population integral.

10. Modules, entry points, and tests

`import numpy as np` imports a library and gives it a short local name. `from price_control import check_dma` imports one object from a neighboring module. Imports can execute a module's top-level code. That is why the legacy cleaning scripts should not be imported merely for inspection: they perform data work at module level.

```
if __name__ == "__main__":
    main()
```

When Python runs a file directly, its `__name__` is `"__main__"`, so this block calls its entry point. When another file imports it, the guard is false and the guarded work is skipped. ArgumentParser translates command-line text into attributes such as `args.draws`; `type=int` converts text to integers and `action="store_true"` creates a Boolean flag.

A class groups behavior. `class Checks(unittest.TestCase):` inherits the testing tools. A method is a function attached to a class; `self` refers to the current instance and is supplied automatically in calls such as `self.frame()`. `self.assertEqual(actual, expected)` fails the test if the values differ. Floating-point results need tolerances: `.1 + .2` is generally not represented exactly as .3. `assert_allclose` compares arrays element by element with stated

absolute/relative tolerances. A passing test confirms its constructed case, not every possible input or the truth of an economic assumption.

11. A tiny trace you can do on paper

```
stock = np.zeros(3)
counts = np.array([0., 2., 0.])
share = counts / counts.sum()
before = stock.copy()
stock = stock + share
```

Initially stock is `[0,0,0]`. Counts sum to 2, so share is `[0,1,0]`. `before` preserves zero history; stock becomes `[0,1,0]`. The order explains why the current purchase does not enter its own pre-choice state. Reversing the last two statements would change the statistical model, even though both versions are valid Python.

Try these questions before looking back: what does `np.arange(4)` return? Why is `frame[["price"]]` different from `frame["price"]`? When does a nested objective run: when it is defined or when SciPy calls it? Does `new = old` copy a table? Does `.all(axis=1)` summarize rows or columns? The answers are 0–3, table versus Series, when called, no, and one result per row after combining columns. These are the building blocks used repeatedly below.

Current workflow: where to return after a long break

This chapter describes the current `final_simple` code. The source chapters that follow also retain earlier implementations; their existence is not an instruction to run them instead of the current job.

The shortest reading path

1. Read `final_simple/flavor_specs.py` to see what each category means.
2. Read `data/summaries.py::run_summaries` and `run` to see how the completed matched data become summaries and labeled estimation inputs.
3. Read `demand/estimation.py::add_stocks`, `parameter_names`, `household_loglike`, and `demand_objective_gradient` for the estimated model.
4. Read `demand/plain_berry_pricing.py::run_comparison`, then `split_pricing.py::targeted_coupons` for the seller's two-period decisions.
5. Read `demand/gamma_calibration.py` for the separate binary information exercise.
6. Read `server/run_after_data.py::run_pipeline` and `export_uncontrolled_pricing.py::export_results` for execution order and recoverable outputs.

The complete production source from `final_simple` is included in this edition, with function and statement annotations. The main job is `server/run_after_data.sh`; the Python driver names its steps explicitly.

Keep the completed data build

The expensive build converts raw panel/retail records, cleans and combines years, merges tables, and matches purchases with prices. These steps remain in `data/build_data.py`. The current request is to reuse their completed outputs and begin again with summaries. No new local data copy is needed or permitted.

On the research server, from the checkout:

```
sbatch Code/final_simple/server/run_after_data.sh \
  /scratch/dtm63837/Kilts_Panel/rebuilds/run_ORIGINAL_JOB_ID
```

The optional second argument selects the full descriptor reference. A recovered reference from the old build is preferred when available. An old three-category mapping alone lacks the detail needed for the new groups.

The driver creates a fresh `after_data_NEW_JOB_ID` child. It estimates both specifications before attempting either pricing stage. The original build and earlier fits stay intact.

Two specifications are two separate analyses

Name	Category order
`three`	other, berry, plain

Name	Category order
`five`	other, berry, plain, other fruits, desserts

Category order matters: the same order must determine count columns, price columns, history arrays, fixed-effect names, and basket coordinates. `categories(specification)` returns an ordered list, not a set. `other` is the reference flavor in both fits, although what remains in that group changes.

The five-group classification gives plain/unflavored first priority, then desserts, then berries, then other fruits, then residual other. Strawberry cheesecake is dessert; peach is other fruit. Cherry/black cherry moves from the legacy berry rule to other fruits in `five`. Descriptor patterns and numeric checks document these choices. Classification rules are not estimates of consumer substitution.

Both runs use uncontrolled heterogeneous split-history demand, the original novelty measure, seven-choice capacity, 128 integration draws, three starts and Hessian inference. Extra categories add fixed effects and alternatives, not extra random novelty/habit coefficients.

What novelty means here

Let w be flavor j's share of last week's yogurt packages. The original predictor is N = 1 - w. Last week's 75% plain / 25% berry basket gives this week's plain N=.25, berry N=.75, and absent categories N=1. The predictor is known before choosing the current basket.

It measures novelty relative to recent exposure. All plain → berry gives berry a full novelty score, and an all-berry current basket removes that score next week. Mixed baskets adjust novelty gradually. This is not a strict one-time reward for every mixed-basket switch. The proposed lagged-switch-away predictor was not adopted.

H sums prior observed weekly own-flavor shares without decay. Positive H coefficients mean reinforcement; negative ones mean satiation. N and H precede current purchases, which update the next week's state. Missing purchase weeks reset next-week N to zero but do not erase H. Purchase records are proxies for exposure, not observed consumption.

What the firm chooses and what is held fixed

The main entry point solves two seller problems, plain and berry. Focal sellers and demand alternatives are different lists: all three or five categories stay in demand even though only plain/berry sellers are reported. Rival prices stay fixed; these are separate counterfactuals, not market equilibrium.

The firm uses recovered household posterior mean price, novelty and habit coefficients. Low/high novelty quartiles receive individually optimized coupons; the middle half receives none by design. These groups also differ in price sensitivity and stock preferences. Coupons range from 0% to 100% in 1% steps, with discount factor .92.

An observed initial basket establishes h0. The firm chooses p1, observes basket h1, and then chooses p2 conditional on h1. Exact basket integration gives 120 states for three inside categories and 792 for five at capacity seven. Every rival's own history remains distinct.

Fixed beliefs describe decisions; persistent types describe consumers

The auxiliary binary theory exercise is separate from coupon pricing. Its uninformed firm uses the same low/middle/high type probabilities after either first-period purchase outcome. It does not learn. Actual consumer types remain fixed over both periods.

For a fixed offer policy, persistent-type evaluation first computes each type's full path revenue, including the probability that this type reaches each second-period state. It then averages across initial types. The informed and uninformed policies must be evaluated over the same initial population.

`profit` retains the optimization objective. `persistent_type_revenue` reports the common policy evaluation. `value_of_information` is known-type revenue minus uninformed-policy revenue under persistent types. This is a different comparison from personalized coupons versus no coupons.

Read failure states as results of checks

The screenshot's exception came from calibrating the separate binary supply model to observed price moments. The original message grouped convergence, boundary and rank failures together. The revised code tries a larger numerical budget, selects the best admissible start, and records the rejection reason and numerical diagnostics. It retains bounds and identification checks.

If that auxiliary calibration remains unsupported, coupon pricing still finishes. The job exports aggregate calibration moments and a failed status; it does not produce information-value figures from a rejected fit. Demand convergence or primary-pricing errors remain failures, and `run_status.json` identifies which specification failed.

Recover only the result boundary

Read the new job's `RESULTS_ONLY/run_status.json` first. Each specification contains summary tables/figures, population estimates and WTP, aggregate pricing and policy benchmarks, provenance, and successfully calibrated information figures. Cross-spec CSVs place estimates and benchmark revenues alongside each other.

Only `RESULTS_ONLY` may be copied locally. Prepared samples, weekly likelihood tables, household histories, reference mappings, data partitions and logs remain on the server. The exporter uses explicit allowed names and verifies parameter hashes and `price_control=false`. Rebuilding this textbook reads source code only and does not cross the empirical boundary.

Worked reading lessons: unpacking the intimidating lines

The file-by-file chapters below are a reference. These short lessons demonstrate how to slow down and reason through a dense expression. The expanded versions are teaching equivalents, not suggested edits to the production code.

Lesson 1: a table pipeline is a sequence of ordinary calls

The estimator contains this expression:

```
weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum()
          .unstack("flavor", fill_value=0)
          .reindex(columns=FLAVORS, fill_value=0).reset_index())
```

Read left to right after identifying the innermost list expression:

1. `keys + ["flavor"]` concatenates the household/week key list with one more column label. The plus sign joins lists; it is not adding column values.
2. `lines.groupby(...)` splits purchase rows into household/week/flavor groups.
3. `["quantity"]` selects the quantity Series from that grouped object.
4. `.sum()` adds package counts within each group. Two separate berry purchases in the same household-week become one berry total.
5. `.unstack("flavor", fill_value=0)` moves the flavor part of the grouped index into columns. An unobserved flavor within an existing week gets zero count.
6. `.reindex(columns=FLAVORS, fill_value=0)` fixes the columns to other, berry, plain, in that order, adding any completely absent flavor with zeros.
7. `.reset_index()` turns household/week index levels into ordinary columns.
8. `weekly = ...` names the resulting table only after the whole chain completes.

You could write intermediate objects instead:

```
group_columns = keys + ["flavor"]
grouped = lines.groupby(group_columns)["quantity"]
totals = grouped.sum()
wide = totals.unstack("flavor", fill_value=0)
ordered = wide.reindex(columns=FLAVORS, fill_value=0)
weekly = ordered.reset_index()
```

Both forms express the same intended pandas operations. Intermediates are useful while learning or debugging because you can inspect each type and shape. Chaining keeps a completed transformation compact once its stages are familiar. It is not intrinsically more statistically correct, and it is not automatically lazy just because it is written across several lines.

For a household with berry purchases 2 and 1 and plain purchase 4 in one week, the final count columns are `(other=0, berry=3, plain=4)`. Index order becomes important only when those columns are converted to position-based arrays.

Lesson 2: masks express a sentence one cell at a time

```
complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Suppose one week has counts `[0,2,1]` and availability `[True,True,False]`. The zero-count comparison gives `[True,False,False]`. OR with availability gives `[True,True,False]`: the plain purchase has no price. `.all(axis=1)` therefore gives False for that row. Although `.any(axis=1)` confirms a nonempty menu, AND combines it with False, so the week is rejected.

If plain count were zero, the OR vector would be all True, so the missing plain price would be acceptable: plain can be excluded from the choice set. This distinguishes "unpriced and unpurchased" from "unpriced but purchased."

The expression could be expanded into a row loop and an inner flavor loop:

```
complete_rows = []
for row_counts, row_available in zip(counts, available):
    nonempty_menu = any(row_available)
    every_purchase_covered = all(
        q == 0 or offered
        for q, offered in zip(row_counts, row_available)
    )
    complete_rows.append(nonempty_menu and every_purchase_covered)
```

The loop uses scalar `or` and `and` because q and offered are individual values. The production expression uses `|` and `&` because its comparisons concern whole arrays. Vectorization moves the element loops into NumPy, but the logical rule remains the same. Parentheses are necessary: do not rely on English-like operator precedence in an array expression.

Lesson 3: from named coefficients to a utility matrix

```
p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
```

The first call obtains parameter labels for this particular model. `zip` pairs each with the coefficient in the same position; `dict` makes the lookup mapping. For a naive three-parameter model, `[.4, -.2, 0.]` corresponds to berry intercept .4, plain intercept -.2, and mean log-minus-alpha zero. It does not mean alpha is zero: the later transform is `-exp(0) = -1`.

With no gamma or price-control effect and one week's prices `[1,1.2,1.5]`, the utility calculation is

```
beta = np.array([0., .4, -.2])
alpha = -1.
prices = np.array([[1., 1.2, 1.5]])
utility = beta + alpha * prices
# utility is [[-1., -.8, -1.7]]
```

Beta has shape `(3,)`, prices `(1,3)`, and the output `(1,3)`. Each flavor's intercept is added to that flavor's price term. Alpha's negative sign means higher prices lower utility. The other-flavor intercept is fixed at zero as a normalization; that does not imply other flavor has zero total utility.

Why not use positions such as `theta[0]` everywhere? That is possible, but a different model inserts/removes coefficients and can change positions. A single ordered label helper plus dictionary lookup makes the interpretation visible. The numeric values remain in an array for efficient optimization.

Lesson 4: softmax, log probabilities, and zero counts

```
logprob = utility - logsumexp(utility, axis=1, keepdims=True)
row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
```

For logit, the logarithm of a probability is utility minus the log of the sum of exponentiated utilities. `axis=1` performs that denominator calculation across alternatives within each week. `keepdims=True` preserves a W-by-1 column so it can be subtracted from the W-by-J utility matrix.

An unavailable alternative has utility minus infinity and probability zero. Its log probability is minus infinity. A zero purchase count should contribute zero to the log likelihood, but floating point computes `0 * -inf` as NaN. The `where` replaces that log probability with zero **where count is zero** before multiplying, protecting the valid zero contribution.

For counts `[0,2,1]`, the row contribution is two times berry log probability plus one times plain log probability. This is why changing an actual package count changes the likelihood's weight; it is not just a single flavor label. Subtracting raw `np.exp(utility).sum()` rather than its log would be the wrong formula, and computing exp then log naively can overflow. `logsumexp` handles the same mathematics in a stable numerical form.

Lesson 5: one household must keep its latent type across weeks

Assume two possible types, equally weighted. Type A gives the household's chosen flavor probability .9 in each of two weeks; type B gives .1 in each. Persistent types imply the history probability

```
likelihood = (.9 * .9 + .1 * .1) / 2
# .41
```

Averaging probabilities independently each week instead gives

```
wrong_history = ((.9 + .1) / 2) * ((.9 + .1) / 2)
# .25
```

The difference is not rounding: it is whether the household can redraw its type between weeks. The estimator first adds weekly log likelihoods within household for each draw, then integrates those full-history likelihoods. `np.bincount` performs the within-household sum. A list accumulates one household vector per draw. The final `logsumexp(..., axis=0) - log(R)` integrates the draw dimension while preserving the household dimension.

This also explains why the gradient uses posterior draw weights. A type assigning higher probability to the entire observed history receives greater weight in that household's score. Those weights are calculation intermediates, not new estimated household coefficients written by the program.

Lesson 6: understanding the gradient's einsum without memorizing it

```
gradient = -np.einsum("rh,rhp->p", weights, np.asarray(scores)) / data["n_households"]
```

The quoted letters label axes; they are not Python variable names. `r` is draw, `h` household, and `p` parameter. Weights has shape `(R,H)`, scores `(R,H,P)`. Multiply matching draw/household entries, sum over r and h because they do not appear after the arrow, and retain p. The resulting vector has length P.

Here is the same sum written in a teaching loop:

```
gradient = np.zeros(P)
for parameter in range(P):
    for draw in range(R):
        for household in range(H):
            gradient[parameter] += (
                weights[draw, household] * scores[draw, household, parameter]
            )
gradient = -gradient / H
```

The negative sign converts a log-likelihood score to a negative-log-likelihood gradient. Division by H matches the mean loss returned to the optimizer. The compact NumPy form avoids Python executing three nested numerical loops, while the teaching loop makes the axis meaning explicit. In this example P/R/H are dimension labels you would obtain from the actual arrays, not existing global constants in the source.

Lesson 7: first-stage least squares does not mean matrix inversion

```
z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
```

Starting inside, `lstsq(controls,z,...)` fits z on the control columns. Its return contains coefficients plus diagnostics, so `[0]` extracts the coefficient vector. Matrix multiplication produces fitted z, then subtraction leaves the part not linearly explained by those controls. `rcond=None` requests NumPy's default singular-value cutoff convention for that calculation.

For a three-row example, let controls be a column of ones and z `[1,2,6]`. Least squares estimates the mean 3, fitted values `[3,3,3]`, and residual `[-2,-1,3]`. The production controls are Xi after fixed-effect absorption, but the projection arithmetic has the same structure.

The alternative formula involving an inverse of `X.T @ X` requires nonsingular cross-products and can be numerically less stable. `lstsq` handles rank deficiency using its numerical decomposition. It cannot create excluded-instrument variation where none exists; the following sum-of-squares check rejects that case.

Lesson 8: symbolic expressions are not hidden numerical functions

```
alpha, beta, gamma = sp.symbols("alpha beta gamma", real=True)
price = sp.Symbol("price", real=True)
index = beta + alpha * price + gamma
direct = sp.diff(index, gamma)
```

`direct` is the symbolic number 1. No household data, optimizer, or probability is involved. If a separate symbol s is introduced, SymPy does not know that s depends on price unless the dependence is encoded. This is why the supply code implements explicit total-derivative rules.

`sp.lambdify((alpha,beta,gamma,price), index, "numpy")` builds a callable expecting four numbers in that order. A call with `(-1,2,.5,1)` gives 1.5. Passing beta where alpha belongs gives a different value without necessarily raising an error. Named dictionaries at output do not remove the need to keep this positional bridge correct. Symbolic algebra improves formula reuse, while numerical tests at asymmetric states catch argument-order mistakes.

Lesson 9: closures and the expanding price bracket

```
f = lambda p: alpha*(p-effective_cost) + 1 + np.exp(
    np.clip(intercept + alpha*p, -745, 700)
)
lo = effective_cost
width = max(2., 2/abs(alpha))
hi = lo + width
while f(hi) > 0:
    width *= 2
    hi = lo + width
```

Only p is written as a lambda parameter; alpha, effective cost, and intercept come from the enclosing call. Evaluating `f(hi)` substitutes the current hi without changing those fixed inputs. Under the maintained negative-alpha domain, f starts positive at lo and eventually becomes negative as p increases. Doubling width searches for a reliable sign-changing bracket rather than guessing a universal maximum price.

Once bracketed, Brent's root method solves f(p)=0. It is solving the transformed FOC, not trying to make the price itself zero. A loop that forgot to update hi could never change its condition; a loop with `width += 2` would expand more slowly for far-away roots. The mathematical guide establishes why this unique FOC root is the optimum for the current unconstrained logit problem.

Lesson 10: practice before editing the real scripts

For each exercise, predict the result before checking the answer:

1. For model `naive`, outside False, control True, what parameter names are returned? Answer: beta_berry, beta_plain, mu_log_minus_alpha, price_control.
2. If a household is selected twice by bootstrap, why change its ID twice? Answer: the two sampled histories must remain separate likelihood contributions.
3. What does `.copy()` protect when changing bootstrap IDs? Answer: the original household subtable that may be selected again in another draw.
4. Is a lazy query just a synonym for a small table? Answer: no; it is a plan, and executing it can still need large intermediate state.
5. In `return fitted, results, bounds`, why do the commas matter? Answer: they construct a tuple the caller can unpack; there are three outputs.
6. Why is saving `.tex` not the same as saving `.pdf`? Answer: the former is source text requiring a TeX engine to typeset it.
7. Can a zero outside count justify adding missing purchase weeks? Answer: no; the implemented sample contains only observed purchase weeks.
8. Does a first-stage F statistic establish exclusion? Answer: no; it describes conditional instrument relevance, while exclusion is an economic assumption.

You can learn most of the syntax by running tiny independent arrays and tables. Read the legacy scripts as text before executing them: several contain top-level filesystem writes and replacements. The following chapters reproduce that source without executing it. The numbered listings are fragments for reading; a fragment beginning inside a function is not necessarily runnable on its own.

Current empirical workflow: separate switching and habit

These lessons update the earlier combined-gamma examples. Begin with the readable files under `Code/final_simple/demand/`; the source chapters later in this book include both these files and their original `final_draft` counterparts. The old model remains a comparison, rather than the current empirical specification.

Lesson 11: two coefficients mean two independently estimated effects

The original combined index was

$$\Xi_{ijt} = (1-\lambda)N_{ijt} - \lambda S_{ijt}, \qquad V_{ijt} = \beta_j - a_i p_{ijt} + \gamma_i \Xi_{ijt} + \rho c f_{ijt}.$$

It forced novelty and own-stock coefficients to move together. The split model instead estimates

$$V_{ijt} = b_0 + \beta_j - a_i p_{ijt} + b_{Ni} N_{ijt} + b_{Si} S_{ijt} + \rho c f_{ijt}.$$

`N` is recent other-flavor purchase shares, `S` is accumulated own-flavor shares, and `a` is positive price sensitivity. A positive novelty coefficient rewards switching away from recently purchased rivals. Positive own-stock effects mean reinforcement; negative ones mean satiation. The name habit does not constrain its estimated sign. These are purchase histories, not observed consumption.

For example, let berry's N be .75 and S be 2, with b_N=.4 and b_S=-.1. The history part of berry utility is $.4.75\text{-}.12{=}.1$. You do not multiply these coefficients by .3 and -.7 again. Those weights belong to the older combined-index model.

In `final_simple/demand/estimation.py`, choose:

```
model = "heterogeneous_history"
capacity = 7
```

The estimator already supports this model. Updating distribution exports does not require changing its likelihood or re-estimating a saved fit. Capacity 7 adds constructed nonpurchase counts; it is a maintained participation convention, not observed purchase-free weeks. Capacity 0 remains available for conditional flavor demand, but the present pricing solver requires positive integer capacity.

Lesson 12: a population distribution is not an estimation confidence interval

The split population model uses independent normal novelty/stock effects and independent lognormal price sensitivity:

```
b_n = mean_novelty + sd_novelty * draws[:, 1]
b_s = mean_own_stock + sd_own_stock * draws[:, 2]
a = np.exp(mu_log_minus_alpha + sd_log_minus_alpha * draws[:, 0])
```

Every row of `draws` defines one persistent type across all weeks. The normal SDs describe variation across consumers, not uncertainty about the estimated population means. Hessian or bootstrap SEs answer the latter question.

For the previously saved controlled split fit, the exported novelty mean/SD are about -.4646/.8466 and own-stock mean/SD about -.01703/.09117. These are existing fit values, not results of a new optimization or new pricing run. Their fitted normal sign shares are about 70.84% and 57.41% below zero respectively. A negative novelty coefficient reverses the recent-other-flavor incentive; it need not imply positive cumulative reinforcement when the two effects are estimated separately.

`export_history_distributions` evaluates each normal's PDF and CDF, stores quantiles, and draws both marginal densities. With SD zero, it records a point mass instead of evaluating a normal density with zero denominator. It writes:

- `history_distribution_summary.csv`: parameters, sign shares and quantiles.
- `history_distribution_grid.csv`: density/CDF values for plotting.
- `history_distributions.pdf` and `.png`: the two population marginals.

The readable saved-fit entry point is `demand/history_distributions.py`. Existing plots are copied under `final_simple/demand/fitted_distributions/`. Their source fits and interpretation are recorded there. No household table is needed merely to export these marginal distributions from saved coefficients.

Lesson 13: recover both household effects, but rank only by switching

A household's posterior type probabilities use its complete history:

$$\ell_{id} = \sum_{t,k} n_{ikt} \log P_{ikt}(a_d, b_{Nd}, b_{Sd}), \qquad w_{id} = \frac{\exp \ell_{id}}{\sum_m \exp \ell_{im}}.$$

The numerical implementation subtracts log-sum-exp rather than exponentiating large log likelihoods directly. Add weeks before integrating draws; averaging weekly probabilities would redraw the person's type every week.

The split pricing script returns a D-by-2 array `history`. Column zero is b_N; column one is b_S. For one household:

```
weights = np.array([.75, .25])
history = np.array([[.4, -.1], [-.2, .3]])
mean_type = weights @ history        # [.25, 0.]
```

The posterior mean switching coefficient is .25 and mean stock effect is zero. The code ranks households by the first mean only, assigns bottom/top quarters, and keeps both means in utility. It does not set stock effects to zero just because stock is absent from the ranking variable. Posterior coefficient uncertainty can be correlated even when population integration draws are independent.

The default firm treats posterior means as known fixed types. The optional posterior-integrated exercise retains the joint draw weights and updates them after the first counterfactual basket. A terminal coupon must optimize revenue averaged over remaining unobserved types; optimizing separately for each true draw would give the firm information it does not have.

Type estimates use the full observed sample, but the first observed basket alone initializes counterfactual history. This is a retrospective exercise with known estimated types, not a prospective holdout forecast. Comparing novelty quartiles does not isolate novelty causally: their habits, prices and initial states differ.

Lesson 14: focal sellers and demand alternatives are different lists

For focal berry, combine other and plain as the outside alternative. Include constructed nonpurchase when present. If utilities are V_o,V_b,V_p,0, then

$$I_{-b} = \log\{e^{V_o} + e^{V_p} + 1\}, \qquad P_b = \text{logistic}(V_b - I_{-b}).$$

This equals berry's original multinomial-logit probability exactly. The outside utility is a log-sum-exp, not the arithmetic mean of rival utilities. With all four utilities zero, each original alternative has probability 1/4; the composite outside has utility log(3), so the binary formula also gives 1/4, not 1/2.

`focal_flavor = "all"` repeats the seller exercise for every category of the saved fit: three or five. The main comparison intentionally selects only plain and berry. Only that run's focal product earns revenue and changes price. These are separate counterfactuals with fixed competitors, not three firms solving a joint equilibrium. The solver internally uses coordinate one for the priced product. `run_households` swaps the focal product's counts, prices, histories, controls and intercept into that coordinate together. Household posterior types are recovered before that swap under the original fitted labels.

Combining rivals in a current choice probability does not erase their histories. Other and plain baskets can produce different future stocks and inclusive values. Keep their identities in basket integration. The seller observes the full basket before the terminal coupon; observing only focal sales would require additional beliefs about which rival flavors were bought.

Lesson 15: read backward induction in the order it is solved

There are three choice dates but two pricing decisions:

1. The first observed basket establishes initial h0 at model t=0.
2. The firm chooses p1(h0); a basket is realized and establishes h1.
3. It observes h1 and chooses terminal p2(h1).

For each basket n and each known estimated type, the terminal value is

$$W_2(n) = \max_{d_2} \; p(d_2) KP_{f,2}(n, d_2), \qquad p(d) = (1-d)p_{\text{posted}}.$$

The initial action solves

$$\max_{d_1} \; p(d_1) KP_{f,1}(d_1) + \delta \sum_n M(n \mid d_1) W_2(n).$$

Read `targeted_coupons` in `final_simple/demand/split_pricing.py`: the outer loop visits one household. STEP A calculates terminal outcomes for all coupons and baskets. STEP B calculates first-action basket probabilities and chooses each basket's best terminal coupon. STEP C picks the best first coupon. The terminal maximization happens before averaging over baskets. Both focal-purchase and no-focal-purchase continuation branches count, including all-nonpurchase baskets.

For basket shares x, update S2=S1+x and N2=sum(x)-x. All-nonpurchase means x=0, leaving stock unchanged and setting next novelty to zero. Capacity 7 yields 120 possible baskets. Updating only at expected shares would generally yield a different value function because logit demand and optimized prices are nonlinear.

The known-type, observed-state timing matches the supply baseline. Empirical binary aggregation is exact at a given full state, but it does not reduce the whole split-stock model to the theory's symmetric +gamma/-gamma one-lag model. The empirical solver also restricts prices to coupons from 0% through 100%, while theory allows unrestricted prices and introductory subsidies. Both new split pricing and theoretical supply default to delta=.92. With zero cost, net revenue is profit. Consumers follow current-period demand without strategically waiting.

Lesson 16: reconstruct deleted inputs without estimating demand again

The weekly export was deleted; regenerate it on the server where its underlying sample lives. `demand/regenerate_weekly_inputs.py` has three conceptual stages:

```
frame = prepare_data(saved_settings)
frame, first_stage = add_price_controls(frame)  # for a controlled fit
# Compare regenerated counts and the fixed-parameter likelihood with fit.json.
# Write weekly_inputs.csv only after both checks pass.
```

This schematic omits the implementation's guards and temporary-file publication. It does not call `optimize_demand`. Reconstructing price controls is necessary to recover the same regressors; it does not replace the saved demand estimates or standard errors. The procedure restores original years, sample selection, history weight, capacity, draw seed/count and model from the saved fit. Moved sample or flavor-reference paths can be overridden explicitly. A mismatch stops rather than silently pairing old coefficients with a different weekly sample.

For pricing alone from an existing completed rebuild, the server command is:

```
sbatch Code/final_simple/server/run_split_pricing.sh \
  /scratch/dtm63837/Kilts_Panel/rebuilds/run_ORIGINAL_JOB_ID
```

It regenerates the uncontrolled fit's matching weekly inputs, verifies the saved likelihood, runs plain/berry pricing and stages allowed results. It does not refit demand. To change category grouping, use the after-data job in lesson 18 instead. Temporary weekly inputs remain on the server and the pricing-only shell removes its job-created inputs on exit. SIGKILL cannot run cleanup handlers.

Reading or rebuilding the book never executes these commands. Begin recovered results with `run_status.json` and `pricing_provenance.json`, then population estimates, aggregate policy CSVs and figures. Full fit records and internal manifests remain on the server. Never recover household tables or weekly inputs. The original novelty measure is retained.

Lesson 17: restricting focal sellers does not restrict demand alternatives

For the main comparison, use `final_simple/demand/plain_berry_pricing.py`. Its selection is:

```
FOCAL_FLAVORS = ("plain", "berry")
```

This tuple controls the seller problems that are solved, not which products are in the estimated choice set. The shared solver still uses all three or five demand categories and both estimated history coefficients. For focal plain, berry and other form the composite outside alternative; for focal berry, plain and other do so. Constructed nonpurchase remains outside too. There is no refit and no new flavor-specific switching distribution: household coefficients remain shared across flavors as estimated.

Plain and berry are easier to interpret as focal products than the broad other aggregate. The all-category focal script remains a sensitivity exercise. The two-focal entry point creates only plain/ and berry/ result folders and a combined comparison table. It changes neither competitor prices jointly nor the underlying demand sample. Run it on the server after weekly-input regeneration, using the same fit directory and weekly table belonging to that category specification.

Lesson 18: one category list controls several aligned arrays

The shared mapping is ordinary Python:

```
SPECIFICATIONS = {
    'three': ['other', 'berry', 'plain'],
```

```
    'five': ['other', 'berry', 'plain', 'other_fruits', 'desserts'],
}
```

Dictionary lookup chooses a specification; `list(...)` returns a fresh ordered list. A price matrix with five columns must use that order, as must count, availability, N and H matrices. `FLAVORS[:] = categories(specification)` changes the existing list in place. This matters because some function defaults already refer to that list; rebinding `FLAVORS` alone would leave those defaults referring to the old object.

`['beta_' + f for f in FLAVORS[1:]]` constructs fixed-effect names for every nonreference flavor. A hardcoded `['beta_berry','beta_plain']` would omit the two new intercepts. The residual gradient uses the same order to give each new coefficient its own derivative. Adding groups therefore changes parameter meaning as well as matrix width.

The original three-group descriptor rule is retained separately. In `five`, dessert conditions precede fruit conditions. A chained Polars `when(...).then(...).when(...)` uses the first satisfied condition, so ordering is an economic grouping decision, not just style.

Lesson 19: novelty is a score for an alternative before choosing it

```
previous_share = np.array([0., .25, .75])  # other, berry, plain
novelty = previous_share.sum() - previous_share
```

The result is `[1., .75, .25]`. This is the original recent-exposure measure, not the realized change in the current basket. Current purchases determine the next state only after current predictors have been recorded. A binary reward for a contemporaneously observed switch would be an outcome-dependent regressor; our N uses prior shares and avoids that construction.

With five categories, the subtraction is identical but the vector is longer. An absent category receives one when last week's basket exists. After a gap, the implemented convention assigns zero instead. The bonus is not restricted to one purchase in a mixed-basket episode. H accumulates past weekly shares, separately from N.

Lesson 20: decisions and evaluation need not use the same averaging order

Suppose persistent type r has initial weight lambda_r and first-period purchase probability s1_r. Under fixed beliefs, the firm's second-period optimization still uses lambda_r. But the consumers actually reaching the purchased branch are weighted in revenue by lambda_r*s1_r.

A schematic persistent-type revenue calculation is:

```
revenue_r = s1_r*p1 + delta*(s1_r*p2j*s2j_r + (1-s1_r)*p20*s20_r)
revenue = prior @ revenue_by_type
```

The `@` symbol computes a weighted sum here. The uninformed offer policy remains the fixed-belief optimizer; evaluation does not retroactively give the firm information it did not use. Compare informed and uninformed policies under the same persistent types. Keep the objective column and evaluated-revenue column distinct.

This auxiliary fixed-belief experiment also differs from the full-demand `posterior-integrated` sensitivity, which updates beliefs after a simulated basket. Read the entry point rather than treating every use of uncertain types as the same specification.

Lesson 21: an exception can reject one optional component without erasing another

`CalibrationError` carries aggregate numerical diagnostics. In `information_frictions`, `try` attempts the auxiliary calibration and `except CalibrationError` records its failed status and returns. The primary coupon solver can

then finish writing its aggregate outputs.

Only that named rejection is handled. An arbitrary programming exception is not converted into a successful calibration. An unidentified model is not repaired by deleting rank checks. The code exports price-moment diagnostics, omits unsupported information charts, and exposes status to the driver and results exporter.

In `calibrate`, filter starts for convergence, interior parameters and numerical rank before choosing the smallest objective. Selecting a numerically smaller but inadmissible result first could reject the whole run even if another valid start exists.

Lesson 22: restarting after data is an explicit sequence, not a hidden resume

`server/run_after_data.py` calls visible subprocesses: summaries, estimation, pricing, figures, and export. `subprocess.run(..., check=True)` waits for completion and raises on a failed child. The driver does not invoke `build_data.py`, so raw conversion and matching are skipped.

Every output is under a new job directory. Both estimates are completed before either pricing stage, and one pricing failure does not prevent attempting the other specification. The exporter copies only allowed results; it does not copy a server work directory wholesale. `run_status.json` and aggregate comparison CSVs make the partial/completed state reviewable.

The saved `flavor_spec` is also used when regenerating weekly inputs. A file with familiar column names is not sufficient: counts and the fixed-parameter likelihood must match the fit. Reconstructing inputs never means changing a saved three-group fit into a five-group fit by editing its metadata.

data_code: source walkthrough

data_code/cleaning_merging/master_dat.py

This matches household purchases to retail quotes. The four-element key list is reused for projection, deduplication, and joins, reducing accidental differences between stages. The filtered retail yogurt table is collected once into memory; panel purchases are scanned and joined year by year to bound panel-side work. An inner join means a price match is a sample-selection condition.

Yearly results are staged as Parquet and scanned together into the final sample. `os.remove` deletes the generated temporary files and `os.rmdir` removes their directory after consolidation. These are real filesystem mutations. Final diagnostic queries return scalar counts or small tables. `next(..., None)` picks supported household/price names rather than assuming a suffix. Building all years eagerly in one pandas merge would be shorter but much more memory-hungry. Resolve its retail input path against the retail merge's actual output before running a fresh pipeline.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–15:

```
9  from pathlib import Path
10 import glob
11 import os
12 import polars as pl
13
14 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
15 final_out = os.path.join(input_dir, "yogurt_estimation_sample.parquet")
```

Read this block:

- Line 9: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `glob`. An as-name is a convenient local alias, not another library.
- Line 11: import `os`. An as-name is a convenient local alias, not another library.
- Line 12: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 14, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 15, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 16–17:

```
16 if Path(final_out).resolve().is_relative_to(Path(__file__).resolve().parents[4]):
17     raise ValueError("Estimation samples must stay outside the code checkout; use the server data directory.")
```

Read this block:

- Line 16: test `Path(final_out).resolve().is_relative_to(Path(__file__).resolve().parents[4])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 17: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 18–27:

```
18 temp_dir = os.path.join(input_dir, "temp_years")
19 os.makedirs(temp_dir, exist_ok=True)
20
21 composite_keys = ["panel_year", "week_end", "upc", "store_code_uc"]
22
23 # 1. Load RMS with explicit column projection (ONLY load what you need)
24 print("[STEP 1/3] Materializing unique Yogurt RMS keys...", flush=True)
25
26 # Define exact RMS columns needed (join keys + filter col + price/movement cols)
27 rms_needed_cols = composite_keys + ["product_module_code", "price"]
```

Read this block:

- Line 18, `temp_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.

- Line 19: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 21, `composite_keys`: Set this name to the list `['panel_year', 'week_end', 'upc', 'store_code_uc']`.
- Line 24: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 27, `rms_needed_cols`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 29–42:

```
29 rms_yogurt = (
30     pl.scan_parquet(os.path.join(input_dir, "merged_rms.parquet"))
31     # Select ONLY required columns up front (drops ~85% of memory loading)
32     .select([c for c in rms_needed_cols])
33     # Filter immediately
34     .filter(
35         pl.col("product_module_code").is_in([3603, 3612])
36     )  # Update to your yogurt code
37     # Deduplicate on composite keys
38     .unique(subset=composite_keys, keep="first")
39     .collect()
40 )
41
42 print(f"--> Yogurt RMS loaded successfully: {len(rms_yogurt):,} rows.", flush=True)
```

Read this block:

- Line 29, `rms_yogurt`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 42: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 45–50:

```
45 panel_years = list(range(2006, 2016))
46
47 print(
48     f"[STEP 2/3] Processing panel year-by-year across {len(panel_years)} years...",
49     flush=True,
50 )
```

Read this block:

- Line 45, `panel_years`: Set this name to the result of calling `list(...)`; the arguments and preceding method chain are shown above.
- Line 47: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `list`: Materialize iterable entries as an ordered Python list.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 52–72:

```
52 for idx, yr in enumerate(panel_years, 1):
53     yr_out = os.path.join(temp_dir, f"yogurt_{yr}.parquet")
54     print(
55         f"  --> [{idx}/{len(panel_years)}] Joining & streaming Year {yr}...",
56         flush=True,
57     )
58 
59     # Scan HMS
60     hms_yr = pl.scan_parquet(os.path.join(input_dir, "merged_panel.parquet")).filter(
61         pl.col("panel_year") == yr
62     )
63 
64     # Filter in-memory RMS to current year
65     rms_yr = rms_yogurt.filter(pl.col("panel_year") == yr)
66 
67     # Stream inner join
68     hms_yr.join(
69         rms_yr.lazy(), on=composite_keys, how="inner", suffix="_rms"
70     ).sink_parquet(yr_out)
71 
72     print(f"      [DONE] Saved Year {yr} chunk.", flush=True)
```

Read this block:

- Line 52: for each entry yielded by `enumerate(panel_years, 1)`, bind/unpack it into `(idx, yr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53, `yr_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 54: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 60, `hms_yr`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 65, `rms_yr`: Set this name to the result of calling `rms_yogurt.filter(...)`; the arguments and preceding method chain are shown above.
- Line 68: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

Source lines 75–79:

```
print(
    "\n[STEP 3/3] Combining yearly files into final estimation sample...",
    flush=True,
)
pl.scan_parquet(os.path.join(temp_dir, "yogurt_*.parquet")).sink_parquet(final_out)
```

Read this block:

- Line 75: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 82–83:

```
for f in glob.glob(os.path.join(temp_dir, "yogurt_*.parquet")):
    os.remove(f)
```

Read this block:

- Line 82: for each entry yielded by `glob.glob(os.path.join(temp_dir, 'yogurt_*.parquet'))`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 83: call `os.remove` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.remove`: Delete the named file; in these scripts it cleans generated temporary year chunks.

Source lines 84–85:

```
if os.path.exists(temp_dir):
    os.rmdir(temp_dir)
```

Read this block:

- Line 84: test `os.path.exists(temp_dir)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: call `os.rmdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.rmdir`: Remove an empty directory after its temporary files have been deleted.

Source lines 87–90:

```
print(
    f"\n[SUCCESS] Saved 10-year Yogurt estimation sample to: {final_out}",
    flush=True,
)
```

Read this block:

- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 95–102:

```
print("\n" + "=" * 70, flush=True)
print("RUNNING ESTIMATION SAMPLE AUDIT DIAGNOSTICS", flush=True)
print("=" * 70, flush=True)
```

```
98
99 # Scan final dataset lazily for memory-efficient auditing
100 sample_scan = pl.scan_parquet(final_out)
101 schema = sample_scan.collect_schema()
102 col_names = schema.names()
```

Read this block:

- Line 95: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 100, `sample_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 101, `schema`: Set this name to the result of calling `sample_scan.collect_schema(...)`; the arguments and preceding method chain are shown above.
- Line 102, `col_names`: Set this name to the result of calling `schema.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

Source lines 105–115:

```
105 total_rows = sample_scan.select(pl.len()).collect().item()
106 unique_upcs = sample_scan.select(pl.col("upc").n_unique()).collect().item()
107 unique_stores = sample_scan.select(pl.col("store_code_uc").n_unique()).collect().item()
108
109 # Check household column dynamically depending on HMS schema
110 hh_col = next((c for c in ["household_code", "hh_id"] if c in col_names), None)
111 unique_hhs = (
112     sample_scan.select(pl.col(hh_col).n_unique()).collect().item() if hh_col else "N/A"
113 )
114
115 print("\n[1] RECORD COUNTS & ENTITY COVERAGE:", flush=True)
```

Read this block:

- Line 105, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 106, `unique_upcs`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 107, `unique_stores`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 110, `hh_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 111, `unique_hhs`: Set this name to the true-branch value if `hh_col` holds, otherwise the else-branch value.
- Line 115: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 116–127:

```
116 print(f"    - Total Estimation Rows:        {total_rows:,}", flush=True)
117 print(
118     f"    - Unique Households Matched:    {unique_hhs:,}"
119     if isinstance(unique_hhs, int)
120     else f"    - Unique Households Matched:    {unique_hhs}",
121     flush=True,
122 )
123 print(f"    - Unique Yogurt UPCs Matched:   {unique_upcs:,}", flush=True)
124 print(f"    - Unique Retail Stores Matched: {unique_stores:,}", flush=True)
125
126 # --- DIAGNOSTIC 2: Join Key Integrity & Null Check ---
127 print("\n[2] JOIN KEY INTEGRITY (NULL CHECK):", flush=True)
```

Read this block:

- Line 116: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 117: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 124: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 128–132:

```
128 null_counts = (
129     sample_scan.select([pl.col(k).null_count().alias(k) for k in composite_keys])
130     .collect()
131     .to_dict(as_series=False)
132 )
```

Read this block:

- Line 128, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 134–136:

```
134 for key, count in null_counts.items():
135     null_pct = (count[0] / total_rows) * 100 if total_rows > 0 else 0
136     print(f"    - Nulls in '{key}': {count[0]:,} ({null_pct:.2f}%)", flush=True)
```

Read this block:

- Line 134: for each entry yielded by `null_counts.items()`, bind/unpack it into `(key, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 135, `null_pct`: Set this name to the true-branch value if `total_rows > 0` holds, otherwise the else-branch value.

- Line 136: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 139–145:

```
print("\n[3] PANEL YEAR DISTRIBUTION:", flush=True)
year_counts = (
    sample_scan.group_by("panel_year")
    .agg(pl.len().alias("count"))
    .sort("panel_year")
    .collect()
)
```

Read this block:

- Line 139: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140, `year_counts`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 147–151:

```
for row in year_counts.iter_rows(named=True):
    print(
        f"    - Year {row['panel_year']}: {row['count']:,} observations",
        flush=True,
    )
```

Read this block:

- Line 147: for each entry yielded by `year_counts.iter_rows(named=True)`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 148: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 154–154:

```
price_col = next((c for c in ["price", "price_rms"] if c in col_names), None)
```

Read this block:

- Line 154, `price_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Source lines 155–189:

```
if price_col:
    price_stats = (
        sample_scan.select(
            [
                pl.col(price_col).min().alias("min_price"),
                pl.col(price_col).mean().alias("mean_price"),
                pl.col(price_col).median().alias("median_price"),
                pl.col(price_col).max().alias("max_price"),
                pl.col(price_col).null_count().alias("null_price_count"),
```

```
164                 (pl.col(price_col) <= 0).sum().alias("zero_price_count"),
165             ]
166         )
167         .collect()
168         .to_dict(as_series=False)
169     )
170 
171     print(
172         f"\n[4] PRICE COLUMN DIAGNOSTICS ('{price_col}'):",
173         flush=True,
174     )
175     print(f"    - Min Price:    ${price_stats['min_price'][0]:.2f}", flush=True)
176     print(f"    - Mean Price:   ${price_stats['mean_price'][0]:.2f}", flush=True)
177     print(
178         f"    - Median Price: ${price_stats['median_price'][0]:.2f}",
179         flush=True,
180     )
181     print(f"    - Max Price:    ${price_stats['max_price'][0]:.2f}", flush=True)
182     print(
183         f"    - Null Prices:  {price_stats['null_price_count'][0]:,}",
184         flush=True,
185     )
186     print(
187         f"    - Zero/Negative:{price_stats['zero_price_count'][0]:,}",
188         flush=True,
189     )
```

Read this block:

- Line 155: test `price_col`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156, `price_stats`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.
- Line 171: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 176: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 177: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 181: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 182: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 192–199:

```
192 file_size_mb = os.path.getsize(final_out) / (1024**2)
193
194 print("\n[5] OUTPUT FILE SUMMARY:", flush=True)
195 print(f"    - Total Schema Columns: {len(col_names)}", flush=True)
196 print(f"    - File Size on Disk:    {file_size_mb:.2f} MB", flush=True)
197 print(f"    - Output Location:      {final_out}", flush=True)
198
199 print("=" * 70, flush=True)
```

Read this block:

- Line 192, `file_size_mb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 194: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 195: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 196: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 197: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 199: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 200–200:

```
200 print("AUDIT COMPLETE — Yogurt estimation sample verified.", flush=True)
```

Read this block:

- Line 200: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data__code/cleaning__merging/panel__concat.py

This stacks annual cleaned panel tables into combined files. `concat_list` is an ordinary list of three table names. In the loop, an f-string builds a wildcard such as `cleaned_trips_*.parquet`. A lazy scan over those files followed by `sink_parquet` writes one combined table. It does not add rows by matching keys: concatenation stacks records, whereas a join combines columns through matches. There is no deduplication or explicit year reconciliation here. A stale matching file is therefore part of the input unless the directory is audited.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
  ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–10:

```
9  import os
10 import polars as pl
```

Read this block:

- Line 9: import `os`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 14–16:

```
14 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
15 final_dir = os.path.join(base_out_dir, "combined")
16 os.makedirs(final_dir, exist_ok=True)
```

Read this block:

- Line 14, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 15, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 16: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 20–20:

```
20 concat_list = ["panelists", "purchases", "trips"]
```

Read this block:

- Line 20, `concat_list`: Set this name to the list `['panelists', 'purchases', 'trips']`.

Source lines 21–28:

```
21 for table in concat_list:
22     file_pattern = os.path.join(base_out_dir, f"cleaned_{table}_*.parquet")
23     concat_out = os.path.join(final_dir, f"{table}.parquet")
```

```
24
25      print(f"Combining {table} files...")
26
27      (pl.scan_parquet(file_pattern).sink_parquet(concat_out))
28      print(f"Saved: {concat_out}")
```

Read this block:

- Line 21: for each entry yielded by `concat_list`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 22, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 23, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 25: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 27: call `pl.scan_parquet(file_pattern).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

data__code/cleaning__merging/panel__convert.py

This script runs at module level: importing it can perform data work. First the imports make filesystem and Polars tools available; path strings select raw/server locations. Integer switches `annual` and `master` are truth-tested to decide which conversion loops run. `range(2006, 2016)` includes 2006 through 2015, excluding the upper endpoint. Dictionaries translate source table names to output stems. Nested loops construct one TSV/Parquet pair per year/table, and `exists` prevents trying to read missing files.

After conversion, wildcard scans discover Parquet files. `columns_to_keep` is a dictionary of table-specific projection lists, sometimes containing Polars alias expressions. `split("_")[0]` extracts a table prefix before selecting its projection. Scanning and sinking keep large tables out of ordinary Python lists; eagerly reading every annual table would need much more memory. Lowercasing column names standardizes downstream joins but does not reconcile every schema difference or verify raw-codebook meaning. No CLI parser protects the hard-coded paths; inspect them before running.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
  ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 10–12:

```
10 import glob
11 import os
12 import polars as pl
```

Read this block:

- Line 10: import `glob`. An as-name is a convenient local alias, not another library.
- Line 11: import `os`. An as-name is a convenient local alias, not another library.
- Line 12: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 16–22:

```
16 annual = 0
17 master = 0
18
19 years = range(2006, 2016)
20 base_raw_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/nielsen_extracts/HMS"
21 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
22 os.makedirs(base_out_dir, exist_ok=True)
```

Read this block:

- Line 16, `annual`: Set this name to the literal value `0`.
- Line 17, `master`: Set this name to the literal value `0`.
- Line 19, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 20, `base_raw_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 21, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 22: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 24–29:

```
24 base_mapping = {
25     "panelists": "panelists",
26     "purchases": "purchases",
27     "trips": "trips",
28 }
29 master_mappings = {"products": "products", "retailers": "retailers"}
```

Read this block:

- Line 24, `base_mapping`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`.
- Line 29, `master_mappings`: Set this name to a dictionary with named entries `products`, `retailers`.

Source lines 32–54:

```
32 if annual:
33     for y in years:
34         print(f"\n--- Converting year {y} ---")
35 
36         for base_tsv, out_name in base_mapping.items():
37             tsv_filename = f"{base_tsv}_{y}.tsv"
38             tsv_path = os.path.join(base_raw_dir, str(y), "Annual_Files", tsv_filename)
39             parquet_out = os.path.join(base_out_dir, f"{out_name}_{y}.parquet")
40 
41             if os.path.exists(tsv_path):
42                 (
43                     pl.scan_csv(
44                         tsv_path,
45                         separator="\t",
46                         quote_char=None,
47                         infer_schema_length=10000,
48                     )
49                     .rename(str.lower)
50                     .sink_parquet(parquet_out)
51                 )
52                 print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
53             else:
54                 print(f"WARNING: File not found {tsv_path}")
```

Read this block:

- Line 32: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 33: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 34: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: for each entry yielded by `base_mapping.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 37, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 38, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 39, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 41: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 54: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 55–72:

```
55 if master:
56     for base_tsv, out_name in master_mappings.items():
57         tsv_filename = f"{base_tsv}.tsv"
58         tsv_path = os.path.join(base_raw_dir, "Master_Files/Latest", tsv_filename)
59         parquet_out = os.path.join(base_out_dir, f"{out_name}.parquet")
60 
61         if os.path.exists(tsv_path):
62             (
63                 pl.scan_csv(
64                     tsv_path,
65                     separator="\t",
66                     quote_char=None,
67                     infer_schema_length=10000,
68                     encoding="utf8-lossy",
69                 )
70                 .rename(str.lower)
71                 .sink_parquet(parquet_out)
72             )
```

Read this block:

- Line 55: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56: for each entry yielded by `master_mappings.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 57, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 58, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 59, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 61: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 76–76:

```
76 parquet_files = glob.glob(os.path.join(base_out_dir, "*.parquet"))
```

Read this block:

- Line 76, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 78–82:

```
78 for filepath in sorted(parquet_files):
79     filename = os.path.basename(filepath)
80     cols = pl.scan_parquet(filepath).collect_schema().names()
81     print(f"\n--- {filename} ({len(cols)} columns) ---")
82     print(cols)
```

Read this block:

- Line 78: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 80, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 84–124:

```
84 columns_to_keep = {
85     "panelists": [
86         pl.col("household_cd").alias("household_code"),
87         "panel_year",
88         "household_income",
89         "household_size",
90         "male_head_age",
91         "female_head_age",
92         "male_head_employment",
93         "female_head_employment",
94         "race",
95         "hispanic_origin",
96         "dma_cd",
97         "dma_name",
98     ],
99     "purchases": [
100        "trip_code_uc",
101        "upc",
102        "quantity",
103        "total_price_paid",
104        "deal_flag_uc",
105    ],
106    "trips": [
107        "trip_code_uc",
```

```
        "household_code",
        "purchase_date",
        "store_code_uc",
        "panel_year",
    ],
    "products": [
        "upc",
        "product_module_code",
        "product_module_descr",
        "product_group_code",
        "product_group_descr",
        "brand_code_uc",
        "brand_descr",
        "size1_amount",
        "size1_units",
    ],
}
```

Read this block:

- Line 84, `columns_to_keep`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`, `products`.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 126–140:

```
for filepath in glob.glob(os.path.join(base_out_dir, "*.parquet")):
    filename = os.path.basename(filepath)

    # Remove .parquet extension first
    clean_name = os.path.splitext(filename)[0]  # e.g., "retailers" or "trips_2011"

    # Split by underscore if present to get the table name
    table_type = clean_name.split("_")[0]  # e.g., "retailers", "trips", "panelists"

    if table_type in columns_to_keep:
        target_cols = columns_to_keep[table_type]
        cleaned_path = os.path.join(base_out_dir, f"cleaned_{filename}")

        (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
        print(f"Cleaned {filename} -> saved to {cleaned_path}")
```

Read this block:

- Line 126: for each entry yielded by `glob.glob(os.path.join(base_out_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 127, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 130, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 133, `table_type`: Set this name to the entry/slice selected by `clean_name.split('_')[0]`; brackets select from the object to their left.
- Line 135: test `table_type in columns_to_keep`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 136, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[table_type]`; brackets select from the object to their left.
- Line 137, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and

preceding method chain are shown above.
- Line 139: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

data__code/cleaning__merging/panel__dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

Module setup and direct-execution blocks

There are no statements in this file.

data__code/cleaning__merging/panel__merge.py

Purchase lines anchor the merge. Join-key casts align integer dtypes before an inner trip join, then left joins attach annual household and UPC product metadata. Method chaining expresses a sequence of relational operations whose result is sunk to `merged_panel.parquet`. An inner join drops unmatched purchase lines; a left join retains them but may add missing metadata. Duplicate right keys can multiply rows in either case.

The second half scans the output and prints diagnostics. `collect().item()` extracts a scalar from a one-cell query result; `to_dict(as_series=False)` makes a dictionary of Python lists for null-count reporting. `count[0]` picks its only row. File byte size divided by `1024**3` expresses GiB. The empty-output check raises, but a nonempty result is not proof of correct joins. Explicit key validation would be a stronger alternative to relying only on total row ratios.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–17:

```
9  import os
10 import polars as pl
11
12 base_dir = "/scratch/dtm63837/Kilts_Panel/Data"
13 final_out = os.path.join(base_dir, "merged_panel.parquet")
14
15 ## === Load Data === ##
16 print("Loading lazy frames...")
17 panelists = pl.scan_parquet(os.path.join(base_dir, "combined/panelists.parquet"))
```

Read this block:

- Line 9: import `os`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 13, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 16: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 17, `panelists`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 18–20:

```
18 purchases = pl.scan_parquet(os.path.join(base_dir, "combined/purchases.parquet"))
19 trips = pl.scan_parquet(os.path.join(base_dir, "combined/trips.parquet"))
20 products = pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
```

Read this block:

- Line 18, `purchases`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 19, `trips`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 20, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Source lines 24–38:

```
24 purchases = purchases.with_columns(
25     [pl.col("trip_code_uc").cast(pl.Int64), pl.col("upc").cast(pl.Int64)]
26 )
27
28 trips = trips.with_columns(
```

```
    [
        pl.col("trip_code_uc").cast(pl.Int64),
        pl.col("household_code").cast(pl.Int64),
        pl.col("panel_year").cast(pl.Int32),
    ]
)

panelists = panelists.with_columns(
    [pl.col("household_code").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)]
)
```

Read this block:

- Line 24, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 28, `trips`: Set this name to the result of calling `trips.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 36, `panelists`: Set this name to the result of calling `panelists.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 40–52:

```
products = products.with_columns([pl.col("upc").cast(pl.Int64)])

## === Merge === ##
merged = (
    purchases.join(trips, on="trip_code_uc", how="inner")
    .join(panelists, on=["household_code", "panel_year"], how="left")
    .join(products, on="upc", how="left")
)

print("Merging purchases, trips, panelists, and products...")
print("Streaming to disk (this will take a few minutes)...")
merged.sink_parquet(final_out)
print(f"\n[SUCCESS] Successfully saved merged file to: {final_out}")
```

Read this block:

- Line 40, `products`: Set this name to the result of calling `products.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 43, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 49: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 50: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 55–64:

```
print("\n" + "=" * 50)
print("RUNNING DIAGNOSTICS ON MERGED DATASET")
print("=" * 50)

# 1. Scan output dataset lazily
merged_scan = pl.scan_parquet(final_out)

# 2. Total Row Count Check
total_rows = merged_scan.select(pl.len()).collect().item()
print(f"\n[1] Total Row Count: {total_rows:,}")
```

Read this block:

- Line 55: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 60, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 63, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 66–67:

```
if total_rows == 0:
    raise ValueError("CRITICAL ERROR: Merged dataset contains 0 rows! Check join keys.")
```

Read this block:

- Line 66: test `total_rows == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 67: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 70–77:

```
purchases_rows = purchases.select(pl.len()).collect().item()
pct_retained = (total_rows / purchases_rows) * 100
print(f"    - Anchor purchases rows: {purchases_rows:,}")
print(f"    - Retained post-join: {pct_retained:.2f}%")

# 4. Null / NA Check across key columns
print("\n[2] Null Value Diagnostics (Key Columns):")
key_cols = ["trip_code_uc", "household_code", "upc", "panel_year"]
```

Read this block:

- Line 70, `purchases_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 71, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 72: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77, `key_cols`: Set this name to the list `['trip_code_uc', 'household_code', 'upc', 'panel_year']`.

Source lines 78–84:

```
78 existing_keys = [col for col in key_cols if col in merged_scan.collect_schema().names()]
79
80 null_counts = (
81     merged_scan.select([pl.col(c).null_count().alias(c) for c in existing_keys])
82     .collect()
83     .to_dict(as_series=False)
84 )
```

Read this block:

- Line 78, `existing_keys`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 80, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 86–88:

```
86 for col, count in null_counts.items():
87     null_pct = (count[0] / total_rows) * 100
88     print(f"    - {col}: {count[0]:,} nulls ({null_pct:.2f}%)")
```

Read this block:

- Line 86: for each entry yielded by `null_counts.items()`, bind/unpack it into `(col, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 87, `null_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 91–97:

```
91 file_size_gb = os.path.getsize(final_out) / (1024**3)
92 schema_cols = merged_scan.collect_schema().names()
93
94 print("\n[3] Output File Summary:")
95 print(f"    - Total Columns: {len(schema_cols)}")
96 print(f"    - Disk Size: {file_size_gb:.2f} GB")
97 print(f"    - Columns: {schema_cols}")
```

Read this block:

- Line 91, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 92, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 94: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 98–99:

```
98 print("=" * 50)
99 print("DIAGNOSTICS COMPLETE - File is verified and ready.")
```

Read this block:

- Line 98: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data__code/cleaning__merging/panel__week.py

This adds a Saturday-ending integer key to the merged panel. The nested expression parses purchase-date strings, extracts weekdays, computes a duration, adds it, formats `%Y%m%d`, and casts to Int64. Read inside out. With Polars' Monday=1 through Sunday=7 convention, `6-(weekday % 7)` gives Monday five days, Saturday zero, and Sunday six days. It agrees with the downstream modulo-seven formula over those weekday values.

The query writes a separate temporary file before `os.replace` replaces the original. Writing directly to the source path while scanning it could corrupt the read; staging then replacement avoids that pattern. It still overwrites the original file after success, so this is not an inspection-only script.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
  ↪ "Code/privacy/server_data_policy.py").is_file())
```

```
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–16:

```
9  import os
10 import polars as pl
11
12 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
13 panel_path = os.path.join(input_dir, "merged_panel.parquet")
14 panel_temp = os.path.join(input_dir, "merged_panel_updated.parquet")
15
16 print("Pre-computing week_end on merged_panel.parquet...")
```

Read this block:

- Line 9: import `os`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 13, `panel_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 14, `panel_temp`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 16: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 18–32:

```
18 (
19     pl.scan_parquet(panel_path)
20     .with_columns(
21         # Convert "YYYY-MM-DD" to Date, compute Saturday week-ending, format to YYYYMMDD Int64
22         week_end=(
23             pl.col("purchase_date").str.to_date()
24             + pl.duration(
25                 days=(6 - (pl.col("purchase_date").str.to_date().dt.weekday() % 7))
26             )
27         )
28         .dt.strftime("%Y%m%d")
29         .cast(pl.Int64)
30     )
31     .sink_parquet(panel_temp)
32 )
```

Read this block:

- Line 18: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.strftime`: Format a parsed date according to directives, e.g. %Y%m%d for an eight-digit week-ending key.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 35–36:

```
35 os.replace(panel_temp, panel_path)
36 print("Updated merged_panel.parquet successfully with Int64 'week_end'!")
```

Read this block:

- Line 35: call `os.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

data_code/cleaning_merging/retail_concat.py

This scans annual retail outputs, harmonizes selected dtypes, and vertically concatenates tables. `glob.glob` returns matching filenames; `sorted` supplies deterministic order. A missing match list triggers `continue`, skipping just that target rather than the whole script. The list comprehension creates casts only for columns actually present in a particular schema.

`how="vertical_relaxed"` allows compatible dtype coercion when stacking frames; it is not permission to ignore arbitrary incompatible schemas. Explicit casts for flavor/size codes and join IDs reduce annual type conflicts. Keeping codes as strings can preserve nonnumeric categories that an integer cast would reject.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
  ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–15:

```
9 import glob
10 import os
11 import polars as pl
12
13 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
14 final_dir = os.path.join(base_out_dir, "combined")
15 os.makedirs(final_dir, exist_ok=True)
```

Read this block:

- Line 9: import `glob`. An as-name is a convenient local alias, not another library.
- Line 10: import `os`. An as-name is a convenient local alias, not another library.
- Line 11: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 13, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 14, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 15: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 17–23:

```
17 concat_targets = {
18     "stores": "cleaned_stores_*.parquet",
19     "products_extra": "cleaned_products_extra_*.parquet",
20     "3612": "cleaned_3612_*.parquet",
21     "3603": "cleaned_3603_*.parquet",
22     "rms": "rms_*.parquet",
23 }
```

Read this block:

- Line 17, `concat_targets`: Set this name to a dictionary with named entries `stores`, `products_extra`, `3612`, `3603`, `rms`.

Source lines 26–32:

```
26 CAST_SCHEMA = {
27     "flavor_code": pl.String,
28     "size1_code_uc": pl.String,
29     "upc": pl.Int64,
30     "store_code_uc": pl.Int64,
31     "panel_year": pl.Int32,
32 }
```

Read this block:

- Line 26, `CAST_SCHEMA`: Set this name to a dictionary with named entries `flavor_code`, `size1_code_uc`, `upc`, `store_code_uc`, `panel_year`.

Source lines 34–65:

```
34 for table, pattern in concat_targets.items():
35     file_pattern = os.path.join(base_out_dir, pattern)
36     matching_files = sorted(glob.glob(file_pattern))
37 
38     if not matching_files:
39         print(f"WARNING: No files found matching {file_pattern}. Skipping {table}...")
40         continue
41 
42     concat_out = os.path.join(final_dir, f"{table}.parquet")
43     print(f"Combining {len(matching_files)} files for '{table}'...")
44 
45     # Load files individually and align column types to avoid SchemaError
46     lazy_frames = []
47     for f in matching_files:
48         lf = pl.scan_parquet(f)
49 
50         # Cast any problematic columns present in this table schema
51         schema_names = lf.collect_schema().names()
52         cast_exprs = [
53             pl.col(col).cast(target_type)
54             for col, target_type in CAST_SCHEMA.items()
55             if col in schema_names
56         ]
57 
58         if cast_exprs:
59             lf = lf.with_columns(cast_exprs)
60 
61         lazy_frames.append(lf)
```

```
62
63      # Concatenate using relaxed type resolution
64      (pl.concat(lazy_frames, how="vertical_relaxed").sink_parquet(concat_out))
65      print(f"Saved: {concat_out}")
```

Read this block:

- Line 34: for each entry yielded by `concat_targets.items()`, bind/unpack it into `(table, pattern)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 35, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 36, `matching_files`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.
- Line 38: test `not matching_files`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 39: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 42, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46, `lazy_frames`: Set this name to the list `[]`.
- Line 47: for each entry yielded by `matching_files`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 48, `lf`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 51, `schema_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 52, `cast_exprs`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 58: test `cast_exprs`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 59, `lf`: Set this name to the result of calling `lf.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 61: call `lazy_frames.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

- `len`: Count elements/rows rather than summing their numerical values.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

data_code/cleaning_merging/retail_convert.py

This is the retail analogue of panel conversion. Multiple mapping dictionaries separate annual metadata, movement modules, and master files. All are ordinary Python loops around lazy TSV reads and Parquet writes. `quote_char=None` disables CSV-style quote interpretation for TSV inputs. Schema inference inspects a prefix, so later inconsistent values may still need attention.

Cleaning keys are sorted by descending length, and a boundary-aware prefix test distinguishes `products_extra` from `products`. This is a concrete reason for the `sorted(..., key=len, reverse=True)` choice: testing the shorter prefix first could select the wrong projection. `continue` skips already cleaned files; `break` stops the key-search loop after a match. A `try/except` around schema inspection reports unreadable files without aborting all inspection.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.
- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–15:

```
import glob
import os
import polars as pl

input_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/RMS"
output_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
os.makedirs(output_dir, exist_ok=True)
```

Read this block:

- Line 9: import `glob`. An as-name is a convenient local alias, not another library.
- Line 10: import `os`. An as-name is a convenient local alias, not another library.
- Line 11: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 13, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 14, `output_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 15: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 17–31:

```
annual = 1
master = 1

years = range(2006, 2016)

annual_mapping = {
    "products_extra": "products_extra",
    "rms_versions": "rms",
    "stores": "stores",
}
movement_mapping = {
    "3603": "3603",
    "3612": "3612",
}
master_mapping = {"brand_variations": "brand_variations", "products": "products"}
```

Read this block:

- Line 17, `annual`: Set this name to the literal value `1`.
- Line 18, `master`: Set this name to the literal value `1`.
- Line 20, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 22, `annual_mapping`: Set this name to a dictionary with named entries `products_extra`, `rms_versions`, `stores`.
- Line 27, `movement_mapping`: Set this name to a dictionary with named entries `3603`, `3612`.
- Line 31, `master_mapping`: Set this name to a dictionary with named entries `brand_variations`, `products`.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 33–74:

```
if annual:
    for y in years:
        for base_mapping, out_name in annual_mapping.items():
            tsv_filename = f"{base_mapping}_{y}.tsv"
            tsv_path = os.path.join(input_dir, str(y), "Annual_Files", tsv_filename)
            parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")

            if os.path.exists(tsv_path):
```

```
41                 (
42                     pl.scan_csv(
43                         tsv_path,
44                         separator="\t",
45                         quote_char=None,
46                         infer_schema_length=10000,
47                     )
48                     .rename(str.lower)
49                     .sink_parquet(parquet_out)
50                 )
51                 print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
52             else:
53                 print(f"FAILED: File not found {tsv_filename}")
54         for base_mapping, out_name in movement_mapping.items():
55             tsv_filename = f"{base_mapping}_{y}.tsv"
56             tsv_path = os.path.join(
57                 input_dir, str(y), f"Movement_Files/2510_{y}", tsv_filename
58             )
59             parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")
60 
61             if os.path.exists(tsv_path):
62                 (
63                     pl.scan_csv(
64                         tsv_path,
65                         separator="\t",
66                         quote_char=None,
67                         infer_schema_length=10000,
68                     )
69                     .rename(str.lower)
70                     .sink_parquet(parquet_out)
71                 )
72                 print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
73             else:
74                 print(f"FAILED: File not found {tsv_filename}")
```

Read this block:

- Line 33: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 35: for each entry yielded by `annual_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 36, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 37, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 38, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 40: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 41: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: for each entry yielded by `movement_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 55, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 56, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 59, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 61: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 76–98:

```
76 if master:
77     for base_mapping, out_name in master_mapping.items():
78         tsv_filename = f"{base_mapping}.tsv"
79         tsv_path = os.path.join(
80             input_dir, "Master_Files_2006-2020/Latest", tsv_filename
81         )
82         parquet_out = os.path.join(output_dir, f"{out_name}.parquet")
83 
84         if os.path.exists(tsv_path):
85             (
86                 pl.scan_csv(
87                     tsv_path,
88                     separator="\t",
89                     quote_char=None,
90                     infer_schema_length=10000,
91                     encoding="utf8-lossy",
92                 )
93                 .rename(str.lower)
94                 .sink_parquet(parquet_out)
95             )
96             print(f"Converted: {tsv_filename} -> {out_name}.parquet")
97         else:
98             print(f"FAILED: File not found {tsv_filename}")
```

Read this block:

- Line 76: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: for each entry yielded by `master_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 78, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 79, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 82, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 84: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 101–101:

```
101 parquet_files = glob.glob(os.path.join(output_dir, "*.parquet"))
```

Read this block:

- Line 101, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 103–116:

```
103 for filepath in sorted(parquet_files):
104     filename = os.path.basename(filepath)
105 
106     # Check if file is empty before reading
107     if os.path.getsize(filepath) == 0:
108         print(f"\n--- {filename} is EMPTY (0 bytes), skipping ---")
109         continue
110 
111     try:
112         cols = pl.scan_parquet(filepath).collect_schema().names()
113         print(f"\n--- {filename} ({len(cols)} columns) ---")
114         print(cols)
115     except Exception as e:
116         print(f"\n--- Could not read {filename} (corrupted or partial file): {e} ---")
```

Read this block:

- Line 103: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 107: test `os.path.getsize(filepath) == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 111: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 112, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

- Line 113: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 114: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 115: catch `Exception` from the preceding try; the optional as-name holds the error object for logging.
- Line 116: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 118–146:

```
118 columns_to_keep = {
119     "3603": ["store_code_uc", "upc", "week_end", "price"],
120     "3612": ["store_code_uc", "upc", "week_end", "price"],
121     "products": [
122         "upc",
123         "upc_ver_uc",
124         "product_module_code",
125         "brand_code_uc",
126         "brand_descr",
127         "size1_code_uc",
128         "size1_amount",
129         "size1_units",
130     ],
131     "products_extra": [
132         "upc",
133         "upc_ver_uc",
134         "panel_year",
135         "flavor_code",
136         "flavor_descr",
137     ],
138     "stores": [
139         "store_code_uc",
140         "year",  # Keep as string 'year' during initial extraction
141         "retailer_code",
142         "channel_code",
143         "dma_code",
144         "dma_descr",
145     ],
146 }
```

Read this block:

- Line 118, `columns_to_keep`: Set this name to a dictionary with named entries `3603`, `3612`, `products`, `products_extra`, `stores`.

Source lines 149–149:

```
149 table_keys_by_length = sorted(columns_to_keep.keys(), key=len, reverse=True)
```

Read this block:

- Line 149, `table_keys_by_length`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Source lines 151–175:

```
151 for filepath in glob.glob(os.path.join(output_dir, "*.parquet")):
152     filename = os.path.basename(filepath)
153 
154     # Skip files that have already been cleaned
155     if filename.startswith("cleaned_"):
156         continue
157 
158     # Remove extension -> "products_extra_2011"
159     clean_name = os.path.splitext(filename)[0]
160 
161     # Match against dictionary keys taking exact boundaries into account
162     matched_table = None
163     for key in table_keys_by_length:
164         if clean_name == key or clean_name.startswith(f"{key}_"):
165             matched_table = key
166             break
167 
168     if matched_table:
169         target_cols = columns_to_keep[matched_table]
170         cleaned_path = os.path.join(output_dir, f"cleaned_{filename}")
171 
172         (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
173         print(
174             f"Cleaned {filename} (Matched: {matched_table}) -> saved to cleaned_{filename}"
175         )
```

Read this block:

- Line 151: for each entry yielded by `glob.glob(os.path.join(output_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 152, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 155: test `filename.startswith('cleaned_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 159, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 162, `matched_table`: Set this name to the literal value `None`.
- Line 163: for each entry yielded by `table_keys_by_length`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 164: test `clean_name == key or clean_name.startswith(f'{key}_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 165, `matched_table`: Set this name to the object currently named `key`; plain assignment does not copy it.
- Line 166: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 168: test `matched_table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 169, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[matched_table]`; brackets select from the object to their left.
- Line 170, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 172: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 173: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

data__code/cleaning__merging/retail__dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

Module setup and direct-execution blocks

There are no statements in this file.

data__code/cleaning__merging/retail__merge.py

This concatenates movement modules and uses them as the retail anchor. Integer floor division by 10000 extracts a calendar year from a YYYYMMDD integer. Several metadata tables are deduplicated before left joins to avoid multiplying movement rows. `suffix` names colliding non-key columns, while composite `on=[...]` lists specify exact matching keys.

First-row deduplication is a policy, not an inference of the correct UPC version. Extra-product sorting prioritizes nonnull flavor descriptions before retaining a UPC/year row. The diagnostics compare anchor and merged row counts and inspect nonnull metadata. A conditional expression chooses which brand field is available for a match diagnostic, so that diagnostic may inspect a source key rather than a successful dictionary match. Output is under `Data/RMS/`, unlike the current master script's `Data/merged_rms.parquet` input path.

Module setup and direct-execution blocks

Source lines 3–8:

```
3 from pathlib import Path as _PolicyPath
4 import sys as _policy_sys
5 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
  ↪  "Code/privacy/server_data_policy.py").is_file())
6 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
7 from server_data_policy import server_only, require_server, require_legacy_server
8 require_legacy_server()
```

Read this block:

- Line 3: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 5, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 6: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: from `server_data_policy`, import `server_only, require_server, require_legacy_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: call `require_legacy_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

- `require_legacy_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 9–15:

```
import os
import polars as pl

base_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
final_out = os.path.join(base_dir, "merged_rms.parquet")

print("Loading lazy frames...")
```

Read this block:

- Line 9: import `os`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 13, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 15: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 18–30:

```
movements = pl.concat(
    [
        pl.scan_parquet(os.path.join(base_dir, "combined/3612.parquet")),
        pl.scan_parquet(os.path.join(base_dir, "combined/3603.parquet")),
    ]
).with_columns(
    [
        pl.col("upc").cast(pl.Int64),
        pl.col("store_code_uc").cast(pl.Int64),
        # Integer division converts YYYYMMDD (e.g., 20120107) directly to YYYY (2012)
        (pl.col("week_end") // 10000).cast(pl.Int32).alias("panel_year"),
    ]
)
```

Read this block:

- Line 18, `movements`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 33–44:

```
rms = (
    pl.scan_parquet(os.path.join(base_dir, "combined/rms.parquet"))
    .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
```

```
    .unique(subset=["upc"], keep="first")
)

# 3. Products table (deduplicated on exact UPC + Version Code composite key)
products = (
    pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
    .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
    .unique(subset=["upc", "upc_ver_uc"], keep="first")
)
```

Read this block:

- Line 33, `rms`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 40, `products`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 47–57:

```
stores = (
    pl.scan_parquet(os.path.join(base_dir, "combined/stores.parquet"))
    .rename({"year": "panel_year"})  # Safely renames 'year' to 'panel_year'
    .with_columns(
        [
            pl.col("store_code_uc").cast(pl.Int64),
            pl.col("panel_year").cast(pl.Int32),
        ]
    )
    .unique(subset=["store_code_uc", "panel_year"], keep="first")
)
```

Read this block:

- Line 47, `stores`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.

Source lines 60–72:

```
products_extra = (
    pl.scan_parquet(os.path.join(base_dir, "combined/products_extra.parquet"))
    .with_columns([pl.col("upc").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)])
    .sort("flavor_descr", descending=False, nulls_last=True)
    .unique(subset=["upc", "panel_year"], keep="first")
)

# 6. Brand Variations (deduplicated on brand code)
brand_var = (
    pl.scan_parquet(os.path.join(base_dir, "brand_variations.parquet"))
    .with_columns(pl.col("brand_code_uc").cast(pl.Int64))
    .unique(subset=["brand_code_uc"], keep="first")
)
```

Read this block:

- Line 60, `products_extra`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

- Line 68, `brand_var`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.

Source lines 75–89:

```
75 merged = (
76     movements
77     # Step A: Attach upc_ver_uc from RMS
78     .join(rms, on="upc", how="left", suffix="_rms")
79     # Step B: Match exact version in products
80     .join(products, on=["upc", "upc_ver_uc"], how="left", suffix="_prod")
81     # Step C: Match store metadata by store
82     .join(stores, on=["store_code_uc", "panel_year"], how="left", suffix="_str")
83     # Step D: Match extra annual metadata
84     .join(products_extra, on=["upc", "panel_year"], how="left", suffix="_xtra")
85     # Step E: Match master brand dictionary
86     .join(brand_var, on="brand_code_uc", how="left", suffix="_brand")
87 )
88
89 print("Streaming merged RMS dataset to disk...")
```

Read this block:

- Line 75, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 90–98:

```
90 merged.sink_parquet(final_out)
91 print(f"Successfully saved: {final_out}")
92
93 ## === Run Post-Merge Audit === ##
94 print("\n" + "=" * 60)
95 print("RUNNING POST-MERGE AUDIT: RMS DATASET")
96 print("=" * 60)
97
98 merged_scan = pl.scan_parquet(final_out)
```

Read this block:

- Line 90: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 101–107:

```
101 total_merged_rows = merged_scan.select(pl.len()).collect().item()
102 anchor_rows = movements.select(pl.len()).collect().item()
103 pct_retained = (total_merged_rows / anchor_rows) * 100
104
105 print(f"\n[1] Row Count & Integrity Audit:")
106 print(f"    - Movements Anchor Rows: {anchor_rows:,}")
107 print(f"    - Merged File Rows:     {total_merged_rows:,}")
```

Read this block:

- Line 101, `total_merged_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 102, `anchor_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 103, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 107: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 108–108:

```
108 print(f"    - Row Ratio:           {pct_retained:.2f}%")
```

Read this block:

- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 110–113:

```
110 if total_merged_rows != anchor_rows:
111     print("    ⚠  WARNING: Row count discrepancy detected! Check join keys.")
112 else:
113     print("    ✓ PERFECT MATCH: Exact 1:1 row retention (0 rows added or dropped).")
```

Read this block:

- Line 110: test `total_merged_rows != anchor_rows`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 111: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 116–124:

```
116 print("\n[2] Dimension Table Match Rates:")
117 match_targets = {
118     "products (brand_descr)": "brand_descr",
119     "stores (retailer_code)": "retailer_code",
120     "products_extra (flavor_descr)": "flavor_descr",
121     "brand_var": "brand_code_uc_brand"
122     if "brand_code_uc_brand" in merged_scan.collect_schema().names()
123     else "brand_code_uc",
124 }
```

Read this block:

- Line 116: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 117, `match_targets`: Set this name to a dictionary with named entries `products (brand_descr)`, `stores (retailer_code)`, `products_extra (flavor_descr)`, `brand_var`.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 126–135:

```
126 for dim_name, col_name in match_targets.items():
127     if col_name in merged_scan.collect_schema().names():
128         matched_count = (
129             merged_scan.filter(pl.col(col_name).is_not_null())
130             .select(pl.len())
131             .collect()
132             .item()
133         )
134         match_pct = (matched_count / total_merged_rows) * 100
135         print(f"    - {dim_name:<30}: {matched_count:,} matched ({match_pct:.2f}%)")
```

Read this block:

- Line 126: for each entry yielded by `match_targets.items()`, bind/unpack it into `(dim_name, col_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 127: test `col_name in merged_scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128, `matched_count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 134, `match_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 135: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.

Source lines 138–144:

```
138 file_size_gb = os.path.getsize(final_out) / (1024**3)
139 schema_cols = merged_scan.collect_schema().names()
140
```

```
print("\n[3] Output File Summary:")
print(f"    - Total Columns: {len(schema_cols)}")
print(f"    - File Size:     {file_size_gb:.2f} GB")
print(f"    - Output Path:   {final_out}")
```

Read this block:

- Line 138, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 139, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 141: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 142: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 143: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 144: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 145–146:

```
print("=" * 60)
print("AUDIT COMPLETE.")
```

Read this block:

- Line 145: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 146: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data__code/recover__flavors.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
"""Supplement the flavor CSV from annual RMS product metadata on the server."""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
 ↪  "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 4: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 6, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 9–13:

```
9  import argparse
10 from pathlib import Path
11
12
13 from summ_stats import DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover
```

Read this block:

- Line 9: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 10: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 13: from `summ_stats`, import `DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 28–29:

```
28 if __name__ == "__main__":
29     main()
```

Read this block:

- Line 28: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 29: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 19.

This small command-line wrapper imports the existing recovery function under the shorter local name `recover`. `as recover` changes only the imported name, not the implementation. Three Path arguments specify the original CSV, annual products, and output file; `recover(...)` does the actual work. Reusing that function avoids a

second recovery algorithm drifting away from the summary pipeline. The guarded entry point lets tests import the module without running the recovery job.

Definition and stated intent:

Source lines 19–19:

```
19 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–25:

```
20 parser = argparse.ArgumentParser(description=__doc__)
21 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
22 parser.add_argument("--products", type=Path, default=DATA_DIR / "RMS/combined/products_extra.parquet")
23 parser.add_argument("--output", type=Path, default=DATA_DIR / "recovered_flavor_reference.csv")
24 args = parser.parse_args()
25 recover(args.flavors, args.products, args.output)
```

Read this block:

- Line 20, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 21: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 25: call `recover` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `recover`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

data__code/summ__stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–12:

```
1 """Unweighted single-person summaries for the full panel and matched yogurt sample.
2
3 Run on the server: python3 summ_stats.py --help
4 No data are loaded when this file is imported.
```

```
"""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 8: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 10, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 11: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 12: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–20:

```
from pathlib import Path
import argparse
import tempfile
import time

import numpy as np
import pandas as pd
```

Read this block:

- Line 14: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 16: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 17: import `time`. An as-name is a convenient local alias, not another library.
- Line 19: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 20: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 21–29:

```
import polars as pl

DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
# Set this to the flavor-reference CSV's full path on the Slurm server.
FLAVOR_FILE = (
    DATA_DIR
```

```
    / "../nielsen_extracts/nielsen_extracts/RMS/Reference_Documentation/2006-
    ↪  2020_Documentation/Latest_Flavor_2010.csv"
)
FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 21: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 23, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 25, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 29, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 34–37:

```
RACE_LABELS = {}
INCOME_LABELS = {}
HISPANIC_LABELS = {}
AGE_LABELS = {}
```

Read this block:

- Line 34, `RACE_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 35, `INCOME_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 36, `HISPANIC_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 37, `AGE_LABELS`: Set this name to a dictionary from the displayed keys and values.

Source lines 691–692:

```
if __name__ == "__main__":
    main()
```

Read this block:

- Line 691: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 692: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 40.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

Definition and stated intent:

Source lines 40–41:

```
40 def normalize_upcs(values):
41     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 42–44:

```
42 text = values.astype("string").str.strip()
43 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
44 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 42, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 43, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 44, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 45–45:

```
45 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 45: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 48.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

Definition and stated intent:

Source lines 48–48:

```
48 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 49–51:

```
49 text = pl.col("upc").cast(pl.String).str.strip_chars()
50 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
51 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 49, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 50, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 51, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 52–54:

```
52 return pl.when(numeric).then(
53     pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical)
54 ).otherwise(text).alias("upc")
```

Read this block:

- Line 52: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

valid flavor labels

Original function: `valid_flavor_labels`. Source begins at line 57.

Input is a pandas table with `flavor_descr`; output is a Boolean Series. Missing text becomes an empty string, is cast to string, stripped, and lowercased. `.isin` checks a list of unusable sentinel strings, then `~` negates the result. Thus `" N/A "` is invalid while `"Strawberry"` is valid. Normalizing once before comparison is clearer than writing separate conditions for every case and whitespace variant. This helper exists alongside directly written lazy filters; it is not automatically applied to every flavor-processing path.

Definition and stated intent:

Source lines 57–57:

```
57 def valid_flavor_labels(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 58–58:

```
58 labels = frame["flavor_descr"].fillna("").astype(str).str.strip().str.lower()
```

Read this block:

- Line 58, `labels`: Set this name to the result of calling `.lower(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 59–59:

```
59 return ~labels.isin(["", "na", "n/a", "nan", "null", "none"])
```

Read this block:

- Line 59: return the negated or sign-adjusted expression `~labels.isin(['', 'na', 'n/a', 'nan', 'null', 'none'])`. This ends the call.

Syntax and calls used here:

- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

scan purchases

Original function: `scan_purchases`. Source begins at line 63.

This builds and returns a lazy, projected purchase query. It validates required column labels and maps optional source labels to a common schema. `aliases.items()` supplies output name plus candidate source names. `next(..., None)` picks the first available candidate; otherwise `pl.lit(None, dtype=pl.String)` makes an explicitly typed missing column. A consistent schema simplifies later summaries.

The comprehension builds expressions without reading rows. Dates, UPCs, quantities, and prices get consistent representations. Head-age zero codes become null, then `pl.coalesce` picks the first nonmissing head-age category, preferring female then male. This is a priority rule, not a mean. Eagerly loading all columns first would use more memory; dropping rows because an optional demographic is absent would change the summary population unnecessarily.

Definition and stated intent:

Source lines 63–64:

```
63 def scan_purchases(path, first_year, last_year):
64     """Project needed columns before loading; accept the names in current code."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `first_year`, `last_year`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 65–75:

```
scan = pl.scan_parquet(path)
names = scan.collect_schema().names()
required = [
    "household_code",
    "panel_year",
    "purchase_date",
    "trip_code_uc",
    "upc",
    "quantity",
]
missing = set(required) - set(names)
```

Read this block:

- Line 65, `scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 66, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 67, `required`: Set this name to a list containing 6 displayed entries.
- Line 75, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 76–77:

```
if missing:
    raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 76: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 78–87:

```
aliases = {
    "race": ["race"],
    "income": ["household_income", "income"],
    "hispanic_origin": ["hispanic_origin", "hispanic"],
    "household_size": ["household_size"],
    "female_head_age": ["female_head_age"],
    "male_head_age": ["male_head_age"],
    "dma_cd": ["dma_cd"],
}
expressions = [pl.col(c) for c in required]
```

Read this block:

- Line 78, `aliases`: Set this name to a dictionary with named entries `race`, `income`, `hispanic_origin`, `household_size`, `female_head_age`, `male_head_age`, `dma_cd`.
- Line 87, `expressions`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 88–93:

```
88 for target, choices in aliases.items():
89     source = next((c for c in choices if c in names), None)
90     expression = (
91         pl.col(source).cast(pl.String) if source else pl.lit(None, dtype=pl.String)
92     )
93     expressions.append(expression.alias(target))
```

Read this block:

- Line 88: for each entry yielded by `aliases.items()`, bind/unpack it into `(target, choices)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 89, `source`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 90, `expression`: Set this name to the true-branch value if `source` holds, otherwise the else-branch value.
- Line 93: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 95–109:

```
95 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
96 expressions.append(
97     pl.col(price_column).cast(pl.Float64).alias("price")
98     if price_column
99     else pl.lit(None, dtype=pl.Float64).alias("price")
100 )
101 purchases = (
102     scan.select(expressions)
103     .filter(pl.col("panel_year").is_between(first_year, last_year))
104     .with_columns(
105         pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date(),
106         normalized_upc_expr(),
107         pl.col("quantity").cast(pl.Float64),
108     )
109 )
```

Read this block:

- Line 95, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

- Line 96: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 101, `purchases`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 112–118:

```
112 age_columns = ["female_head_age", "male_head_age"]
113 purchases = purchases.with_columns(
114     [
115         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
116         for c in age_columns
117     ]
118 )
```

Read this block:

- Line 112, `age_columns`: Set this name to the list `['female_head_age', 'male_head_age']`.
- Line 113, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.

Source lines 119–119:

```
119 return purchases.with_columns(pl.coalesce(age_columns).alias("head_age"))
```

Read this block:

- Line 119: return the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

single person purchases

Original function: `single_person_purchases`. Source begins at line 122.

Filters purchase records to household size one before purchase weeks or histories are counted. This is a record-level sample restriction: filtering households after constructing their histories would retain excluded purchases in the stocks.

Definition and stated intent:

Source lines 122–123:

```
122 def single_person_purchases(scan):
123     """Match estimation: retain size-one purchase records before histories/weeks."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 124–125:

```
124 if "household_size" not in scan.collect_schema().names():
125     raise ValueError("Single-person summaries require household_size.")
```

Read this block:

- Line 124: test `'household_size' not in scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 125: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 126–126:

```
126 return scan.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 126: return the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

repeated yogurt households

Original function: `repeated_yogurt_households`. Source begins at line 129.

This keeps household histories with at least two distinct purchase weeks after the caller's year/price filters. `week` is a Polars expression using modulo arithmetic to reach Saturday. The query obtains each household's minimum and maximum week: unequal endpoints prove at least two distinct weeks without retaining a full distinct-week set.

Only eligible household IDs are collected. A semi join filters the original lazy purchase table while preserving its columns and all eligible purchase rows. Joining an expanded eligibility table could multiply rows; the semi join expresses membership only. Multiple trips in the same week do not satisfy this rule. Estimation later repeats a similar restriction after additional exclusions.

Definition and stated intent:

Source lines 129–134:

```
129 def repeated_yogurt_households(scan):
130     """Keep households with two distinct Saturday-ending purchase weeks.
131
132     Apply after year selection; multiple lines/trips in one week count once.
133     Summaries precede the estimation-only menu and IV exclusions.
134     """
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 135–145:

```
135 week = pl.col("purchase_date") + pl.duration(
136     days=(6 - pl.col("purchase_date").dt.weekday()) % 7
137 )
138 ids = (
139     scan.group_by("household_code")
140     .agg(week.min().alias("first_week"), week.max().alias("last_week"))
141     .filter(pl.col("first_week") != pl.col("last_week"))
142     .select("household_code")
143 )
144 # Materialize only household IDs so the repeated scan cannot cache all purchases.
145 ids = ids.collect(engine="streaming")
```

Read this block:

- Line 135, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 138, `ids`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 145, `ids`: Set this name to the result of calling `ids.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

Source lines 146–146:

```
146 return scan.join(ids.lazy(), on="household_code", how="semi")
```

Read this block:

- Line 146: return the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

household summary

Original function: `household_summary`. Source begins at line 149.

Input is a lazy purchase table; output is a lazy one-row-per-household query. The `expressions` list defines exact purchase totals, distinct counts, and observed years. Demographic history follows a separate path: select a narrow

table, deduplicate, stably sort, and take each field's first nonmissing value. Aggregating the full purchase table separately preserves exact trip/UPC/line counts while avoiding repeated wide demographic sorts.

`group_by(...).agg([...])` evaluates each listed expression per group. A validated one-to-one join reunites totals and demographic values. Later `with_columns` steps preserve original race, apply the Hispanic override, and calculate approximate weeks/trip rate. The second step can refer to the new `approx_observed_weeks` created in the first. Counting only observed years avoids filling panel gaps but still assumes full-year exposure within those years.

Definition and stated intent:

Source lines 149–150:

```
149 def household_summary(scan):
150     """One row per household, so frequent shoppers do not dominate demographics."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 151–160:

```
151 demographics = [
152     "race",
153     "income",
154     "hispanic_origin",
155     "household_size",
156     "head_age",
157     "female_head_age",
158     "male_head_age",
159     "dma_cd",
160 ]
```

Read this block:

- Line 151, `demographics`: Set this name to a list containing 8 displayed entries.

Source lines 162–167:

```
162 scan = scan.with_columns(
163     [
164         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
165         for c in ["female_head_age", "male_head_age"]
166     ]
167 )
```

Read this block:

- Line 162, `scan`: Set this name to the result of calling `scan.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 168–182:

```
168 expressions = [
169     pl.col("panel_year").n_unique().alias("observed_years"),
```

```
    pl.col("panel_year").min().alias("first_year"),
    pl.col("panel_year").max().alias("last_year"),
    pl.col("trip_code_uc").n_unique().alias("trips"),
    pl.col("purchase_date").n_unique().alias("shopping_days"),
    pl.col("upc").n_unique().alias("distinct_upcs"),
    pl.len().alias("purchase_lines"),
    pl.col("quantity").sum().alias("packages"),
]
# Keep exact purchase counts separate from the much narrower demographic
# history. Sort that history once for all fields, preserving input order
# for conflicting values on the same date. Removing identical rows cannot
# change the first nonmissing value.
totals = scan.group_by("household_code").agg(expressions)
```

Read this block:

- Line 168, `expressions`: Set this name to a list containing 8 displayed entries.
- Line 182, `totals`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 183–189:

```
history = (scan.select(["household_code", "panel_year", "purchase_date"] + demographics)
           .unique(maintain_order=True)
           .sort(["household_code", "panel_year", "purchase_date"], maintain_order=True))
values = history.group_by("household_code").agg([
    pl.col(c).drop_nulls().first().alias(c) for c in demographics
])
households = totals.join(values, on="household_code", how="left", validate="1:1")
```

Read this block:

- Line 183, `history`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 186, `values`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 189, `households`: Set this name to the result of calling `totals.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 190–197:

```
190 return households.with_columns(
191     pl.col("race").alias("race_original"),
192     pl.when(pl.col("hispanic_origin") == "1").then(pl.lit("Hispanic"))
193     .otherwise(pl.col("race")).alias("race"),
194     (pl.col("observed_years") * 365.25 / 7).alias("approx_observed_weeks"),
195 ).with_columns(
196     (pl.col("trips") / pl.col("approx_observed_weeks")).alias("trips_per_observed_week")
197 )
```

Read this block:

- Line 190: return the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

as lazy

Original function: `as_lazy`. Source begins at line 200.

This small adapter always returns a Polars LazyFrame. `isinstance` checks the runtime type. An existing LazyFrame is returned unchanged, a pandas table is converted through `pl.from_pandas(...).lazy()`, and the remaining expected Polars DataFrame receives `.lazy()`. This lets callers supply convenient small eager tables in tests while production uses lazy scans. It is not a universal converter: an arbitrary unsupported object without `.lazy()` will still fail. Avoiding a copy/conversion when already lazy preserves the existing query plan.

Definition and stated intent:

Source lines 200–200:

```
200 def as_lazy(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 201–202:

```
201 if isinstance(frame, pl.LazyFrame):
202     return frame
```

Read this block:

- Line 201: test `isinstance(frame, pl.LazyFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 202: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 203–204:

```
203 if isinstance(frame, pd.DataFrame):
204     return pl.from_pandas(frame).lazy()
```

Read this block:

- Line 203: test `isinstance(frame, pd.DataFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 204: return the result of calling `.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 205–205:

```
205 return frame.lazy()
```

Read this block:

- Line 205: return the result of calling `frame.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

partition purchases

Original function: `partition_purchases`. Source begins at line 209.

This executes a lazy scan once to write physical household-hash partitions, then returns a list of lazy scans over those files. `hasattr(pl, "PartitionBy")` checks API availability before starting. `(hash(seed=0) % batches)` maps every row with the same household ID to the same bucket. This is deterministic routing, not random sample selection.

`sink_parquet(pl.PartitionBy(...))` writes by bucket. The list comprehension includes only existing bucket directories, important because small examples can leave buckets empty. `sorted(directory.glob(...))` makes input order predictable. `hive_partitioning=False` prevents folder names from automatically adding a partition column that was intentionally excluded. The alternative of filtering/recollecting the original source for every bucket repeats large scans; physical partitions trade temporary disk space for reusable reads.

Definition and stated intent:

Source lines 209–210:

```
209 def partition_purchases(scan, work, prefix, batches):
210     """Read the projected source once, writing household hash partitions."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 211–212:

```
211 if not hasattr(pl, "PartitionBy"):
212     raise RuntimeError("Summary partitioning requires Polars >= 2.0; install est_code/requirements.txt.")
```

Read this block:

- Line 211: test `not hasattr(pl, 'PartitionBy')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 212: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `hasattr`: Check that an object exposes a named attribute before using an optional/version-dependent API.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 213–219:

```
213 directory = work / f"{prefix}_purchases"
214 began = time.monotonic()
215 print(f"{prefix}: partitioning purchases once into {batches} batches", flush=True)
216 (scan.with_columns((pl.col("household_code").hash(seed=0) % batches).alias("_batch"))
217  .sink_parquet(pl.PartitionBy(directory, key="_batch", include_key=False), engine="streaming"))
218 parts = [pl.scan_parquet(sorted(directory.glob(f"_batch={batch}/*.parquet")), hive_partitioning=False)
219          for batch in range(batches) if (directory / f"_batch={batch}").exists()]
```

Read this block:

- Line 213, `directory`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 214, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 215: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 216: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 218, `parts`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 220–221:

```
220 if not parts:
221     raise ValueError(f"Empty {prefix} sample in selected years.")
```

Read this block:

- Line 220: test `not parts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 221: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 222–222:

```
222 print(f"{prefix}: partitioning completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 222: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 223–223:

```
223 return parts
```

Read this block:

- Line 223: return the object currently named `parts`; plain assignment does not copy it. This ends the call.

household batches

Original function: `household_batches`. Source begins at line 227.

This accepts an optional list of already written partitions. The conditional expression reuses them if supplied; otherwise it creates them once. `enumerate` gives each partition an output filename suffix, `household_summary` runs on it, and `sink_parquet` writes its reduced result. The final wildcard scan returns one logical table of all household outputs without first concatenating them eagerly in Python. Households never cross partition boundaries, so exact household summaries can be computed independently and then stacked.

Definition and stated intent:

Source lines 227–228:

```
227 def household_batches(scan, work, prefix, batches, partitions=None):
228     # Each physical partition is read on its own; no full-source rescan per batch.
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`, `partitions`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 229–229:

```
229 partitions = partitions if partitions is not None else partition_purchases(scan, work, prefix, batches)
```

Read this block:

- Line 229, `partitions`: Set this name to the true-branch value if `partitions is not None` holds, otherwise the else-branch value.

Syntax and calls used here:

- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 230–234:

```
230 for batch, subset in enumerate(partitions):
231     began = time.monotonic()
232     print(f"{prefix}: household batch {batch + 1}/{len(partitions)}", flush=True)
233     household_summary(subset).sink_parquet(work / f"{prefix}_{batch}.parquet", engine="streaming")
234     print(f"{prefix}: household batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 230: for each entry yielded by `enumerate(partitions)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 231, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 232: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 233: call `household_summary(subset).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 234: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `household_summary`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 235–235:

```
235 return pl.scan_parquet(work / f"{prefix}_*.parquet")
```

Read this block:

- Line 235: return the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

save table

Original function: `save_table`. Source begins at line 239.

This writes both a machine-readable CSV and an escaped LaTeX table and prints a readable version. Polars inputs are collected and converted to pandas because the chosen table formatting/export methods belong to pandas. A pandas input already has those methods, so conversion is skipped.

`lambda value: f"{value:.2f}"` formats display numbers without changing the stored numerical data. `index=False` suppresses an unwanted row-number column. `escape=True` protects LaTeX syntax characters in labels. CSV retains reusable data whereas LaTeX supplies paper-ready formatting; either output alone would serve only one of those purposes. This function's useful result is its side effects, so no explicit return is required.

Definition and stated intent:

Source lines 239–240:

```
239 def save_table(frame, output, name, caption):
240     """Keep machine-readable CSV beside the paper-ready LaTeX table."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `output`, `name`, `caption`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 241–242:

```
241 if isinstance(frame, (pl.LazyFrame, pl.DataFrame)):
242     frame = as_lazy(frame).collect(engine="streaming").to_pandas()
```

Read this block:

- Line 241: test `isinstance(frame, (pl.LazyFrame, pl.DataFrame))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 242, `frame`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 243–248:

```
243 print(f"\n{caption} [{name}]", flush=True)
244 print(frame.to_string(index=False, float_format=lambda value: f"{value:.2f}"), flush=True)
245 frame.to_csv(output / f"{name}.csv", index=False)
246 (output / f"{name}.tex").write_text(
247     frame.to_latex(index=False, escape=True, float_format="%.2f", caption=caption)
248 )
```

Read this block:

- Line 243: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 244: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 245: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 246: call `(output / f'{name}.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_latex`: Serialize a pandas table as LaTeX source, which needs a separate TeX compilation to become PDF.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

demographic tables

Original function: `demographic_tables`. Source begins at line 251.

Input is a one-row-per-household table, an output directory, and filename prefix. The initial count is its denominator. The loop builds one categorical table per demographic using null-to-`Missing` replacement, grouped counts, optional codebook mappings, and percentages of all households. `.map(labels).fillna(code)` keeps source codes visible when a verified label is absent.

The `specs` list pairs human-readable metric names with Polars aggregation expressions. Enumerated string aliases let all metrics be computed in a single query and then zipped back to names. `rows.extend(generator)` appends race percentages to the overview. Equal household weighting is achieved by summarizing households first; directly counting purchase rows would overweight frequent shoppers. Numeric means of income/head-age codes remain means of categories, not economic levels.

Definition and stated intent:

Source lines 251–251:

```
251 def demographic_tables(households, output, prefix):
```

`def` creates this callable; the parentheses name its inputs (`households`, `output`, `prefix`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 252–254:

```
252 households = as_lazy(households)
253 count = households.select(pl.len()).collect(engine="streaming").item()
254 race_table = None
```

Read this block:

- Line 252, `households`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 253, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 254, `race_table`: Set this name to the literal value `None`.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 255–267:

```
255 for column, labels in [("race", RACE_LABELS), ("hispanic_origin", HISPANIC_LABELS),
256                        ("income", INCOME_LABELS), ("household_size", {}),
257                        ("head_age", AGE_LABELS), ("female_head_age", AGE_LABELS),
258                        ("male_head_age", AGE_LABELS)]:
259     table = (households.select(pl.col(column).cast(pl.String).fill_null("Missing").alias("code"))
260              .group_by("code").len(name="households").sort("code")
261              .collect(engine="streaming").to_pandas())
262     table["category"] = table["code"].map(labels).fillna(table["code"])
263     table["percent_all_households"] = 100 * table["households"] / count
264     save_table(table, output, f"{prefix}_{column}",
265                f"{prefix.replace('_', ' ').title()}: {column.replace('_', ' ')} (unweighted)")
266     if column == "race":
267         race_table = table
```

Read this block:

- Line 255: for each entry yielded by `[('race', RACE_LABELS), ('hispanic_origin', HISPANIC_LABELS), ...]`, bind/unpack it into `(column, labels)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 259, `table`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 262, `table['category']`: Store the result of calling `.fillna(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 263, `table['percent_all_households']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 264: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 266: test `column == 'race'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 267, `race_table`: Set this name to the object currently named `table`; plain assignment does not copy it.

Syntax and calls used here:

- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 268–282:

```
268 specs = [
269     ("Households", pl.len()),
270     ("Mean observed panel years", pl.col("observed_years").mean()),
271     ("Mean trips per household", pl.col("trips").mean()),
272     ("Mean trips per approximate observed week", pl.col("trips_per_observed_week").mean()),
273     ("Median trips per approximate observed week", pl.col("trips_per_observed_week").median()),
274     ("Mean distinct UPCs per household", pl.col("distinct_upcs").mean()),
275     ("Mean purchase lines per household", pl.col("purchase_lines").mean()),
276     ("Mean packages per household", pl.col("packages").mean()),
277     ("Mean head age (source category code)", pl.col("head_age").cast(pl.Float64, strict=False).mean()),
278     ("Households with numeric head age", pl.col("head_age").cast(pl.Float64, strict=False).count()),
279     ("Mean household income (source category code)", pl.col("income").cast(pl.Float64, strict=False).mean()),
280     ("Households with numeric income", pl.col("income").cast(pl.Float64, strict=False).count()),
281 ]
282 values = households.select([expr.alias(str(i)) for i, (_, expr) in
    ↪  enumerate(specs)]).collect(engine="streaming").row(0)
```

Read this block:

- Line 268, `specs`: Set this name to a list containing 12 displayed entries.
- Line 282, `values`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `str`: Convert to text for paths, messages, or serialization.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 283–287:

```
283 rows = [(name, value) for (name, _), value in zip(specs, values)]
284 rows.extend((f"Race: {row.category} (% of all households)", row.percent_all_households)
285             for row in race_table.itertuples())
286 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, f"{prefix}_overview",
287           f"{prefix.replace('_', ' ').title()}: Household purchase summary (unweighted)")
```

Read this block:

- Line 283, `rows`: Set this name to a list computed by the displayed for-expression.
- Line 284: call `rows.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 286: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

scan reference

Original function: `scan_reference`. Source begins at line 291.

This wraps `pl.scan_csv` with consistent parsing choices. `infer_schema=False` keeps reference fields as strings instead of guessing that UPCs are numbers. `null_values=[...]` explicitly recognizes several missing-value spellings. The return is lazy, so calling the function sets up a CSV read without loading the entire table. Centralizing these options prevents different recovery and classification paths from interpreting the same file differently.

Definition and stated intent:

Source lines 291–291:

```
291 def scan_reference(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 292–293:

```
292 return pl.scan_csv(path, infer_schema=False,
293                   null_values=["NA", "N/A", "NaN", "nan", "NULL", "null", "None", "<NA>"])
```

Read this block:

- Line 292: return the result of calling `pl.scan_csv(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

recover flavor reference

Original function: `recover_flavor_reference`. Source begins at line 297.

This supplements, rather than overwrites, the descriptor reference. Both input tables are lazy; an anti join excludes product UPCs already present in the CSV. The filter removes missing/placeholder candidate descriptors. If annual metadata exists, each UPC's candidate columns take earliest-year values; otherwise first row deduplication is used. The output is a new combined reference plus a separate CSV of just recovered rows.

`.with_name(output.stem + ...)` constructs a sibling output filename. Comparing resolved paths rejects accidental overwrite of the original reference. After writing additions, `scan_reference` rereads them under the same schema conventions before diagonal relaxed concatenation allows unequal optional columns. The return is the output

Path, enabling `run` to use it for subsequent classification. It does not repair blank descriptors on UPCs already in the original reference.

Definition and stated intent:

Source lines 297–297:

```
297 def recover_flavor_reference(reference_path, products_path, output):
```

`def` creates this callable; the parentheses name its inputs (`reference_path`, `products_path`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 298–298:

```
298 reference = scan_reference(reference_path).with_columns(normalized_upc_expr())
```

Read this block:

- Line 298, `reference`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 299–300:

```
299 if not {"upc", "flavor_descr"}.issubset(reference.collect_schema().names()):
300     raise ValueError("Reference needs upc and flavor_descr columns.")
```

Read this block:

- Line 299: test `not {'upc', 'flavor_descr'}.issubset(reference.collect_schema().names())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 300: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 301–302:

```
301 products = pl.scan_parquet(products_path)
302 names = products.collect_schema().names()
```

Read this block:

- Line 301, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 302, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 303–304:

```
303 if not {"upc", "flavor_descr"}.issubset(names):
304     raise ValueError(f"{products_path}: needs upc and flavor_descr columns.")
```

Read this block:

- Line 303: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 304: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 305–312:

```
305 columns = [c for c in ["upc", "flavor_descr", "panel_year", "product_descr"] if c in names]
306 products = (products.select(columns).with_columns(normalized_upc_expr())
307             .join(reference.select("upc").unique(), on="upc", how="anti"))
308 products = products.filter(
309     pl.col("upc").is_not_null() & pl.col("flavor_descr").is_not_null()
310     & ~pl.col("flavor_descr").cast(pl.String).str.strip_chars().str.to_lowercase()
311     .is_in(["", "na", "n/a", "nan", "null", "none"])
312 )
```

Read this block:

- Line 305, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 306, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 308, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 313–319:

```
313 if "panel_year" in names:
314     recovered = products.group_by("upc").agg([
315         pl.col(c).sort_by("panel_year", nulls_last=True).first().alias(c)
316         for c in columns if c != "upc"
317     ])
318 else:
```

```
319     recovered = products.unique(subset="upc", keep="first", maintain_order=True)
```

Read this block:

- Line 313: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 314, `recovered`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 319, `recovered`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 320–321:

```
320 output = Path(output)
321 output.parent.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 320, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 321: call `output.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 322–323:

```
322 if output.resolve() == Path(reference_path).resolve():
323     raise ValueError("Choose a separate output file to preserve the original reference.")
```

Read this block:

- Line 322: test `output.resolve() == Path(reference_path).resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 323: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 324–329:

```
324 recovered_path = output.with_name(output.stem + "_recovered_rows.csv")
325 recovered.sink_csv(recovered_path, engine="streaming")
326 additions = scan_reference(recovered_path)
327 pl.concat([reference, additions], how="diagonal_relaxed").sink_csv(output, engine="streaming")
328 count = additions.select(pl.len()).collect(engine="streaming").item()
329 print(f"Recovered labels for {count:,} additional UPCs. Existing entries retained. Saved {output}", flush=True)
```

Read this block:

- Line 324, `recovered_path`: Set this name to the result of calling `output.with_name(...)`; the arguments and preceding method chain are shown above.
- Line 325: call `recovered.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 326, `additions`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 327: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 328, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 329: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 330–330:

```
330 return output
```

Read this block:

- Line 330: return the object currently named `output`; plain assignment does not copy it. This ends the call.

flavor reference

Original function: `flavor_reference`. Source begins at line 333.

This resolves one label per normalized UPC and returns an eager Polars mapping. `detailed` selects two different expression recipes. Detailed labels use trimmed lowercase descriptors with whitespace collapsed; grouped labels use ordered plain/berry text matching and an other fallback. `.replace` maps specific whole values; regex `.str.replace_all` instead matches patterns within text.

Before deduplicating, the function counts UPCs assigned multiple distinct labels and can save their rows for review. `unique(..., keep="first", maintain_order=True)` implements first-reference-row precedence reproducibly. This is a transparent rule but not a semantic resolution of conflicting descriptions. Returning only UPC and final flavor gives downstream joins a small unique lookup table.

Definition and stated intent:

Source lines 333–334:

```
333 def flavor_reference(path, detailed=False, output=None):
334     """Use first reference row per normalized UPC; keep missing descriptors."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `detailed`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 335–337:

```
335 frame = scan_reference(path)
336 names = frame.collect_schema().names()
337 missing = {"upc", "flavor_descr"} - set(names)
```

Read this block:

- Line 335, `frame`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 336, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 337, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 338–339:

```
338 if missing:
339     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 338: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 339: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 340–340:

```
340 frame = frame.with_columns(normalized_upc_expr())
```

Read this block:

- Line 340, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 341–342:

```
341 if frame.select((pl.col("upc").is_null() | (pl.col("upc") == "")).any()).collect(engine="streaming").item():
342     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 341: test `frame.select((pl.col('upc').is_null() | (pl.col('upc') == '')).any()).collect(...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 342: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 343–343:

```
343 text = pl.col("flavor_descr").fill_null("")
```

Read this block:

- Line 343, `text`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.

Source lines 344–345:

```
344 if "product_descr" in names:
345     text = text + pl.lit(" ") + pl.col("product_descr").fill_null("")
```

Read this block:

- Line 344: test `'product_descr' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 345, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

Source lines 346–346:

```
346 text = text.str.to_lowercase()
```

Read this block:

- Line 346, `text`: Set this name to the result of calling `text.str.to_lowercase(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.

Source lines 347–353:

```
347 if detailed:
348     flavor = (pl.col("flavor_descr").fill_null("").str.to_lowercase().str.strip_chars()
349               .str.replace_all(r"\s+", " ").replace({"": "unknown", "unflavored": "plain"}))
350 else:
351     flavor = (pl.when(text.str.contains("plain|unflavored")).then(pl.lit("plain"))
352               .when(text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"))
353               .then(pl.lit("berry")).otherwise(pl.lit("other")))
```

Read this block:

- Line 347: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 348, `flavor`: Set this name to the result of calling `.replace(...)`; the arguments and preceding method chain are shown above.
- Line 351, `flavor`: Set this name to the result of calling `.otherwise(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.replace_all`: Replace every regex match within each string, such as collapsing repeated whitespace.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.

Source lines 354–356:

```
354 frame = frame.with_columns(flavor.alias("flavor"))
355 conflicting = frame.group_by("upc").agg(pl.col("flavor").n_unique().alias("n")).filter(pl.col("n") > 1)
356 count = conflicting.select(pl.len()).collect(engine="streaming").item()
```

Read this block:

- Line 354, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 355, `conflicting`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 356, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.

Source lines 357–361:

```
357 if count:
358     print(f"{count:,} UPCs have conflicting flavor classifications; keeping the first CSV row.", flush=True)
359     if output is not None:
360         name = "detailed_flavor_conflicts.csv" if detailed else "aggregated_flavor_conflicts.csv"
361         frame.join(conflicting.select("upc"), on="upc", how="semi").sink_csv(output / name, engine="streaming")
```

Read this block:

- Line 357: test `count`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 358: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 359: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 360, `name`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 361: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 362–363:

```
362 blanks = frame.select((pl.col("flavor_descr").fill_null("").str.strip_chars() ==
    ↪  "").sum()).collect(engine="streaming").item()
363 print(f"Flavor-reference rows with blank descriptors: {blanks:,}", flush=True)
```

Read this block:

- Line 362, `blanks`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 363: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 364–364:

```
364 return frame.select("upc", "flavor").unique(subset="upc", keep="first",
    ↪  maintain_order=True).collect(engine="streaming")
```

Read this block:

- Line 364: return the result of calling `.collect(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

yogurt weeks

Original function: `yogurt_weeks`. Source begins at line 367.

This attaches flavor labels, audits matched purchase lines, and returns a lazy weekly quantity query plus an eager small audit table. A left join preserves unmatched purchases long enough to count/report them. Unmatched UPCs are exported and cause a ValueError, preventing an apparently successful summary of a silently reduced sample. Nonpositive/missing quantities also stop the run.

The Saturday expression adds a row-dependent duration to purchase date. Grouping by household/week/flavor and summing quantity keeps only observed combinations. The alternative dense pivot would create a column for every detailed flavor and potentially a great many empty cells. Price problems are audited here; the main matched-sample caller has already applied its positive-price filter.

Definition and stated intent:

Source lines 367–368:

```
367 def yogurt_weeks(scan, reference, flavors=FLAVORS, output=None):
368     """Keep sparse household/week/flavor counts rather than a dense pivot."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `reference`, `flavors`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 369–375:

```
369 joined = scan.join(reference.lazy(), on="upc", how="left")
370 audit = joined.select(
371     pl.len().alias("purchase_lines"),
372     pl.col("flavor").is_null().sum().alias("unmatched_flavor_lines"),
373     ((~pl.col("price").is_finite()) | (pl.col("price") <=
    ↪ 0)).fill_null(True).sum().alias("missing_or_nonpositive_quoted_prices"),
374     ((~pl.col("quantity").is_finite()) | (pl.col("quantity") <=
    ↪ 0)).fill_null(True).sum().alias("invalid_quantity_lines"),
375 ).collect(engine="streaming")
```

Read this block:

- Line 369, `joined`: Set this name to the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above.
- Line 370, `audit`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 376–381:

```
376 if audit["unmatched_flavor_lines"][0]:
377     unmatched = (joined.filter(pl.col("flavor").is_null()).group_by("upc")
378                  .agg(pl.len().alias("purchase_lines"), pl.col("quantity").sum().alias("packages")))
379     if output is not None:
380         unmatched.sink_csv(output / "unmatched_purchase_upcs.csv", engine="streaming")
381     raise ValueError("Purchased UPCs still lack flavor labels; see unmatched_purchase_upcs.csv.")
```

Read this block:

- Line 376: test `audit['unmatched_flavor_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 377, `unmatched`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 379: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 380: call `unmatched.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 381: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Source lines 382–383:

```
382 if audit["invalid_quantity_lines"][0]:
383     raise ValueError("Nonpositive or missing quantities; audit the matched sample first.")
```

Read this block:

- Line 382: test `audit['invalid_quantity_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 384–386:

```
384 week = pl.col("purchase_date") + pl.duration(days=(6 - pl.col("purchase_date").dt.weekday()) % 7)
385 grouped = (joined.with_columns(week.alias("week"))
386            .group_by(["household_code", "week", "flavor"]).agg(pl.col("quantity").sum()))
```

Read this block:

- Line 384, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 385, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

Source lines 387–387:

```
387 return grouped, audit
```

Read this block:

- Line 387: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

flavor summaries

Original function: `flavor_summaries`. Source begins at line 391.

This is the main descriptive-statistics engine. It returns pandas share and transition tables for plotting, while writing much richer outputs. First it sums quantities by flavor and computes package shares; a rounding filter removes rows whose displayed share is zero but leaves the denominator unchanged. Next it aggregates household/flavor totals to number of flavors, squared quantities, and largest quantity, producing HHI and largest-flavor share.

The basket query groups again by household/week. `pl.when(pl.len() == 1)` gives a single-flavor label only to unmixed baskets. `.shift().over("household_code")` looks at the previous basket within the household, not across household borders. Date differences identify adjacent weeks. A cumulative sum of spell-start flags creates run identifiers; mixed baskets and gaps break the runs.

The expensive basket query is checkpointed once in a temporary directory so transitions, spells, CSV exports, and global medians can reuse it. Transition counts pivot into a small matrix; dividing by row sums produces probabilities. Zero denominators become NaN. Spells are averaged as spells, not as households. Finally a small metric table records basket frequencies and eligible-pair switching. This sequence uses several aggregation units intentionally; replacing them all with one grand groupby would obscure their different denominators.

Definition and stated intent:

Source lines 391–391:

```
391 def flavor_summaries(weekly, output, flavors=FLAVORS):
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `output`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 392–393:

```
392 weekly = as_lazy(weekly)
393 totals = weekly.group_by("flavor").agg(pl.col("quantity").sum().alias("packages")).collect(engine="streaming")
```

Read this block:

- Line 392, `weekly`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 393, `totals`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 394–395:

```
394 if flavors != FLAVORS:
395     flavors = sorted(totals["flavor"].to_list())
```

Read this block:

- Line 394: test `flavors != FLAVORS`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 395, `flavors`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 397–405:

```
397 shares = (pl.DataFrame({"flavor": flavors}).join(totals, on="flavor", how="left")
398           .with_columns(pl.col("packages").fill_null(0))
399           .with_columns((100 * pl.col("packages") / pl.col("packages").sum()).alias("percent_packages")))
400 # Match the table's two-decimal display; keep the original denominator.
401 shares = shares.filter(pl.col("percent_packages").round(2) > 0).sort(
402     ["percent_packages", "flavor"], descending=[True, False]
403 )
```

```
404 save_table(shares, output, "yogurt_flavor_shares", "Flavor shares of purchased packages")
405 counts = weekly.group_by(["household_code", "flavor"]).agg(pl.col("quantity").sum())
```

Read this block:

- Line 397, `shares`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 401, `shares`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 404: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 405, `counts`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 406–415:

```
406 households = (counts.group_by("household_code").agg(
407     pl.len().alias("flavors_bought"), pl.col("quantity").sum().alias("packages"),
408     pl.col("quantity").pow(2).sum().alias("squared_packages"),
409     pl.col("quantity").max().alias("largest_packages"))
410     .with_columns((pl.col("squared_packages") / pl.col("packages").pow(2)).alias("flavor_HHI"),
411                   (pl.col("largest_packages") / pl.col("packages")).alias("largest_flavor_share"))
412     .select("household_code", "flavors_bought", "flavor_HHI", "largest_flavor_share"))
413 hh_path = output / "yogurt_household_flavors.csv"
414 households.sink_csv(hh_path, engine="streaming")
415 hh_metrics = pl.scan_csv(hh_path).select(pl.len(), pl.col("flavors_bought").mean(),
    ↪  pl.col("flavor_HHI").mean()).collect(engine="streaming").row(0)
```

Read this block:

- Line 406, `households`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 413, `hh_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 414: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 415, `hh_metrics`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.pow`: Raise each expression value to a power, such as squaring quantities for concentration calculations.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

Source lines 416–425:

```
416 baskets = (weekly.group_by(["household_code", "week"]).agg(
417     pl.col("quantity").sum().alias("packages"), pl.len().alias("flavors_in_basket"),
418     pl.when(pl.len() == 1).then(pl.col("flavor").first()).otherwise(None).alias("single_flavor"))
419     .sort(["household_code", "week"])
420     .with_columns(pl.col("single_flavor").shift().over("household_code").alias("previous_flavor"),
421                   (pl.col("week").diff().over("household_code").dt.total_days() / 7).alias("gap_weeks"))
422     .with_columns((pl.col("single_flavor").is_not_null() & pl.col("previous_flavor").is_not_null()
423                    & (pl.col("gap_weeks") == 1)).fill_null(False).alias("eligible"))
424     .with_columns((~(pl.col("eligible") & (pl.col("single_flavor") == pl.col("previous_flavor")))).fill_null(False))
425                   .cast(pl.UInt64).cum_sum().alias("spell")))
```

Read this block:

- Line 416, `baskets`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.cum_sum`: Accumulate running totals, used to turn spell-start flags into spell IDs.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.over`: Evaluate a Polars expression within the specified group/window, preserving row-level results.
- `.total_days`: Convert a duration expression into its elapsed number of days before comparing weekly gaps.
- `.shift`: Move values by one row by default; over(household) prevents the shift crossing household boundaries.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 427–460:

```
427 with tempfile.TemporaryDirectory(prefix="baskets-", dir=output) as tmp:
428     checkpoint = Path(tmp) / "baskets.parquet"
429     baskets.sink_parquet(checkpoint, engine="streaming")
430     baskets = pl.scan_parquet(checkpoint)
431     baskets.sink_csv(output / "yogurt_weekly_baskets.csv", engine="streaming")
432     weekly.sink_csv(output / "yogurt_weekly_flavor_quantities.csv", engine="streaming")
433     transitions = (baskets.filter(pl.col("eligible")).group_by(["previous_flavor", "single_flavor"])
434                    .len().collect(engine="streaming").to_pandas())
435     if transitions.empty:
436         matrix = pd.DataFrame(0, index=flavors, columns=flavors)
437     else:
438         matrix = transitions.pivot(index="previous_flavor", columns="single_flavor",
    ↪ values="len").fillna(0).reindex(index=flavors, columns=flavors, fill_value=0)
439     matrix.index.name = "previous_flavor"
440     matrix.columns.name = "single_flavor"
441     matrix.to_csv(output / "yogurt_transition_counts.csv")
442     probabilities = matrix.div(matrix.sum(axis=1).replace(0, np.nan), axis=0)
443     probabilities.to_csv(output / "yogurt_transition_probabilities.csv")
444     print("\nYogurt transition counts", flush=True)
445     print(matrix.to_string(), flush=True)
```

```
446     print("\nYogurt transition probabilities", flush=True)
447     print(probabilities.to_string(float_format=lambda v: f"{v:.2f}"), flush=True)
448     spells = (baskets.filter(pl.col("single_flavor").is_not_null()).group_by(["household_code", "spell"])
449               .agg(pl.col("single_flavor").first().alias("flavor"), pl.len().alias("weeks")))
450     spells_path = Path(tmp) / "spells.parquet"
451     spells.sink_parquet(spells_path, engine="streaming")
452     spells = pl.scan_parquet(spells_path)
453     spells.sink_csv(output / "yogurt_single_flavor_spells.csv", engine="streaming")
454     spell_mean = spells.select(pl.col("weeks").mean()).collect(engine="streaming").item()
455     bm = baskets.select(
456         pl.len().alias("weeks"), (100 * (pl.col("flavors_in_basket") > 1).mean()).alias("mixed"),
457         (100 * (pl.col("packages") > 7).mean()).alias("above_seven"), pl.col("eligible").sum().alias("pairs"),
458         (100 * (pl.col("single_flavor") !=
    ↪ pl.col("previous_flavor")).filter(pl.col("eligible")).mean()).alias("switch_rate"),
459         pl.col("gap_weeks").median().alias("gap"),
460     ).collect(engine="streaming").row(0, named=True)
```

Read this block:

- Line 427: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 428, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 429: call `baskets.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 430, `baskets`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 431: call `baskets.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 432: call `weekly.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 433, `transitions`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 435: test `transitions.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 436, `matrix`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 438, `matrix`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 439, `matrix.index.name`: Set this name to the literal text `'previous_flavor'`.
- Line 440, `matrix.columns.name`: Set this name to the literal text `'single_flavor'`.
- Line 441: call `matrix.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 442, `probabilities`: Set this name to the result of calling `matrix.div(...)`; the arguments and preceding method chain are shown above.
- Line 443: call `probabilities.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 444: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 445: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 446: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 447: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 448, `spells`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

- Line 450, `spells_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 451: call `spells.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 452, `spells`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 453: call `spells.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 454, `spell_mean`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 455, `bm`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.div`: Divide a pandas object with explicit alignment options; axis=0 aligns the denominator by rows.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 461–468:

```
461 rows = [
462     ("Matched households", hh_metrics[0]), ("Observed yogurt purchase weeks", bm["weeks"]),
463     ("Mean flavors bought per household", hh_metrics[1]), ("Mean household flavor HHI (0 to 1)", hh_metrics[2]),
464     ("Mixed-flavor weeks (%)", bm["mixed"]), ("Weeks above seven packages (%)", bm["above_seven"]),
465     ("Eligible adjacent single-flavor week pairs", bm["pairs"]), ("Switch rate in eligible pairs (%)",
    ↪  bm["switch_rate"]),
466     ("Mean single-flavor spell length (consecutive calendar weeks)", spell_mean), ("Median gap between purchase
    ↪  weeks", bm["gap"]),
467 ]
468 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, "yogurt_dynamics", "Matched yogurt sample:
    ↪  purchase dynamics")
```

Read this block:

- Line 461, `rows`: Set this name to a list containing 10 displayed entries.

- Line 468: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 469–469:

```
469 return shares.to_pandas(), probabilities
```

Read this block:

- Line 469: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

make hhi figures

Original function: `make_hhi_figures`. Source begins at line 473.

This reads just the saved `flavor_HHI` column and writes histogram and CDF PDFs. `usecols` avoids reading irrelevant household fields. `.dropna().to_numpy()` leaves a numeric vector. If it is empty, a bare return avoids dividing by zero. The histogram weights each household by `100 / n`, so heights are percentages. `linspace(0,1,41)` defines 40 bins.

The CDF uses sorted observations and cumulative percentages. `np.r_` concatenates an initial zero with the vectors to draw the starting segment. `where="post"` creates a right-continuous step plot. A loop over two `kind` strings shares styling/export code without confusing their different calculations. It reuses saved household summaries instead of rerunning a large private-data pipeline.

Definition and stated intent:

Source lines 473–474:

```
473 def make_hhi_figures(output):
474     """Equal-household distributions using all flavor labels, without top-20 truncation."""
```

`def` creates this callable; the parentheses name its inputs (`output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 475–479:

```
475 import matplotlib
476 matplotlib.use("Agg")
477 import matplotlib.pyplot as plt
478
479 values = pd.read_csv(output / "yogurt_household_flavors.csv",
    ↪  usecols=["flavor_HHI"])["flavor_HHI"].dropna().to_numpy()
```

Read this block:

- Line 475: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 476: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 477: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 479, `values`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 480–481:

```
480 if not len(values):
481     return
```

Read this block:

- Line 480: test `not len(values)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 481: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 482–494:

```
482 for kind in ["histogram", "cdf"]:
483     fig, ax = plt.subplots(figsize=(7, 5))
484     if kind == "histogram":
485         ax.hist(values, bins=np.linspace(0, 1, 41), weights=np.full(len(values), 100 / len(values)),
    ↪  color="steelblue")
486         ax.set_ylabel("Households (%)")
487     else:
488         ax.step(np.r_[0., np.sort(values)], np.r_[0., 100 * np.arange(1, len(values) + 1) / len(values)],
    ↪  where="post")
489         ax.set_ylabel("Cumulative households (%)")
490     ax.set(xlim=(0, 1), xlabel="Household flavor HHI (quantity shares)", title="Yogurt flavor concentration")
491     fig.tight_layout()
492     fig.savefig(output / f"flavor_hhi_{kind}.pdf")
493     plt.close(fig)
494     print(f"Saved HHI figure: {output / f'flavor_hhi_{kind}.pdf'}", flush=True)
```

Read this block:

- Line 482: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 483, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 484: test `kind == 'histogram'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 485: call `ax.hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 486: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 488: call `ax.step` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 489: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 490: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 491: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 492: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 493: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 494: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.step`: Draw an empirical step curve; where='post' extends each value to the right.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

make figures

Original function: `make_figures`. Source begins at line 497.

Inputs are already computed flavor-share and transition tables. Sorting then `head(20)` selects a readable subset for plots, and `reindex` aligns matrix rows and columns with the displayed flavor order. It does not recompute transition denominators within that subset, so a plotted row need not sum to one.

`fig, ax = plt.subplots` separates a Figure container from its plotting Axes. `ax.bar` draws shares; `ax.imshow` colors transition cells. Two nested index loops place formatted text inside each cell. Tick rotation improves long-label readability. `tight_layout`, `savefig`, and `close` finish each output. Plotting inside the data aggregation itself would make it harder to skip figures or recreate them from existing summary tables.

Definition and stated intent:

Source lines 497–498:

```
497 def make_figures(shares, transitions, output):
498     # Limit figure dimensions; all flavors remain in the saved tables.
```

`def` creates this callable; the parentheses name its inputs (`shares`, `transitions`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 499–506:

```
499 shares = shares.sort_values("packages", ascending=False).head(20)
500 flavors = shares["flavor"].tolist()
501 transitions = transitions.reindex(index=flavors, columns=flavors)
502 # Import plotting only if requested (use --no-figures for tables alone).
503 import matplotlib
504
505 matplotlib.use("Agg")
506 import matplotlib.pyplot as plt
```

Read this block:

- Line 499, `shares`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 500, `flavors`: Set this name to the result of calling `shares['flavor'].tolist(...)`; the arguments and preceding method chain are shown above.
- Line 501, `transitions`: Set this name to the result of calling `transitions.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 503: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 505: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 506: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 508–513:

```
508 fig, ax = plt.subplots(figsize=(max(6, len(flavors) * 0.35), 5))
509 ax.bar(shares["flavor"], shares["percent_packages"], color="steelblue")
510 ax.tick_params(axis="x", labelrotation=90)
511 ax.set(ylabel="Share of purchased packages (%)", title="Yogurt flavor shares (up to 20 most purchased flavors)")
512 fig.tight_layout()
513 fig.savefig(output / "flavor_shares.pdf")
```

Read this block:

- Line 508, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 509: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 510: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 511: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 512: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 513: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.tick_params`: Adjust axis tick appearance, for example rotating long category labels.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 514–527:

```
514 plt.close(fig)
515 side = max(6, len(flavors) * 0.4)
516 fig, ax = plt.subplots(figsize=(side, side))
517 image = ax.imshow(transitions.to_numpy(), vmin=0, vmax=1, cmap="Blues")
518 ax.set(
519     xticks=range(len(flavors)),
520     yticks=range(len(flavors)),
521     xticklabels=flavors,
522     yticklabels=flavors,
523     xlabel="Current flavor",
524     ylabel="Previous flavor",
525     title="Adjacent single-flavor purchase weeks",
526 )
527 ax.tick_params(axis="x", labelrotation=90)
```

Read this block:

- Line 514: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 515, `side`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 516, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 517, `image`: Set this name to the result of calling `ax.imshow(...)`; the arguments and preceding method chain are shown above.
- Line 518: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 527: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.imshow`: Display a numerical matrix as colored cells; row/column label alignment remains the caller's responsibility.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 528–537:

```
528 for i in range(len(flavors)):
529     for j in range(len(flavors)):
530         value = transitions.iloc[i, j]
531         ax.text(
532             j,
533             i,
534             f"{value:.2f}" if pd.notna(value) else "N/A",
535             ha="center",
536             va="center",
537         )
```

Read this block:

- Line 528: for each entry yielded by `range(len(flavors))`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 529: for each entry yielded by `range(len(flavors))`, bind/unpack it into `j` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 530, `value`: Set this name to the entry/slice selected by `transitions.iloc[i, j]`; brackets select from the object to their left.
- Line 531: call `ax.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 538–541:

```
538 fig.colorbar(image, ax=ax)
539 fig.tight_layout()
540 fig.savefig(output / "flavor_transitions.pdf")
```

```
541 plt.close(fig)
```

Read this block:

- Line 538: call `fig.colorbar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 539: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 540: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 541: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.colorbar`: Add a legend relating matrix colors to numerical values.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run

Original function: `run`. Source begins at line 545.

This orchestrates the summary workflow and writes final artifacts. `sources` and `source_partitions` are dictionaries keyed by sample prefix, keeping full panel and matched yogurt objects separate. It optionally supplements flavor metadata, then prepares each population, writes reusable hash partitions, computes household/demographic summaries, and audits annual coverage.

The second loop runs detailed then grouped flavor taxonomies. Partition-level weekly outputs remain sparse, while small audit tables are summed. The last reference therefore holds the grouped mapping, which is saved for estimation. The final export intentionally reopens the original purchase file, rather than the demographic projection, to preserve store/DMA and other fields. A semi join keeps eligible households and a left join attaches the resolved flavor.

`prepared.drop("flavor")` avoids two competing label columns when reprocessing an already labeled input. The resolved-path check prevents writing over the original source. This orchestration keeps the population restrictions explicit; running the estimator on an arbitrary intermediate table can produce another sample even if its filename looks similar.

Definition and stated intent:

Source lines 545–546:

```
545 def run(args, output, work):
546     # Keep purchase-level estimation samples out of this code checkout.
```

`def` creates this callable; the parentheses name its inputs (`args`, `output`, `work`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 547–547:

```
547 sample_destination = (Path(output) / "yogurt_estimation_sample.parquet").resolve()
```

Read this block:

- Line 547, `sample_destination`: Set this name to the result of calling `.resolve(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 548–549:

```
548 if sample_destination.is_relative_to(Path(__file__).resolve().parents[3]):
549     raise ValueError("Estimation samples must stay outside the code checkout; use the server data directory.")
```

Read this block:

- Line 548: test `sample_destination.is_relative_to(Path(__file__).resolve().parents[3])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 549: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 551–552:

```
551 products_extra = args.products_extra or args.data_dir / "RMS/combined/products_extra.parquet"
552 flavor_path = args.flavors
```

Read this block:

- Line 551, `products_extra`: Set this name to the Boolean condition `args.products_extra or args.data_dir / 'RMS/combined/products_...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 552, `flavor_path`: Set this name to the attribute `args.flavors` of the object before the dot.

Source lines 553–560:

```
553 if products_extra.exists():
554     recovered_path = output / "recovered_flavor_reference.csv"
555     if recovered_path.resolve() == flavor_path.resolve():
556         recovered_path = output / "supplemented_flavor_reference.csv"
557     recover_flavor_reference(flavor_path, products_extra, recovered_path)
558     flavor_path = recovered_path
559 else:
560     print(f"Flavor recovery source not found: {products_extra}; using the CSV reference.", flush=True)
```

Read this block:

- Line 553: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 554, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 555: test `recovered_path.resolve() == flavor_path.resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 556, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 557: call `recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 558, `flavor_path`: Set this name to the object currently named `recovered_path`; plain assignment does not copy it.
- Line 560: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `recover_flavor_reference`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 561–564:

```
561 sources = {}
562 prepared_eligible = None
563 print("Summary sample: single-person purchase records (household_size == 1).", flush=True)
564 source_partitions = {}
```

Read this block:

- Line 561, `sources`: Set this name to a dictionary from the displayed keys and values.
- Line 562, `prepared_eligible`: Set this name to the literal value `None`.
- Line 563: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 564, `source_partitions`: Set this name to a dictionary from the displayed keys and values.

Source lines 565–602:

```
565 for prefix, filename in [("full_panel", "merged_panel.parquet"), ("matched_yogurt",
    ↪ "yogurt_estimation_sample.parquet")]:
566     source = scan_purchases(args.data_dir / filename, args.first_year, args.last_year)
567     if "household_size" not in pl.scan_parquet(args.data_dir / filename).collect_schema().names():
568         raise ValueError(f"{filename}: single-person summaries require household_size.")
569     if prefix == "matched_yogurt":
570         # Preserve all-household retail coverage in the prepared estimation
571         # input. est.py restricts demand records while retaining these quotes.
572         covered_all = source.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
573         prepared_eligible = (repeated_yogurt_households(covered_all)
574                              .select("household_code").unique().collect(engine="streaming"))
575     source = single_person_purchases(source)
576     if prefix == "matched_yogurt":
577         covered = (pl.col("price").is_finite() & (pl.col("price") > 0)).fill_null(False)
578         audit = source.select(pl.len().alias("purchase_lines_before_price_filter"),
579                               (~covered).sum().alias("purchase_lines_without_price_coverage"))
580         save_table(audit, output, "yogurt_price_coverage", "Yogurt matched-price coverage")
581         source = source.filter(covered)
582         source = repeated_yogurt_households(source)
583     parts = partition_purchases(source, work, prefix, args.household_batches)
584     source_partitions[prefix] = parts
585     source = pl.concat(parts)
586     sources[prefix] = source
587     households = household_batches(source, work, prefix, args.household_batches, partitions=parts)
588     if households.select(pl.len()).collect(engine="streaming").item() == 0:
589         raise ValueError(f"Empty {prefix} sample in selected years.")
590     demographic_tables(households, output, prefix)
591     households.sink_csv(output / f"{prefix}_households.csv", engine="streaming")
592     # Exact distinct-trip counts for one year at a time bound aggregation state.
593     years = source.select("panel_year").unique().sort("panel_year").collect(engine="streaming")["panel_year"]
594     coverage_rows = []
595     for year in years:
596         coverage_rows.append(source.filter(pl.col("panel_year") == year).select(
597             pl.lit(year).alias("panel_year"),
598             pl.col("household_code").n_unique().alias("households"),
599             pl.col("trip_code_uc").n_unique().alias("trips"), pl.len().alias("purchase_lines"),
600             pl.col("quantity").sum().alias("packages")).collect(engine="streaming"))
601     coverage = pl.concat(coverage_rows)
602     save_table(coverage, output, f"{prefix}_year_coverage", "Sample coverage by year")
```

Read this block:

- Line 565: for each entry yielded by `[('full_panel', 'merged_panel.parquet'), ('matched_yogurt', 'y...`,

bind/unpack it into `(prefix, filename)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 566, `source`: Set this name to the result of calling `scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 567: test `'household_size' not in pl.scan_parquet(args.data_dir / filename).collect_sch...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 568: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 569: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 572, `covered_all`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 573, `prepared_eligible`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 575, `source`: Set this name to the result of calling `single_person_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 576: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 577, `covered`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.
- Line 578, `audit`: Set this name to the result of calling `source.select(...)`; the arguments and preceding method chain are shown above.
- Line 580: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 581, `source`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 582, `source`: Set this name to the result of calling `repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 583, `parts`: Set this name to the result of calling `partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 584, `source_partitions[prefix]`: Store the object currently named `parts`; plain assignment does not copy it in the selected entry/column.
- Line 585, `source`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 586, `sources[prefix]`: Store the object currently named `source`; plain assignment does not copy it in the selected entry/column.
- Line 587, `households`: Set this name to the result of calling `household_batches(...)`; the arguments and preceding method chain are shown above.
- Line 588: test `households.select(pl.len()).collect(engine='streaming').item() == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 589: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 590: call `demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 591: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 593, `years`: Set this name to the entry/slice selected by `source.select('panel_year').unique().sort('panel_year').c...`; brackets select from the object to their left.
- Line 594, `coverage_rows`: Set this name to the list `[]`.
- Line 595: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 596: call `coverage_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 601, `coverage`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 602: call `save_table` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.

Syntax and calls used here:

- `scan_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `single_person_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_batches`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demographic_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `repeated_yogurt_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 603–627:

```
603 for detailed in [True, False]:
604     folder = output if detailed else output / "aggregated_flavors"
605     folder.mkdir(parents=True, exist_ok=True)
606     reference = flavor_reference(flavor_path, detailed=detailed, output=output)
607     flavors = sorted(reference["flavor"].unique().to_list()) if detailed else FLAVORS
608     if detailed:
609         reference.write_csv(output / "detailed_flavor_reference.csv")
610     # Process household batches independently, keeping weekly baskets long.
611     audit_total = None
612     tag = "detailed" if detailed else "grouped"
613     parts = source_partitions["matched_yogurt"]
614     for batch, subset in enumerate(parts):
615         began = time.monotonic()
616         print(f"{tag}: yogurt week batch {batch + 1}/{len(parts)}", flush=True)
```

```
617         weekly, audit = yogurt_weeks(subset, reference, flavors, output=folder)
618         weekly.sink_parquet(work / f"{tag}_weeks_{batch}.parquet", engine="streaming")
619         print(f"{tag}: yogurt week batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
620         audit_total = audit if audit_total is None else pl.concat([audit_total, audit]).select(pl.all().sum())
621     if detailed:
622         save_table(audit_total, output, "yogurt_sample_audit", "Yogurt sample audit")
623     weekly = pl.scan_parquet(work / f"{tag}_weeks_*.parquet")
624     shares, transitions = flavor_summaries(weekly, folder, flavors)
625     if not args.no_figures:
626         make_hhi_figures(folder)
627         make_figures(shares, transitions, folder)
```

Read this block:

- Line 603: for each entry yielded by `[True, False]`, bind/unpack it into `detailed` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 604, `folder`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 605: call `folder.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 606, `reference`: Set this name to the result of calling `flavor_reference(...)`; the arguments and preceding method chain are shown above.
- Line 607, `flavors`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 608: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 609: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 611, `audit_total`: Set this name to the literal value `None`.
- Line 612, `tag`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 613, `parts`: Set this name to the entry/slice selected by `source_partitions['matched_yogurt']`; brackets select from the object to their left.
- Line 614: for each entry yielded by `enumerate(parts)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 615, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 616: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 617, `(weekly, audit)`: Unpack the result of calling `yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 618: call `weekly.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 619: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 620, `audit_total`: Set this name to the true-branch value if `audit_total is None` holds, otherwise the else-branch value.
- Line 621: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 622: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 623, `weekly`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 624, `(shares, transitions)`: Unpack the result of calling `flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 625: test `not args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 626: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 627: call `make_figures` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `flavor_summaries`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.write_csv`: Write an eager Polars table to CSV; it does not return a newly analyzed dataset.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `yogurt_weeks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 630–637:

```
630 reference.write_csv(output / "estimation_flavor_reference.csv")
631 print(f"Saved estimation flavor mapping to {output / 'estimation_flavor_reference.csv'}", flush=True)
632 # Retain every original purchase column needed downstream, including store
633 # and DMA fields that demographic projections do not otherwise need.
634 eligible = prepared_eligible
635 prepared = pl.scan_parquet(args.data_dir / "yogurt_estimation_sample.parquet")
636 original_names = prepared.collect_schema().names()
637 price_column = next((c for c in ["price", "price_rms"] if c in original_names), None)
```

Read this block:

- Line 630: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 634, `eligible`: Set this name to the object currently named `prepared_eligible`; plain assignment does not copy it.
- Line 635, `prepared`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 636, `original_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 637, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 638–639:

```
638 if price_column is None:
639     raise ValueError("Yogurt purchase table needs price or price_rms for matched-price coverage.")
```

Read this block:

- Line 638: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 639: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 640–641:

```
640 prepared = prepared.filter(pl.col(price_column).cast(pl.Float64).is_finite()
641                            & (pl.col(price_column).cast(pl.Float64) > 0))
```

Read this block:

- Line 640, `prepared`: Set this name to the result of calling `prepared.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 642–643:

```
642 if "flavor" in original_names:
643     prepared = prepared.drop("flavor")
```

Read this block:

- Line 642: test `'flavor' in original_names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 643, `prepared`: Set this name to the result of calling `prepared.drop(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Source lines 644–649:

```
644 prepared = (prepared.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
645             .with_columns(normalized_upc_expr(),
646                           pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
647             .join(eligible.lazy(), on="household_code", how="semi")
648             .join(reference.lazy(), on="upc", how="left"))
649 sample_path = output / "yogurt_estimation_sample.parquet"
```

Read this block:

- Line 644, `prepared`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 649, `sample_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 650–651:

```
650 if sample_path.resolve() == (args.data_dir / "yogurt_estimation_sample.parquet").resolve():
651     raise ValueError("Choose an output folder separate from the original data directory.")
```

Read this block:

- Line 650: test `sample_path.resolve() == (args.data_dir / 'yogurt_estimation_sample.parquet')...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 651: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 652–654:

```
652 prepared.sink_parquet(sample_path, engine="streaming")
653 print(f"Saved full prepared yogurt purchase table to {sample_path}", flush=True)
654 print(f"Saved summaries to {output}", flush=True)
```

Read this block:

- Line 652: call `prepared.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 653: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 654: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 658.

This parses summary options and chooses ordinary pipeline work or HHI-only work. `args.output or default_path` uses a fallback when the optional output is None. The HHI-only branch checks its saved input, makes graphs, optionally repeats for grouped summaries, then returns before raw-data setup. It is a true early exit, not merely a plotting flag.

Ordinary runs create the output directory and enter a TemporaryDirectory context there, passing its Path to `run`. Context management ensures intermediate partitions are cleaned on ordinary completion or exceptions. Keeping temporary work under output makes disk requirements predictable. `--no-figures` and `--hhi-only` are rejected together because their requested actions conflict.

Definition and stated intent:

Source lines 658–658:

```
658 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 659–664:

```
659 parser = argparse.ArgumentParser(description=__doc__)
660 parser.add_argument("--data-dir", type=Path, default=DATA_DIR)
661 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
662 parser.add_argument("--products-extra", type=Path)
663 parser.add_argument("--output", type=Path)
664 parser.add_argument("--first-year", type=int, default=2006)
```

Read this block:

- Line 659, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 660: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 661: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 662: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 663: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 664: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 665–671:

```
665 parser.add_argument("--last-year", type=int, default=2015)
666 parser.add_argument("--household-batches", type=int, default=16,
667                     help="Household disk partitions; more batches reduce aggregation memory (default 16).")
668 parser.add_argument("--no-figures", action="store_true")
669 parser.add_argument("--hhi-only", action="store_true",
670                     help="Plot HHI from existing yogurt_household_flavors.csv in --output; skip raw data and all
                         ↪  summaries.")
671 args = parser.parse_args()
```

Read this block:

- Line 665: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 666: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 668: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 669: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 671, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 672–673:

```
672 if args.hhi_only and args.no_figures:
673     parser.error("--hhi-only cannot be combined with --no-figures")
```

Read this block:

- Line 672: test `args.hhi_only and args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 673: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 674–675:

```
674 if args.household_batches < 1:
675     parser.error("--household-batches must be positive")
```

Read this block:

- Line 674: test `args.household_batches < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 675: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 676–676:

```
676 output = args.output or args.data_dir / "summary_outputs"
```

Read this block:

- Line 676, `output`: Set this name to the Boolean condition `args.output or args.data_dir / 'summary_outputs'`; it can be a scalar or a row/array mask depending on its inputs.

Source lines 677–685:

```
677 if args.hhi_only:
678     source = output / "yogurt_household_flavors.csv"
679     if not source.exists():
680         parser.error(f"HHI-only needs the saved detailed household table: {source}")
681     make_hhi_figures(output)
682     grouped = output / "aggregated_flavors"
683     if (grouped / "yogurt_household_flavors.csv").exists():
684         make_hhi_figures(grouped)
685     return
```

Read this block:

- Line 677: test `args.hhi_only`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 678, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 679: test `not source.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 680: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 681: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 682, `grouped`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 683: test `(grouped / 'yogurt_household_flavors.csv').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 684: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 685: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 686–686:

```
686 output.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 686: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 687–688:

```
687 with tempfile.TemporaryDirectory(prefix="summary-work-", dir=output) as tmp:
688     run(args, output, Path(tmp))
```

Read this block:

- Line 687: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 688: call `run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `run`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

est_code: source walkthrough

est_code/bootstrap_se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 """Household bootstrap for the two-stage demand estimator."""
2
3 # Empirical I/O is permitted only on the research server, never on this computer.
4 from pathlib import Path as _PolicyPath
5 import sys as _policy_sys
6 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
7 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
8 from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 4: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 6, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 9–12:

```
9 import numpy as np
10 import pandas as pd
11
```

```
12 from price_control import add_price_controls
```

Read this block:

- Line 9: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 10: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 12: from `price_control`, import `add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

resample households

Original function: `resample_households`. Source begins at line 15.

Input is the weekly frame and a NumPy random Generator. Output is a concatenated bootstrap sample with the same number of household draws as the original. The comprehension stores whole household subtables. `rng.integers(0, H, size=H)` draws indices with replacement; its upper endpoint H is excluded.

`enumerate(selected)` supplies a fresh ID and selected old-group position. `.copy()` is essential before changing that ID so the original history is not modified. If selection is `[2,2,0]`, two copies of old household 2 get new IDs 0 and 1, keeping their likelihoods separate. Concatenating with `ignore_index=True` gives clean row labels. Resampling individual weeks would break persistent household preferences and within-household dependence.

Definition and stated intent:

Source lines 15–20:

```
15 def resample_households(frame, rng):
16     """Keep whole histories; repeated households get distinct bootstrap IDs.
17
18     Stocks/Xi stay attached to the original histories. Rebuilding stocks after
19     dropping incomplete price weeks would incorrectly erase observed purchases.
20     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `rng`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 21–23:

```
21 groups = [group for _, group in frame.groupby("household_code", sort=True)]
22 selected = rng.integers(0, len(groups), size=len(groups))
23 samples = []
```

Read this block:

- Line 21, `groups`: Set this name to a list computed by the displayed for-expression.
- Line 22, `selected`: Set this name to the result of calling `rng.integers(...)`; the arguments and preceding method chain are shown above.
- Line 23, `samples`: Set this name to the list `[]`.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.integers`: Draw integer positions with replacement by default; the high endpoint is excluded.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 24–27:

```
24 for new_id, old_id in enumerate(selected):
25     group = groups[old_id].copy()
26     group["household_code"] = new_id
```

```
27     samples.append(group)
```

Read this block:

- Line 24: for each entry yielded by `enumerate(selected)`, bind/unpack it into `(new_id, old_id)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 25, `group`: Set this name to the result of calling `groups[old_id].copy(...)`; the arguments and preceding method chain are shown above.
- Line 26, `group['household_code']`: Store the object currently named `new_id`; plain assignment does not copy it in the selected entry/column.
- Line 27: call `samples.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 28–28:

```
28 return pd.concat(samples, ignore_index=True)
```

Read this block:

- Line 28: return the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above. This ends the call.

bootstrap standard errors

Original function: `bootstrap_standard_errors`. Source begins at line 32.

The `estimate` argument is a callable, not an estimate array. Each repetition resamples households, refits enabled price controls, calls `estimate(sample, initial)`, and records its result. `record.update` adds named fields to an existing dictionary. Accepted vectors go to `values`; all attempts go to `records`, so failure does not disappear from the audit trail.

The `try/except` catches expected data/linear-algebra problems. Every attempt checkpoints the records as CSV. After the loop, the threshold requires at least 20 usable fits and at least 80% success. Returning `(None, report)` explicitly signals inference was withheld. Otherwise, `std(ddof=1)` uses sample rather than population SD, `quantile` forms empirical percentile intervals, and `np.cov(..., rowvar=False)` treats columns as parameters. Analytic Hessian SEs would be cheaper but would require handling the estimated first-stage residuals and household dependence; this implementation uses a two-stage cluster bootstrap.

Definition and stated intent:

Source lines 32–38:

```
32 def bootstrap_standard_errors(frame, args, estimate, initial, names):
33     """Refit price controls AND demand in every household-cluster replication.
34 
35     estimate(frame, initial) returns a scipy optimizer result. All replications
36     use the same integration draws as the main fit. Retail instruments stay
37     fixed: this is inference conditional on the observed retail price universe.
38     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `args`, `estimate`, `initial`, `names`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 39–41:

```
39 rng = np.random.default_rng(args.seed + 10000)
40 records = []
41 values = []
```

Read this block:

- Line 39, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 40, `records`: Set this name to the list `[]`.
- Line 41, `values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 42–61:

```
42 for replication in range(args.bootstrap_reps):
43     record = {"replication": replication + 1, "converged": False}
44     try:
45         sample = resample_households(frame, rng)
46         if not args.no_price_control:
47             sample, _ = add_price_controls(sample)
48         fit = estimate(sample, initial)
49         record.update(converged=bool(fit.success), objective=float(fit.fun),
50                       message=str(fit.message))
51         record.update(dict(zip(names, fit.x)))
52         if fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun):
53             values.append(fit.x)
54     except (ValueError, np.linalg.LinAlgError) as error:
55         # An unidentifiable resample is recorded, never silently accepted.
56         record["message"] = str(error)
57     records.append(record)
58     # Checkpoint after each replication in case a long cluster job ends.
59     pd.DataFrame(records).to_csv(args.output / "bootstrap_draws.csv", index=False)
60     print(f"Bootstrap {replication+1}/{args.bootstrap_reps}: "
61           f"{len(values)} usable fits", flush=True)
```

Read this block:

- Line 42: for each entry yielded by `range(args.bootstrap_reps)`, bind/unpack it into `replication` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `record`: Set this name to a dictionary with named entries `replication`, `converged`.
- Line 44: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 45, `sample`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 46: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 47, `(sample, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48, `fit`: Set this name to the result of calling `estimate(...)`; the arguments and preceding method chain are shown above.
- Line 49: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: test `fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 53: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: catch `(ValueError, np.linalg.LinAlgError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 56, `record['message']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 57: call `records.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: call `pd.DataFrame(records).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 60: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `estimate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `str`: Convert to text for paths, messages, or serialization.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 62–66:

```
62 valid = len(values)
63 report = dict(method=("household bootstrap" if args.no_price_control
64                       else "household bootstrap, refitting both stages"),
65              requested=args.bootstrap_reps, usable=valid,
66              conditional_on="retail instruments and fixed integration draws")
```

Read this block:

- Line 62, `valid`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 63, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 68–70:

```
68 if valid < max(20, int(np.ceil(.8 * args.bootstrap_reps))):
69     report["status"] = "insufficient converged replications; no SEs reported"
70     return None, report
```

Read this block:

- Line 68: test `valid < max(20, int(np.ceil(0.8 * args.bootstrap_reps)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69, `report['status']`: Store the literal text `'insufficient converged replications; no SEs reported'` in the selected entry/column.
- Line 70: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.ceil`: Round upward, so the minimum usable-bootstrap threshold meets the requested success fraction.

Source lines 71–78:

```
71 estimates = np.asarray(values)
72 table = pd.DataFrame({"parameter": names,
73                       "std_error": estimates.std(axis=0, ddof=1),
74                       "ci_lower": np.quantile(estimates, .025, axis=0),
75                       "ci_upper": np.quantile(estimates, .975, axis=0)})
76 covariance = pd.DataFrame(np.cov(estimates, rowvar=False), index=names, columns=names)
77 covariance.to_csv(args.output / "covariance.csv")
78 report["status"] = "computed; percentile 95% intervals"
```

Read this block:

- Line 71, `estimates`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 72, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 76, `covariance`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 77: call `covariance.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78, `report['status']`: Store the literal text `'computed; percentile 95% intervals'` in the selected entry/column.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `.cov`: Compute covariance; rowvar=False means columns represent parameters rather than treating rows as variables.

Source lines 79–79:

```
79 return table, report
```

Read this block:

- Line 79: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est_code/est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–12:

```
"""Weekly yogurt demand: fixed or mixed logit, with or without gamma.

Read README.md first. Run `python3 est.py --help` for server paths and options.
The raw data are not required for importing or testing the likelihood.
"""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 8: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 10, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 11: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 12: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–20:

```
from pathlib import Path
import argparse
import json
import time

import numpy as np
import pandas as pd
```

Read this block:

- Line 14: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `argparse`. An as-name is a convenient local alias, not another library.

- Line 16: import `json`. An as-name is a convenient local alias, not another library.
- Line 17: import `time`. An as-name is a convenient local alias, not another library.
- Line 19: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 20: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 21–27:

```
21 import polars as pl
22 from scipy.optimize import minimize
23 from scipy.special import logsumexp
24
25 from price_control import check_dma, add_price_controls
26 from bootstrap_se import bootstrap_standard_errors
27 from hessian_se import hessian_standard_errors
```

Read this block:

- Line 21: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 22: from `scipy.optimize`, import `minimize` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 23: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 25: from `price_control`, import `check_dma, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 26: from `bootstrap_se`, import `bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 27: from `hessian_se`, import `hessian_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 28–33:

```
28 from wtp import implied_wtp
29
30 FLAVORS = ["other", "berry", "plain"]
31 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
32 # Set this to the flavor-reference CSV on the Slurm server.
33 FLAVOR_FILE = DATA_DIR / "summary_outputs/estimation_flavor_reference.csv"
```

Read this block:

- Line 28: from `wtp`, import `implied_wtp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 30, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 31, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 33, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 835–836:

```
835 if __name__ == "__main__":
836     main()
```

Read this block:

- Line 835: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 836: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 36.

Job and contract. Accept a pandas Series of UPC-like values and return a Series of canonical string identifiers. `values.astype("string")` selects pandas' nullable string dtype; `.str` applies a string operation to every element. `.strip()` removes surrounding whitespace. `numeric` is a Boolean Series, not a single true/false answer.

`fullmatch` requires the entire string to fit the numeric pattern. Replacement removes a trailing decimal-zero suffix, then `lstrip("0")` removes leading zeros. The last `where` keeps the original text where `~numeric` is true and uses the canonical version elsewhere. Thus `"00123.0"` becomes `"123"`, `"000"` becomes `"0"`, and `"A001"` stays `"A001"`. The inner `.replace("", "0")` fixes the empty string produced when all digits were zero.

Why this choice? Converting to `float` then `int` looks shorter but can round long identifiers and fails on nonnumeric UPCs. A Python loop would work but duplicates the same string operations per row and loses convenient missing-value handling. `where` is easy to misread: its true branch retains the receiver.

Definition and stated intent:

Source lines 36–37:

```
36 def normalize_upcs(values):
37     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 38–40:

```
38 text = values.astype("string").str.strip()
39 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
40 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 38, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 39, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 40, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 41–41:

```
41 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 41: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 44.

This returns a **Polars expression**, not a loaded Series. Calling it constructs a reusable recipe for the `upc` column. `pl.col` identifies that column; `cast(pl.String)` standardizes its type. The local names `text`, `numeric`, and `canonical` hold expression objects. Their calculations run later when the surrounding lazy query is executed.

Nested `when/then/otherwise` first decides whether a UPC is numeric, then whether stripping zeros would leave an empty identifier. `pl.lit("0")` means literal text, not a column named zero. `.alias("upc")` ensures `with_columns` replaces the intended column. This version avoids collecting an entire lazy table just to normalize one field. A pandas conversion would forfeit the lazy query's memory and projection advantages.

Definition and stated intent:

Source lines 44–44:

```
44 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–47:

```
45 text = pl.col("upc").cast(pl.String).str.strip_chars()
46 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
47 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 45, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 46, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 47, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 48–53:

```
48 return (
49     pl.when(numeric)
50     .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
```

```
51     .otherwise(text)
52     .alias("upc")
53 )
```

Read this block:

- Line 48: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

read flavors

Original function: `read_flavors`. Source begins at line 57.

Input `path` is a CSV path; optional `products_extra` is a Parquet path. Output is a two-column pandas table with one row per UPC. This function has an early return: if the CSV already contains resolved `upc,flavor`, validation succeeds and the rest of the recovery/classification code is never executed.

`{"upc", "flavor"}.issubset(frame.columns)` asks whether both required labels exist. `missing = required - set(frame.columns)` is set subtraction. Validation uses `.any()` for "at least one invalid row" and `.all()` for "every label is allowed." `if products_extra is not None` distinguishes an omitted optional argument from a supplied path that happens not to exist.

The metadata path scans only useful columns and anti-joins existing reference UPCs. It appends absent UPCs, rather than overwriting existing classifications. `[... for c in columns if c != "upc"]` builds one aggregation expression per non-key field. `np.select` picks the first matching condition, so plain wins when both plain and berry text occur. A generic substring loop could be easier to write at first, but would make precedence and vectorized missing-value handling less explicit. `drop_duplicates(..., keep="first")` is a reconciliation rule; it does not prove the retained label is correct.

Definition and stated intent:

Source lines 57–58:

```
57 def read_flavors(path, products_extra=None):
58     """Use the same plain-first classification as data_code/summ_stats.py."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `products_extra`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–59:

```
59 frame = pd.read_csv(path, dtype={"upc": str})
```

Read this block:

- Line 59, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 60–74:

```
60 if {"upc", "flavor"}.issubset(frame.columns):
61     # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
62     frame["upc"] = normalize_upcs(frame["upc"])
63     if (
64         frame["upc"].isna().any()
65         or frame["upc"].eq("").any()
66         or frame["upc"].duplicated().any()
67     ):
68         raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
69     if not frame["flavor"].isin(FLAVORS).all():
70         raise ValueError(f"{path}: saved mapping must use other, berry, plain.")
71     print(
72         f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
73     )
74     return frame[["upc", "flavor"]]
```

Read this block:

- Line 60: test `{'upc', 'flavor'}.issubset(frame.columns)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 63: test `frame['upc'].isna().any() or frame['upc'].eq('').any() or frame['upc'].duplic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 68: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 69: test `not frame['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 70: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 71: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: return the entry/slice selected by `frame[['upc', 'flavor']]`; brackets select from the object to their left. This ends the call.

Syntax and calls used here:

- `normalize_upcs`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 75–75:

```
75 missing = {"upc", "flavor_descr"} - set(frame.columns)
```

Read this block:

- Line 75, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subex-

pressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 76–77:

```
76 if missing:
77     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 76: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 78–78:

```
78 frame["upc"] = normalize_upcs(frame["upc"])
```

Read this block:

- Line 78, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 79–80:

```
79 if frame["upc"].isna().any() or frame["upc"].eq("").any():
80     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 79: test `frame['upc'].isna().any() or frame['upc'].eq('').any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 80: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 81–122:

```
81 if products_extra is not None:
82     products_extra = Path(products_extra)
83     if products_extra.exists():
84         products = pl.scan_parquet(products_extra)
85         names = products.collect_schema().names()
86         if not {"upc", "flavor_descr"}.issubset(names):
87             raise ValueError(
88                 f"{products_extra}: needs upc and flavor_descr columns."
89             )
90         columns = [
91             c
92             for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
93             if c in names
94         ]
95         products = (
96             products.select(columns)
97             .with_columns(normalized_upc_expr())
98             .join(
99                 pl.DataFrame(
100                     {"upc": frame["upc"].dropna().unique().tolist()}
101                 ).lazy(),
```

```
                on="upc",
                how="anti",
            )
        )
        products = products.filter(
            pl.col("upc").is_not_null()
            & pl.col("flavor_descr").is_not_null()
            & ~pl.col("flavor_descr")
            .cast(pl.String)
            .str.strip_chars()
            .str.to_lowercase()
            .is_in(["", "na", "n/a", "nan", "null", "none"])
        )
        if "panel_year" in names:
            products = products.group_by("upc").agg(
                [
                    pl.col(c)
                    .sort_by("panel_year", nulls_last=True)
                    .first()
                    .alias(c)
                    for c in columns
```

Source lines 123–140:

```
                if c != "upc"
            ]
        )
    else:
        products = products.unique(
            subset="upc", keep="first", maintain_order=True
        )
    recovered = products.collect(engine="streaming").to_pandas()
    frame = pd.concat([frame, recovered], ignore_index=True)
    print(
        f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
        flush=True,
    )
else:
    print(
        f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
        flush=True,
    )
```

Read this block:

- Line 81: test `products_extra is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 82, `products_extra`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 83: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 84, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 85, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 86: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 87: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 90, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries

passing its if filter.
- Line 95, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 106, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.
- Line 115: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 116, `products`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 127, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.
- Line 130, `recovered`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 131, `frame`: Set this name to the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above.
- Line 132: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 137: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 141–141:

```
141 text = frame["flavor_descr"].fillna("")
```

Read this block:

- Line 141, `text`: Set this name to the result of calling `frame['flavor_descr'].fillna(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 142–143:

```
142 if "product_descr" in frame.columns:
143     text = text + " " + frame["product_descr"].fillna("")
```

Read this block:

- Line 142: test `'product_descr' in frame.columns`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 143, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 144–154:

```
144 text = text.str.lower()
145 frame["flavor"] = np.select(
146     [
147         text.str.contains("plain|unflavored"),
148         text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"),
149     ],
150     ["plain", "berry"],
151     default="other",
152 )
153 # Match summ_stats: retain missing labels and use the first CSV row per UPC.
154 conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
```

Read this block:

- Line 144, `text`: Set this name to the result of calling `text.str.lower(...)`; the arguments and preceding method chain are shown above.
- Line 145, `frame['flavor']`: Store the result of calling `np.select(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 154, `conflicts`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 155–160:

```
155 if conflicts:
156     print(
157         f"{conflicts:,} UPCs have conflicting flavor classifications; "
158         "keeping the first flavor in CSV row order.",
159         flush=True,
160     )
```

Read this block:

- Line 155: test `conflicts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 161–161:

```
161 return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

Read this block:

- Line 161: return the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

week dates

Original function: `week_dates`. Source begins at line 164.

`pd.to_datetime` interprets strings with an explicit `%Y%m%d` format: four-digit year, two-digit month, two-digit day. The regex removes a trailing `.0` from integer dates that were stored as float-like text. `errors="raise"` rejects a malformed date instead of silently creating a missing date.

For example, `20100109` should mean January 9, 2010. Passing that integer directly to a general timestamp constructor can instead mean nanoseconds from an epoch. Explicit parsing prevents that mistake. This helper exists in the module, but the current main preparation path parses purchase-date strings with Polars directly; a function's presence does not mean the entry point uses it.

Definition and stated intent:

Source lines 164–165:

```
164 def week_dates(values):
165     """Parse YYYYMMDD safely (integer dates are NOT nanoseconds)."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 166–170:

```
166 return pd.to_datetime(
167     values.astype(str).str.replace(r"\.0$", "", regex=True),
168     format="%Y%m%d",
169     errors="raise",
170 )
```

Read this block:

- Line 166: return the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

prepare data

Original function: `prepare_data`. Source begins at line 174.

This is the data assembly function. `args` is an argparse Namespace: a small object whose attributes hold command-line settings. It returns a pandas table with one row per usable household-week. Its work crosses a deliberate boundary: large input filtering and quote aggregation use lazy Polars, while the smaller household choice table becomes pandas for subsequent array calculations.

Read it in phases. First it scans and filters years, resolves labels, chooses the price field, and projects columns. `getattr(args, name, default)` provides compatibility when a manually constructed Namespace lacks an optional attribute. `next((c for c in candidates if c in names), None)` chooses the first supported price field. It is a small loop written as a generator, not a special price API.

Next it collects selected households, checks quantities, sums counts, and chooses an exposure store. Sorting by descending quantity followed by ascending store ID then keeping the first household-week row implements the tie-break exactly. `groupby(...).sum().unstack(...)` turns long flavor counts into three columns; `reindex` fixes column order even if one flavor is absent from an intermediate table. Merge `validate` arguments catch unintended duplicate keys.

The quote section continues to use full `base`, not the household-limited `selected`. It deduplicates quotes and checks conflicts before constructing menus. The DMA section computes sums and counts and subtracts own-DMA totals. This algebra avoids looping over every possible outside DMA for every household. The same pooled IV is copied into all three flavor-named IV columns.

Finally it constructs histories **before** menu/IV likelihood exclusions, builds Boolean availability masks, filters unusable rows, reapplies the two-week rule, and adds outside counts. `complete &= iv_complete` combines masks in place; it is not a comparison. `transform("nunique")` produces a row-aligned count so the household restriction can be applied without a second merge. Moving `add_stocks` below these exclusions is a plausible-looking alternative that changes the information used to build history.

Definition and stated intent:

Source lines 174–179:

```
174 def prepare_data(args):
175     """Use only the merged purchase table, including its existing prices.
176 
177     The IV pools all flavors: the same-week average observed price outside the
178     consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
179     """
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 180–181:

```
180 base = pl.scan_parquet(args.sample)
181 names = base.collect_schema().names()
```

Read this block:

- Line 180, `base`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

- Line 181, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 182–183:

```
182 if "household_size" not in names:
183     raise ValueError("Estimation requires household_size to restrict purchases to single-person households.")
```

Read this block:

- Line 182: test `'household_size' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 183: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 184–184:

```
184 base = base.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
```

Read this block:

- Line 184, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 185–190:

```
185 if "flavor" not in names:
186     products_extra = getattr(args, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet"
187     flavor = read_flavors(args.flavors, products_extra)
188     base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
189 else:
190     base = base.with_columns(normalized_upc_expr())
```

Read this block:

- Line 185: test `'flavor' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 186, `products_extra`: Set this name to the Boolean condition `getattr(args, 'products_extra', None) or DATA_DIR / 'RMS/combi...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 187, `flavor`: Set this name to the result of calling `read_flavors(...)`; the arguments and preceding method chain are shown above.
- Line 188, `base`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 190, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `read_flavors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 191–191:

```
191 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
```

Read this block:

- Line 191, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 192–193:

```
192 if price_column is None:
193     raise ValueError("Merged purchase table needs a price or price_rms column.")
```

Read this block:

- Line 192: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 193: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 194–196:

```
194 consumer_dma = getattr(args, "household_dma_column", "dma_cd")
195 columns = ["household_code", "household_size", "panel_year", "purchase_date", "trip_code_uc", "upc",
196            "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
```

Read this block:

- Line 194, `consumer_dma`: Set this name to the result of calling `getattr(...)`; the arguments and preceding method chain are shown above.
- Line 195, `columns`: Set this name to a list containing 10 displayed entries.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 197–198:

```
197 if not args.no_price_control:
198     columns.append(pl.col(consumer_dma).alias("dma_cd"))
```

Read this block:

- Line 197: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 198: call `columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 199–206:

```
199 base = base.select(columns).with_columns(
200     pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
201 base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
202 base = base.with_columns((pl.col("purchase_date") + pl.duration(
203     days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
204 # Keep the full merged table for prices/IVs, even in a household-limited run.
205 # Restrict demand histories before weekly aggregation; retain all quotes for menus/IVs.
206 selected = base.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 199, `base`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 201, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.
- Line 202, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 206, `selected`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 207–210:

```
207 if args.max_households:
208     ids = (selected.select("household_code").unique().sort("household_code")
209            .limit(args.max_households).collect(engine="streaming")["household_code"])
210     selected = selected.filter(pl.col("household_code").is_in(ids.implode()))
```

Read this block:

- Line 207: test `args.max_households`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 208, `ids`: Set this name to the entry/slice selected by `selected.select('household_code').unique().sort('househol...`; brackets select from the object to their left.
- Line 210, `selected`: Set this name to the result of calling `selected.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 211–212:

```
211 print(f"Loading merged purchase table: {args.sample}", flush=True)
212 lines = selected.collect(engine="streaming").to_pandas()
```

Read this block:

- Line 211: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 212, `lines`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.

Source lines 213–214:

```
213 if lines.empty:
214     raise ValueError("No purchase lines in selected sample.")
```

Read this block:

- Line 213: test `lines.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 214: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 215–216:

```
215 if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
216     raise ValueError("Merged purchase table has missing or invalid flavor labels.")
```

Read this block:

- Line 215: test `lines['flavor'].isna().any() or not lines['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 216: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 217–217:

```
217 q = lines["quantity"].to_numpy(float)
```

Read this block:

- Line 217, `q`: Set this name to the result of calling `lines['quantity'].to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 218–219:

```
218 if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
219     raise ValueError("Quantity must be positive integer package counts.")
```

Read this block:

- Line 218: test `not np.isfinite(q).all() or (q <= 0).any() or (not np.allclose(q, np.round(q)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 219: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 220–226:

```
220 lines["week"] = pd.to_datetime(lines["week"])
221 keys = ["household_code", "week"]
222 weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
223           .reindex(columns=FLAVORS, fill_value=0).reset_index())
224 stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
225 stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
226 stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
```

Read this block:

- Line 220, `lines['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 221, `keys`: Set this name to the list `['household_code', 'week']`.
- Line 222, `weekly`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 224, `stores`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 225, `stores`: Set this name to the result of calling `stores.sort_values(...)`; the arguments and preceding method chain are shown above.
- Line 226, `stores`: Set this name to the entry/slice selected by `stores.drop_duplicates(keys)[keys + ['store_code_uc']]`; brackets select from the object to their left.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

Source lines 227–227:

```
227 weekly = weekly.merge(stores, on=keys, validate="one_to_one")
```

Read this block:

- Line 227, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 228–232:

```
228 if not args.no_price_control:
229     lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
230     if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
231         raise ValueError("Multiple consumer DMAs within a household-week.")
232     weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")
```

Read this block:

- Line 228: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 229, `lines['dma_cd']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 230: test `(lines.groupby(keys)['dma_cd'].nunique() > 1).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 231: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 232, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 234–235:

```
234 quote_keys = ["store_code_uc", "week", "upc"]
235 price_columns = quote_keys + ["flavor", "price"]
```

Read this block:

- Line 234, `quote_keys`: Set this name to the list `['store_code_uc', 'week', 'upc']`.
- Line 235, `price_columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 236–237:

```
236 if not args.no_price_control:
237     price_columns.append("dma_cd")
```

Read this block:

- Line 236: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 237: call `price_columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 238–244:

```
238 quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
239           .select(price_columns).unique())
240 # A store/week/UPC should have one price and one flavor. Consumer DMA may
241 # differ when consumers from different markets visit the same store.
242 local = quotes.select(quote_keys + ["flavor", "price"]).unique()
243 conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
244                .limit(1).collect(engine="streaming"))
```

Read this block:

- Line 238, `quotes`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 242, `local`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 243, `conflicting`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 245–246:

```
245 if len(conflicting):
246     raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
```

Read this block:

- Line 245: test `len(conflicting)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 246: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 247–253:

```
247 menus = (local.group_by(["store_code_uc", "week", "flavor"])
248          .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
249 menus["week"] = pd.to_datetime(menus["week"])
250 menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
251          .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
252 weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
253 print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
```

Read this block:

- Line 247, `menus`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 249, `menus['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 250, `menus`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 252, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 253: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 254–278:

```
254 if not args.no_price_control:
255     invalid_dma = quotes.select(
256         ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
257          | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
258          | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64,
              ↪  strict=False).floor()))
259         .fill_null(True).any()).collect(engine="streaming").item()
260     if invalid_dma:
261         raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
262     # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
263     by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
264               .group_by(["week", "dma_cd"]).agg(
265                   pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
266               .collect(engine="streaming").to_pandas())
267     by_dma["week"] = pd.to_datetime(by_dma["week"])
268     totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
269               .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
270     weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
271     weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
272     denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
273     weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
274                             / denominator.where(denominator > 0))
275     for f in FLAVORS:
276         weekly["iv_" + f] = weekly["iv_overall"]
277     weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
278     print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
```

Read this block:

- Line 254: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 255, `invalid_dma`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 260: test `invalid_dma`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 261: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 263, `by_dma`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 267, `by_dma['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 268, `totals`: Set this name to the result of calling `.rename(...)`; the arguments and preceding method chain are shown above.
- Line 270, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 271, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 272, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 273, `weekly['iv_overall']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 275: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 276, `weekly['iv_' + f]`: Store the entry/slice selected by `weekly['iv_overall']`; brackets select from the object to their left in the selected entry/column.
- Line 277, `weekly`: Set this name to the result of calling `weekly.drop(...)`; the arguments and preceding method chain are shown above.
- Line 278: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 279–283:

```
279 weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), args.habit_weight)
280 price_cols = ["price_" + f for f in FLAVORS]
281 available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
282 counts = weekly[FLAVORS].to_numpy(float)
283 complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Read this block:

- Line 279, `weekly`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 280, `price_cols`: Set this name to a list computed by the displayed for-expression.
- Line 281, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 282, `counts`: Set this name to the result of calling `weekly[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 283, `complete`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 284–285:

```
284 for j, f in enumerate(FLAVORS):
285     weekly["available_" + f] = available[:, j]
```

Read this block:

- Line 284: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 285, `weekly['available_' + f]`: Store the entry/slice selected by `available[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 286–286:

```
286 print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
```

Read this block:

- Line 286: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 287–291:

```
287 if not args.no_price_control:
288     iv_cols = ["iv_" + f for f in FLAVORS]
289     iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
290     print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True)
291     complete &= iv_complete
```

Read this block:

- Line 287: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 288, `iv_cols`: Set this name to a list computed by the displayed for-expression.
- Line 289, `iv_complete`: Set this name to the result of calling `.all(...)`; the arguments and preceding method chain are shown above.
- Line 290: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 291: update `complete` using `&=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 292–296:

```
292 weekly = weekly.loc[complete].copy()
293 repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
294 print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated,
    ↪  'household_code'].nunique():,}", flush=True)
295 weekly = weekly.loc[repeated].reset_index(drop=True)
296 weekly["outside"] = np.maximum(args.capacity - weekly[FLAVORS].sum(axis=1), 0)
```

Read this block:

- Line 292, `weekly`: Set this name to the result of calling `weekly.loc[complete].copy(...)`; the arguments and preceding method chain are shown above.
- Line 293, `repeated`: Set this name to the Boolean condition `weekly.groupby('household_code')['week'].transform('nunique') ...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 294: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 295, `weekly`: Set this name to the result of calling `weekly.loc[repeated].reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 296, `weekly['outside']`: Store the result of calling `np.maximum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.transform`: Compute a pandas group result broadcast back to original rows, unlike a reduced one-row-per-group aggregation.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.

Source lines 297–298:

```
297 if weekly.empty:
298     raise ValueError("No households with at least two usable purchase weeks.")
```

Read this block:

- Line 297: test `weekly.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 298: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 299–299:

```
299 return weekly
```

Read this block:

- Line 299: return the object currently named `weekly`; plain assignment does not copy it. This ends the call.

add stocks

Original function: `add_stocks`. Source begins at line 302.

Input is a weekly pandas table; output is a sorted copy with stock, novelty, and Xi columns. The outer loop resets history for each household. `for _, group` ignores the household label but keeps its subtable. The inner `iterrows()` loop supplies a row index and a Series containing that row's values.

`stocks[index] = stock` stores pre-choice history into one output row. The seven-day condition tests genuine adjacent calendar weeks. Then counts are normalized into shares, stock is updated, and the current date becomes the previous date. A gap resets recent novelty by leaving its preallocated zeros, but does not reset cumulative stock. A loop is reasonable here because state at the next observation depends on the preceding update; vectorization requires careful shifted cumulative sums and calendar-gap checks.

The final `enumerate` loop writes one named column per flavor. `stocks[:, j]` extracts all rows of the jth flavor. With `habit_weight=.7`, current Xi uses .3 times novelty minus .7 times own stock. The separately preallocated `stocks` matrix means historical output rows do not change when `stock` is reassigned. Try a one-household, two-week example before reading the likelihood.

Definition and stated intent:

Source lines 302–313:

```
302 def add_stocks(weekly, habit_weight=0.7):
303     """All past observed occasions, delta=1; no rolling window or decay.
304 
305     S_jt = sum_{s<t} share_js
306     N_jt = sum_{k != j} share_k,t-1  (immediately preceding calendar week)
307     Xi_jt = (1-habit_weight)*N_jt - habit_weight*S_jt
308 
309     Positive gamma rewards recent other-flavor consumption and penalizes own
310     fatigue. Negative gamma represents habit. A purchase occasion is an observed
311     household purchase week. Current purchases enter history only AFTER Xi is
312     recorded. The first observed occasion starts with zero history.
313     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 314–318:

```
314 result = (
315     weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
316 )
317 stocks = np.zeros((len(result), 3))
318 novelty = np.zeros_like(stocks)
```

Read this block:

- Line 314, `result`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 317, `stocks`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 318, `novelty`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 319–330:

```
319 for _, group in result.groupby("household_code", sort=False):
320     stock = np.zeros(3)
321     previous_share = np.zeros(3)
322     previous_week = None
323     for index, row in group.iterrows():
324         stocks[index] = stock
325         if previous_week is not None and (row["week"] - previous_week).days == 7:
326             novelty[index] = previous_share.sum() - previous_share
327         counts = row[FLAVORS].to_numpy(float)
328         previous_share = counts / counts.sum()
329         stock = stock + previous_share
330         previous_week = row["week"]
```

Read this block:

- Line 319: for each entry yielded by `result.groupby('household_code', sort=False)`, bind/unpack it into `(_, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 320, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 321, `previous_share`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 322, `previous_week`: Set this name to the literal value `None`.
- Line 323: for each entry yielded by `group.iterrows()`, bind/unpack it into `(index, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 324, `stocks[index]`: Store the object currently named `stock`; plain assignment does not copy it in the selected entry/column.
- Line 325: test `previous_week is not None and (row['week'] - previous_week).days == 7`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 326, `novelty[index]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 327, `counts`: Set this name to the result of calling `row[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 328, `previous_share`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 329, `stock`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 330, `previous_week`: Set this name to the entry/slice selected by `row['week']`; brackets select from the object to their left.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 331–336:

```
331 for j, name in enumerate(FLAVORS):
332     result["stock_" + name] = stocks[:, j]
333     result["novelty_" + name] = novelty[:, j]
334     result["xi_" + name] = (1 - habit_weight) * novelty[
335         :, j
336     ] - habit_weight * stocks[:, j]
```

Read this block:

- Line 331: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 332, `result['stock_' + name]`: Store the entry/slice selected by `stocks[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 333, `result['novelty_' + name]`: Store the entry/slice selected by `novelty[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 334, `result['xi_' + name]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 337–337:

```
337 return result
```

Read this block:

- Line 337: return the object currently named `result`; plain assignment does not copy it. This ends the call.

make arrays

Original function: `make_arrays`. Source begins at line 340.

This converts a labeled table into a dictionary of NumPy arrays. `pd.factorize` returns both dense integer household codes and their original labels: household IDs `[10,10,30]` become codes `[0,0,1]`. Dense codes let `bincount` sum efficiently. The expression `FLAVORS + (["outside"] if capacity else [])` builds either three or four outcome columns without changing the inside-price array's three columns.

The function validates counts/Xi, checks menu consistency, and inserts zero placeholders for missing prices. The separate availability mask ensures those zeros are not interpreted as offered free goods. `all(c in frame for c in cf_cols)` asks whether every residual column exists. If controls are absent, `zeros_like` supplies the right-shaped

residual array. The returned dictionary deliberately keeps flags such as `outside` and `control` beside arrays so the likelihood can use one interface for several models. Passing raw tables directly into every optimization step would repeatedly perform label lookup and conversion.

Definition and stated intent:

Source lines 340–341:

```
340 def make_arrays(frame, capacity):
341     """Use household codes to sum log likelihoods before integrating types."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 342–348:

```
342 hh, labels = pd.factorize(frame["household_code"], sort=True)
343 counts = frame[FLAVORS + (["outside"] if capacity else [])].to_numpy(float)
344 prices = frame[["price_" + f for f in FLAVORS]].to_numpy(float)
345 xi = frame[["xi_" + f for f in FLAVORS]].to_numpy(float)
346 history = {key: frame[[prefix + f for f in FLAVORS]].to_numpy(float)
347            for key, prefix in [("novelty", "novelty_"), ("stock", "stock_")]
348            if all(prefix + f in frame for f in FLAVORS)}
```

Read this block:

- Line 342, `(hh, labels)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 343, `counts`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 344, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 345, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 346, `history`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 349–350:

```
349 if any(not np.isfinite(value).all() for value in history.values()):
350     raise ValueError("Nonfinite separate history inputs.")
```

Read this block:

- Line 349: test `any((not np.isfinite(value).all() for value in history.values()))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 350: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 351–351:

```
351 available = np.isfinite(prices) & (prices > 0)
```

Read this block:

- Line 351, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 352–353:

```
352 if not all(np.isfinite(x).all() for x in [counts, xi]):
353     raise ValueError("Nonfinite estimation inputs.")
```

Read this block:

- Line 352: test `not all((np.isfinite(x).all() for x in [counts, xi]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 353: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 354–355:

```
354 if ((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all():
355     raise ValueError("Purchased flavors must have covered prices and a nonempty choice set.")
```

Read this block:

- Line 354: test `((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 355: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 357–360:

```
357 prices = np.where(available, prices, 0.)
358 cf_cols = ["cf_" + f for f in FLAVORS]
359 control = all(c in frame for c in cf_cols)
360 cf = frame[cf_cols].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 357, `prices`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 358, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 359, `control`: Set this name to the result of calling `all(...)`; the arguments and preceding method chain are shown above.
- Line 360, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 361–362:

```
361 if not np.isfinite(cf).all():
362     raise ValueError("Nonfinite price-control residuals.")
```

Read this block:

- Line 361: test `not np.isfinite(cf).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 362: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 363–375:

```
363 return dict(
364     counts=counts,
365     prices=prices,
366     available=available,
367     xi=xi,
368     **history,
369     cf=cf,
370     control=control,
371     hh=hh,
372     labels=labels,
373     n_households=len(labels),
374     outside=bool(capacity),
375 )
```

Read this block:

- Line 363: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

make draws

Original function: `make_draws`. Source begins at line 378.

Inputs are an even draw count, an integer seed, and an optional model name. Output has shape `(number,3)` for heterogeneous_history and `(number,2)` otherwise. `number % 2` is the remainder after division by two: zero for even numbers, one for odd. `number // 2` is integer division. The generator draws half the rows, then `vstack([z, -z])` appends their negatives. With two draws, an initial row `[.4,-1.2]` gives a second row `[-.4,1.2]`.

The seed makes repeated calls reproducible, and a local Generator avoids changing NumPy's global random state. Reusing these fixed antithetic nodes reduces noise when comparing candidate parameter vectors. Drawing afresh inside each objective call would make the optimizer chase random changes as well as parameter effects. More draws still cost more computation.

Split-model update. heterogeneous_history uses D-by-3 draws with alpha, novelty and own-stock coordinates; other models keep D-by-2 draws. The draw dimension and seed must match the saved fit when reconstructing likelihoods.

Definition and stated intent:

Source lines 378–379:

```
378 def make_draws(number, seed, model=None):
379     """Fixed antithetic normal draws, reused throughout optimization."""
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 380–381:

```
380 if number < 2 or number % 2:
381     raise ValueError("Use an even number of draws, at least two.")
```

Read this block:

- Line 380: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 381: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 382–382:

```
382 z = np.random.default_rng(seed).standard_normal((number // 2, 3 if model == "heterogeneous_history" else 2))
```

Read this block:

- Line 382, `z`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 383–383:

```
383 return np.vstack([z, -z])
```

Read this block:

- Line 383: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

parameter names

Original function: `parameter_names`. Source begins at line 386.

This returns the ordered labels defining the parameter vector. `names += [...]` extends a list; it does not numerically add its string entries. Membership tests such as `model in ("mixed_naive", "heterogeneous")` let related specifications share branches. The default `control=False` omits a residual coefficient unless the caller explicitly enables it.

For naive conditional demand without controls, the returned labels are two betas and mean log-minus-alpha. Heterogeneous demand adds mean gamma and both SDs; the optional outside constant is prepended and price control appended. Keeping this logic in one helper is safer than independently remembering vector positions in optimization, utility, and output. The order is part of the API: zipping a reordered list with old coefficients silently changes their meaning.

Split layouts. separate_history adds mean_novelty and mean_own_stock plus random price sensitivity; heterogeneous_history also adds sd_novelty and sd_own_stock. This helper is also used when checking the saved likelihood during weekly-export regeneration.

Definition and stated intent:

Source lines 386–386:

```
386 def parameter_names(model, outside, control=False):
```

`def` creates this callable; the parentheses name its inputs (`model`, `outside`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 387–388:

```
387 names = (["inside_constant"] if outside else []) + ["beta_berry", "beta_plain"]
388 names += ["mu_log_minus_alpha"]
```

Read this block:

- Line 387, `names`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 388: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 389–390:

```
389 if model in ("homogeneous", "heterogeneous"):
390     names += ["mean_gamma"]
```

Read this block:

- Line 389: test `model in ('homogeneous', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 390: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 391–392:

```
391 if model in ("mixed_naive", "heterogeneous"):
392     names += ["sd_log_minus_alpha"]
```

Read this block:

- Line 391: test `model in ('mixed_naive', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 392: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 393–394:

```
393 if model == "heterogeneous":
394     names += ["sd_gamma"]
```

Read this block:

- Line 393: test `model == 'heterogeneous'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 394: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 395–397:

```
395 if model == "mixture":
396     names += ["mean_gamma", "sd_log_minus_alpha", "sd_gamma",
397               "gamma_mean_gap", "sd_gamma_2", "mixture_weight"]
```

Read this block:

- Line 395: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 396: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 398–399:

```
398 if model in ("separate_history", "heterogeneous_history"):
399     names += ["mean_novelty", "mean_own_stock", "sd_log_minus_alpha"]
```

Read this block:

- Line 398: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 399: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 400–401:

```
400 if model == "heterogeneous_history":
401     names += ["sd_novelty", "sd_own_stock"]
```

Read this block:

- Line 400: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 401: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 402–403:

```
402 if control:
403     names += ["price_control"]
```

Read this block:

- Line 402: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 403: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 404–404:

```
404 return names
```

Read this block:

- Line 404: return the object currently named `names`; plain assignment does not copy it. This ends the call.

mixture components

Original function: `mixture_components`. Source begins at line 407.

Unpacks a parameter vector into two ordered normal-gamma components and a common price distribution. Adding a nonnegative gap orders component means and prevents label switching; it does not force either component to have positive gamma.

Definition and stated intent:

Source lines 407–408:

```
407 def mixture_components(theta, data):
408     """Ordered component means prevent label switching; alpha is shared."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 409–414:

```
409 names = parameter_names("mixture", data["outside"], data["control"])
410 p = dict(zip(names, theta))
411 normal_names = parameter_names("heterogeneous", data["outside"], data["control"])
412 first = np.array([p[n] for n in normal_names])
413 second = first.copy()
414 second[normal_names.index("mean_gamma")] += p["gamma_mean_gap"]
```

Read this block:

- Line 409, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 410, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 411, `normal_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 412, `first`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 413, `second`: Set this name to the result of calling `first.copy(...)`; the arguments and preceding method chain are shown above.
- Line 414: update `second[normal_names.index('mean_gamma')]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 415–415:

```
415 second[normal_names.index("sd_gamma")] = p["sd_gamma_2"]
```

Read this block:

- Line 415, `second[normal_names.index('sd_gamma')]`: Store the entry/slice selected by `p['sd_gamma_2']`; brackets select from the object to their left in the selected entry/column.

Source lines 416–416:

```
416 return names, normal_names, first, second, p["mixture_weight"]
```

Read this block:

- Line 416: return a tuple of 5 results. The caller can unpack them; return ends the call, unlike print.

mixture loglike score

Original function: `mixture_loglike_score`. Source begins at line 419.

Combines complete household likelihoods across persistent mixture components, using stable log-sum-exp. Mixing each week separately would change the model by allowing household membership to change over time. The score combines component derivatives with posterior membership weights.

Definition and stated intent:

Source lines 419–420:

```
419 def mixture_loglike_score(theta, data, draws, score=False):
420     """Mix whole household histories, never individual weeks."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `score`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 421–421:

```
421 names, normal_names, first, second, weight = mixture_components(theta, data)
```

Read this block:

- Line 421, `(names, normal_names, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 422–427:

```
422 if score:
423     ll1, g1 = demand_objective_gradient(first, data, draws, "heterogeneous", household=True)
424     ll2, g2 = demand_objective_gradient(second, data, draws, "heterogeneous", household=True)
425 else:
426     ll1 = household_loglike(first, data, draws, "heterogeneous")
427     ll2 = household_loglike(second, data, draws, "heterogeneous")
```

Read this block:

- Line 422: test `score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 423, `(ll1, g1)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 424, `(ll2, g2)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 426, `ll1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 427, `ll2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 428–429:

```
428 terms = np.array([np.log(weight) + ll1, np.log1p(-weight) + ll2])
429 ll = logsumexp(terms, axis=0)
```

Read this block:

- Line 428, `terms`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 429, `ll`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.

Source lines 430–431:

```
430 if not score:
431     return ll
```

Read this block:

- Line 430: test `not score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 431: return the object currently named `ll`; plain assignment does not copy it. This ends the call.

Source lines 432–433:

```
432 responsibility = np.exp(terms - ll)
433 gradient = np.zeros((data["n_households"], len(names)))
```

Read this block:

- Line 432, `responsibility`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 433, `gradient`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 434–439:

```
434 for k, name in enumerate(normal_names):
435     if name == "sd_gamma":
436         gradient[:, names.index(name)] = responsibility[0] * g1[:, k]
437         gradient[:, names.index("sd_gamma_2")] = responsibility[1] * g2[:, k]
438     else:
439         gradient[:, names.index(name)] = responsibility[0] * g1[:, k] + responsibility[1] * g2[:, k]
```

Read this block:

- Line 434: for each entry yielded by `enumerate(normal_names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 435: test `name == 'sd_gamma'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 436, `gradient[:, names.index(name)]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 437, `gradient[:, names.index('sd_gamma_2')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 439, `gradient[:, names.index(name)]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 440–441:

```
440 gradient[:, names.index("gamma_mean_gap")] = responsibility[1] * g2[:, normal_names.index("mean_gamma")]
441 gradient[:, names.index("mixture_weight")] = responsibility[0] / weight - responsibility[1] / (1 - weight)
```

Read this block:

- Line 440, `gradient[:, names.index('gamma_mean_gap')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 441, `gradient[:, names.index('mixture_weight')]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 442–442:

```
442 return -ll.mean(), -gradient.mean(axis=0)
```

Read this block:

- Line 442: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

history utility

Original function: `history_utility`. Source begins at line 445.

Chooses the history formula from the model name. Split models use b_N*N*+*b*_*S*S directly, with distinct draw coordinates for the two random coefficients. Combined-gamma models use gamma*Xi. The habit-weight parameter belongs to Xi and does not multiply the independently estimated split coefficients.

Definition and stated intent:

Source lines 445–445:

```
445 def history_utility(p, data, z, model):
```

`def` creates this callable; the parentheses name its inputs (`p`, `data`, `z`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 446–453:

```
446 if model in ("separate_history", "heterogeneous_history"):
447     novelty = p["mean_novelty"] + p.get("sd_novelty", 0.) * z[1]
448     own_stock = p["mean_own_stock"]
449     if model == "heterogeneous_history":
450         if len(z) < 3:
451             raise ValueError("heterogeneous_history requires three-dimensional draws.")
452         own_stock += p["sd_own_stock"] * z[2]
```

```
453     return novelty * data["novelty"] + own_stock * data["stock"]
```

Read this block:

- Line 446: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 447, `novelty`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 448, `own_stock`: Set this name to the entry/slice selected by `p['mean_own_stock']`; brackets select from the object to their left.
- Line 449: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 450: test `len(z) < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 451: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 452: update `own_stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 453: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `len`: Count elements/rows rather than summing their numerical values.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 454–454:

```
454 gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
```

Read this block:

- Line 454, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 455–455:

```
455 return gamma * data["xi"]
```

Read this block:

- Line 455: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

household loglike

Original function: `household_loglike`. Source begins at line 458.

Input `theta` is a one-dimensional parameter array, `data` is the dictionary from `make_arrays`, and `draws` is a draw-by-two matrix for combined models or draw-by-three for split heterogeneity. The return is one log likelihood per household, not a scalar. `dict(zip(...))` attaches coefficient names so the rest of the function need not remember their numeric positions.

For each latent-type draw, negative exponential constructs alpha and an affine normal transformation constructs gamma. Broadcasting makes beta and these scalar coefficients operate across every row/flavor. Unavailable utility is set to minus infinity; an optional zero outside column is appended. `logprob = utility - logsumexp(..., axis=1, keepdims=True)` normalizes each row. The retained column dimension makes subtraction broadcast across that row.

The count-weighted row sum uses `where(counts > 0, logprob, 0)` to avoid zero times minus infinity. `bincount` adds all weeks of the same household. Only after this aggregation does the final `logsumexp` integrate draws. This order implements persistent preferences. Averaging weekly probabilities first would be simpler computationally but would estimate a different preference process.

Current split utility. history_utility supplies b_NN+b_SS, with separate means and normal draw coordinates. Gamma*Xi is retained only for combined models. In a mixture model, the separate component helper mixes complete household likelihoods. Neither split coefficient receives the old fixed lambda weight.

Definition and stated intent:

Source lines 458–464:

```
458 def household_loglike(theta, data, draws, model):
459     """L_i = mean_r exp(sum_t sum_j count_itj * log P_itj,r).
460 
461     Gamma is normal and common across flavors; alpha = -exp(mu + sd*z). Beta never varies by draw.
462     Process one draw at a time to avoid a weeks x draws x products array.
463     The multinomial combinatorial constant is omitted (independent of theta).
464     """
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 465–466:

```
465 if model == "mixture":
466     return mixture_loglike_score(theta, data, draws)
```

Read this block:

- Line 465: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 466: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 467–473:

```
467 p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
468 beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
469 beta += p.get("inside_constant", 0.0)
470 loglikes = []
471 active_draws = (
472     draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
473 )
```

Read this block:

- Line 467, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 468, `beta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 469: update `beta` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 470, `loglikes`: Set this name to the list `[]`.
- Line 471, `active_draws`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 474–492:

```
474 for z in active_draws:
475     alpha = -np.exp(
476         p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.0) * z[0]
477     )
478     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
479     utility = (
480         beta
481         + alpha * data["prices"]
482         + history_utility(p, data, z, model)
483         + p.get("price_control", 0.0) * data["cf"]
484     )
485     utility = np.where(data["available"], utility, -np.inf)
486     if data["outside"]:
487         utility = np.column_stack([utility, np.zeros(len(utility))])
488     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
489     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
490     loglikes.append(
491         np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"])
492     )
```

Read this block:

- Line 474: for each entry yielded by `active_draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 475, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 478, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 479, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 485, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 486: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 487, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 488, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 489, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 490: call `loglikes.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere;

np.where instead takes explicit true/false branches.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 493–493:

```
493 return logsumexp(loglikes, axis=0) - np.log(len(active_draws))
```

Read this block:

- Line 493: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

demand objective gradient

Original function: `demand_objective_gradient`. Source begins at line 496.

This returns a tuple `(mean_negative_log_likelihood, gradient)`, which SciPy requests when `jac=True`. The utility and likelihood steps mirror `household_loglike`; the additional work computes derivatives in the same draw loop. `total[:, None]` expands a weekly count vector to a column so it can multiply the matrix of choice probabilities. `residual` is observed count minus predicted count for the three inside alternatives.

The `columns` dictionary holds one weekly score vector per coefficient name. For example, the berry intercept takes column 1 of residual, while the alpha location score uses the price-weighted residual multiplied by alpha itself because differentiating `-exp(mu + sd*z)` contributes alpha. Each score is aggregated to households, and `column_stack` places parameters on a final axis.

`np.einsum("rh,rhp->p", weights, scores)` means multiply posterior weight at draw r/household h by its parameter-p score, then sum r and h, retaining p. A nested loop over r, h, p would express the same sum more verbosely and slowly. The minus sign and household-count division match minimization of average NLL. This is an analytic derivative of the finite-draw objective, not automatic differentiation or a finite-difference estimate.

Definition and stated intent:

Source lines 496–497:

```
496 def demand_objective_gradient(theta, data, draws, model, household=False):
497     """Exact simulated likelihood score, weighted by household type posteriors."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`, `household`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 498–499:

```
498 if model == "mixture":
499     return mixture_loglike_score(theta, data, draws, score=True)
```

Read this block:

- Line 498: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 499: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 500–504:

```
500 names = parameter_names(model, data["outside"], data["control"])
501 p = dict(zip(names, theta))
502 active = draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
503 likelihoods, scores = [], []
504 total = data["counts"].sum(axis=1)
```

Read this block:

- Line 500, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 501, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 502, `active`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.
- Line 503, `(likelihoods, scores)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 504, `total`: Set this name to the result of calling `data['counts'].sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 505–531:

```
505 for z in active:
506     alpha = -np.exp(p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.) * z[0])
507     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
508     utility = (np.array([0., p["beta_berry"], p["beta_plain"]]) + p.get("inside_constant", 0.)
509                + alpha * data["prices"] + history_utility(p, data, z, model) + p.get("price_control", 0.) *
                    ↪  data["cf"])
510     utility = np.where(data["available"], utility, -np.inf)
511     if data["outside"]:
512         utility = np.column_stack([utility, np.zeros(len(utility))])
513     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
514     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
515     likelihoods.append(np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"]))
516     residual = data["counts"][:, :3] - total[:, None] * np.exp(logprob[:, :3])
517     price_score = (residual * data["prices"]).sum(axis=1) * alpha
518     columns = {"inside_constant": residual.sum(axis=1), "beta_berry": residual[:, 1],
519                "beta_plain": residual[:, 2], "mu_log_minus_alpha": price_score,
520                "sd_log_minus_alpha": price_score * z[0],
```

```
521                 "price_control": (residual * data["cf"]).sum(axis=1)}
522     columns["mean_gamma"] = (residual * data["xi"]).sum(axis=1)
523     columns["sd_gamma"] = columns["mean_gamma"] * z[1]
524     if model in ("separate_history", "heterogeneous_history"):
525         columns["mean_novelty"] = (residual * data["novelty"]).sum(axis=1)
526         columns["mean_own_stock"] = (residual * data["stock"]).sum(axis=1)
527         columns["sd_novelty"] = columns["mean_novelty"] * z[1]
528         if model == "heterogeneous_history":
529             columns["sd_own_stock"] = columns["mean_own_stock"] * z[2]
530     scores.append(np.column_stack([np.bincount(data["hh"], weights=columns[n],
531                                                minlength=data["n_households"]) for n in names]))
```

Read this block:

- Line 505: for each entry yielded by `active`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 506, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 507, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 508, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 510, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 511: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 512, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 513, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 514, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 515: call `likelihoods.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 516, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 517, `price_score`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 518, `columns`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `price_control`.
- Line 522, `columns['mean_gamma']`: Store the result of calling `(residual * data['xi']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 523, `columns['sd_gamma']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 524: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 525, `columns['mean_novelty']`: Store the result of calling `(residual * data['novelty']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 526, `columns['mean_own_stock']`: Store the result of calling `(residual * data['stock']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 527, `columns['sd_novelty']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 528: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 529, `columns['sd_own_stock']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 530: call `scores.append` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `len`: Count elements/rows rather than summing their numerical values.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 532–535:

```
532 likelihoods = np.asarray(likelihoods)
533 normalizer = logsumexp(likelihoods, axis=0)
534 weights = np.exp(likelihoods - normalizer)
535 household_scores = np.einsum("rh,rhp->hp", weights, np.asarray(scores))
```

Read this block:

- Line 532, `likelihoods`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 533, `normalizer`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 534, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 535, `household_scores`: Set this name to the result of calling `np.einsum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.

Source lines 536–537:

```
536 if household:
537     return normalizer - np.log(len(active)), household_scores
```

Read this block:

- Line 536: test `household`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 537: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Source lines 538–538:

```
538 gradient = -household_scores.mean(axis=0)
```

Read this block:

- Line 538, `gradient`: Set this name to the negated or sign-adjusted expression `-household_scores.mean(axis=0)`.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 539–539:

```
539 return -(normalizer - np.log(len(active))).mean(), gradient
```

Read this block:

- Line 539: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

optimize demand

Original function: `optimize_demand`. Source begins at line 542.

This sets up and runs multiple bounded local searches and returns the selected fit, every starting solution, and the bounds. Initial SDs are .3 because zero can be stationary under symmetric integration. Bounds are pairs `(lower, upper)` in the same order as parameter names. The optional initial vector is copied before perturbation, preventing caller-owned coefficients from being changed.

Each start uses `minimize(..., method="L-BFGS-B", jac=True)`. The callback reports progress every ten iterations, while completed starts are saved to JSON. `np.clip` keeps perturbed starts feasible. `min(results, key=lambda r: r.fun)` uses each result's loss as its comparison key; it selects the smallest loss, not necessarily a converged fit. An alternative is to select only converged results, but that is not the rule this code implements. Multistart search probes local solutions and does not establish a global optimum.

Definition and stated intent:

Source lines 542–542:

```
542 def optimize_demand(data, args, initial=None, number_starts=None, quiet=False):
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `initial`, `number_starts`, `quiet`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 543–546:

```
543 draws = make_draws(args.draws, args.seed, args.model)
544 names = parameter_names(args.model, data["outside"], data["control"])
545 start = np.zeros(len(names))
546 bounds = []
```

Read this block:

- Line 543, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 544, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 545, `start`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 546, `bounds`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 547–560:

```
547 for k, name in enumerate(names):
548     if name.startswith("sd_"):
549         start[k] = 0.3  # Zero SD is a stationary point: start away from zero.
550         bounds.append((0.0, 3.0))
551     elif name == "gamma_mean_gap":
552         start[k] = 0.4
553         bounds.append((0.0, 40.0))
554     elif name == "mixture_weight":
555         start[k] = 0.5
556         bounds.append((0.001, 0.999))
557     elif name == "mu_log_minus_alpha":
558         bounds.append((-6.0, 3.0))
559     else:
560         bounds.append((-20.0, 20.0))
```

Read this block:

- Line 547: for each entry yielded by `enumerate(names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 548: test `name.startswith('sd_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 549, `start[k]`: Store the literal value `0.3` in the selected entry/column.
- Line 550: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 551: test `name == 'gamma_mean_gap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 552, `start[k]`: Store the literal value `0.4` in the selected entry/column.
- Line 553: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 554: test `name == 'mixture_weight'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 555, `start[k]`: Store the literal value `0.5` in the selected entry/column.
- Line 556: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 557: test `name == 'mu_log_minus_alpha'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 558: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 560: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 561–562:

```
561 if args.model == "mixture":
562     start[names.index("mean_gamma")] = -0.2
```

Read this block:

- Line 561: test `args.model == 'mixture'`. Execute the indented branch only when it is true; an else/elif

branch handles its alternative.
- Line 562, `start[names.index('mean_gamma')]`: Store the negated or sign-adjusted expression `-0.2` in the selected entry/column.

Source lines 563–564:

```
if initial is not None:
    start = np.asarray(initial).copy()
```

Read this block:

- Line 563: test `initial is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 564, `start`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 565–566:

```
rng = np.random.default_rng(args.seed + 1)
results = []
```

Read this block:

- Line 565, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 566, `results`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 568–569:

```
def objective(theta):
    return demand_objective_gradient(theta, data, draws, args.model)
```

Read this block:

- Line 568: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 571–605:

```
for s in range(args.starts if number_starts is None else number_starts):
    initial = start.copy()
    if s:
        initial += rng.normal(0, 0.5, len(start))
        initial = np.clip(initial, np.array(bounds)[:, 0], np.array(bounds)[:, 1])
    began = time.monotonic()
    iterations = [0]

    def progress(theta):
        iterations[0] += 1
        if not quiet and iterations[0] % 10 == 0:
            print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)

    fit = minimize(
        objective,
```

```
        initial,
        method="L-BFGS-B",
        jac=True,
        callback=progress,
        bounds=bounds,
        options={"maxiter": args.maxiter, "ftol": 1e-10, "gtol": 1e-6},
    )
    results.append(fit)
    if not quiet:
        args.output.mkdir(parents=True, exist_ok=True)
        (args.output / "starts_checkpoint.json").write_text(json.dumps([
            dict(theta=r.x.tolist(), nll=float(r.fun * data["n_households"]),
                 converged=bool(r.success), iterations=int(r.nit), evaluations=int(r.nfev))
            for r in results], indent=2))
    if not quiet:
        print(
            f"Start {s + 1}: NLL={fit.fun * data['n_households']:.4f}; "
            f"converged={fit.success}",
            flush=True,
        )
```

Read this block:

- Line 571: for each entry yielded by `range(args.starts if number_starts is None else number_starts)`, bind/unpack it into `s` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 572, `initial`: Set this name to the result of calling `start.copy(...)`; the arguments and preceding method chain are shown above.
- Line 573: test `s`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 574: update `initial` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 575, `initial`: Set this name to the result of calling `np.clip(...)`; the arguments and preceding method chain are shown above.
- Line 576, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 577, `iterations`: Set this name to the list `[0]`.
- Line 579: define nested callable `progress`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 584, `fit`: Set this name to the result of calling `minimize(...)`; the arguments and preceding method chain are shown above.
- Line 593: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 594: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 595: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 596: call `(args.output / 'starts_checkpoint.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 600: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 601: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `minimize`: Run iterative numerical minimization; jac=True expects the objective to return value plus gradient.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.

- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 607–607:

```
607 best = min(results, key=lambda r: r.fun)
```

Read this block:

- Line 607, `best`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 608–608:

```
608 return best, results, bounds
```

Read this block:

- Line 608: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

optimize demand.objective

Original function: `optimize_demand.objective`. Source begins at line 568.

This small nested adapter has one explicit input, `theta`. It forwards theta and the enclosing call's data/draws/model to the actual objective/gradient function. SciPy expects a callable accepting the changing candidate vector; the closure supplies the fixed objects. A global variable for data could work but would make concurrent or repeated fits harder to reason about. Returning the whole pair is intentional because the surrounding minimizer sets `jac=True`.

Definition and stated intent:

Source lines 568–568:

```
568 def objective(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 569–569:

```
569 return demand_objective_gradient(theta, data, draws, args.model)
```

Read this block:

- Line 569: return the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimize demand.progress

Original function: `optimize_demand.progress`. Source begins at line 579.

SciPy calls this nested callback after iterations, passing the current vector. The body increments `iterations[0]`, checks a remainder modulo ten, and prints elapsed time unless quiet mode is active. The callback does not return an objective value and does not choose the next parameter vector.

The enclosing variable is a one-element list so the nested function can mutate its element without rebinding the outer name. An ordinary integer counter with assignment would require `nonlocal iterations`. That alternative is often clearer; the existing list technique works because objects can be mutated through a captured reference. `time.monotonic` measures elapsed duration without wall-clock adjustments changing the result.

Definition and stated intent:

Source lines 579–579:

```
579 def progress(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 580–580:

```
580 iterations[0] += 1
```

Read this block:

- Line 580: update `iterations[0]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 581–582:

```
581 if not quiet and iterations[0] % 10 == 0:
582     print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
```

Read this block:

- Line 581: test `not quiet and iterations[0] % 10 == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 582: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot gamma distribution

Original function: `plot_gamma_distribution`. Source begins at line 611.

Input is an estimate table, output directory, and convergence flag. This is a side-effect function: it saves a figure and implicitly returns `None`. `set_index("parameter")["estimate"]` creates a name-indexed Series for coefficient lookup. The early bare `return` skips naive models with no gamma.

For positive SD, `linspace` makes a smooth horizontal grid, `norm.pdf` computes density, and `fill_between(..., where=x < 0)` shades the negative region. For zero SD a continuous density would divide by zero or misrepresent a point mass, so the `else` branch draws a vertical line. `norm.cdf(0, ...)` computes the implied population habit share. `Agg` supports file rendering without a GUI; `plt.close(fig)` releases the figure. This is a distribution of latent types, not an uncertainty interval for the mean.

Definition and stated intent:

Source lines 611–612:

```
611 def plot_gamma_distribution(table, output, converged=True):
612     """Plot the fitted population distribution, not uncertainty in its mean."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 613–613:

```
613 estimates = table.set_index("parameter")["estimate"]
```

Read this block:

- Line 613, `estimates`: Set this name to the entry/slice selected by `table.set_index('parameter')['estimate']`; brackets select from the object to their left.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 614–615:

```
614 if "mean_gamma" not in estimates:
615     return  # Naive models have no gamma parameter.
```

Read this block:

- Line 614: test `'mean_gamma' not in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 615: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 616–622:

```
616 import matplotlib
617 matplotlib.use("Agg")
618 import matplotlib.pyplot as plt
619 from scipy.stats import norm
620
621 mean = float(estimates["mean_gamma"])
622 sd = float(estimates.get("sd_gamma", 0.))
```

Read this block:

- Line 616: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 617: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 618: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 619: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 621, `mean`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 622, `sd`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 623–623:

```
623 fig, ax = plt.subplots(figsize=(7, 5))
```

Read this block:

- Line 623, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 624–659:

```
624 if "mixture_weight" in estimates:
625     weight = float(estimates["mixture_weight"])
626     mean2 = mean + float(estimates["gamma_mean_gap"])
627     sd2 = float(estimates["sd_gamma_2"])
628     x = np.linspace(min(mean - 4 * sd, mean2 - 4 * sd2) - .01,
629                     max(mean + 4 * sd, mean2 + 4 * sd2) + .01, 1001)
630     density = np.zeros_like(x)
631     negative = 0.
632     for mu, sigma, share in [(mean, sd, weight), (mean2, sd2, 1 - weight)]:
633         if sigma > 0:
634             component = share * norm.pdf(x, loc=mu, scale=sigma)
635             density += component
636             ax.plot(x, component, linestyle=":", label=f"Component (weight {share:.1%})")
637             negative += share * norm.cdf(0, loc=mu, scale=sigma)
638         else:
639             ax.axvline(mu, linestyle=":", label=f"Point mass (weight {share:.1%})")
640             negative += share * (mu < 0)
641     ax.plot(x, density, color="steelblue", label="Mixture continuous density")
642     ax.set_ylabel("Population density")
643     ax.legend()
644     description = f"Component means = {mean:.4g}, {mean2:.4g}; P(gamma < 0) = {negative:.1%}"
645 elif sd > 0:
646     x = np.linspace(mean - 4 * sd, mean + 4 * sd, 801)
647     density = norm.pdf(x, loc=mean, scale=sd)
648     ax.plot(x, density, color="steelblue")
649     ax.fill_between(x, density, where=x < 0, color="steelblue", alpha=.25)
650     ax.set_ylabel("Population density")
```

```
    negative = norm.cdf(0, loc=mean, scale=sd)
    description = f"Mean = {mean:.4g}; SD = {sd:.4g}; P(gamma < 0) = {negative:.1%}"
else:
    ax.axvline(mean, color="steelblue", linewidth=2, label="Point mass (SD = 0)")
    ax.set_xlim(mean - 1, mean + 1)
    ax.set_yticks([])
    ax.set_ylabel("Point mass")
    ax.legend()
    description = f"All households: gamma = {mean:.4g} (SD = 0)"
```

Read this block:

- Line 624: test `'mixture_weight' in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 625, `weight`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 626, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 627, `sd2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 628, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 630, `density`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.
- Line 631, `negative`: Set this name to the literal value `0.0`.
- Line 632: for each entry yielded by `[(mean, sd, weight), (mean2, sd2, 1 - weight)]`, bind/unpack it into `(mu, sigma, share)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 633: test `sigma > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 634, `component`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 635: update `density` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 636: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 637: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 639: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 640: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 641: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 642: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 643: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 644, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 645: test `sd > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 646, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 647, `density`: Set this name to the result of calling `norm.pdf(...)`; the arguments and preceding method chain are shown above.
- Line 648: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 649: call `ax.fill_between` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 650: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 651, `negative`: Set this name to the result of calling `norm.cdf(...)`; the arguments and preceding method chain are shown above.
- Line 652, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 654: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 655: call `ax.set_xlim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 656: call `ax.set_yticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 657: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 658: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.pdf`: Evaluate a probability density (here a normal distribution), not a PDF document file.
- `.fill_between`: Shade the area between curves/zero, optionally only where a Boolean condition is true.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set_xlim`: Set the visible x-axis limits; this changes presentation rather than the calculated results.
- `.set_yticks`: Choose which vertical tick marks to show, including an empty list for a point-mass display.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 660–661:

```
660 ax.axvline(0, color="gray", linestyle="--", linewidth=1)
661 title = "Fitted population gamma distribution"
```

Read this block:

- Line 660: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 661, `title`: Set this name to the literal text `'Fitted population gamma distribution'`.

Source lines 662–663:

```
662 if not converged or table["at_bound"].any():
663     title += " (provisional fit)"
```

Read this block:

- Line 662: test `not converged or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 663: update `title` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 664–668:

```
664 ax.set(xlabel="Gamma (common across flavors)", title=title)
665 fig.text(.5, .02, description, ha="center")
666 fig.tight_layout(rect=(0, .06, 1, 1))
667 fig.savefig(output / "gamma_distribution.pdf")
668 plt.close(fig)
```

Read this block:

- Line 664: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 665: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 666: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 667: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 668: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

plot history distributions

Original function: `plot_history_distributions`. Source begins at line 671.

Delegates to the population-distribution export routine immediately after a split fit. Its side effects are tables and plots, not new coefficient estimates. The readable estimator keeps the full export calculation in the same file; the original estimator imports that helper.

Definition and stated intent:

Source lines 671–672:

```
671 def plot_history_distributions(table, output, converged=True):
672     """Export fitted marginal population distributions after every split fit."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 673–674:

```
673 from history_distributions import export_history_distributions
674 export_history_distributions(table, output, converged)
```

Read this block:

- Line 673: from `history_distributions`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 674: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

fit model

Original function: `fit_model`. Source begins at line 678.

This coordinates point fitting, reporting, optional inference, and saved files. It turns `best.x` into a labeled table. The boundary list comprehension pairs each coefficient with its bounds and tests distance below `1e-4`; flags do not automatically change the estimate. Saving point estimates before bootstrap protects the useful main result if a long inference job is interrupted.

The nested `estimate` callback supplies demand fitting to the generic bootstrap. The bootstrap runs only after a converged main fit and with the original weekly frame available. Missing inference columns are explicitly filled with NaN so an absent SE cannot be mistaken for zero. `vars(args).items()` enumerates settings; Path objects become strings so JSON serialization can encode them. The returned `best` is still a SciPy result object, while CSV/JSON provide portable outputs.

Distribution outputs. After the main split fit, both history marginals are exported as parameter/quantile summaries, density/CDF grids and PDF/PNG plots. Their SDs are population variation; SEs in estimates.csv describe sampling uncertainty. The saved fit can regenerate these plots without optimization.

Definition and stated intent:

Source lines 678–679:

```
678 def fit_model(data, args, frame=None):
679     """Estimate demand, compute requested inference, and save readable outputs."""
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 680–687:

```
680 best, results, bounds = optimize_demand(data, args)
681 names = parameter_names(args.model, data["outside"], data["control"])
682 table = pd.DataFrame({"parameter": names, "estimate": best.x})
683 table["at_bound"] = [
684     min(abs(v - lo), abs(v - hi)) < 1e-4 for v, (lo, hi) in zip(best.x, bounds)
685 ]
686 args.output.mkdir(parents=True, exist_ok=True)
687 table.to_csv(args.output / "point_estimates.csv", index=False)
```

Read this block:

- Line 680, `(best, results, bounds)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 681, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 682, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 683, `table['at_bound']`: Store a list computed by the displayed for-expression in the selected entry/column.
- Line 686: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 687: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 688–693:

```
688 plot_gamma_distribution(table, args.output, converged=best.success)
689 plot_history_distributions(table, args.output, converged=best.success)
690 wtp = implied_wtp(table, getattr(args, "habit_weight", .7))
691 wtp["provisional_fit"] = not best.success or bool(table["at_bound"].any())
692 wtp.to_csv(args.output / "implied_wtp.csv", index=False)
693 inference = {"method": args.se_method, "status": "disabled"}
```

Read this block:

- Line 688: call `plot_gamma_distribution` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 689: call `plot_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 690, `wtp`: Set this name to the result of calling `implied_wtp(...)`; the arguments and preceding method chain are shown above.
- Line 691, `wtp['provisional_fit']`: Store the Boolean condition `not best.success or bool(table['at_bound'].any())`; it can be a scalar or a row/array mask depending on its inputs in the selected entry/column.
- Line 692: call `wtp.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 693, `inference`: Set this name to a dictionary with named entries `method`, `status`.

Syntax and calls used here:

- `plot_gamma_distribution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `plot_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 694–724:

```
694 if args.se_method == "bootstrap":
695     if not best.success:
696         inference["status"] = "main fit not converged; bootstrap not run"
697     elif frame is None:
698         raise ValueError("Bootstrap requires the weekly input frame.")
699     else:
```

```

        def estimate(sample, initial):
            bootstrap_data = make_arrays(sample, args.capacity)
            fitted, _, _ = optimize_demand(
                bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
            )
            return fitted

        standard_errors, inference = bootstrap_standard_errors(
            frame, args, estimate, best.x, names
        )
        if standard_errors is not None:
            table = table.merge(
                standard_errors, on="parameter", validate="one_to_one"
            )
elif args.se_method == "hessian":
    if not best.success:
        inference["status"] = "main fit not converged; Hessian inference not run"
    else:
        draws = make_draws(args.draws, args.seed, args.model)
        gradient = lambda theta: demand_objective_gradient(theta, data, draws, args.model)[1]
        standard_errors, inference = hessian_standard_errors(
            best.x, gradient, data["n_households"], bounds, names, args.output)
        if standard_errors is not None:
            table = table.merge(standard_errors, on="parameter", validate="one_to_one")
```

Read this block:

- Line 694: test `args.se_method == 'bootstrap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 695: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 696, `inference['status']`: Store the literal text `'main fit not converged; bootstrap not run'` in the selected entry/column.
- Line 697: test `frame is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 698: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 701: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 708, `(standard_errors, inference)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 711: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 712, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.
- Line 715: test `args.se_method == 'hessian'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 716: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 717, `inference['status']`: Store the literal text `'main fit not converged; Hessian inference not run'` in the selected entry/column.
- Line 719, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 720, `gradient`: Set this name to a one-expression callable, with input `theta` and returned expression `demand_objective_gradient(theta, data, draws, args.m...`.
- Line 721, `(standard_errors, inference)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 723: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 724, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 726–728:

```
726 for column in ["std_error", "ci_lower", "ci_upper"]:
727     if column not in table:
728         table[column] = np.nan
```

Read this block:

- Line 726: for each entry yielded by `['std_error', 'ci_lower', 'ci_upper']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 727: test `column not in table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 728, `table[column]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Source lines 729–729:

```
729 table.to_csv(args.output / "estimates.csv", index=False)
```

Read this block:

- Line 729: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 730–756:

```
730 summary = dict(
731     model=args.model,
732     price_control=data["control"],
733     history_specification="separate" if args.model in ("separate_history", "heterogeneous_history") else
    ↪ "combined_or_none",
734     sample_restriction="household_size == 1 on each purchase record",
735     households=data["n_households"],
736     weeks=len(data["hh"]),
737     capacity=args.capacity,
738     inference=inference,
739     converged=bool(best.success),
740     message=str(best.message),
741     nll=float(best.fun * data["n_households"]),
742     parameters=len(names),
```

```
    aic=float(2 * best.fun * data["n_households"] + 2 * len(names)),
    bic=float(2 * best.fun * data["n_households"] + np.log(data["n_households"]) * len(names)),
    settings={
        k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()
    },
    starts=[
        dict(
            nll=float(r.fun * data["n_households"]),
            converged=bool(r.success),
            theta=r.x.tolist(),
        )
        for r in results
    ],
)
```

Read this block:

- Line 730, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 757–759:

```
(args.output / "fit.json").write_text(json.dumps(summary, indent=2))
print("Inference: " + inference["status"], flush=True)
print(table.to_string(index=False))
```

Read this block:

- Line 757: call `(args.output / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 758: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 759: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 760–761:

```
if not best.success or table["at_bound"].any():
    print("Provisional fit: inspect convergence/bounds before interpretation.")
```

Read this block:

- Line 760: test `not best.success or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 761: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 762–762:

```
762 return best
```

Read this block:

- Line 762: return the object currently named `best`; plain assignment does not copy it. This ends the call.

fit model.estimate

Original function: `fit_model.estimate`. Source begins at line 701.

This nested function accepts one resampled weekly table and a starting coefficient vector. It rebuilds numerical arrays, runs demand optimization with bootstrap start count, and returns only the selected fit. `fitted, _, _` discards other optimizer outputs. It captures capacity and other settings from outer `args`. Passing this callback avoids importing `est.py` back into `bootstrap_se.py`, which could create a circular module dependency. Price controls are reestimated by the bootstrap before this function is called, not inside this callback.

Definition and stated intent:

Source lines 701–701:

```
701 def estimate(sample, initial):
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 702–705:

```
702 bootstrap_data = make_arrays(sample, args.capacity)
703 fitted, _, _ = optimize_demand(
704     bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
705 )
```

Read this block:

- Line 702, `bootstrap_data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 703, `(fitted, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 706–706:

```
706 return fitted
```

Read this block:

- Line 706: return the object currently named `fitted`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 766.

This is the command-line entry point. `default_model` lets small wrapper scripts reuse the entire interface while selecting a different default model. Each `add_argument` describes a setting, conversion, default, or permitted choices. The parser turns a flag such as `--no-price-control` into the attribute `args.no_price_control`; hyphens become underscores in attribute names.

Validation runs before expensive data loading. `parser.error` prints usage and exits for invalid settings. After parsing, execution is preparation, optional first stages, weekly input export, array construction, and fitting. This division lets tests construct a small `SimpleNamespace` and call internal functions without pretending to run a shell command. Repeating this code in all four entry scripts would allow defaults and validations to drift apart.

Definition and stated intent:

Source lines 766–766:

```
766 def main(default_model="heterogeneous", default_no_price_control=False):
```

`def` creates this callable; the parentheses name its inputs (`default_model`, `default_no_price_control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 767–779:

```
767 parser = argparse.ArgumentParser(description=__doc__)
768 parser.add_argument(
769     "--sample",
770     type=Path,
771     default=DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet",
772 )
773 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
774 parser.add_argument(
775     "--products-extra",
776     type=Path,
777     help="Annual flavor metadata; defaults to RMS/combined/products_extra.parquet beside the RMS data directory.",
778 )
779 parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results/estimation_split"))
```

Read this block:

- Line 767, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 768: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 773: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 774: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 779: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 780–793:

```
780 parser.add_argument(
781     "--model",
782     choices=["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
    ↪ "heterogeneous_history"],
```

```
    default=default_model,
)
parser.add_argument("--first-year", type=int, default=2006)
parser.add_argument("--last-year", type=int, default=2015)
parser.add_argument("--max-households", type=int)
parser.add_argument(
    "--capacity",
    type=int,
    default=0,
    help="0: conditional flavor demand; 7: outside=max(7-Q,0)",
)
```

Read this block:

- Line 780: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 785: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 786: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 787: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 788: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 794–806:

```
parser.add_argument(
    "--no-price-control",
    action="store_true",
    default=default_no_price_control,
    help="Disable the price control using the overall other-DMA mean-price IV",
)
parser.add_argument("--household-dma-column", default="dma_cd")
parser.add_argument(
    "--se-method", choices=["hessian", "bootstrap", "none"], default="hessian"
)
parser.add_argument("--bootstrap-reps", type=int, default=49)
parser.add_argument("--bootstrap-starts", type=int, default=2)
parser.add_argument("--habit-weight", type=float, default=0.7)
```

Read this block:

- Line 794: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 800: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 801: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 804: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 805: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 806: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 807–811:

```
parser.add_argument("--draws", type=int, default=128)
parser.add_argument("--starts", type=int, default=3)
parser.add_argument("--maxiter", type=int, default=600)
```

```
parser.add_argument("--seed", type=int, default=306)
args = parser.parse_args()
```

Read this block:

- Line 807: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 808: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 809: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 810: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 811, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 812–813:

```
if not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1:
    parser.error("Check habit weight, capacity, and starts.")
```

Read this block:

- Line 812: test `not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 813: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 814–817:

```
if args.first_year > args.last_year or (
    args.max_households is not None and args.max_households < 1
):
    parser.error("Check years and household limit.")
```

Read this block:

- Line 814: test `args.first_year > args.last_year or (args.max_households is not None and args....`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 817: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 818–819:

```
if args.draws < 2 or args.draws % 2 or args.maxiter < 1:
    parser.error("Use an even draw count >= 2 and maxiter >= 1.")
```

Read this block:

- Line 818: test `args.draws < 2 or args.draws % 2 or args.maxiter < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 819: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 820–823:

```
820 if args.se_method == "bootstrap" and (
821     args.bootstrap_reps < 20 or args.bootstrap_starts < 1
822 ):
823     parser.error("Bootstrap requires at least 20 replications and one start.")
```

Read this block:

- Line 820: test `args.se_method == 'bootstrap' and (args.bootstrap_reps < 20 or args.bootstrap...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 823: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 824–824:

```
824 frame = prepare_data(args)
```

Read this block:

- Line 824, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 825–829:

```
825 if not args.no_price_control:
826     frame, first_stage = add_price_controls(frame)
827     args.output.mkdir(parents=True, exist_ok=True)
828     first_stage.to_csv(args.output / "first_stage.csv", index=False)
829     print(first_stage.to_string(index=False), flush=True)
```

Read this block:

- Line 825: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 826, `(frame, first_stage)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 827: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 828: call `first_stage.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 829: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 830–832:

```
830 args.output.mkdir(parents=True, exist_ok=True)
831 frame.to_csv(args.output / "weekly_inputs.csv", index=False)
832 fit_model(make_arrays(frame, args.capacity), args, frame)
```

Read this block:

- Line 830: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 831: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 832: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/est_gamma.py

This wrapper selects the `homogeneous` default, meaning common alpha and common unrestricted gamma. It imports and calls the shared `main`; it does not define a function locally. The name of a script is not itself a Python function or an instruction to fit a different formula unless its actual call says so.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta, alpha, and flavor-specific gamma. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("homogeneous")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_het_gamma.py

This wrapper supplies the `heterogeneous` default to shared `main`. Both alpha and gamma vary by persistent type. Parentheses call the imported function; writing just `main` would retrieve the function object without executing it.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta; heterogeneous gamma AND alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_het_history.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Independent normal novelty and own-stock coefficients; random alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous_history", default_no_price_control=False)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__het__naive.py

This guarded wrapper supplies the `mixed_naive` default. The common estimator then includes price heterogeneity while excluding gamma. The small file is a convenient command name and shares all data, optimizer, and inference options with the main script.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta, heterogeneous alpha, and no gamma. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("mixed_naive")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__mixture__gamma.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Two-normal gamma mixture; shared lognormal alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("mixture")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__naive.py

This wrapper imports `main` and calls it with the `naive` default only under the entry-point guard. The wrapper contains no new estimation algorithm. `--model` can still override the supplied default because the common parser processes the user's options. Reusing one main function avoids four copies of data preparation and CLI validation.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta/alpha and no history effect. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("naive")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_no_price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Original heterogeneous gamma model without the price control function. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous", default_no_price_control=True)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_separate_history.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Separate common novelty and own-stock coefficients; random alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("separate_history", default_no_price_control=False)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/hessian_se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 """Observed-information inference, conditional on first-stage fits."""
2 import numpy as np
3 import pandas as pd
4 from scipy.stats import norm
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 4: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.

observed covariance

Original function: `observed_covariance`. Source begins at line 7.

Central-differences the analytic mean-household NLL gradient to form observed information, checks bounds and eigenvalues, and divides its inverse by the household count. Missing household scaling would give incorrect standard errors. Nonpositive or ill-conditioned information is rejected instead of inverted blindly.

Definition and stated intent:

Source lines 7–8:

```
7 def observed_covariance(theta, gradient, households, bounds):
8     """Differentiate the mean negative-log-likelihood gradient, then scale by I."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 9–11:

```
9 theta = np.asarray(theta, dtype=float)
10 steps = np.cbrt(np.finfo(float).eps) * np.maximum(1., np.abs(theta))
11 lower, upper = np.asarray(bounds, dtype=float).T
```

Read this block:

- Line 9, `theta`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 10, `steps`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 11, `(lower, upper)`: Unpack the attribute `np.asarray(bounds, dtype=float).T` of the object before the dot into these targets in order.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 12–13:

```
12 if np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 1e-4)):
13     raise ValueError("parameter at or near bounds; normal inference withheld")
```

Read this block:

- Line 12: test `np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 0.0001))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 13: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 14–14:

```
14 hessian = np.empty((len(theta), len(theta)))
```

Read this block:

- Line 14, `hessian`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 15–18:

```
15 for j, step in enumerate(steps):
16     delta = np.zeros(len(theta))
17     delta[j] = step
18     hessian[:, j] = (gradient(theta + delta) - gradient(theta - delta)) / (2 * step)
```

Read this block:

- Line 15: for each entry yielded by `enumerate(steps)`, bind/unpack it into `(j, step)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `delta`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 17, `delta[j]`: Store the object currently named `step`; plain assignment does not copy it in the selected entry/column.
- Line 18, `hessian[:, j]`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–19:

```
19 hessian = (hessian + hessian.T) / 2
```

Read this block:

- Line 19, `hessian`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 20–21:

```
20 if not np.isfinite(hessian).all():
21     raise ValueError("nonfinite observed information; SEs withheld")
```

Read this block:

- Line 20: test `not np.isfinite(hessian).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 21: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 22–22:

```
22 eigenvalues = np.linalg.eigvalsh(hessian)
```

Read this block:

- Line 22, `eigenvalues`: Set this name to the result of calling `np.linalg.eigvalsh(...)`; the arguments and preceding method chain are shown above.

Source lines 23–24:

```
23 if eigenvalues[0] <= max(0., eigenvalues[-1] * 1e-10):
24     raise ValueError("observed information not positive definite or ill-conditioned; SEs withheld")
```

Read this block:

- Line 23: test `eigenvalues[0] <= max(0.0, eigenvalues[-1] * 1e-10)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 24: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 25–25:

```
25 return np.linalg.solve(hessian, np.eye(len(theta))) / households
```

Read this block:

- Line 25: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

hessian standard errors

Original function: `hessian_standard_errors`. Source begins at line 28.

Converts an accepted observed-information covariance into parameter SEs and normal intervals, writes the covariance, and returns inference metadata. These are conditional model-based SEs; they do not include price-first-stage uncertainty or provide a robust sandwich covariance.

Definition and stated intent:

Source lines 28–28:

```
28 def hessian_standard_errors(theta, gradient, households, bounds, names, output):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`, `names`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–31:

```
29 report = dict(method="observed-information Hessian",
30               conditional_on="estimated price controls, retail instruments, and fixed integration draws",
31               first_stage_uncertainty="not included", robust=False)
```

Read this block:

- Line 29, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 32–36:

```
32 try:
33     covariance = observed_covariance(theta, gradient, households, bounds)
34 except ValueError as error:
35     report["status"] = str(error)
36     return None, report
```

Read this block:

- Line 32: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 33, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 34: catch `ValueError` from the preceding try; the optional as-name holds the error object for logging.
- Line 35, `report['status']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 36: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 37–42:

```
37 pd.DataFrame(covariance, index=names, columns=names).to_csv(output / "covariance.csv")
38 se = np.sqrt(np.diag(covariance))
39 critical = norm.ppf(.975)
40 table = pd.DataFrame(dict(parameter=names, std_error=se,
41                           ci_lower=theta - critical * se, ci_upper=theta + critical * se))
42 report["status"] = "computed; normal 95% intervals; first-stage uncertainty omitted"
```

Read this block:

- Line 37: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 39, `critical`: Set this name to the result of calling `norm.ppf(...)`; the arguments and preceding method chain are shown above.
- Line 40, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 42, `report['status']`: Store the literal text `'computed; normal 95% intervals; first-stage uncertainty omitted'` in the selected entry/column.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 43–43:

```
43 return table, report
```

Read this block:

- Line 43: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est__code/history__distributions.py

Saved-fit distribution exporter. Read this when you want the two estimated population marginals without rerunning optimization. It reads fit metadata and coefficients, not household histories. The zero-SD case is a point mass. The reusable function is called automatically after a split fit.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Export fitted population history distributions, including from saved fits."""
2 from pathlib import Path
3 import os
4 import tempfile
5 from statistics import NormalDist
6 import argparse
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: from `statistics`, import `NormalDist` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `argparse`. An as-name is a convenient local alias, not another library.

Source lines 7–9:

```
7 import json
8 import numpy as np
9 import pandas as pd
```

Read this block:

- Line 7: import `json`. An as-name is a convenient local alias, not another library.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 55–64:

```
55 if __name__ == '__main__':
56     parser = argparse.ArgumentParser(description=__doc__)
57     parser.add_argument('--fit-dir', required=True, type=Path)
58     parser.add_argument('--output', type=Path)
59     args = parser.parse_args()
60     fit = json.loads((args.fit_dir/'fit.json').read_text())
61     if fit['model'] not in ['separate_history', 'heterogeneous_history']:
62         parser.error('Requires a split-history fit')
63     export_history_distributions(pd.read_csv(args.fit_dir/'point_estimates.csv'),
64                                  args.output or args.fit_dir, fit.get('converged', False))
```

Read this block:

- Line 55: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 60, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 61: test `fit['model'] not in ['separate_history', 'heterogeneous_history']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 63: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

export history distributions

Original function: `export_history_distributions`. Source begins at line 12.

Inputs and outputs. The coefficient table supplies mean_novelty, mean_own_stock and their SDs; output is a directory. For each marginal, the function writes a summary of mean, SD, sign share and quantiles, a grid of densities/CDFs, and PDF/PNG plots. These describe population heterogeneity, not confidence intervals or household posterior means.

`NormalDist(mean, sd)` represents a fitted normal distribution; `.cdf(0)` gives its share below zero and `.inv_cdf(.25)` gives its lower quartile. A zero SD is handled as a point mass instead of dividing by zero. A negative own-stock coefficient means satiation, while a positive one means reinforcement. Convergence/boundary status marks provisional exports. This calculation recovers the already estimated distribution without optimizing demand again.

Definition and stated intent:

Source lines 12–12:

```
12 def export_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 p = table.set_index('parameter').estimate
```

Read this block:

- Line 13, `p`: Set this name to the attribute `table.set_index('parameter').estimate` of the object before the dot.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 14–15:

```
14 if 'mean_novelty' not in p:
15     return
```

Read this block:

- Line 14: test `'mean_novelty' not in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 15: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 16–21:

```
16 output = Path(output)
17 output.mkdir(parents=True, exist_ok=True)
18 provisional = not converged or table.get('at_bound', pd.Series(False)).astype(str).str.lower().eq('true').any()
19 summaries, curves = [], []
20 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
21 import matplotlib
```

Read this block:

- Line 16, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 18, `provisional`: Set this name to the Boolean condition `not converged or table.get('at_bound', pd.Series(False)).astyp...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 19, `(summaries, curves)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 20: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 21: import `matplotlib`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 22–24:

```
22 matplotlib.use('Agg')
23 import matplotlib.pyplot as plt
24 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

Read this block:

- Line 22: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 24, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 25–45:

```
25 for ax, term, label in zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'Own-stock habit']):
26     mean, sd = float(p['mean_'+term]), float(p.get('sd_'+term, 0.))
27     if not np.isfinite([mean, sd]).all() or sd < 0:
28         raise ValueError('Invalid distribution parameters')
29     normal = NormalDist(mean, sd) if sd > 0 else None
30     row = dict(coefficient=term, mean=mean, sd=sd, point_mass=sd == 0,
31                share_below_zero=normal.cdf(0) if normal else float(mean < 0), provisional=bool(provisional))
32     for q in [.01, .05, .25, .5, .75, .95, .99]:
33         row[f'q{int(100*q):02d}'] = normal.inv_cdf(q) if normal else mean
34     summaries.append(row)
35     if normal:
36         x = np.linspace(mean-4*sd, mean+4*sd, 801)
37         density = np.exp(-.5*((x-mean)/sd)**2)/(sd*np.sqrt(2*np.pi))
38         ax.plot(x, density)
39         curves.extend(dict(coefficient=term, value=float(v), density=float(d), cdf=normal.cdf(float(v)),
    ↪ point_mass=False)
40                       for v, d in zip(x, density))
41     else:
42         ax.axvline(mean, label='Point mass')
43         curves.append(dict(coefficient=term, value=mean, density=np.nan, cdf=1., point_mass=True))
44     ax.axvline(0, color='gray', ls='--')
45     ax.set(title=f'{label}\nMean={mean:.4g}; SD={sd:.4g}', xlabel='Utility coefficient', ylabel='Population
    ↪ density')
```

Read this block:

- Line 25: for each entry yielded by `zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'O...`, bind/unpack it into `(ax, term, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 26, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.

- Line 27: test `not np.isfinite([mean, sd]).all() or sd < 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 29, `normal`: Set this name to the true-branch value if `sd > 0` holds, otherwise the else-branch value.
- Line 30, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 32: for each entry yielded by `[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]`, bind/unpack it into `q` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 33, `row[f'q{int(100 * q):02d}']`: Store the true-branch value if `normal` holds, otherwise the else-branch value in the selected entry/column.
- Line 34: call `summaries.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: test `normal`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 36, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 37, `density`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 38: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39: call `curves.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `curves.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `NormalDist`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another

expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 46–49:

```
46 pd.DataFrame(summaries).to_csv(output/'history_distribution_summary.csv', index=False)
47 pd.DataFrame(curves).to_csv(output/'history_distribution_grid.csv', index=False)
48 fig.suptitle('Fitted population history distributions'+(' (provisional)' if provisional else ''))
49 fig.tight_layout()
```

Read this block:

- Line 46: call `pd.DataFrame(summaries).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47: call `pd.DataFrame(curves).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 50–51:

```
50 for extension in ['pdf', 'png']:
51     fig.savefig(output/f'history_distributions.{extension}')
```

Read this block:

- Line 50: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 51: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 52–52:

```
52 plt.close(fig)
```

Read this block:

- Line 52: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

est_code/price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–7:

```
"""Other-DMA instruments and the price first stage. See README.md."""
import numpy as np
import pandas as pd

from scipy.stats import t as student_t

FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.stats`, import `t as student_t` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

check dma

Original function: `check_dma`. Source begins at line 10.

This converts a pandas Series to integer DMA IDs, rejecting missing, nonnumeric, nonpositive, and fractional entries. `pd.to_numeric(errors="coerce")` turns bad strings into missing numbers so one Boolean mask can find all invalid rows. `codes == np.floor(codes)` checks integrality without rounding fractional inputs into a valid ID. Only after validation does `astype("int64")` convert the dtype. `description` makes an error identify the consumer or retail source. Quietly replacing an invalid DMA with zero would contaminate own-market exclusion.

Definition and stated intent:

Source lines 10–11:

```
def check_dma(values, description):
    """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
```

`def` creates this callable; the parentheses name its inputs (`values`, `description`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–13:

```
codes = pd.to_numeric(values, errors="coerce")
valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
```

Read this block:

- Line 12, `codes`: Set this name to the result of calling `pd.to_numeric(...)`; the arguments and preceding method chain are shown above.
- Line 13, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 14–15:

```
14 if not valid.all():
15     raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
```

Read this block:

- Line 14: test `not valid.all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 15: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 16–16:

```
16 return codes.astype("int64")
```

Read this block:

- Line 16: return the result of calling `codes.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

other dma instruments

Original function: `other_dma_instruments`. Source begins at line 19.

This retained helper accepts three pandas tables and returns weekly rows with UPC-matched flavor IVs. It is **not** the pooled instrument construction used by current `est.py`. It first averages retail-DMA prices outside the consumer's DMA per UPC/week, then averages across the exposure store's UPC basket.

Renaming the own-DMA table's columns makes its keys match the merge. `present` is a Boolean Series; converting it to integers subtracts one from the DMA count only when that own-market quote exists. The grouped `count` ignores missing values whereas `size` counts all rows. Comparing them detects an incompletely instrumented basket and marks the whole flavor IV missing, rather than quietly averaging only a subset. Several joins replace a much slower nested lookup loop.

Definition and stated intent:

Source lines 19–25:

```
19 def other_dma_instruments(weekly, exposure_upcs, dma_prices):
20     """Match each exposure-store UPC to its same-week price in OTHER DMAs.
21
22     dma_prices has ONE mean package price per UPC/week/retail DMA. First average
23     these means over all DMAs except the CONSUMER's DMA, then average over the
24     exposure store's UPCs in a flavor. This matches the local price's UPC basket.
25     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `exposure_upcs`, `dma_prices`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–33:

```
dma_prices = dma_prices.copy()
dma_prices["dma_code"] = check_dma(dma_prices["dma_code"], "retail")
pk = ["upc", "week"]
totals = (dma_prices.groupby(pk)["dma_price"].agg(["sum", "count"])
          .rename(columns={"sum": "sum_dma", "count": "n_dma"}).reset_index())
exposure = weekly[["store_code_uc", "week", "dma_cd"]].drop_duplicates()
exposure = exposure.merge(exposure_upcs, on=["store_code_uc", "week"],
                          how="left", validate="many_to_many")
```

Read this block:

- Line 26, `dma_prices`: Set this name to the result of calling `dma_prices.copy(...)`; the arguments and preceding method chain are shown above.
- Line 27, `dma_prices['dma_code']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `pk`: Set this name to the list `['upc', 'week']`.
- Line 29, `totals`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 31, `exposure`: Set this name to the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above.
- Line 32, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 34–40:

```
exposure = exposure.merge(totals, on=pk, how="left", validate="many_to_one")
own = dma_prices.rename(columns={"dma_code": "dma_cd", "dma_price": "own_dma_price"})
exposure = exposure.merge(own, on=pk + ["dma_cd"], how="left", validate="many_to_one")
present = exposure["own_dma_price"].notna()
denominator = exposure["n_dma"] - present.astype(int)
exposure["iv"] = ((exposure["sum_dma"] - exposure["own_dma_price"].fillna(0))
                  / denominator.where(denominator > 0))
```

Read this block:

- Line 34, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 35, `own`: Set this name to the result of calling `dma_prices.rename(...)`; the arguments and preceding method chain are shown above.
- Line 36, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

- Line 37, `present`: Set this name to the result of calling `exposure['own_dma_price'].notna(...)`; the arguments and preceding method chain are shown above.
- Line 38, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 39, `exposure['iv']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 42–46:

```
42 keys = ["store_code_uc", "week", "dma_cd", "flavor"]
43 grouped = exposure.groupby(keys)["iv"].agg(["mean", "count", "size"])
44 grouped.loc[grouped["count"] != grouped["size"], "mean"] = np.nan
45 result = (grouped["mean"].unstack("flavor").reindex(columns=FLAVORS)
46           .rename(columns={f: "iv_" + f for f in FLAVORS}).reset_index())
```

Read this block:

- Line 42, `keys`: Set this name to the list `['store_code_uc', 'week', 'dma_cd', 'flavor']`.
- Line 43, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 44, `grouped.loc[grouped['count'] != grouped['size'], 'mean']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 45, `result`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 47–48:

```
47 return weekly.merge(result, on=["store_code_uc", "week", "dma_cd"], how="left",
48                     validate="many_to_one")
```

Read this block:

- Line 47: return the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above. This ends the call.

absorb effects

Original function: `absorb_effects`. Source begins at line 51.

Input `values` is a numerical matrix; `groups` lists row-aligned group labels such as week and DMA. Output is a copy with group means removed. Factorization converts arbitrary labels to dense integers. `np.add.at(means, code, residual)` accumulates rows into group sums even when indices repeat; ordinary fancy-index `means[code] += residual` may not accumulate repeats as intended.

`counts[:, None]` makes group counts a column so every variable is divided by its group size. The function cycles week and DMA demeaning until the biggest absolute change falls below tolerance, then returns. A maximum iteration limit raises on failure. A huge explicit dummy matrix is a valid alternative used in tests, but its memory grows with the number of fixed-effect groups. One pass is insufficient when group cells are unbalanced.

Definition and stated intent:

Source lines 51–56:

```
51 def absorb_effects(values, groups, tolerance=1e-10, maxiter=2000):
52     """Remove week and DMA means without a huge matrix of dummy variables.
53 
54     Repeated demeaning is needed because DMA/week cells are unbalanced.
55     Each household-week exposure has equal weight in the first stage.
56     """
```

`def` creates this callable; the parentheses name its inputs (`values`, `groups`, `tolerance`, `maxiter`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 57–58:

```
57 residual = np.asarray(values, dtype=float).copy()
58 codes = [pd.factorize(group, sort=True)[0] for group in groups]
```

Read this block:

- Line 57, `residual`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 58, `codes`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 59–68:

```
59 for _ in range(maxiter):
60     previous = residual.copy()
61     for code in codes:
62         counts = np.bincount(code)
63         means = np.zeros((len(counts), residual.shape[1]))
64         np.add.at(means, code, residual)
65         means /= counts[:, None]
66         residual -= means[code]
67     if np.max(np.abs(residual - previous)) < tolerance:
68         return residual
```

Read this block:

- Line 59: for each entry yielded by `range(maxiter)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60, `previous`: Set this name to the result of calling `residual.copy(...)`; the arguments and preceding method chain are shown above.
- Line 61: for each entry yielded by `codes`, bind/unpack it into `code` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 62, `counts`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.

- Line 63, `means`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 64: call `np.add.at` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: update `means` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 66: update `residual` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 67: test `np.max(np.abs(residual - previous)) < tolerance`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 68: return the object currently named `residual`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.add.at`: Accumulate in place at possibly repeated indices; this avoids the lost-repeat behavior of buffered fancy-index updates.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 69–69:

```
69 raise ValueError("First-stage fixed-effect demeaning did not converge.")
```

Read this block:

- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

add price controls

Original function: `add_price_controls`. Source begins at line 72.

This returns `(augmented_frame, diagnostic_table)`. For each flavor it selects covered prices, absorbs week/DMA effects jointly from price, instrument, and Xi, then partials Xi out of y and z. `within[:, 0]`, `[:, 1]`, `[:, 2:]` split those roles by known column order. `lstsq(...)[0]` keeps only the coefficient vector from the least-squares result tuple.

Dot products compute the IV slope after partialling. A small residual instrument sum of squares raises instead of dividing by an almost zero denominator. `np.linalg.lstsq` tolerates redundant controls more gracefully than explicitly inverting their cross-product matrix. The residual is assigned only to covered rows; unavailable alternatives retain zero placeholders. Cluster score sums, partial R-squared, and residual SD describe first-stage relevance and fit. They do not prove instrument validity. Reusing this function in bootstrap replications keeps the two-stage estimator consistent across samples.

Definition and stated intent:

Source lines 72–77:

```
72 def add_price_controls(frame):
73     """OLS separately by flavor: price ~ IV + all three Xi + week FE + DMA FE.
74 
75     Refit this function in every household bootstrap sample. Return residuals
76     in DOLLARS (no arbitrary rescaling) for the choice-model control function.
```

```
    """
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–79:

```
result = frame.copy()
reports = []
```

Read this block:

- Line 78, `result`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 79, `reports`: Set this name to the list `[]`.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 80–119:

```
for flavor in FLAVORS:
    columns = ["price_" + flavor, "iv_" + flavor] + ["xi_" + f for f in FLAVORS]
    covered = np.isfinite(result["price_" + flavor]) & (result["price_" + flavor] > 0)
    sample = result.loc[covered]
    if sample.empty:
        raise ValueError(f"No covered prices for the {flavor} first stage.")
    hh, households = pd.factorize(sample["household_code"], sort=True)
    values = sample[columns].to_numpy(float)
    if not np.isfinite(values).all():
        raise ValueError(f"Nonfinite first-stage inputs for {flavor}.")
    within = absorb_effects(values, [sample["week"], sample["dma_cd"]])
    y, z, controls = within[:, 0], within[:, 1], within[:, 2:]
    # Partial out Xi before estimating the excluded instrument's coefficient.
    # This tolerates redundant Xi columns, but NOT a redundant instrument.
    z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
    y_partial = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
    z_ss = z_partial @ z_partial
    if z_ss <= 1e-12 * max(1., values[:, 1] @ values[:, 1]):
        raise ValueError(f"No excluded-instrument variation after controls for {flavor}.")
    pi = (z_partial @ y_partial) / z_ss
    control_beta = np.linalg.lstsq(controls, y - pi*z, rcond=None)[0]
    residual = y - pi*z - controls @ control_beta
    if np.std(residual) < 1e-10:
        raise ValueError(f"First-stage residual for {flavor} is essentially zero.")
    result["cf_" + flavor] = 0.
    result.loc[covered, "cf_" + flavor] = residual
    # Household-clustered partial-IV t/F diagnostic (not a weak-IV-robust test).
    scores = np.bincount(hh, weights=z_partial * residual, minlength=len(households))
    pi_se = np.sqrt((scores @ scores) / z_ss**2)
    if len(households) > 1:
        pi_se *= np.sqrt(len(households) / (len(households) - 1))
    f_stat = float((pi / pi_se)**2) if pi_se > 0 else np.nan
    restricted_ss = y_partial @ y_partial
    partial_r2 = 1 - (residual @ residual) / restricted_ss if restricted_ss > 0 else np.nan
    reports.append(dict(flavor=flavor, iv_coefficient=float(pi),
                        iv_cluster_se=float(pi_se), iv_cluster_F=f_stat,
                        iv_pvalue=float(2*student_t.sf(np.sqrt(f_stat), len(households)-1))
                        if len(households) > 1 else np.nan,
                        partial_R2=float(partial_r2), residual_sd=float(np.std(residual)),
                        observations=len(sample), households=len(households)))
```

Read this block:

- Line 80: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `covered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `sample`: Set this name to the entry/slice selected by `result.loc[covered]`; brackets select from the object to their left.
- Line 84: test `sample.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 86, `(hh, households)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 87, `values`: Set this name to the result of calling `sample[columns].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 88: test `not np.isfinite(values).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 89: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 90, `within`: Set this name to the result of calling `absorb_effects(...)`; the arguments and preceding method chain are shown above.
- Line 91, `(y, z, controls)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 94, `z_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 95, `y_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 96, `z_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97: test `z_ss <= 1e-12 * max(1.0, values[:, 1] @ values[:, 1])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 98: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 99, `pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 100, `control_beta`: Set this name to the entry/slice selected by `np.linalg.lstsq(controls, y - pi * z, rcond=None)[0]`; brackets select from the object to their left.
- Line 101, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 102: test `np.std(residual) < 1e-10`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 103: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 104, `result['cf_' + flavor]`: Store the literal value `0.0` in the selected entry/column.
- Line 105, `result.loc[covered, 'cf_' + flavor]`: Store the object currently named `residual`; plain assignment does not copy it in the selected entry/column.
- Line 107, `scores`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 108, `pi_se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 109: test `len(households) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 110: update `pi_se` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 111, `f_stat`: Set this name to the true-branch value if `pi_se > 0` holds, otherwise the else-branch value.
- Line 112, `restricted_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 113, `partial_r2`: Set this name to the true-branch value if `restricted_ss > 0` holds, otherwise the else-branch value.
- Line 114: call `reports.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `len`: Count elements/rows rather than summing their numerical values.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.sf`: Evaluate survival probability above a threshold, useful for a two-sided t diagnostic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 120–120:

```
120 return result, pd.DataFrame(reports)
```

Read this block:

- Line 120: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est_code/regenerate_weekly_inputs.py

Server-only workflow for the deleted household export. Saved settings reconstruct the original sample/history/menu table and optional first stages. Exact household/week totals and likelihood agreement are required before writing weekly_inputs.csv. Demand estimates and inference are preserved.

Module setup and direct-execution blocks

Source lines 1–8:

```
"""Regenerate weekly inputs on the server without refitting saved demand parameters."""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server, require_data_path
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 4: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 6, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: from `server_data_policy`, import `server_only, require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 9–15:

```
import argparse
import json
from pathlib import Path
import tempfile
from types import SimpleNamespace

import numpy as np
```

Read this block:

- Line 9: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 13: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 16–19:

```
import pandas as pd

from est import prepare_data, make_arrays, make_draws, parameter_names, household_loglike
from price_control import add_price_controls
```

Read this block:

- Line 16: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 18: from `est`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 19: from `price_control`, import `add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 89–90:

```
89 if __name__ == '__main__':
90     main()
```

Read this block:

- Line 89: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 90: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

regenerate

Original function: `regenerate`. Source begins at line 23.

Job and contract. Rebuild the deleted weekly_inputs.csv on the server using the existing fit directory, optionally overriding moved sample/reference paths. The function reads saved settings, calls the same sample/history/menu preparation, and reconstructs price controls. It never calls the demand optimizer or replaces fitted parameters and inference.

`SimpleNamespace(**settings)` turns dictionary keys into ordinary attributes. Converting saved path strings back to Path objects restores the input contract. The reconstructed table must match household/week counts and the saved integrated likelihood, using the exact parameter order, draw count and seed. A mismatch raises before publishing a CSV.

`NamedTemporaryFile` creates a private intermediate beside the destination; `temporary.replace(target)` publishes only a verified complete export. The finally block cleans up an unfinished intermediate. The JSON regeneration record stores diagnostics, not household rows. Run this entry point on the server; reading the textbook does not execute it.

Definition and stated intent:

Source lines 23–23:

```
23 def regenerate(fit_dir, sample=None, flavors=None, weekly_output=None):
```

`def` creates this callable; the parentheses name its inputs (`fit_dir`, `sample`, `flavors`, `weekly_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–29:

```
24 fit_dir = Path(fit_dir)
25 fit = json.loads((fit_dir / 'fit.json').read_text())
26 settings = dict(fit['settings'])
27 required = ['sample', 'first_year', 'last_year', 'max_households', 'capacity',
28             'no_price_control', 'household_dma_column', 'habit_weight', 'draws', 'seed']
29 missing = set(required)-settings.keys()
```

Read this block:

- Line 24, `fit_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 25, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 26, `settings`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 27, `required`: Set this name to a list containing 10 displayed entries.
- Line 29, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 30–31:

```
30 if missing:
31     raise ValueError(f'Saved fit lacks reconstruction settings: {sorted(missing)}')
```

Read this block:

- Line 30: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 31: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 32–33:

```
32 settings['model'] = fit['model']
33 settings['output'] = Path(weekly_output).parent if weekly_output is not None else
   ↪  Path('/scratch/dtm63837/Kilts_Panel/Data/temporary') / fit_dir.name
```

Read this block:

- Line 32, `settings['model']`: Store the entry/slice selected by `fit['model']`; brackets select from the object to their left in the selected entry/column.
- Line 33, `settings['output']`: Store the true-branch value if `weekly_output is not None` holds, otherwise the else-branch value in the selected entry/column.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 34–38:

```
34 for key in ['sample', 'flavors', 'products_extra']:
35     if settings.get(key) is not None:
36         settings[key] = Path(settings[key])
37     else:
38         settings[key] = None
```

Read this block:

- Line 34: for each entry yielded by `['sample', 'flavors', 'products_extra']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 35: test `settings.get(key) is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 36, `settings[key]`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 38, `settings[key]`: Store the literal value `None` in the selected entry/column.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 39–40:

```
39 if sample is not None:
40     settings['sample'] = Path(sample)
```

Read this block:

- Line 39: test `sample is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 40, `settings['sample']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 41–42:

```
41 if flavors is not None:
42     settings['flavors'] = Path(flavors)
```

Read this block:

- Line 41: test `flavors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42, `settings['flavors']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 43–44:

```
43 args = SimpleNamespace(**settings)
44 frame = prepare_data(args)
```

Read this block:

- Line 43, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 44, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 45–47:

```
45 if not args.no_price_control:
46     # Reconstruct the original first stage; preserve saved inference artifacts.
47     frame, _ = add_price_controls(frame)
```

Read this block:

- Line 45: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 47, `(frame, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 48–48:

```
48 data = make_arrays(frame, args.capacity)
```

Read this block:

- Line 48, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 49–50:

```
49 if len(frame) != fit['weeks'] or data['n_households'] != fit['households']:
50     raise ValueError('Regenerated household/week totals do not match the saved fit; no CSV written.')
```

Read this block:

- Line 49: test `len(frame) != fit['weeks'] or data['n_households'] != fit['households']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 50: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 51–56:

```
51 estimates = pd.read_csv(fit_dir / 'point_estimates.csv')
52 parameters = estimates.set_index('parameter').estimate
53 names = parameter_names(args.model, data['outside'], data['control'])
54 theta = np.array([parameters[name] for name in names])
55 draws = make_draws(args.draws, args.seed, args.model)
56 nll = -float(household_loglike(theta, data, draws, args.model).sum())
```

Read this block:

- Line 51, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 52, `parameters`: Set this name to the attribute `estimates.set_index('parameter').estimate` of the object before the dot.
- Line 53, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 54, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 55, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 56, `nll`: Set this name to the negated or sign-adjusted expression `-float(household_loglike(theta, data, draws, args.model).sum())`.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 57–58:

```
57 if not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-9, atol=1e-4):
58     raise ValueError(f'Regenerated likelihood {nll} differs from saved {fit["nll"]}; no CSV written.')
```

Read this block:

- Line 57: test `not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 58: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 59–61:

```
59 target = Path(weekly_output) if weekly_output is not None else Path('/scratch/dtm63837/Kilts_Panel/Data/temporary')
   ↪  / fit_dir.name / 'weekly_inputs.csv'
60 require_data_path(target)
61 target.parent.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 59, `target`: Set this name to the true-branch value if `weekly_output is not None` holds, otherwise the else-branch value.
- Line 60: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 61: call `target.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 63–64:

```
63 with tempfile.NamedTemporaryFile(dir=target.parent, prefix='.weekly_inputs_', suffix='.csv', delete=False) as
   ↪  handle:
64     temporary = Path(handle.name)
```

Read this block:

- Line 63: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 64, `temporary`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Source lines 65–69:

```
65 try:
66     frame.to_csv(temporary, index=False)
67     temporary.replace(target)
68 finally:
69     temporary.unlink(missing_ok=True)
```

Read this block:

- Line 65: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 66: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 67: call `temporary.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 69: call `temporary.unlink` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 70–74:

```
70 report = dict(households=data['n_households'], weeks=len(frame), reconstructed_nll=nll,
71              saved_nll=fit['nll'], model=args.model, sample=str(args.sample),
72              demand_refitted=False, price_control_refitted=not args.no_price_control)
73 (fit_dir / 'weekly_inputs_regeneration.json').write_text(json.dumps(report, indent=2)+'\n')
74 print(f'Regenerated {target}; household/week totals and saved likelihood verified.', flush=True)
```

Read this block:

- Line 70, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 73: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 75–75:

```
75 return report
```

Read this block:

- Line 75: return the object currently named `report`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 79.

Parses a fit directory and optional moved input paths, then calls server-side reconstruction. Missing required paths stop before processing purchases. No optimizer is invoked.

Definition and stated intent:

Source lines 79–79:

```
79 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 80–85:

```
80 parser = argparse.ArgumentParser(description=__doc__)
81 parser.add_argument('--fit-dir', type=Path, required=True)
82 parser.add_argument('--sample', type=Path, help='Override saved prepared-sample path if moved.')
83 parser.add_argument('--flavors', type=Path, help='Override saved flavor-reference path if moved.')
84 parser.add_argument('--weekly-output', type=Path, help='Temporary weekly CSV outside the code checkout.')
85 args = parser.parse_args()
```

Read this block:

- Line 80, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 86–86:

```
86 regenerate(args.fit_dir, args.sample, args.flavors, args.weekly_output)
```

Read this block:

- Line 86: call `regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `regenerate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/wtp.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Population WTP moments under independent gamma and lognormal -alpha."""
2 import numpy as np
3 import pandas as pd
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.

implied wtp

Original function: `implied_wtp`. Source begins at line 6.

Computes population moments of coefficient/positive price sensitivity. Independence permits multiplying the coefficient moments by inverse-lognormal moments. Averaging these household ratios differs from dividing a mean coefficient by mean price sensitivity. Split history rows use their own coefficients without lambda weights.

Definition and stated intent:

Source lines 6–7:

```
6 def implied_wtp(estimates, habit_weight=.7):
7     """Return population moments, not sampling standard errors, in price units."""
```

`def` creates this callable; the parentheses name its inputs (`estimates`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 8–12:

```
8 p = dict(zip(estimates.parameter, estimates.estimate))
9 mu, sigma = p['mu_log_minus_alpha'], p.get('sd_log_minus_alpha', 0.)
10 inverse_mean = np.exp(-mu + sigma**2 / 2)
11 inverse_second = np.exp(-2 * mu + 2 * sigma**2)
12 rows = []
```

Read this block:

- Line 8, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 9, `(mu, sigma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 10, `inverse_mean`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 11, `inverse_second`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 12, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 14–18:

```
14 def add(term, mean, second, interpretation):
15     wtp_mean = mean * inverse_mean
16     variance = max(0., second * inverse_second - wtp_mean**2)
17     rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
18                      interpretation=interpretation))
```

Read this block:

- Line 14: define nested callable `add`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 20–24:

```
20 for name in ['inside_constant', 'beta_berry', 'beta_plain']:
21     if name in p:
22         interpretation = ('Other flavor vs outside, at zero history and control residual'
23                           if name == 'inside_constant' else 'Flavor premium relative to other')
24         add(name, p[name], p[name]**2, interpretation)
```

Read this block:

- Line 20: for each entry yielded by `['inside_constant', 'beta_berry', 'beta_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 21: test `name in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22, `interpretation`: Set this name to the true-branch value if `name == 'inside_constant'` holds, otherwise the else-branch value.
- Line 24: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 25–39:

```
25 if 'mean_gamma' in p:
26     mean, sd = p['mean_gamma'], p.get('sd_gamma', 0.)
27     second = mean**2 + sd**2
28     if 'mixture_weight' in p:
29         weight = p['mixture_weight']
30         mean2 = mean + p['gamma_mean_gap']
31         second2 = mean2**2 + p['sd_gamma_2']**2
32         add('gamma_component_1', mean, second, 'Per unit Xi, conditional on component 1')
33         add('gamma_component_2', mean2, second2, 'Per unit Xi, conditional on component 2')
34         mean, second = weight * mean + (1 - weight) * mean2, weight * second + (1 - weight) * second2
35     add('gamma', mean, second, 'Per unit increase in history index Xi')
36     add('gamma_novelty', (1 - habit_weight) * mean, (1 - habit_weight)**2 * second,
37         'Per unit novelty N, holding own-flavor stock fixed')
38     add('gamma_own_stock', -habit_weight * mean, habit_weight**2 * second,
39         'Per unit own-flavor stock S, holding novelty fixed')
```

Read this block:

- Line 25: test `'mean_gamma' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 27, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28: test `'mixture_weight' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 29, `weight`: Set this name to the entry/slice selected by `p['mixture_weight']`; brackets select from the object to their left.
- Line 30, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 31, `second2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 32: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34, `(mean, second)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 35: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 40–44:

```
40 for term in ["novelty", "own_stock"]:
41     if "mean_" + term in p:
42         mean, sd = p["mean_" + term], p.get("sd_" + term, 0.)
43         add(term, mean, mean**2 + sd**2,
44             "Per unit " + term + ", holding the other history measure fixed")
```

Read this block:

- Line 40: for each entry yielded by `['novelty', 'own_stock']`, bind/unpack it into `term` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 41: test `'mean_' + term in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 43: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 45–45:

```
45 return pd.DataFrame(rows)
```

Read this block:

- Line 45: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

implied wtp.add

Original function: `implied_wtp.add`. Source begins at line 14.

This nested helper adds one named WTP row from coefficient first and second moments. It uses variables captured from the enclosing WTP call, converts coefficient moments into ratio moments, and appends to the

enclosing output list. Its side effect is a new row; it does not refit coefficients.

Definition and stated intent:

Source lines 14–14:

```
14 def add(term, mean, second, interpretation):
```

`def` creates this callable; the parentheses name its inputs (`term`, `mean`, `second`, `interpretation`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–18:

```
15 wtp_mean = mean * inverse_mean
16 variance = max(0., second * inverse_second - wtp_mean**2)
17 rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
18                  interpretation=interpretation))
```

Read this block:

- Line 15, `wtp_mean`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 16, `variance`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code: source walkthrough

supply_code/comparative_statics/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/comparative_statics/alpha_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price1_derivative
3 dp1_dalpha = price1_derivative(alpha)
```

Read this block:

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dalpha`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/alpha_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price2_derivative
3 dp2_dalpha = price2_derivative(alpha)
```

Read this block:

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dalpha`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price1_derivative
3 dp1_dbeta = price1_derivative(beta)
```

Read this block:

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dbeta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_p1gamma.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.gamma_p1beta import dp1dbeta_dgamma_clean
```

Read this block:

- Line 1: from `comparative_statics.gamma_p1beta`, import `dp1dbeta_dgamma_clean` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/beta_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import *
2 from comparative_statics.canonical import price1_derivative, price2_derivative
3 ratio_derivative = (p2*price1_derivative(beta)-p1*price2_derivative(beta))/p2**2
```

Read this block:

- Line 1: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

supply_code/comparative_statics/beta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price2_derivative
3 dp2_dbeta = price2_derivative(beta)
```

Read this block:

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dbeta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dbeta
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dPi_dbeta` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/canonical.py

Functions provide generic derivative rules; module-level assignments call them to define named gamma, beta, curvature, and ratio expressions. The quotient-rule ratio is created once with generic terminal symbols, then `.subs({...})` specializes it to matched-inside, matched-outside, and acquisition histories. This keeps three ratio definitions tied to one algebraic rule instead of three independent hand-written implementations.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """All derivatives follow one implicit FOC and the terminal envelope."""
2 import sympy as sp
3 from model.symbols import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 33–38:

```
33 dp1_dgamma = price1_derivative(gamma)
34 dp2_dgamma = price2_derivative(gamma)
35 dVj_dgamma, dV0_dgamma = value2_derivative(gamma,u,-1,p2j), value2_derivative(gamma,v,1,p20)
36 ds_dgamma = sp.factor(sj1*s01*(q+alpha*dp1_dgamma))
37 du_dgamma, dv_dgamma = -u*(1-u)**2, v*(1-v)**2
```

```
38 d2p1_dgamma2 = total_derivative(dp1_dgamma,gamma)
```

Read this block:

- Line 33, `dp1_dgamma`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 34, `dp2_dgamma`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 35, `(dVj_dgamma, dV0_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 36, `ds_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 37, `(du_dgamma, dv_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `d2p1_dgamma2`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 53–58:

```
53 d2p2_dgamma2 = terminal_total_derivative(dp2_dgamma,gamma)
54 ds2_dgamma = sp.factor(sj2*s02*(q2+alpha*dp2_dgamma))
55 dp2dbeta_dgamma = terminal_total_derivative(price2_derivative(beta),gamma)
56 dPi_dgamma = profit_derivative(gamma)
57 dPi_dbeta = profit_derivative(beta)
58 ratio_derivative = (p2*dp1_dgamma-p1*dp2_dgamma)/p2**2
```

Read this block:

- Line 53, `d2p2_dgamma2`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 54, `ds2_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 55, `dp2dbeta_dgamma`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 56, `dPi_dgamma`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 57, `dPi_dbeta`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 58, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `terminal_total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `profit_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 59–61:

```
matched_j = ratio_derivative.subs({q:-1,p2:p2j,sj2:u,q2:-1})
matched_0 = ratio_derivative.subs({q:1,p2:p20,sj2:v,q2:1})
induce_harvest = ratio_derivative.subs({q:1,p2:p2j,sj2:u,q2:-1})
```

Read this block:

- Line 59, `matched_j`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 60, `matched_0`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 61, `induce_harvest`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

price2 derivative

Original function: `price2_derivative`. Source begins at line 6.

This returns a **symbolic expression** for the terminal optimal-price derivative with respect to symbolic parameter theta. Default arguments `sj2`, `q2`, and `p2` are shared symbols; callers substitute a particular branch by passing u/-1/p2j or v/+1/p20. `sp.diff(beta, theta)` is one when theta is beta and zero for an independent parameter, so one generic expression covers several comparative statics.

`direct` is the fixed-price derivative of the utility index. The returned formula already incorporates implicit share/price feedback; `simplify` reduces algebra. Differentiating the markup while holding its share fixed would omit that feedback. Unlike numerical finite differences, symbolic derivatives do not need a step size, though they still depend on a correct symbolic model.

Definition and stated intent:

Source lines 6–6:

```
def price2_derivative(theta, share=sj2, history=q2, price=p2):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

Read this block:

- Line 7, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 8–8:

```
return sp.simplify((1-share)*sp.diff(c, theta) + sp.diff(alpha, theta)/alpha**2 - share*direct/alpha)
```

Read this block:

- Line 8: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

value2 derivative

Original function: `value2_derivative`. Source begins at line 11.

This computes the symbolic envelope derivative of a reoptimized continuation value. It uses the same fixed-price index derivative as terminal pricing, then returns `-share*direct/alpha - share*diff(c,theta)`. There is no explicit derivative of optimal price because the terminal FOC cancels that first-order effect. Omitting price responses here is justified by the envelope theorem, not a general Python or SymPy rule. Substituting fully expanded optimal prices would be harder algebra with the same result under the maintained optimum conditions.

Definition and stated intent:

Source lines 11–11:

```
11 def value2_derivative(theta, share, history, price):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–12:

```
12 direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

Read this block:

- Line 12, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 13–13:

```
13 return -share*direct/alpha - share*sp.diff(c, theta)
```

Read this block:

- Line 13: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

price1 derivative

Original function: `price1_derivative`. Source begins at line 16.

This forms the symbolic first-period price derivative. It builds its direct utility effect, obtains the two terminal envelope derivatives, subtracts them, and inserts the result into the implicit first-price formula. `sp.diff(delta, theta)` includes a direct discount-factor change when theta is delta; for gamma that term is zero but the continuation values still change.

The two helper calls use the correct opposite signed terminal histories. Maintaining a single generic function avoids separate alpha/beta/delta/gamma formulas disagreeing about those branches. `sp.simplify` does algebraic cleanup; it does not decide whether a term is positive under the economic domain.

Definition and stated intent:

Source lines 16–16:

```
16 def price1_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–18:

```
17 direct = sp.diff(beta, theta) + p1*sp.diff(alpha, theta) + q*sp.diff(gamma, theta)
18 dDV = value2_derivative(theta,u,-1,p2j)-value2_derivative(theta,v,1,p20)
```

Read this block:

- Line 17, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 18, `dDV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–20:

```
19 return sp.simplify(s01*sp.diff(c,theta)+sp.diff(alpha,theta)/alpha**2
20                  -sj1*direct/alpha-s01*(sp.diff(delta,theta)*(Vj-V0)+delta*dDV))
```

Read this block:

- Line 19: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

total derivative

Original function: `total_derivative`. Source begins at line 23.

Input is a symbolic expression and differentiation parameter. Output is its total derivative along the solved equilibrium. The first assignments compute price derivatives, then logistic chain rules compute derivatives of the three shares. A dictionary maps each endogenous symbol to its derivative.

The return is explicit partial differentiation plus `sum(diff(expr,k)*val for k,val in substitutions.items())`. This is the chain rule over every mapped endogenous object. The dictionary is a derivative map, not values actually substituted with `.subs` in this function. `sp.factor` collects common algebraic factors. Plain `sp.diff(expr,theta)` would treat shares, prices, and values as independent symbols and miss their indirect changes.

Definition and stated intent:

Source lines 23–23:

```
23 def total_derivative(expr, theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–30:

```
24 dp = price1_derivative(theta)
25 dpj, dp0 = price2_derivative(theta,u,-1,p2j), price2_derivative(theta,v,1,p20)
26 ds = sj1*s01*(sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)+alpha*dp)
27 du = u*(1-u)*(sp.diff(beta,theta)+p2j*sp.diff(alpha,theta)-sp.diff(gamma,theta)+alpha*dpj)
28 dv = v*(1-v)*(sp.diff(beta,theta)+p20*sp.diff(alpha,theta)+sp.diff(gamma,theta)+alpha*dp0)
29 substitutions = {p1:dp, p2j:dpj, p20:dp0, sj1:ds, u:du, v:dv,
30                  Vj:value2_derivative(theta,u,-1,p2j), V0:value2_derivative(theta,v,1,p20)}
```

Read this block:

- Line 24, `dp`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 25, `(dpj, dp0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 26, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27, `du`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `dv`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 29, `substitutions`: Set this name to a dictionary from the displayed keys and values.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return sp.factor(sp.diff(expr,theta)+sum(sp.diff(expr,k)*val for k,val in substitutions.items()))
```

Read this block:

- Line 31: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

terminal total derivative

Original function: `terminal_total_derivative`. Source begins at line 39.

This is the smaller one-branch version of the total derivative. It calculates terminal price response, computes terminal share response including repricing, and adds explicit theta, price, and share chain-rule terms. Because there is only one terminal branch, a short direct expression is clearer than building the full period-1 derivative dictionary. It is used for terminal curvature and mixed derivatives; a second plain symbolic partial derivative would hold the share fixed and give the wrong comparative static.

Definition and stated intent:

Source lines 39–39:

```
39 def terminal_total_derivative(expr,theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 dp = price2_derivative(theta)
41 ds = sj2*s02*(sp.diff(beta,theta)+p2*sp.diff(alpha,theta)+q2*sp.diff(gamma,theta)+alpha*dp)
```

Read this block:

- Line 40, `dp`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 41, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 42–42:

```
42 return sp.factor(sp.diff(expr,theta)+sp.diff(expr,p2)*dp+sp.diff(expr,sj2)*ds)
```

Read this block:

- Line 42: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.

profit derivative

Original function: `profit_derivative`. Source begins at line 45.

This returns the optimized two-period profit derivative with respect to theta. `partial` combines direct current demand/cost effects, direct discounting, and both branch envelope effects. `.subs(p1, markup_expression)` inserts the first-period optimal pricing equation, then `simplify` reduces the result.

The code does not explicitly multiply by dp1/dtheta because the first-period FOC makes that contribution zero at the optimum. Replacing the entire objective with current revenue before differentiating would lose continuation effects. This helper is reusable for gamma and beta, while relying on the same fixed-type, two-branch, interior optimum as the rest of the model.

Definition and stated intent:

Source lines 45–45:

```
45 def profit_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–49:

```
46 direct = sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)
47 partial = (direct*sj1*s01*(p1-c+delta*(Vj-V0))-sj1*sp.diff(c,theta)
48            +sp.diff(delta,theta)*(sj1*Vj+s01*V0)
49            +delta*(sj1*value2_derivative(theta,u,-1,p2j)+s01*value2_derivative(theta,v,1,p20)))
```

Read this block:

- Line 46, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `partial`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 50–50:

```
50 return sp.simplify(partial.subs(p1,c-1/(alpha*s01)-delta*(Vj-V0)))
```

Read this block:

- Line 50: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

supply_code/comparative_statics/delta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price1_derivative
3 dp1_ddelta = price1_derivative(delta)
```

Read this block:

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_ddelta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/delta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price2_derivative
3 dp2_ddelta = price2_derivative(delta)
```

Read this block:

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_ddelta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/gamma2_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p1_dgamma2
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `d2p1_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma2_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p2_dgamma2
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `d2p2_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp1_dgamma, ds_dgamma as dsj1_dgamma_total
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp1_dgamma, ds_dgamma as dsj1_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p1beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 from model.symbols import beta, gamma
2 from comparative_statics.canonical import price1_derivative, total_derivative
3 dp1dbeta_dgamma_raw = total_derivative(price1_derivative(beta), gamma)
4 dp1dbeta_dgamma_clean = dp1dbeta_dgamma_raw
```

Read this block:

- Line 1: from `model.symbols`, import `beta, gamma` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, total_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1dbeta_dgamma_raw`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 4, `dp1dbeta_dgamma_clean`: Set this name to the object currently named `dp1dbeta_dgamma_raw`; plain assignment does not copy it.

Syntax and calls used here:

- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/gamma_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import ratio_derivative, matched_j, matched_0, induce_harvest
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `ratio_derivative, matched_j, matched_0, induce_harvest` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2_dgamma, ds2_dgamma as dsj2_dgamma_total
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp2_dgamma, ds2_dgamma as dsj2_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p2beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2dbeta_dgamma
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp2dbeta_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dgamma
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dPi_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics_symbolic.py

This convenience script imports canonical expressions and, only when run directly, iterates over `(label, expression)` pairs and prints each. Tuple unpacking connects a human-readable label to its actual symbolic object. It does not compute independent derivative formulas. Reusing named expressions keeps terminal display consistent with production numeric evaluation.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Print verified canonical gamma derivatives."""
2 from comparative_statics.canonical import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `comparative_statics.canonical`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 3–6:

```
3 if __name__ == "__main__":
4     for label,expr in [("dp1/dgamma",dp1_dgamma),("d2p1/dgamma2",d2p1_dgamma2),
5                        ("dprofit/dgamma",dPi_dgamma),("ratio derivative",ratio_derivative)]:
6         print(label, "=", expr)
```

Read this block:

- Line 3: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 4: for each entry yielded by `[('dp1/dgamma', dp1_dgamma), ('d2p1/dgamma2', d2p1_dgamma2), (...,` bind/unpack it into `(label, expr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

supply_code/export/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/export/latex_export.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Export the model's actual SymPy expressions as display equations."""
2 import sympy as sp
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.

display equation

Original function: `display_equation`. Source begins at line 5.

This returns a LaTeX display-equation string. `lhs` is already a LaTeX label; `sp.latex(expression)` serializes the actual symbolic right-hand side. `'\\[\n'` contains a backslash followed by a bracket and a newline; Python string escapes are why source contains doubled backslashes. Concatenation places the equation between display delimiters. Deriving the right side from SymPy avoids manually copying a formula that later changes in the code. The function returns text and does not itself invoke a TeX compiler.

Definition and stated intent:

Source lines 5–6:

```
5 def display_equation(lhs, expression):
6     """Keep the mathematical right-hand side sourced directly from SymPy."""
```

`def` creates this callable; the parentheses name its inputs (`lhs`, `expression`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
7 return '\\[\n' + lhs + ' = ' + sp.latex(expression) + '\n\\]\n'
```

Read this block:

- Line 7: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.latex`: Serialize symbolic algebra as LaTeX source rather than recomputing the mathematical formula manually.

supply__code/export/pretty__print.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 import sympy as sp
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.

show

Original function: `show`. Source begins at line 4.

This is a terminal-output helper returning None. Blank `print()` creates a new line; `"=" * 70` repeats a string seventy times, whereas multiplication of numeric values has a different meaning. `sp.pprint` renders an expression in readable mathematical form in the terminal. It is useful for inspection, while `sp.latex` is the appropriate alternative for document source.

Definition and stated intent:

Source lines 4–4:

```
4 def show(title, expr):
```

`def` creates this callable; the parentheses name its inputs (`title`, `expr`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 5–9:

```
5 print()
6 print("=" * 70)
7 print(title)
8 print("=" * 70)
9 sp.pprint(expr)
```

Read this block:

- Line 5: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9: call `sp.pprint` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.pprint`: Pretty-print symbolic mathematics to the terminal for human inspection.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply__code/make__comparative__static__figures.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Generate only the requested symmetric-switching comparative-static figures."""
2 from pathlib import Path
3 import csv
4 import argparse
5 import numpy as np
6 import matplotlib
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `csv`. An as-name is a convenient local alias, not another library.
- Line 4: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `matplotlib`. An as-name is a convenient local alias, not another library.

Source lines 7–13:

```
7 matplotlib.use('Agg')
8 import matplotlib.pyplot as plt
9 PARAMETERS = dict(beta=2., alpha=-1., delta=.92, c=0.)
10
11 ROOT=Path(__file__).resolve().parent
12 OUT=ROOT/'writeup_math_graphs'/'figures'
13 GAMMAS=np.linspace(-3,3,601)
```

Read this block:

- Line 7: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 9, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 11, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.
- Line 12, `OUT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 13, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 14–15:

```
14 LABELS={'j':r'Initially inside ($x_0=j$)','0':r'Initially outside ($x_0=0$)'}
15 STATES=[('0','0'),('0','j'),('j','j'),('j','0')]
```

Read this block:

- Line 14, `LABELS`: Set this name to a dictionary with named entries `j`, `0`.
- Line 15, `STATES`: Set this name to a list containing 4 displayed entries.

Source lines 156–157:

```
156 if __name__=='__main__':
157     main()
```

Read this block:

- Line 156: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 157: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

state outcomes

Original function: `state_outcomes`. Source begins at line 18.

Conditions on each initial/first-choice history, retains terminal demand expectation, and produces histories with their path weights. This separates conditional economic mechanisms from ex ante comparisons.

Definition and stated intent:

Source lines 18–19:

```
18 def state_outcomes(curves):
19     """Condition on (x0, x1); terminal demand remains integrated out."""
```

`def` creates this callable; the parentheses name its inputs (`curves`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–20:

```
20 rows=[]
```

Read this block:

- Line 20, `rows`: Set this name to the list `[]`.

Source lines 21–35:

```
21 for initial, realized in STATES:
22     for r in curves[initial]:
23         p2=r['p2j' if realized=='j' else 'p20']
24         dp2=r['dp2j' if realized=='j' else 'dp20']
25         value=r['V2j' if realized=='j' else 'V20']
26         rows.append(dict(
27             gamma=r['gamma'], x0=initial, x1=realized,
28             probability=r['s'] if realized=='j' else 1-r['s'],
29             share1=r['s'], outside_share1=1-r['s'],
30             share2=r['u' if realized=='j' else 'v'],
31             outside_share2=1-r['u' if realized=='j' else 'v'],
32             price_ratio=r['p1']/p2,
33             price_ratio_derivative=(r['dp1']*p2-r['p1']*dp2)/p2**2,
```

```
34            realized_profit=(r['p1']-PARAMETERS['c'])*(realized=='j')
35                            +PARAMETERS['delta']*value))
```

Read this block:

- Line 21: for each entry yielded by `STATES`, bind/unpack it into `(initial, realized)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 22: for each entry yielded by `curves[initial]`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 23, `p2`: Set this name to the entry/slice selected by `r['p2j' if realized == 'j' else 'p20']`; brackets select from the object to their left.
- Line 24, `dp2`: Set this name to the entry/slice selected by `r['dp2j' if realized == 'j' else 'dp20']`; brackets select from the object to their left.
- Line 25, `value`: Set this name to the entry/slice selected by `r['V2j' if realized == 'j' else 'V20']`; brackets select from the object to their left.
- Line 26: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 36–36:

```
36 return rows
```

Read this block:

- Line 36: return the object currently named `rows`; plain assignment does not copy it. This ends the call.

plot state outcomes

Original function: `plot_state_outcomes`. Source begins at line 39.

Plots conditional prices, ratios or profits across gamma for the history-specific outcomes. Histories that are rare under a policy should not be confused with an equally weighted population average.

Definition and stated intent:

Source lines 39–39:

```
39 def plot_state_outcomes(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–45:

```
40 specs=[('all_state_price_ratios','price_ratio',r'$p_1^*(x_0)/p_2^*(x_1)$',
41         'Price ratios by realized history'),
42        ('all_state_price_ratio_derivatives','price_ratio_derivative',r'$dR_{x_0,x_1}/d\gamma$',
43         'Price-ratio derivatives by realized history'),
44        ('all_state_realized_profit','realized_profit',r'$\Pi^*(x_0,x_1)$',
45         'Profit conditional on realized first-period choice')]
```

Read this block:

- Line 40, `specs`: Set this name to a list containing 3 displayed entries.

Source lines 46–59:

```
for name,key,ylabel,title in specs:
    fig,ax=plt.subplots(figsize=(8,5))
    for (initial,realized),style in zip(STATES,['-','--','-.',':']):
        group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
        ax.plot([r['gamma'] for r in group],[r[key] for r in group],
                ls=style,lw=2,label=rf'$(x_0,x_1)=({initial},{realized})$')
    ax.axhline(0,color='gray',lw=.7)
    ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
    ax.legend();ax.grid(alpha=.2)
    if key=='realized_profit':
        fig.text(.5,.01,'First-period margin if purchased + discounted expected terminal profit',
                 ha='center',fontsize=9)
        fig.tight_layout(rect=(0,.04,1,1))
    save(fig,name)
```

Read this block:

- Line 46: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 49, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 50: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 55: test `key == 'realized_profit'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot state shares

Original function: `plot_state_shares`. Source begins at line 62.

Plots how policy-induced probabilities allocate households across realized histories. These probability weights explain why an ex ante profit comparison can differ from comparing conditional histories alone.

Definition and stated intent:

Source lines 62–62:

```
62 def plot_state_shares(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 63–63:

```
63 fig,axes=plt.subplots(2,2,figsize=(11,8),sharex=True,sharey=True)
```

Read this block:

- Line 63, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 64–74:

```
64 for ax,key,title in zip(axes.flat,
65         ['share1','share2','outside_share1','outside_share2'],
66         ['Period 1: inside share','Period 2: inside share',
67          'Period 1: outside share','Period 2: outside share']):
68     for (initial,realized),style in zip(STATES,['-','--','-.',':']):
69         group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
70         ax.plot([r['gamma'] for r in group],[r[key] for r in group],
71                 ls=style,lw=2,label=rf'$({initial},{realized})$')
72     ax.set(xlabel=r'$\gamma$',ylabel='Share',title=title,ylim=(0,1))
73     ax.grid(alpha=.2)
74     ax.legend(title=r'$(x_0,x_1)$',fontsize=8,ncol=2)
```

Read this block:

- Line 64: for each entry yielded by `zip(axes.flat, ['share1', 'share2', 'outside_share1', 'outside...`, bind/unpack it into `(ax, key, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 68: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 69, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 70: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 72: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 75–75:

```
75 save(fig,'all_state_shares')
```

Read this block:

- Line 75: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

save

Original function: `save`. Source begins at line 78.

This formatting/output helper takes a Matplotlib Figure and filename stem. `tight_layout` adjusts spacing; a two-element suffix loop saves both PDF and PNG; `plt.close` frees the Figure. `OUT / f"{name}.{suffix}"` constructs each Path. Shared export code prevents one figure accidentally lacking its vector PDF version. DPI matters chiefly for raster PNG content; it does not make an ordinary vector line in a PDF more mathematically precise.

Definition and stated intent:

Source lines 78–78:

```
78 def save(fig,name):
```

`def` creates this callable; the parentheses name its inputs (`fig`, `name`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 79–79:

```
79 fig.tight_layout()
```

Read this block:

- Line 79: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 80–81:

```
80 for suffix in ['pdf','png']:
81     fig.savefig(OUT/f'{name}.{suffix}',dpi=160)
```

Read this block:

- Line 80: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `suffix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 82–82:

```
82 plt.close(fig)
```

Read this block:

- Line 82: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 85.

This generates all benchmark rows from the production `evaluate` function, writes a solution CSV, and plots six requested figures. The dictionary comprehension creates a list of gamma evaluations for each initial-state key. `csv.DictWriter` uses dictionary keys as field names, so output records retain branch meanings. Plot `specs` is a data list describing several similar figures; the loop executes the common plotting logic.

Separate blocks handle acquisition ratios and the two-panel price-level figure. Terminal prices can be read from either initial-state collection because their realized history determines them. A final generator computes the largest absolute FOC residual and raises if it exceeds tolerance. Plotting algebra typed anew here would risk drifting from the derivative expressions; numerical values instead come from the common symbolic-to-numerical path.

Definition and stated intent:

Source lines 85–85:

```
85 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–90:

```
86 parser=argparse.ArgumentParser(description=__doc__)
87 parser.add_argument('--from-solutions',action='store_true',
88                     help='Use the saved benchmark solutions instead of solving again.')
```

```
89 args=parser.parse_args()
90 OUT.mkdir(parents=True,exist_ok=True)
```

Read this block:

- Line 86, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 87: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `OUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 91–100:

```
91 if args.from_solutions:
92     with (OUT/'solutions.csv').open(newline='') as file:
93         saved=[{k:v if k=='initial_state' else float(v) for k,v in row.items()}
94                for row in csv.DictReader(file)]
95     curves={state:[r for r in saved if r['initial_state']==state] for state in ['j','0']}
96     for group in curves.values():
97         np.testing.assert_allclose([r['gamma'] for r in group],GAMMAS,atol=1e-12)
98 else:
99     from model.numerical import evaluate
100    curves={state:[evaluate(float(g),state,**PARAMETERS) for g in GAMMAS] for state in ['j','0']}
```

Read this block:

- Line 91: test `args.from_solutions`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 92: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 93, `saved`: Set this name to a list computed by the displayed for-expression.
- Line 95, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 96: for each entry yielded by `curves.values()`, bind/unpack it into `group` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 97: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: from `model.numerical`, import `evaluate` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 100, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 101–101:

```
101 outcomes=state_outcomes(curves)
```

Read this block:

- Line 101, `outcomes`: Set this name to the result of calling `state_outcomes(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 102–105:

```
102 with (OUT/'all_state_outcomes.csv').open('w',newline='') as file:
103     writer=csv.DictWriter(file,fieldnames=list(outcomes[0]))
104     writer.writeheader()
105     writer.writerows(outcomes)
```

Read this block:

- Line 102: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 103, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 104: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.
- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 107–111:

```
107 for initial in ['0','j']:
108     groups={a:[r for r in outcomes if r['x0']==initial and r['x1']==a] for a in ['0','j']}
109     expected=[sum(groups[a][i]['probability']*groups[a][i]['realized_profit']
110                   for a in ['0','j']) for i in range(len(GAMMAS))]
111     np.testing.assert_allclose(expected,[r['profit'] for r in curves[initial]],atol=1e-12)
```

Read this block:

- Line 107: for each entry yielded by `['0', 'j']`, bind/unpack it into `initial` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 108, `groups`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 109, `expected`: Set this name to a list computed by the displayed for-expression.
- Line 111: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `len`: Count elements/rows rather than summing their numerical values.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 112–112:

```
112 plot_state_outcomes(outcomes)
```

Read this block:

- Line 112: call `plot_state_outcomes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 113–116:

```
113 for r in outcomes:
114     for period in [1,2]:
115         assert 0<=r[f'share{period}']<=1
116         np.testing.assert_allclose(r[f'share{period}']+r[f'outside_share{period}'],1)
```

Read this block:

- Line 113: for each entry yielded by `outcomes`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 114: for each entry yielded by `[1, 2]`, bind/unpack it into `period` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 115: require `0 <= r[f'share{period}'] <= 1` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 116: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 117–117:

```
117 plot_state_shares(outcomes)
```

Read this block:

- Line 117: call `plot_state_shares` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_shares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 118–122:

```
118 with (OUT/'solutions.csv').open('w',newline='') as file:
119     writer=csv.DictWriter(file,fieldnames=list(curves['j'][0]))
120     writer.writeheader()
121     for state in ['j','0']:
122         writer.writerows(curves[state])
```

Read this block:

- Line 118: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 119, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 120: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 121: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 123–126:

```
123 specs=[('profit_by_gamma','profit',r'$\Pi^*$','Optimized two-period profit'),
124        ('gamma_profit_by_gamma','dprofit',r'$d\Pi^*/d\gamma$','Optimized profit derivative'),
125        ('gamma_p1_by_gamma','dp1',r'$dp_1^*/d\gamma$','Optimal period-1 price derivative'),
126        ('matched_price_ratio_derivative','d_matched_ratio',r'$dR_a/d\gamma$','State-matched price-ratio
    ↪ derivatives')]
```

Read this block:

- Line 123, `specs`: Set this name to a list containing 4 displayed entries.

Source lines 127–134:

```
127 for name,key,ylabel,title in specs:
128     fig,ax=plt.subplots(figsize=(7.2,4.5))
129     for state in ['j','0']:
130         ax.plot(GAMMAS,[r[key] for r in curves[state]],label=LABELS[state],lw=2)
131     ax.axhline(0,color='gray',lw=.7)
132     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
133     ax.legend();ax.grid(alpha=.2)
134     save(fig,name)
```

Read this block:

- Line 127: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 129: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 130: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 134: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

Source lines 135–140:

```
135 fig,ax=plt.subplots(figsize=(7.2,4.5))
136 ax.plot(GAMMAS,[r['d_induce_ratio'] for r in curves['0']],lw=2,
137         label=r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$')
138 ax.axhline(0,color='gray',lw=.7)
139 ax.set(xlabel=r'$\gamma$',ylabel=r'$dR_{0\to j}/d\gamma$',title='Induce-then-harvest price-ratio derivative')
140 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
```

Read this block:

- Line 135, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 136: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 140–141:

```
140 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
141 fig,axes=plt.subplots(1,2,figsize=(10,4.2))
```

Read this block:

- Line 140: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 141, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 142–143:

```
142 for state in ['j','0']:
143     axes[0].plot(GAMMAS,[r['p1'] for r in curves[state]],label=LABELS[state],lw=2)
```

Read this block:

- Line 142: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 143: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 144–145:

```
144 for state,key,label in [('j','p2j',r'Previous inside ($x_1=j$)'),('0','p20',r'Previous outside ($x_1=0$)')]:
145     axes[1].plot(GAMMAS,[r[key] for r in curves['j']],label=label,lw=2)
```

Read this block:

- Line 144: for each entry yielded by `[('j', 'p2j', 'Previous inside ($x_1=j$)'), ('0', 'p20', 'Prev...`, bind/unpack it into `(state, key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 145: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 146–148:

```
146 for ax,title,ylabel in zip(axes,['Optimal period-1 prices','Optimal period-2 prices'],[r'$p_1^*$',r'$p_2^*$']):
147     ax.axhline(0,color='gray',lw=.7);ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
148     ax.legend(fontsize=8);ax.grid(alpha=.2)
```

Read this block:

- Line 146: for each entry yielded by `zip(axes, ['Optimal period-1 prices', 'Optimal period-2 prices...`, bind/unpack it into `(ax, title, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 147: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 147: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 149–150:

```
149 save(fig,'optimal_prices_by_gamma')
150 maximum=max(abs(r[k]) for state in curves for r in curves[state] for k in ['foc1','foc2j','foc20'])
```

Read this block:

- Line 149: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 150, `maximum`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 151–152:

```
151 if maximum>1e-9:
152     raise AssertionError(f'FOC residual {maximum}')
```

Read this block:

- Line 151: test `maximum > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 152: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 153–153:

```
153 print(f'Saved ten figures, 1,202 solved observations, and {len(outcomes):,} conditional outcomes; max FOC
    ↪  residual={maximum:.3g}')
```

Read this block:

- Line 153: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/make_latex_document.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Generate a verified symmetric-switching appendix with six selected figures."""
2 from pathlib import Path
3 import sympy as sp
4 from model.symbols import *
5 from model.utility import U1,U2
6 from model.profit import Pi
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
7 from pricing.period1_foc import foc1_collected
8 from pricing.period2_foc import foc2
9 from pricing.period1_price import p1_star
10 from pricing.period2_price import p2_star
11 from comparative_statics import canonical as C
12 from model.numerical import evaluate
```

Read this block:

- Line 7: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 8: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `pricing.period1_price`, import `p1_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period2_price`, import `p2_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `model.numerical`, import `evaluate` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–15:

```
13 from export.latex_export import display_equation
14
15 ROOT=Path(__file__).resolve().parent/'writeup_math_graphs'
```

Read this block:

- Line 13: from `export.latex_export`, import `display_equation` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15, `ROOT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 276–277:

```
276 if __name__=='__main__':
277     main()
```

Read this block:

- Line 276: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 18.

This writes a research appendix as text. The opening loop pairs LaTeX left-hand side strings with actual SymPy expressions and delegates equation formatting. The very long raw triple-quoted `document` string is document **data** inside the Python program: its LaTeX commands are interpreted later by a TeX engine, not executed as Python statements. A raw string preserves the many backslashes.

Later loops append benchmark rows and figure sections. An f-string inserts computed numbers with three decimals; adjacent raw-string fragments preserve LaTeX row-ending backslashes. The figure-existence check stops on missing assets instead of generating a broken paper. `write_text` creates `.tex`, not a PDF; compilation is a separate step. Repeated string `+=` is acceptable for this small document; accumulating a list and `"".join(parts)` would scale better for very large text assembly.

Definition and stated intent:

Source lines 18–18:

```
18 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–21:

```
19 ROOT.mkdir(exist_ok=True)
20 equations='\\section{Symbolically derived equations}\n'
21 equations += r'Here $q_2=q(x_1)$, $s_2=s_2(x_1)$ and $p_2=p_2(x_1)$ describe a fixed terminal history. Price
    ↪  equations and comparative-static right-hand sides use equilibrium shares; the profit and FOCs use shares at the
    ↪  candidate prices.'+'\n'
```

Read this block:

- Line 19: call `ROOT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `equations`: Set this name to the literal text `'\\section{Symbolically derived equations}\n'`.
- Line 21: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 22–28:

```
22 for lhs,expression in [(r'v_1',U1),(r'v_2',U2),(r'\Pi',Pi),
23                        (r'F_1',foc1_collected),(r'F_2',foc2),
24                        (r'p_1^*',p1_star),(r'p_2^*',p2_star),
25                        (r'\frac{dp_1^*}{d\gamma}',C.dp1_dgamma),
26                        (r'\frac{dp_2^*}{d\gamma}',C.dp2_dgamma),
27                        (r'\frac{d\Pi^*}{d\gamma}',C.dPi_dgamma)]:
28     equations+=display_equation(lhs,expression)
```

Read this block:

- Line 22: for each entry yielded by `[('v_1', U1), ('v_2', U2), ('\\Pi', Pi), ('F_1', foc1_collecte...`, bind/unpack it into `(lhs, expression)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 28: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `display_equation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 29–29:

```
29 (ROOT/'sympy_equations.tex').write_text(equations)
```

Read this block:

- Line 29: call `(ROOT / 'sympy_equations.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Source lines 30–71:

```
    document=r'''\documentclass[11pt]{article}
\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm,graphicx,booktabs,array}
\usepackage[hidelinks]{hyperref}
\newtheorem{proposition}{Proposition}
\newcommand{\dd}{\mathrm{d}}
\setlength{\parindent}{0pt}
\setlength{\parskip}{5pt}
\title{Two-period pricing with symmetric switching preferences}
\author{}\date{October 30, 2026}
\begin{document}\maketitle
\section{Model, histories, and normalization}
One firm sells one inside product $j$ against outside option $0$ in periods
$t=1,2$. Type $r$ is fixed and observed by the firm, which prices separately for
that type. All formulas below are conditional on $r$; its subscript and the
fixed product index are suppressed. There is no pooling of heterogeneous types.
The initial realized choice is $x_0\in\{j,0\}$. The second-period price can depend
on the realized first-period choice $x_1$. Independent Type-I extreme-value
shocks enter both alternatives in each period.

Before normalization the deterministic utilities are
\[
 V_{jt}^{\rm raw}=\beta+\alpha p_t+\gamma\mathbf1\{x_{t-1}=0\},
 \qquad V_{0t}^{\rm raw}=\gamma\mathbf1\{x_{t-1}=j\}.
\]
The payoff to switching applies in either direction. Subtracting the raw outside
utility from both alternatives gives normalized outside utility zero and
\[
 v_t(h)=\beta+\alpha p_t(h)+q(h)\gamma,\qquad
 q(j)=-1,\quad q(0)=+1.
\]
Normalization retains the outside switching payoff through the signed inside
utility difference; it does not delete that payoff. Negative $\gamma$ denotes
switching aversion and positive $\gamma$ denotes novelty preference.
\[
 s_t(h)=\frac{e^{v_t(h)}}{1+e^{v_t(h)}},\quad s_{0t}(h)=1-s_t(h),\quad
 \partial_p s_t=\alpha s_t(1-s_t),\quad
 (\partial_\gamma s_t)_p=q(h)s_t(1-s_t).
\]
A realized state $x_t$, a hypothetical switching indicator, and the probability
$s_t$ are distinct objects. Every derivative below holds the discrete history fixed.
```

Source lines 72–113:

```
\textbf{Assumptions.} $\alpha<0$, $0<\delta<1$, fixed marginal cost $c$, finite
parameters, and interior binary-logit shares. Prices are unrestricted real
numbers: introductory subsidies are allowed. Continuation values are conditional
on realized states and have no direct dependence on $p_1$. Type does not evolve.
The case $\delta=0$ is used only as a static counterfactual.

\begin{center}\begin{tabular}{ll}\toprule
Notation & Meaning\\\midrule
$q=q(x_0)$ & Signed initial-state coefficient\\
$s=s_1^*(x_0),\quad r_0=1-s$ & First-period equilibrium shares\\
$u=s_2^*(j),\quad v=s_2^*(0)$ & Inside shares in the two continuation states\\
$V_j=V_2(j),\quad V_0=V_2(0)$ & Optimized continuation profits\\
$\Delta V=V_j-V_0,\quad B=u+v$ & Continuation difference and sum of terminal shares\\
$\Pi^*$ & Expected discounted profit per initial consumer\\\bottomrule
\end{tabular}\end{center}

\section{Backward induction and optimality}
For each realized first-period choice $a\in\{j,0\}$, solve
```

```
\[
 V_2(a)=\max_{p_2(a)}s_2(a)[p_2(a)-c].
\]
Its first-order condition is
\[
 0=s_2+\alpha s_2(1-s_2)(p_2-c),\qquad
 p_2^*(a)=c-\frac1{\alpha[1-s_2^*(a)]}.
\]
This is an implicit equation: the share depends on the same price being solved.
Both $V_j$ and $V_0$ must be computed before period 1.

First-period profit integrates over both possible realized histories:
\begin{align*}
 \Pi(p_1;x_0)&=s_1(p_1-c)+\delta[s_1V_j+(1-s_1)V_0]\\
 &=\delta V_0+s_1[p_1-c+\delta\Delta V].
\end{align*}
Because the branch values do not depend directly on $p_1$,
\[
 \partial_{p_1}\{\delta[s_1V_j+(1-s_1)V_0]\}
 =\delta\alpha s_1(1-s_1)(V_j-V_0).
\]
Consequently,
\[
 0=s_1+\alpha s_1(1-s_1)[p_1-c+\delta\Delta V],\qquad
```

Source lines 114–155:

```
 p_1^*=c-\frac1{\alpha(1-s)}-\delta\Delta V.
\]
The static FOC component $c-1/[\alpha(1-s)]$ uses the share at the dynamic
optimum. It is not the counterfactual static optimum, which must be solved
separately with $\delta=0$.

\begin{proposition}[Unique optimal prices]
Under the stated assumptions each continuation price and the first-period price
has a unique finite global optimum, with no lower bound imposed on $p_1$.
\end{proposition}
\begin{proof}
Both price problems reduce, up to a constant, to maximizing
$s(p)(p-k)$ with $s(p)=\operatorname{logit}^{-1}(b+\alpha p)$.
Use $k=c$ in period 2 and $k=c-\delta\Delta V$ in period 1.
The sign of the profit derivative is the sign of
\[
 H(p)=1+e^{b+\alpha p}+\alpha(p-k),\quad
 H'(p)=\alpha(1+e^{b+\alpha p})<0.
\]
$H$ ranges from positive infinity to negative infinity, so its root is unique
and the objective increases before it and decreases afterward. The price may
lie below physical cost $c$ in period 1 when continuation incentives justify it.
\end{proof}

\section{Envelope derivatives and conditional comparative statics}
Differentiating each continuation value using the envelope theorem gives
\[
 V_2'(a)=q(a)s_2(1-s_2)(p_2^*-c)=-\frac{q(a)s_2}{\alpha}.
\]
Implicit differentiation of the terminal FOC gives the same expression for the
optimal price derivative:
\[
 p_2^{*\prime}(a)=-\frac{q(a)s_2}{\alpha},\quad
 V_j'=u/\alpha,\quad V_0'=-v/\alpha,\quad
 \Delta V'=(u+v)/\alpha.
```

```
\]
In particular, $p_2^{*\prime}(j)<0$ and $p_2^{*\prime}(0)>0$.
The total equilibrium terminal-share derivative is
$s_2'=q(a)s_2(1-s_2)^2$, including the optimal-price response.

For the first-period implicit pricing equation let
$F=p_1-c+1/[\alpha(1-s_1)]+\delta\Delta V$. At fixed history,
```

Source lines 156–197:

```
\[
 F_{p_1}=\frac1{r_0},\qquad
 F_\gamma=\frac{qs}{\alpha r_0}+\frac{\delta B}{\alpha},\qquad
 \boxed{p_1^{*\prime}=-\frac{qs+\delta r_0 B}{\alpha}}.
\]
The share therefore changes according to
\[
 s'=sr_0(q+\alpha p_1^{*\prime})=sr_0^2(q-\delta B).
\]
The first-period envelope theorem retains both continuation branches:
\begin{align*}
 \Pi^{*\prime}
 &=qsr_0[p_1^*-c+\delta\Delta V]+\delta[sV_j'+r_0V_0']\\
 &=\boxed{\frac{-qs+\delta(su-r_0v)}{\alpha}}.
\end{align*}
The term $\delta V_0'$ must not be dropped. The envelope partial derivative holds
the optimizing price fixed inside the partial derivative; it does not hold the
continuation value fixed as $\gamma$ varies.

For the analytical second price derivative define
\[
 u'=-u(1-u)^2,\quad v'=v(1-v)^2,\quad B'=u'+v'.
\]
Differentiating the first derivative and substituting $s'$ gives
\[
 \boxed{p_1^{*\prime\prime}
 =-\frac{sr_0^2(q-\delta B)^2+\delta r_0B'}{\alpha}}.
\]
This expression is used analytically in checks, rather than obtained from noisy
twice-differenced optimized prices. No second-derivative figure is included.

\begin{proposition}[Conditional signs]
For an initially outside consumer, $p_1^{*\prime}>0$ and $\Pi^{*\prime}>0$.
For an initially inside consumer,
\[
 p_1^{*\prime}>0\ \Longleftrightarrow\ \delta r_0(u+v)>s,\qquad
 \Pi^{*\prime}>0\ \Longleftrightarrow\ \delta r_0v>s(1+\delta u).
\]
Equalities give zero derivatives; reversed inequalities give negative derivatives.
\end{proposition}
\begin{proof}
For $q=1$, the price numerator $s+\delta r_0B$ is positive and $-1/\alpha>0$.
```

Source lines 198–239:

```
The profit numerator is $-s(1-\delta u)-\delta r_0v<0$ because $\delta u<1$;
dividing by $\alpha<0$ makes the derivative positive. Substituting $q=-1$
gives the two stated thresholds. These statements do not establish a common
monotonicity sign for both histories or a global sign for the price ratios.
\end{proof}

\section{Price ratios: three distinct conditioning statements}
```

```
The state-matched ratios are descriptive comparisons:
\[
 R_j=\frac{p_1^*(x_0=j)}{p_2^*(x_1=j)},\qquad
 R_0=\frac{p_1^*(x_0=0)}{p_2^*(x_1=0)}.
\]
They do not necessarily describe one realized consumer path. The acquisition
comparison is instead
\[
 R_{0\to j}=\frac{p_1^*(x_0=0)}{p_2^*(x_1=j)},
\]
conditional on acquisition in period 1. It is not an expectation over histories.
For each definition use its own numerator, denominator and state derivatives:
\[
 \boxed{R'=\frac{p_2^*p_1^{*\prime}-p_1^*p_2^{*\prime}}{(p_2^*)^2}}.
\]
Terminal prices exceed $c$; the benchmark $c=0$ therefore has strictly positive
ratio denominators. For other calibrations a ratio is only defined when its
denominator is nonzero. No expected ratio or ratio of expectations is substituted.

\section{Numerical benchmark and verification}
Every plotted point reoptimizes both terminal branches and period 1:
\[
 \beta=2,\qquad\alpha=-1,\qquad\delta=0.92,\qquad c=0,\qquad
 \gamma\in[-3,3]\ \text{on a 601-point grid}.
\]
The calibration is illustrative, not estimated. SymPy verifies raw-utility
normalization, both branch weights, the profit-derived FOCs, implicit price
derivatives and the envelope identities. Numerical checks compare all 1,202
benchmark state/grid solutions with the handoff's independent reference.
Central differences reoptimize the full model at nearby parameters; an
independent scalar profit optimizer checks optimality. Additional calibrations
check derivatives and an introductory-subsidy case. Numerical evidence is not
a proof of unqualified global comparative-static signs.

\subsection*{Selected optimized values}
```

Source lines 240–242:

```
\begin{center}\small\begin{tabular}{crrrrrr}\toprule
$x_0$ & $\gamma$ & $p_1^*$ & $p_2^*(j)$ & $p_2^*(0)$ & $\Pi^*$ & $\Pi^{*\prime}$\\\midrule
'''
```

Read this block:

- Line 30, `document`: Set this name to a multiline text template; its contents are data, not Python instructions.

Source lines 243–246:

```
for state in ['j','0']:
    for g in [-3,-1,0,1,3]:
        r=evaluate(g,state)
        document+=f"${state}$ & {g} & {r['p1']:.3f} & {r['p2j']:.3f} & {r['p20']:.3f} & {r['profit']:.3f} &
 ↪  {r['dprofit']:.3f}"+r'\\'+'\n'
```

Read this block:

- Line 243: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 244: for each entry yielded by `[-3, -1, 0, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 245, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

- Line 246: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 247–250:

```
    document+=r'''\bottomrule\end{tabular}\end{center}
\clearpage
\input{sympy_equations.tex}
'''
```

Read this block:

- Line 247: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 251–264:

```
specs=[
    ('matched_price_ratio_derivative','State-matched price-ratio derivatives',
     r'$R_j=p_1^*(j)/p_2^*(j)$ and $R_0=p_1^*(0)/p_2^*(0)$. Each curve holds its stated histories fixed and uses
      ↪  the analytical quotient rule.'),
    ('induce_harvest_price_ratio_derivative','Induce-then-harvest price-ratio derivative',
     r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$, conditional on a first-period acquisition. This is a path-specific
      ↪  comparison, not an ex-ante expectation.'),
    ('profit_by_gamma','Optimized profit levels',
     r'Expected discounted profit includes both realized continuation histories. The two curves condition on the
      ↪  initial realized state.'),
    ('gamma_profit_by_gamma','Optimized profit derivative',
     r'Total derivatives use the envelope theorem and both continuation-value derivatives. Their signs depend on
      ↪  the initial history and the stated thresholds.'),
    ('optimal_prices_by_gamma','Optimal prices in both periods',
     r'Left: period-1 prices for both initial states. Right: period-2 prices for both realized first-period states.
      ↪  Prices are unconstrained; all shares adjust.'),
    ('gamma_p1_by_gamma','Optimal period-1 price derivative',
     r'Analytical total derivatives, with both continuation branches reoptimized. Initial-state signs are
      ↪  interpreted using the proved conditional statements.'),
]
```

Read this block:

- Line 251, `specs`: Set this name to a list containing 6 displayed entries.

Source lines 265–270:

```
for name,title,caption in specs:
    if not (ROOT/'figures'/f'{name}.pdf').exists():
        raise FileNotFoundError(name)
    document+='\\clearpage\n\\section*{'+title+'}\n'
    document+='\\begin{figure}[htbp]\\centering\n\\includegraphics[width=\\linewidth]{figures/'+name+'.pdf}\n'
    document+='\\caption{'+caption+r' Benchmark: $\beta=2$, $\alpha=-1$, $\delta=0.92$, $c=0$;
 ↪  $\gamma\in[-3,3]$.}'+'\n\\end{figure}\n'
```

Read this block:

- Line 265: for each entry yielded by `specs`, bind/unpack it into `(name, title, caption)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 266: test `not (ROOT / 'figures' / f'{name}.pdf').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 267: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 268: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 269: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 270: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 271–273:

```
271 document+='\\end{document}\n'
272 (ROOT/'comparative_statics_figures.tex').write_text(document)
273 print('Generated verified symbolic equations and appendix with six requested figures.')
```

Read this block:

- Line 271: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 272: call `(ROOT / 'comparative_statics_figures.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 273: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/model/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/model/assumptions.py

This dictionary stores human-readable assumptions as strings. Python and SymPy do not enforce alpha<0 just because a dictionary says so. It serves documentation; the numerical evaluator performs explicit checks. Distinguish descriptive metadata from executable validation.

Module setup and direct-execution blocks

Source lines 1–3:

```
assumptions = {"alpha": "<0", "delta": "0<delta<1", "prices": "unconstrained real",
               "cost": "fixed", "shares": "strictly interior", "type": "fixed and observed",
               "history": "realized j or 0; both continuation states integrated"}
```

Read this block:

- Line 1, `assumptions`: Set this name to a dictionary with named entries `alpha`, `delta`, `prices`, `cost`, `shares`, `type`, `history`.

supply_code/model/numerical.py

Module-level tuples `_ARGS`, `_EXPRESSIONS`, and `_KEYS` establish the ordered bridge from symbolic algebra to named numerical derivative results. A leading underscore is a convention for an internal name, not access control. `sp.lambdify` executes once at import, avoiding repeated compilation of the same symbolic formulas for all 1,202 benchmark evaluations. The functions below reuse that callable. Positional order, rather than matching name spelling at runtime, connects its inputs.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Unconstrained backward induction; derivatives evaluated from SymPy."""
import numpy as np
import sympy as sp
from scipy.special import expit
from scipy.optimize import brentq
from model import symbols as S
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model`, import `symbols as S` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
from model.utility import signed_state
from comparative_statics import canonical as C

PARAMETERS = dict(beta=2., alpha=-1., delta=.92, c=0.)
_KEYS = ("dp1", "d2p1", "dprofit", "matched_j", "matched_0", "induce_harvest", "dp2j", "dp20")
_ARGS = (S.alpha,S.beta,S.gamma,S.delta,S.c,S.q,S.sj1,S.u,S.v,S.p1,S.p2j,S.p20,S.Vj,S.V0)
_EXPRESSIONS = (C.dp1_dgamma,C.d2p1_dgamma2,C.dPi_dgamma,C.matched_j,C.matched_0,C.induce_harvest,
                C.price2_derivative(S.gamma,S.u,-1,S.p2j),C.price2_derivative(S.gamma,S.v,1,S.p20))
```

Read this block:

- Line 7: from `model.utility`, import `signed_state` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

- Line 11, `_KEYS`: Set this name to a tuple containing 8 displayed entries.
- Line 12, `_ARGS`: Set this name to a tuple containing 14 displayed entries.
- Line 13, `_EXPRESSIONS`: Set this name to a tuple containing 8 displayed entries.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 15–15:

```
15 _EVALUATE = sp.lambdify(_ARGS,_EXPRESSIONS,"numpy")
```

Read this block:

- Line 15, `_EVALUATE`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.

inside share

Original function: `inside_share`. Source begins at line 18.

This computes one numeric binary-logit probability. `signed_state` validates the state; `expit` evaluates logistic demand stably, then `float` turns a NumPy scalar into an ordinary Python number. Direct `exp(x)/(1+exp(x))` can overflow at large x. The defaults beta=2 and alpha=-1 support the benchmark, while callers can supply other calibrations. This function alone does not validate alpha's sign; full parameter-domain checks live in `evaluate`.

Definition and stated intent:

Source lines 18–18:

```
18 def inside_share(price,gamma,state,beta=2.,alpha=-1.):
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 return float(expit(beta+signed_state(state)*gamma+alpha*price))
```

Read this block:

- Line 19: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimal price

Original function: `optimal_price`. Source begins at line 22.

This finds one optimal price for a utility intercept, negative alpha, and an effective cost. `f = lambda p: ...` defines the transformed scalar FOC. `clip` limits the exponential's argument to a representable range; it is a numerical safeguard rather than a theoretical cap on utility.

The initial lower endpoint is effective cost, where f is positive. `max` sets a reasonable initial width, and `while f(hi) > 0` doubles it until the bracket changes sign. `brentq` can then locate the zero to `xtol=1e-12`. A fixed interval is simpler but can miss a root under another calibration. Unconstrained generic minimization ignores the monotonic scalar structure and can need more evaluations. The maintained alpha<0 assumption establishes a unique root; the caller validates it.

Definition and stated intent:

Source lines 22–23:

```
22 def optimal_price(intercept,alpha,effective_cost):
23     # Multiplying the FOC by alpha gives a strictly decreasing function.
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–27:

```
24 f = lambda p: alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p,-745,700))
25 lo = effective_cost
26 width = max(2.,2/abs(alpha))
27 hi = lo+width
```

Read this block:

- Line 24, `f`: Set this name to a one-expression callable, with input `p` and returned expression `alpha * (p - effective_cost) + 1 + np.exp(np.clip(in...`.
- Line 25, `lo`: Set this name to the object currently named `effective_cost`; plain assignment does not copy it.
- Line 26, `width`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 27, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 28–30:

```
28 while f(hi) > 0:
29     width *= 2
30     hi = lo+width
```

Read this block:

- Line 28: repeat while `f(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 29: update `width` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 30, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `f`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return float(brentq(f,lo,hi,xtol=1e-12))
```

Read this block:

- Line 31: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

evaluate

Original function: `evaluate`. Source begins at line 34.

This is backward induction expressed as ordinary assignments. It accepts gamma, initial history, and optional calibration numbers, then returns a dictionary. `np.isfinite([...]).all()` validates every numeric input at once. The terminal list comprehension solves two prices, unpacked into p2j and p20. Subsequent tuple assignments compute their shares and values; first-period effective cost then uses the value difference.

`_EVALUATE(...)` calls the numerical function built from symbolic derivatives at import time. Its argument order must match `_ARGS`; a positional swap can produce plausible but wrong derivatives. `result.update(dict(zip(_KEYS, map(float, values))))` converts and labels those outputs. State-inapplicable ratio derivatives become NaN intentionally. Final FOC residuals use original profit derivatives as a diagnostic independent of the root's transformed form. An external simultaneous optimizer would conceal the model's simpler sequential structure.

Definition and stated intent:

Source lines 34–34:

```
34 def evaluate(gamma,initial_state,beta=2.,alpha=-1.,delta=.92,c=0.):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`, `beta`, `alpha`, `delta`, `c`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–35:

```
35 q = signed_state(initial_state)
```

Read this block:

- Line 35, `q`: Set this name to the result of calling `signed_state(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 36–37:

```
36 if not (np.isfinite([gamma,beta,alpha,delta,c]).all() and alpha<0 and 0<=delta<1):
37     raise ValueError("Require finite parameters, alpha<0, and 0<=delta<1 (delta=0 is the static counterfactual).")
```

Read this block:

- Line 36: test `not (np.isfinite([gamma, beta, alpha, delta, c]).all() and alpha < 0 and (0 <...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 37: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 38–43:

```
38 p2j,p20 = [optimal_price(beta+signed_state(state)*gamma,alpha,c) for state in ["j","0"]]
39 u,v = inside_share(p2j,gamma,"j",beta,alpha),inside_share(p20,gamma,"0",beta,alpha)
40 Vj,V0 = (p2j-c)*u,(p20-c)*v
41 DV = Vj-V0
42 p1 = optimal_price(beta+q*gamma,alpha,c-delta*DV)
43 s = inside_share(p1,gamma,initial_state,beta,alpha)
```

Read this block:

- Line 38, `(p2j, p20)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 39, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 41, `DV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 42, `p1`: Set this name to the result of calling `optimal_price(...)`; the arguments and preceding method chain are shown above.
- Line 43, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `optimal_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 44–51:

```
44 profit = s*(p1-c)+delta*(s*Vj+(1-s)*V0)
45 values = _EVALUATE(alpha,beta,gamma,delta,c,q,s,u,v,p1,p2j,p20,Vj,V0)
46 result = dict(gamma=float(gamma),initial_state=initial_state,p1=p1,p2j=p2j,p20=p20,s=s,u=u,v=v,
47               V2j=Vj,V20=V0,DeltaV=DV,profit=profit,
48               static_component=c-1/(alpha*(1-s)),continuation_adjustment=-delta*DV,
49               static_counterfactual=optimal_price(beta+q*gamma,alpha,c))
50 result.update(dict(zip(_KEYS,map(float,values))))
51 result['matched_j' if initial_state=='0' else 'matched_0'] = float('nan')
```

Read this block:

- Line 44, `profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45, `values`: Set this name to the result of calling `_EVALUATE(...)`; the arguments and preceding method chain are shown above.
- Line 46, `result`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `result.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51, `result['matched_j' if initial_state == '0' else 'matched_0']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `_EVALUATE`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `map`: Apply a callable to each iterable entry lazily, such as converting result scalars to Python float.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 52–53:

```
52 if initial_state=='j':
53     result['induce_harvest'] = float('nan')
```

Read this block:

- Line 52: test `initial_state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 53, `result['induce_harvest']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 54–58:

```
54 result['d_matched_ratio'] = result['matched_j'] if initial_state=='j' else result['matched_0']
55 result['d_induce_ratio'] = result['induce_harvest'] if initial_state=='0' else float('nan')
56 result['foc1'] = s+alpha*s*(1-s)*(p1-c+delta*DV)
57 result['foc2j'] = u+alpha*u*(1-u)*(p2j-c)
58 result['foc20'] = v+alpha*v*(1-v)*(p20-c)
```

Read this block:

- Line 54, `result['d_matched_ratio']`: Store the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value in the selected entry/column.
- Line 55, `result['d_induce_ratio']`: Store the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value in the selected entry/column.
- Line 56, `result['foc1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 57, `result['foc2j']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 58, `result['foc20']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 59–59:

```
59 return result
```

Read this block:

- Line 59: return the object currently named `result`; plain assignment does not copy it. This ends the call.

solve

Original function: `solve`. Source begins at line 62.

This is a compact convenience interface. The bare `*` makes `initial_state` keyword-only; `**parameters` collects other keywords. Membership tests reject obsolete z1/z2 inputs. `parameters.pop("gamma", 0.)` retrieves and removes gamma, allowing the rest to be expanded into `evaluate` without passing gamma twice. The function returns seven values as a tuple. It is convenient for unpacking, while `evaluate` is preferable when you need named derivative or diagnostic fields.

Definition and stated intent:

Source lines 62–62:

```
62 def solve(*, initial_state="0", **parameters):
```

`def` creates this callable; the parentheses name its inputs (`initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**parameters` collects additional keyword arguments in a dictionary.

Source lines 63–64:

```
63 if 'z1' in parameters or 'z2' in parameters:
64     raise ValueError("Old 0/1 histories are superseded: use initial_state='j' or '0'; both continuation branches
       ↪ are solved.")
```

Read this block:

- Line 63: test `'z1' in parameters or 'z2' in parameters`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 64: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 65–66:

```
65 gamma = parameters.pop('gamma',0.)
66 r = evaluate(gamma,initial_state,**parameters)
```

Read this block:

- Line 65, `gamma`: Set this name to the result of calling `parameters.pop(...)`; the arguments and preceding method chain are shown above.
- Line 66, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.pop`: Retrieve and remove a dictionary entry, using the supplied default if it is absent.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–67:

```
67 return r['p1'],r['p2j'],r['p20'],r['s'],r['u'],r['v'],r['profit']
```

Read this block:

- Line 67: return a tuple of 7 results. The caller can unpack them; return ends the call, unlike print.

supply_code/model/profit.py

These assignments construct symbolic current and continuation profit. `pi2` weights both optimized terminal values by first-period share and outside share. `Pi = pi1 + delta*pi2` creates total discounted profit. Capitalization matters: Python treats `Pi` and `pi1` as unrelated names. No optimizer runs in this file; it provides the expression that FOC modules and tests inspect.

Module setup and direct-execution blocks

Source lines 1–6:

```
from model.symbols import *
pi1 = sj1*(p1-c)
profit2 = sj2*(p2-c)
pi2 = sj1*Vj + (1-sj1)*V0
Pi = pi1 + delta*pi2
DeltaV = Vj-V0
```

Read this block:

- Line 1: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2, `pi1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 3, `profit2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `pi2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `Pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `DeltaV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

supply_code/model/shares.py

This module builds symbolic logit shares from normalized utility and defines derivative expressions. The star import makes shared symbols available directly without a module prefix, which shortens formulas but makes their origins harder for beginners to track. Expressions such as `alpha*sj1*(1-sj1)` encode the logistic price derivative using share placeholders. They are algebra, not evaluated probabilities for a dataset. The explicit-share expression and the share symbol are intentionally distinct objects.

Module setup and direct-execution blocks

Source lines 1–6:

```
import sympy as sp
from model.symbols import *
from model.utility import U1, U2
sj1_expr = 1/(1+sp.exp(-U1))
sj2_expr = 1/(1+sp.exp(-U2))
s01_expr, s02_expr = 1-sj1_expr, 1-sj2_expr
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `sj1_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `sj2_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `(s01_expr, s02_expr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 7–11:

```
dsj1_dp1, dsj2_dp2 = alpha*sj1*s01, alpha*sj2*s02
ds01_dp1, ds02_dp2 = -dsj1_dp1, -dsj2_dp2
dsj1_dgamma = q*sj1*s01
dsj2_dgamma_direct = q2*sj2*s02
dsj2_dutility = sj2*s02
```

Read this block:

- Line 7, `(dsj1_dp1, dsj2_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(ds01_dp1, ds02_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 9, `dsj1_dgamma`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 10, `dsj2_dgamma_direct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 11, `dsj2_dutility`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

supply_code/model/symbols.py

`sp.symbols` constructs formal mathematical objects and tuple unpacking gives them Python names. The name displayed in equations can differ from the Python identifier, e.g. `p1` represents printed `p_1`. `real=True` records a symbolic domain; `positive=True` on shares does not also encode share below one. Economic restrictions such as alpha<0 remain assumptions used in interpretation and numeric validation. `s01 = 1-sj1` defines an expression, not another independent symbol. `z1,z2 = q,q2` aliases existing signed symbols for legacy code.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbols for a fixed type and signed realized histories."""
import sympy as sp
alpha, beta, gamma, delta, c = sp.symbols("alpha beta gamma delta c", real=True)
p1, p2, p2j, p20 = sp.symbols("p_1 p_2 p_{2j} p_{20}", real=True)
q, q2 = sp.symbols("q q_2", real=True)
sj1, sj2, u, v = sp.symbols("s s_2 u v", positive=True)
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3, `(alpha, beta, gamma, delta, c)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 4, `(p1, p2, p2j, p20)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 5, `(q, q2)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 6, `(sj1, sj2, u, v)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.symbols`: Create SymPy symbols, which stand for algebraic variables rather than observed numbers.

Source lines 7–12:

```
s01, s02 = 1-sj1, 1-sj2
Vj, V0 = sp.symbols("V_{2j} V_{20}", real=True)
h0, h1 = sp.symbols("h_0 h_1")
p1_star_sym, p2_star_sym = sp.symbols("p_1^* p_2^*")
# Legacy names are signed coefficients now, never the former 0/1 indicators.
z1, z2 = q, q2
```

Read this block:

- Line 7, `(s01, s02)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(Vj, V0)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 9, `(h0, h1)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 10, `(p1_star_sym, p2_star_sym)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 12, `(z1, z2)`: Unpack the tuple `(q, q2)` into these targets in order.

supply_code/model/utility.py

After defining its two helpers, module-level code calls `raw_utilities` for each period, unpacks inside/outside expressions, and subtracts outside utility. `sp.expand` distributes products to obtain the normalized linear index. This module combines function definitions with actual symbolic setup work at import time. It does not read private data or solve numeric prices.

Module setup and direct-execution blocks

Source lines 1–2:

```
import sympy as sp
from model.symbols import *
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 17–20:

```
raw_inside1, raw_outside1 = raw_utilities(p1, q)
raw_inside2, raw_outside2 = raw_utilities(p2, q2)
U1 = sp.expand(raw_inside1 - raw_outside1)
U2 = sp.expand(raw_inside2 - raw_outside2)
```

Read this block:

- Line 17, `(raw_inside1, raw_outside1)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 18, `(raw_inside2, raw_outside2)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 19, `U1`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.
- Line 20, `U2`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.expand`: Distribute products and combine terms in a symbolic expression without solving its unknowns.

signed state

Original function: `signed_state`. Source begins at line 5.

This converts the two accepted string state names into integer signs. Two independent `if` blocks are fine because each successful branch returns at once. Any other value reaches `raise ValueError`. `signed_state("j")` returns -1; `signed_state("0")` returns +1; numeric `0` is not string `"0"` and is rejected. A dictionary lookup could shorten this to a mapping, but these explicit branches make both accepted values and the custom error easy for a novice to see.

Definition and stated intent:

Source lines 5–5:

```
5 def signed_state(state):
```

`def` creates this callable; the parentheses name its inputs (`state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 6–7:

```
6 if state == "j":
7     return -1
```

Read this block:

- Line 6: test `state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 7: return the negated or sign-adjusted expression `-1`. This ends the call.

Source lines 8–9:

```
8 if state == "0":
9     return 1
```

Read this block:

- Line 8: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 9: return the literal value `1`. This ends the call.

Source lines 10–10:

```
10 raise ValueError("state must be 'j' or '0'")
```

Read this block:

- Line 10: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

raw utilities

Original function: `raw_utilities`. Source begins at line 13.

Input `price` may be a SymPy symbol and `signed_history` a sign or symbolic q. The return statement contains two comma-separated expressions, so it returns a tuple `(inside_utility, outside_utility)`. The combinations `(1+q)/2` and `(1-q)/2` act as complementary indicators when q is ±1. At q=-1 the first is zero and second one; at q=+1 they reverse.

Arithmetic with SymPy objects builds algebraic expressions rather than computing a numerical utility immediately. Returning both raw utilities preserves the outside switching payoff before normalization. Writing only an inside bonus would encode a different model. Module-level unpacking later names each result and subtracts the outside utility explicitly.

Definition and stated intent:

Source lines 13–13:

```
13 def raw_utilities(price, signed_history):
```

`def` creates this callable; the parentheses name its inputs (`price`, `signed_history`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 14–15:

```
14 return (beta + alpha*price + gamma*(1+signed_history)/2,
15         gamma*(1-signed_history)/2)
```

Read this block:

- Line 14: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

supply_code/pricing/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/pricing/period1_foc.py

This module creates equivalent raw and collected forms of the dynamic first derivative. Factoring `(p1-c+delta*DeltaV)` displays the effective-margin meaning. `sp.Eq(expression, 0)` constructs a symbolic equation object; it does not solve that equation or assign zero to the expression. Keeping an equation for export beside an expression for differentiation avoids manually retyping formulas.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 import sympy as sp
2 from model.symbols import *
3 from model.profit import DeltaV
4 foc1_raw = sj1 + alpha*sj1*s01*(p1-c) + delta*alpha*sj1*s01*DeltaV
5 foc1_collected = sj1 + alpha*sj1*s01*(p1-c+delta*DeltaV)
6 foc1_eq = sp.Eq(foc1_collected, 0)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.profit`, import `DeltaV` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `foc1_raw`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 5, `foc1_collected`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `foc1_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period1_price.py

`p1_star` is an implicit markup expression containing an equilibrium-share symbol and both continuation values. `sp.Eq` labels it with a formal starred price for display. It is not a numerical optimal price until those endogenous objects are solved consistently. Direct substitution of an arbitrary share would not solve the model.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p1_star = c-1/(alpha*s01)-delta*(Vj-V0)
4 p1_star_eq = sp.Eq(p1_star_sym, p1_star)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p1_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p1_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period2_foc.py

The terminal FOC contains current share plus margin times its price derivative. There is no discount term because period 2 is terminal. `sp.Eq` wraps it as an equation for presentation; actual root solving happens in `model/numerical.py`. Separating the expression permits tests to compare it with a literal derivative of profit instead of trusting a displayed equation alone.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 foc2 = sj2 + alpha*sj2*s02*(p2-c)
4 foc2_eq = sp.Eq(foc2, 0)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 3, `foc2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `foc2_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period2_price.py

The markup expression is cost minus `1/(alpha*s02)`, positive above cost for negative alpha and interior shares. Parentheses make the whole product the denominator: `1/alpha*s02` would mean `(1/alpha)*s02`, a different formula. This is another implicit representation because outside share depends on price.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p2_star = c-1/(alpha*s02)
4 p2_star_eq = sp.Eq(p2_star_sym, p2_star)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p2_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p2_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/run_all.py

`Path(__file__).resolve().parent` locates this script's directory regardless of the shell's current directory. A list of command-argument lists describes the three reproducibility steps. `[sys.executable, *command]` uses the current Python interpreter and expands each command's arguments without a shell. `cwd=ROOT` makes relative imports and outputs predictable; `check=True` raises if a step exits unsuccessfully, preventing later artifact generation after a failed check. Running a shell string with semicolons would be less portable and would require additional quoting/exit-status care.

Module setup and direct-execution blocks

Source lines 1–5:

```
1 """Reproducibility entry point: tests, requested figures, symbolic LaTeX."""
2 import subprocess
3 import sys
4 from pathlib import Path
5 ROOT=Path(__file__).resolve().parent
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 3: import `sys`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 6–11:

```
for command in [
    ['-m','unittest','-v','checks.test_symmetric'],
    ['make_comparative_static_figures.py'],
    ['make_latex_document.py'],
]:
    subprocess.run([sys.executable,*command],cwd=ROOT,check=True)
```

Read this block:

- Line 6: for each entry yielded by `[['-m', 'unittest', '-v', 'checks.test_symmetric'], ['make_com...`, bind/unpack it into `command` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 11: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 12–12:

```
print('All checks passed; figures and LaTeX regenerated. Compile the TeX with latexmk.')
```

Read this block:

- Line 12: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

supply_code/structural_switching_reference.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–10:

```
"""Two-period symmetric switching model: solve, differentiate, verify, plot.

Run: python structural_switching_reference.py
Requires: numpy, scipy, matplotlib.  Outputs to ./outputs.
```

```
"""
from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: import `csv`. An as-name is a convenient local alias, not another library.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 10: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 12–14:

```
BETA, ALPHA, DELTA, COST = 2.0, -1.0, 0.92, 0.0
GAMMAS = np.linspace(-3.0, 3.0, 601)
OUTPUT = Path('outputs')
```

Read this block:

- Line 12, `(BETA, ALPHA, DELTA, COST)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 13, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 14, `OUTPUT`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 130–131:

```
if __name__ == '__main__':
    main()
```

Read this block:

- Line 130: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 131: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit safe

Original function: `expit_safe`. Source begins at line 17.

This imports and returns SciPy's stable logistic transform. The local import occurs when the function is called; Python caches imported modules, so it is not reloading SciPy from scratch each time. A module-level import

would be more conventional here. The numerical advantage over a naive exponential ratio is avoiding overflow at extreme utility indices.

Definition and stated intent:

Source lines 17–17:

```
17 def expit_safe(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 18–18:

```
18 from scipy.special import expit
```

Read this block:

- Line 18: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–19:

```
19 return expit(x)
```

Read this block:

- Line 19: return the result of calling `expit(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

inside share

Original function: `inside_share`. Source begins at line 22.

This independent reference maps a state to a sign with nested conditional expressions, rejects invalid states, and evaluates logistic demand. The final `else None` makes invalid input detectable before multiplying it by gamma. Defaults capture the benchmark constants at function definition time. Unlike the production state helper, the sign logic is written independently to help cross-check the implementation rather than simply calling the same helper twice.

Definition and stated intent:

Source lines 22–23:

```
22 def inside_share(price, gamma, state, beta=BETA, alpha=ALPHA):
23     """state='j': existing customer, state='0': outside previously."""
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–24:

```
24 q = -1 if state == 'j' else +1 if state == '0' else None
```

Read this block:

- Line 24, `q`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 25–26:

```
25 if q is None:
26     raise ValueError("state must be 'j' or '0'")
```

Read this block:

- Line 25: test `q is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 27–27:

```
27 return expit_safe(beta + q * gamma + alpha * price)
```

Read this block:

- Line 27: return the result of calling `expit_safe(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `expit_safe`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

p2 opt

Original function: `p2_opt`. Source begins at line 30.

This reference solves the terminal markup equation using a lambda and a fixed bracket [-30,40]. Its gamma/state inputs enter through the closed-over variables. The return is the `brentq` root. It is appropriate for the reference benchmark; the production solver's expanding bracket handles a wider calibration range without assuming the root remains in this interval. Separate implementations make agreement more informative than two wrappers around the same solver.

Definition and stated intent:

Source lines 30–31:

```
30 def p2_opt(gamma, state):
31     """Unique finite price solving p=c-1/[alpha*(1-s)] (interior)."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 32–32:

```
32 f = lambda p: p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, state)))
```

Read this block:

- Line 32, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ...`.

Syntax and calls used here:

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 33–33:

```
33 return brentq(f, -30.0, 40.0, xtol=1e-13)
```

Read this block:

- Line 33: return the result of calling `brentq(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

evaluate

Original function: `evaluate`. Source begins at line 36.

This independently computes backward-induction prices, branch values, profit, and hand-written analytic gamma derivatives. Comma-separated assignments unpack small tuples; `r = 1-s` names the outside share. Conditional expressions choose the correct matched denominator, and acquisition ratios are NaN outside their applicable initial history. An `assert np.isclose(...)` checks the price decomposition internally. Production uses lambdified canonical formulas, whereas the reference writes formulas directly to provide an independent oracle. Its interface and benchmark constants are narrower than production's.

Definition and stated intent:

Source lines 36–36:

```
36 def evaluate(gamma, initial_state):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–42:

```
37 p2j, p20 = p2_opt(gamma, 'j'), p2_opt(gamma, '0')
38 u, v = inside_share(p2j, gamma, 'j'), inside_share(p20, gamma, '0')
39 Vj, V0 = (p2j-COST)*u, (p20-COST)*v
40 dV = Vj - V0
41 q = -1 if initial_state == 'j' else 1
42 f = lambda p: p-COST+1/(ALPHA*(1-inside_share(p, gamma, initial_state)))+DELTA*dV
```

Read this block:

- Line 37, `(p2j, p20)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 39, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `dV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 41, `q`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 42, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ...`.

Syntax and calls used here:

- `p2_opt`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 43–48:

```
p1 = brentq(f, -30.0, 40.0, xtol=1e-13)
s = inside_share(p1, gamma, initial_state)
r = 1-s
pi = s*(p1-COST)+DELTA*(s*Vj+r*V0)
static_piece = COST - 1/(ALPHA*r)
dynamic_piece = -DELTA*dV
```

Read this block:

- Line 43, `p1`: Set this name to the result of calling `brentq(...)`; the arguments and preceding method chain are shown above.
- Line 44, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.
- Line 45, `r`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 46, `pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `static_piece`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 48, `dynamic_piece`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

Source lines 50–56:

```
d_pi = (-q*s + DELTA*(s*u-r*v))/ALPHA
dp1 = -(q*s + DELTA*r*(u+v))/ALPHA
# ds/dgamma = s*r*(q + alpha*dp1) = s*r^2*(q - delta*(u+v)).
ds = s*r*r*(q-DELTA*(u+v))
dB = -u*(1-u)**2 + v*(1-v)**2
d2p1 = -(s*r*r*(q-DELTA*(u+v))**2 + DELTA*r*dB)/ALPHA
dp2j = -(-1)*u/ALPHA
```

Read this block:

- Line 50, `d_pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `dp1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 53, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 54, `dB`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 55, `d2p1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 56, `dp2j`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 57–61:

```
57 dp20 = -(+1)*v/ALPHA
58 matched_p2 = p2j if initial_state == 'j' else p20
59 matched_dp2 = dp2j if initial_state == 'j' else dp20
60 d_matched_ratio = (dp1*matched_p2-p1*matched_dp2)/matched_p2**2
61 d_induce_ratio = (dp1*p2j - p1*dp2j)/p2j**2 if initial_state == '0' else np.nan
```

Read this block:

- Line 57, `dp20`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58, `matched_p2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 59, `matched_dp2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 60, `d_matched_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 61, `d_induce_ratio`: Set this name to the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value.

Source lines 62–62:

```
62 assert np.isclose(p1, static_piece+dynamic_piece, atol=1e-9)
```

Read this block:

- Line 62: require `np.isclose(p1, static_piece + dynamic_piece, atol=1e-09)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 63–67:

```
63 return dict(gamma=gamma, initial_state=initial_state, p1=p1, p2j=p2j,
64             p20=p20, profit=pi, V2j=Vj, V20=V0, DeltaV=dV,
65             static_component=static_piece, continuation_adjustment=dynamic_piece,
66             dp1=dp1, d2p1=d2p1, dprofit=d_pi,
67             d_matched_ratio=d_matched_ratio, d_induce_ratio=d_induce_ratio)
```

Read this block:

- Line 63: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

verify

Original function: `verify`. Source begins at line 70.

This loops over selected gamma values and both histories. For each it re-solves at gamma+h and gamma-h, then compares analytic derivatives with centered finite differences. The step h balances approximation and floating-point error; making it arbitrarily tiny does not ensure accuracy. `assert condition, details` raises AssertionError carrying a diagnostic tuple when a check fails. Ratios must be recomputed from reoptimized price levels, not differentiated holding terminal prices fixed. This is a numerical check, not a symbolic sign proof.

Definition and stated intent:

Source lines 70–70:

```
70 def verify():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 71–71:

```
71 h = 1e-4
```

Read this block:

- Line 71, `h`: Set this name to the literal value `0.0001`.

Source lines 72–86:

```
72 for g in [-2.4, -1.2, 0, 0.7, 2.6]:
73     for state in ['j', '0']:
74         result = evaluate(g, state)
75         plus, minus = evaluate(g+h, state), evaluate(g-h, state)
76         for value, key in [('dp1', 'p1'), ('dprofit', 'profit')]:
77             numerical = (plus[key]-minus[key])/(2*h)
78             assert np.isclose(result[value], numerical, atol=2e-7), (g,state,value,result[value],numerical)
79         numerical2 = (plus['dp1']-minus['dp1'])/(2*h)
80         assert np.isclose(result['d2p1'], numerical2, atol=2e-7), (g,state,'d2p1')
81         numerical_ratio = (plus['p1']/ (plus['p2j'] if state=='j' else plus['p20']) -
82                            minus['p1']/(minus['p2j'] if state=='j' else minus['p20']))/(2*h)
83         assert np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-7)
84         if state == '0':
85             numerical_ih = (plus['p1']/plus['p2j'] - minus['p1']/minus['p2j'])/(2*h)
86             assert np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-7)
```

Read this block:

- Line 72: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 74, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 75, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 76: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit')]`, bind/unpack it into `(value, key)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 77, `numerical`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 78: require `np.isclose(result[value], numerical, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 79, `numerical2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80: require `np.isclose(result['d2p1'], numerical2, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 81, `numerical_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83: require `np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 84: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 85, `numerical_ih`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 86: require `np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 87–87:

```
87 print('All analytical derivative checks passed.')
```

Read this block:

- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot series

Original function: `plot_series`. Source begins at line 90.

This plots one result field for two initial histories and saves a raster image. `[r[key] for r in rows[state]]` extracts that field from every result dictionary. Separating the data key from axis label/filename lets one helper generate price, derivative, and ratio plots. The function expects rows already aligned to GAMMAS. Its outputs belong to the retained reference program, not the six-figure production path used by `run_all.py`.

Definition and stated intent:

Source lines 90–90:

```
90 def plot_series(rows, key, ylabel, filename, title):
```

`def` creates this callable; the parentheses name its inputs (`rows`, `key`, `ylabel`, `filename`, `title`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 91–91:

```
91 fig, ax = plt.subplots(figsize=(8.6, 5.2))
```

Read this block:

- Line 91, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 92–93:

```
92 for state,label in [('j',r'Initially inside ($x_0=j$)'),('0',r'Initially outside ($x_0=0$)')]:
93     ax.plot(GAMMAS, [r[key] for r in rows[state]], label=label, linewidth=2)
```

Read this block:

- Line 92: for each entry yielded by `[('j', 'Initially inside ($x_0=j$)'), ('0', 'Initially outside...`, bind/unpack it into `(state, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 93: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 94–99:

```
94 ax.axhline(0, color='gray', linewidth=.8)
95 ax.set(xlabel=r'$\gamma$', ylabel=ylabel, title=title)
96 ax.legend()
97 fig.tight_layout()
98 fig.savefig(OUTPUT/filename, dpi=180)
99 plt.close(fig)
```

Read this block:

- Line 94: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 102.

This first verifies the reference formulas, creates its `outputs` directory, solves benchmark rows, writes CSV, and saves several reference plots. `with ... open(..., newline="")` lets the CSV writer handle line endings correctly. Some lines contain semicolons separating multiple statements; Python executes them left-to-right just as if written

on separate lines. That compact style saves space but is less friendly to beginners. Production uses another entry point and another output folder; running this does not replace `run_all.py`.

Definition and stated intent:

Source lines 102–102:

```
102 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 103–105:

```
103 verify()
104 OUTPUT.mkdir(exist_ok=True)
105 rows = {state:[evaluate(float(g),state) for g in GAMMAS] for state in ['j','0']}
```

Read this block:

- Line 103: call `verify` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104: call `OUTPUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105, `rows`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `verify`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 106–110:

```
106 with (OUTPUT/'model_results.csv').open('w', newline='') as file:
107     writer = csv.DictWriter(file, fieldnames=list(rows['j'][0]))
108     writer.writeheader()
109     for state in ['j','0']:
110         writer.writerows(rows[state])
```

Read this block:

- Line 106: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 107, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 108: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 110: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.

- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 111–116:

```
plot_series(rows, 'p1', r'$p_1^*$', 'price1.png', 'Optimal period-1 prices')
plot_series(rows, 'dp1', r'$dp_1^*/d\gamma$', 'dp1.png', 'First derivative of period-1 price')
plot_series(rows, 'd2p1', r'$d^2p_1^*/d\gamma^2$', 'd2p1.png', 'Second derivative of period-1 price (analytic)')
plot_series(rows, 'dprofit', r'$d\Pi^*/d\gamma$', 'dprofit.png', 'Optimized profit derivative')
plot_series(rows, 'd_matched_ratio', r'$d(p_1/p_2)/d\gamma$', 'matched_price_ratio.png', 'State-matched
 ↪  price-ratio derivatives')
fig,ax=plt.subplots(figsize=(8.6,5.2))
```

Read this block:

- Line 111: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 114: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 115: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `plot_series`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 117–120:

```
ax.plot(GAMMAS, [r['d_induce_ratio'] for r in rows['0']], linewidth=2)
ax.axhline(0,color='gray',linewidth=.8)
ax.set(xlabel=r'$\gamma$', ylabel=r'$d[p_1(0)/p_2(j)]/d\gamma$', title='Induce-then-harvest price-ratio
 ↪  derivative')
fig.tight_layout();fig.savefig(OUTPUT/'induce_harvest_ratio.png',dpi=180);plt.close(fig)
```

Read this block:

- Line 117: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

Source lines 121–127:

```
121 for state in ['j','0']:
122     fig,ax=plt.subplots(figsize=(8.6,5.2))
123     for key,label in [('p1',r'$p_1^*$'),('static_component','Static FOC
        ↪ component'),('continuation_adjustment',r'$-\delta\Delta V_2$')]:
124         ax.plot(GAMMAS,[r[key] for r in rows[state]],label=label,linewidth=2)
125     ax.axhline(0,color='gray',linewidth=.8)
126     ax.set(xlabel=r'$\gamma$',ylabel='Price / component',title=f'First-period decomposition: initial state
    ↪ {state}')
127     ax.legend();fig.tight_layout();fig.savefig(OUTPUT/f'decomposition_{state}.png',dpi=180);plt.close(fig)
```

Read this block:

- Line 121: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 123: for each entry yielded by `[('p1', '$p_1^*$'), ('static_component', 'Static FOC component...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.legend`: Render labels previously attached to plotted series, identifying curves.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 128–128:

```
128 print('Saved outputs to', OUTPUT.resolve())
```

Read this block:

- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises: source walkthrough

pricing_exercises/run_empirical_targeting.py

Earlier combined-gamma berry-only pricing comparison. It does not accept the split fit. Use run_split_targeting.py or final_simple/demand/split_pricing.py for the current model. Its .95 revenue discount default differs from the new split exercise's .92.

Module setup and direct-execution blocks

Source lines 2–13:

```
2 """Standalone empirical berry targeting, separate from demand estimation.
3
4 User-supplied paths only. No estimation is performed. --self-test uses only
5 generated synthetic inputs. See EMPIRICAL_TARGETING.md for definitions.
6 """
7 from __future__ import annotations
8
9 # Empirical I/O is permitted only on the research server, never on this computer.
10 from pathlib import Path as _PolicyPath
11 import sys as _policy_sys
12 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
13 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 12, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 13: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–19:

```
from server_data_policy import server_only, require_server
import argparse
import hashlib
import json
import math
import os
```

Read this block:

- Line 14: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 16: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 17: import `json`. An as-name is a convenient local alias, not another library.
- Line 18: import `math`. An as-name is a convenient local alias, not another library.
- Line 19: import `os`. An as-name is a convenient local alias, not another library.

Source lines 20–26:

```
from pathlib import Path
import sys
import tempfile
import numpy as np
import pandas as pd

FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 20: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 21: import `sys`. An as-name is a convenient local alias, not another library.
- Line 22: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 23: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 24: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 26, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 582–583:

```
if __name__ == "__main__":
    main()
```

Read this block:

- Line 582: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 583: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 28.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 28–28:

```
28 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–30:

```
29 x = np.asarray(x)
30 small = np.exp(-np.abs(x))
```

Read this block:

- Line 29, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 30, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 31–31:

```
31 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 31: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 33.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 33–33:

```
33 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–36:

```
34 x = np.asarray(x)
35 maximum = np.max(x, axis=axis, keepdims=True)
36 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 34, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 35, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 36, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 37–37:

```
37 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 37: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 39.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 39–39:

```
39 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–43:

```
40 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
41 import matplotlib
42 matplotlib.use("Agg")
43 import matplotlib.pyplot as plt
```

Read this block:

- Line 40: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 42: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 44–44:

```
44 return plt
```

Read this block:

- Line 44: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 46.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 46–46:

```
46 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–48:

```
47 if number < 2 or number % 2:
48     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 47: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 49–49:

```
49 half = np.random.default_rng(seed).standard_normal((number//2, 2))
```

Read this block:

- Line 49, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 50–50:

```
50 return np.vstack([half, -half])
```

Read this block:

- Line 50: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 52.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 52–53:

```
52 def posterior(frame, parameters, draws, control=True):
53     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 54–59:

```
54 labels = np.sort(frame.household_code.unique())
55 hh = pd.Categorical(frame.household_code, categories=labels).codes
56 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
57 available = np.isfinite(prices) & (prices > 0)
58 counts = frame[FLAVORS+["outside"]].to_numpy(float)
59 xi = frame[["xi_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 54, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 55, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 56, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 57, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 59, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 60–60:

```
60 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 60, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 61–64:

```
61 if (not np.isfinite(counts).all() or (counts < 0).any()
62         or not np.isfinite(xi).all() or not np.isfinite(cf).all()
63         or ((counts[:, :3] > 0) & ~available).any()):
64     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 61: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(xi).a...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 64: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 65–68:

```
65 beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
66 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
67 gamma = parameters["mean_gamma"] + parameters["sd_gamma"]*draws[:, 1]
68 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 65, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 67, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 69–74:

```
for d, (a, g) in enumerate(zip(alpha, gamma)):
    u = beta-a*np.nan_to_num(prices)+g*xi+parameters.get("price_control", 0.)*cf
    u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
    logp = u-logsumexp(u, axis=1, keepdims=True)
    ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
    loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 69: for each entry yielded by `enumerate(zip(alpha, gamma))`, bind/unpack it into `(d, (a, g))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 70, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 71, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 72, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 73, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 74, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 75–81:

```
weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
means = weights @ gamma
# Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
order = np.argsort(means, kind="stable")
groups = np.full(len(labels), "middle", dtype=object)
n = len(labels)//4
```

Read this block:

- Line 75, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 76, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.

- Line 77, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 79, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 80, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 81, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 82–83:

```
82 if n < 1:
83     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 82: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 83: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 84–90:

```
84 groups[order[:n]], groups[order[-n:]] = "low", "high"
85 population_q = np.quantile(gamma, [.25, .75])
86 table = pd.DataFrame(dict(household_code=labels, group=groups, gamma_mean=means,
87                           gamma_sd=np.sqrt(np.maximum(weights @ gamma**2-means**2, 0)),
88                           alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
89                           probability_gamma_below_draw_q25=weights @ (gamma <= population_q[0]),
90                           probability_gamma_above_draw_q75=weights @ (gamma >= population_q[1])))
```

Read this block:

- Line 84, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 85, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 86, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 91–91:

```
91 return table, weights, alpha, gamma, beta, nll
```

Read this block:

- Line 91: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 93.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 93–94:

```
93 def compositions(capacity):
94     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 95–96:

```
95 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
96                  for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 95: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 98.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 98–103:

```
98 def forecast_context(frame, households):
99     """First observed choice establishes h0; price the next two consecutive weeks.
100
101    Carry the initial observation's menu, posted prices and controls forward.
102    Type estimation still uses the complete observed sample.
103    """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 104–110:

```
104 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
105 first = first.set_index("household_code").loc[households.household_code]
106 counts = first[FLAVORS].to_numpy(float)
```

```
totals = counts.sum(axis=1, keepdims=True)
shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
# Saved stocks precede the observed choice; add its basket exactly once.
stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 104, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 105, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 106, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 107, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 108, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 110, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 111–116:

```
novelty = shares.sum(axis=1, keepdims=True)-shares
prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
available = np.isfinite(prices) & (prices > 0)
eligible = available[:, 1]
households["berry_eligible"] = eligible
initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 111, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 112, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 113, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 114, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 115, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 116, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 117–121:

```
households["initial_observed_week"] = initial_weeks
households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
cf_cols = ["cf_"+f for f in FLAVORS]
cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 117, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 118, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 119, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 120, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 121, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 122–123:

```
if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
    raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 122: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 123: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 124–125:

```
return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
            available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 124: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 127.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 127–127:

```
127 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 128–128:

```
128 h = hashlib.sha256()
```

Read this block:

- Line 128, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 129–131:

```
129 with path.open("rb") as handle:
130     for chunk in iter(lambda: handle.read(1024*1024), b""):
131         h.update(chunk)
```

Read this block:

- Line 129: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 130: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 131: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 132–132:

```
132 return h.hexdigest()
```

Read this block:

- Line 132: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 136.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits

one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1+b_S$S1, history2=b_N$N2+b_S$S2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 136–146:

```
136 def targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
137                      delta=1., capacity=7, habit_weight=.7, control_coefficient=0.,
138                      batch_size=24, type_treatment="posterior-mean", state_output=None):
139     """Observed h0, then p1(h0), realized h1, and terminal p2(h1).
140 
141     The observed initialization and two priced choices form three choice dates.
142     Individual backward induction; gamma quartiles are reporting groups only.
143 
144     Baseline: posted prices in both weeks. Dynamic: choose first coupon accounting for optimized
145     state-contingent terminal revenue. The seller observes the full basket.
146     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `gamma`, `beta`, `discounts`, `delta`, `capacity`, `habit_weight`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 147–152:

```
147 baskets = compositions(capacity)
148 inside = baskets[:, :3]
149 totals = inside.sum(axis=1, keepdims=True)
150 shares = np.divide(inside, totals, out=np.zeros_like(inside, dtype=float), where=totals > 0)
151 coeff = math.lgamma(capacity+1)-np.vectorize(math.lgamma)(baskets+1).sum(axis=1)
152 k, nstates = len(discounts), len(baskets)
```

Read this block:

- Line 147, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 148, `inside`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 149, `totals`: Set this name to the result of calling `inside.sum(...)`; the arguments and preceding method chain are shown above.
- Line 150, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 151, `coeff`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 152, `(k, nstates)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 153–156:

```
rows, maximum_mass_error, wrote_states = [], 0., False
indices = np.flatnonzero(context["eligible"])
# Only the two tails receive targeting; middle households remain at posted prices.
targeted = households.group.isin(["low", "high"]).to_numpy()
```

Read this block:

- Line 153, `(rows, maximum_mass_error, wrote_states)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 154, `indices`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 156, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 157–198:

```
for start in range(0, len(indices), batch_size):
    ids = indices[start:start+batch_size]
    h = len(ids)
    prices, availability = context["prices"][ids], context["available"][ids]
    stock, novelty, cf = context["stocks"][ids], context["novelty"][ids], context["cf"][ids]
    if type_treatment == "posterior-mean":
        # Base supply information assumption: an estimated fixed type is known.
        a, g = (weights[ids] @ alpha)[:, None], (weights[ids] @ gamma)[:, None]
        w = np.ones((h, 1))
    else:
        a = np.broadcast_to(alpha, (h, len(alpha)))
        g = np.broadcast_to(gamma, (h, len(gamma)))
        w = weights[ids]
    xi1 = (1-habit_weight)*novelty-habit_weight*stock
    xi2 = (1-habit_weight)*(shares.sum(axis=1, keepdims=True)-shares)[None, :, :] - habit_weight*(stock[:, None,
    :]+shares[None, :, :])
    u1 = beta[None, None, :]-a[..., None]*prices[:, None, :]+g[..., None]*xi1[:, None, :]+control_coefficient*cf[:,
    None, :]
    u2 = beta[None, None, None, :]-a[:, :, None, None]*prices[:, None, None, :]+g[:, :, None, None]*xi2[:, None, :,
    :]+control_coefficient*cf[:, None, None, :]
    u1 = np.where(availability[:, None, :], u1, -np.inf)
    u2 = np.where(availability[:, None, None, :], u2, -np.inf)
    terminal = []
    for discount in discounts:
        u = u2.copy()
        u[..., 1] += a[:, :, None]*prices[:, None, None, 1]*discount
        denominator = logsumexp(np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1), axis=-1)
        quantity = capacity*np.exp(u[..., 1]-denominator)
        terminal.append(np.stack([quantity*prices[:, None, None, 1]*(1-discount), quantity,
                                  quantity*prices[:, None, None, 1]*discount], axis=-1))
    # Metrics: net revenue, berry units, coupon expenditure.
    first = np.zeros((h, k, 3))
    second = np.zeros((h, k, 3))
    stage0_second = None
    paths = []
```

```
    for i, discount in enumerate(discounts):
        u = u1.copy()
        u[..., 1] += a*prices[:, None, 1]*discount
        full = np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1)
        logp = full-logsumexp(full, axis=-1, keepdims=True)
        terms = np.zeros((h, a.shape[1], nstates, 4))
        np.multiply(baskets[None, None, :, :], logp[:, :, None, :], out=terms,
                    where=baskets[None, None, :, :] > 0)
        probability = np.exp(coeff[None, None, :]+terms.sum(axis=-1))
        maximum_mass_error = max(maximum_mass_error, float(np.abs(probability.sum(axis=-1)-1).max()))
```

Source lines 199–240:

```
    joint = w[:, :, None]*probability
    state_probability = joint.sum(axis=1)
    quantity = capacity*(w*np.exp(logp[..., 1])).sum(axis=1)
    first[:, i] = np.stack([quantity*prices[:, 1]*(1-discount), quantity,
                            quantity*prices[:, 1]*discount], axis=-1)
    # Integrate types BEFORE choosing a terminal coupon: no foresight of
    # an unobserved true draw in the posterior-integrated sensitivity.
    metrics = np.stack([np.einsum("hds,hdsm->hsm", joint, candidate)
                        for candidate in terminal], axis=2)  # household,state,action,metric
    best = metrics[..., 0].argmax(axis=2)
    selected = np.take_along_axis(metrics, best[:, :, None, None], axis=2)[:, :, 0, :]
    # E_h1[p2(h1)*E(q2|h1,p2,type)] under this candidate p1(h0).
    # Optimize p2 separately in each state BEFORE summing its revenue.
    second[:, i] = selected.sum(axis=1)
    if i == 0:
        stage0_second = metrics[:, :, 0, :].sum(axis=1)
    # Storing these compact arrays allows exporting only chosen first actions.
    paths.append((best, selected, state_probability))
zero_choice = np.zeros(h, dtype=int)
dynamic_choice = (first[..., 0]+delta*second[..., 0]).argmax(axis=1)
dynamic_choice = np.where(targeted[ids], dynamic_choice, 0)
for name, choice in [("no_coupon", zero_choice), ("personalized_dynamic", dynamic_choice)]:
    for local, hh in enumerate(ids):
        action = int(choice[local])
        initial = first[local, action]
        is_target = targeted[hh] and name != "no_coupon"
        terminal_metrics = second[local, action] if is_target else stage0_second[local]
        best, selected, probability = paths[action]
        state_coupon = discounts[best[local]] if is_target else np.zeros(nstates)
        mass = probability[local]
        p1 = prices[local, 1]*(1-discounts[action])
        p2 = prices[local, 1]*(1-state_coupon)
        mean_p2 = float(mass @ p2)
        ratio = np.divide(p1, p2, out=np.full(nstates, np.nan), where=p2 > 0)
        positive_mass = mass > 1e-14
        expected_ratio = float(mass[positive_mass] @ ratio[positive_mass]) if
    ↪ np.isfinite(ratio[positive_mass]).all() else np.nan
        row = dict(household_code=households.iloc[hh].household_code,
                   group=households.iloc[hh].group, policy=name,
                   discount1=discounts[action], expected_discount2=float(mass @ state_coupon),
                   p1=p1, expected_p2=mean_p2,
                   ratio_of_expected_prices=p1/mean_p2 if mean_p2 > 0 else np.nan,
                   expected_price_ratio=expected_ratio,
```

Source lines 241–259:

```
                   revenue1=initial[0], revenue2=terminal_metrics[0],
                   discounted_revenue=initial[0]+delta*terminal_metrics[0],
                   berry_units1=initial[1], berry_units2=terminal_metrics[1],
```

```
                    coupon_expenditure=initial[2]+delta*terminal_metrics[2])
        for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
            branch_mass = mass[branch].sum()
            row[f"probability_{label}"] = branch_mass
            row[f"mean_p2_given_{label}"] = float(mass[branch] @ p2[branch]/branch_mass) if branch_mass > 0 else
    ↪ np.nan
        rows.append(row)
        if state_output is not None and is_target:
            detail = pd.DataFrame(dict(household_code=row["household_code"], group=row["group"], policy=name,
                                       state=np.arange(nstates), other=baskets[:, 0], berry=baskets[:, 1],
                                       plain=baskets[:, 2], outside=baskets[:, 3], probability=mass,
                                       discount1=discounts[action], discount2=state_coupon, p1=p1, p2=p2,
    ↪ price_ratio=ratio))
            detail.drop(columns=["household_code"]).to_csv(state_output, mode="a" if wrote_states else "w",
    ↪ header=not wrote_states,
                           index=False, compression="gzip")
            wrote_states = True
if start % (batch_size*40) == 0 or start+batch_size >= len(indices):
    print(f"Targeted pricing: integrated {min(start+batch_size,len(indices))}/{len(indices)} eligible households",
    ↪ flush=True)
```

Read this block:

- Line 157: for each entry yielded by `range(0, len(indices), batch_size)`, bind/unpack it into `start` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 158, `ids`: Set this name to the entry/slice selected by `indices[start:start + batch_size]`; brackets select from the object to their left.
- Line 159, `h`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 160, `(prices, availability)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 161, `(stock, novelty, cf)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 162: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 164, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 165, `w`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 167, `a`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 168, `g`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 169, `w`: Set this name to the entry/slice selected by `weights[ids]`; brackets select from the object to their left.
- Line 170, `xi1`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 171, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 172, `u1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 173, `u2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 174, `u1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 175, `u2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 176, `terminal`: Set this name to the list `[]`.
- Line 177: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 178, `u`: Set this name to the result of calling `u2.copy(...)`; the arguments and preceding method chain

are shown above.
- Line 179: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 180, `denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 181, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 182: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 185, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 186, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 187, `stage0_second`: Set this name to the literal value `None`.
- Line 188, `paths`: Set this name to the list `[]`.
- Line 189: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(i, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 190, `u`: Set this name to the result of calling `u1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 191: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 192, `full`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 193, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 194, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 195: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 197, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 198, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 199, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 200, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 201, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 202, `first[:, i]`: Store the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 206, `metrics`: Set this name to the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above.
- Line 208, `best`: Set this name to the result of calling `metrics[..., 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 209, `selected`: Set this name to the entry/slice selected by `np.take_along_axis(metrics, best[:, :, None, None], axis=...`; brackets select from the object to their left.
- Line 212, `second[:, i]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 213: test `i == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 214, `stage0_second`: Set this name to the result of calling `metrics[:, :, 0, :].sum(...)`; the arguments and preceding method chain are shown above.
- Line 216: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 217, `zero_choice`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding

method chain are shown above.
- Line 218, `dynamic_choice`: Set this name to the result of calling `.argmax(...)`; the arguments and preceding method chain are shown above.
- Line 219, `dynamic_choice`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 220: for each entry yielded by `[('no_coupon', zero_choice), ('personalized_dynamic', dynamic_...`, bind/unpack it into `(name, choice)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 221: for each entry yielded by `enumerate(ids)`, bind/unpack it into `(local, hh)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 222, `action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 223, `initial`: Set this name to the entry/slice selected by `first[local, action]`; brackets select from the object to their left.
- Line 224, `is_target`: Set this name to the Boolean condition `targeted[hh] and name != 'no_coupon'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 225, `terminal_metrics`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 226, `(best, selected, probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 227, `state_coupon`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 228, `mass`: Set this name to the entry/slice selected by `probability[local]`; brackets select from the object to their left.
- Line 229, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 230, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 231, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 232, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 233, `positive_mass`: Set this name to the Boolean condition `mass > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 234, `expected_ratio`: Set this name to the true-branch value if `np.isfinite(ratio[positive_mass]).all()` holds, otherwise the else-branch value.
- Line 235, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 245: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 246, `branch_mass`: Set this name to the result of calling `mass[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 247, `row[f'probability_{label}']`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 248, `row[f'mean_p2_given_{label}']`: Store the true-branch value if `branch_mass > 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 249: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 250: test `state_output is not None and is_target`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 251, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 255: call `detail.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 257, `wrote_states`: Set this name to the literal value `True`.

- Line 258: test `start % (batch_size * 40) == 0 or start + batch_size >= len(indices)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 259: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 260–261:

```
260 if maximum_mass_error > 1e-9:
261     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 260: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 261: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 262–267:

```
262 table = pd.DataFrame(rows)
263 baseline = table[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
264 table["baseline_revenue"] = table.household_code.map(baseline)
265 table["revenue_change"] = table.discounted_revenue-table.baseline_revenue
266 table["revenue_change_pct"] = 100*table.revenue_change/table.baseline_revenue
267 wide = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 262, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 263, `baseline`: Set this name to the attribute `table[table.policy == 'no_coupon'].set_index('household_c...` of the object before the dot.
- Line 264, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 265, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 266, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 267, `wide`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 268–269:

```
268 if (wide.personalized_dynamic+1e-8 < wide.no_coupon).any():
269     raise AssertionError("Household dynamic policy does not dominate the feasible no-coupon baseline.")
```

Read this block:

- Line 268: test `(wide.personalized_dynamic + 1e-08 < wide.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 269: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 270–270:

```
270 return table, maximum_mass_error
```

Read this block:

- Line 270: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

run households

Original function: `run_households`. Source begins at line 273.

Reads an explicitly selected fitted-results directory and its weekly table, checks the model, convergence, bounds, capacity and input schema, then recovers posterior types and reproduces the saved likelihood. For a split run, it permutes the selected focal flavor into the solver's berry coordinate, moving counts, prices, stocks, novelty, controls and intercepts consistently. Type recovery happens before permutation using the original fitted flavor labels.

It calls the household solver, renames public berry-coordinate columns to focal names, and writes policy/group/aggregate tables and figures. The manifest records how rival state coordinates map back to flavors. Swapping only prices or only counts would silently price the wrong product. The three runs are separate focal sellers with fixed rival prices, not a simultaneous equilibrium.

Definition and stated intent:

Source lines 273–275:

```
273 def run_households(args):
274     # This branch is entered only when the person running the script supplies
275     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 276–276:

```
276 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 276, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 277–278:

```
277 if fit["model"] != "heterogeneous" or not fit.get("converged", False):
278     raise ValueError("This exercise requires a converged single-normal heterogeneous-gamma fit.")
```

Read this block:

- Line 277: test `fit['model'] != 'heterogeneous' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 278: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 279–279:

```
279 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 279, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 280–281:

```
280 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
281     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 280: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 281: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 282–284:

```
282 parameters = dict(zip(estimates.parameter, estimates.estimate))
283 settings = fit["settings"]
284 capacity = settings["capacity"]
```

Read this block:

- Line 282, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 283, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 284, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 285–286:

```
285 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
286     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 285: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 286: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 287–290:

```
287 frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
288 required = ["household_code", "week", "outside", *FLAVORS]
289 required += [prefix+f for prefix in ["price_", "stock_", "novelty_", "xi_"] for f in FLAVORS]
290 control = not settings.get("no_price_control", False)
```

Read this block:

- Line 287, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 288, `required`: Set this name to a list containing 4 displayed entries.

- Line 289: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 290, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 291–292:

```
291 if control:
292     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 291: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 292: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 293–293:

```
293 missing = set(required)-set(frame)
```

Read this block:

- Line 293, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 294–295:

```
294 if missing:
295     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 294: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 296–297:

```
296 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪  "week"]).any():
297     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 296: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 297: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 298–299:

```
298 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
299     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 298: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 299: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 300–300:

```
300 weight = settings.get("habit_weight", .7)
```

Read this block:

- Line 300, `weight`: Set this name to the result of calling `settings.get(...)`; the arguments and preceding method chain are shown above.

Source lines 301–304:

```
301 for f in FLAVORS:
302     expected = (1-weight)*frame["novelty_"+f]-weight*frame["stock_"+f]
303     if not np.allclose(expected, frame["xi_"+f], atol=1e-8):
304         raise ValueError("Saved Xi does not match the fit's history definition.")
```

Read this block:

- Line 301: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 302, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 303: test `not np.allclose(expected, frame['xi_' + f], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 304: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 305–306:

```
305 draws = persistent_draws(settings["draws"], settings["seed"])
306 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 305, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.

- Line 306, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 308–309:

```
308 if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
309     raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 308: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 309: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 310–311:

```
310 context = forecast_context(frame, households)
311 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 310, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 311, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 312–313:

```
312 if any(n == 0 for n in counts):
313     raise ValueError("Each gamma segment must have berry-eligible forecast households.")
```

Read this block:

- Line 312: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 313: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 314–325:

```
314 table, mass_error = targeted_coupons(
315     context, households, weights, alpha, gamma, beta, args.discounts,
316     args.demand_delta, capacity, weight,
317     parameters.get("price_control", 0.) if control else 0., args.batch_size,
318     args.type_treatment,
319     args.output / "targeted_state_policies.csv.gz" if args.save_state_policies else None)
320 households.drop(columns=["household_code"]).to_csv(args.output / "household_posterior_summary.csv", index=False)
321 table.drop(columns=["household_code"]).to_csv(args.output / "targeted_household_results.csv", index=False)
322 totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
323           "berry_units1", "berry_units2", "coupon_expenditure"]
324 aggregate = table.groupby("policy", sort=False)[totals].sum()
325 aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
```

Read this block:

- Line 314, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 320: call `households.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 321: call `table.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 322, `totals`: Set this name to a list containing 8 displayed entries.
- Line 324, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 325, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 326–332:

```
326 aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
327 group = table.groupby(["policy", "group"], sort=False)[totals].sum()
328 group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
329 means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
330                                              "ratio_of_expected_prices", "expected_price_ratio"]].mean()
331 group = group.join(means)
332 group["eligible_households"] = table.groupby(["policy", "group"]).size()
```

Read this block:

- Line 326: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 327, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 328, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 329, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.

- Line 331, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 332, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 333–342:

```
group.to_csv(args.output / "targeted_revenue_by_gamma_group.csv")
households.groupby("group").agg(households=("household_code", "size"),
                                berry_eligible=("berry_eligible", "sum"),
                                gamma_mean=("gamma_mean", "mean"),
                                gamma_mean_min=("gamma_mean", "min"),
                                gamma_mean_max=("gamma_mean", "max"),
                                mean_posterior_sd=("gamma_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(args.output /
 ↪  "household_group_summary.csv")
make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta)
```

Read this block:

- Line 333: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 334: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 342: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 343–353:

```
return dict(households=len(households), eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
            reconstructed_nll=nll, basket_mass_error=mass_error,
            input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
            parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
            posterior_draws=len(draws), posterior_seed=settings["seed"],
            capacity=capacity, habit_weight=weight, price_control=control,
            initial_history="first_observed_basket", pricing_periods=2,
            baskets=len(compositions(capacity)),
            discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
            type_treatment=args.type_treatment,
            aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 343: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.

- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 356.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 356–357:

```
356 def make_targeting_figures(households, table, aggregate, output, delta=.95):
357     """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 358–363:

```
358 plt = plotting()
359 groups = ["low", "middle", "high"]
360 labels = ["Low gamma\n(bottom 25%)", "Middle gamma\n(middle 50%)", "High gamma\n(top 25%)"]
361 colors = ["#3575AD", "#999999", "#D17925"]
362 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
363 bins = np.histogram_bin_edges(households.gamma_mean, bins=30)
```

Read this block:

- Line 358, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 359, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 360, `labels`: Set this name to the list `['Low gamma\n(bottom 25%)', 'Middle gamma\n(middle 50%)',...`.
- Line 361, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 362, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 363, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 364–366:

```
364 for g, label, color in zip(groups, labels, colors):
365     axes[0].hist(households.loc[households.group == g, "gamma_mean"],
366                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 364: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 365: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 367–374:

```
367 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $\gamma$)",
368             ylabel="Number of households")
369 axes[0].legend(fontsize=9)
370 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
371 x = np.arange(len(groups))
372 # Average over state-contingent terminal prices first, discount, then total.
373 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
374 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 367: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 369: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 370, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 371, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 373, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 374: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 375–381:

```
375 axes[1].set_xticks(x, labels)
376 axes[1].set(title="Total Expected Revenue",
377             xlabel="Consumer type group", ylabel="Total Expected Revenue")
378 axes[1].set_ylim(bottom=0)
379 axes[1].grid(axis="y", alpha=.2)
380 axes[1].set_axisbelow(True)
381 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 375: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 376: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 378: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 379: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 380: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 381: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 382–386:

```
382 fig.text(.5, .015,
383          f"First-period revenue + {delta:g} × expected terminal revenue. "
384          "Low/high types receive personalized berry offers; middle types receive no coupons.",
385          ha="center", fontsize=9)
386 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 382: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 386: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 387–388:

```
387 for extension in ["png", "pdf"]:
388     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 387: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 388: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 389–396:

```
389 plt.close(fig)
390
391 fig, ax = plt.subplots(figsize=(9, 5))
392 x = np.arange(3)
```

```
means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    "expected_discount2"]].mean().reindex(groups)
ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
ax.bar(x+.18, 100*means.expected_discount2, width=.36,
       label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 389: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 391, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 392, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 393, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 394: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 395: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 397–403:

```
ax.set_xticks(x, labels)
ax.set(xlabel="Consumer type group", ylim=(0, 105),
       ylabel="Mean optimal berry discount (% of posted price)")
ax.grid(axis="y", alpha=.2)
ax.set_axisbelow(True)
ax.legend(fontsize=8, loc="upper left")
fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 397: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 398: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 400: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 401: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 402: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 403: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 404–406:

```
fig.text(.5, .015, "Household-weighted means among berry-eligible households. Middle group receives no coupons by
    policy design.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 404: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 406: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 407–408:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 407: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 408: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 409–413:

```
plt.close(fig)

dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
    households[["household_code", "gamma_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 409: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 411, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 413, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 414–426:

```
for column, (variable, xlabel) in enumerate([
        ("gamma_mean", r"Estimated household type (posterior mean $\gamma$)"),
        ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
    for row, (discount, title) in enumerate([
            ("discount1", r"Optimal discount at $p_1(h_0)$"),
            ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
        ax = axes[row, column]
        for g, label, color in zip(groups, labels, colors):
            subset = dynamic.loc[dynamic.group == g]
            ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
                       color=color, label=label.replace("\n", " "))
        ax.set(xlabel=xlabel, ylabel="Berry discount (% of posted price)", title=title, ylim=(-2, 102))
        ax.grid(alpha=.2)
```

Read this block:

- Line 414: for each entry yielded by `enumerate([('gamma_mean', 'Estimated household type (posterior...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 417: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 420, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 421: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 422, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 423: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 425: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 426: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 427–431:

```
427 axes[0, 0].legend(fontsize=8)
428 fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
429 fig.text(.5, .015, "Each point is one berry-eligible household. Terminal offers average over the household's
    ↪  realized h1 histories.",
430          ha="center", fontsize=9)
431 fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 427: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 428: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 429: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 431: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 432–433:

```
432 for extension in ["png", "pdf"]:
433     fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 432: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 433: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 434–434:

```
434 plt.close(fig)
```

Read this block:

- Line 434: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 437.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 437–438:

```
437 def self_test():
438     """Generated synthetic households only; no user-supplied input is accessed."""
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 439–443:

```
439 rng = np.random.default_rng(41)
440 parameters = dict(inside_constant=.8, beta_berry=-.2, beta_plain=-.1,
441                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
442                   mean_gamma=-.1, sd_gamma=.2)
443 rows = []
```

Read this block:

- Line 439, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 440, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 443, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 444–457:

```
444 for household in range(12):
445     stock, previous = np.zeros(3), np.zeros(3)
446     for t in range(3):
447         counts = rng.multinomial(2, [.25, .3, .2, .25])
448         novelty = previous.sum()-previous
449         row = dict(household_code=f"synthetic_{household:02}",
450                    week=pd.Timestamp("2020-01-04")+pd.Timedelta(weeks=t), outside=counts[3])
451         for j, flavor in enumerate(FLAVORS):
452             row.update({flavor: counts[j], "price_"+flavor: 1.+.1*j,
453                         "stock_"+flavor: stock[j], "novelty_"+flavor: novelty[j],
454                         "xi_"+flavor: .3*novelty[j]-.7*stock[j]})
455         rows.append(row)
456         previous = counts[:3]/max(counts[:3].sum(), 1)
457         stock += previous
```

Read this block:

- Line 444: for each entry yielded by `range(12)`, bind/unpack it into `household` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 445, `(stock, previous)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 446: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 447, `counts`: Set this name to the result of calling `rng.multinomial(...)`; the arguments and preceding method chain are shown above.
- Line 448, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 449, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 451: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 452: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 455: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 456, `previous`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 457: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 458–460:

```
458 frame = pd.DataFrame(rows)
459 draws = persistent_draws(8, 19)
460 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, False)
```

Read this block:

- Line 458, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 459, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 460, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 461–461:

```
461 assert np.allclose(weights.sum(axis=1), 1.)
```

Read this block:

- Line 461: require `np.allclose(weights.sum(axis=1), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError

checks are preferable for essential input validation.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 462–462:

```
462 scalar_ll = []
```

Read this block:

- Line 462, `scalar_ll`: Set this name to the list `[]`.

Source lines 463–468:

```
463 for a, g in zip(alpha, gamma):
464     value = 0.
465     for row in rows[:3]:
466         u = [beta[j]-a*row["price_"+f]+g*row["xi_"+f] for j, f in enumerate(FLAVORS)]+[0.]
467         value += np.array([row[f] for f in FLAVORS]+[row["outside"]]) @ (np.array(u)-logsumexp(u))
468     scalar_ll.append(value)
```

Read this block:

- Line 463: for each entry yielded by `zip(alpha, gamma)`, bind/unpack it into `(a, g)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 464, `value`: Set this name to the literal value `0.0`.
- Line 465: for each entry yielded by `rows[:3]`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 466, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 467: update `value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 468: call `scalar_ll.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 469–469:

```
469 assert np.allclose(weights[0], np.exp(scalar_ll-logsumexp(scalar_ll)))
```

Read this block:

- Line 469: require `np.allclose(weights[0], np.exp(scalar_ll - logsumexp(scalar_ll)))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 470–471:

```
context = forecast_context(frame, households)
discounts = np.array([0., .25, .5])
```

Read this block:

- Line 470, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 471, `discounts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 472–479:

```
for treatment in ["posterior-mean", "posterior-integrated"]:
    table, error = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
                                    capacity=2, batch_size=4, type_treatment=treatment)
    assert error < 1e-12 and len(table) == 24
    # Accounting and treating the middle segment as a holdout.
    assert np.allclose(table.discounted_revenue+table.coupon_expenditure,
                       1.1*(table.berry_units1+table.berry_units2))
    assert np.allclose(table.loc[table.group == "middle", "revenue_change"], 0.)
```

Read this block:

- Line 472: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 473, `(table, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 475: require `error < 1e-12 and len(table) == 24` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 477: require `np.allclose(table.discounted_revenue + table.coupon_expenditure, 1.1 * (table.berr...` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 479: require `np.allclose(table.loc[table.group == 'middle', 'revenue_change'], 0.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 480–481:

```
no_history, _ = targeted_coupons(context, households, weights, np.ones_like(alpha),
                                 np.zeros_like(gamma), beta, discounts, capacity=2)
```

Read this block:

- Line 480, `(no_history, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 482–482:

```
482 assert np.allclose(no_history.revenue1, no_history.revenue2)
```

Read this block:

- Line 482: require `np.allclose(no_history.revenue1, no_history.revenue2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 483–483:

```
483 assert np.allclose(no_history.discount1, no_history.expected_discount2)
```

Read this block:

- Line 483: require `np.allclose(no_history.discount1, no_history.expected_discount2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 485–488:

```
485 swapped = households.copy()
486 swapped["group"] = swapped.group.replace({"low": "high", "high": "low"})
487 changed, _ = targeted_coupons(context, swapped, weights, alpha, gamma, beta, discounts, capacity=2)
488 original, _ = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts, capacity=2)
```

Read this block:

- Line 485, `swapped`: Set this name to the result of calling `households.copy(...)`; the arguments and preceding method chain are shown above.
- Line 486, `swapped['group']`: Store the result of calling `swapped.group.replace(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 487, `(changed, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 488, `(original, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 489–489:

```
489 assert np.allclose(changed.discount1, original.discount1)
```

Read this block:

- Line 489: require `np.allclose(changed.discount1, original.discount1)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 490–490:

```
490 assert np.allclose(changed.discounted_revenue, original.discounted_revenue)
```

Read this block:

- Line 490: require `np.allclose(changed.discounted_revenue, original.discounted_revenue)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 491–491:

```
491 assert len(compositions(7)) == 120
```

Read this block:

- Line 491: require `len(compositions(7)) == 120` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 492–496:

```
492 restricted = {k: v.copy() for k,v in context.items()}
493 restricted["available"][:, 2] = False
494 restricted["prices"][:, 2] = 0.
495 _, error = targeted_coupons(restricted, households, weights, alpha, gamma, beta,
496                              discounts[:1], capacity=7, batch_size=4)
```

Read this block:

- Line 492, `restricted`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 493, `restricted['available'][:, 2]`: Store the literal value `False` in the selected entry/column.
- Line 494, `restricted['prices'][:, 2]`: Store the literal value `0.0` in the selected entry/column.
- Line 495, `(_, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 497–497:

```
497 assert error < 1e-12
```

Read this block:

- Line 497: require `error < 1e-12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 498–539:

```
498 with tempfile.TemporaryDirectory(prefix="targeting_synthetic_") as directory:
499     root = Path(directory)
500     synthetic_input = root / "synthetic_records.csv"
501     frame.to_csv(synthetic_input, index=False)
502     pd.DataFrame({"parameter": list(parameters), "estimate": list(parameters.values()), "at_bound":
    ↪  False}).to_csv(root / "point_estimates.csv", index=False)
503     fit = dict(model="heterogeneous", converged=True, nll=nll, weeks=len(frame), households=12,
504                settings=dict(capacity=2, draws=8, seed=19, no_price_control=True, habit_weight=.7))
505     (root / "fit.json").write_text(json.dumps(fit))
506     args = argparse.Namespace(fit_dir=root, household_input=synthetic_input, output=root,
507                               discounts=discounts, demand_delta=1., batch_size=4,
508                               type_treatment="posterior-mean", save_state_policies=True)
509     report = run_households(args)
510     assert report["households"] == 12
511     states = pd.read_csv(root / "targeted_state_policies.csv.gz")
512     assert "household_code" not in states.columns
```

```
    expected_households = int(households.group.isin(["low", "high"]).sum())
    assert np.allclose(states.groupby("policy").probability.sum(), expected_households)
    # Independent exhaustive two-period calculation for the first targeted
    # household: all first actions and all terminal actions at each basket.
    household = int(np.flatnonzero(households.group.eq("low").to_numpy())[0])
    a, g = weights[household] @ alpha, weights[household] @ gamma
    basket_rows = compositions(2)
    values = []
    for d1 in discounts:
        prices = context["prices"][household].copy()
        prices[1] *= 1-d1
        xi = .3*context["novelty"][household]-.7*context["stocks"][household]
        u = np.r_[beta-a*prices+g*xi, 0.]
        prob = np.exp(u-logsumexp(u))
        value = 2*prob[1]*prices[1]
        continuation = 0.
        for basket in basket_rows:
            mass = math.factorial(2)/math.prod(math.factorial(int(n)) for n in basket)*np.prod(prob**basket)
            x = basket[:3]/max(basket[:3].sum(), 1)
            xi2 = .3*(x.sum()-x)-.7*(context["stocks"][household]+x)
            returns = []
            for d2 in discounts:
                second_prices = context["prices"][household].copy()
                second_prices[1] *= 1-d2
                u2 = np.r_[beta-a*second_prices+g*xi2, 0.]
                prob2 = np.exp(u2-logsumexp(u2))
                returns.append(2*prob2[1]*second_prices[1])
```

Source lines 540–545:

```
        continuation += mass*max(returns)
    values.append(value+continuation)
actual = pd.read_csv(root / "targeted_household_results.csv")
assert "household_code" not in actual.columns
predicted = actual.loc[actual.policy.eq("personalized_dynamic"), "discounted_revenue"]
assert np.isclose(predicted, max(values)).any()
```

Read this block:

- Line 498: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 499, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 500, `synthetic_input`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 501: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 502: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 503, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 505: call `(root / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 506, `args`: Set this name to the result of calling `argparse.Namespace(...)`; the arguments and preceding method chain are shown above.
- Line 509, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 510: require `report['households'] == 12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

- Line 511, `states`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 512: require `'household_code' not in states.columns` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 513, `expected_households`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 514: require `np.allclose(states.groupby('policy').probability.sum(), expected_households)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 517, `household`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 518, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 519, `basket_rows`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 520, `values`: Set this name to the list `[]`.
- Line 521: for each entry yielded by `discounts`, bind/unpack it into `d1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 522, `prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 523: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 524, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 525, `u`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.
- Line 526, `prob`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 527, `value`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 528, `continuation`: Set this name to the literal value `0.0`.
- Line 529: for each entry yielded by `basket_rows`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 530, `mass`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 531, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 532, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 533, `returns`: Set this name to the list `[]`.
- Line 534: for each entry yielded by `discounts`, bind/unpack it into `d2` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 535, `second_prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 536: update `second_prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 537, `u2`: Set this name to the entry/slice selected by `np.r_[beta - a * second_prices + g * xi2, 0.0]`; brackets select from the object to their left.
- Line 538, `prob2`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 539: call `returns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 540: update `continuation` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 541: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 542, `actual`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 543: require `'household_code' not in actual.columns` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 544, `predicted`: Set this name to the entry/slice selected by `actual.loc[actual.policy.eq('personalized_dynamic'), 'dis...`; brackets select from the object to their left.
- Line 545: require `np.isclose(predicted, max(values)).any()` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `list`: Materialize iterable entries as an ordered Python list.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 546–546:

```
546 print("Synthetic targeting checks passed: posterior likelihood, per-household optimization, full-basket backward
    ↪  induction, reporting-group invariance, policy dominance, missing menus, exact basket mass, exports.")
```

Read this block:

- Line 546: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 550.

Parses explicit fit and weekly-input paths, validates coupon settings, and runs the earlier combined-gamma berry-only exercise. Its self-test uses generated temporary inputs. The new split fit requires the separate split entry point.

Definition and stated intent:

Source lines 550–550:

```
550 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 551–557:

```
551 parser = argparse.ArgumentParser(description=__doc__)
552 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
553 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
554 parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results") /
    ↪  "empirical_outputs")
555 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
556                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
557 parser.add_argument("--demand-delta", type=float, default=.95)
```

Read this block:

- Line 551, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 552: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 553: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 554: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 555: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 557: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 558–563:

```
558 parser.add_argument("--batch-size", type=int, default=24)
559 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪  default="posterior-mean",
560                     help="Default treats estimated household types as known, matching base supply; optional
                        ↪  posterior uncertainty sensitivity.")
561 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
    ↪  terminal coupon to compressed CSV.")
562 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
563 args = parser.parse_args()
```

Read this block:

- Line 558: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 559: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 561: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 562: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 563, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 564–566:

```
564 if args.self_test:
565     self_test()
566     return
```

Read this block:

- Line 564: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 565: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 566: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 567–568:

```
567 if args.household_input is None or args.fit_dir is None:
568     parser.error("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 567: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 568: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 569–569:

```
569 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 569, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 570–572:

```
570 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
571         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
572     parser.error("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 570: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 572: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 573–579:

```
573 args.output.mkdir(parents=True, exist_ok=True)
574 report = run_households(args)
575 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
576                 results=report, script_sha256=digest(Path(__file__).resolve()),
577                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
578 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
579 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 573: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 574, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 575, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 578: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 579: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

pricing_exercises/run_pricing_exercises.py

Compatibility launcher forwarding to the earlier supply or combined-gamma exercise. The current split workflow has a separate explicit entry point and is not selected by the old households branch.

Module setup and direct-execution blocks

Source lines 2–9:

```
"""Compatibility launcher. Prefer the two explicit standalone entry points."""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
 ↪  "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 5: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 7, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 10–13:

```
import argparse
from pathlib import Path
import subprocess
import sys
```

Read this block:

- Line 10: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 11: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 13: import `sys`. An as-name is a convenient local alias, not another library.

Source lines 27–28:

```
if __name__ == "__main__":
    main()
```

Read this block:

- Line 27: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 17.

Forwards compatibility arguments to the standalone theoretical supply or earlier combined-gamma household script. The launcher does not estimate demand or select the new split workflow automatically. Use the explicit split-pricing entry point for the current exercise.

Definition and stated intent:

Source lines 17–17:

```
17 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 18–20:

```
18 parser = argparse.ArgumentParser(add_help=False)
19 parser.add_argument("--exercise", choices=["supply", "households", "both"], default="supply")
20 args, remaining = parser.parse_known_args()
```

Read this block:

- Line 18, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 19: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `(args, remaining)`: Unpack the result of calling `parser.parse_known_args(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 21–22:

```
21 if args.exercise == "both":
22     raise SystemExit("Use run_supply_exercises.py and run_empirical_targeting.py separately; see README.md.")
```

Read this block:

- Line 21: test `args.exercise == 'both'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `SystemExit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 23–24:

```
23 script = "run_supply_exercises.py" if args.exercise == "supply" else "run_empirical_targeting.py"
24 subprocess.run([sys.executable, str(Path(__file__).resolve().parent / script), *remaining], check=True)
```

Read this block:

- Line 23, `script`: Set this name to the true-branch value if `args.exercise == 'supply'` holds, otherwise the else-branch value.
- Line 24: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises/run_split_targeting.py

Current original split-model empirical pricing implementation. It recovers alpha, novelty and own-stock types; ranks by novelty; retains both effects in demand; and runs all three focal flavors with fixed rivals. This file uses household batches; the readable final_simple version loops one household at a time with equivalent results.

Module setup and direct-execution blocks

Source lines 2–13:

```
2 """Split-history pricing for each focal flavor, separate from demand estimation.
3
4 User-supplied paths only. No estimation is performed. --self-test uses only
5 generated synthetic inputs. See SPLIT_TARGETING.md for definitions.
6 """
7 from __future__ import annotations
8
9 # Empirical I/O is permitted only on the research server, never on this computer.
10 from pathlib import Path as _PolicyPath
11 import sys as _policy_sys
12 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
13 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 12, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

- Line 13: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–19:

```
14 from server_data_policy import server_only, require_server
15 import argparse
16 import hashlib
17 import json
18 import math
19 import os
```

Read this block:

- Line 14: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 16: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 17: import `json`. An as-name is a convenient local alias, not another library.
- Line 18: import `math`. An as-name is a convenient local alias, not another library.
- Line 19: import `os`. An as-name is a convenient local alias, not another library.

Source lines 20–26:

```
20 from pathlib import Path
21 import sys
22 import tempfile
23 import numpy as np
24 import pandas as pd
25
26 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 20: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 21: import `sys`. An as-name is a convenient local alias, not another library.
- Line 22: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 23: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 24: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 26, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 512–513:

```
512 if __name__ == "__main__":
513     main()
```

Read this block:

- Line 512: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 513: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 28.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 28–28:

```
28 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–30:

```
29 x = np.asarray(x)
30 small = np.exp(-np.abs(x))
```

Read this block:

- Line 29, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 30, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 31–31:

```
31 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 31: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 33.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 33–33:

```
33 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–36:

```
34 x = np.asarray(x)
35 maximum = np.max(x, axis=axis, keepdims=True)
36 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 34, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 35, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 36, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 37–37:

```
37 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 37: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 39.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 39–39:

```
39 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–43:

```
40 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
41 import matplotlib
42 matplotlib.use("Agg")
43 import matplotlib.pyplot as plt
```

Read this block:

- Line 40: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 42: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 44–44:

```
44 return plt
```

Read this block:

- Line 44: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 46.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 46–46:

```
46 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–48:

```
47 if number < 2 or number % 2:
48     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 47: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 49–49:

```
49 half = np.random.default_rng(seed).standard_normal((number//2, 3))
```

Read this block:

- Line 49, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 50–50:

```
50 return np.vstack([half, -half])
```

Read this block:

- Line 50: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 52.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 52–53:

```
52 def posterior(frame, parameters, draws, control=True):
53     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 54–59:

```
54 labels = np.sort(frame.household_code.unique())
55 hh = pd.Categorical(frame.household_code, categories=labels).codes
56 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
57 available = np.isfinite(prices) & (prices > 0)
58 counts = frame[FLAVORS+["outside"]].to_numpy(float)
59 novelty = frame[["novelty_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 54, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 55, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 56, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 57, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 59, `novelty`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 60–61:

```
60 stock = frame[["stock_"+f for f in FLAVORS]].to_numpy(float)
61 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 60, `stock`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 61, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 62–65:

```
62 if (not np.isfinite(counts).all() or (counts < 0).any()
63         or not np.isfinite(novelty).all() or not np.isfinite(stock).all() or not np.isfinite(cf).all()
64         or ((counts[:, :3] > 0) & ~available).any()):
65     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 62: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(novel...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 65: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 66–70:

```
beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
switching = parameters["mean_novelty"] + parameters["sd_novelty"]*draws[:, 1]
habit = parameters["mean_own_stock"] + parameters["sd_own_stock"]*draws[:, 2]
loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 66, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 67, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 68, `switching`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `habit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 70, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 71–76:

```
for d, (a, g, b) in enumerate(zip(alpha, switching, habit)):
    u = beta-a*np.nan_to_num(prices)+g*novelty+b*stock+parameters.get("price_control", 0.)*cf
    u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
    logp = u-logsumexp(u, axis=1, keepdims=True)
    ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
    loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 71: for each entry yielded by `enumerate(zip(alpha, switching, habit))`, bind/unpack it into `(d, (a, g, b))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 73, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 74, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 75, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 76, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.

- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 77–83:

```
77 weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
78 nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
79 means = weights @ switching
80 # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
81 order = np.argsort(means, kind="stable")
82 groups = np.full(len(labels), "middle", dtype=object)
83 n = len(labels)//4
```

Read this block:

- Line 77, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 78, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 79, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 82, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 83, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 84–85:

```
84 if n < 1:
85     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 84: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 86–94:

```
86 groups[order[:n]], groups[order[-n:]] = "low", "high"
87 population_q = np.quantile(switching, [.25, .75])
88 table = pd.DataFrame(dict(household_code=labels, group=groups, switching_mean=means,
89                           switching_sd=np.sqrt(np.maximum(weights @ switching**2-means**2, 0)),
90                           alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
91                           probability_switching_below_draw_q25=weights @ (switching <= population_q[0]),
```

```
92                          probability_switching_above_draw_q75=weights @ (switching >= population_q[1])))
93 table["habit_mean"] = weights @ habit
94 table["habit_sd"] = np.sqrt(np.maximum(weights @ habit**2 - table.habit_mean.to_numpy()**2, 0))
```

Read this block:

- Line 86, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 87, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 88, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 93, `table['habit_mean']`: Store matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 94, `table['habit_sd']`: Store the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 95–95:

```
95 return table, weights, alpha, np.column_stack([switching, habit]), beta, nll
```

Read this block:

- Line 95: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 97.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 97–98:

```
97 def compositions(capacity):
98     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 99–100:

```
99 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
100                 for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 99: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 102.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 102–107:

```
102 def forecast_context(frame, households):
103     """First observed choice establishes h0; price the next two consecutive weeks.
104 
105     Carry the initial observation's menu, posted prices and controls forward.
106     Type estimation still uses the complete observed sample.
107     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 108–114:

```
108 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
109 first = first.set_index("household_code").loc[households.household_code]
110 counts = first[FLAVORS].to_numpy(float)
111 totals = counts.sum(axis=1, keepdims=True)
112 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
113 # Saved stocks precede the observed choice; add its basket exactly once.
114 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 108, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 109, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 110, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 111, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 112, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 114, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 115–120:

```
115 novelty = shares.sum(axis=1, keepdims=True)-shares
116 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
117 available = np.isfinite(prices) & (prices > 0)
118 eligible = available[:, 1]
119 households["berry_eligible"] = eligible
120 initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 115, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 116, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 117, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 118, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 119, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 120, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 121–125:

```
121 households["initial_observed_week"] = initial_weeks
122 households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
123 households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
124 cf_cols = ["cf_"+f for f in FLAVORS]
125 cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 121, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 122, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 123, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 124, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 125, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 126–127:

```
126 if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
127     raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 126: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 127: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 128–129:

```
128 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
129             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 128: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 131.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 131–131:

```
131 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 132–132:

```
132 h = hashlib.sha256()
```

Read this block:

- Line 132, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 133–135:

```
133 with path.open("rb") as handle:
134     for chunk in iter(lambda: handle.read(1024*1024), b""):
135         h.update(chunk)
```

Read this block:

- Line 133: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 134: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 135: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 136–136:

```
136 return h.hexdigest()
```

Read this block:

- Line 136: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 140.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 140–150:

```
140 def targeted_coupons(context, households, weights, alpha, switching, beta, discounts,
141                      delta=1., capacity=7, control_coefficient=0.,
142                      batch_size=24, type_treatment="posterior-mean", state_output=None):
143     """Observed h0, then p1(h0), realized h1, and terminal p2(h1).
144 
145     The observed initialization and two priced choices form three choice dates.
146     Individual backward induction; switching quartiles are reporting groups only.
147 
148     Baseline: posted prices in both weeks. Dynamic: choose first coupon accounting for optimized
149     state-contingent terminal revenue. The seller observes the full basket.
150     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `switching`, `beta`, `discounts`, `delta`, `capacity`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 151–156:

```
151 baskets = compositions(capacity)
152 inside = baskets[:, :3]
153 totals = inside.sum(axis=1, keepdims=True)
154 shares = np.divide(inside, totals, out=np.zeros_like(inside, dtype=float), where=totals > 0)
155 coeff = math.lgamma(capacity+1)-np.vectorize(math.lgamma)(baskets+1).sum(axis=1)
156 k, nstates = len(discounts), len(baskets)
```

Read this block:

- Line 151, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 152, `inside`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 153, `totals`: Set this name to the result of calling `inside.sum(...)`; the arguments and preceding method chain are shown above.
- Line 154, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 155, `coeff`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 156, `(k, nstates)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 157–160:

```
157 rows, maximum_mass_error, wrote_states = [], 0., False
158 indices = np.flatnonzero(context["eligible"])
159 # Only the two tails receive targeting; middle households remain at posted prices.
160 targeted = households.group.isin(["low", "high"]).to_numpy()
```

Read this block:

- Line 157, `(rows, maximum_mass_error, wrote_states)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 158, `indices`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 160, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 161–202:

```
161 for start in range(0, len(indices), batch_size):
162     ids = indices[start:start+batch_size]
163     h = len(ids)
164     prices, availability = context["prices"][ids], context["available"][ids]
```

```
    stock, novelty, cf = context["stocks"][ids], context["novelty"][ids], context["cf"][ids]
    if type_treatment == "posterior-mean":
        # Base supply information assumption: an estimated fixed type is known.
        a, g = (weights[ids] @ alpha)[:, None], (weights[ids] @ switching)[:, None]
        w = np.ones((h, 1))
    else:
        a = np.broadcast_to(alpha, (h, len(alpha)))
        g = np.broadcast_to(switching, (h, *switching.shape))
        w = weights[ids]
    # g[...,0] is switching/novelty; g[...,1] is signed own-stock habit.
    history1 = g[..., 0, None]*novelty[:, None, :] + g[..., 1, None]*stock[:, None, :]
    novelty2 = shares.sum(axis=1, keepdims=True)-shares
    stock2 = stock[:, None, :]+shares[None, :, :]
    history2 = (g[..., 0, None, None]*novelty2[None, None, :, :]
                +g[..., 1, None, None]*stock2[:, None, :, :])
    u1 = beta[None, None, :]-a[..., None]*prices[:, None, :]+history1+control_coefficient*cf[:, None, :]
    u2 = beta[None, None, None, :]-a[:, :, None, None]*prices[:, None, None, :]+history2+control_coefficient*cf[:,
    None, None, :]
    u1 = np.where(availability[:, None, :], u1, -np.inf)
    u2 = np.where(availability[:, None, None, :], u2, -np.inf)
    terminal = []
    for discount in discounts:
        u = u2.copy()
        u[..., 1] += a[:, :, None]*prices[:, None, None, 1]*discount
        denominator = logsumexp(np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1), axis=-1)
        quantity = capacity*np.exp(u[..., 1]-denominator)
        terminal.append(np.stack([quantity*prices[:, None, None, 1]*(1-discount), quantity,
                                  quantity*prices[:, None, None, 1]*discount], axis=-1))
    # Metrics: net revenue, berry units, coupon expenditure.
    first = np.zeros((h, k, 3))
    second = np.zeros((h, k, 3))
    stage0_second = None
    paths = []
    for i, discount in enumerate(discounts):
        u = u1.copy()
        u[..., 1] += a*prices[:, None, 1]*discount
        full = np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1)
        logp = full-logsumexp(full, axis=-1, keepdims=True)
        terms = np.zeros((h, a.shape[1], nstates, 4))
```

Source lines 203–244:

```
    np.multiply(baskets[None, None, :, :], logp[:, :, None, :], out=terms,
                where=baskets[None, None, :, :] > 0)
    probability = np.exp(coeff[None, None, :]+terms.sum(axis=-1))
    maximum_mass_error = max(maximum_mass_error, float(np.abs(probability.sum(axis=-1)-1).max()))
    joint = w[:, :, None]*probability
    state_probability = joint.sum(axis=1)
    quantity = capacity*(w*np.exp(logp[..., 1])).sum(axis=1)
    first[:, i] = np.stack([quantity*prices[:, 1]*(1-discount), quantity,
                            quantity*prices[:, 1]*discount], axis=-1)
    # Integrate types BEFORE choosing a terminal coupon: no foresight of
    # an unobserved true draw in the posterior-integrated sensitivity.
    metrics = np.stack([np.einsum("hds,hdsm->hsm", joint, candidate)
                        for candidate in terminal], axis=2)  # household,state,action,metric
    best = metrics[..., 0].argmax(axis=2)
    selected = np.take_along_axis(metrics, best[:, :, None, None], axis=2)[:, :, 0, :]
    # E_h1[p2(h1)*E(q2|h1,p2,type)] under this candidate p1(h0).
    # Optimize p2 separately in each state BEFORE summing its revenue.
    second[:, i] = selected.sum(axis=1)
    if i == 0:
        stage0_second = metrics[:, :, 0, :].sum(axis=1)
```

```
    # Storing these compact arrays allows exporting only chosen first actions.
    paths.append((best, selected, state_probability))
zero_choice = np.zeros(h, dtype=int)
dynamic_choice = (first[..., 0]+delta*second[..., 0]).argmax(axis=1)
dynamic_choice = np.where(targeted[ids], dynamic_choice, 0)
for name, choice in [("no_coupon", zero_choice), ("personalized_dynamic", dynamic_choice)]:
    for local, hh in enumerate(ids):
        action = int(choice[local])
        initial = first[local, action]
        is_target = targeted[hh] and name != "no_coupon"
        terminal_metrics = second[local, action] if is_target else stage0_second[local]
        best, selected, probability = paths[action]
        state_coupon = discounts[best[local]] if is_target else np.zeros(nstates)
        mass = probability[local]
        p1 = prices[local, 1]*(1-discounts[action])
        p2 = prices[local, 1]*(1-state_coupon)
        mean_p2 = float(mass @ p2)
        ratio = np.divide(p1, p2, out=np.full(nstates, np.nan), where=p2 > 0)
        positive_mass = mass > 1e-14
        expected_ratio = float(mass[positive_mass] @ ratio[positive_mass]) if
    np.isfinite(ratio[positive_mass]).all() else np.nan
        row = dict(household_code=households.iloc[hh].household_code,
                   group=households.iloc[hh].group, policy=name,
```

Source lines 245–267:

```
                   discount1=discounts[action], expected_discount2=float(mass @ state_coupon),
                   p1=p1, expected_p2=mean_p2,
                   ratio_of_expected_prices=p1/mean_p2 if mean_p2 > 0 else np.nan,
                   expected_price_ratio=expected_ratio,
                   revenue1=initial[0], revenue2=terminal_metrics[0],
                   discounted_revenue=initial[0]+delta*terminal_metrics[0],
                   berry_units1=initial[1], berry_units2=terminal_metrics[1],
                   coupon_expenditure=initial[2]+delta*terminal_metrics[2])
        for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
            branch_mass = mass[branch].sum()
            row[f"probability_{label}"] = branch_mass
            row[f"mean_p2_given_{label}"] = float(mass[branch] @ p2[branch]/branch_mass) if branch_mass > 0 else
    np.nan
        rows.append(row)
        if state_output is not None and is_target:
            detail = pd.DataFrame(dict(household_code=row["household_code"], group=row["group"], policy=name,
                                       state=np.arange(nstates), rival1_units=baskets[:, 0], focal_units=baskets[:, 1],
                                           rival2_units=baskets[:, 2], outside=baskets[:, 3], probability=mass,
                                           discount1=discounts[action], discount2=state_coupon, p1=p1, p2=p2,
    price_ratio=ratio))
            detail.drop(columns=["household_code"]).to_csv(state_output, mode="a" if wrote_states else "w",
    header=not wrote_states,
                           index=False, compression="gzip")
            wrote_states = True
if start % (batch_size*40) == 0 or start+batch_size >= len(indices):
    print(f"Targeted pricing: integrated {min(start+batch_size,len(indices))}/{len(indices)} eligible households",
        flush=True)
```

Read this block:

- Line 161: for each entry yielded by `range(0, len(indices), batch_size)`, bind/unpack it into `start` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 162, `ids`: Set this name to the entry/slice selected by `indices[start:start + batch_size]`; brackets select from the object to their left.
- Line 163, `h`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are

shown above.
- Line 164, `(prices, availability)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 165, `(stock, novelty, cf)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 166: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 168, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 169, `w`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 171, `a`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 172, `g`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 173, `w`: Set this name to the entry/slice selected by `weights[ids]`; brackets select from the object to their left.
- Line 175, `history1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 176, `novelty2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 177, `stock2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 178, `history2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 180, `u1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 181, `u2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 182, `u1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 183, `u2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 184, `terminal`: Set this name to the list `[]`.
- Line 185: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 186, `u`: Set this name to the result of calling `u2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 187: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 188, `denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 189, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 190: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 194, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 195, `stage0_second`: Set this name to the literal value `None`.
- Line 196, `paths`: Set this name to the list `[]`.
- Line 197: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(i, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 198, `u`: Set this name to the result of calling `u1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 199: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 200, `full`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding

method chain are shown above.
- Line 201, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 202, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 203: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 205, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 206, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 207, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 208, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 209, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 210, `first[:, i]`: Store the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 214, `metrics`: Set this name to the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above.
- Line 216, `best`: Set this name to the result of calling `metrics[..., 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 217, `selected`: Set this name to the entry/slice selected by `np.take_along_axis(metrics, best[:, :, None, None], axis=...`; brackets select from the object to their left.
- Line 220, `second[:, i]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 221: test `i == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 222, `stage0_second`: Set this name to the result of calling `metrics[:, :, 0, :].sum(...)`; the arguments and preceding method chain are shown above.
- Line 224: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 225, `zero_choice`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 226, `dynamic_choice`: Set this name to the result of calling `.argmax(...)`; the arguments and preceding method chain are shown above.
- Line 227, `dynamic_choice`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 228: for each entry yielded by `[('no_coupon', zero_choice), ('personalized_dynamic', dynamic_...`, bind/unpack it into `(name, choice)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 229: for each entry yielded by `enumerate(ids)`, bind/unpack it into `(local, hh)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 230, `action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 231, `initial`: Set this name to the entry/slice selected by `first[local, action]`; brackets select from the object to their left.
- Line 232, `is_target`: Set this name to the Boolean condition `targeted[hh] and name != 'no_coupon'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 233, `terminal_metrics`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 234, `(best, selected, probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 235, `state_coupon`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.

- Line 236, `mass`: Set this name to the entry/slice selected by `probability[local]`; brackets select from the object to their left.
- Line 237, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 238, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 239, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 240, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 241, `positive_mass`: Set this name to the Boolean condition `mass > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 242, `expected_ratio`: Set this name to the true-branch value if `np.isfinite(ratio[positive_mass]).all()` holds, otherwise the else-branch value.
- Line 243, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 253: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 254, `branch_mass`: Set this name to the result of calling `mass[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 255, `row[f'probability_{label}']`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 256, `row[f'mean_p2_given_{label}']`: Store the true-branch value if `branch_mass > 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 257: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 258: test `state_output is not None and is_target`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 259, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 263: call `detail.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 265, `wrote_states`: Set this name to the literal value `True`.
- Line 266: test `start % (batch_size * 40) == 0 or start + batch_size >= len(indices)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 267: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 268–269:

```
268 if maximum_mass_error > 1e-9:
269     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 268: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 269: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 270–275:

```
270 table = pd.DataFrame(rows)
271 baseline = table[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
272 table["baseline_revenue"] = table.household_code.map(baseline)
273 table["revenue_change"] = table.discounted_revenue-table.baseline_revenue
274 table["revenue_change_pct"] = 100*table.revenue_change/table.baseline_revenue
275 wide = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 270, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 271, `baseline`: Set this name to the attribute `table[table.policy == 'no_coupon'].set_index('household_c...` of the object before the dot.
- Line 272, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 273, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

- Line 274, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 275, `wide`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 276–277:

```
276 if (wide.personalized_dynamic+1e-8 < wide.no_coupon).any():
277     raise AssertionError("Household dynamic policy does not dominate the feasible no-coupon baseline.")
```

Read this block:

- Line 276: test `(wide.personalized_dynamic + 1e-08 < wide.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 278–278:

```
278 return table, maximum_mass_error
```

Read this block:

- Line 278: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

run households

Original function: `run_households`. Source begins at line 281.

Reads an explicitly selected fitted-results directory and its weekly table, checks the model, convergence, bounds, capacity and input schema, then recovers posterior types and reproduces the saved likelihood. For a split run, it permutes the selected focal flavor into the solver's berry coordinate, moving counts, prices, stocks, novelty, controls and intercepts consistently. Type recovery happens before permutation using the original fitted flavor labels.

It calls the household solver, renames public berry-coordinate columns to focal names, and writes policy/group/aggregate tables and figures. The manifest records how rival state coordinates map back to flavors. Swapping only prices or only counts would silently price the wrong product. The three runs are separate focal sellers with fixed rival prices, not a simultaneous equilibrium.

Definition and stated intent:

Source lines 281–283:

```
281 def run_households(args):
282     # This branch is entered only when the person running the script supplies
283     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 284–284:

```
284 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 284, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 285–286:

```
285 if fit["model"] != "heterogeneous_history" or not fit.get("converged", False):
286     raise ValueError("This exercise requires a converged split heterogeneous-history fit.")
```

Read this block:

- Line 285: test `fit['model'] != 'heterogeneous_history' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 286: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 287–287:

```
287 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 287, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 288–289:

```
288 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
289     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 288: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 289: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 290–292:

```
290 parameters = dict(zip(estimates.parameter, estimates.estimate))
291 settings = fit["settings"]
292 capacity = settings["capacity"]
```

Read this block:

- Line 290, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 291, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 292, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 293–294:

```
293 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
294     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 293: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 294: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 295–298:

```
295 frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
296 required = ["household_code", "week", "outside", *FLAVORS]
297 required += [prefix+f for prefix in ["price_", "stock_", "novelty_"] for f in FLAVORS]
298 control = not settings.get("no_price_control", False)
```

Read this block:

- Line 295, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 296, `required`: Set this name to a list containing 4 displayed entries.
- Line 297: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 298, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 299–300:

```
299 if control:
300     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 299: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 300: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 301–301:

```
301 missing = set(required)-set(frame)
```

Read this block:

- Line 301, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 302–303:

```
302 if missing:
303     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 302: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 303: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 304–305:

```
304 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪ "week"]).any():
305     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 304: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 305: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 306–307:

```
306 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
307     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 306: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 307: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 308–309:

```
draws = persistent_draws(settings["draws"], settings["seed"])
households, weights, alpha, switching, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 308, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 309, `(households, weights, alpha, switching, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 311–312:

```
if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
    raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 311: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 312: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 313–313:

```
focal = args.focal_flavor
```

Read this block:

- Line 313, `focal`: Set this name to the attribute `args.focal_flavor` of the object before the dot.

Source lines 314–322:

```
if focal != "berry":
    mapping = {"berry": focal, focal: "berry"}
    original = frame.copy()
    original_beta = beta.copy()
    for destination, source in mapping.items():
        for prefix in ["", "price_", "stock_", "novelty_", "cf_"]:
            if prefix+source in original:
                frame[prefix+destination] = original[prefix+source]
        beta[FLAVORS.index(destination)] = original_beta[FLAVORS.index(source)]
```

Read this block:

- Line 314: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 315, `mapping`: Set this name to a dictionary with named entries `berry`.
- Line 316, `original`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 317, `original_beta`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.
- Line 318: for each entry yielded by `mapping.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 319: for each entry yielded by `['', 'price_', 'stock_', 'novelty_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 320: test `prefix + source in original`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 321, `frame[prefix + destination]`: Store the entry/slice selected by `original[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 322, `beta[FLAVORS.index(destination)]`: Store the entry/slice selected by `original_beta[FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 323–324:

```
323 context = forecast_context(frame, households)
324 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 323, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 324, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 325–326:

```
325 if any(n == 0 for n in counts):
326     raise ValueError("Each switching segment must have focal-eligible forecast households.")
```

Read this block:

- Line 325: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 326: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 327–337:

```
327 table, mass_error = targeted_coupons(
328     context, households, weights, alpha, switching, beta, args.discounts,
329     args.demand_delta, capacity,
330     parameters.get("price_control", 0.) if control else 0., args.batch_size,
331     args.type_treatment,
332     args.output / "targeted_state_policies.csv.gz" if args.save_state_policies else None)
333 households.rename(columns={"berry_eligible": "focal_eligible"}, inplace=True)
334 table.rename(columns={c: c.replace("berry", "focal") for c in table if "berry" in c}, inplace=True)
335 table["focal_flavor"] = focal
336 households.drop(columns=["household_code"]).to_csv(args.output / "household_posterior_summary.csv", index=False)
```

```
337 table.drop(columns=["household_code"]).to_csv(args.output / "targeted_household_results.csv", index=False)
```

Read this block:

- Line 327, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 333: call `households.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 334: call `table.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 335, `table['focal_flavor']`: Store the object currently named `focal`; plain assignment does not copy it in the selected entry/column.
- Line 336: call `households.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 337: call `table.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Source lines 338–344:

```
338 totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
339           "focal_units1", "focal_units2", "coupon_expenditure"]
340 aggregate = table.groupby("policy", sort=False)[totals].sum()
341 aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
342 aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
343 group = table.groupby(["policy", "group"], sort=False)[totals].sum()
344 group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
```

Read this block:

- Line 338, `totals`: Set this name to a list containing 8 displayed entries.
- Line 340, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 341, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 342: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 343, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 344, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 345–359:

```
means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
                                            "ratio_of_expected_prices", "expected_price_ratio"]].mean()
group = group.join(means)
group["eligible_households"] = table.groupby(["policy", "group"]).size()
group.to_csv(args.output / "targeted_revenue_by_switching_group.csv")
households.groupby("group").agg(households=("household_code", "size"),
                                focal_eligible=("focal_eligible", "sum"),
                                switching_mean=("switching_mean", "mean"),
                                switching_mean_min=("switching_mean", "min"),
                                switching_mean_max=("switching_mean", "max"),
                                mean_posterior_sd=("switching_sd", "mean"),
                                habit_mean=("habit_mean", "mean"),
                                mean_habit_posterior_sd=("habit_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(args.output /
 ↪  "household_group_summary.csv")
```

Read this block:

- Line 345, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.
- Line 347, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 348, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 349: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 350: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

Source lines 360–361:

```
make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta,
                       focal_flavor=focal)
```

Read this block:

- Line 360: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 362–374:

```
return dict(focal_flavor=focal, state_coordinates={"focal_units": focal,
            "rival1_units": "berry" if focal == "other" else "other",
            "rival2_units": "berry" if focal == "plain" else "plain"}, households=len(households),
             ↪  eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
            reconstructed_nll=nll, basket_mass_error=mass_error,
            input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
            parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
            posterior_draws=len(draws), posterior_seed=settings["seed"],
            capacity=capacity, history_model="b_N*N + b_S*S", price_control=control,
```

```
            initial_history="first_observed_basket", pricing_periods=2,
            baskets=len(compositions(capacity)),
            discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
            type_treatment=args.type_treatment,
            aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 362: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 377.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 377–378:

```
def make_targeting_figures(households, table, aggregate, output, delta=.95, focal_flavor="berry"):
    """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`, `focal_flavor`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 379–384:

```
plt = plotting()
groups = ["low", "middle", "high"]
labels = ["Low switching\n(bottom 25%)", "Middle switching\n(middle 50%)", "High switching\n(top 25%)"]
colors = ["#3575AD", "#999999", "#D17925"]
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
bins = np.histogram_bin_edges(households.switching_mean, bins=30)
```

Read this block:

- Line 379, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 380, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 381, `labels`: Set this name to the list `['Low switching\n(bottom 25%)', 'Middle switching\n(middl...`.
- Line 382, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 383, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 384, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 385–387:

```
385 for g, label, color in zip(groups, labels, colors):
386     axes[0].hist(households.loc[households.group == g, "switching_mean"],
387                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 385: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 386: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 388–395:

```
388 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $b_N$)",
389             ylabel="Number of households")
390 axes[0].legend(fontsize=9)
391 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
392 x = np.arange(len(groups))
393 # Average over state-contingent terminal prices first, discount, then total.
394 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
395 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 388: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 390: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 391, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 392, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 394, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 395: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 396–402:

```
396 axes[1].set_xticks(x, labels)
397 axes[1].set(title="Total Expected Revenue",
398            xlabel="Consumer type group", ylabel="Total Expected Revenue")
399 axes[1].set_ylim(bottom=0)
400 axes[1].grid(axis="y", alpha=.2)
401 axes[1].set_axisbelow(True)
402 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 396: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 397: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 399: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 400: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 401: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 402: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 403–407:

```
403 fig.text(.5, .015,
404          f"First-period revenue + {delta:g} × expected terminal revenue. "
405          f"Low/high types receive personalized {focal_flavor} offers; middle types receive no coupons.",
406          ha="center", fontsize=9)
407 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 403: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 407: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 408–409:

```
408 for extension in ["png", "pdf"]:
409     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 408: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 409: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 410–417:

```
410 plt.close(fig)
411
412 fig, ax = plt.subplots(figsize=(9, 5))
413 x = np.arange(3)
414 means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    ↪  "expected_discount2"]].mean().reindex(groups)
415 ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
416 ax.bar(x+.18, 100*means.expected_discount2, width=.36,
417        label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 410: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 412, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 413, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 414, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 415: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 416: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 418–424:

```
418 ax.set_xticks(x, labels)
419 ax.set(xlabel="Consumer type group", ylim=(0, 105),
420        ylabel=f"Mean optimal {focal_flavor} discount (% of posted price)")
421 ax.grid(axis="y", alpha=.2)
422 ax.set_axisbelow(True)
423 ax.legend(fontsize=8, loc="upper left")
424 fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 418: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 419: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 421: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 422: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 423: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 424: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 425–427:

```
fig.text(.5, .015, f"Household-weighted means among {focal_flavor}-eligible households. Middle group receives no coupons by policy design.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 425: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 427: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 428–429:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 428: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 429: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 430–434:

```
plt.close(fig)

dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
    households[["household_code", "switching_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 430: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 432, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 434, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 435–447:

```
for column, (variable, xlabel) in enumerate([
        ("switching_mean", r"Estimated household type (posterior mean $b_N$)"),
        ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
    for row, (discount, title) in enumerate([
            ("discount1", r"Optimal discount at $p_1(h_0)$"),
```

```
            ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
        ax = axes[row, column]
        for g, label, color in zip(groups, labels, colors):
            subset = dynamic.loc[dynamic.group == g]
            ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
                       color=color, label=label.replace("\n", " "))
        ax.set(xlabel=xlabel, ylabel=f"{focal_flavor.title()} discount (% of posted price)", title=title, ylim=(-2,
    102))
        ax.grid(alpha=.2)
```

Read this block:

- Line 435: for each entry yielded by `enumerate([('switching_mean', 'Estimated household type (poste...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 438: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 441, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 442: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 443, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 444: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 446: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 447: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 448–452:

```
axes[0, 0].legend(fontsize=8)
fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
fig.text(.5, .015, f"Each point is one {focal_flavor}-eligible household. Terminal offers average over the
    household's realized h1 histories.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 448: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 449: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 450: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 452: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 453–454:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 453: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 454: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 455–455:

```
455 plt.close(fig)
```

Read this block:

- Line 455: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 458.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 458–458:

```
458 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 459–460:

```
459 import unittest
460 suite = unittest.defaultTestLoader.discover(str(Path(__file__).parent), pattern="test_split_targeting.py")
```

Read this block:

- Line 459: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 460, `suite`: Set this name to the result of calling `unittest.defaultTestLoader.discover(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 461–462:

```
461 if not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful():
462     raise RuntimeError("Split targeting checks failed")
```

Read this block:

- Line 461: test `not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 462: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 466.

Parses explicit empirical inputs and loops over the selected focal-flavor cases, writing per-flavor outputs and a combined comparison. Defaults select all three flavors and delta=.92. The self-test branch uses generated inputs.

Definition and stated intent:

Source lines 466–466:

```
466 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 467–473:

```
467 parser = argparse.ArgumentParser(description=__doc__)
468 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
469 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
470 parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results") /
    ↪ "split_empirical_outputs")
471 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
472                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
473 parser.add_argument("--demand-delta", type=float, default=.92)
```

Read this block:

- Line 467, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 468: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 469: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 470: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 471: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 473: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 474–480:

```
474 parser.add_argument("--batch-size", type=int, default=24)
475 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪ default="posterior-mean",
476                     help="Default treats estimated household types as known, matching base supply; optional
                        ↪ posterior uncertainty sensitivity.")
477 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
    ↪ terminal coupon to compressed CSV.")
```

```
478 parser.add_argument("--focal-flavor", choices=["all", *FLAVORS], default="all")
479 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
480 args = parser.parse_args()
```

Read this block:

- Line 474: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 475: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 477: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 478: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 479: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 480, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 481–483:

```
481 if args.self_test:
482     self_test()
483     return
```

Read this block:

- Line 481: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 482: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 483: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 484–485:

```
484 if args.household_input is None or args.fit_dir is None:
485     parser.error("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 484: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 485: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 486–486:

```
486 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 486, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 487–489:

```
487 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
488         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
489     parser.error("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 487: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 489: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 490–493:

```
490 args.output.mkdir(parents=True, exist_ok=True)
491 output_root = args.output
492 selected = FLAVORS if args.focal_flavor == "all" else [args.focal_flavor]
493 reports, comparisons = {}, []
```

Read this block:

- Line 490: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 491, `output_root`: Set this name to the attribute `args.output` of the object before the dot.
- Line 492, `selected`: Set this name to the true-branch value if `args.focal_flavor == 'all'` holds, otherwise the else-branch value.
- Line 493, `(reports, comparisons)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 494–500:

```
494 for focal in selected:
495     args.focal_flavor = focal
496     args.output = output_root / focal
497     args.output.mkdir(parents=True, exist_ok=True)
498     reports[focal] = run_households(args)
499     for row in reports[focal]["aggregate"]:
500         comparisons.append(dict(focal_flavor=focal, **row))
```

Read this block:

- Line 494: for each entry yielded by `selected`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 495, `args.focal_flavor`: Set this name to the object currently named `focal`; plain assignment does not copy it.
- Line 496, `args.output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 497: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 498, `reports[focal]`: Store the result of calling `run_households(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 499: for each entry yielded by `reports[focal]['aggregate']`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 500: call `comparisons.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 501–508:

```
501 args.output = output_root
502 args.focal_flavor = "all" if len(selected) == 3 else selected[0]
503 pd.DataFrame(comparisons).to_csv(output_root / "focal_flavor_comparison.csv", index=False)
504 report = reports
505 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
506                 results=report, script_sha256=digest(Path(__file__).resolve()),
507                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
508 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 501, `args.output`: Set this name to the object currently named `output_root`; plain assignment does not copy it.
- Line 502, `args.focal_flavor`: Set this name to the true-branch value if `len(selected) == 3` holds, otherwise the else-branch value.
- Line 503: call `pd.DataFrame(comparisons).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 504, `report`: Set this name to the object currently named `reports`; plain assignment does not copy it.
- Line 505, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 508: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 509–509:

```
509 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 509: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

pricing__exercises/run__supply__exercises.py

All three theoretical information cases: known initial history, hidden initial history, and advance commitment to two prices. The binary symmetric switching model is a mechanism benchmark; it is not the cumulative-stock split demand likelihood. Both continuation branches and unrestricted theoretical prices are retained.

Module setup and direct-execution blocks

Source lines 2–11:

```
2  """Run all three theoretical supply cases, prices, ratios and profit, in one file.
3
4  No empirical input is read. Dependencies: numpy, pandas, matplotlib.
5  python3 run_supply_exercises.py [--self-test] [--output DIRECTORY]
6  """
7  from __future__ import annotations
8  import argparse
9  import hashlib
10 import json
11 import math
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: import `math`. An as-name is a convenient local alias, not another library.

Source lines 12–17:

```
12 import os
13 from pathlib import Path
14 import sys
15 import tempfile
16 import numpy as np
17 import pandas as pd
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `sys`. An as-name is a convenient local alias, not another library.
- Line 15: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 16: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 17: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 19–19:

```
19 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 19, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 162–163:

```
162 CASES = ["known_state", "hidden_initial_state", "ex_ante_commitment"]
163 LABELS = dict(zip(CASES, ["Known state (base)", "Hidden initial state", "Ex ante commitment"]))
```

Read this block:

- Line 162, `CASES`: Set this name to the list `['known_state', 'hidden_initial_state', 'ex_ante_commitme...`.
- Line 163, `LABELS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 343–344:

```
343 if __name__ == "__main__":
344     main()
```

Read this block:

- Line 343: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 344: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 21.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 21–21:

```
21 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–23:

```
22 x = np.asarray(x)
23 small = np.exp(-np.abs(x))
```

Read this block:

- Line 22, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 23, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 24–24:

```
24 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 24: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

golden maximum

Original function: `golden_maximum`. Source begins at line 26.

Refines a bracketed scalar maximizer by golden-section interval reduction, reusing previous function values. This is a numerical search, not a proof that a multimodal objective has only one peak.

Definition and stated intent:

Source lines 26–26:

```
26 def golden_maximum(function, lo, hi):
```

`def` creates this callable; the parentheses name its inputs (`function`, `lo`, `hi`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 27–29:

```
27 ratio = (math.sqrt(5)-1)/2
28 left, right = hi-ratio*(hi-lo), lo+ratio*(hi-lo)
29 fl, fr = function(left), function(right)
```

Read this block:

- Line 27, `ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `(left, right)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 29, `(fl, fr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 30–40:

```
30 for _ in range(100):
31     if hi-lo <= 1e-10*(1+abs(left)+abs(right)):
32         break
33     if fl > fr:
34         hi, right, fr = right, left, fl
35         left = hi-ratio*(hi-lo)
36         fl = function(left)
37     else:
38         lo, left, fl = left, right, fr
39         right = lo+ratio*(hi-lo)
40         fr = function(right)
```

Read this block:

- Line 30: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 31: test `hi - lo <= 1e-10 * (1 + abs(left) + abs(right))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 33: test `fl > fr`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34, `(hi, right, fr)`: Unpack the tuple `(right, left, fl)` into these targets in order.
- Line 35, `left`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 36, `fl`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.
- Line 38, `(lo, left, fl)`: Unpack the tuple `(left, right, fr)` into these targets in order.
- Line 39, `right`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 40, `fr`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 41–41:

```
41 return (left, fl) if fl > fr else (right, fr)
```

Read this block:

- Line 41: return the true-branch value if `fl > fr` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

scalar maximum

Original function: `scalar_maximum`. Source begins at line 43.

Evaluates a scalar objective on a grid, refines every detected local peak and includes endpoints, then keeps the best candidate. Hidden-state mixtures can be multimodal, so assuming a single global peak would be unsafe.

Definition and stated intent:

Source lines 43–44:

```
43 def scalar_maximum(function, bounds, points=161):
44     """Search every grid-local maximum, including boundaries (mixtures may multimode)."""
```

`def` creates this callable; the parentheses name its inputs (`function`, `bounds`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–45:

```
45 grid = np.linspace(*bounds, points)
```

Read this block:

- Line 45, `grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 47–52:

```
47 try:
48     values = np.asarray(function(grid))
49     if values.shape != grid.shape:
50         raise ValueError("Scalar objective")
51 except (ValueError, TypeError):
52     values = np.array([function(x) for x in grid])
```

Read this block:

- Line 47: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 48, `values`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 49: test `values.shape != grid.shape`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 50: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 51: catch `(ValueError, TypeError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 52, `values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 53–53:

```
53 candidates = [(values[0], grid[0]), (values[-1], grid[-1])]
```

Read this block:

- Line 53, `candidates`: Set this name to a list containing 2 displayed entries.

Source lines 54–57:

```
54 for i in range(1, points - 1):
55     if values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i - 1], values[i + 1]):
56         price, value = golden_maximum(function, grid[i - 1], grid[i + 1])
57         candidates.append((value, price))
```

Read this block:

- Line 54: for each entry yielded by `range(1, points - 1)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 55: test `values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i -...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56, `(price, value)`: Unpack the result of calling `golden_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 57: call `candidates.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `golden_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

Source lines 58–58:

```
58 value, price = max(candidates)
```

Read this block:

- Line 58, `(value, price)`: Unpack the result of calling `max(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 59–59:

```
59 return float(price), float(value)
```

Read this block:

- Line 59: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

static price

Original function: `static_price`. Source begins at line 61.

Solves the transformed binary-logit price first-order condition by bracketing and bisection for signed alpha<0. The effective cost can incorporate continuation value, allowing introductory subsidies; do not impose the coupon floor from empirical pricing on this theoretical solver.

Definition and stated intent:

Source lines 61–62:

```
61 def static_price(intercept, alpha, effective_cost=0.):
62     """Signed alpha<0; monotone transformed FOC, including introductory subsidies."""
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 63–64:

```
63 def foc(p):
64     return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 63: define nested callable `foc`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 65–65:

```
65 lo, hi = effective_cost, effective_cost+max(2., 2/abs(alpha))
```

Read this block:

- Line 65, `(lo, hi)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 66–67:

```
66 while foc(hi) > 0:
67     hi = lo+2*(hi-lo)
```

Read this block:

- Line 66: repeat while `foc(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 67, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `foc`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 68–73:

```
68 for _ in range(100):
69     mid = (lo+hi)/2
70     if foc(mid) > 0:
71         lo = mid
72     else:
73         hi = mid
```

Read this block:

- Line 68: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 69, `mid`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 70: test `foc(mid) > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 71, `lo`: Set this name to the object currently named `mid`; plain assignment does not copy it.
- Line 73, `hi`: Set this name to the object currently named `mid`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 74–74:

```
74 return (lo+hi)/2
```

Read this block:

- Line 74: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

static price.foc

Original function: `static_price.foc`. Source begins at line 63.

This nested function returns the transformed price first-order-condition residual for a trial scalar. Captured slope, intercept and effective cost remain fixed during the root search. A zero is a candidate optimal price under the maintained monotonicity argument.

Definition and stated intent:

Source lines 63–63:

```
63 def foc(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 64–64:

```
64 return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 64: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.

supply solution

Original function: `supply_solution`. Source begins at line 76.

Solves the known-state baseline, hidden-initial-state case, and commitment case for one fixed type and prior. Both terminal histories enter expected profits. The hidden-state case pools initial histories within a known type; it does not pool different gamma types. The commitment case chooses two numbers before the first purchase and cannot adapt p2 to h1.

Definition and stated intent:

Source lines 76–77:

```
76 def supply_solution(gamma, prior=.5, beta=2., alpha=-1., delta=.92, cost=0.):
77     """Known type; hidden h0, then either observe h1 or commit to p1,p2."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `prior`, `beta`, `alpha`, `delta`, `cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–80:

```
78 if not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all()
79         and alpha < 0 and 0 <= prior <= 1 and 0 <= delta <= 1):
80     raise ValueError("Need finite inputs, alpha<0, prior/delta in [0,1].")
```

Read this block:

- Line 78: test `not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all() and alpha < ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 80: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 82–87:

```
82 intercepts = beta + np.array([-gamma, gamma])
83 weights = np.array([prior, 1 - prior])
84 p2 = np.array([static_price(x, alpha, cost) for x in intercepts])
85 s2 = expit(intercepts + alpha * p2)
86 W2 = (p2 - cost) * s2
87 adjustment = delta * (W2[0] - W2[1])
```

Read this block:

- Line 82, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 84, `p2`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 85, `s2`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 86, `W2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 87, `adjustment`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `static_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 89–90:

```
89 def share1(p):
90     return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 89: define nested callable `share1`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 92–94:

```
92 def adaptive_value(p):
93     s = share1(p)
94     return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 92: define nested callable `adaptive_value`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 96–100:

```
96 scale = (20 + abs(beta) + abs(gamma) + abs(alpha * cost)) / abs(alpha)
97 bounds = (cost - scale, cost + scale)
98 pooled_p1, adaptive = scalar_maximum(adaptive_value, bounds)
99 # Rechecking a wider domain catches a binding numerical search boundary.
100 wide_p1, wide_value = scalar_maximum(adaptive_value, (cost - 2*scale, cost + 2*scale), 321)
```

Read this block:

- Line 96, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97, `bounds`: Set this name to a tuple containing 2 displayed entries.
- Line 98, `(pooled_p1, adaptive)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 100, `(wide_p1, wide_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 101–102:

```
101 if wide_value > adaptive + 1e-8:
102     pooled_p1, adaptive = wide_p1, wide_value
```

Read this block:

- Line 101: test `wide_value > adaptive + 1e-08`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 102, `(pooled_p1, adaptive)`: Unpack the tuple `(wide_p1, wide_value)` into these targets in order.

Source lines 103–104:

```
103 known_p1 = np.array([static_price(x, alpha, cost - adjustment) for x in intercepts])
104 known_values = []
```

Read this block:

- Line 103, `known_p1`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 104, `known_values`: Set this name to the list `[]`.

Source lines 105–107:

```
105 for x, price in zip(intercepts, known_p1):
106     s = expit(x + alpha * price)
107     known_values.append(s*(price-cost) + delta*(s*W2[0]+(1-s)*W2[1]))
```

Read this block:

- Line 105: for each entry yielded by `zip(intercepts, known_p1)`, bind/unpack it into `(x, price)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 106, `s`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 107: call `known_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 109–116:

```
109 def committed_profile(second, return_price=False):
110     terminal = expit(intercepts + alpha * second)
111     Wj, W0 = terminal*(second-cost)
112     def objective(first):
113         s = share1(first)
114         return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
115     price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
116     return (price, value) if return_price else value
```

Read this block:

- Line 109: define nested callable `committed_profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 120–131:

```
120 if delta == 0:
121     cp1, cp2, committed = pooled_p1, static_price(beta, alpha, cost), adaptive
122 else:
123     cp2, committed = scalar_maximum(committed_profile, (cost, cost+scale))
124     cp1, committed = committed_profile(cp2, True)
125     # Independent wider and denser profile scan.
126     check_p2, check_value = scalar_maximum(committed_profile, (cost, cost+2*scale), 321)
127     if check_value > committed+1e-9:
128         cp2 = check_p2
129         cp1, committed = committed_profile(cp2, True)
130     if min(cp2-cost, cost+2*scale-cp2) < scale*1e-5:
131         raise RuntimeError("Committed optimum is near a search bound; inspect calibration.")
```

Read this block:

- Line 120: test `delta == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 121, `(cp1, cp2, committed)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 123, `(cp2, committed)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 124, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 126, `(check_p2, check_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 127: test `check_value > committed + 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128, `cp2`: Set this name to the object currently named `check_p2`; plain assignment does not copy it.
- Line 129, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 130: test `min(cp2 - cost, cost + 2 * scale - cp2) < scale * 1e-05`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 131: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `committed_profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 132–132:

```
132 known = float(weights @ known_values)
```

Read this block:

- Line 132, `known`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 133–134:

```
133 if not known + 1e-7 >= adaptive >= committed - 1e-7:
134     raise AssertionError("Information-value ordering failed.")
```

Read this block:

- Line 133: test `not known + 1e-07 >= adaptive >= committed - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 134: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 135–141:

```
135 s = share1(pooled_p1)
136 derivative = float(weights @ (alpha*expit(intercepts+alpha*pooled_p1)
137                              * expit(-intercepts-alpha*pooled_p1)))
138 committed_first_share = share1(cp1)
139 committed_first_derivative = float(weights @ (alpha*expit(intercepts+alpha*cp1)*expit(-intercepts-alpha*cp1)))
140 terminal = expit(intercepts+alpha*cp2)
141 terminal_weights = np.array([committed_first_share, 1-committed_first_share])
```

Read this block:

- Line 135, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 136, `derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 138, `committed_first_share`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 139, `committed_first_derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 140, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 141, `terminal_weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 142–143:

```
142 committed_foc1 =
    ↪  committed_first_share+committed_first_derivative*(cp1-cost+delta*(cp2-cost)*(terminal[0]-terminal[1]))
143 committed_foc2 = float(terminal_weights @ (terminal+alpha*(cp2-cost)*terminal*(1-terminal))) if delta > 0 else 0.
```

Read this block:

- Line 142, `committed_foc1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 143, `committed_foc2`: Set this name to the true-branch value if `delta > 0` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 144–152:

```
144 return dict(gamma=gamma, prior_inside=prior, beta=beta, alpha=alpha, delta=delta, cost=cost,
145             known_p1_inside=known_p1[0], known_p1_outside=known_p1[1],
146             known_value=known, hidden_p1=pooled_p1, adaptive_p2_inside=p2[0],
147             adaptive_p2_outside=p2[1], hidden_value=adaptive,
148             committed_p1=cp1, committed_p2=cp2, committed_value=committed,
149             initial_information_value=known-adaptive,
150             second_period_flexibility_value=adaptive-committed,
151             committed_foc1=committed_foc1, committed_foc2=committed_foc2,
152             hidden_foc=s+derivative*(pooled_p1-cost+adjustment))
```

Read this block:

- Line 144: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

supply solution.share1

Original function: `supply_solution.share1`. Source begins at line 89.

Returns first-period binary-logit demand, or its initial-state mixture, at a trial price. The function closes over the current theoretical type and prior. Initial history is integrated as probabilities, not averaged as utilities.

Definition and stated intent:

Source lines 89–89:

```
89 def share1(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 90–90:

```
90 return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 90: return matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

supply solution.adaptive value

Original function: `supply_solution.adaptive_value`. Source begins at line 92.

Evaluates current profit plus discounted, state-contingent optimized terminal values. The first choice changes the probabilities of both continuation histories, which is why its derivative contains their value difference.

Definition and stated intent:

Source lines 92–92:

```
92 def adaptive_value(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 93–93:

```
93 s = share1(p)
```

Read this block:

- Line 93, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 94–94:

```
94 return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 94: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

supply solution.committed profile

Original function: `supply_solution.committed_profile`. Source begins at line 109.

For a trial first price, solves the best single committed terminal price and returns its value/profile. This inner optimization must respect the information restriction: p2 is common across realized h1.

Definition and stated intent:

Source lines 109–109:

```
109 def committed_profile(second, return_price=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `return_price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 110–111:

```
110 terminal = expit(intercepts + alpha * second)
111 Wj, W0 = terminal*(second-cost)
```

Read this block:

- Line 110, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 111, `(Wj, W0)`: Unpack multiplication of the displayed operands; parentheses control which subexpressions are evaluated together into these targets in order.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 112–114:

```
112 def objective(first):
113     s = share1(first)
114     return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 112: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 115–115:

```
115 price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
```

Read this block:

- Line 115, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 116–116:

```
116 return (price, value) if return_price else value
```

Read this block:

- Line 116: return the true-branch value if `return_price` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

supply solution.committed profile.objective

Original function: `supply_solution.committed_profile.objective`. Source begins at line 112.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 112–112:

```
112 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 113–113:

```
113 s = share1(first)
```

Read this block:

- Line 113, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 114–114:

```
114 return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 114: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

plotting

Original function: `plotting`. Source begins at line 154.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 154–154:

```
154 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 155–158:

```
155 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
156 import matplotlib
157 matplotlib.use("Agg")
158 import matplotlib.pyplot as plt
```

Read this block:

- Line 155: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 156: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 157: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 158: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 159–159:

```
159 return plt
```

Read this block:

- Line 159: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

outcome tables

Original function: `outcome_tables`. Source begins at line 166.

Exports history-specific prices, quantities and profits plus properly probability-weighted ex ante outcomes. Conditional realized-history values and expected profits answer different questions; averaging conditional profits without their path probabilities would be wrong.

Definition and stated intent:

Source lines 166–167:

```
166 def outcome_tables(result):
167     """All four realized histories per case, plus correctly weighted ex ante objects."""
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 168–169:

```
168 g, beta, alpha, delta, cost, pi = [result[k] for k in ["gamma", "beta", "alpha", "delta", "cost", "prior_inside"]]
169 paths, summary = [], []
```

Read this block:

- Line 168, `(g, beta, alpha, delta, cost, pi)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 169, `(paths, summary)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 170–208:

```
170 for case in CASES:
171     case_paths = []
172     for h0, weight in [("j", pi), ("0", 1-pi)]:
173         if case == "known_state":
174             p1 = result["known_p1_inside" if h0 == "j" else "known_p1_outside"]
175         else:
176             p1 = result["hidden_p1" if case == "hidden_initial_state" else "committed_p1"]
177         s1 = float(expit(beta+(-1 if h0 == "j" else 1)*g+alpha*p1))
178         conditional_value = 0.
179         local = []
180         for h1, transition in [("j", s1), ("0", 1-s1)]:
181             p2 = result["committed_p2"] if case == "ex_ante_commitment" else result["adaptive_p2_inside" if h1 ==
    ↪ "j" else "adaptive_p2_outside"]
182             s2 = float(expit(beta+(-1 if h1 == "j" else 1)*g+alpha*p2))
183             # Realize the first choice; integrate the terminal taste shock.
184             conditional_profit = (p1-cost)*(h1 == "j")+delta*(p2-cost)*s2
185             conditional_value += transition*conditional_profit
186             row = dict(gamma=g, prior_inside=pi, case=case, h0=h0, h1=h1,
187                        initial_probability=weight, transition_probability=transition,
188                        path_probability=weight*transition, p1=p1, p2=p2,
189                        price_ratio=p1/p2 if abs(p2) > 1e-12 else np.nan,
190                        first_purchase_probability=s1, second_purchase_probability=s2,
191                        conditional_profit=conditional_profit)
192             local.append(row)
193         for row in local:
194             row["optimal_profit_given_h0"] = conditional_value
```

```
195         case_paths.extend(local)
196     probability = np.array([r["path_probability"] for r in case_paths])
197     if not np.isclose(probability.sum(), 1.):
198         raise AssertionError("Path probabilities do not sum to one.")
199     mean_p1, mean_p2, profit = [float(probability @ np.array([r[k] for r in case_paths]))
200                                 for k in ["p1", "p2", "conditional_profit"]]
201     expected_ratio = float(probability @ np.array([r["price_ratio"] for r in case_paths]))
202     expected_key = {"known_state": "known_value", "hidden_initial_state": "hidden_value", "ex_ante_commitment":
    ↪ "committed_value"}[case]
203     if not np.isclose(profit, result[expected_key], atol=1e-8):
204         raise AssertionError("Path profits do not reproduce optimized objective.")
205     summary.append(dict(gamma=g, prior_inside=pi, case=case, expected_p1=mean_p1,
206                         expected_p2=mean_p2, ratio_of_expected_prices=mean_p1/mean_p2 if abs(mean_p2) > 1e-12 else
    ↪ np.nan,
207                         expected_price_ratio=expected_ratio, optimal_profit=profit))
208     paths.extend(case_paths)
```

Read this block:

- Line 170: for each entry yielded by `CASES`, bind/unpack it into `case` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 171, `case_paths`: Set this name to the list `[]`.
- Line 172: for each entry yielded by `[('j', pi), ('0', 1 - pi)]`, bind/unpack it into `(h0, weight)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 173: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 174, `p1`: Set this name to the entry/slice selected by `result['known_p1_inside' if h0 == 'j' else 'known_p1_outs...`; brackets select from the object to their left.
- Line 176, `p1`: Set this name to the entry/slice selected by `result['hidden_p1' if case == 'hidden_initial_state' else...`; brackets select from the object to their left.
- Line 177, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 178, `conditional_value`: Set this name to the literal value `0.0`.
- Line 179, `local`: Set this name to the list `[]`.
- Line 180: for each entry yielded by `[('j', s1), ('0', 1 - s1)]`, bind/unpack it into `(h1, transition)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 181, `p2`: Set this name to the true-branch value if `case == 'ex_ante_commitment'` holds, otherwise the else-branch value.
- Line 182, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 184, `conditional_profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 185: update `conditional_value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 186, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 192: call `local.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193: for each entry yielded by `local`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 194, `row['optimal_profit_given_h0']`: Store the object currently named `conditional_value`; plain assignment does not copy it in the selected entry/column.
- Line 195: call `case_paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 196, `probability`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 197: test `not np.isclose(probability.sum(), 1.0)`. Execute the indented branch only when it is true;

an else/elif branch handles its alternative.
- Line 198: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 199, `(mean_p1, mean_p2, profit)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 201, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 202, `expected_key`: Set this name to the entry/slice selected by `{'known_state': 'known_value', 'hidden_initial_state': 'h...`; brackets select from the object to their left.
- Line 203: test `not np.isclose(profit, result[expected_key], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 204: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 205: call `summary.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 208: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 209–209:

```
209 return paths, summary
```

Read this block:

- Line 209: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

make figures

Original function: `make_figures`. Source begins at line 212.

Turns already computed supply solution tables into case-comparison figures; it does not add another solver or change the information assumptions. Plot labels distinguish state-contingent policies from committed prices.

Definition and stated intent:

Source lines 212–212:

```
212 def make_figures(paths, summary, output):
```

`def` creates this callable; the parentheses name its inputs (`paths`, `summary`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 213–213:

```
213 plt = plotting()
```

Read this block:

- Line 213, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 214–255:

```
214 for prior in summary.prior_inside.unique():
215     stem = f"prior_{prior:g}".replace(".", "p")
216     subset = paths[paths.prior_inside == prior]
217     fig, axes = plt.subplots(3, 3, figsize=(16, 12))
218     for column, case in enumerate(CASES):
219         data = subset[subset.case == case]
220         if case == "known_state":
221             for h0 in ["0", "j"]:
222                 initial = data[(data.h0 == h0) & (data.h1 == "j")]
223                 axes[0, column].plot(initial.gamma, initial.p1, label=f"p1(h0={h0})")
224         else:
225             initial = data[(data.h0 == "0") & (data.h1 == "j")]
226             axes[0, column].plot(initial.gamma, initial.p1, label="p1")
227         if case == "ex_ante_commitment":
228             axes[0, column].plot(initial.gamma, initial.p2, label="p2")
229         else:
230             for h1 in ["0", "j"]:
231                 terminal = data[(data.h0 == "0") & (data.h1 == h1)]
232                 axes[0, column].plot(terminal.gamma, terminal.p2, "--", label=f"p2(h1={h1})")
233         for (h0, h1), history in data.groupby(["h0", "h1"]):
234             # Duplicate ratios in restricted cases need only one curve.
235             if case != "known_state" and h0 != "0":
236                 continue
237             if case == "ex_ante_commitment" and h1 != "j":
238                 continue
239             axes[1, column].plot(history.gamma, history.price_ratio, label=f"h0={h0}, h1={h1}")
240         for h0 in ["0", "j"]:
241             initial = data[(data.h0 == h0) & (data.h1 == "j")]
242             axes[2, column].plot(initial.gamma, initial.optimal_profit_given_h0, label=f"E[profit|h0={h0}]")
243         expected = summary[(summary.prior_inside == prior) & (summary.case == case)]
244         axes[2, column].plot(expected.gamma, expected.optimal_profit, "k--", label="Ex ante E[profit]")
245         axes[0, column].set_title(LABELS[case])
246     for row, ylabel in enumerate(["Optimal prices", "Path-specific p1 / p2", "Optimal discounted profit"]):
247         for ax in axes[row]:
248             ax.set(xlabel="gamma", ylabel=ylabel)
249             ax.axhline(0, color="gray", linewidth=.5)
250             ax.legend(fontsize=7)
251     fig.suptitle(f"Supply cases: Pr(h0=j)={prior:g}")
252     fig.tight_layout()
253     fig.savefig(output / f"supply_cases_{stem}.png", dpi=180)
254     fig.savefig(output / f"supply_cases_{stem}.pdf")
255     plt.close(fig)
```

Source lines 256–280:

```
256
257 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
258 for case, group in summary[summary.prior_inside == prior].groupby("case", sort=False):
259     axes[0].plot(group.gamma, group.optimal_profit, label=LABELS[case])
```

```
260     axes[1].plot(group.gamma, group.ratio_of_expected_prices, label=LABELS[case])
261     axes[2].plot(group.gamma, group.expected_price_ratio, label=LABELS[case])
262 for ax, title in zip(axes, ["Optimal expected profit", "E[p1] / E[p2]", "E[p1 / p2]"]):
263     ax.set(xlabel="gamma", title=title)
264     ax.legend(fontsize=7)
265 fig.tight_layout()
266 fig.savefig(output / f"supply_comparison_{stem}.png", dpi=180)
267 fig.savefig(output / f"supply_comparison_{stem}.pdf")
268 plt.close(fig)
269
270 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
271 for ax, case in zip(axes, CASES):
272     data = subset[subset.case == case]
273     for (h0, h1), history in data.groupby(["h0", "h1"]):
274         ax.plot(history.gamma, history.conditional_profit, label=f"h0={h0}, h1={h1}")
275     ax.set(xlabel="gamma", ylabel="Path-conditional discounted profit", title=LABELS[case])
276     ax.legend(fontsize=7)
277 fig.tight_layout()
278 fig.savefig(output / f"supply_history_profit_{stem}.png", dpi=180)
279 fig.savefig(output / f"supply_history_profit_{stem}.pdf")
280 plt.close(fig)
```

Read this block:

- Line 214: for each entry yielded by `summary.prior_inside.unique()`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 215, `stem`: Set this name to the result of calling `f'prior_{prior:g}'.replace(...)`; the arguments and preceding method chain are shown above.
- Line 216, `subset`: Set this name to the entry/slice selected by `paths[paths.prior_inside == prior]`; brackets select from the object to their left.
- Line 217, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 218: for each entry yielded by `enumerate(CASES)`, bind/unpack it into `(column, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 219, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 220: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 221: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 222, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 223: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 225, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 226: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 227: test `case == 'ex_ante_commitment'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 228: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 230: for each entry yielded by `['0', 'j']`, bind/unpack it into `h1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 231, `terminal`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == h1)]`; brackets select from the object to their left.
- Line 232: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 233: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 235: test `case != 'known_state' and h0 != '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 236: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 237: test `case == 'ex_ante_commitment' and h1 != 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 238: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 239: call `axes[1, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 240: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 241, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 242: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 243, `expected`: Set this name to the entry/slice selected by `summary[(summary.prior_inside == prior) & (summary.case =...`; brackets select from the object to their left.
- Line 244: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 245: call `axes[0, column].set_title` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 246: for each entry yielded by `enumerate(['Optimal prices', 'Path-specific p1 / p2', 'Optimal...`, bind/unpack it into `(row, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 247: for each entry yielded by `axes[row]`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 248: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 249: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 250: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 251: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 252: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 253: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 254: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 255: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 257, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 258: for each entry yielded by `summary[summary.prior_inside == prior].groupby('case', sort=Fa...`, bind/unpack it into `(case, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 259: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 260: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 261: call `axes[2].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 262: for each entry yielded by `zip(axes, ['Optimal expected profit', 'E[p1] / E[p2]', 'E[p1 /...`, bind/unpack it into `(ax, title)` and run the indented body. Changes inside an iteration can feed subsequent

iterations.
- Line 263: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 264: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 265: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 266: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 267: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 268: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 270, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 271: for each entry yielded by `zip(axes, CASES)`, bind/unpack it into `(ax, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 272, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 273: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 274: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 275: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 276: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 277: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 278: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 279: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 280: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 283.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 283–283:

```
283 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 284–295:

```
284 for gamma, prior, delta in [(0., .5, .92), (-2., .25, .92), (1.5, .75, .92),
285                             (-1., 0., .92), (1., 1., .92), (1., .5, 0.)]:
286     solution = supply_solution(gamma, prior, delta=delta)
287     paths, summary = outcome_tables(solution)
288     assert len(paths) == 12 and len(summary) == 3
289     for key in ["hidden_foc", "committed_foc1", "committed_foc2"]:
290         assert abs(solution[key]) < 1e-6
291     if gamma == 0:
292         assert np.ptp([row["optimal_profit"] for row in summary]) < 1e-7
293         assert all(np.isclose(row["price_ratio"], 1., atol=1e-6) for row in paths)
294     if prior in (0., 1.):
295         assert np.isclose(solution["known_value"], solution["hidden_value"], atol=1e-7)
```

Read this block:

- Line 284: for each entry yielded by `[(0.0, 0.5, 0.92), (-2.0, 0.25, 0.92), (1.5, 0.75, 0.92), (-1....`, bind/unpack it into `(gamma, prior, delta)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 286, `solution`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 287, `(paths, summary)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 288: require `len(paths) == 12 and len(summary) == 3` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 289: for each entry yielded by `['hidden_foc', 'committed_foc1', 'committed_foc2']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 290: require `abs(solution[key]) < 1e-06` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 291: test `gamma == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 292: require `np.ptp([row['optimal_profit'] for row in summary]) < 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 293: require `all((np.isclose(row['price_ratio'], 1.0, atol=1e-06) for row in paths))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

- Line 294: test `prior in (0.0, 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295: require `np.isclose(solution['known_value'], solution['hidden_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `len`: Count elements/rows rather than summing their numerical values.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 296–296:

```
296 print("Supply checks passed: all cases, four histories, probabilities, profit reconstruction, FOCs, degenerate
    ↪  priors, gamma=0, delta=0.")
```

Read this block:

- Line 296: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 299.

Parses the theoretical gamma grid, initial-history priors, cost/discount settings and output directory, then solves all three information cases and writes solution/outcome tables and figures. The self-test branch uses theoretical calibrations. No household sample or fitted empirical distribution is needed.

Definition and stated intent:

Source lines 299–299:

```
299 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 300–305:

```
300 parser = argparse.ArgumentParser(description=__doc__)
301 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "supply_outputs")
302 parser.add_argument("--gamma-min", type=float, default=-3.)
303 parser.add_argument("--gamma-max", type=float, default=3.)
304 parser.add_argument("--gamma-points", type=int, default=61)
305 parser.add_argument("--priors", type=float, nargs="+", default=[.25, .5, .75])
```

Read this block:

- Line 300, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 301: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 302: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 303: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 304: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 305: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 306–311:

```
306 parser.add_argument("--beta", type=float, default=2.)
307 parser.add_argument("--alpha", type=float, default=-1., help="Signed price coefficient, negative.")
308 parser.add_argument("--delta", type=float, default=.92)
309 parser.add_argument("--cost", type=float, default=0.)
310 parser.add_argument("--self-test", action="store_true")
311 args = parser.parse_args()
```

Read this block:

- Line 306: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 307: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 308: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 309: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 310: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 311, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 312–314:

```
312 if args.self_test:
313     self_test()
314     return
```

Read this block:

- Line 312: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 313: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 314: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 315–316:

```
315 if args.gamma_points < 2 or args.gamma_max <= args.gamma_min:
316     parser.error("Need >=2 gamma points and gamma-max > gamma-min.")
```

Read this block:

- Line 315: test `args.gamma_points < 2 or args.gamma_max <= args.gamma_min`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 316: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 317–318:

```
317 args.output.mkdir(parents=True, exist_ok=True)
318 solutions, paths, summaries = [], [], []
```

Read this block:

- Line 317: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 318, `(solutions, paths, summaries)`: Unpack a tuple containing 3 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 319–327:

```
319 for prior in args.priors:
320     for i, gamma in enumerate(np.linspace(args.gamma_min, args.gamma_max, args.gamma_points)):
321         result = supply_solution(gamma, prior, args.beta, args.alpha, args.delta, args.cost)
322         history_rows, summary_rows = outcome_tables(result)
323         solutions.append(result)
324         paths.extend(history_rows)
325         summaries.extend(summary_rows)
326         if i % 10 == 0 or i == args.gamma_points-1:
327             print(f"Prior {prior:g}: solved {i+1}/{args.gamma_points} gamma values", flush=True)
```

Read this block:

- Line 319: for each entry yielded by `args.priors`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 320: for each entry yielded by `enumerate(np.linspace(args.gamma_min, args.gamma_max, args.gam...`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 321, `result`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 322, `(history_rows, summary_rows)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 323: call `solutions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 324: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 325: call `summaries.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 326: test `i % 10 == 0 or i == args.gamma_points - 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 327: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 328–332:

```
328 raw, histories, expected = pd.DataFrame(solutions), pd.DataFrame(paths), pd.DataFrame(summaries)
329 raw.to_csv(args.output / "supply_solutions.csv", index=False)
330 histories.to_csv(args.output / "supply_history_outcomes.csv", index=False)
331 expected.to_csv(args.output / "supply_expected_outcomes.csv", index=False)
332 residual = float(raw[["hidden_foc", "committed_foc1", "committed_foc2"]].abs().to_numpy().max())
```

Read this block:

- Line 328, `(raw, histories, expected)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 329: call `raw.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 330: call `histories.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 331: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 332, `residual`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 333–334:

```
333 if residual > 1e-6:
334     raise AssertionError(f"FOC residual too large: {residual}")
```

Read this block:

- Line 333: test `residual > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 334: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 335–340:

```
335 make_figures(histories, expected, args.output)
336 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k,v in vars(args).items()},
337                 maximum_foc_residual=residual, calibrations=len(raw), cases=CASES,
338                 script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest())
339 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
340 print(f"All three supply cases completed. CSVs and PNG/PDF figures: {args.output}")
```

Read this block:

- Line 335: call `make_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 336, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 339: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 340: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

pricing__exercises/unknown__type/plot__supply__information.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 2–7:

```
2 """Recreate supply_information for unknown gamma and observed initial history."""
3 from pathlib import Path
4 from types import SimpleNamespace
5 import argparse
6 import hashlib
```

```
import json
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 6: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 7: import `json`. An as-name is a convenient local alias, not another library.

Source lines 8–11:

```
import sys

import numpy as np
import pandas as pd
```

Read this block:

- Line 8: import `sys`. An as-name is a convenient local alias, not another library.
- Line 10: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 11: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 13–16:

```
if __package__:
    from .run_unknown_type import GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting
else:
    from run_unknown_type import GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting
```

Read this block:

- Line 13: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 14: from `run_unknown_type`, import `GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `run_unknown_type`, import `GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–24:

```
output = Path(__file__).resolve().parent / "outputs"
probability_points = 41
beta = 2.0
alpha = -1.0
delta = 0.92
cost = 0.0
```

Read this block:

- Line 19, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 20, `probability_points`: Set this name to the literal value `41`.
- Line 21, `beta`: Set this name to the literal value `2.0`.
- Line 22, `alpha`: Set this name to the negated or sign-adjusted expression `-1.0`.
- Line 23, `delta`: Set this name to the literal value `0.92`.
- Line 24, `cost`: Set this name to the literal value `0.0`.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 155–160:

```
155 if __name__ == "__main__":
156     settings = SimpleNamespace(output=output, probability_points=probability_points,
157                                beta=beta, alpha=alpha, delta=delta, cost=cost)
158     if len(sys.argv) > 1:
159         settings = read_command_line(settings)
160     run_information(settings)
```

Read this block:

- Line 155: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 158: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 159, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 160: call `run_information` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_information`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

committed policy

Original function: `committed_policy`. Source begins at line 27.

Chooses two common prices before outcomes when initial history is known but gamma is hidden. Both prices must apply across possible types; the method profiles over the terminal price and verifies first-order conditions. This is a commitment/Bayesian-theory comparison, distinct from empirical fixed beliefs.

Definition and stated intent:

Source lines 27–28:

```
27 def committed_policy(h0, weights, beta, alpha, delta, cost, points=81):
28     """Two common prices chosen before purchases; h0 is observed, gamma is not."""
```

`def` creates this callable; the parentheses name its inputs (`h0`, `weights`, `beta`, `alpha`, `delta`, `cost`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–30:

```
29 intercepts = beta+(-1 if h0 == "j" else 1)*GAMMAS
30 scale = (20+abs(beta)+max(abs(GAMMAS))+abs(alpha*cost))/abs(alpha)
```

Read this block:

- Line 29, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 30, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 32–42:

```
def profile(second, details=False):
    sj = expit(beta-GAMMAS+alpha*second)
    s0 = expit(beta+GAMMAS+alpha*second)

    def objective(first):
        shares = expit(intercepts+alpha*np.asarray(first)[..., None])
        return ((np.asarray(first)-cost)*(shares @ weights)
                + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))

    first, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale), points)
    return (first, value) if details else value
```

Read this block:

- Line 32: define nested callable `profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 44–51:

```
if delta:
    second, value = scalar_maximum(profile, (cost, cost+2*scale), points)
    first, value = profile(second, True)
    if min(second-cost, cost+2*scale-second) < 1e-6*scale:
        raise RuntimeError("Committed terminal price is near a search bound.")
else:
    second = cost+1/abs(alpha)  # Unidentified by the objective at delta=0.
    first, value = profile(second, True)
```

Read this block:

- Line 44: test `delta`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45, `(second, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 46, `(first, value)`: Unpack the result of calling `profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 47: test `min(second - cost, cost + 2 * scale - second) < 1e-06 * scale`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 50, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `(first, value)`: Unpack the result of calling `profile(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 52–58:

```
52 shares = expit(intercepts+alpha*first)
53 sj = expit(beta-GAMMAS+alpha*second)
54 s0 = expit(beta+GAMMAS+alpha*second)
55 ds = alpha*shares*(1-shares)
56 foc1 = float(weights @ (shares+ds*(first-cost+delta*(second-cost)*(sj-s0))))
57 foc2 = float(weights @ (shares*(sj+alpha*(second-cost)*sj*(1-sj))
58                        +(1-shares)*(s0+alpha*(second-cost)*s0*(1-s0)))) if delta else 0.
```

Read this block:

- Line 52, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 53, `sj`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 54, `s0`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 55, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 56, `foc1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 57, `foc2`: Set this name to the true-branch value if `delta` holds, otherwise the else-branch value.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 59–60:

```
59 if max(abs(foc1), abs(foc2)) > 1e-6:
60     raise AssertionError("Committed policy FOC failed.")
```

Read this block:

- Line 59: test `max(abs(foc1), abs(foc2)) > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 60: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 61–61:

```
61 return first, second, value
```

Read this block:

- Line 61: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

committed policy.profile

Original function: `committed_policy.profile`. Source begins at line 32.

For a candidate committed second price, optimizes the first price and returns either its value or detailed price/value pair. The details flag lets the outer optimizer use a scalar while the caller later recovers the selected policy.

Definition and stated intent:

Source lines 32–32:

```
32 def profile(second, details=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `details`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 33–34:

```
33 sj = expit(beta-GAMMAS+alpha*second)
34 s0 = expit(beta+GAMMAS+alpha*second)
```

Read this block:

- Line 33, `sj`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 34, `s0`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 36–39:

```
36 def objective(first):
37     shares = expit(intercepts+alpha*np.asarray(first)[..., None])
38     return ((np.asarray(first)-cost)*(shares @ weights)
39             + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))
```

Read this block:

- Line 36: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 41–41:

```
41 first, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale), points)
```

Read this block:

- Line 41, `(first, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 42–42:

```
42 return (first, value) if details else value
```

Read this block:

- Line 42: return the true-branch value if `details` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

committed policy.profile.objective

Original function: `committed_policy.profile.objective`. Source begins at line 36.

Weights each persistent type's first purchase and second-period branch revenues under a single committed policy. Broadcasting permits evaluating a price grid without changing which consumers keep their type.

Definition and stated intent:

Source lines 36–36:

```
36 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–37:

```
37 shares = expit(intercepts+alpha*np.asarray(first)[..., None])
```

Read this block:

- Line 37, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 38–39:

```
38 return ((np.asarray(first)-cost)*(shares @ weights)
39         + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))
```

Read this block:

- Line 38: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

draw figures

Original function: `draw_figures`. Source begins at line 64.

Draws prices, expected profits and the value of observing first-period history as type probabilities vary. Grouping selects each focal gamma and initial state. These are theoretical curves, not newly estimated empirical revenues.

Definition and stated intent:

Source lines 64–65:

```
64 def draw_figures(results, output):
65     """Use the original figure's horizontal three-panel line-plot layout."""
```

`def` creates this callable; the parentheses name its inputs (`results`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 66–66:

```
66 plt = plotting()
```

Read this block:

- Line 66, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–96:

```
67 for (focal_gamma, h0), data in results.groupby(["focal_gamma", "h0"], sort=False):
68     state = "inside" if h0 == "j" else "outside"
69     tag = {-1.25: "negative", 0.: "zero", 1.25: "positive"}[focal_gamma]
70     stem = ("supply_information" if focal_gamma == 1.25 and h0 == "0"
71             else f"supply_information_{tag}_h0_{state}")
72     x = data.focal_probability
73     with plt.rc_context({"font.size": 11, "axes.titlesize": 13}):
74         fig, axes = plt.subplots(1, 3, figsize=(17, 4.6), layout="constrained")
75         for key, label in [("p1", "Unknown gamma: adaptive p1"),
76                            ("committed_p1", "Unknown gamma: committed p1"),
77                            ("committed_p2", "Unknown gamma: committed p2")]:
78             axes[0].plot(x, data[key], label=label, linewidth=2)
79         axes[0].set(title="Optimal prices", ylabel="Price")
80         for key, label in [("known_gamma_profit", "Gamma revealed before pricing"),
81                            ("optimal_profit", "Gamma unknown: observe h1"),
82                            ("committed_profit", "Gamma unknown: commit to both prices")]:
83             axes[1].plot(x, data[key], label=label, linewidth=2)
84         axes[1].set(title="Expected two-period profit", ylabel="Discounted expected profit")
85         axes[2].plot(x, data.value_observing_h1, linewidth=2,
86                      label="Adaptive profit - committed profit")
87         axes[2].set(title="Value of observing h1", ylabel="Expected profit gain")
88         axes[2].set_ylim(bottom=0)
89         for ax in axes:
90             ax.set(xlabel=f"Pr(gamma = {focal_gamma:g})", xlim=(0, 1))
91             ax.legend(fontsize=8, frameon=True)
92         fig.suptitle(f"Known h0: {state} | Gamma unobserved in {{-1.25, 0, +1.25}} | Remaining probability split
   ↪ equally",
93                      fontsize=13)
94         for extension in ["png", "pdf"]:
95             fig.savefig(output / f"{stem}.{extension}", dpi=180)
96         plt.close(fig)
```

Read this block:

- Line 67: for each entry yielded by `results.groupby(['focal_gamma', 'h0'], sort=False)`, bind/unpack it into `((focal_gamma, h0), data)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 68, `state`: Set this name to the true-branch value if `h0 == 'j'` holds, otherwise the else-branch value.
- Line 69, `tag`: Set this name to the entry/slice selected by `{-1.25: 'negative', 0.0: 'zero', 1.25: 'positive'}[focal_...`; brackets select from the object to their left.
- Line 70, `stem`: Set this name to the true-branch value if `focal_gamma == 1.25 and h0 == '0'` holds, otherwise the else-branch value.
- Line 72, `x`: Set this name to the attribute `data.focal_probability` of the object before the dot.
- Line 73: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 74, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 75: for each entry yielded by `[('p1', 'Unknown gamma: adaptive p1'), ('committed_p1', 'Unkno...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 78: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.
- Line 79: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: for each entry yielded by `[('known_gamma_profit', 'Gamma revealed before pricing'), ('op...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 83: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `axes[2].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `axes[2].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `axes[2].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: for each entry yielded by `axes`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 90: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 95: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run information

Original function: `run_information`. Source begins at line 99.

Loops over known initial states and probability grids, compares known gamma, adaptive hidden gamma and committed prices, checks value bounds, then saves theoretical figures and provenance. Its adaptive hidden-type model learns via outcomes; it is not the auxiliary fixed-belief specification.

Definition and stated intent:

Source lines 99–99:

```
def run_information(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 100–101:

```
if settings.probability_points < 3:
    raise ValueError("Need at least three probability points.")
```

Read this block:

- Line 100: test `settings.probability_points < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 101: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 102–103:

```
grid = np.unique(np.r_[np.linspace(0, 1, settings.probability_points), 1/3, .5])
rows = []
```

Read this block:

- Line 102, `grid`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.
- Line 103, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 104–131:

```
for h0 in ["0", "j"]:
    # A benchmark only: reveal gamma before pricing, while retaining known h0.
    known_values = np.array([supply_solution(g, float(h0 == "j"), settings.beta,
                             settings.alpha, settings.delta, settings.cost)["known_value"] for g in GAMMAS])
    for focal_index, gamma in enumerate(GAMMAS):
        for probability in grid:
            weights = np.full(3, (1-probability)/2)
            weights[focal_index] = probability
            _, policy, _ = solve(h0, weights, settings.beta, settings.alpha, settings.delta, settings.cost)
            first, second, committed_value = committed_policy(h0, weights, settings.beta,
                                             settings.alpha, settings.delta, settings.cost)
            adaptive_value = policy[0]["optimal_profit"]
            known_value = float(weights @ known_values)
            if not known_value+1e-7 >= adaptive_value >= committed_value-1e-7:
                raise AssertionError("Known gamma >= adaptive unknown gamma >= committed unknown gamma failed.")
            if any(np.isclose(probability, p, atol=1e-12) for p in [0., 1/3, .5, 1.]):
                _, dense_policy, _ = solve(h0, weights, settings.beta, settings.alpha,
                                           settings.delta, settings.cost, points=161)
                _, _, dense_committed = committed_policy(h0, weights, settings.beta, settings.alpha,
                                                         settings.delta, settings.cost, points=161)
                assert np.isclose(dense_policy[0]["optimal_profit"], adaptive_value, atol=1e-7, rtol=0)
                assert np.isclose(dense_committed, committed_value, atol=1e-7, rtol=0)
```

```
126             row = dict(policy[0], focal_gamma=gamma, focal_probability=probability,
127                        committed_p1=first, committed_p2=second, committed_profit=committed_value,
128                        known_gamma_profit=known_value,
129                        value_observing_h1=max(0., adaptive_value-committed_value))
130             rows.append(row)
131         print(f"Information curves: known h0={h0}, varying Pr(gamma={gamma:g})", flush=True)
```

Read this block:

- Line 104: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 106, `known_values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 108: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(focal_index, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 109: for each entry yielded by `grid`, bind/unpack it into `probability` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 110, `weights`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 111, `weights[focal_index]`: Store the object currently named `probability`; plain assignment does not copy it in the selected entry/column.
- Line 112, `(_, policy, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 113, `(first, second, committed_value)`: Unpack the result of calling `committed_policy(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 115, `adaptive_value`: Set this name to the entry/slice selected by `policy[0]['optimal_profit']`; brackets select from the object to their left.
- Line 116, `known_value`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 117: test `not known_value + 1e-07 >= adaptive_value >= committed_value - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 118: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 119: test `any((np.isclose(probability, p, atol=1e-12) for p in [0.0, 1 / 3, 0.5, 1.0]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 120, `(_, dense_policy, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `(_, _, dense_committed)`: Unpack the result of calling `committed_policy(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 124: require `np.isclose(dense_policy[0]['optimal_profit'], adaptive_value, atol=1e-07, rtol=0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 125: require `np.isclose(dense_committed, committed_value, atol=1e-07, rtol=0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 126, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 130: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress

promptly.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `committed_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 132–141:

```
settings.output.mkdir(parents=True, exist_ok=True)
results = pd.DataFrame(rows)
results.to_csv(settings.output / "supply_information_results.csv", index=False)
draw_figures(results, settings.output)
manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k, v in vars(settings).items()},
                information="h0 known in every case; gamma uncertain except in revealed-type benchmark",
                varying="Probability of one gamma; other two types split remaining probability equally",
                script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),

 ↪  solver_sha256=hashlib.sha256(Path(__file__).with_name("run_unknown_type.py").read_bytes()).hexdigest())
(settings.output / "supply_information_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 132: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133, `results`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 134: call `results.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `draw_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 141: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `draw_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `isinstance`: Check a runtime type to select the appropriate conversion path.

- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 142–142:

```
142 print(f"Saved six additional supply-information PNG/PDF figures to {settings.output}")
```

Read this block:

- Line 142: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 145.

Parses theoretical output/grid and beta/alpha/delta/cost overrides. These ordinary numerical parameters do not select empirical flavor groups.

Definition and stated intent:

Source lines 145–145:

```
145 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 146–148:

```
146 parser = argparse.ArgumentParser(description=__doc__)
147 parser.add_argument("--output", type=Path)
148 parser.add_argument("--probability-points", type=int)
```

Read this block:

- Line 146, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 147: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 149–150:

```
149 for name in ["beta", "alpha", "delta", "cost"]:
150     parser.add_argument(f"--{name}", type=float)
```

Read this block:

- Line 149: for each entry yielded by `['beta', 'alpha', 'delta', 'cost']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 150: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 151–151:

```
151 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 151: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 152–152:

```
152 return parser.parse_args()
```

Read this block:

- Line 152: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

pricing_exercises/unknown_type/run_unknown_type.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 2–7:

```
2 """Observed initial history, hidden fixed gamma, Bayesian learning from choices."""
3 from pathlib import Path
4 import argparse
5 import hashlib
6 import json
7 import sys
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 5: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 6: import `json`. An as-name is a convenient local alias, not another library.

- Line 7: import `sys`. An as-name is a convenient local alias, not another library.

Source lines 9–21:

```
9  import numpy as np
10 import pandas as pd
11
12 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
13 from run_supply_exercises import expit, scalar_maximum, supply_solution, plotting
14
15 GAMMAS = np.array([-1.25, 0., 1.25])
16 PRIORS = {
17     "favor_negative": np.array([.5, .25, .25]),
18     "favor_zero": np.array([.25, .5, .25]),
19     "favor_positive": np.array([.25, .25, .5]),
20     "equal": np.full(3, 1/3),
21 }
```

Read this block:

- Line 9: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 10: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 12: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 13: from `run_supply_exercises`, import `expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15, `GAMMAS`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 16, `PRIORS`: Set this name to a dictionary with named entries `favor_negative`, `favor_zero`, `favor_positive`, `equal`.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.

Source lines 219–220:

```
219 if __name__ == "__main__":
220     main()
```

Read this block:

- Line 219: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 220: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

solve

Original function: `solve`. Source begins at line 22.

Solves the historical Bayesian hidden-gamma theory for a known initial history. One common policy applies across possible types, with posterior learning after the first purchase. It returns theoretical path rows, policy summaries and numerical residuals; it is distinct from current fixed-belief empirical calibration.

Definition and stated intent:

Source lines 22–23:

```
def solve(h0, weights, beta=2., alpha=-1., delta=.92, cost=0., points=81):
    """One policy for a known h0 and a distribution over unobserved gamma."""
```

`def` creates this callable; the parentheses name its inputs (`h0`, `weights`, `beta`, `alpha`, `delta`, `cost`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–24:

```
weights = np.asarray(weights, dtype=float)
```

Read this block:

- Line 24, `weights`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 25–29:

```
if (h0 not in ["j", "0"] or weights.shape != (3,) or not np.isfinite(weights).all()
        or (weights < 0).any() or not np.isclose(weights.sum(), 1.)
        or not np.isfinite([beta, alpha, delta, cost]).all() or alpha >= 0
        or not 0 <= delta <= 1):
    raise ValueError("Need known h0, three normalized type probabilities, alpha<0, delta in [0,1].")
```

Read this block:

- Line 25: test `h0 not in ['j', '0'] or weights.shape != (3,) or (not np.isfinite(weights).al...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 29: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 30–31:

```
initial_intercepts = beta + (-1 if h0 == "j" else 1)*GAMMAS
scale = (20+abs(beta)+max(abs(GAMMAS))+abs(alpha*cost))/abs(alpha)
```

Read this block:

- Line 30, `initial_intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 31, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 33–39:

```
def maximize(function, terminal=False):
    lower = cost if terminal else cost-2*scale
    upper = cost+2*scale
    price, value = scalar_maximum(function, (lower, upper), points)
    if min(price-lower, upper-price) < 1e-6*scale:
        raise RuntimeError("Optimum near numerical bound; enlarge search domain.")
    return price, value
```

Read this block:

- Line 33: define nested callable `maximize`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 41–61:

```
def terminal_policy(first):
    # The firm observes h1, not gamma. Bayes' rule weights each possible type.
    inside = expit(initial_intercepts+alpha*first)
    outside = expit(-initial_intercepts-alpha*first)
    policies = []
    for h1, likelihood in [("j", inside), ("0", outside)]:
        joint = weights*likelihood
        probability = float(joint.sum())
        posterior = joint/probability
        intercepts = beta+(-1 if h1 == "j" else 1)*GAMMAS

        def expected_terminal_profit(price):
            shares = expit(intercepts+alpha*np.asarray(price)[..., None])
            return (np.asarray(price)-cost)*(shares @ posterior)

        price, value = maximize(expected_terminal_profit, terminal=True)
        share = expit(intercepts+alpha*price)
        foc = float(posterior @ (share+alpha*(price-cost)*share*(1-share)))
        policies.append(dict(h1=h1, p2=price, value=value,
                             probability=probability, posterior=posterior, foc=foc))
    return policies
```

Read this block:

- Line 41: define nested callable `terminal_policy`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 63–67:

```
def expected_profit(first):
    # Optimize the expectation over gamma, rather than prices for each gamma.
    purchase_probability = float(weights @ expit(initial_intercepts+alpha*first))
    continuation = sum(p["probability"]*p["value"] for p in terminal_policy(first))
    return (first-cost)*purchase_probability+delta*continuation
```

Read this block:

- Line 63: define nested callable `expected_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 69–71:

```
p1, optimal_profit = maximize(expected_profit)
terminal = terminal_policy(p1)
rows = []
```

Read this block:

- Line 69, `(p1, optimal_profit)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 70, `terminal`: Set this name to the result of calling `terminal_policy(...)`; the arguments and preceding method chain are shown above.
- Line 71, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `maximize`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `terminal_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 72–85:

```
72 for i, gamma in enumerate(GAMMAS):
73     s1 = float(expit(initial_intercepts[i]+alpha*p1))
74     for branch in terminal:
75         h1, p2 = branch["h1"], branch["p2"]
76         probability = s1 if h1 == "j" else float(expit(-initial_intercepts[i]-alpha*p1))
77         s2 = float(expit(beta+(-1 if h1 == "j" else 1)*gamma+alpha*p2))
78         row = dict(case="unknown_type", h0=h0, gamma=gamma, h1=h1,
79                    prior_type=weights[i], transition_probability=probability,
80                    path_probability=weights[i]*probability, p1=p1, p2=p2,
81                    second_purchase_probability=s2,
82                    conditional_profit=(p1-cost)*(h1 == "j")+delta*(p2-cost)*s2)
83         for k, name in enumerate(["negative", "zero", "positive"]):
84             row[f"posterior_{name}"] = branch["posterior"][k]
85         rows.append(row)
```

Read this block:

- Line 72: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 74: for each entry yielded by `terminal`, bind/unpack it into `branch` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 75, `(h1, p2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 76, `probability`: Set this name to the true-branch value if `h1 == 'j'` holds, otherwise the else-branch value.
- Line 77, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 78, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 83: for each entry yielded by `enumerate(['negative', 'zero', 'positive'])`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 84, `row[f'posterior_{name}']`: Store the entry/slice selected by `branch['posterior'][k]`; brackets select from the object to their left in the selected entry/column.
- Line 85: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 87–88:

```
87 probabilities = np.array([r["path_probability"] for r in rows])
88 reconstructed = float(probabilities @ [r["conditional_profit"] for r in rows])
```

Read this block:

- Line 87, `probabilities`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 88, `reconstructed`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 89–89:

```
89 assert np.isclose(probabilities.sum(), 1.)
```

Read this block:

- Line 89: require `np.isclose(probabilities.sum(), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 90–90:

```
90 assert np.isclose(reconstructed, optimal_profit, atol=1e-8)
```

Read this block:

- Line 90: require `np.isclose(reconstructed, optimal_profit, atol=1e-08)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 91–91:

```
91 assert np.allclose(sum(p["probability"]*p["posterior"] for p in terminal), weights)
```

Read this block:

- Line 91: require `np.allclose(sum((p['probability'] * p['posterior'] for p in terminal)), weights)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 92–94:

```
92 for branch in terminal:
93     joint = [r["path_probability"] for r in rows if r["h1"] == branch["h1"]]
94     assert np.allclose(branch["posterior"]*branch["probability"], joint, atol=1e-10)
```

Read this block:

- Line 92: for each entry yielded by `terminal`, bind/unpack it into `branch` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 93, `joint`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 94: require `np.allclose(branch['posterior'] * branch['probability'], joint, atol=1e-10)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 97–102:

```
97 shares = expit(initial_intercepts+alpha*p1)
98 derivative = alpha*shares*(1-shares)
99 future = [expit(beta+q*GAMMAS+alpha*p["p2"])*(p["p2"]-cost)
100           for q, p in zip([-1, 1], terminal)]
101 foc1 = float(weights @ (shares+derivative*(p1-cost+delta*(future[0]-future[1]))))
102 residual = max(abs(foc1), *(abs(p["foc"]) for p in terminal))
```

Read this block:

- Line 97, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 98, `derivative`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 99, `future`: Set this name to a list computed by the displayed for-expression.
- Line 101, `foc1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 102, `residual`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 103–104:

```
103 if residual > 1e-6:
104     raise AssertionError(f"FOC residual {residual}")
```

Read this block:

- Line 103: test `residual > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 104: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 105–108:

```
105 summary = dict(case="unknown_type", h0=h0, p1=p1,
106                p2_after_inside=terminal[0]["p2"], p2_after_outside=terminal[1]["p2"],
107                optimal_profit=optimal_profit, first_purchase_probability=float(weights @ shares),
108                prior_negative=weights[0], prior_zero=weights[1], prior_positive=weights[2])
```

Read this block:

- Line 105, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 109–109:

```
109 return rows, [summary], residual
```

Read this block:

- Line 109: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

solve.maximize

Original function: `solve.maximize`. Source begins at line 33.

Chooses bounds appropriate to an initial or terminal price and delegates grid/refinement maximization. A boundary optimum raises rather than being silently accepted.

Definition and stated intent:

Source lines 33–33:

```
33 def maximize(function, terminal=False):
```

`def` creates this callable; the parentheses name its inputs (`function`, `terminal`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–36:

```
34 lower = cost if terminal else cost-2*scale
35 upper = cost+2*scale
36 price, value = scalar_maximum(function, (lower, upper), points)
```

Read this block:

- Line 34, `lower`: Set this name to the true-branch value if `terminal` holds, otherwise the else-branch value.
- Line 35, `upper`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 36, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 37–38:

```
37 if min(price-lower, upper-price) < 1e-6*scale:
38     raise RuntimeError("Optimum near numerical bound; enlarge search domain.")
```

Read this block:

- Line 37: test `min(price - lower, upper - price) < 1e-06 * scale`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 38: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 39–39:

```
39 return price, value
```

Read this block:

- Line 39: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

solve.terminal policy

Original function: `solve.terminal_policy`. Source begins at line 41.

Updates hidden-type weights by Bayes' rule after each possible first-period purchase outcome and optimizes one terminal price per history. Consumers retain type; this older theory learns, unlike the current fixed-belief auxiliary exercise.

Definition and stated intent:

Source lines 41–42:

```
41 def terminal_policy(first):
42     # The firm observes h1, not gamma. Bayes' rule weights each possible type.
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 43–45:

```
43 inside = expit(initial_intercepts+alpha*first)
44 outside = expit(-initial_intercepts-alpha*first)
45 policies = []
```

Read this block:

- Line 43, `inside`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 44, `outside`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 45, `policies`: Set this name to the list `[]`.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 46–60:

```
46 for h1, likelihood in [("j", inside), ("0", outside)]:
47     joint = weights*likelihood
48     probability = float(joint.sum())
49     posterior = joint/probability
50     intercepts = beta+(-1 if h1 == "j" else 1)*GAMMAS
51 
52     def expected_terminal_profit(price):
53         shares = expit(intercepts+alpha*np.asarray(price)[..., None])
54         return (np.asarray(price)-cost)*(shares @ posterior)
55 
56     price, value = maximize(expected_terminal_profit, terminal=True)
57     share = expit(intercepts+alpha*price)
58     foc = float(posterior @ (share+alpha*(price-cost)*share*(1-share)))
59     policies.append(dict(h1=h1, p2=price, value=value,
60                          probability=probability, posterior=posterior, foc=foc))
```

Read this block:

- Line 46: for each entry yielded by `[('j', inside), ('0', outside)]`, bind/unpack it into `(h1, likelihood)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 48, `probability`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 49, `posterior`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 50, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 52: define nested callable `expected_terminal_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 56, `(price, value)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 57, `share`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 58, `foc`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 59: call `policies.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `maximize`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 61–61:

```
61 return policies
```

Read this block:

- Line 61: return the object currently named `policies`; plain assignment does not copy it. This ends the call.

solve.terminal policy.expected terminal profit

Original function: `solve.terminal_policy.expected_terminal_profit`. Source begins at line 52.

Computes terminal expected margin using the branch posterior over types. One price applies to all types in that observed history; optimizing separately by true gamma would give the firm information it lacks.

Definition and stated intent:

Source lines 52–52:

```
52 def expected_terminal_profit(price):
```

`def` creates this callable; the parentheses name its inputs (`price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 53–53:

```
53 shares = expit(intercepts+alpha*np.asarray(price)[..., None])
```

Read this block:

- Line 53, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 54–54:

```
54 return (np.asarray(price)-cost)*(shares @ posterior)
```

Read this block:

- Line 54: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

solve.expected_profit

Original function: `solve.expected_profit`. Source begins at line 63.

Combines initial expected margin and continuation values from the optimized history-specific policies. The first price affects both state probabilities and inferred type composition.

Definition and stated intent:

Source lines 63–64:

```
63 def expected_profit(first):
64     # Optimize the expectation over gamma, rather than prices for each gamma.
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 65–66:

```
65 purchase_probability = float(weights @ expit(initial_intercepts+alpha*first))
66 continuation = sum(p["probability"]*p["value"] for p in terminal_policy(first))
```

Read this block:

- Line 65, `purchase_probability`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 66, `continuation`: Set this name to the result of calling `sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `terminal_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 67–67:

```
67 return (first-cost)*purchase_probability+delta*continuation
```

Read this block:

- Line 67: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

figures

Original function: `figures`. Source begins at line 112.

Draws one theoretical page per prior showing hidden gamma weights, optimal history-dependent prices and expected profit. All values are generated from model parameters rather than purchase data.

Definition and stated intent:

Source lines 112–113:

```
112 def figures(paths, summary, output):
113     """One readable page per prior: distribution, policy, and expected profit."""
```

`def` creates this callable; the parentheses name its inputs (`paths`, `summary`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 114–119:

```
114 plt = plotting()
115 colors = ["#3366a6", "#78909c", "#d48435"]
116 titles = {"favor_negative": "Pr(gamma = -1.25) = 0.50",
117           "favor_zero": "Pr(gamma = 0) = 0.50",
118           "favor_positive": "Pr(gamma = +1.25) = 0.50",
119           "equal": "Equal probability on all three gamma values"}
```

Read this block:

- Line 114, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 115, `colors`: Set this name to the list `['#3366a6', '#78909c', '#d48435']`.
- Line 116, `titles`: Set this name to a dictionary with named entries `favor_negative`, `favor_zero`, `favor_positive`, `equal`.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 120–154:

```
120 for scenario, weights in PRIORS.items():
121     data = summary[summary.scenario == scenario].set_index("h0").loc[["0", "j"]]
122     with plt.rc_context({"font.size": 13, "axes.titlesize": 15, "axes.labelsize": 13}):
123         fig = plt.figure(figsize=(13, 7), layout="constrained")
124         grid = fig.add_gridspec(2, 2, width_ratios=[1, 1.7])
125         prior_ax = fig.add_subplot(grid[:, 0])
126         price_ax = fig.add_subplot(grid[0, 1])
127         profit_ax = fig.add_subplot(grid[1, 1])
128         bars = prior_ax.bar(["-1.25", "0", "+1.25"], weights, color="#8267a0", width=.6)
129         prior_ax.bar_label(bars, labels=[f"{p:.0%}" if scenario != "equal" else "1/3" for p in weights], padding=6)
130         prior_ax.set(xlabel="Possible gamma (unobserved)", ylabel="Firm's prior probability",
131                      title="Uncertainty faced by the firm", ylim=(0, .65))
132         x = np.arange(2)
133         width = .23
134         for offset, key, label, color in zip([-1, 0, 1],
135                 ["p1", "p2_after_inside", "p2_after_outside"],
```

```
136                 ["Period 1", "Period 2: bought in period 1", "Period 2: did not buy in period 1"], colors):
137             bars = price_ax.bar(x+offset*width, data[key], width, color=color, label=label)
138             price_ax.bar_label(bars, fmt="%.3f", padding=3, fontsize=11)
139         price_ax.set(title="Optimal prices under gamma uncertainty", ylabel="Price")
140         price_ax.set_ylim(min(0, float(data[["p1", "p2_after_inside", "p2_after_outside"]].min().min())*1.2),
141                           float(data[["p1", "p2_after_inside", "p2_after_outside"]].max().max())*1.6)
142         price_ax.legend(loc="upper left", bbox_to_anchor=(0, 1.01), fontsize=9, frameon=False)
143         bars = profit_ax.bar(x, data.optimal_profit, color="#467d5d", width=.5)
144         profit_ax.bar_label(bars, fmt="%.3f", padding=4)
145         profit_ax.set(title="Maximized expected two-period profit", ylabel="Discounted expected profit",
146                       ylim=(min(0, data.optimal_profit.min()*1.2), data.optimal_profit.max()*1.2))
147         for ax in [price_ax, profit_ax]:
148             ax.set_xticks(x, ["Known h0: outside", "Known h0: inside"])
149             ax.spines[["top", "right"]].set_visible(False)
150         prior_ax.spines[["top", "right"]].set_visible(False)
151         fig.suptitle(titles[scenario], fontsize=18)
152         for extension in ["png", "pdf"]:
153             fig.savefig(output / f"optimal_policy_{scenario}.{extension}", dpi=180)
154         plt.close(fig)
```

Read this block:

- Line 120: for each entry yielded by `PRIORS.items()`, bind/unpack it into `(scenario, weights)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 121, `data`: Set this name to the entry/slice selected by `summary[summary.scenario == scenario].set_index('h0').loc...`; brackets select from the object to their left.
- Line 122: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 123, `fig`: Set this name to the result of calling `plt.figure(...)`; the arguments and preceding method chain are shown above.
- Line 124, `grid`: Set this name to the result of calling `fig.add_gridspec(...)`; the arguments and preceding method chain are shown above.
- Line 125, `prior_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 126, `price_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 127, `profit_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 128, `bars`: Set this name to the result of calling `prior_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 129: call `prior_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 130: call `prior_ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 133, `width`: Set this name to the literal value `0.23`.
- Line 134: for each entry yielded by `zip([-1, 0, 1], ['p1', 'p2_after_inside', 'p2_after_outside'],...`, bind/unpack it into `(offset, key, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 137, `bars`: Set this name to the result of calling `price_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 138: call `price_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: call `price_ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `price_ax.set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 142: call `price_ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 143, `bars`: Set this name to the result of calling `profit_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 144: call `profit_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 145: call `profit_ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 147: for each entry yielded by `[price_ax, profit_ax]`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 148: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 149: call `ax.spines[['top', 'right']].set_visible` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 150: call `prior_ax.spines[['top', 'right']].set_visible` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 151: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 152: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 153: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 154: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 157.

Verifies degenerate-type nesting, common offers across hidden types, Bayesian belief consistency, probability mass, revenue reconstruction and first-order conditions. These checks distinguish this learning theory from fixed-belief pricing.

Definition and stated intent:

Source lines 157–157:

```
157 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 158–173:

```
158 for h0 in ["0", "j"]:
159     for i, gamma in enumerate(GAMMAS):
160         _, summary, _ = solve(h0, np.eye(3)[i])
161         reference = supply_solution(gamma, float(h0 == "j"))
162         actual = summary[0]
163         assert np.isclose(actual["optimal_profit"], reference["known_value"], atol=1e-7)
164         assert np.isclose(actual["p1"], reference["known_p1_inside" if h0 == "j" else "known_p1_outside"],
             ↪  atol=1e-6)
165     for weights in PRIORS.values():
166         rows, summary, _ = solve(h0, weights)
167         # All true types face exactly the same p1 and same p2 at a given h1.
168         assert np.ptp([r["p1"] for r in rows]) == 0
169         for h1 in ["0", "j"]:
170             assert np.ptp([r["p2"] for r in rows if r["h1"] == h1]) == 0
171         # Revealing gamma can only increase the optimized expected profit.
172         known = sum(w*supply_solution(g, float(h0 == "j"))["known_value"] for w, g in zip(weights, GAMMAS))
173         assert known >= summary[0]["optimal_profit"]-1e-7
```

Read this block:

- Line 158: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 159: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 160, `(_, summary, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 161, `reference`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 162, `actual`: Set this name to the entry/slice selected by `summary[0]`; brackets select from the object to their left.
- Line 163: require `np.isclose(actual['optimal_profit'], reference['known_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 164: require `np.isclose(actual['p1'], reference['known_p1_inside' if h0 == 'j' else 'known_p1_o...` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 165: for each entry yielded by `PRIORS.values()`, bind/unpack it into `weights` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 166, `(rows, summary, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 168: require `np.ptp([r['p1'] for r in rows]) == 0` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 169: for each entry yielded by `['0', 'j']`, bind/unpack it into `h1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 170: require `np.ptp([r['p2'] for r in rows if r['h1'] == h1]) == 0` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

- Line 172, `known`: Set this name to the result of calling `sum(...)`; the arguments and preceding method chain are shown above.
- Line 173: require `known >= summary[0]['optimal_profit'] - 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 174–174:

```
174 print("Passed: common policy across gamma, degenerate-type benchmark, Bayesian beliefs, probability mass, profit
    ↪  reconstruction and FOCs.")
```

Read this block:

- Line 174: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 177.

Solves the configured theoretical hidden-gamma scenarios and initial states, records results and draws the policy figures. The main empirical pipeline does not call this Bayesian-learning entry point.

Definition and stated intent:

Source lines 177–177:

```
177 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 178–183:

```
178 parser = argparse.ArgumentParser(description=__doc__)
179 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "outputs")
180 parser.add_argument("--beta", type=float, default=2.)
181 parser.add_argument("--alpha", type=float, default=-1.)
```

```
182 parser.add_argument("--delta", type=float, default=.92)
183 parser.add_argument("--cost", type=float, default=0.)
```

Read this block:

- Line 178, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 179: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 180: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 181: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 182: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 183: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 184–185:

```
184 parser.add_argument("--self-test", action="store_true")
185 args = parser.parse_args()
```

Read this block:

- Line 184: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 185, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 186–188:

```
186 if args.self_test:
187     self_test()
188     return
```

Read this block:

- Line 186: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 187: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 188: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 189–189:

```
189 paths, summaries, residuals = [], [], []
```

Read this block:

- Line 189, `(paths, summaries, residuals)`: Unpack a tuple containing 3 displayed entries into these targets in order.

Source lines 190–202:

```
190 for scenario, weights in PRIORS.items():
191     for h0 in ["0", "j"]:
192         rows, summary, residual = solve(h0, weights, args.beta, args.alpha, args.delta, args.cost)
193         # Independently repeat with a denser price search.
194         _, dense, _ = solve(h0, weights, args.beta, args.alpha, args.delta, args.cost, points=161)
195         if not np.allclose([r["optimal_profit"] for r in summary], [r["optimal_profit"] for r in dense], atol=1e-7,
             ↪  rtol=0):
196             raise AssertionError("Denser search disagrees.")
197         for row in rows+summary:
198             row["scenario"] = scenario
199         paths.extend(rows)
200         summaries.extend(summary)
201         residuals.append(residual)
202         print(f"Solved {scenario}, observed h0={h0}", flush=True)
```

Read this block:

- Line 190: for each entry yielded by `PRIORS.items()`, bind/unpack it into `(scenario, weights)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 191: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 192, `(rows, summary, residual)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 194, `(_, dense, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 195: test `not np.allclose([r['optimal_profit'] for r in summary], [r['optimal_profit'] ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 196: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 197: for each entry yielded by `rows + summary`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 198, `row['scenario']`: Store the object currently named `scenario`; plain assignment does not copy it in the selected entry/column.
- Line 199: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 200: call `summaries.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 201: call `residuals.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 202: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 203–208:

```
203 args.output.mkdir(parents=True, exist_ok=True)
204 histories, expected = pd.DataFrame(paths), pd.DataFrame(summaries)
205 histories.to_csv(args.output / "supply_history_outcomes.csv", index=False)
206 expected.to_csv(args.output / "supply_expected_outcomes.csv", index=False)
207 expected.to_csv(args.output / "optimal_policies.csv", index=False)
208 figures(histories, expected, args.output)
```

Read this block:

- Line 203: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 204, `(histories, expected)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 205: call `histories.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 206: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 207: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 208: call `figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 209–216:

```
209 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()},
210                 information="h0 observed, gamma unobserved and fixed; h1 observed before p2",
211                 gamma_values=GAMMAS.tolist(), priors={k: v.tolist() for k, v in PRIORS.items()},
212                 maximum_foc_residual=max(residuals), denser_search_verified=True,
213                 script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
214                 solver_sha256=hashlib.sha256((Path(__file__).resolve().parents[1] /
    ↪  "run_supply_exercises.py").read_bytes()).hexdigest())
215 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
216 print(f"Saved figures and tables to {args.output}")
```

Read this block:

- Line 209, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 215: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 216: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Current readable research workflow (final_simple)

../final_simple/data/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

Source lines 1–1:

```
"""Data research scripts."""
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

../final_simple/data/build_data.py

Expensive server-only raw conversion, cleaning, concatenation, panel/retail merges and yogurt matching. Current alongside jobs reuse its completed outputs rather than calling it again. Read it for provenance of the matched checkpoint, not as the next command for changing flavor groups.

Module setup and direct-execution blocks

Source lines 1–13:

```
"""Build the purchase data, from raw TSV files through matched yogurt purchases.

Edit Parameters, select the steps you want, then run python3 data/build_data.py.
Functions below follow the same order as the research workflow. Importing this
file does not read data, create output folders, or start any conversion.
"""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 9: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 11, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 12: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 13: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–20:

```
14 from pathlib import Path
15 from types import SimpleNamespace
16 import argparse
17 import os
18 import tempfile
19
20 import polars as pl
```

Read this block:

- Line 14: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 17: import `os`. An as-name is a convenient local alias, not another library.
- Line 18: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 20: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 26–31:

```
26 data_dir = Path('/scratch/dtm63837/Kilts_Panel/Data')
27 panel_raw = Path('/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/nielsen_extracts/HMS')
28 retail_raw = Path('/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/RMS')
29 first_year = 2006
30 last_year = 2015
31 convert_annual = True
```

Read this block:

- Line 26, `data_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 27, `panel_raw`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 28, `retail_raw`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 29, `first_year`: Set this name to the literal value `2006`.
- Line 30, `last_year`: Set this name to the literal value `2015`.

- Line 31, `convert_annual`: Set this name to the literal value `True`.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 32–32:

```
32 convert_master = True
```

Read this block:

- Line 32, `convert_master`: Set this name to the literal value `True`.

Source lines 36–36:

```
36 run_steps = []
```

Read this block:

- Line 36, `run_steps`: Set this name to the list `[]`.

Source lines 44–45:

```
44 retail_merged_input = None  # Set a Path here only if using an existing file elsewhere.
45 yogurt_module_codes = [3603, 3612]
```

Read this block:

- Line 44, `retail_merged_input`: Set this name to the literal value `None`.
- Line 45, `yogurt_module_codes`: Set this name to the list `[3603, 3612]`.

Source lines 49–58:

```
49 PANEL_COLUMNS = {
50     'panelists': [pl.col('household_cd').alias('household_code'), 'panel_year',
51                   'household_income', 'household_size', 'male_head_age', 'female_head_age',
52                   'male_head_employment', 'female_head_employment', 'race', 'hispanic_origin',
53                   'dma_cd', 'dma_name'],
54     'purchases': ['trip_code_uc', 'upc', 'quantity', 'total_price_paid', 'deal_flag_uc'],
55     'trips': ['trip_code_uc', 'household_code', 'purchase_date', 'store_code_uc', 'panel_year'],
56     'products': ['upc', 'product_module_code', 'product_module_descr', 'product_group_code',
57                  'product_group_descr', 'brand_code_uc', 'brand_descr', 'size1_amount', 'size1_units'],
58 }
```

Read this block:

- Line 49, `PANEL_COLUMNS`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`, `products`.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 59–73:

```
59 RETAIL_COLUMNS = {
60     '3603': ['store_code_uc', 'upc', 'week_end', 'price'],
61     '3612': ['store_code_uc', 'upc', 'week_end', 'price'],
62     'products': ['upc', 'upc_ver_uc', 'product_module_code', 'brand_code_uc', 'brand_descr',
63                  'size1_code_uc', 'size1_amount', 'size1_units'],
64     'products_extra': ['upc', 'upc_ver_uc', 'panel_year', 'flavor_code', 'flavor_descr'],
65     'stores': ['store_code_uc', 'year', 'retailer_code', 'channel_code', 'dma_code', 'dma_descr'],
66 }
67 RETAIL_TYPES = {'flavor_code': pl.String, 'size1_code_uc': pl.String,
68                 'upc': pl.Int64, 'store_code_uc': pl.Int64, 'panel_year': pl.Int32}
```

```
MATCH_KEYS = ['panel_year', 'week_end', 'upc', 'store_code_uc']
AVAILABLE_STEPS = [
    'convert_panel', 'clean_panel', 'combine_panel', 'merge_panel', 'panel_weeks',
    'convert_retail', 'clean_retail', 'combine_retail', 'merge_retail', 'match_yogurt',
]
```

Read this block:

- Line 59, `RETAIL_COLUMNS`: Set this name to a dictionary with named entries `3603`, `3612`, `products`, `products_extra`, `stores`.
- Line 67, `RETAIL_TYPES`: Set this name to a dictionary with named entries `flavor_code`, `size1_code_uc`, `upc`, `store_code_uc`, `panel_year`.
- Line 69, `MATCH_KEYS`: Set this name to the list `['panel_year', 'week_end', 'upc', 'store_code_uc']`.
- Line 70, `AVAILABLE_STEPS`: Set this name to a list containing 10 displayed entries.

Source lines 430–440:

```
if __name__ == '__main__':
    settings = SimpleNamespace(
        data_dir=data_dir, panel_raw=panel_raw, retail_raw=retail_raw,
        first_year=first_year, last_year=last_year, convert_annual=convert_annual,
        convert_master=convert_master, run_steps=run_steps,
        retail_merged_input=retail_merged_input, yogurt_module_codes=yogurt_module_codes)
    # Providing flags overrides only those values for this run.
    import sys
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_build(settings)
```

Read this block:

- Line 430: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 431, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 437: import `sys`. An as-name is a convenient local alias, not another library.
- Line 438: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 439, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 440: call `run_build` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_build`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

convert one file

Original function: `convert_one_file`. Source begins at line 80.

On the server, scans one tab-separated raw source, lowercases columns and writes a Parquet conversion. Missing paths are reported and skipped, so the caller must separately verify coverage. This expensive build stage is not invoked by the after-data job.

Definition and stated intent:

Source lines 80–85:

```
80 def convert_one_file(input_path, output_path, lossy_text=False):
81     """Read a tab-separated raw file, lowercase its column names, write Parquet.
82
83     A missing file is reported and skipped, matching the old conversion loops.
84     Read those messages to check coverage; skipping does not establish completeness.
85     """
```

`def` creates this callable; the parentheses name its inputs (`input_path`, `output_path`, `lossy_text`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–88:

```
86 if not input_path.exists():
87     print(f'Missing raw file; skipped: {input_path}', flush=True)
88     return
```

Read this block:

- Line 86: test `not input_path.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 89–94:

```
89 output_path.parent.mkdir(parents=True, exist_ok=True)
90 encoding = 'utf8-lossy' if lossy_text else 'utf8'
91 raw = pl.scan_csv(input_path, separator='\t', quote_char=None,
92                   infer_schema_length=10000, encoding=encoding)
93 raw.rename(str.lower).sink_parquet(output_path)
94 print(f'Converted {input_path.name} -> {output_path}', flush=True)
```

Read this block:

- Line 89: call `output_path.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `encoding`: Set this name to the true-branch value if `lossy_text` holds, otherwise the else-branch value.
- Line 91, `raw`: Set this name to the result of calling `pl.scan_csv(...)`; the arguments and preceding method chain are shown above.
- Line 93: call `raw.rename(str.lower).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.

convert panel files

Original function: `convert_panel_files`. Source begins at line 97.

Loops over requested panel years and annual table names, then optional master tables, composing each path explicitly. Actual I/O passes through convert_one_file and its server guard.

Definition and stated intent:

Source lines 97–97:

```
97 def convert_panel_files(raw_dir, output_dir, years, annual=True, master=True):
```

`def` creates this callable; the parentheses name its inputs (`raw_dir`, `output_dir`, `years`, `annual`, `master`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 98–102:

```
98 if annual:
99     for year in years:
100         for table in ['panelists', 'purchases', 'trips']:
101             input_path = raw_dir / str(year) / 'Annual_Files' / f'{table}_{year}.tsv'
102             convert_one_file(input_path, output_dir / f'{table}_{year}.parquet')
```

Read this block:

- Line 98: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 99: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 100: for each entry yielded by `['panelists', 'purchases', 'trips']`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 101, `input_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 102: call `convert_one_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `convert_one_file`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 103–106:

```
103 if master:
104     for table in ['products', 'retailers']:
105         input_path = raw_dir / 'Master_Files' / 'Latest' / f'{table}.tsv'
106         convert_one_file(input_path, output_dir / f'{table}.parquet', lossy_text=True)
```

Read this block:

- Line 103: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 104: for each entry yielded by `['products', 'retailers']`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 105, `input_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 106: call `convert_one_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

convert retail files

Original function: `convert_retail_files`. Source begins at line 109.

Converts annual metadata and movement modules 3603/3612, plus selected retail masters. The raw rms_versions filename maps to rms in output; the loop does not estimate demand or classify yogurt flavors.

Definition and stated intent:

Source lines 109–109:

```
109 def convert_retail_files(raw_dir, output_dir, years, annual=True, master=True):
```

`def` creates this callable; the parentheses name its inputs (`raw_dir`, `output_dir`, `years`, `annual`, `master`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 110–119:

```
110 if annual:
111     for year in years:
112         # RMS version files are named rms_versions in raw data, rms in outputs.
113         for raw_name, table in [('products_extra', 'products_extra'),
114                                 ('rms_versions', 'rms'), ('stores', 'stores')]:
115             input_path = raw_dir / str(year) / 'Annual_Files' / f'{raw_name}_{year}.tsv'
116             convert_one_file(input_path, output_dir / f'{table}_{year}.parquet')
117         for module in ['3603', '3612']:
118             input_path = raw_dir / str(year) / 'Movement_Files' / f'2510_{year}' / f'{module}_{year}.tsv'
119             convert_one_file(input_path, output_dir / f'{module}_{year}.parquet')
```

Read this block:

- Line 110: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 111: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 113: for each entry yielded by `[('products_extra', 'products_extra'), ('rms_versions', 'rms')...`, bind/unpack it into `(raw_name, table)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 115, `input_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 116: call `convert_one_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 117: for each entry yielded by `['3603', '3612']`, bind/unpack it into `module` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 118, `input_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 119: call `convert_one_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `convert_one_file`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 120–123:

```
120 if master:
121     for table in ['brand_variations', 'products']:
122         input_path = raw_dir / 'Master_Files_2006-2020' / 'Latest' / f'{table}.tsv'
123         convert_one_file(input_path, output_dir / f'{table}.parquet', lossy_text=True)
```

Read this block:

- Line 120: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 121: for each entry yielded by `['brand_variations', 'products']`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `input_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 123: call `convert_one_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

clean parquet files

Original function: `clean_parquet_files`. Source begins at line 127.

Keeps the specified columns in new cleaned_ files, preserving original conversions. Longer table names are matched first so products_extra is not confused with products. Column projection changes retained information, not just file size.

Definition and stated intent:

Source lines 127–128:

```
127 def clean_parquet_files(directory, columns_by_table):
128     """Write cleaned_ copies. Do not change or delete the original conversions."""
```

`def` creates this callable; the parentheses name its inputs (`directory`, `columns_by_table`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 130–130:

```
130 table_names = sorted(columns_by_table, key=len, reverse=True)
```

Read this block:

- Line 130, `table_names`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 131–140:

```
131 for input_path in sorted(directory.glob('*.parquet')):
132     if input_path.name.startswith('cleaned_'):
133         continue
134     for table in table_names:
135         if input_path.stem == table or input_path.stem.startswith(table + '_'):
136             output_path = directory / ('cleaned_' + input_path.name)
137             source = pl.scan_parquet(input_path)
138             source.select(columns_by_table[table]).sink_parquet(output_path)
```

```
139             print(f'Kept {table} columns: {output_path}', flush=True)
140             break
```

Read this block:

- Line 131: for each entry yielded by `sorted(directory.glob('*.parquet'))`, bind/unpack it into `input_path` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 132: test `input_path.name.startswith('cleaned_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 133: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 134: for each entry yielded by `table_names`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 135: test `input_path.stem == table or input_path.stem.startswith(table + '_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 136, `output_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 137, `source`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 138: call `source.select(columns_by_table[table]).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

combine panel files

Original function: `combine_panel_files`. Source begins at line 147.

Stacks cleaned annual panel tables into combined Parquet outputs. Wildcard inputs mean a stale converted year can enter this old build step; the after-data job avoids rerunning it and reuses the completed build.

Definition and stated intent:

Source lines 147–147:

```
147 def combine_panel_files(directory):
```

`def` creates this callable; the parentheses name its inputs (`directory`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 148–149:

```
148 combined = directory / 'combined'
149 combined.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 148, `combined`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 149: call `combined.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 150–156:

```
150 for table in ['panelists', 'purchases', 'trips']:
151     # This preserves the original wildcard rule: all matching years are stacked.
152     files = sorted(directory.glob(f'cleaned_{table}_*.parquet'))
153     if not files:
154         raise FileNotFoundError(f'No cleaned annual panel files for {table}: {directory}')
155     pl.scan_parquet(files).sink_parquet(combined / f'{table}.parquet')
156     print(f'Combined {len(files)} files for {table}', flush=True)
```

Read this block:

- Line 150: for each entry yielded by `['panelists', 'purchases', 'trips']`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 152, `files`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.
- Line 153: test `not files`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 154: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 155: call `pl.scan_parquet(files).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 156: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

align retail types

Original function: `align_retail_types`. Source begins at line 159.

Casts known retail identifiers and codes to compatible types before concatenation. Alignment avoids a schema mismatch across years; it does not reconcile inconsistent economic meanings of codes.

Definition and stated intent:

Source lines 159–160:

```
159 def align_retail_types(frame):
160     """Make identifiers/codes compatible across annual retail schemas."""
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 161–162:

```
161 names = frame.collect_schema().names()
162 casts = []
```

Read this block:

- Line 161, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 162, `casts`: Set this name to the list `[]`.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 163–165:

```
163 for column, dtype in RETAIL_TYPES.items():
164     if column in names:
165         casts.append(pl.col(column).cast(dtype))
```

Read this block:

- Line 163: for each entry yielded by `RETAIL_TYPES.items()`, bind/unpack it into `(column, dtype)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 164: test `column in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 165: call `casts.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 166–166:

```
166 return frame.with_columns(casts)
```

Read this block:

- Line 166: return the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

combine retail files

Original function: `combine_retail_files`. Source begins at line 170.

Concatenates compatible cleaned annual retail tables into combined outputs, applying retail type alignment. This is part of the expensive build rather than a flavor-specification transformation.

Definition and stated intent:

Source lines 170–170:

```
170 def combine_retail_files(directory):
```

`def` creates this callable; the parentheses name its inputs (`directory`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 171–176:

```
171 combined = directory / 'combined'
172 combined.mkdir(parents=True, exist_ok=True)
173 patterns = {'stores': 'cleaned_stores_*.parquet',
174             'products_extra': 'cleaned_products_extra_*.parquet',
175             '3612': 'cleaned_3612_*.parquet', '3603': 'cleaned_3603_*.parquet',
176             'rms': 'rms_*.parquet'}
```

Read this block:

- Line 171, `combined`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 172: call `combined.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 173, `patterns`: Set this name to a dictionary with named entries `stores`, `products_extra`, `3612`, `3603`, `rms`.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 177–186:

```
177 for table, pattern in patterns.items():
178     files = sorted(directory.glob(pattern))
179     if not files:
180         print(f'No annual files for {table}; skipped {directory / pattern}', flush=True)
181         continue
182     annual_frames = []
183     for path in files:
184         annual_frames.append(align_retail_types(pl.scan_parquet(path)))
185     pl.concat(annual_frames, how='vertical_relaxed').sink_parquet(combined / f'{table}.parquet')
186     print(f'Combined {len(files)} files for {table}', flush=True)
```

Read this block:

- Line 177: for each entry yielded by `patterns.items()`, bind/unpack it into `(table, pattern)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 178, `files`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.
- Line 179: test `not files`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 180: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 181: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 182, `annual_frames`: Set this name to the list `[]`.
- Line 183: for each entry yielded by `files`, bind/unpack it into `path` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 184: call `annual_frames.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 185: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `align_retail_types`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

join panel tables

Original function: `join_panel_tables`. Source begins at line 192.

Anchors on purchases, inner joins trip information, then attaches demographics and product attributes. Read each join key and join type: they determine which purchases survive and whether metadata can remain missing.

Definition and stated intent:

Source lines 192–197:

```
192 def join_panel_tables(purchases, trips, panelists, products):
193     """Pure table calculation: purchases are the anchor, trips use an inner join.
194 
195     Demographic and product joins are left joins. No new deduplication or sample
196     restriction is imposed here; the original join rules are retained.
197     """
```

`def` creates this callable; the parentheses name its inputs (`purchases`, `trips`, `panelists`, `products`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 198–204:

```
198 purchases = purchases.with_columns(pl.col('trip_code_uc').cast(pl.Int64), pl.col('upc').cast(pl.Int64))
199 trips = trips.with_columns(pl.col('trip_code_uc').cast(pl.Int64),
200                            pl.col('household_code').cast(pl.Int64), pl.col('panel_year').cast(pl.Int32))
201 panelists = panelists.with_columns(pl.col('household_code').cast(pl.Int64), pl.col('panel_year').cast(pl.Int32))
202 products = products.with_columns(pl.col('upc').cast(pl.Int64))
203 merged = purchases.join(trips, on='trip_code_uc', how='inner')
204 merged = merged.join(panelists, on=['household_code', 'panel_year'], how='left')
```

Read this block:

- Line 198, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.

- Line 199, `trips`: Set this name to the result of calling `trips.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 201, `panelists`: Set this name to the result of calling `panelists.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 202, `products`: Set this name to the result of calling `products.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 203, `merged`: Set this name to the result of calling `purchases.join(...)`; the arguments and preceding method chain are shown above.
- Line 204, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 205–205:

```
205 merged = merged.join(products, on='upc', how='left')
```

Read this block:

- Line 205, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.

Source lines 206–206:

```
206 return merged
```

Read this block:

- Line 206: return the object currently named `merged`; plain assignment does not copy it. This ends the call.

merge panel files

Original function: `merge_panel_files`. Source begins at line 210.

Reads the combined panel tables, calls join_panel_tables and writes the merged server checkpoint. The pure join helper separates table logic from empirical file I/O.

Definition and stated intent:

Source lines 210–210:

```
210 def merge_panel_files(directory):
```

`def` creates this callable; the parentheses name its inputs (`directory`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 211–216:

```
211 combined = directory / 'combined'
212 purchases = pl.scan_parquet(combined / 'purchases.parquet')
213 trips = pl.scan_parquet(combined / 'trips.parquet')
214 panelists = pl.scan_parquet(combined / 'panelists.parquet')
215 products = pl.scan_parquet(directory / 'cleaned_products.parquet')
216 merged = join_panel_tables(purchases, trips, panelists, products)
```

Read this block:

- Line 211, `combined`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 212, `purchases`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 213, `trips`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 214, `panelists`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 215, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 216, `merged`: Set this name to the result of calling `join_panel_tables(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `join_panel_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 217–219:

```
217 output_path = directory / 'merged_panel.parquet'
218 merged.sink_parquet(output_path)
219 audit_file(output_path, anchor=purchases, keys=['trip_code_uc', 'household_code', 'upc', 'panel_year'])
```

Read this block:

- Line 217, `output_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 218: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 219: call `audit_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `audit_file`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

add saturday week

Original function: `add_saturday_week`. Source begins at line 222.

Transforms purchase dates to the Saturday week-end used by retail matching, including Sunday and year-boundary cases. Calendar arithmetic defines matching eligibility; it is not a rolling seven-observation window.

Definition and stated intent:

Source lines 222–223:

```
222 def add_saturday_week(frame):
223     """Attach YYYYMMDD Saturday week-end, including Sunday and year boundaries."""
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 224–225:

```
224 date = pl.col('purchase_date').cast(pl.String).str.to_date()
225 saturday = date + pl.duration(days=(6 - date.dt.weekday()) % 7)
```

Read this block:

- Line 224, `date`: Set this name to the result of calling `.to_date(...)`; the arguments and preceding method chain are shown above.
- Line 225, `saturday`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 226–226:

```
226 return frame.with_columns(saturday.dt.strftime('%Y%m%d').cast(pl.Int64).alias('week_end'))
```

Read this block:

- Line 226: return the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.strftime`: Format a parsed date according to directives, e.g. %Y%m%d for an eight-digit week-ending key.

attach panel weeks

Original function: `attach_panel_weeks`. Source begins at line 230.

Reads the merged panel checkpoint, adds its Saturday week identifier, and publishes the updated checkpoint on the server. Current alongside runs reuse this already-completed result.

Definition and stated intent:

Source lines 230–231:

```
230 def attach_panel_weeks(directory):
231     # A temporary replacement avoids reading/writing the same Parquet simultaneously.
```

`def` creates this callable; the parentheses name its inputs (`directory`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 232–232:

```
232 panel_path = directory / 'merged_panel.parquet'
```

Read this block:

- Line 232, `panel_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 233–236:

```
233 with tempfile.TemporaryDirectory(prefix='panel-week-', dir=directory) as work:
234     replacement = Path(work) / 'merged_panel.parquet'
235     add_saturday_week(pl.scan_parquet(panel_path)).sink_parquet(replacement)
236     os.replace(replacement, panel_path)
```

Read this block:

- Line 233: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 234, `replacement`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 235: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 236: call `os.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `add_saturday_week`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 237–237:

```
237 print(f'Attached Saturday week-end: {panel_path}', flush=True)
```

Read this block:

- Line 237: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

join retail tables

Original function: `join_retail_tables`. Source begins at line 243.

Applies the original key casts, first-row deduplication and left joins between movement, versions, products, stores and metadata. A left join retains movement records even when an attribute is absent.

Definition and stated intent:

Source lines 243–244:

```
243 def join_retail_tables(movements, rms, products, stores, products_extra, brand_variations):
244     """Retain the original key casts, first-row rules, and left-join sequence."""
```

`def` creates this callable; the parentheses name its inputs (`movements`, `rms`, `products`, `stores`, `products_extra`, `brand_variations`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 245–252:

```
245 movements = movements.with_columns(
246     pl.col('upc').cast(pl.Int64), pl.col('store_code_uc').cast(pl.Int64),
247     (pl.col('week_end') // 10000).cast(pl.Int32).alias('panel_year'))
248 rms = rms.with_columns(pl.col('upc').cast(pl.Int64), pl.col('upc_ver_uc').cast(pl.Int32))
249 rms = rms.unique(subset=['upc'], keep='first')
250 products = products.with_columns(pl.col('upc').cast(pl.Int64), pl.col('upc_ver_uc').cast(pl.Int32))
251 products = products.unique(subset=['upc', 'upc_ver_uc'], keep='first')
252 stores = stores.rename({'year': 'panel_year'})
```

Read this block:

- Line 245, `movements`: Set this name to the result of calling `movements.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 248, `rms`: Set this name to the result of calling `rms.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 249, `rms`: Set this name to the result of calling `rms.unique(...)`; the arguments and preceding method chain are shown above.
- Line 250, `products`: Set this name to the result of calling `products.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 251, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.
- Line 252, `stores`: Set this name to the result of calling `stores.rename(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The // operator is floor division, used here for integer counts or calendar-year extraction.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.

Source lines 253–258:

```
253 stores = stores.with_columns(pl.col('store_code_uc').cast(pl.Int64), pl.col('panel_year').cast(pl.Int32))
254 stores = stores.unique(subset=['store_code_uc', 'panel_year'], keep='first')
255 products_extra = products_extra.with_columns(pl.col('upc').cast(pl.Int64), pl.col('panel_year').cast(pl.Int32))
256 products_extra = products_extra.sort('flavor_descr', descending=False, nulls_last=True)
257 products_extra = products_extra.unique(subset=['upc', 'panel_year'], keep='first')
258 brand_variations = brand_variations.with_columns(pl.col('brand_code_uc').cast(pl.Int64))
```

Read this block:

- Line 253, `stores`: Set this name to the result of calling `stores.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 254, `stores`: Set this name to the result of calling `stores.unique(...)`; the arguments and preceding method chain are shown above.
- Line 255, `products_extra`: Set this name to the result of calling `products_extra.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 256, `products_extra`: Set this name to the result of calling `products_extra.sort(...)`; the arguments and preceding method chain are shown above.
- Line 257, `products_extra`: Set this name to the result of calling `products_extra.unique(...)`; the arguments and preceding method chain are shown above.
- Line 258, `brand_variations`: Set this name to the result of calling `brand_variations.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.

Source lines 259–265:

```
259 brand_variations = brand_variations.unique(subset=['brand_code_uc'], keep='first')
260
261 merged = movements.join(rms, on='upc', how='left', suffix='_rms')
262 merged = merged.join(products, on=['upc', 'upc_ver_uc'], how='left', suffix='_prod')
263 merged = merged.join(stores, on=['store_code_uc', 'panel_year'], how='left', suffix='_str')
264 merged = merged.join(products_extra, on=['upc', 'panel_year'], how='left', suffix='_xtra')
265 merged = merged.join(brand_variations, on='brand_code_uc', how='left', suffix='_brand')
```

Read this block:

- Line 259, `brand_variations`: Set this name to the result of calling `brand_variations.unique(...)`; the arguments and preceding method chain are shown above.
- Line 261, `merged`: Set this name to the result of calling `movements.join(...)`; the arguments and preceding method chain are shown above.
- Line 262, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.
- Line 263, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.
- Line 264, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.
- Line 265, `merged`: Set this name to the result of calling `merged.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 266–266:

```
266 return merged
```

Read this block:

- Line 266: return the object currently named `merged`; plain assignment does not copy it. This ends the call.

merge retail files

Original function: `merge_retail_files`. Source begins at line 270.

Collects combined retail sources, calls the join helper and writes the merged retail checkpoint. Downstream yogurt matching reads that checkpoint; changing category definitions does not require rebuilding it.

Definition and stated intent:

Source lines 270–270:

```
270 def merge_retail_files(directory):
```

`def` creates this callable; the parentheses name its inputs (`directory`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 271–283:

```
271 combined = directory / 'combined'
272 movements = pl.concat([pl.scan_parquet(combined / '3612.parquet'),
273                       pl.scan_parquet(combined / '3603.parquet')])
274 merged = join_retail_tables(
275     movements, pl.scan_parquet(combined / 'rms.parquet'),
276     pl.scan_parquet(directory / 'cleaned_products.parquet'),
277     pl.scan_parquet(combined / 'stores.parquet'),
278     pl.scan_parquet(combined / 'products_extra.parquet'),
279     pl.scan_parquet(directory / 'brand_variations.parquet'))
```

```
280 output_path = directory / 'merged_rms.parquet'
281 merged.sink_parquet(output_path)
282 audit_file(output_path, anchor=movements, keys=['upc', 'store_code_uc', 'panel_year'],
283            metadata=['brand_descr', 'retailer_code', 'flavor_descr', 'brand_code_uc'])
```

Read this block:

- Line 271, `combined`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 272, `movements`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 274, `merged`: Set this name to the result of calling `join_retail_tables(...)`; the arguments and preceding method chain are shown above.
- Line 280, `output_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 281: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 282: call `audit_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `join_retail_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `audit_file`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

yogurt retail quotes

Original function: `yogurt_retail_quotes`. Source begins at line 289.

Builds one retained retail price quote for each year/week/UPC/store matching key. The year is part of the key; duplicates are not independent additional prices.

Definition and stated intent:

Source lines 289–290:

```
289 def yogurt_retail_quotes(retail, module_codes):
290     """One retained retail quote for each year/week/UPC/store matching key."""
```

`def` creates this callable; the parentheses name its inputs (`retail`, `module_codes`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 291–291:

```
291 needed = MATCH_KEYS + ['product_module_code', 'price']
```

Read this block:

- Line 291, `needed`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 292–293:

```
292 return (retail.select(needed).filter(pl.col('product_module_code').is_in(module_codes))
293         .unique(subset=MATCH_KEYS, keep='first'))
```

Read this block:

- Line 292: return the result of calling `.unique(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

match one year

Original function: `match_one_year`. Source begins at line 296.

Inner joins the selected year of yogurt purchases to retained retail quotes. Purchases without matching prices are excluded rather than imputed or converted to observed outside choices.

Definition and stated intent:

Source lines 296–297:

```
296 def match_one_year(panel, retail_quotes, year):
297     """Only purchases with a matching quote survive this inner join."""
```

`def` creates this callable; the parentheses name its inputs (`panel`, `retail_quotes`, `year`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 298–299:

```
298 panel_year = panel.filter(pl.col('panel_year') == year)
299 retail_year = retail_quotes.filter(pl.col('panel_year') == year)
```

Read this block:

- Line 298, `panel_year`: Set this name to the result of calling `panel.filter(...)`; the arguments and preceding method chain are shown above.
- Line 299, `retail_year`: Set this name to the result of calling `retail_quotes.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 300–300:

```
300 return panel_year.join(retail_year, on=MATCH_KEYS, how='inner', suffix='_rms')
```

Read this block:

- Line 300: return the result of calling `panel_year.join(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

match yogurt files

Original function: `match_yogurt_files`. Source begins at line 304.

Runs the annual yogurt matching steps using a private temporary directory and explicit year outputs, then writes the matched purchase checkpoint. This is the final expensive data-build stage skipped by run_after_data.

Definition and stated intent:

Source lines 304–305:

```
304 def match_yogurt_files(directory, retail_path, years, module_codes):
305     # Keep purchase-level estimation samples out of this code checkout.
```

`def` creates this callable; the parentheses name its inputs (`directory`, `retail_path`, `years`, `module_codes`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 306–306:

```
306 sample_destination = (Path(directory) / "yogurt_estimation_sample.parquet").resolve()
```

Read this block:

- Line 306, `sample_destination`: Set this name to the result of calling `.resolve(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 307–308:

```
307 if sample_destination.is_relative_to(Path(__file__).resolve().parents[3]):
308     raise ValueError("Estimation samples must stay outside the code checkout; use the server data directory.")
```

Read this block:

- Line 307: test `sample_destination.is_relative_to(Path(__file__).resolve().parents[3])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 308: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 313–316:

```
313 retail = pl.scan_parquet(retail_path)
314 retail_quotes = yogurt_retail_quotes(retail, module_codes).collect()
315 panel = pl.scan_parquet(directory / 'merged_panel.parquet')
316 output_path = directory / 'yogurt_estimation_sample.parquet'
```

Read this block:

- Line 313, `retail`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 314, `retail_quotes`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 315, `panel`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 316, `output_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `yogurt_retail_quotes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 317–324:

```
317 with tempfile.TemporaryDirectory(prefix='yogurt-years-', dir=directory) as work:
318     year_files = []
319     for year in years:
320         year_file = Path(work) / f'yogurt_{year}.parquet'
321         match_one_year(panel, retail_quotes.lazy(), year).sink_parquet(year_file)
322         year_files.append(year_file)
323         print(f'Matched purchase year {year}', flush=True)
324     pl.scan_parquet(year_files).sink_parquet(output_path)
```

Read this block:

- Line 317: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 318, `year_files`: Set this name to the list `[]`.
- Line 319: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 320, `year_file`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 321: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 322: call `year_files.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 323: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 324: call `pl.scan_parquet(year_files).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `match_one_year`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 325–325:

```
325 audit_file(output_path, keys=MATCH_KEYS, show_years=True)
```

Read this block:

- Line 325: call `audit_file` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `audit_file`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

audit file

Original function: `audit_file`. Source begins at line 332.

Reports aggregate schema, coverage, counts and key diagnostics for a server file. A row-count ratio alone is not proof of correct matching; the audit does not print individual purchase records.

Definition and stated intent:

Source lines 332–333:

```
332 def audit_file(path, anchor=None, keys=(), metadata=(), show_years=False):
333     """Report what the file contains; a row-count ratio alone is not a join proof."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `anchor`, `keys`, `metadata`, `show_years`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 334–337:

```
334 frame = pl.scan_parquet(path)
335 names = frame.collect_schema().names()
336 count = frame.select(pl.len()).collect().item()
337 print(f'\nOutput: {path}\nRows: {count:,}\nColumns: {names}', flush=True)
```

Read this block:

- Line 334, `frame`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 335, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 336, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 337: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 338–339:

```
338 if count == 0:
339     raise ValueError(f'Empty merged output; inspect inputs and matching keys: {path}')
```

Read this block:

- Line 338: test `count == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 339: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 340–343:

```
340 if anchor is not None:
341     original_count = anchor.select(pl.len()).collect().item()
342     ratio = 100 * count / original_count if original_count else float('nan')
343     print(f'Anchor rows: {original_count:,}; output/anchor: {ratio:.2f}%', flush=True)
```

Read this block:

- Line 340: test `anchor is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 341, `original_count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 342, `ratio`: Set this name to the true-branch value if `original_count` holds, otherwise the else-branch value.
- Line 343: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 344–347:

```
344 for column in keys:
345     if column in names:
346         missing = frame.select(pl.col(column).null_count()).collect().item()
347         print(f'Missing {column}: {missing:,} ({100*missing/count:.2f}%)', flush=True)
```

Read this block:

- Line 344: for each entry yielded by `keys`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 345: test `column in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 346, `missing`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 347: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 348–351:

```
348 for column in metadata:
349     if column in names:
350         covered = frame.select(pl.col(column).is_not_null().sum()).collect().item()
351         print(f'Nonmissing {column}: {covered:,} ({100*covered/count:.2f}%)', flush=True)
```

Read this block:

- Line 348: for each entry yielded by `metadata`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 349: test `column in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 350, `covered`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 351: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.

Source lines 352–355:

```
352 for column in ['household_code', 'hh_id', 'upc', 'store_code_uc']:
353     if column in names:
354         unique = frame.select(pl.col(column).n_unique()).collect().item()
355         print(f'Distinct {column}: {unique:,}', flush=True)
```

Read this block:

- Line 352: for each entry yielded by `['household_code', 'hh_id', 'upc', 'store_code_uc']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 353: test `column in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 354, `unique`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 355: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.

Source lines 356–357:

```
356 if show_years and 'panel_year' in names:
357     print(frame.group_by('panel_year').len().sort('panel_year').collect(), flush=True)
```

Read this block:

- Line 356: test `show_years and 'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 357: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 358–358:

```
358 price_column = next((c for c in ['price', 'price_rms'] if c in names), None)
```

Read this block:

- Line 358, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 359–365:

```
359 if price_column:
360     stats = frame.select(
361         pl.col(price_column).min().alias('minimum'), pl.col(price_column).mean().alias('mean'),
362         pl.col(price_column).median().alias('median'), pl.col(price_column).max().alias('maximum'),
363         pl.col(price_column).null_count().alias('missing'),
364         (pl.col(price_column) <= 0).sum().alias('zero_or_negative')).collect()
365     print(f'Price summary ({price_column}):\n{stats}', flush=True)
```

Read this block:

- Line 359: test `price_column`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 360, `stats`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 365: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.

Source lines 366–366:

```
366 print(f'File size: {path.stat().st_size / 1024**2:.2f} MB', flush=True)
```

Read this block:

- Line 366: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 372.

Parses optional raw-path, year and step overrides for the data builder. Parsing an argument does not authorize empirical I/O locally; server guards still apply.

Definition and stated intent:

Source lines 372–372:

```
372 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 373–378:

```
373 parser = argparse.ArgumentParser(description=__doc__)
374 parser.add_argument('--data-dir', type=Path)
375 parser.add_argument('--panel-raw', type=Path)
376 parser.add_argument('--retail-raw', type=Path)
```

```
377 parser.add_argument('--first-year', type=int)
378 parser.add_argument('--last-year', type=int)
```

Read this block:

- Line 373, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 374: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 375: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 376: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 377: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 378: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 379–383:

```
379 parser.add_argument('--steps', nargs='+', choices=AVAILABLE_STEPS, dest='run_steps')
380 parser.add_argument('--retail-merged-input', type=Path)
381 parser.add_argument('--skip-annual', dest='convert_annual', action='store_false')
382 parser.add_argument('--skip-master', dest='convert_master', action='store_false')
383 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 379: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 380: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 381: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 382: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 383: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 384–384:

```
384 return parser.parse_args()
```

Read this block:

- Line 384: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run build

Original function: `run_build`. Source begins at line 391.

Executes explicitly selected build steps in dependency order and audits outputs. The current after-data pipeline does not call this function; it requires completed merged panel and matched yogurt checkpoints instead.

Definition and stated intent:

Source lines 391–391:

```
391 def run_build(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 392–395:

```
392 if not settings.run_steps:
393     print('Choose run_steps in the Parameters section. Available steps:')
394     print(', '.join(AVAILABLE_STEPS))
395     return
```

Read this block:

- Line 392: test `not settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 393: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 394: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 395: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 396–397:

```
396 if settings.first_year > settings.last_year:
397     raise ValueError('first_year must not exceed last_year')
```

Read this block:

- Line 396: test `settings.first_year > settings.last_year`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 397: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 398–398:

```
398 unknown = set(settings.run_steps) - set(AVAILABLE_STEPS)
```

Read this block:

- Line 398, `unknown`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 399–400:

```
399 if unknown:
400     raise ValueError(f'Unknown build steps: {sorted(unknown)}')
```

Read this block:

- Line 399: test `unknown`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 400: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 401–405:

```
401 directory = settings.data_dir
402 retail_directory = directory / 'RMS'
403 years = range(settings.first_year, settings.last_year + 1)
404 directory.mkdir(parents=True, exist_ok=True)
405 retail_directory.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 401, `directory`: Set this name to the attribute `settings.data_dir` of the object before the dot.
- Line 402, `retail_directory`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 403, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 404: call `directory.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 405: call `retail_directory.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 407–408:

```
407 if 'convert_panel' in settings.run_steps:
408     convert_panel_files(settings.panel_raw, directory, years, settings.convert_annual, settings.convert_master)
```

Read this block:

- Line 407: test `'convert_panel' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 408: call `convert_panel_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `convert_panel_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 409–410:

```
409 if 'clean_panel' in settings.run_steps:
410     clean_parquet_files(directory, PANEL_COLUMNS)
```

Read this block:

- Line 409: test `'clean_panel' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 410: call `clean_parquet_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `clean_parquet_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 411–412:

```
411 if 'combine_panel' in settings.run_steps:
412     combine_panel_files(directory)
```

Read this block:

- Line 411: test `'combine_panel' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 412: call `combine_panel_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `combine_panel_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 413–414:

```
413 if 'merge_panel' in settings.run_steps:
414     merge_panel_files(directory)
```

Read this block:

- Line 413: test `'merge_panel' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 414: call `merge_panel_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `merge_panel_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 415–416:

```
415 if 'panel_weeks' in settings.run_steps:
416     attach_panel_weeks(directory)
```

Read this block:

- Line 415: test `'panel_weeks' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 416: call `attach_panel_weeks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `attach_panel_weeks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 417–418:

```
417 if 'convert_retail' in settings.run_steps:
418     convert_retail_files(settings.retail_raw, retail_directory, years, settings.convert_annual,
    ↪  settings.convert_master)
```

Read this block:

- Line 417: test `'convert_retail' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 418: call `convert_retail_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `convert_retail_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 419–420:

```
419 if 'clean_retail' in settings.run_steps:
420     clean_parquet_files(retail_directory, RETAIL_COLUMNS)
```

Read this block:

- Line 419: test `'clean_retail' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 420: call `clean_parquet_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 421–422:

```
421 if 'combine_retail' in settings.run_steps:
422     combine_retail_files(retail_directory)
```

Read this block:

- Line 421: test `'combine_retail' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 422: call `combine_retail_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `combine_retail_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 423–424:

```
423 if 'merge_retail' in settings.run_steps:
424     merge_retail_files(retail_directory)
```

Read this block:

- Line 423: test `'merge_retail' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 424: call `merge_retail_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `merge_retail_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 425–427:

```
425 if 'match_yogurt' in settings.run_steps:
426     retail_path = settings.retail_merged_input or retail_directory / 'merged_rms.parquet'
427     match_yogurt_files(directory, retail_path, years, settings.yogurt_module_codes)
```

Read this block:

- Line 425: test `'match_yogurt' in settings.run_steps`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 426, `retail_path`: Set this name to the Boolean condition `settings.retail_merged_input or retail_directory / 'merged_rms...'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 427: call `match_yogurt_files` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `match_yogurt_files`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

../final_simple/data/summaries.py

Server summary and prepared-input stage. A flavor_spec chooses grouped classifications after detailed summaries; the exact mapping is carried into demand. The two-spec job gives each run a separate output, preserving prior artifacts.

Module setup and direct-execution blocks

Source lines 1–13:

```
1  """Summaries, flavor recovery, and the prepared input for demand estimation.
2
3  Edit Parameters, then run python3 data/summaries.py. No data are read on import.
4  Detailed summaries are retained; grouped summaries select three or five categories.
5  The work is split into named research steps below rather than separate launchers.
6  """
7
8  # Empirical I/O is permitted only on the research server, never on this computer.
9  from pathlib import Path as _PolicyPath
10 import sys as _policy_sys
11 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
12 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
13 from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 9: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 11, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 12: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 13: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–19:

```
14 from pathlib import Path
15 from types import SimpleNamespace
16 import argparse
17 import os
18 import sys
19 import tempfile
```

Read this block:

- Line 14: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 17: import `os`. An as-name is a convenient local alias, not another library.
- Line 18: import `sys`. An as-name is a convenient local alias, not another library.
- Line 19: import `tempfile`. An as-name is a convenient local alias, not another library.

Source lines 20–27:

```
20 import time
21 from pprint import pformat
22
23 import numpy as np
24 import pandas as pd
25 import polars as pl
26
27 os.environ.setdefault('MPLCONFIGDIR', str(Path(tempfile.gettempdir()) / 'final_simple_matplotlib'))
```

Read this block:

- Line 20: import `time`. An as-name is a convenient local alias, not another library.
- Line 21: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 23: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 24: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 25: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 27: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 32–37:

```
32 data_dir = Path('/scratch/dtm63837/Kilts_Panel/Data')
33 flavors = data_dir / '../nielsen_extracts/nielsen_extracts/RMS/Reference_Documentation/2006-
   ↪  2020_Documentation/Latest_Flavor_2010.csv'
34 products_extra = None  # None uses data_dir/RMS/combined/products_extra.parquet.
35 output = None          # None uses the selected data_dir / 'summary_outputs'.
36 first_year = 2006
37 last_year = 2015
```

Read this block:

- Line 32, `data_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 33, `flavors`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 34, `products_extra`: Set this name to the literal value `None`.
- Line 35, `output`: Set this name to the literal value `None`.
- Line 36, `first_year`: Set this name to the literal value `2006`.
- Line 37, `last_year`: Set this name to the literal value `2015`.

Source lines 38–43:

```
household_batches = 16 # Physical household partitions to bound aggregation memory.
no_figures = False
hhi_only = False       # True redraws saved HHI tables without reading raw purchases.
flavor_spec = "three"  # "five" adds other_fruits and desserts; dessert cues win.
recover_only = False   # True only supplements the flavor reference; no summaries.
recovery_output = None # None uses output/recovered_flavor_reference.csv.
```

Read this block:

- Line 38, `household_batches`: Set this name to the literal value `16`.
- Line 39, `no_figures`: Set this name to the literal value `False`.
- Line 40, `hhi_only`: Set this name to the literal value `False`.
- Line 41, `flavor_spec`: Set this name to the literal text `'three'`.
- Line 42, `recover_only`: Set this name to the literal value `False`.
- Line 43, `recovery_output`: Set this name to the literal value `None`.

Source lines 46–53:

```
RACE_LABELS = {}
INCOME_LABELS = {}
HISPANIC_LABELS = {}
AGE_LABELS = {}

# Shared descriptor rules; the selected categories are recorded with each fit.
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from flavor_specs import categories, classification_expression
```

Read this block:

- Line 46, `RACE_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 47, `INCOME_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 48, `HISPANIC_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 49, `AGE_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 52: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: from `flavor_specs`, import `categories, classification_expression` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 55–57:

```
FLAVORS = ['other', 'berry', 'plain']
DATA_DIR = data_dir
FLAVOR_FILE = flavors
```

Read this block:

- Line 55, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 56, `DATA_DIR`: Set this name to the object currently named `data_dir`; plain assignment does not copy it.
- Line 57, `FLAVOR_FILE`: Set this name to the object currently named `flavors`; plain assignment does not copy it.

Source lines 837–845:

```
837 if __name__ == '__main__':
838     settings = SimpleNamespace(
839         data_dir=data_dir, flavors=flavors, products_extra=products_extra, output=output,
840         first_year=first_year, last_year=last_year, household_batches=household_batches,
841         no_figures=no_figures, hhi_only=hhi_only, recover_only=recover_only,
842         recovery_output=recovery_output, flavor_spec=flavor_spec)
843     if len(sys.argv) > 1:
844         settings = read_command_line(settings)
845     run_summaries(settings)
```

Read this block:

- Line 837: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 838, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 843: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 844, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 845: call `run_summaries` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_summaries`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 66.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

Definition and stated intent:

Source lines 66–67:

```
66 def normalize_upcs(values):
67     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 68–70:

```
68 text = values.astype("string").str.strip()
69 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
70 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 68, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 69, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 70, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 71–71:

```
71 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 71: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 73.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

Definition and stated intent:

Source lines 73–73:

```
73 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 74–76:

```
74 text = pl.col("upc").cast(pl.String).str.strip_chars()
75 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
76 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 74, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 75, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.

- Line 76, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 77–79:

```
77 return pl.when(numeric).then(
78     pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical)
79 ).otherwise(text).alias("upc")
```

Read this block:

- Line 77: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

valid flavor labels

Original function: `valid_flavor_labels`. Source begins at line 81.

Input is a pandas table with `flavor_descr`; output is a Boolean Series. Missing text becomes an empty string, is cast to string, stripped, and lowercased. `.isin` checks a list of unusable sentinel strings, then `~` negates the result. Thus `" N/A "` is invalid while `"Strawberry"` is valid. Normalizing once before comparison is clearer than writing separate conditions for every case and whitespace variant. This helper exists alongside directly written lazy filters; it is not automatically applied to every flavor-processing path.

Definition and stated intent:

Source lines 81–81:

```
81 def valid_flavor_labels(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 82–82:

```
82 labels = frame["flavor_descr"].fillna("").astype(str).str.strip().str.lower()
```

Read this block:

- Line 82, `labels`: Set this name to the result of calling `.lower(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 83–83:

```
83 return ~labels.isin(["", "na", "n/a", "nan", "null", "none"])
```

Read this block:

- Line 83: return the negated or sign-adjusted expression `~labels.isin(['', 'na', 'n/a', 'nan', 'null', 'none'])`. This ends the call.

Syntax and calls used here:

- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

scan purchases

Original function: `scan_purchases`. Source begins at line 88.

This builds and returns a lazy, projected purchase query. It validates required column labels and maps optional source labels to a common schema. `aliases.items()` supplies output name plus candidate source names. `next(..., None)` picks the first available candidate; otherwise `pl.lit(None, dtype=pl.String)` makes an explicitly typed missing column. A consistent schema simplifies later summaries.

The comprehension builds expressions without reading rows. Dates, UPCs, quantities, and prices get consistent representations. Head-age zero codes become null, then `pl.coalesce` picks the first nonmissing head-age category, preferring female then male. This is a priority rule, not a mean. Eagerly loading all columns first would use more memory; dropping rows because an optional demographic is absent would change the summary population unnecessarily.

Definition and stated intent:

Source lines 88–89:

```
88 def scan_purchases(path, first_year, last_year):
89     """Project needed columns before loading; accept the names in current code."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `first_year`, `last_year`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 90–100:

```
90 scan = pl.scan_parquet(path)
91 names = scan.collect_schema().names()
92 required = [
93     "household_code",
94     "panel_year",
95     "purchase_date",
96     "trip_code_uc",
97     "upc",
98     "quantity",
99 ]
100 missing = set(required) - set(names)
```

Read this block:

- Line 90, `scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

- Line 91, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 92, `required`: Set this name to a list containing 6 displayed entries.
- Line 100, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 101–102:

```
101 if missing:
102     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 101: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 102: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 103–112:

```
103 aliases = {
104     "race": ["race"],
105     "income": ["household_income", "income"],
106     "hispanic_origin": ["hispanic_origin", "hispanic"],
107     "household_size": ["household_size"],
108     "female_head_age": ["female_head_age"],
109     "male_head_age": ["male_head_age"],
110     "dma_cd": ["dma_cd"],
111 }
112 expressions = [pl.col(c) for c in required]
```

Read this block:

- Line 103, `aliases`: Set this name to a dictionary with named entries `race`, `income`, `hispanic_origin`, `household_size`, `female_head_age`, `male_head_age`, `dma_cd`.
- Line 112, `expressions`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 113–118:

```
113 for target, choices in aliases.items():
114     source = next((c for c in choices if c in names), None)
115     expression = (
116         pl.col(source).cast(pl.String) if source else pl.lit(None, dtype=pl.String)
117     )
```

```
118     expressions.append(expression.alias(target))
```

Read this block:

- Line 113: for each entry yielded by `aliases.items()`, bind/unpack it into `(target, choices)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 114, `source`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 115, `expression`: Set this name to the true-branch value if `source` holds, otherwise the else-branch value.
- Line 118: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 120–134:

```
120 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
121 expressions.append(
122     pl.col(price_column).cast(pl.Float64).alias("price")
123     if price_column
124     else pl.lit(None, dtype=pl.Float64).alias("price")
125 )
126 purchases = (
127     scan.select(expressions)
128     .filter(pl.col("panel_year").is_between(first_year, last_year))
129     .with_columns(
130         pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date(),
131         normalized_upc_expr(),
132         pl.col("quantity").cast(pl.Float64),
133     )
134 )
```

Read this block:

- Line 120, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 121: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126, `purchases`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 137–143:

```
137 age_columns = ["female_head_age", "male_head_age"]
138 purchases = purchases.with_columns(
139     [
140         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
141         for c in age_columns
142     ]
143 )
```

Read this block:

- Line 137, `age_columns`: Set this name to the list `['female_head_age', 'male_head_age']`.
- Line 138, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.

Source lines 144–144:

```
144 return purchases.with_columns(pl.coalesce(age_columns).alias("head_age"))
```

Read this block:

- Line 144: return the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

single person purchases

Original function: `single_person_purchases`. Source begins at line 147.

Filters purchase records to household size one before purchase weeks or histories are counted. This is a record-level sample restriction: filtering households after constructing their histories would retain excluded purchases in the stocks.

Definition and stated intent:

Source lines 147–148:

```
147 def single_person_purchases(scan):
148     """Match estimation: retain size-one purchase records before histories/weeks."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 149–150:

```
149 if "household_size" not in scan.collect_schema().names():
150     raise ValueError("Single-person summaries require household_size.")
```

Read this block:

- Line 149: test `'household_size' not in scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 150: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 151–151:

```
151 return scan.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 151: return the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

repeated yogurt households

Original function: `repeated_yogurt_households`. Source begins at line 153.

This keeps household histories with at least two distinct purchase weeks after the caller's year/price filters. `week` is a Polars expression using modulo arithmetic to reach Saturday. The query obtains each household's minimum and maximum week: unequal endpoints prove at least two distinct weeks without retaining a full distinct-week set.

Only eligible household IDs are collected. A semi join filters the original lazy purchase table while preserving its columns and all eligible purchase rows. Joining an expanded eligibility table could multiply rows; the semi join expresses membership only. Multiple trips in the same week do not satisfy this rule. Estimation later repeats a similar restriction after additional exclusions.

Definition and stated intent:

Source lines 153–158:

```
153 def repeated_yogurt_households(scan):
154     """Keep households with two distinct Saturday-ending purchase weeks.
155 
156     Apply after year selection; multiple lines/trips in one week count once.
157     Summaries precede the estimation-only menu and IV exclusions.
158     """
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 159–169:

```
159 week = pl.col("purchase_date") + pl.duration(
160     days=(6 - pl.col("purchase_date").dt.weekday()) % 7
161 )
162 ids = (
163     scan.group_by("household_code")
164     .agg(week.min().alias("first_week"), week.max().alias("last_week"))
165     .filter(pl.col("first_week") != pl.col("last_week"))
166     .select("household_code")
167 )
168 # Materialize only household IDs so the repeated scan cannot cache all purchases.
169 ids = ids.collect(engine="streaming")
```

Read this block:

- Line 159, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 162, `ids`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 169, `ids`: Set this name to the result of calling `ids.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

Source lines 170–170:

```
170 return scan.join(ids.lazy(), on="household_code", how="semi")
```

Read this block:

- Line 170: return the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

household summary

Original function: `household_summary`. Source begins at line 180.

Input is a lazy purchase table; output is a lazy one-row-per-household query. The `expressions` list defines exact purchase totals, distinct counts, and observed years. Demographic history follows a separate path: select a narrow table, deduplicate, stably sort, and take each field's first nonmissing value. Aggregating the full purchase table separately preserves exact trip/UPC/line counts while avoiding repeated wide demographic sorts.

`group_by(...).agg([...])` evaluates each listed expression per group. A validated one-to-one join reunites totals and demographic values. Later `with_columns` steps preserve original race, apply the Hispanic override, and calculate approximate weeks/trip rate. The second step can refer to the new `approx_observed_weeks` created in the first. Counting only observed years avoids filling panel gaps but still assumes full-year exposure within those years.

Definition and stated intent:

Source lines 180–181:

```
180 def household_summary(scan):
181     """One row per household, so frequent shoppers do not dominate demographics."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 182–191:

```
182 demographics = [
183     "race",
184     "income",
185     "hispanic_origin",
186     "household_size",
187     "head_age",
188     "female_head_age",
189     "male_head_age",
190     "dma_cd",
191 ]
```

Read this block:

- Line 182, `demographics`: Set this name to a list containing 8 displayed entries.

Source lines 193–198:

```
193 scan = scan.with_columns(
194     [
195         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
196         for c in ["female_head_age", "male_head_age"]
197     ]
198 )
```

Read this block:

- Line 193, `scan`: Set this name to the result of calling `scan.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 199–213:

```
199 expressions = [
200     pl.col("panel_year").n_unique().alias("observed_years"),
201     pl.col("panel_year").min().alias("first_year"),
202     pl.col("panel_year").max().alias("last_year"),
203     pl.col("trip_code_uc").n_unique().alias("trips"),
204     pl.col("purchase_date").n_unique().alias("shopping_days"),
205     pl.col("upc").n_unique().alias("distinct_upcs"),
206     pl.len().alias("purchase_lines"),
207     pl.col("quantity").sum().alias("packages"),
208 ]
209 # Keep exact purchase counts separate from the much narrower demographic
210 # history. Sort that history once for all fields, preserving input order
211 # for conflicting values on the same date. Removing identical rows cannot
```

```
# change the first nonmissing value.
totals = scan.group_by("household_code").agg(expressions)
```

Read this block:

- Line 199, `expressions`: Set this name to a list containing 8 displayed entries.
- Line 213, `totals`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 214–220:

```
history = (scan.select(["household_code", "panel_year", "purchase_date"] + demographics)
           .unique(maintain_order=True)
           .sort(["household_code", "panel_year", "purchase_date"], maintain_order=True))
values = history.group_by("household_code").agg([
    pl.col(c).drop_nulls().first().alias(c) for c in demographics
])
households = totals.join(values, on="household_code", how="left", validate="1:1")
```

Read this block:

- Line 214, `history`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 217, `values`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 220, `households`: Set this name to the result of calling `totals.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 221–228:

```
return households.with_columns(
    pl.col("race").alias("race_original"),
    pl.when(pl.col("hispanic_origin") == "1").then(pl.lit("Hispanic"))
    .otherwise(pl.col("race")).alias("race"),
    (pl.col("observed_years") * 365.25 / 7).alias("approx_observed_weeks"),
).with_columns(
    (pl.col("trips") / pl.col("approx_observed_weeks")).alias("trips_per_observed_week")
)
```

Read this block:

- Line 221: return the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

as lazy

Original function: `as_lazy`. Source begins at line 230.

This small adapter always returns a Polars LazyFrame. `isinstance` checks the runtime type. An existing LazyFrame is returned unchanged, a pandas table is converted through `pl.from_pandas(...).lazy()`, and the remaining expected Polars DataFrame receives `.lazy()`. This lets callers supply convenient small eager tables in tests while production uses lazy scans. It is not a universal converter: an arbitrary unsupported object without `.lazy()` will still fail. Avoiding a copy/conversion when already lazy preserves the existing query plan.

Definition and stated intent:

Source lines 230–230:

```
230 def as_lazy(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 231–232:

```
231 if isinstance(frame, pl.LazyFrame):
232     return frame
```

Read this block:

- Line 231: test `isinstance(frame, pl.LazyFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 232: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 233–234:

```
233 if isinstance(frame, pd.DataFrame):
234     return pl.from_pandas(frame).lazy()
```

Read this block:

- Line 233: test `isinstance(frame, pd.DataFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 234: return the result of calling `.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 235–235:

```
235 return frame.lazy()
```

Read this block:

- Line 235: return the result of calling `frame.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

partition purchases

Original function: `partition_purchases`. Source begins at line 240.

This executes a lazy scan once to write physical household-hash partitions, then returns a list of lazy scans over those files. `hasattr(pl, "PartitionBy")` checks API availability before starting. `(hash(seed=0) % batches)` maps every row with the same household ID to the same bucket. This is deterministic routing, not random sample selection.

`sink_parquet(pl.PartitionBy(...))` writes by bucket. The list comprehension includes only existing bucket directories, important because small examples can leave buckets empty. `sorted(directory.glob(...))` makes input order predictable. `hive_partitioning=False` prevents folder names from automatically adding a partition column that was intentionally excluded. The alternative of filtering/recollecting the original source for every bucket repeats large scans; physical partitions trade temporary disk space for reusable reads.

Definition and stated intent:

Source lines 240–241:

```
240 def partition_purchases(scan, work, prefix, batches):
241     """Read the projected source once, writing household hash partitions."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 242–243:

```
242 if not hasattr(pl, "PartitionBy"):
243     raise RuntimeError("Summary partitioning requires Polars >= 2.0; install final_simple/requirements.txt.")
```

Read this block:

- Line 242: test `not hasattr(pl, 'PartitionBy')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 243: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `hasattr`: Check that an object exposes a named attribute before using an optional/version-dependent API.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 244–250:

```
244 directory = work / f"{prefix}_purchases"
245 began = time.monotonic()
246 print(f"{prefix}: partitioning purchases once into {batches} batches", flush=True)
247 (scan.with_columns((pl.col("household_code").hash(seed=0) % batches).alias("_batch"))
248  .sink_parquet(pl.PartitionBy(directory, key="_batch", include_key=False), engine="streaming"))
249 parts = [pl.scan_parquet(sorted(directory.glob(f"_batch={batch}/*.parquet")), hive_partitioning=False)
250          for batch in range(batches) if (directory / f"_batch={batch}").exists()]
```

Read this block:

- Line 244, `directory`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 245, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 246: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 247: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 249, `parts`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 251–252:

```
251 if not parts:
252     raise ValueError(f"Empty {prefix} sample in selected years.")
```

Read this block:

- Line 251: test `not parts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 252: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 253–253:

```
253 print(f"{prefix}: partitioning completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 253: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 254–254:

```
254 return parts
```

Read this block:

- Line 254: return the object currently named `parts`; plain assignment does not copy it. This ends the call.

summarize household batches

Original function: `summarize_household_batches`. Source begins at line 257.

Computes household summaries batch by batch from server partitions, then combines the results. Batching bounds memory use while keeping each household history intact; it is not a sample-size restriction.

Definition and stated intent:

Source lines 257–258:

```
257 def summarize_household_batches(scan, work, prefix, batches, partitions=None):
258     # Each physical partition is read on its own; no full-source rescan per batch.
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`, `partitions`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 259–259:

```
259 partitions = partitions if partitions is not None else partition_purchases(scan, work, prefix, batches)
```

Read this block:

- Line 259, `partitions`: Set this name to the true-branch value if `partitions is not None` holds, otherwise the else-branch value.

Syntax and calls used here:

- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 260–264:

```
260 for batch, subset in enumerate(partitions):
261     began = time.monotonic()
262     print(f"{prefix}: household batch {batch + 1}/{len(partitions)}", flush=True)
263     household_summary(subset).sink_parquet(work / f"{prefix}_{batch}.parquet", engine="streaming")
264     print(f"{prefix}: household batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 260: for each entry yielded by `enumerate(partitions)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 261, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 262: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 263: call `household_summary(subset).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 264: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `household_summary`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 265–265:

```
265 return pl.scan_parquet(work / f"{prefix}_*.parquet")
```

Read this block:

- Line 265: return the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

save table

Original function: `save_table`. Source begins at line 268.

This writes both a machine-readable CSV and an escaped LaTeX table and prints a readable version. Polars inputs are collected and converted to pandas because the chosen table formatting/export methods belong to pandas. A pandas input already has those methods, so conversion is skipped.

`lambda value: f"{value:.2f}"` formats display numbers without changing the stored numerical data. `index=False` suppresses an unwanted row-number column. `escape=True` protects LaTeX syntax characters in labels. CSV retains reusable data whereas LaTeX supplies paper-ready formatting; either output alone would serve only one of those purposes. This function's useful result is its side effects, so no explicit return is required.

Definition and stated intent:

Source lines 268–269:

```
268 def save_table(frame, output, name, caption):
269     """Keep machine-readable CSV beside the paper-ready LaTeX table."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `output`, `name`, `caption`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 270–271:

```
270 if isinstance(frame, (pl.LazyFrame, pl.DataFrame)):
271     frame = as_lazy(frame).collect(engine="streaming").to_pandas()
```

Read this block:

- Line 270: test `isinstance(frame, (pl.LazyFrame, pl.DataFrame))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 271, `frame`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 272–277:

```
272 print(f"\n{caption} [{name}]", flush=True)
273 print(frame.to_string(index=False, float_format=lambda value: f"{value:.2f}"), flush=True)
274 frame.to_csv(output / f"{name}.csv", index=False)
275 (output / f"{name}.tex").write_text(
276     frame.to_latex(index=False, escape=True, float_format="%.2f", caption=caption)
277 )
```

Read this block:

- Line 272: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 273: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 274: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 275: call `(output / f'{name}.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_latex`: Serialize a pandas table as LaTeX source, which needs a separate TeX compilation to become PDF.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

demographic tables

Original function: `demographic_tables`. Source begins at line 281.

Input is a one-row-per-household table, an output directory, and filename prefix. The initial count is its denominator. The loop builds one categorical table per demographic using null-to-`Missing` replacement, grouped counts, optional codebook mappings, and percentages of all households. `.map(labels).fillna(code)` keeps source codes visible when a verified label is absent.

The `specs` list pairs human-readable metric names with Polars aggregation expressions. Enumerated string aliases let all metrics be computed in a single query and then zipped back to names. `rows.extend(generator)` appends race percentages to the overview. Equal household weighting is achieved by summarizing households first; directly counting purchase rows would overweight frequent shoppers. Numeric means of income/head-age codes remain means of categories, not economic levels.

Definition and stated intent:

Source lines 281–281:

```
281 def demographic_tables(households, output, prefix):
```

`def` creates this callable; the parentheses name its inputs (`households`, `output`, `prefix`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 282–284:

```
282 households = as_lazy(households)
283 count = households.select(pl.len()).collect(engine="streaming").item()
```

```
284 race_table = None
```

Read this block:

- Line 282, `households`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 283, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 284, `race_table`: Set this name to the literal value `None`.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 285–297:

```
285 for column, labels in [("race", RACE_LABELS), ("hispanic_origin", HISPANIC_LABELS),
286                        ("income", INCOME_LABELS), ("household_size", {}),
287                        ("head_age", AGE_LABELS), ("female_head_age", AGE_LABELS),
288                        ("male_head_age", AGE_LABELS)]:
289     table = (households.select(pl.col(column).cast(pl.String).fill_null("Missing").alias("code"))
290              .group_by("code").len(name="households").sort("code")
291              .collect(engine="streaming").to_pandas())
292     table["category"] = table["code"].map(labels).fillna(table["code"])
293     table["percent_all_households"] = 100 * table["households"] / count
294     save_table(table, output, f"{prefix}_{column}",
295                f"{prefix.replace('_', ' ').title()}: {column.replace('_', ' ')} (unweighted)")
296     if column == "race":
297         race_table = table
```

Read this block:

- Line 285: for each entry yielded by `[('race', RACE_LABELS), ('hispanic_origin', HISPANIC_LABELS), ...]`, bind/unpack it into `(column, labels)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 289, `table`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 292, `table['category']`: Store the result of calling `.fillna(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 293, `table['percent_all_households']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 294: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 296: test `column == 'race'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 297, `race_table`: Set this name to the object currently named `table`; plain assignment does not copy it.

Syntax and calls used here:

- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 298–312:

```
298 specs = [
299     ("Households", pl.len()),
300     ("Mean observed panel years", pl.col("observed_years").mean()),
301     ("Mean trips per household", pl.col("trips").mean()),
302     ("Mean trips per approximate observed week", pl.col("trips_per_observed_week").mean()),
303     ("Median trips per approximate observed week", pl.col("trips_per_observed_week").median()),
304     ("Mean distinct UPCs per household", pl.col("distinct_upcs").mean()),
305     ("Mean purchase lines per household", pl.col("purchase_lines").mean()),
306     ("Mean packages per household", pl.col("packages").mean()),
307     ("Mean head age (source category code)", pl.col("head_age").cast(pl.Float64, strict=False).mean()),
308     ("Households with numeric head age", pl.col("head_age").cast(pl.Float64, strict=False).count()),
309     ("Mean household income (source category code)", pl.col("income").cast(pl.Float64, strict=False).mean()),
310     ("Households with numeric income", pl.col("income").cast(pl.Float64, strict=False).count()),
311 ]
312 values = households.select([expr.alias(str(i)) for i, (_, expr) in
    ↪  enumerate(specs)]).collect(engine="streaming").row(0)
```

Read this block:

- Line 298, `specs`: Set this name to a list containing 12 displayed entries.
- Line 312, `values`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `str`: Convert to text for paths, messages, or serialization.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 313–317:

```
313 rows = [(name, value) for (name, _), value in zip(specs, values)]
314 rows.extend((f"Race: {row.category} (% of all households)", row.percent_all_households)
315             for row in race_table.itertuples())
316 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, f"{prefix}_overview",
317            f"{prefix.replace('_', ' ').title()}: Household purchase summary (unweighted)")
```

Read this block:

- Line 313, `rows`: Set this name to a list computed by the displayed for-expression.
- Line 314: call `rows.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 316: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

scan reference

Original function: `scan_reference`. Source begins at line 326.

Returns a lazy reference scan, selecting CSV or Parquet behavior from the suffix. String-preserving CSV settings prevent numeric guessing from corrupting UPC identifiers; neither branch collects a table until the caller executes the plan.

Definition and stated intent:

Source lines 326–327:

```
326 def scan_reference(path):
327     # Parquet references allow a complete synthetic check without any CSV reads.
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 328–329:

```
328 if Path(path).suffix.lower() == '.parquet':
329     return pl.scan_parquet(path)
```

Read this block:

- Line 328: test `Path(path).suffix.lower() == '.parquet'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 329: return the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 330–331:

```
330 return pl.scan_csv(path, infer_schema=False,
331                    null_values=["NA", "N/A", "NaN", "nan", "NULL", "null", "None", "<NA>"])
```

Read this block:

- Line 330: return the result of calling `pl.scan_csv(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

recover flavor reference

Original function: `recover_flavor_reference`. Source begins at line 336.

This supplements, rather than overwrites, the descriptor reference. Both input tables are lazy; an anti join excludes product UPCs already present in the CSV. The filter removes missing/placeholder candidate descriptors. If annual

metadata exists, each UPC's candidate columns take earliest-year values; otherwise first row deduplication is used. The output is a new combined reference plus a separate CSV of just recovered rows.

`.with_name(output.stem + ...)` constructs a sibling output filename. Comparing resolved paths rejects accidental overwrite of the original reference. After writing additions, `scan_reference` rereads them under the same schema conventions before diagonal relaxed concatenation allows unequal optional columns. The return is the output Path, enabling `run` to use it for subsequent classification. It does not repair blank descriptors on UPCs already in the original reference.

Definition and stated intent:

Source lines 336–336:

```
336 def recover_flavor_reference(reference_path, products_path, output):
```

`def` creates this callable; the parentheses name its inputs (`reference_path`, `products_path`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 337–337:

```
337 reference = scan_reference(reference_path).with_columns(normalized_upc_expr())
```

Read this block:

- Line 337, `reference`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 338–339:

```
338 if not {"upc", "flavor_descr"}.issubset(reference.collect_schema().names()):
339     raise ValueError("Reference needs upc and flavor_descr columns.")
```

Read this block:

- Line 338: test `not {'upc', 'flavor_descr'}.issubset(reference.collect_schema().names())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 339: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 340–341:

```
340 products = pl.scan_parquet(products_path)
341 names = products.collect_schema().names()
```

Read this block:

- Line 340, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 341, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 342–343:

```
342 if not {"upc", "flavor_descr"}.issubset(names):
343     raise ValueError(f"{products_path}: needs upc and flavor_descr columns.")
```

Read this block:

- Line 342: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 343: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 344–351:

```
344 columns = [c for c in ["upc", "flavor_descr", "panel_year", "product_descr"] if c in names]
345 products = (products.select(columns).with_columns(normalized_upc_expr())
346             .join(reference.select("upc").unique(), on="upc", how="anti"))
347 products = products.filter(
348     pl.col("upc").is_not_null() & pl.col("flavor_descr").is_not_null()
349     & ~pl.col("flavor_descr").cast(pl.String).str.strip_chars().str.to_lowercase()
350     .is_in(["", "na", "n/a", "nan", "null", "none"])
351 )
```

Read this block:

- Line 344, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 345, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 347, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 352–358:

```
352 if "panel_year" in names:
353     recovered = products.group_by("upc").agg([
354         pl.col(c).sort_by("panel_year", nulls_last=True).first().alias(c)
355         for c in columns if c != "upc"
356     ])
357 else:
358     recovered = products.unique(subset="upc", keep="first", maintain_order=True)
```

Read this block:

- Line 352: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 353, `recovered`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 358, `recovered`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 359–360:

```
359 output = Path(output)
360 output.parent.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 359, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 360: call `output.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 361–362:

```
361 if output.resolve() == Path(reference_path).resolve():
362     raise ValueError("Choose a separate output file to preserve the original reference.")
```

Read this block:

- Line 361: test `output.resolve() == Path(reference_path).resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 362: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 364–364:

```
364 recovered_path = output.with_name(output.stem + "_recovered_rows.csv")
```

Read this block:

- Line 364, `recovered_path`: Set this name to the result of calling `output.with_name(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.

Source lines 365–375:

```
365 with tempfile.TemporaryDirectory(prefix="reference-", dir=output.parent) as tmp:
366     checkpoint = Path(tmp) / "additions.parquet"
367     recovered.sink_parquet(checkpoint, engine="streaming")
368     additions = pl.scan_parquet(checkpoint)
369     additions.sink_csv(recovered_path, engine="streaming")
370     supplemented = pl.concat([reference, additions], how="diagonal_relaxed")
371     if output.suffix.lower() == '.parquet':
372         supplemented.sink_parquet(output, engine="streaming")
373     else:
374         supplemented.sink_csv(output, engine="streaming")
375     count = additions.select(pl.len()).collect(engine="streaming").item()
```

Read this block:

- Line 365: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 366, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 367: call `recovered.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 368, `additions`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 369: call `additions.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 370, `supplemented`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 371: test `output.suffix.lower() == '.parquet'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 372: call `supplemented.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 374: call `supplemented.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 375, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

Source lines 376–376:

```
376 print(f"Recovered labels for {count:,} additional UPCs. Existing entries retained. Saved {output}", flush=True)
```

Read this block:

- Line 376: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 377–377:

```
377 return output
```

Read this block:

- Line 377: return the object currently named `output`; plain assignment does not copy it. This ends the call.

flavor reference

Original function: `flavor_reference`. Source begins at line 381.

Normalizes UPCs and returns one resolved category per UPC with first-reference-row precedence. Detailed labels use normalized descriptors; grouped labels call the shared three/five classification expression, including dessert precedence in five. Conflicting assignments can be recorded server-side for review; they are not silently resolved semantically.

Definition and stated intent:

Source lines 381–382:

```
381 def flavor_reference(path, detailed=False, output=None, flavor_spec="three"):
382     """Use first reference row per normalized UPC; keep missing descriptors."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `detailed`, `output`, `flavor_spec`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 383–385:

```
383 frame = scan_reference(path)
384 names = frame.collect_schema().names()
385 missing = {"upc", "flavor_descr"} - set(names)
```

Read this block:

- Line 383, `frame`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 384, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 385, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 386–387:

```
386 if missing:
387     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 386: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 387: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 388–388:

```
388 frame = frame.with_columns(normalized_upc_expr())
```

Read this block:

- Line 388, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 389–390:

```
389 if frame.select((pl.col("upc").is_null() | (pl.col("upc") == "")).any()).collect(engine="streaming").item():
390     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 389: test `frame.select((pl.col('upc').is_null() | (pl.col('upc') == '')).any()).collect...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 390: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 391–391:

```
391 text = pl.col("flavor_descr").fill_null("")
```

Read this block:

- Line 391, `text`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.

Source lines 392–393:

```
392 if "product_descr" in names:
393     text = text + pl.lit(" ") + pl.col("product_descr").fill_null("")
```

Read this block:

- Line 392: test `'product_descr' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 393, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

Source lines 394–394:

```
394 text = text.str.to_lowercase()
```

Read this block:

- Line 394, `text`: Set this name to the result of calling `text.str.to_lowercase(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.

Source lines 395–399:

```
395 if detailed:
396     flavor = (pl.col("flavor_descr").fill_null("").str.to_lowercase().str.strip_chars()
397               .str.replace_all(r"\s+", " ").replace({"": "unknown", "unflavored": "plain"}))
398 else:
399     flavor = classification_expression(text, flavor_spec)
```

Read this block:

- Line 395: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 396, `flavor`: Set this name to the result of calling `.replace(...)`; the arguments and preceding method chain are shown above.
- Line 399, `flavor`: Set this name to the result of calling `classification_expression(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `classification_expression`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `.replace_all`: Replace every regex match within each string, such as collapsing repeated whitespace.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.

Source lines 400–402:

```
400 frame = frame.with_columns(flavor.alias("flavor"))
401 conflicting = frame.group_by("upc").agg(pl.col("flavor").n_unique().alias("n")).filter(pl.col("n") > 1)
402 count = conflicting.select(pl.len()).collect(engine="streaming").item()
```

Read this block:

- Line 400, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.

- Line 401, `conflicting`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 402, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.

Source lines 403–407:

```
403 if count:
404     print(f"{count:,} UPCs have conflicting flavor classifications; keeping the first CSV row.", flush=True)
405     if output is not None:
406         name = "detailed_flavor_conflicts.csv" if detailed else "aggregated_flavor_conflicts.csv"
407         frame.join(conflicting.select("upc"), on="upc", how="semi").sink_csv(output / name, engine="streaming")
```

Read this block:

- Line 403: test `count`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 404: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 405: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 406, `name`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 407: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 408–409:

```
408 blanks = frame.select((pl.col("flavor_descr").fill_null("").str.strip_chars() ==
    ↪  "").sum()).collect(engine="streaming").item()
409 print(f"Flavor-reference rows with blank descriptors: {blanks:,}", flush=True)
```

Read this block:

- Line 408, `blanks`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 409: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 410–410:

```
410 return frame.select("upc", "flavor").unique(subset="upc", keep="first",
    ↪ maintain_order=True).collect(engine="streaming")
```

Read this block:

- Line 410: return the result of calling `.collect(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

yogurt weeks

Original function: `yogurt_weeks`. Source begins at line 420.

This attaches flavor labels, audits matched purchase lines, and returns a lazy weekly quantity query plus an eager small audit table. A left join preserves unmatched purchases long enough to count/report them. Unmatched UPCs are exported and cause a ValueError, preventing an apparently successful summary of a silently reduced sample. Nonpositive/missing quantities also stop the run.

The Saturday expression adds a row-dependent duration to purchase date. Grouping by household/week/flavor and summing quantity keeps only observed combinations. The alternative dense pivot would create a column for every detailed flavor and potentially a great many empty cells. Price problems are audited here; the main matched-sample caller has already applied its positive-price filter.

Definition and stated intent:

Source lines 420–421:

```
420 def yogurt_weeks(scan, reference, flavors=FLAVORS, output=None):
421     """Keep sparse household/week/flavor counts rather than a dense pivot."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `reference`, `flavors`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 422–428:

```
422 joined = scan.join(reference.lazy(), on="upc", how="left")
423 audit = joined.select(
424     pl.len().alias("purchase_lines"),
425     pl.col("flavor").is_null().sum().alias("unmatched_flavor_lines"),
426     ((~pl.col("price").is_finite()) | (pl.col("price") <=
    ↪ 0)).fill_null(True).sum().alias("missing_or_nonpositive_quoted_prices"),
427     ((~pl.col("quantity").is_finite()) | (pl.col("quantity") <=
    ↪ 0)).fill_null(True).sum().alias("invalid_quantity_lines"),
428 ).collect(engine="streaming")
```

Read this block:

- Line 422, `joined`: Set this name to the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above.
- Line 423, `audit`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 429–434:

```
429 if audit["unmatched_flavor_lines"][0]:
430     unmatched = (joined.filter(pl.col("flavor").is_null()).group_by("upc")
431                  .agg(pl.len().alias("purchase_lines"), pl.col("quantity").sum().alias("packages")))
432     if output is not None:
433         unmatched.sink_csv(output / "unmatched_purchase_upcs.csv", engine="streaming")
434     raise ValueError("Purchased UPCs still lack flavor labels; see unmatched_purchase_upcs.csv.")
```

Read this block:

- Line 429: test `audit['unmatched_flavor_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 430, `unmatched`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 432: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 433: call `unmatched.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 434: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Source lines 435–436:

```
435 if audit["invalid_quantity_lines"][0]:
436     raise ValueError("Nonpositive or missing quantities; audit the matched sample first.")
```

Read this block:

- Line 435: test `audit['invalid_quantity_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 436: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 437–439:

```
437 week = pl.col("purchase_date") + pl.duration(days=(6 - pl.col("purchase_date").dt.weekday()) % 7)
438 grouped = (joined.with_columns(week.alias("week"))
439            .group_by(["household_code", "week", "flavor"]).agg(pl.col("quantity").sum()))
```

Read this block:

- Line 437, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 438, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

Source lines 440–440:

```
440 return grouped, audit
```

Read this block:

- Line 440: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

flavor summaries

Original function: `flavor_summaries`. Source begins at line 445.

This is the main descriptive-statistics engine. It returns pandas share and transition tables for plotting, while writing much richer outputs. First it sums quantities by flavor and computes package shares; a rounding filter removes rows whose displayed share is zero but leaves the denominator unchanged. Next it aggregates household/flavor totals to number of flavors, squared quantities, and largest quantity, producing HHI and largest-flavor share.

The basket query groups again by household/week. `pl.when(pl.len() == 1)` gives a single-flavor label only to unmixed baskets. `.shift().over("household_code")` looks at the previous basket within the household, not across household borders. Date differences identify adjacent weeks. A cumulative sum of spell-start flags creates run identifiers; mixed baskets and gaps break the runs.

The expensive basket query is checkpointed once in a temporary directory so transitions, spells, CSV exports, and global medians can reuse it. Transition counts pivot into a small matrix; dividing by row sums produces probabilities. Zero denominators become NaN. Spells are averaged as spells, not as households. Finally a small metric table records basket frequencies and eligible-pair switching. This sequence uses several aggregation units intentionally; replacing them all with one grand groupby would obscure their different denominators.

Definition and stated intent:

Source lines 445–445:

```
445 def flavor_summaries(weekly, output, flavors=FLAVORS):
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `output`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 446–447:

```
446 weekly = as_lazy(weekly)
447 totals = weekly.group_by("flavor").agg(pl.col("quantity").sum().alias("packages")).collect(engine="streaming")
```

Read this block:

- Line 446, `weekly`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 447, `totals`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 448–449:

```
448 if flavors != FLAVORS:
449     flavors = sorted(totals["flavor"].to_list())
```

Read this block:

- Line 448: test `flavors != FLAVORS`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 449, `flavors`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 451–459:

```
451 shares = (pl.DataFrame({"flavor": flavors}).join(totals, on="flavor", how="left")
452           .with_columns(pl.col("packages").fill_null(0))
453           .with_columns((100 * pl.col("packages") / pl.col("packages").sum()).alias("percent_packages")))
454 # Match the table's two-decimal display; keep the original denominator.
455 shares = shares.filter(pl.col("percent_packages").round(2) > 0).sort(
456     ["percent_packages", "flavor"], descending=[True, False]
457 )
458 save_table(shares, output, "yogurt_flavor_shares", "Flavor shares of purchased packages")
459 counts = weekly.group_by(["household_code", "flavor"]).agg(pl.col("quantity").sum())
```

Read this block:

- Line 451, `shares`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 455, `shares`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 458: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 459, `counts`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.

- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 460–473:

```
460 households = (counts.group_by("household_code").agg(
461     pl.len().alias("flavors_bought"), pl.col("quantity").sum().alias("packages"),
462     pl.col("quantity").pow(2).sum().alias("squared_packages"),
463     pl.col("quantity").max().alias("largest_packages"))
464     .with_columns((pl.col("squared_packages") / pl.col("packages").pow(2)).alias("flavor_HHI"),
465                   (pl.col("largest_packages") / pl.col("packages")).alias("largest_flavor_share"))
466     .select("household_code", "flavors_bought", "flavor_HHI", "largest_flavor_share"))
467 hh_path = output / "yogurt_household_flavors.csv"
468 # The same checkpoint supplies the export, HHI plots, and household means.
469 # It avoids repeatedly deriving household quantities or rereading a CSV.
470 household_checkpoint = output / "yogurt_household_flavors.parquet"
471 households.sink_parquet(household_checkpoint, engine="streaming")
472 households = pl.scan_parquet(household_checkpoint)
473 households.sink_csv(hh_path, engine="streaming")
```

Read this block:

- Line 460, `households`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 467, `hh_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 470, `household_checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 471: call `households.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 472, `households`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 473: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.pow`: Raise each expression value to a power, such as squaring quantities for concentration calculations.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.

Source lines 474–484:

```
474 hh_metrics = households.select(pl.len(), pl.col("flavors_bought").mean(),
    ↪  pl.col("flavor_HHI").mean()).collect(engine="streaming").row(0)
475 baskets = (weekly.group_by(["household_code", "week"]).agg(
476     pl.col("quantity").sum().alias("packages"), pl.len().alias("flavors_in_basket"),
477     pl.when(pl.len() == 1).then(pl.col("flavor").first()).otherwise(None).alias("single_flavor"))
478     .sort(["household_code", "week"])
479     .with_columns(pl.col("single_flavor").shift().over("household_code").alias("previous_flavor"),
480                   (pl.col("week").diff().over("household_code").dt.total_days() / 7).alias("gap_weeks"))
481     .with_columns((pl.col("single_flavor").is_not_null() & pl.col("previous_flavor").is_not_null()
482                    & (pl.col("gap_weeks") == 1)).fill_null(False).alias("eligible"))
483     .with_columns((~(pl.col("eligible") & (pl.col("single_flavor") == pl.col("previous_flavor")))).fill_null(False))
484                   .cast(pl.UInt64).cum_sum().alias("spell")))
```

Read this block:

- Line 474, `hh_metrics`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.
- Line 475, `baskets`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.cum_sum`: Accumulate running totals, used to turn spell-start flags into spell IDs.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.over`: Evaluate a Polars expression within the specified group/window, preserving row-level results.
- `.total_days`: Convert a duration expression into its elapsed number of days before comparing weekly gaps.
- `.shift`: Move values by one row by default; over(household) prevents the shift crossing household boundaries.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 486–519:

```
with tempfile.TemporaryDirectory(prefix="baskets-", dir=output) as tmp:
    checkpoint = Path(tmp) / "baskets.parquet"
    baskets.sink_parquet(checkpoint, engine="streaming")
    baskets = pl.scan_parquet(checkpoint)
    baskets.sink_csv(output / "yogurt_weekly_baskets.csv", engine="streaming")
    weekly.sink_csv(output / "yogurt_weekly_flavor_quantities.csv", engine="streaming")
    transitions = (baskets.filter(pl.col("eligible")).group_by(["previous_flavor", "single_flavor"])
                   .len().collect(engine="streaming").to_pandas())
    if transitions.empty:
        matrix = pd.DataFrame(0, index=flavors, columns=flavors)
    else:
        matrix = transitions.pivot(index="previous_flavor", columns="single_flavor",
↪   values="len").fillna(0).reindex(index=flavors, columns=flavors, fill_value=0)
    matrix.index.name = "previous_flavor"
    matrix.columns.name = "single_flavor"
    matrix.to_csv(output / "yogurt_transition_counts.csv")
    probabilities = matrix.div(matrix.sum(axis=1).replace(0, np.nan), axis=0)
    probabilities.to_csv(output / "yogurt_transition_probabilities.csv")
    print("\nYogurt transition counts", flush=True)
    print(matrix.to_string(), flush=True)
    print("\nYogurt transition probabilities", flush=True)
    print(probabilities.to_string(float_format=lambda v: f"{v:.2f}"), flush=True)
    spells = (baskets.filter(pl.col("single_flavor").is_not_null()).group_by(["household_code", "spell"])
              .agg(pl.col("single_flavor").first().alias("flavor"), pl.len().alias("weeks")))
    spells_path = Path(tmp) / "spells.parquet"
    spells.sink_parquet(spells_path, engine="streaming")
    spells = pl.scan_parquet(spells_path)
    spells.sink_csv(output / "yogurt_single_flavor_spells.csv", engine="streaming")
    spell_mean = spells.select(pl.col("weeks").mean()).collect(engine="streaming").item()
    bm = baskets.select(
        pl.len().alias("weeks"), (100 * (pl.col("flavors_in_basket") > 1).mean()).alias("mixed"),
        (100 * (pl.col("packages") > 7).mean()).alias("above_seven"), pl.col("eligible").sum().alias("pairs"),
        (100 * (pl.col("single_flavor") !=
↪   pl.col("previous_flavor")).filter(pl.col("eligible")).mean()).alias("switch_rate"),
```

```
518         pl.col("gap_weeks").median().alias("gap"),
519     ).collect(engine="streaming").row(0, named=True)
```

Read this block:

- Line 486: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 487, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 488: call `baskets.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 489, `baskets`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 490: call `baskets.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 491: call `weekly.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 492, `transitions`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 494: test `transitions.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 495, `matrix`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 497, `matrix`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 498, `matrix.index.name`: Set this name to the literal text `'previous_flavor'`.
- Line 499, `matrix.columns.name`: Set this name to the literal text `'single_flavor'`.
- Line 500: call `matrix.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 501, `probabilities`: Set this name to the result of calling `matrix.div(...)`; the arguments and preceding method chain are shown above.
- Line 502: call `probabilities.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 503: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 504: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 505: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 506: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 507, `spells`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 509, `spells_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 510: call `spells.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 511, `spells`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 512: call `spells.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 513, `spell_mean`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 514, `bm`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.div`: Divide a pandas object with explicit alignment options; axis=0 aligns the denominator by rows.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 520–527:

```
520 rows = [
521     ("Matched households", hh_metrics[0]), ("Observed yogurt purchase weeks", bm["weeks"]),
522     ("Mean flavors bought per household", hh_metrics[1]), ("Mean household flavor HHI (0 to 1)", hh_metrics[2]),
523     ("Mixed-flavor weeks (%)", bm["mixed"]), ("Weeks above seven packages (%)", bm["above_seven"]),
524     ("Eligible adjacent single-flavor week pairs", bm["pairs"]), ("Switch rate in eligible pairs (%)",
    ↪  bm["switch_rate"]),
525     ("Mean single-flavor spell length (consecutive calendar weeks)", spell_mean), ("Median gap between purchase
    ↪  weeks", bm["gap"]),
526 ]
527 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, "yogurt_dynamics", "Matched yogurt sample:
    ↪  purchase dynamics")
```

Read this block:

- Line 520, `rows`: Set this name to a list containing 10 displayed entries.
- Line 527: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 528–528:

```
528 return shares.to_pandas(), probabilities
```

Read this block:

- Line 528: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

make hhi figures

Original function: `make_hhi_figures`. Source begins at line 537.

Reads the saved household HHI checkpoint on the server and draws household-weighted histogram/CDF figures. It prefers Parquet and supports the older CSV fallback. Only aggregate figures, not the household checkpoint, belong in local exports.

Definition and stated intent:

Source lines 537–538:

```
537 def make_hhi_figures(output):
538     """Equal-household distributions using all flavor labels, without top-20 truncation."""
```

`def` creates this callable; the parentheses name its inputs (`output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 539–543:

```
539 import matplotlib
540 matplotlib.use("Agg")
541 import matplotlib.pyplot as plt
542
543 checkpoint = output / "yogurt_household_flavors.parquet"
```

Read this block:

- Line 539: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 540: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 541: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 543, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 544–548:

```
544 if checkpoint.exists():
545     values =
    ↪  pl.scan_parquet(checkpoint).select("flavor_HHI").collect().to_pandas()["flavor_HHI"].dropna().to_numpy()
546 else:
547     # Backward compatibility for an older output folder containing CSV only.
548     values = pd.read_csv(output / "yogurt_household_flavors.csv",
    ↪  usecols=["flavor_HHI"])["flavor_HHI"].dropna().to_numpy()
```

Read this block:

- Line 544: test `checkpoint.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 545, `values`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 548, `values`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 549–550:

```
549 if not len(values):
550     return
```

Read this block:

- Line 549: test `not len(values)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 550: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 551–563:

```
551 for kind in ["histogram", "cdf"]:
552     fig, ax = plt.subplots(figsize=(7, 5))
553     if kind == "histogram":
554         ax.hist(values, bins=np.linspace(0, 1, 41), weights=np.full(len(values), 100 / len(values)),
    ↪  color="steelblue")
555         ax.set_ylabel("Households (%)")
556     else:
557         ax.step(np.r_[0., np.sort(values)], np.r_[0., 100 * np.arange(1, len(values) + 1) / len(values)],
    ↪  where="post")
558         ax.set_ylabel("Cumulative households (%)")
559     ax.set(xlim=(0, 1), xlabel="Household flavor HHI (quantity shares)", title="Yogurt flavor concentration")
560     fig.tight_layout()
561     fig.savefig(output / f"flavor_hhi_{kind}.pdf")
562     plt.close(fig)
563     print(f"Saved HHI figure: {output / f'flavor_hhi_{kind}.pdf'}", flush=True)
```

Read this block:

- Line 551: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 552, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 553: test `kind == 'histogram'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 554: call `ax.hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 555: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 557: call `ax.step` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 558: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 559: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 560: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 561: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 562: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 563: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.step`: Draw an empirical step curve; where='post' extends each value to the right.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

make figures

Original function: `make_figures`. Source begins at line 565.

Inputs are already computed flavor-share and transition tables. Sorting then `head(20)` selects a readable subset for plots, and `reindex` aligns matrix rows and columns with the displayed flavor order. It does not recompute transition denominators within that subset, so a plotted row need not sum to one.

`fig, ax = plt.subplots` separates a Figure container from its plotting Axes. `ax.bar` draws shares; `ax.imshow` colors transition cells. Two nested index loops place formatted text inside each cell. Tick rotation improves long-label readability. `tight_layout`, `savefig`, and `close` finish each output. Plotting inside the data aggregation itself would make it harder to skip figures or recreate them from existing summary tables.

Definition and stated intent:

Source lines 565–566:

```
565 def make_figures(shares, transitions, output):
566     # Limit figure dimensions; all flavors remain in the saved tables.
```

`def` creates this callable; the parentheses name its inputs (`shares`, `transitions`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 567–574:

```
567 shares = shares.sort_values("packages", ascending=False).head(20)
568 flavors = shares["flavor"].tolist()
569 transitions = transitions.reindex(index=flavors, columns=flavors)
570 # Import plotting only if requested (use --no-figures for tables alone).
571 import matplotlib
572
573 matplotlib.use("Agg")
574 import matplotlib.pyplot as plt
```

Read this block:

- Line 567, `shares`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 568, `flavors`: Set this name to the result of calling `shares['flavor'].tolist(...)`; the arguments and preceding method chain are shown above.
- Line 569, `transitions`: Set this name to the result of calling `transitions.reindex(...)`; the arguments and preceding method chain are shown above.

- Line 571: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 573: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 574: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 576–581:

```
576 fig, ax = plt.subplots(figsize=(max(6, len(flavors) * 0.35), 5))
577 ax.bar(shares["flavor"], shares["percent_packages"], color="steelblue")
578 ax.tick_params(axis="x", labelrotation=90)
579 ax.set(ylabel="Share of purchased packages (%)", title="Yogurt flavor shares (up to 20 most purchased flavors)")
580 fig.tight_layout()
581 fig.savefig(output / "flavor_shares.pdf")
```

Read this block:

- Line 576, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 577: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 578: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 579: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 580: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 581: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.tick_params`: Adjust axis tick appearance, for example rotating long category labels.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 582–595:

```
582 plt.close(fig)
583 side = max(6, len(flavors) * 0.4)
584 fig, ax = plt.subplots(figsize=(side, side))
585 image = ax.imshow(transitions.to_numpy(), vmin=0, vmax=1, cmap="Blues")
586 ax.set(
587     xticks=range(len(flavors)),
588     yticks=range(len(flavors)),
589     xticklabels=flavors,
590     yticklabels=flavors,
591     xlabel="Current flavor",
```

```
592     ylabel="Previous flavor",
593     title="Adjacent single-flavor purchase weeks",
594 )
595 ax.tick_params(axis="x", labelrotation=90)
```

Read this block:

- Line 582: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 583, `side`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 584, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 585, `image`: Set this name to the result of calling `ax.imshow(...)`; the arguments and preceding method chain are shown above.
- Line 586: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 595: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.imshow`: Display a numerical matrix as colored cells; row/column label alignment remains the caller's responsibility.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 596–605:

```
596 for i in range(len(flavors)):
597     for j in range(len(flavors)):
598         value = transitions.iloc[i, j]
599         ax.text(
600             j,
601             i,
602             f"{value:.2f}" if pd.notna(value) else "N/A",
603             ha="center",
604             va="center",
605         )
```

Read this block:

- Line 596: for each entry yielded by `range(len(flavors))`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 597: for each entry yielded by `range(len(flavors))`, bind/unpack it into `j` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 598, `value`: Set this name to the entry/slice selected by `transitions.iloc[i, j]`; brackets select from the object to their left.
- Line 599: call `ax.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 606–609:

```
606 fig.colorbar(image, ax=ax)
607 fig.tight_layout()
608 fig.savefig(output / "flavor_transitions.pdf")
609 plt.close(fig)
```

Read this block:

- Line 606: call `fig.colorbar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 607: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 608: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 609: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.colorbar`: Add a legend relating matrix colors to numerical values.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

prepare summary samples

Original function: `prepare_summary_samples`. Source begins at line 620.

Creates full-panel and matched-yogurt summary samples, applies single-person restrictions, and writes aggregate demographics and year coverage. The prepared estimation handoff retains additional households for retail-price coverage, while demand later restricts its consumers.

Definition and stated intent:

Source lines 620–620:

```
620 def prepare_summary_samples(settings, output, work):
```

`def` creates this callable; the parentheses name its inputs (`settings`, `output`, `work`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 621–623:

```
621 prepared_eligible = None
622 print("Summary sample: single-person purchase records (household_size == 1).", flush=True)
623 source_partitions = {}
```

Read this block:

- Line 621, `prepared_eligible`: Set this name to the literal value `None`.
- Line 622: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 623, `source_partitions`: Set this name to a dictionary from the displayed keys and values.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 624–660:

```
624 for prefix, filename in [("full_panel", "merged_panel.parquet"), ("matched_yogurt",
    ↪ "yogurt_estimation_sample.parquet")]:
625     source = scan_purchases(settings.data_dir / filename, settings.first_year, settings.last_year)
```

```
626     if "household_size" not in pl.scan_parquet(settings.data_dir / filename).collect_schema().names():
627         raise ValueError(f"{filename}: single-person summaries require household_size.")
628     if prefix == "matched_yogurt":
629         # Preserve all-household retail coverage in the prepared estimation
630         # input. demand/estimation.py restricts demand records while retaining these quotes.
631         covered_all = source.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
632         prepared_eligible = (repeated_yogurt_households(covered_all)
633                              .select("household_code").unique().collect(engine="streaming"))
634     source = single_person_purchases(source)
635     if prefix == "matched_yogurt":
636         covered = (pl.col("price").is_finite() & (pl.col("price") > 0)).fill_null(False)
637         audit = source.select(pl.len().alias("purchase_lines_before_price_filter"),
638                               (~covered).sum().alias("purchase_lines_without_price_coverage"))
639         save_table(audit, output, "yogurt_price_coverage", "Yogurt matched-price coverage")
640         source = source.filter(covered)
641         source = repeated_yogurt_households(source)
642     parts = partition_purchases(source, work, prefix, settings.household_batches)
643     source_partitions[prefix] = parts
644     source = pl.concat(parts)
645     households = summarize_household_batches(source, work, prefix, settings.household_batches, partitions=parts)
646     if households.select(pl.len()).collect(engine="streaming").item() == 0:
647         raise ValueError(f"Empty {prefix} sample in selected years.")
648     demographic_tables(households, output, prefix)
649     households.sink_csv(output / f"{prefix}_households.csv", engine="streaming")
650     # Exact distinct-trip counts for one year at a time bound aggregation state.
651     years = source.select("panel_year").unique().sort("panel_year").collect(engine="streaming")["panel_year"]
652     coverage_rows = []
653     for year in years:
654         coverage_rows.append(source.filter(pl.col("panel_year") == year).select(
655             pl.lit(year).alias("panel_year"),
656             pl.col("household_code").n_unique().alias("households"),
657             pl.col("trip_code_uc").n_unique().alias("trips"), pl.len().alias("purchase_lines"),
658             pl.col("quantity").sum().alias("packages")).collect(engine="streaming"))
659     coverage = pl.concat(coverage_rows)
660     save_table(coverage, output, f"{prefix}_year_coverage", "Sample coverage by year")
```

Read this block:

- Line 624: for each entry yielded by `[('full_panel', 'merged_panel.parquet'), ('matched_yogurt', 'y...`, bind/unpack it into `(prefix, filename)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 625, `source`: Set this name to the result of calling `scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 626: test `'household_size' not in pl.scan_parquet(settings.data_dir / filename).collect...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 627: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 628: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 631, `covered_all`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 632, `prepared_eligible`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 634, `source`: Set this name to the result of calling `single_person_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 635: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 636, `covered`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.

- Line 637, `audit`: Set this name to the result of calling `source.select(...)`; the arguments and preceding method chain are shown above.
- Line 639: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 640, `source`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 641, `source`: Set this name to the result of calling `repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 642, `parts`: Set this name to the result of calling `partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 643, `source_partitions[prefix]`: Store the object currently named `parts`; plain assignment does not copy it in the selected entry/column.
- Line 644, `source`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 645, `households`: Set this name to the result of calling `summarize_household_batches(...)`; the arguments and preceding method chain are shown above.
- Line 646: test `households.select(pl.len()).collect(engine='streaming').item() == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 647: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 648: call `demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 649: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 651, `years`: Set this name to the entry/slice selected by `source.select('panel_year').unique().sort('panel_year').c...`; brackets select from the object to their left.
- Line 652, `coverage_rows`: Set this name to the list `[]`.
- Line 653: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 654: call `coverage_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659, `coverage`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 660: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `scan_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `single_person_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `summarize_household_batches`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demographic_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct

representations.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `repeated_yogurt_households`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 661–661:

```
661 return prepared_eligible, source_partitions
```

Read this block:

- Line 661: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

summarize flavor groups

Original function: `summarize_flavor_groups`. Source begins at line 671.

Runs detailed-label summaries and then summaries using the selected three/five grouping. The same resolved grouped mapping is returned for estimation, preventing summaries and demand from silently using different classification rules.

Definition and stated intent:

Source lines 671–671:

```
671 def summarize_flavor_groups(settings, output, work, flavor_path, source_partitions):
```

`def` creates this callable; the parentheses name its inputs (`settings`, `output`, `work`, `flavor_path`, `source_partitions`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 672–697:

```
672 for detailed in [True, False]:
673     folder = output if detailed else output / "aggregated_flavors"
674     folder.mkdir(parents=True, exist_ok=True)
675     reference = flavor_reference(flavor_path, detailed=detailed, output=output,
676                                  flavor_spec=getattr(settings, "flavor_spec", "three"))
677     flavors = sorted(reference["flavor"].unique().to_list()) if detailed else FLAVORS
678     if detailed:
679         reference.write_csv(output / "detailed_flavor_reference.csv")
680     # Process household batches independently, keeping weekly baskets long.
681     audit_total = None
682     tag = "detailed" if detailed else "grouped"
683     parts = source_partitions["matched_yogurt"]
```

```
684     for batch, subset in enumerate(parts):
685         began = time.monotonic()
686         print(f"{tag}: yogurt week batch {batch + 1}/{len(parts)}", flush=True)
687         weekly, audit = yogurt_weeks(subset, reference, flavors, output=folder)
688         weekly.sink_parquet(work / f"{tag}_weeks_{batch}.parquet", engine="streaming")
689         print(f"{tag}: yogurt week batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
690         audit_total = audit if audit_total is None else pl.concat([audit_total, audit]).select(pl.all().sum())
691     if detailed:
692         save_table(audit_total, output, "yogurt_sample_audit", "Yogurt sample audit")
693     weekly = pl.scan_parquet(work / f"{tag}_weeks_*.parquet")
694     shares, transitions = flavor_summaries(weekly, folder, flavors)
695     if not settings.no_figures:
696         make_hhi_figures(folder)
697         make_figures(shares, transitions, folder)
```

Read this block:

- Line 672: for each entry yielded by `[True, False]`, bind/unpack it into `detailed` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 673, `folder`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 674: call `folder.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 675, `reference`: Set this name to the result of calling `flavor_reference(...)`; the arguments and preceding method chain are shown above.
- Line 677, `flavors`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 678: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 679: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 681, `audit_total`: Set this name to the literal value `None`.
- Line 682, `tag`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 683, `parts`: Set this name to the entry/slice selected by `source_partitions['matched_yogurt']`; brackets select from the object to their left.
- Line 684: for each entry yielded by `enumerate(parts)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 685, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 686: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 687, `(weekly, audit)`: Unpack the result of calling `yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 688: call `weekly.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 689: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 690, `audit_total`: Set this name to the true-branch value if `audit_total is None` holds, otherwise the else-branch value.
- Line 691: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 692: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 693, `weekly`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 694, `(shares, transitions)`: Unpack the result of calling `flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 695: test `not settings.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 696: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 697: call `make_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `flavor_summaries`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.write_csv`: Write an eager Polars table to CSV; it does not return a newly analyzed dataset.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `yogurt_weeks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `len`: Count elements/rows rather than summing their numerical values.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 698–698:

```
698 return reference
```

Read this block:

- Line 698: return the object currently named `reference`; plain assignment does not copy it. This ends the call.

save estimation sample

Original function: `save_estimation_sample`. Source begins at line 710.

Writes the specification-specific labeled purchase input and exact UPC mapping on the research server, outside the checkout. The expanded input uses the same completed matched data with new labels; none of these purchase-level files belongs in RESULTS_ONLY.

Definition and stated intent:

Source lines 710–711:

```
710 def save_estimation_sample(settings, output, prepared_households, reference):
711     # Keep purchase-level estimation samples out of this code checkout.
```

`def` creates this callable; the parentheses name its inputs (`settings`, `output`, `prepared_households`, `reference`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 712–712:

```
712 sample_destination = (Path(output) / "yogurt_estimation_sample.parquet").resolve()
```

Read this block:

- Line 712, `sample_destination`: Set this name to the result of calling `.resolve(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 713–714:

```
713 if sample_destination.is_relative_to(Path(__file__).resolve().parents[3]):
714     raise ValueError("Estimation samples must stay outside the code checkout; use the server data directory.")
```

Read this block:

- Line 713: test `sample_destination.is_relative_to(Path(__file__).resolve().parents[3])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 714: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 718–725:

```
718 reference.write_csv(output / "estimation_flavor_reference.csv")
719 print(f"Saved estimation flavor mapping to {output / 'estimation_flavor_reference.csv'}", flush=True)
720 # Retain every original purchase column needed downstream, including store
721 # and DMA fields that demographic projections do not otherwise need.
722 eligible = prepared_households
723 prepared = pl.scan_parquet(settings.data_dir / "yogurt_estimation_sample.parquet")
724 original_names = prepared.collect_schema().names()
725 price_column = next((c for c in ["price", "price_rms"] if c in original_names), None)
```

Read this block:

- Line 718: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 719: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 722, `eligible`: Set this name to the object currently named `prepared_households`; plain assignment does not copy it.
- Line 723, `prepared`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 724, `original_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 725, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.write_csv`: Write an eager Polars table to CSV; it does not return a newly analyzed dataset.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 726–727:

```
726 if price_column is None:
727     raise ValueError("Yogurt purchase table needs price or price_rms for matched-price coverage.")
```

Read this block:

- Line 726: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 727: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 728–729:

```
728 prepared = prepared.filter(pl.col(price_column).cast(pl.Float64).is_finite()
729                            & (pl.col(price_column).cast(pl.Float64) > 0))
```

Read this block:

- Line 728, `prepared`: Set this name to the result of calling `prepared.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 730–731:

```
730 if "flavor" in original_names:
731     prepared = prepared.drop("flavor")
```

Read this block:

- Line 730: test `'flavor' in original_names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 731, `prepared`: Set this name to the result of calling `prepared.drop(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Source lines 732–737:

```
prepared = (prepared.filter(pl.col("panel_year").is_between(settings.first_year, settings.last_year))
            .with_columns(normalized_upc_expr(),
                          pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
            .join(eligible.lazy(), on="household_code", how="semi")
            .join(reference.lazy(), on="upc", how="left"))
sample_path = output / "yogurt_estimation_sample.parquet"
```

Read this block:

- Line 732, `prepared`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 737, `sample_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 738–739:

```
if sample_path.resolve() == (settings.data_dir / "yogurt_estimation_sample.parquet").resolve():
    raise ValueError("Choose an output folder separate from the original data directory.")
```

Read this block:

- Line 738: test `sample_path.resolve() == (settings.data_dir / 'yogurt_estimation_sample.parqu...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 739: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 740–742:

```
prepared.sink_parquet(sample_path, engine="streaming")
print(f"Saved full prepared yogurt purchase table to {sample_path}", flush=True)
print(f"Saved summaries to {output}", flush=True)
```

Read this block:

- Line 740: call `prepared.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 741: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 742: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run

Original function: `run`. Source begins at line 751.

This orchestrates the summary workflow and writes final artifacts. `sources` and `source_partitions` are dictionaries keyed by sample prefix, keeping full panel and matched yogurt objects separate. It optionally supplements flavor metadata, then prepares each population, writes reusable hash partitions, computes household/demographic summaries, and audits annual coverage.

The second loop runs detailed then grouped flavor taxonomies. Partition-level weekly outputs remain sparse, while small audit tables are summed. The last reference therefore holds the grouped mapping, which is saved for estimation. The final export intentionally reopens the original purchase file, rather than the demographic projection, to preserve store/DMA and other fields. A semi join keeps eligible households and a left join attaches the resolved flavor.

`prepared.drop("flavor")` avoids two competing label columns when reprocessing an already labeled input. The resolved-path check prevents writing over the original source. This orchestration keeps the population restrictions explicit; running the estimator on an arbitrary intermediate table can produce another sample even if its filename looks similar.

Definition and stated intent:

Source lines 751–752:

```
751 def run(settings, output, work):
752     """The three main research steps; no files are imported from final_draft."""
```

`def` creates this callable; the parentheses name its inputs (`settings`, `output`, `work`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 754–754:

```
754 sample_destination = (Path(output) / "yogurt_estimation_sample.parquet").resolve()
```

Read this block:

- Line 754, `sample_destination`: Set this name to the result of calling `.resolve(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 755–756:

```
755 if sample_destination.is_relative_to(Path(__file__).resolve().parents[3]):
756     raise ValueError("Estimation samples must stay outside the code checkout; use the server data directory.")
```

Read this block:

- Line 755: test `sample_destination.is_relative_to(Path(__file__).resolve().parents[3])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 756: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 758–759:

```
758 products_extra = settings.products_extra or settings.data_dir / "RMS/combined/products_extra.parquet"
759 flavor_path = settings.flavors
```

Read this block:

- Line 758, `products_extra`: Set this name to the Boolean condition `settings.products_extra or settings.data_dir / 'RMS/combined/p...`; it can be a scalar or a row/array mask depending on its inputs.

- Line 759, `flavor_path`: Set this name to the attribute `settings.flavors` of the object before the dot.

Source lines 760–768:

```
760 if products_extra.exists():
761     suffix = ".parquet" if Path(flavor_path).suffix.lower() == ".parquet" else ".csv"
762     recovered_path = output / ("recovered_flavor_reference" + suffix)
763     if recovered_path.resolve() == flavor_path.resolve():
764         recovered_path = output / ("supplemented_flavor_reference" + suffix)
765     recover_flavor_reference(flavor_path, products_extra, recovered_path)
766     flavor_path = recovered_path
767 else:
768     print(f"Flavor recovery source not found: {products_extra}; using the supplied reference.", flush=True)
```

Read this block:

- Line 760: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 761, `suffix`: Set this name to the true-branch value if `Path(flavor_path).suffix.lower() == '.parquet'` holds, otherwise the else-branch value.
- Line 762, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 763: test `recovered_path.resolve() == flavor_path.resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 764, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 765: call `recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 766, `flavor_path`: Set this name to the object currently named `recovered_path`; plain assignment does not copy it.
- Line 768: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `recover_flavor_reference`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 769–773:

```
769 prepared_households, source_partitions = prepare_summary_samples(settings, output, work)
770 reference = summarize_flavor_groups(settings, output, work, flavor_path, source_partitions)
771 save_estimation_sample(settings, output, prepared_households, reference)
772 notes = "SUMMARY RUN SETTINGS\n\n" + pformat(vars(settings), sort_dicts=False)
773 (output / "run_notes.txt").write_text(notes + "\n")
```

Read this block:

- Line 769, `(prepared_households, source_partitions)`: Unpack the result of calling `prepare_summary_samples(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 770, `reference`: Set this name to the result of calling `summarize_flavor_groups(...)`; the arguments and preceding method chain are shown above.

- Line 771: call `save_estimation_sample` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 772, `notes`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 773: call `(output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `prepare_summary_samples`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `summarize_flavor_groups`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `save_estimation_sample`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 782.

Parses summary paths, years, partition count and flavor_spec, together with recovery/figure-only switches. Three remains the baseline; five adds the new categories. The full descriptor reference is needed to construct the expanded mapping.

Definition and stated intent:

Source lines 782–782:

```
782 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 783–788:

```
783 parser = argparse.ArgumentParser(description=__doc__)
784 parser.add_argument("--flavor-spec", choices=["three", "five"], default=flavor_spec)
785 parser.add_argument("--data-dir", type=Path, default=DATA_DIR)
786 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
787 parser.add_argument("--products-extra", "--products", type=Path)
788 parser.add_argument("--output", type=Path)
```

Read this block:

- Line 783, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 784: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 785: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 786: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 787: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 788: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 789–796:

```
789 parser.add_argument("--first-year", type=int, default=2006)
790 parser.add_argument("--last-year", type=int, default=2015)
791 parser.add_argument("--household-batches", type=int, default=16,
792                     help="Household disk partitions; more batches reduce aggregation memory (default 16).")
793 parser.add_argument("--no-figures", action="store_true")
794 parser.add_argument("--hhi-only", action="store_true",
795                     help="Plot saved household HHI (Parquet preferred; legacy CSV supported); skip raw data and all
                     ↪  summaries.")
796 parser.add_argument("--recover-only", action="store_true")
```

Read this block:

- Line 789: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 790: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 791: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 793: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 794: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 796: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 797–798:

```
797 parser.add_argument("--recovery-output", type=Path)
798 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 797: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 798: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 799–799:

```
799 return parser.parse_args()
```

Read this block:

- Line 799: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run summaries

Original function: `run_summaries`. Source begins at line 809.

Configures the shared ordered category list, selects recovery-only, figure-only or full summaries, and uses a temporary work directory. In-place list replacement keeps default arguments aligned with the chosen categories. All empirical work stays behind the server guard.

Definition and stated intent:

Source lines 809–809:

```
809 def run_summaries(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 810–810:

```
810 FLAVORS[:] = categories(getattr(settings, "flavor_spec", "three"))
```

Read this block:

- Line 810, `FLAVORS[:]`: Store the result of calling `categories(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.

Source lines 811–816:

```
811 if settings.recover_only:
812     output = settings.output or settings.data_dir / "summary_outputs"
813     products = settings.products_extra or settings.data_dir / "RMS/combined/products_extra.parquet"
814     destination = settings.recovery_output or output / "recovered_flavor_reference.csv"
815     recover_flavor_reference(settings.flavors, products, destination)
816     return
```

Read this block:

- Line 811: test `settings.recover_only`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 812, `output`: Set this name to the Boolean condition `settings.output or settings.data_dir / 'summary_outputs'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 813, `products`: Set this name to the Boolean condition `settings.products_extra or settings.data_dir / 'RMS/combined/p...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 814, `destination`: Set this name to the Boolean condition `settings.recovery_output or output / 'recovered_flavor_referen...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 815: call `recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 816: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `recover_flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 817–818:

```
817 if settings.first_year > settings.last_year:
818     raise ValueError("first_year must not exceed last_year")
```

Read this block:

- Line 817: test `settings.first_year > settings.last_year`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 818: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 819–820:

```
819 if settings.hhi_only and settings.no_figures:
820     raise ValueError("--hhi-only cannot be combined with --no-figures")
```

Read this block:

- Line 819: test `settings.hhi_only and settings.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 820: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 821–822:

```
821 if settings.household_batches < 1:
822     raise ValueError("--household-batches must be positive")
```

Read this block:

- Line 821: test `settings.household_batches < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 822: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 823–823:

```
823 output = settings.output or settings.data_dir / "summary_outputs"
```

Read this block:

- Line 823, `output`: Set this name to the Boolean condition `settings.output or settings.data_dir / 'summary_outputs'`; it can be a scalar or a row/array mask depending on its inputs.

Source lines 824–832:

```
824 if settings.hhi_only:
825     source = output / "yogurt_household_flavors.csv"
826     if not source.exists() and not source.with_suffix(".parquet").exists():
827         raise ValueError(f"HHI-only needs the saved detailed household table: {source}")
828     make_hhi_figures(output)
829     grouped = output / "aggregated_flavors"
830     if (grouped / "yogurt_household_flavors.csv").exists() or (grouped /
        ↪ "yogurt_household_flavors.parquet").exists():
831         make_hhi_figures(grouped)
832     return
```

Read this block:

- Line 824: test `settings.hhi_only`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 825, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 826: test `not source.exists() and (not source.with_suffix('.parquet').exists())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 827: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 828: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 829, `grouped`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 830: test `(grouped / 'yogurt_household_flavors.csv').exists() or (grouped / 'yogurt_hou...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 831: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 832: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 833–833:

```
833 output.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 833: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 834–835:

```
834 with tempfile.TemporaryDirectory(prefix="summary-work-", dir=output) as tmp:
835     run(settings, output, Path(tmp))
```

Read this block:

- Line 834: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 835: call `run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `run`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

../final_simple/demand/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

Source lines 1–1:

```
"""Demand research scripts."""
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

../final_simple/demand/estimation.py

Current split-history estimator with dynamic three/five alternative dimensions, shared random price/novelty/stock coefficients, analytic gradients, inference and population WTP. The pipeline explicitly selects uncontrolled demand and original previous-share novelty; controlled and earlier models remain sensitivity options.

Module setup and direct-execution blocks

Source lines 1–13:

```
"""Demand estimation. Edit Parameters, then run python3 demand/estimation.py.

Read from top to bottom: inputs -> weekly data -> histories -> probabilities ->
likelihood -> optimization -> standard errors -> saved results.
No calculations are imported from final_draft. All seven model choices remain.
"""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
 ↪  "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 9: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 11, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 12: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 13: from `server_data_policy`, import `server_only, require_server` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–19:

```
from pathlib import Path
from statistics import NormalDist
from types import SimpleNamespace
```

```
17 import argparse
18 import json
19 import os
```

Read this block:

- Line 14: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: from `statistics`, import `NormalDist` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 17: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 18: import `json`. An as-name is a convenient local alias, not another library.
- Line 19: import `os`. An as-name is a convenient local alias, not another library.

Source lines 20–28:

```
20 import sys
21 import tempfile
22 from pprint import pformat
23
24 import numpy as np
25 import pandas as pd
26
27 # Use a writable plotting cache, including on a shared research server.
28 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "final_simple_matplotlib"))
```

Read this block:

- Line 20: import `sys`. An as-name is a convenient local alias, not another library.
- Line 21: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 22: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 24: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 25: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 28: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 30–35:

```
30 import time
31 import polars as pl
32 from scipy.optimize import minimize
33 from scipy.special import logsumexp
34 from scipy.stats import t as student_t
35 from scipy.stats import norm
```

Read this block:

- Line 30: import `time`. An as-name is a convenient local alias, not another library.
- Line 31: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 32: from `scipy.optimize`, import `minimize` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 33: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 34: from `scipy.stats`, import `t as student_t` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 35: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 42–46:

```
DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
sample = DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet"
flavors = DATA_DIR / "summary_outputs/estimation_flavor_reference.csv"
products_extra = None                 # Optional flavor metadata for older inputs.
output = Path("/scratch/dtm63837/Kilts_Panel/results") / "estimation_split"
```

Read this block:

- Line 42, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 43, `sample`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 44, `flavors`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45, `products_extra`: Set this name to the literal value `None`.
- Line 46, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 51–56:

```
model = "heterogeneous_history"
flavor_spec = "three"               # "three" baseline or "five" expanded groups.
first_year = 2006
last_year = 2015
max_households = None                # Use 100 for a development sample.
capacity = 7                         # Pricing setup; use 0 for conditional flavor choice.
```

Read this block:

- Line 51, `model`: Set this name to the literal text `'heterogeneous_history'`.
- Line 52, `flavor_spec`: Set this name to the literal text `'three'`.
- Line 53, `first_year`: Set this name to the literal value `2006`.
- Line 54, `last_year`: Set this name to the literal value `2015`.
- Line 55, `max_households`: Set this name to the literal value `None`.
- Line 56, `capacity`: Set this name to the literal value `7`.

Source lines 57–62:

```
no_price_control = False             # False means include the IV residual control.
household_dma_column = "dma_cd"
habit_weight = 0.7                   # Xi = (1-lambda)*novelty - lambda*own stock.
draws = 128                          # Even number of fixed integration draws.
starts = 3                           # Number of optimizer starting points.
maxiter = 600
```

Read this block:

- Line 57, `no_price_control`: Set this name to the literal value `False`.
- Line 58, `household_dma_column`: Set this name to the literal text `'dma_cd'`.
- Line 59, `habit_weight`: Set this name to the literal value `0.7`.
- Line 60, `draws`: Set this name to the literal value `128`.
- Line 61, `starts`: Set this name to the literal value `3`.
- Line 62, `maxiter`: Set this name to the literal value `600`.

Source lines 63–66:

```
seed = 306
se_method = "hessian"                # hessian, bootstrap, or none.
```

```
bootstrap_reps = 49
bootstrap_starts = 2
```

Read this block:

- Line 63, `seed`: Set this name to the literal value `306`.
- Line 64, `se_method`: Set this name to the literal text `'hessian'`.
- Line 65, `bootstrap_reps`: Set this name to the literal value `49`.
- Line 66, `bootstrap_starts`: Set this name to the literal value `2`.

Source lines 70–74:

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from flavor_specs import categories

FLAVORS = ["other", "berry", "plain"] # This column order is used in all arrays.
FLAVOR_FILE = flavors
```

Read this block:

- Line 70: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 71: from `flavor_specs`, import `categories` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 73, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 74, `FLAVOR_FILE`: Set this name to the object currently named `flavors`; plain assignment does not copy it.

Source lines 1292–1320:

```
if __name__ == "__main__":
    # Collect the ordinary Python variables above into one settings record.
    # settings.draws means exactly the variable draws from the Parameters section.
    settings = SimpleNamespace(
        sample=sample,
        flavors=flavors,
        products_extra=products_extra,
        output=output,
        model=model,
        flavor_spec=flavor_spec,
        first_year=first_year,
        last_year=last_year,
        max_households=max_households,
        capacity=capacity,
        no_price_control=no_price_control,
        household_dma_column=household_dma_column,
        se_method=se_method,
        bootstrap_reps=bootstrap_reps,
        bootstrap_starts=bootstrap_starts,
        habit_weight=habit_weight,
        novelty_measure="previous_share",
        draws=draws,
        starts=starts,
        maxiter=maxiter,
        seed=seed,
    )
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_estimation(settings)
```

Read this block:

- Line 1292: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1295, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 1318: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1319, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 1320: call `run_estimation` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_estimation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

check dma

Original function: `check_dma`. Source begins at line 83.

This converts a pandas Series to integer DMA IDs, rejecting missing, nonnumeric, nonpositive, and fractional entries. `pd.to_numeric(errors="coerce")` turns bad strings into missing numbers so one Boolean mask can find all invalid rows. `codes == np.floor(codes)` checks integrality without rounding fractional inputs into a valid ID. Only after validation does `astype("int64")` convert the dtype. `description` makes an error identify the consumer or retail source. Quietly replacing an invalid DMA with zero would contaminate own-market exclusion.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 83–84:

```
83 def check_dma(values, description):
84     """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
```

`def` creates this callable; the parentheses name its inputs (`values`, `description`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 85–86:

```
85 codes = pd.to_numeric(values, errors="coerce")
86 valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
```

Read this block:

- Line 85, `codes`: Set this name to the result of calling `pd.to_numeric(...)`; the arguments and preceding method chain are shown above.
- Line 86, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 87–88:

```
87 if not valid.all():
88     raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
```

Read this block:

- Line 87: test `not valid.all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 88: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 89–89:

```
89 return codes.astype("int64")
```

Read this block:

- Line 89: return the result of calling `codes.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

other dma instruments

Original function: `other_dma_instruments`. Source begins at line 92.

This retained helper accepts three pandas tables and returns weekly rows with UPC-matched flavor IVs. It is **not** the pooled instrument construction used by current `est.py`. It first averages retail-DMA prices outside the consumer's DMA per UPC/week, then averages across the exposure store's UPC basket.

Renaming the own-DMA table's columns makes its keys match the merge. `present` is a Boolean Series; converting it to integers subtracts one from the DMA count only when that own-market quote exists. The grouped `count` ignores missing values whereas `size` counts all rows. Comparing them detects an incompletely instrumented basket and marks the whole flavor IV missing, rather than quietly averaging only a subset. Several joins replace a much slower nested lookup loop.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 92–98:

```
92 def other_dma_instruments(weekly, exposure_upcs, dma_prices):
93     """Match each exposure-store UPC to its same-week price in OTHER DMAs.
94 
95     dma_prices has ONE mean package price per UPC/week/retail DMA. First average
96     these means over all DMAs except the CONSUMER's DMA, then average over the
97     exposure store's UPCs in a flavor. This matches the local price's UPC basket.
```

```
98      """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `exposure_upcs`, `dma_prices`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 99–106:

```
99  dma_prices = dma_prices.copy()
100 dma_prices["dma_code"] = check_dma(dma_prices["dma_code"], "retail")
101 pk = ["upc", "week"]
102 totals = (dma_prices.groupby(pk)["dma_price"].agg(["sum", "count"])
103           .rename(columns={"sum": "sum_dma", "count": "n_dma"}).reset_index())
104 exposure = weekly[["store_code_uc", "week", "dma_cd"]].drop_duplicates()
105 exposure = exposure.merge(exposure_upcs, on=["store_code_uc", "week"],
106                           how="left", validate="many_to_many")
```

Read this block:

- Line 99, `dma_prices`: Set this name to the result of calling `dma_prices.copy(...)`; the arguments and preceding method chain are shown above.
- Line 100, `dma_prices['dma_code']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 101, `pk`: Set this name to the list `['upc', 'week']`.
- Line 102, `totals`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 104, `exposure`: Set this name to the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above.
- Line 105, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 107–113:

```
107 exposure = exposure.merge(totals, on=pk, how="left", validate="many_to_one")
108 own = dma_prices.rename(columns={"dma_code": "dma_cd", "dma_price": "own_dma_price"})
109 exposure = exposure.merge(own, on=pk + ["dma_cd"], how="left", validate="many_to_one")
110 present = exposure["own_dma_price"].notna()
111 denominator = exposure["n_dma"] - present.astype(int)
112 exposure["iv"] = ((exposure["sum_dma"] - exposure["own_dma_price"].fillna(0))
113                   / denominator.where(denominator > 0))
```

Read this block:

- Line 107, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

- Line 108, `own`: Set this name to the result of calling `dma_prices.rename(...)`; the arguments and preceding method chain are shown above.
- Line 109, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 110, `present`: Set this name to the result of calling `exposure['own_dma_price'].notna(...)`; the arguments and preceding method chain are shown above.
- Line 111, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 112, `exposure['iv']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 115–119:

```
115 keys = ["store_code_uc", "week", "dma_cd", "flavor"]
116 grouped = exposure.groupby(keys)["iv"].agg(["mean", "count", "size"])
117 grouped.loc[grouped["count"] != grouped["size"], "mean"] = np.nan
118 result = (grouped["mean"].unstack("flavor").reindex(columns=FLAVORS)
119           .rename(columns={f: "iv_" + f for f in FLAVORS}).reset_index())
```

Read this block:

- Line 115, `keys`: Set this name to the list `['store_code_uc', 'week', 'dma_cd', 'flavor']`.
- Line 116, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 117, `grouped.loc[grouped['count'] != grouped['size'], 'mean']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 118, `result`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 120–121:

```
120 return weekly.merge(result, on=["store_code_uc", "week", "dma_cd"], how="left",
121                     validate="many_to_one")
```

Read this block:

- Line 120: return the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above. This ends the call.

absorb effects

Original function: `absorb_effects`. Source begins at line 126.

Input `values` is a numerical matrix; `groups` lists row-aligned group labels such as week and DMA. Output is a copy with group means removed. Factorization converts arbitrary labels to dense integers. `np.add.at(means, code, residual)` accumulates rows into group sums even when indices repeat; ordinary fancy-index `means[code] += residual` may not accumulate repeats as intended.

`counts[:, None]` makes group counts a column so every variable is divided by its group size. The function cycles week and DMA demeaning until the biggest absolute change falls below tolerance, then returns. A maximum iteration limit raises on failure. A huge explicit dummy matrix is a valid alternative used in tests, but its memory grows with the number of fixed-effect groups. One pass is insufficient when group cells are unbalanced.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 126–131:

```
126 def absorb_effects(values, groups, tolerance=1e-10, maxiter=2000):
127     """Remove week and DMA means without a huge matrix of dummy variables.
128 
129     Repeated demeaning is needed because DMA/week cells are unbalanced.
130     Each household-week exposure has equal weight in the first stage.
131     """
```

`def` creates this callable; the parentheses name its inputs (`values`, `groups`, `tolerance`, `maxiter`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 132–133:

```
132 residual = np.asarray(values, dtype=float).copy()
133 codes = [pd.factorize(group, sort=True)[0] for group in groups]
```

Read this block:

- Line 132, `residual`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 133, `codes`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 134–143:

```
134 for _ in range(maxiter):
135     previous = residual.copy()
136     for code in codes:
137         counts = np.bincount(code)
138         means = np.zeros((len(counts), residual.shape[1]))
139         np.add.at(means, code, residual)
140         means /= counts[:, None]
141         residual -= means[code]
142     if np.max(np.abs(residual - previous)) < tolerance:
143         return residual
```

Read this block:

- Line 134: for each entry yielded by `range(maxiter)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 135, `previous`: Set this name to the result of calling `residual.copy(...)`; the arguments and preceding method chain are shown above.
- Line 136: for each entry yielded by `codes`, bind/unpack it into `code` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 137, `counts`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 138, `means`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 139: call `np.add.at` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: update `means` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 141: update `residual` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 142: test `np.max(np.abs(residual - previous)) < tolerance`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 143: return the object currently named `residual`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.add.at`: Accumulate in place at possibly repeated indices; this avoids the lost-repeat behavior of buffered fancy-index updates.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 144–144:

```
144 raise ValueError("First-stage fixed-effect demeaning did not converge.")
```

Read this block:

- Line 144: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

add price controls

Original function: `add_price_controls`. Source begins at line 149.

This returns `(augmented_frame, diagnostic_table)`. For each flavor it selects covered prices, absorbs week/DMA effects jointly from price, instrument, and Xi, then partials Xi out of y and z. `within[:, 0]`, `[:, 1]`, `[:, 2:]` split those roles by known column order. `lstsq(...)[0]` keeps only the coefficient vector from the least-squares result tuple.

Dot products compute the IV slope after partialling. A small residual instrument sum of squares raises instead of dividing by an almost zero denominator. `np.linalg.lstsq` tolerates redundant controls more gracefully than explicitly inverting their cross-product matrix. The residual is assigned only to covered rows; unavailable alternatives retain zero placeholders. Cluster score sums, partial R-squared, and residual SD describe first-stage

relevance and fit. They do not prove instrument validity. Reusing this function in bootstrap replications keeps the two-stage estimator consistent across samples.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 149–154:

```
149 def add_price_controls(frame):
150     """OLS separately by flavor: price ~ IV + all selected flavor Xi + week FE + DMA FE.
151 
152     Refit this function in every household bootstrap sample. Return residuals
153     in DOLLARS (no arbitrary rescaling) for the choice-model control function.
154     """
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 155–156:

```
155 result = frame.copy()
156 reports = []
```

Read this block:

- Line 155, `result`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 156, `reports`: Set this name to the list `[]`.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 157–196:

```
157 for flavor in FLAVORS:
158     columns = ["price_" + flavor, "iv_" + flavor] + ["xi_" + f for f in FLAVORS]
159     covered = np.isfinite(result["price_" + flavor]) & (result["price_" + flavor] > 0)
160     sample = result.loc[covered]
161     if sample.empty:
162         raise ValueError(f"No covered prices for the {flavor} first stage.")
163     hh, households = pd.factorize(sample["household_code"], sort=True)
164     values = sample[columns].to_numpy(float)
165     if not np.isfinite(values).all():
166         raise ValueError(f"Nonfinite first-stage inputs for {flavor}.")
167     within = absorb_effects(values, [sample["week"], sample["dma_cd"]])
168     y, z, controls = within[:, 0], within[:, 1], within[:, 2:]
169     # Partial out Xi before estimating the excluded instrument's coefficient.
170     # This tolerates redundant Xi columns, but NOT a redundant instrument.
171     z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
172     y_partial = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
173     z_ss = z_partial @ z_partial
174     if z_ss <= 1e-12 * max(1., values[:, 1] @ values[:, 1]):
175         raise ValueError(f"No excluded-instrument variation after controls for {flavor}.")
176     pi = (z_partial @ y_partial) / z_ss
177     control_beta = np.linalg.lstsq(controls, y - pi*z, rcond=None)[0]
178     residual = y - pi*z - controls @ control_beta
179     if np.std(residual) < 1e-10:
180         raise ValueError(f"First-stage residual for {flavor} is essentially zero.")
181     result["cf_" + flavor] = 0.
182     result.loc[covered, "cf_" + flavor] = residual
183     # Household-clustered partial-IV t/F diagnostic (not a weak-IV-robust test).
```

```
    scores = np.bincount(hh, weights=z_partial * residual, minlength=len(households))
    pi_se = np.sqrt((scores @ scores) / z_ss**2)
    if len(households) > 1:
        pi_se *= np.sqrt(len(households) / (len(households) - 1))
    f_stat = float((pi / pi_se)**2) if pi_se > 0 else np.nan
    restricted_ss = y_partial @ y_partial
    partial_r2 = 1 - (residual @ residual) / restricted_ss if restricted_ss > 0 else np.nan
    reports.append(dict(flavor=flavor, iv_coefficient=float(pi),
                        iv_cluster_se=float(pi_se), iv_cluster_F=f_stat,
                        iv_pvalue=float(2*student_t.sf(np.sqrt(f_stat), len(households)-1))
                        if len(households) > 1 else np.nan,
                        partial_R2=float(partial_r2), residual_sd=float(np.std(residual)),
                        observations=len(sample), households=len(households)))
```

Read this block:

- Line 157: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 158, `columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 159, `covered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 160, `sample`: Set this name to the entry/slice selected by `result.loc[covered]`; brackets select from the object to their left.
- Line 161: test `sample.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 162: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 163, `(hh, households)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 164, `values`: Set this name to the result of calling `sample[columns].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 165: test `not np.isfinite(values).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 166: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 167, `within`: Set this name to the result of calling `absorb_effects(...)`; the arguments and preceding method chain are shown above.
- Line 168, `(y, z, controls)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 171, `z_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 172, `y_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 173, `z_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 174: test `z_ss <= 1e-12 * max(1.0, values[:, 1] @ values[:, 1])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 175: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 176, `pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 177, `control_beta`: Set this name to the entry/slice selected by `np.linalg.lstsq(controls, y - pi * z, rcond=None)[0]`; brackets select from the object to their left.
- Line 178, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 179: test `np.std(residual) < 1e-10`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 180: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 181, `result['cf_' + flavor]`: Store the literal value `0.0` in the selected entry/column.
- Line 182, `result.loc[covered, 'cf_' + flavor]`: Store the object currently named `residual`; plain assignment does not copy it in the selected entry/column.
- Line 184, `scores`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 185, `pi_se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 186: test `len(households) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 187: update `pi_se` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 188, `f_stat`: Set this name to the true-branch value if `pi_se > 0` holds, otherwise the else-branch value.
- Line 189, `restricted_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 190, `partial_r2`: Set this name to the true-branch value if `restricted_ss > 0` holds, otherwise the else-branch value.
- Line 191: call `reports.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `len`: Count elements/rows rather than summing their numerical values.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.sf`: Evaluate survival probability above a threshold, useful for a two-sided t diagnostic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 197–197:

```
197 return result, pd.DataFrame(reports)
```

Read this block:

- Line 197: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 205.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 205–206:

```
205 def normalize_upcs(values):
206     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 207–209:

```
207 text = values.astype("string").str.strip()
208 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
209 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 207, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 208, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 209, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 210–210:

```
210 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 210: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 212.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 212–212:

```
212 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 213–215:

```
213 text = pl.col("upc").cast(pl.String).str.strip_chars()
214 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
215 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 213, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 214, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 215, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 216–221:

```
216 return (
217     pl.when(numeric)
218     .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
219     .otherwise(text)
220     .alias("upc")
221 )
```

Read this block:

- Line 216: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

read flavors

Original function: `read_flavors`. Source begins at line 224.

Input `path` is a CSV path; optional `products_extra` is a Parquet path. Output is a two-column pandas table with one row per UPC. This function has an early return: if the CSV already contains resolved `upc,flavor`, validation succeeds and the rest of the recovery/classification code is never executed.

`{"upc", "flavor"}.issubset(frame.columns)` asks whether both required labels exist. `missing = required - set(frame.columns)` is set subtraction. Validation uses `.any()` for "at least one invalid row" and `.all()` for "every label is allowed." `if products_extra is not None` distinguishes an omitted optional argument from a supplied path that happens not to exist.

The metadata path scans only useful columns and anti-joins existing reference UPCs. It appends absent UPCs, rather than overwriting existing classifications. `[... for c in columns if c != "upc"]` builds one aggregation expression per non-key field. `np.select` picks the first matching condition, so plain wins when both plain and berry text occur. A generic substring loop could be easier to write at first, but would make precedence and vectorized missing-value handling less explicit. `drop_duplicates(..., keep="first")` is a reconciliation rule; it does not prove the retained label is correct.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 224–225:

```
224 def read_flavors(path, products_extra=None):
225     """Use the same plain-first classification as data_code/summ_stats.py."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `products_extra`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 226–226:

```
226 frame = pd.read_csv(path, dtype={"upc": str})
```

Read this block:

- Line 226, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 227–241:

```
227 if {"upc", "flavor"}.issubset(frame.columns):
228     # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
229     frame["upc"] = normalize_upcs(frame["upc"])
230     if (
231         frame["upc"].isna().any()
232         or frame["upc"].eq("").any()
```

```
233         or frame["upc"].duplicated().any()
234     ):
235         raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
236     if not frame["flavor"].isin(FLAVORS).all():
237         raise ValueError(f"{path}: saved mapping must use {FLAVORS}.")
238     print(
239         f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
240     )
241     return frame[["upc", "flavor"]]
```

Read this block:

- Line 227: test `{'upc', 'flavor'}.issubset(frame.columns)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 229, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 230: test `frame['upc'].isna().any() or frame['upc'].eq('').any() or frame['upc'].duplic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 235: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 236: test `not frame['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 237: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 238: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 241: return the entry/slice selected by `frame[['upc', 'flavor']]`; brackets select from the object to their left. This ends the call.

Syntax and calls used here:

- `normalize_upcs`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 242–242:

```
242 missing = {"upc", "flavor_descr"} - set(frame.columns)
```

Read this block:

- Line 242, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 243–244:

```
243 if missing:
244     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 243: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 244: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 245–245:

```
245 frame["upc"] = normalize_upcs(frame["upc"])
```

Read this block:

- Line 245, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 246–247:

```
246 if frame["upc"].isna().any() or frame["upc"].eq("").any():
247     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 246: test `frame['upc'].isna().any() or frame['upc'].eq('').any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 247: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 248–289:

```
248 if products_extra is not None:
249     products_extra = Path(products_extra)
250     if products_extra.exists():
251         products = pl.scan_parquet(products_extra)
252         names = products.collect_schema().names()
253         if not {"upc", "flavor_descr"}.issubset(names):
254             raise ValueError(
255                 f"{products_extra}: needs upc and flavor_descr columns."
256             )
257         columns = [
258             c
259             for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
260             if c in names
261         ]
262         products = (
263             products.select(columns)
264             .with_columns(normalized_upc_expr())
265             .join(
266                 pl.DataFrame(
267                     {"upc": frame["upc"].dropna().unique().tolist()}
268                 ).lazy(),
269                 on="upc",
270                 how="anti",
271             )
272         )
273         products = products.filter(
274             pl.col("upc").is_not_null()
```

```
            & pl.col("flavor_descr").is_not_null()
            & ~pl.col("flavor_descr")
            .cast(pl.String)
            .str.strip_chars()
            .str.to_lowercase()
            .is_in(["", "na", "n/a", "nan", "null", "none"])
        )
        if "panel_year" in names:
            products = products.group_by("upc").agg(
                [
                    pl.col(c)
                    .sort_by("panel_year", nulls_last=True)
                    .first()
                    .alias(c)
                    for c in columns
```

Source lines 290–307:

```
                if c != "upc"
            ]
        )
    else:
        products = products.unique(
            subset="upc", keep="first", maintain_order=True
        )
    recovered = products.collect(engine="streaming").to_pandas()
    frame = pd.concat([frame, recovered], ignore_index=True)
    print(
        f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
        flush=True,
    )
else:
    print(
        f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
        flush=True,
    )
```

Read this block:

- Line 248: test `products_extra is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 249, `products_extra`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 250: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 251, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 252, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 253: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 254: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 257, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 262, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 273, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.
- Line 282: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch

handles its alternative.
- Line 283, `products`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 294, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.
- Line 297, `recovered`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 298, `frame`: Set this name to the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above.
- Line 299: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 304: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 308–308:

```
308 text = frame["flavor_descr"].fillna("")
```

Read this block:

- Line 308, `text`: Set this name to the result of calling `frame['flavor_descr'].fillna(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 309–310:

```
309 if "product_descr" in frame.columns:
310     text = text + " " + frame["product_descr"].fillna("")
```

Read this block:

- Line 309: test `'product_descr' in frame.columns`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 310, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 311–316:

```
311 text = text.str.lower()
312 from flavor_specs import classify_descriptor
313 specification = "five" if len(FLAVORS) == 5 else "three"
314 frame["flavor"] = text.map(lambda value: classify_descriptor(value, specification))
315 # Match summ_stats: retain missing labels and use the first CSV row per UPC.
316 conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
```

Read this block:

- Line 311, `text`: Set this name to the result of calling `text.str.lower(...)`; the arguments and preceding method chain are shown above.
- Line 312: from `flavor_specs`, import `classify_descriptor` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 313, `specification`: Set this name to the true-branch value if `len(FLAVORS) == 5` holds, otherwise the else-branch value.
- Line 314, `frame['flavor']`: Store the result of calling `text.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 316, `conflicts`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `classify_descriptor`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 317–322:

```
317 if conflicts:
318     print(
```

```
        f"{conflicts:,} UPCs have conflicting flavor classifications; "
        "keeping the first flavor in CSV row order.",
        flush=True,
    )
```

Read this block:

- Line 317: test `conflicts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 318: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 323–323:

```
return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

Read this block:

- Line 323: return the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

week dates

Original function: `week_dates`. Source begins at line 325.

`pd.to_datetime` interprets strings with an explicit `%Y%m%d` format: four-digit year, two-digit month, two-digit day. The regex removes a trailing `.0` from integer dates that were stored as float-like text. `errors="raise"` rejects a malformed date instead of silently creating a missing date.

For example, `20100109` should mean January 9, 2010. Passing that integer directly to a general timestamp constructor can instead mean nanoseconds from an epoch. Explicit parsing prevents that mistake. This helper exists in the module, but the current main preparation path parses purchase-date strings with Polars directly; a function's presence does not mean the entry point uses it.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 325–326:

```
def week_dates(values):
    """Parse YYYYMMDD safely (integer dates are NOT nanoseconds)."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 327–331:

```
return pd.to_datetime(
    values.astype(str).str.replace(r"\.0$", "", regex=True),
    format="%Y%m%d",
    errors="raise",
)
```

Read this block:

- Line 327: return the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

prepare data

Original function: `prepare_data`. Source begins at line 336.

Configures the saved/selected three-or-five category list before constructing purchase weeks, exposure-store menus and histories. Demand is restricted to single-person households while broader quote coverage remains. Main jobs omit price controls; controlled sensitivity behavior is retained.

Definition and stated intent:

Source lines 336–341:

```
336 def prepare_data(settings):
337     """Use only the merged purchase table, including its existing prices.
338 
339     The IV pools all flavors: the same-week average observed price outside the
340     consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
341     """
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 342–344:

```
342 FLAVORS[:] = categories(getattr(settings, "flavor_spec", "three"))
343 base = pl.scan_parquet(settings.sample)
344 names = base.collect_schema().names()
```

Read this block:

- Line 342, `FLAVORS[:]`: Store the result of calling `categories(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 343, `base`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 344, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 345–346:

```
345 if "household_size" not in names:
346     raise ValueError("Estimation requires household_size to restrict purchases to single-person households.")
```

Read this block:

- Line 345: test `'household_size' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 346: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 347–347:

```
347 base = base.filter(pl.col("panel_year").is_between(settings.first_year, settings.last_year))
```

Read this block:

- Line 347, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 348–353:

```
348 if "flavor" not in names:
349     products_extra = getattr(settings, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet"
350     flavor = read_flavors(settings.flavors, products_extra)
351     base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
352 else:
353     base = base.with_columns(normalized_upc_expr())
```

Read this block:

- Line 348: test `'flavor' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 349, `products_extra`: Set this name to the Boolean condition `getattr(settings, 'products_extra', None) or DATA_DIR / 'RMS/c...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 350, `flavor`: Set this name to the result of calling `read_flavors(...)`; the arguments and preceding method chain are shown above.
- Line 351, `base`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 353, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `read_flavors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 354–354:

```
354 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
```

Read this block:

- Line 354, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 355–356:

```
355 if price_column is None:
356     raise ValueError("Merged purchase table needs a price or price_rms column.")
```

Read this block:

- Line 355: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 356: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 357–359:

```
357 consumer_dma = getattr(settings, "household_dma_column", "dma_cd")
358 columns = ["household_code", "household_size", "panel_year", "purchase_date", "trip_code_uc", "upc",
359            "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
```

Read this block:

- Line 357, `consumer_dma`: Set this name to the result of calling `getattr(...)`; the arguments and preceding method chain are shown above.
- Line 358, `columns`: Set this name to a list containing 10 displayed entries.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 360–361:

```
360 if not settings.no_price_control:
361     columns.append(pl.col(consumer_dma).alias("dma_cd"))
```

Read this block:

- Line 360: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 361: call `columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 362–369:

```
362 base = base.select(columns).with_columns(
363     pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
364 base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
365 base = base.with_columns((pl.col("purchase_date") + pl.duration(
366     days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
367 # Keep the full merged table for prices/IVs, even in a household-limited run.
368 # Restrict demand histories before weekly aggregation; retain all quotes for menus/IVs.
369 selected = base.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 362, `base`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 364, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.
- Line 365, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 369, `selected`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 370–373:

```
370 if settings.max_households:
371     ids = (selected.select("household_code").unique().sort("household_code")
372            .limit(settings.max_households).collect(engine="streaming")["household_code"])
373     selected = selected.filter(pl.col("household_code").is_in(ids.implode()))
```

Read this block:

- Line 370: test `settings.max_households`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 371, `ids`: Set this name to the entry/slice selected by `selected.select('household_code').unique().sort('househol...`; brackets select from the object to their left.
- Line 373, `selected`: Set this name to the result of calling `selected.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 374–375:

```
374 print(f"Loading merged purchase table: {settings.sample}", flush=True)
375 lines = selected.collect(engine="streaming").to_pandas()
```

Read this block:

- Line 374: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 375, `lines`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.

Source lines 376–377:

```
376 if lines.empty:
377     raise ValueError("No purchase lines in selected sample.")
```

Read this block:

- Line 376: test `lines.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 377: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 378–379:

```
378 if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
379     raise ValueError("Merged purchase table has missing or invalid flavor labels.")
```

Read this block:

- Line 378: test `lines['flavor'].isna().any() or not lines['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 379: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 380–380:

```
380 q = lines["quantity"].to_numpy(float)
```

Read this block:

- Line 380, `q`: Set this name to the result of calling `lines['quantity'].to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 381–382:

```
381 if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
382     raise ValueError("Quantity must be positive integer package counts.")
```

Read this block:

- Line 381: test `not np.isfinite(q).all() or (q <= 0).any() or (not np.allclose(q, np.round(q)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 382: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 383–389:

```
383 lines["week"] = pd.to_datetime(lines["week"])
384 keys = ["household_code", "week"]
385 weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
386           .reindex(columns=FLAVORS, fill_value=0).reset_index())
387 stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
388 stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
389 stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
```

Read this block:

- Line 383, `lines['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 384, `keys`: Set this name to the list `['household_code', 'week']`.
- Line 385, `weekly`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 387, `stores`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 388, `stores`: Set this name to the result of calling `stores.sort_values(...)`; the arguments and preceding method chain are shown above.
- Line 389, `stores`: Set this name to the entry/slice selected by `stores.drop_duplicates(keys)[keys + ['store_code_uc']]`; brackets select from the object to their left.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

Source lines 390–390:

```
390 weekly = weekly.merge(stores, on=keys, validate="one_to_one")
```

Read this block:

- Line 390, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 391–395:

```
391 if not settings.no_price_control:
392     lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
393     if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
394         raise ValueError("Multiple consumer DMAs within a household-week.")
395     weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")
```

Read this block:

- Line 391: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 392, `lines['dma_cd']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 393: test `(lines.groupby(keys)['dma_cd'].nunique() > 1).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 394: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 395, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 397–398:

```
397 quote_keys = ["store_code_uc", "week", "upc"]
398 price_columns = quote_keys + ["flavor", "price"]
```

Read this block:

- Line 397, `quote_keys`: Set this name to the list `['store_code_uc', 'week', 'upc']`.
- Line 398, `price_columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 399–400:

```
399 if not settings.no_price_control:
400     price_columns.append("dma_cd")
```

Read this block:

- Line 399: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 400: call `price_columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 401–407:

```
401 quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
402           .select(price_columns).unique())
403 # A store/week/UPC should have one price and one flavor. Consumer DMA may
404 # differ when consumers from different markets visit the same store.
405 local = quotes.select(quote_keys + ["flavor", "price"]).unique()
406 conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
407                 .limit(1).collect(engine="streaming"))
```

Read this block:

- Line 401, `quotes`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

- Line 405, `local`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 406, `conflicting`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 408–409:

```
408 if len(conflicting):
409     raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
```

Read this block:

- Line 408: test `len(conflicting)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 409: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 410–416:

```
410 menus = (local.group_by(["store_code_uc", "week", "flavor"])
411          .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
412 menus["week"] = pd.to_datetime(menus["week"])
413 menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
414          .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
415 weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
416 print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
```

Read this block:

- Line 410, `menus`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 412, `menus['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 413, `menus`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 415, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 416: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 417–441:

```
417 if not settings.no_price_control:
418     invalid_dma = quotes.select(
419         ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
420          | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
421          | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64,
          ↪  strict=False).floor()))
```

```
422         .fill_null(True).any()).collect(engine="streaming").item()
423     if invalid_dma:
424         raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
425     # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
426     by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
427               .group_by(["week", "dma_cd"]).agg(
428                   pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
429               .collect(engine="streaming").to_pandas())
430     by_dma["week"] = pd.to_datetime(by_dma["week"])
431     totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
432               .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
433     weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
434     weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
435     denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
436     weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
437                             / denominator.where(denominator > 0))
438     for f in FLAVORS:
439         weekly["iv_" + f] = weekly["iv_overall"]
440     weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
441     print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
```

Read this block:

- Line 417: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 418, `invalid_dma`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 423: test `invalid_dma`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 424: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 426, `by_dma`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 430, `by_dma['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 431, `totals`: Set this name to the result of calling `.rename(...)`; the arguments and preceding method chain are shown above.
- Line 433, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 434, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 435, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 436, `weekly['iv_overall']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 438: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 439, `weekly['iv_' + f]`: Store the entry/slice selected by `weekly['iv_overall']`; brackets select from the object to their left in the selected entry/column.
- Line 440, `weekly`: Set this name to the result of calling `weekly.drop(...)`; the arguments and preceding method chain are shown above.
- Line 441: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 442–447:

```
442 weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), settings.habit_weight,
443                     getattr(settings, "novelty_measure", "previous_share"))
444 price_cols = ["price_" + f for f in FLAVORS]
445 available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
446 counts = weekly[FLAVORS].to_numpy(float)
447 complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Read this block:

- Line 442, `weekly`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 444, `price_cols`: Set this name to a list computed by the displayed for-expression.
- Line 445, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 446, `counts`: Set this name to the result of calling `weekly[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 447, `complete`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 448–449:

```
448 for j, f in enumerate(FLAVORS):
449     weekly["available_" + f] = available[:, j]
```

Read this block:

- Line 448: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 449, `weekly['available_' + f]`: Store the entry/slice selected by `available[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 450–450:

```
450 print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
```

Read this block:

- Line 450: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 451–455:

```
451 if not settings.no_price_control:
452     iv_cols = ["iv_" + f for f in FLAVORS]
453     iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
454     print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True)
455     complete &= iv_complete
```

Read this block:

- Line 451: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 452, `iv_cols`: Set this name to a list computed by the displayed for-expression.
- Line 453, `iv_complete`: Set this name to the result of calling `.all(...)`; the arguments and preceding method chain are shown above.
- Line 454: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 455: update `complete` using `&=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 456–460:

```
456 weekly = weekly.loc[complete].copy()
457 repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
458 print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated,
    ↪  'household_code'].nunique():,}", flush=True)
459 weekly = weekly.loc[repeated].reset_index(drop=True)
460 weekly["outside"] = np.maximum(settings.capacity - weekly[FLAVORS].sum(axis=1), 0)
```

Read this block:

- Line 456, `weekly`: Set this name to the result of calling `weekly.loc[complete].copy(...)`; the arguments and preceding method chain are shown above.
- Line 457, `repeated`: Set this name to the Boolean condition `weekly.groupby('household_code')['week'].transform('nunique') ...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 458: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 459, `weekly`: Set this name to the result of calling `weekly.loc[repeated].reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 460, `weekly['outside']`: Store the result of calling `np.maximum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.transform`: Compute a pandas group result broadcast back to original rows, unlike a reduced one-row-per-group aggregation.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.

Source lines 461–462:

```
461 if weekly.empty:
462     raise ValueError("No households with at least two usable purchase weeks.")
```

Read this block:

- Line 461: test `weekly.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 462: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 463–463:

```
463 return weekly
```

Read this block:

- Line 463: return the object currently named `weekly`; plain assignment does not copy it. This ends the call.

switch away event

Original function: `switch_away_event`. Source begins at line 467.

Retained helper for the unadopted lagged-switch experiment. It detects displacement or introduction with falling share, but current jobs explicitly select previous_share novelty and do not use this event as their predictor.

Definition and stated intent:

Source lines 467–472:

```
467 def switch_away_event(before, after):
468     """Binary recovery event for each flavor; inputs are consecutive basket shares.
469 
470     A strict majority is dominance. A new entry only rewards flavors whose
471     shares fall. Multiple triggers still produce a single unit boost.
472     """
```

`def` creates this callable; the parentheses name its inputs (`before`, `after`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 473–475:

```
473 entered = (before == 0) & (after > 0)
474 takeover = ((before > .5)[:, None] & (after > .5)[None, :]
475             & ~np.eye(len(before), dtype=bool)).any(axis=1)
```

Read this block:

- Line 473, `entered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 474, `takeover`: Set this name to the result of calling `.any(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `len`: Count elements/rows rather than summing their numerical values.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 476–476:

```
476 return (takeover | (entered.any() & (after < before))).astype(float)
```

Read this block:

- Line 476: return the result of calling `.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

add stocks

Original function: `add_stocks`. Source begins at line 479.

Records cumulative own-share stock and prior-calendar-week novelty before current purchases. Current jobs select N=1-previous own share and zero after missing weeks; stock accumulates all observed prior shares. Three/five vector lengths follow FLAVORS. The optional lagged event branch is retained solely as an unadopted experiment.

Definition and stated intent:

Source lines 479–486:

```
479 def add_stocks(weekly, habit_weight=0.7, novelty_measure="previous_share"):
480     """Record cumulative stock and novelty before current purchases.
481 
482     previous_share: legacy N = 1 - own share in the preceding calendar week.
483     lagged_switch_away: N in t is the binary switch-away event from t-2 to t-1.
484     Both transitions must be adjacent calendar weeks; missing history gives 0.
485     Stock remains the sum of all preceding observed own-flavor shares.
486     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `habit_weight`, `novelty_measure`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 487–488:

```
487 if novelty_measure not in {"previous_share", "lagged_switch_away"}:
488     raise ValueError("Unknown novelty measure: " + str(novelty_measure))
```

Read this block:

- Line 487: test `novelty_measure not in {'previous_share', 'lagged_switch_away'}`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 488: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 489–493:

```
489 result = (
490     weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
491 )
492 stocks = np.zeros((len(result), len(FLAVORS)))
493 novelty = np.zeros_like(stocks)
```

Read this block:

- Line 489, `result`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 492, `stocks`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 493, `novelty`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 494–511:

```
494 for _, group in result.groupby("household_code", sort=False):
495     stock = np.zeros(len(FLAVORS))
496     previous_share = np.zeros(len(FLAVORS))
497     previous_week = None
498     previous_event = np.zeros(len(FLAVORS))
499     for index, row in group.iterrows():
500         stocks[index] = stock
501         adjacent = previous_week is not None and (row["week"] - previous_week).days == 7
502         if adjacent:
503             novelty[index] = (previous_share.sum() - previous_share
504                               if novelty_measure == "previous_share" else previous_event)
505         counts = row[FLAVORS].to_numpy(float)
506         current_share = counts / counts.sum()
507         previous_event = (switch_away_event(previous_share, current_share)
508                           if adjacent else np.zeros(len(FLAVORS)))
509         previous_share = current_share
510         stock = stock + previous_share
511         previous_week = row["week"]
```

Read this block:

- Line 494: for each entry yielded by `result.groupby('household_code', sort=False)`, bind/unpack it into `(_, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 495, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 496, `previous_share`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 497, `previous_week`: Set this name to the literal value `None`.
- Line 498, `previous_event`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 499: for each entry yielded by `group.iterrows()`, bind/unpack it into `(index, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 500, `stocks[index]`: Store the object currently named `stock`; plain assignment does not copy it in the selected entry/column.
- Line 501, `adjacent`: Set this name to the Boolean condition `previous_week is not None and (row['week'] - previous_week).da...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 502: test `adjacent`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 503, `novelty[index]`: Store the true-branch value if `novelty_measure == 'previous_share'` holds, otherwise the else-branch value in the selected entry/column.
- Line 505, `counts`: Set this name to the result of calling `row[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 506, `current_share`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 507, `previous_event`: Set this name to the true-branch value if `adjacent` holds, otherwise the else-branch value.
- Line 509, `previous_share`: Set this name to the object currently named `current_share`; plain assignment

does not copy it.
- Line 510, `stock`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 511, `previous_week`: Set this name to the entry/slice selected by `row['week']`; brackets select from the object to their left.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `switch_away_event`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 512–517:

```
512 for j, name in enumerate(FLAVORS):
513     result["stock_" + name] = stocks[:, j]
514     result["novelty_" + name] = novelty[:, j]
515     result["xi_" + name] = (1 - habit_weight) * novelty[
516         :, j
517     ] - habit_weight * stocks[:, j]
```

Read this block:

- Line 512: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 513, `result['stock_' + name]`: Store the entry/slice selected by `stocks[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 514, `result['novelty_' + name]`: Store the entry/slice selected by `novelty[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 515, `result['xi_' + name]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 518–518:

```
518 return result
```

Read this block:

- Line 518: return the object currently named `result`; plain assignment does not copy it. This ends the call.

make arrays

Original function: `make_arrays`. Source begins at line 522.

This converts a labeled table into a dictionary of NumPy arrays. `pd.factorize` returns both dense integer household codes and their original labels: household IDs `[10,10,30]` become codes `[0,0,1]`. Dense codes let `bincount` sum efficiently. The expression `FLAVORS + (["outside"] if capacity else [])` builds either three or four outcome columns without changing the inside-price array's three columns.

The function validates counts/Xi, checks menu consistency, and inserts zero placeholders for missing prices. The separate availability mask ensures those zeros are not interpreted as offered free goods. `all(c in frame for c in cf_cols)` asks whether every residual column exists. If controls are absent, `zeros_like` supplies the right-shaped residual array. The returned dictionary deliberately keeps flags such as `outside` and `control` beside arrays so the

likelihood can use one interface for several models. Passing raw tables directly into every optimization step would repeatedly perform label lookup and conversion.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 522–523:

```
522 def make_arrays(frame, capacity):
523     """Use household codes to sum log likelihoods before integrating types."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 524–530:

```
524 hh, labels = pd.factorize(frame["household_code"], sort=True)
525 counts = frame[FLAVORS + (["outside"] if capacity else [])].to_numpy(float)
526 prices = frame[["price_" + f for f in FLAVORS]].to_numpy(float)
527 xi = frame[["xi_" + f for f in FLAVORS]].to_numpy(float)
528 history = {key: frame[[prefix + f for f in FLAVORS]].to_numpy(float)
529             for key, prefix in [("novelty", "novelty_"), ("stock", "stock_")]
530             if all(prefix + f in frame for f in FLAVORS)}
```

Read this block:

- Line 524, `(hh, labels)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 525, `counts`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 526, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 527, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 528, `history`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 531–532:

```
531 if any(not np.isfinite(value).all() for value in history.values()):
532     raise ValueError("Nonfinite separate history inputs.")
```

Read this block:

- Line 531: test `any((not np.isfinite(value).all() for value in history.values()))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 532: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 533–533:

```
available = np.isfinite(prices) & (prices > 0)
```

Read this block:

- Line 533, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 534–535:

```
if not all(np.isfinite(x).all() for x in [counts, xi]):
    raise ValueError("Nonfinite estimation inputs.")
```

Read this block:

- Line 534: test `not all((np.isfinite(x).all() for x in [counts, xi]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 535: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 536–537:

```
if ((counts[:, :len(FLAVORS)] > 0) & ~available).any() or not available.any(axis=1).all():
    raise ValueError("Purchased flavors must have covered prices and a nonempty choice set.")
```

Read this block:

- Line 536: test `((counts[:, :len(FLAVORS)] > 0) & ~available).any() or not available.any(axis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 537: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `len`: Count elements/rows rather than summing their numerical values.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 539–542:

```
prices = np.where(available, prices, 0.)
cf_cols = ["cf_" + f for f in FLAVORS]
control = all(c in frame for c in cf_cols)
cf = frame[cf_cols].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 539, `prices`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 540, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 541, `control`: Set this name to the result of calling `all(...)`; the arguments and preceding method chain are shown above.

- Line 542, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 543–544:

```
543 if not np.isfinite(cf).all():
544     raise ValueError("Nonfinite price-control residuals.")
```

Read this block:

- Line 543: test `not np.isfinite(cf).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 544: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 545–557:

```
545 return dict(
546     counts=counts,
547     prices=prices,
548     available=available,
549     xi=xi,
550     **history,
551     cf=cf,
552     control=control,
553     hh=hh,
554     labels=labels,
555     n_households=len(labels),
556     outside=bool(capacity),
557 )
```

Read this block:

- Line 545: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

make draws

Original function: `make_draws`. Source begins at line 567.

Inputs are an even draw count, an integer seed, and an optional model name. Output has shape `(number,3)` for heterogeneous_history and `(number,2)` otherwise. `number % 2` is the remainder after division by two: zero for even numbers, one for odd. `number // 2` is integer division. The generator draws half the rows, then `vstack([z, -z])` appends their negatives. With two draws, an initial row `[.4,-1.2]` gives a second row `[-.4,1.2]`.

The seed makes repeated calls reproducible, and a local Generator avoids changing NumPy's global random state. Reusing these fixed antithetic nodes reduces noise when comparing candidate parameter vectors. Drawing afresh inside each objective call would make the optimizer chase random changes as well as parameter effects. More draws still cost more computation.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Split-model update. heterogeneous_history uses D-by-3 draws with alpha, novelty and own-stock coordinates; other models keep D-by-2 draws. The draw dimension and seed must match the saved fit when reconstructing likelihoods.

Definition and stated intent:

Source lines 567–568:

```
567 def make_draws(number, seed, model=None):
568     """Fixed antithetic normal draws, reused throughout optimization."""
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 569–570:

```
569 if number < 2 or number % 2:
570     raise ValueError("Use an even number of draws, at least two.")
```

Read this block:

- Line 569: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 570: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 571–571:

```
571 z = np.random.default_rng(seed).standard_normal((number // 2, 3 if model == "heterogeneous_history" else 2))
```

Read this block:

- Line 571, `z`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 572–572:

```
572 return np.vstack([z, -z])
```

Read this block:

- Line 572: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

parameter names

Original function: `parameter_names`. Source begins at line 576.

Creates the ordered parameter vector from all nonreference flavor names plus price and history distribution parameters. The five-group split fit adds beta_other_fruits and beta_desserts; order must agree with every utility and gradient array.

Definition and stated intent:

Source lines 576–576:

```
576 def parameter_names(model, outside, control=False):
```

`def` creates this callable; the parentheses name its inputs (`model`, `outside`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 577–578:

```
577 names = (["inside_constant"] if outside else []) + ["beta_" + f for f in FLAVORS[1:]]
578 names += ["mu_log_minus_alpha"]
```

Read this block:

- Line 577, `names`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 578: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 579–580:

```
579 if model in ("homogeneous", "heterogeneous"):
580     names += ["mean_gamma"]
```

Read this block:

- Line 579: test `model in ('homogeneous', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 580: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 581–582:

```
581 if model in ("mixed_naive", "heterogeneous"):
582     names += ["sd_log_minus_alpha"]
```

Read this block:

- Line 581: test `model in ('mixed_naive', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 582: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 583–584:

```
583 if model == "heterogeneous":
584     names += ["sd_gamma"]
```

Read this block:

- Line 583: test `model == 'heterogeneous'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 584: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 585–587:

```
585 if model == "mixture":
586     names += ["mean_gamma", "sd_log_minus_alpha", "sd_gamma",
587               "gamma_mean_gap", "sd_gamma_2", "mixture_weight"]
```

Read this block:

- Line 585: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 586: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 588–589:

```
588 if model in ("separate_history", "heterogeneous_history"):
589     names += ["mean_novelty", "mean_own_stock", "sd_log_minus_alpha"]
```

Read this block:

- Line 588: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 589: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 590–591:

```
590 if model == "heterogeneous_history":
591     names += ["sd_novelty", "sd_own_stock"]
```

Read this block:

- Line 590: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 591: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 592–593:

```
592 if control:
593     names += ["price_control"]
```

Read this block:

- Line 592: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 593: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 594–594:

```
594 return names
```

Read this block:

- Line 594: return the object currently named `names`; plain assignment does not copy it. This ends the call.

mixture components

Original function: `mixture_components`. Source begins at line 598.

Unpacks a parameter vector into two ordered normal-gamma components and a common price distribution. Adding a nonnegative gap orders component means and prevents label switching; it does not force either component to have positive gamma.

Definition and stated intent:

Source lines 598–599:

```
598 def mixture_components(theta, data):
599     """Ordered component means prevent label switching; alpha is shared."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 600–605:

```
600 names = parameter_names("mixture", data["outside"], data["control"])
601 p = dict(zip(names, theta))
602 normal_names = parameter_names("heterogeneous", data["outside"], data["control"])
603 first = np.array([p[n] for n in normal_names])
604 second = first.copy()
605 second[normal_names.index("mean_gamma")] += p["gamma_mean_gap"]
```

Read this block:

- Line 600, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 601, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 602, `normal_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 603, `first`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 604, `second`: Set this name to the result of calling `first.copy(...)`; the arguments and preceding method chain are shown above.
- Line 605: update `second[normal_names.index('mean_gamma')]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 606–606:

```
606 second[normal_names.index("sd_gamma")] = p["sd_gamma_2"]
```

Read this block:

- Line 606, `second[normal_names.index('sd_gamma')]`: Store the entry/slice selected by `p['sd_gamma_2']`; brackets select from the object to their left in the selected entry/column.

Source lines 607–607:

```
607 return names, normal_names, first, second, p["mixture_weight"]
```

Read this block:

- Line 607: return a tuple of 5 results. The caller can unpack them; return ends the call, unlike print.

mixture loglike score

Original function: `mixture_loglike_score`. Source begins at line 610.

Combines complete household likelihoods across persistent mixture components, using stable log-sum-exp. Mixing each week separately would change the model by allowing household membership to change over time. The score combines component derivatives with posterior membership weights.

Definition and stated intent:

Source lines 610–611:

```
610 def mixture_loglike_score(theta, data, draws, score=False):
611     """Mix whole household histories, never individual weeks."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `score`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 612–612:

```
612 names, normal_names, first, second, weight = mixture_components(theta, data)
```

Read this block:

- Line 612, `(names, normal_names, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 613–618:

```
613 if score:
614     ll1, g1 = demand_objective_gradient(first, data, draws, "heterogeneous", household=True)
615     ll2, g2 = demand_objective_gradient(second, data, draws, "heterogeneous", household=True)
616 else:
617     ll1 = household_loglike(first, data, draws, "heterogeneous")
618     ll2 = household_loglike(second, data, draws, "heterogeneous")
```

Read this block:

- Line 613: test `score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 614, `(ll1, g1)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 615, `(ll2, g2)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 617, `ll1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 618, `ll2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 619–620:

```
terms = np.array([np.log(weight) + ll1, np.log1p(-weight) + ll2])
ll = logsumexp(terms, axis=0)
```

Read this block:

- Line 619, `terms`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 620, `ll`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.

Source lines 621–622:

```
if not score:
    return ll
```

Read this block:

- Line 621: test `not score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 622: return the object currently named `ll`; plain assignment does not copy it. This ends the call.

Source lines 623–624:

```
responsibility = np.exp(terms - ll)
gradient = np.zeros((data["n_households"], len(names)))
```

Read this block:

- Line 623, `responsibility`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 624, `gradient`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 625–630:

```
for k, name in enumerate(normal_names):
    if name == "sd_gamma":
        gradient[:, names.index(name)] = responsibility[0] * g1[:, k]
        gradient[:, names.index("sd_gamma_2")] = responsibility[1] * g2[:, k]
    else:
        gradient[:, names.index(name)] = responsibility[0] * g1[:, k] + responsibility[1] * g2[:, k]
```

Read this block:

- Line 625: for each entry yielded by `enumerate(normal_names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 626: test `name == 'sd_gamma'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 627, `gradient[:, names.index(name)]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 628, `gradient[:, names.index('sd_gamma_2')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 630, `gradient[:, names.index(name)]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 631–632:

```
631 gradient[:, names.index("gamma_mean_gap")] = responsibility[1] * g2[:, normal_names.index("mean_gamma")]
632 gradient[:, names.index("mixture_weight")] = responsibility[0] / weight - responsibility[1] / (1 - weight)
```

Read this block:

- Line 631, `gradient[:, names.index('gamma_mean_gap')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 632, `gradient[:, names.index('mixture_weight')]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 633–633:

```
633 return -ll.mean(), -gradient.mean(axis=0)
```

Read this block:

- Line 633: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

history utility

Original function: `history_utility`. Source begins at line 637.

Chooses the history formula from the model name. Split models use b_N*N*+*b*_*S*S directly, with distinct draw coordinates for the two random coefficients. Combined-gamma models use gamma*Xi. The habit-weight parameter belongs to Xi and does not multiply the independently estimated split coefficients.

Definition and stated intent:

Source lines 637–637:

```
637 def history_utility(p, data, z, model):
```

`def` creates this callable; the parentheses name its inputs (`p`, `data`, `z`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 638–645:

```
638 if model in ("separate_history", "heterogeneous_history"):
639     novelty = p["mean_novelty"] + p.get("sd_novelty", 0.) * z[1]
640     own_stock = p["mean_own_stock"]
641     if model == "heterogeneous_history":
642         if len(z) < 3:
643             raise ValueError("heterogeneous_history requires three-dimensional draws.")
644         own_stock += p["sd_own_stock"] * z[2]
```

```
645     return novelty * data["novelty"] + own_stock * data["stock"]
```

Read this block:

- Line 638: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 639, `novelty`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 640, `own_stock`: Set this name to the entry/slice selected by `p['mean_own_stock']`; brackets select from the object to their left.
- Line 641: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 642: test `len(z) < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 643: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 644: update `own_stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 645: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `len`: Count elements/rows rather than summing their numerical values.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 646–646:

```
646 gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
```

Read this block:

- Line 646, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 647–647:

```
647 return gamma * data["xi"]
```

Read this block:

- Line 647: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

household loglike

Original function: `household_loglike`. Source begins at line 651.

Input `theta` is a one-dimensional parameter array, `data` is the dictionary from `make_arrays`, and `draws` is a draw-by-two matrix for combined models or draw-by-three for split heterogeneity. The return is one log likelihood per household, not a scalar. `dict(zip(...))` attaches coefficient names so the rest of the function need not remember their numeric positions.

For each latent-type draw, negative exponential constructs alpha and an affine normal transformation constructs gamma. Broadcasting makes beta and these scalar coefficients operate across every row/flavor. Unavailable utility is set to minus infinity; an optional zero outside column is appended. `logprob = utility - logsumexp(..., axis=1, keepdims=True)` normalizes each row. The retained column dimension makes subtraction broadcast across that row.

The count-weighted row sum uses `where(counts > 0, logprob, 0)` to avoid zero times minus infinity. `bincount` adds all weeks of the same household. Only after this aggregation does the final `logsumexp` integrate draws. This order implements persistent preferences. Averaging weekly probabilities first would be simpler computationally but would estimate a different preference process.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Current split utility. history_utility supplies b_NN+b_SS, with separate means and normal draw coordinates. Gamma*Xi is retained only for combined models. In a mixture model, the separate component helper mixes complete household likelihoods. Neither split coefficient receives the old fixed lambda weight.

Definition and stated intent:

Source lines 651–657:

```
651 def household_loglike(theta, data, draws, model):
652     """L_i = mean_r exp(sum_t sum_j count_itj * log P_itj,r).
653 
654     Gamma is normal and common across flavors; alpha = -exp(mu + sd*z). Beta never varies by draw.
655     Process one draw at a time to avoid a weeks x draws x products array.
656     The multinomial combinatorial constant is omitted (independent of theta).
657     """
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 658–659:

```
658 if model == "mixture":
659     return mixture_loglike_score(theta, data, draws)
```

Read this block:

- Line 658: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 659: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 660–666:

```
660 p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
661 beta = np.array([0.] + [p["beta_" + f] for f in FLAVORS[1:]])
662 beta += p.get("inside_constant", 0.0)
663 loglikes = []
664 active_draws = (
665     draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
666 )
```

Read this block:

- Line 660, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 661, `beta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 662: update `beta` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 663, `loglikes`: Set this name to the list `[]`.

- Line 664, `active_draws`: Set this name to the true-branch value if `model in ('mixed_naive', 'heteroge-neous', 'sepa...` holds, otherwise the else-branch value.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 667–685:

```
667 for z in active_draws:
668     alpha = -np.exp(
669         p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.0) * z[0]
670     )
671     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
672     utility = (
673         beta
674         + alpha * data["prices"]
675         + history_utility(p, data, z, model)
676         + p.get("price_control", 0.0) * data["cf"]
677     )
678     utility = np.where(data["available"], utility, -np.inf)
679     if data["outside"]:
680         utility = np.column_stack([utility, np.zeros(len(utility))])
681     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
682     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
683     loglikes.append(
684         np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"])
685     )
```

Read this block:

- Line 667: for each entry yielded by `active_draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 668, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 671, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 672, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 678, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 679: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 680, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 681, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 682, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

- Line 683: call `loglikes.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 686–686:

```
686 return logsumexp(loglikes, axis=0) - np.log(len(active_draws))
```

Read this block:

- Line 686: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

demand objective gradient

Original function: `demand_objective_gradient`. Source begins at line 690.

This returns a tuple `(mean_negative_log_likelihood, gradient)`, which SciPy requests when `jac=True`. The utility and likelihood steps mirror `household_loglike`; the additional work computes derivatives in the same draw loop. `total[:, None]` expands a weekly count vector to a column so it can multiply the matrix of choice probabilities. `residual` is observed count minus predicted count for the three inside alternatives.

The `columns` dictionary holds one weekly score vector per coefficient name. For example, the berry intercept takes column 1 of residual, while the alpha location score uses the price-weighted residual multiplied by alpha itself because differentiating `-exp(mu + sd*z)` contributes alpha. Each score is aggregated to households, and `column_stack` places parameters on a final axis.

`np.einsum("rh,rhp->p", weights, scores)` means multiply posterior weight at draw r/household h by its parameter-p score, then sum r and h, retaining p. A nested loop over r, h, p would express the same sum more verbosely and slowly. The minus sign and household-count division match minimization of average NLL. This is an analytic derivative of the finite-draw objective, not automatic differentiation or a finite-difference estimate.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 690–691:

```
690 def demand_objective_gradient(theta, data, draws, model, household=False):
691     """Exact simulated likelihood score, weighted by household type posteriors."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`, `household`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 692–693:

```
692 if model == "mixture":
693     return mixture_loglike_score(theta, data, draws, score=True)
```

Read this block:

- Line 692: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 693: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 694–698:

```
694 names = parameter_names(model, data["outside"], data["control"])
695 p = dict(zip(names, theta))
696 active = draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
697 likelihoods, scores = [], []
698 total = data["counts"].sum(axis=1)
```

Read this block:

- Line 694, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 695, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 696, `active`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.
- Line 697, `(likelihoods, scores)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 698, `total`: Set this name to the result of calling `data['counts'].sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 699–726:

```
699 for z in active:
700     alpha = -np.exp(p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.) * z[0])
701     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
702     utility = (np.array([0.] + [p["beta_" + f] for f in FLAVORS[1:]]) + p.get("inside_constant", 0.)
703                + alpha * data["prices"] + history_utility(p, data, z, model) + p.get("price_control", 0.) *
                    ↪  data["cf"])
704     utility = np.where(data["available"], utility, -np.inf)
705     if data["outside"]:
```

```
706         utility = np.column_stack([utility, np.zeros(len(utility))])
707     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
708     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
709     likelihoods.append(np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"]))
710     residual = data["counts"][:, :len(FLAVORS)] - total[:, None] * np.exp(logprob[:, :len(FLAVORS)])
711     price_score = (residual * data["prices"]).sum(axis=1) * alpha
712     columns = {"inside_constant": residual.sum(axis=1), "beta_berry": residual[:, 1],
713                "beta_plain": residual[:, 2], "mu_log_minus_alpha": price_score,
714                "sd_log_minus_alpha": price_score * z[0],
715                "price_control": (residual * data["cf"]).sum(axis=1)}
716     columns.update({"beta_" + f: residual[:, j] for j, f in enumerate(FLAVORS) if j})
717     columns["mean_gamma"] = (residual * data["xi"]).sum(axis=1)
718     columns["sd_gamma"] = columns["mean_gamma"] * z[1]
719     if model in ("separate_history", "heterogeneous_history"):
720         columns["mean_novelty"] = (residual * data["novelty"]).sum(axis=1)
721         columns["mean_own_stock"] = (residual * data["stock"]).sum(axis=1)
722         columns["sd_novelty"] = columns["mean_novelty"] * z[1]
723         if model == "heterogeneous_history":
724             columns["sd_own_stock"] = columns["mean_own_stock"] * z[2]
725     scores.append(np.column_stack([np.bincount(data["hh"], weights=columns[n],
726                                                minlength=data["n_households"]) for n in names]))
```

Read this block:

- Line 699: for each entry yielded by `active`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 700, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 701, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 702, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 704, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 705: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 706, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 707, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 708, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 709: call `likelihoods.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 710, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 711, `price_score`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 712, `columns`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `price_control`.
- Line 716: call `columns.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 717, `columns['mean_gamma']`: Store the result of calling `(residual * data['xi']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 718, `columns['sd_gamma']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 719: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 720, `columns['mean_novelty']`: Store the result of calling `(residual * data['novelty']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 721, `columns['mean_own_stock']`: Store the result of calling `(residual * data['stock']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 722, `columns['sd_novelty']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 723: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 724, `columns['sd_own_stock']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 725: call `scores.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `len`: Count elements/rows rather than summing their numerical values.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 727–733:

```
727 likelihoods = np.asarray(likelihoods)
728 normalizer = logsumexp(likelihoods, axis=0)
729 weights = np.exp(likelihoods - normalizer)
730 # weights is draws-by-households; scores is draws-by-households-by-parameters.
731 # Weight each draw's score, then sum over the draw axis.
732 weighted_scores = weights[:, :, None] * np.asarray(scores)
733 household_scores = weighted_scores.sum(axis=0)
```

Read this block:

- Line 727, `likelihoods`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 728, `normalizer`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 729, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 732, `weighted_scores`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 733, `household_scores`: Set this name to the result of calling `weighted_scores.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 734–735:

```
734 if household:
735     return normalizer - np.log(len(active)), household_scores
```

Read this block:

- Line 734: test `household`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 735: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Source lines 736–736:

```
736 gradient = -household_scores.mean(axis=0)
```

Read this block:

- Line 736, `gradient`: Set this name to the negated or sign-adjusted expression `-household_scores.mean(axis=0)`.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 737–737:

```
737 return -(normalizer - np.log(len(active))).mean(), gradient
```

Read this block:

- Line 737: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

optimize demand

Original function: `optimize_demand`. Source begins at line 747.

This sets up and runs multiple bounded local searches and returns the selected fit, every starting solution, and the bounds. Initial SDs are .3 because zero can be stationary under symmetric integration. Bounds are pairs `(lower, upper)` in the same order as parameter names. The optional initial vector is copied before perturbation, preventing caller-owned coefficients from being changed.

Each start uses `minimize(..., method="L-BFGS-B", jac=True)`. The callback reports progress every ten iterations, while completed starts are saved to JSON. `np.clip` keeps perturbed starts feasible. `min(results, key=lambda r: r.fun)` uses each result's loss as its comparison key; it selects the smallest loss, not necessarily a converged fit. An alternative is to select only converged results, but that is not the rule this code implements. Multistart search probes local solutions and does not establish a global optimum.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 747–747:

```
747 def optimize_demand(data, settings, initial=None, number_starts=None, quiet=False):
```

`def` creates this callable; the parentheses name its inputs (`data`, `settings`, `initial`, `number_starts`, `quiet`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 748–751:

```
748 draws = make_draws(settings.draws, settings.seed, settings.model)
749 names = parameter_names(settings.model, data["outside"], data["control"])
750 start = np.zeros(len(names))
751 bounds = []
```

Read this block:

- Line 748, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 749, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 750, `start`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 751, `bounds`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 752–765:

```
752 for k, name in enumerate(names):
753     if name.startswith("sd_"):
754         start[k] = 0.3  # Zero SD is a stationary point: start away from zero.
755         bounds.append((0.0, 3.0))
756     elif name == "gamma_mean_gap":
757         start[k] = 0.4
758         bounds.append((0.0, 40.0))
759     elif name == "mixture_weight":
760         start[k] = 0.5
761         bounds.append((0.001, 0.999))
762     elif name == "mu_log_minus_alpha":
763         bounds.append((-6.0, 3.0))
764     else:
765         bounds.append((-20.0, 20.0))
```

Read this block:

- Line 752: for each entry yielded by `enumerate(names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 753: test `name.startswith('sd_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 754, `start[k]`: Store the literal value `0.3` in the selected entry/column.
- Line 755: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 756: test `name == 'gamma_mean_gap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 757, `start[k]`: Store the literal value `0.4` in the selected entry/column.
- Line 758: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 759: test `name == 'mixture_weight'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 760, `start[k]`: Store the literal value `0.5` in the selected entry/column.
- Line 761: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 762: test `name == 'mu_log_minus_alpha'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 763: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 765: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 766–767:

```
766 if settings.model == "mixture":
767     start[names.index("mean_gamma")] = -0.2
```

Read this block:

- Line 766: test `settings.model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 767, `start[names.index('mean_gamma')]`: Store the negated or sign-adjusted expression `-0.2` in the selected entry/column.

Source lines 768–769:

```
768 if initial is not None:
769     start = np.asarray(initial).copy()
```

Read this block:

- Line 768: test `initial is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 769, `start`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 770–771:

```
770 rng = np.random.default_rng(settings.seed + 1)
771 results = []
```

Read this block:

- Line 770, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 771, `results`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 773–774:

```
773 def objective(theta):
774     return demand_objective_gradient(theta, data, draws, settings.model)
```

Read this block:

- Line 773: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 776–810:

```
776 for s in range(settings.starts if number_starts is None else number_starts):
777     initial = start.copy()
778     if s:
779         initial += rng.normal(0, 0.5, len(start))
780         initial = np.clip(initial, np.array(bounds)[:, 0], np.array(bounds)[:, 1])
781     began = time.monotonic()
782     iterations = [0]
783 
784     def progress(theta):
785         iterations[0] += 1
786         if not quiet and iterations[0] % 10 == 0:
787             print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
788 
789     fit = minimize(
790         objective,
791         initial,
792         method="L-BFGS-B",
793         jac=True,
794         callback=progress,
795         bounds=bounds,
796         options={"maxiter": settings.maxiter, "ftol": 1e-10, "gtol": 1e-6},
797     )
798     results.append(fit)
799     if not quiet:
800         settings.output.mkdir(parents=True, exist_ok=True)
801         (settings.output / "starts_checkpoint.json").write_text(json.dumps([
802             dict(theta=r.x.tolist(), nll=float(r.fun * data["n_households"]),
803                  converged=bool(r.success), iterations=int(r.nit), evaluations=int(r.nfev))
804             for r in results], indent=2))
805     if not quiet:
806         print(
807             f"Start {s + 1}: NLL={fit.fun * data['n_households']:.4f}; "
808             f"converged={fit.success}",
809             flush=True,
810         )
```

Read this block:

- Line 776: for each entry yielded by `range(settings.starts if number_starts is None else number_sta...`, bind/unpack it into `s` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 777, `initial`: Set this name to the result of calling `start.copy(...)`; the arguments and preceding method chain are shown above.
- Line 778: test `s`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 779: update `initial` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 780, `initial`: Set this name to the result of calling `np.clip(...)`; the arguments and preceding method chain are shown above.
- Line 781, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 782, `iterations`: Set this name to the list `[0]`.
- Line 784: define nested callable `progress`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 789, `fit`: Set this name to the result of calling `minimize(...)`; the arguments and preceding method chain are shown above.
- Line 798: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 799: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 800: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 801: call `(settings.output / 'starts_checkpoint.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 805: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 806: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `minimize`: Run iterative numerical minimization; jac=True expects the objective to return value plus gradient.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 812–812:

```
812 best = min(results, key=lambda r: r.fun)
```

Read this block:

- Line 812, `best`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 813–813:

```
813 return best, results, bounds
```

Read this block:

- Line 813: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

optimize demand.objective

Original function: `optimize_demand.objective`. Source begins at line 773.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 773–773:

```
773 def objective(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 774–774:

```
774 return demand_objective_gradient(theta, data, draws, settings.model)
```

Read this block:

- Line 774: return the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimize demand.progress

Original function: `optimize_demand.progress`. Source begins at line 784.

SciPy calls this nested callback after iterations, passing the current vector. The body increments `iterations[0]`, checks a remainder modulo ten, and prints elapsed time unless quiet mode is active. The callback does not return an objective value and does not choose the next parameter vector.

The enclosing variable is a one-element list so the nested function can mutate its element without rebinding the outer name. An ordinary integer counter with assignment would require `nonlocal iterations`. That alternative is often clearer; the existing list technique works because objects can be mutated through a captured reference. `time.monotonic` measures elapsed duration without wall-clock adjustments changing the result.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 784–784:

```
784 def progress(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 785–785:

```
785 iterations[0] += 1
```

Read this block:

- Line 785: update `iterations[0]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 786–787:

```
786 if not quiet and iterations[0] % 10 == 0:
787     print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
```

Read this block:

- Line 786: test `not quiet and iterations[0] % 10 == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 787: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

resample households

Original function: `resample_households`. Source begins at line 823.

Input is the weekly frame and a NumPy random Generator. Output is a concatenated bootstrap sample with the same number of household draws as the original. The comprehension stores whole household subtables. `rng.integers(0, H, size=H)` draws indices with replacement; its upper endpoint H is excluded.

`enumerate(selected)` supplies a fresh ID and selected old-group position. `.copy()` is essential before changing that ID so the original history is not modified. If selection is `[2,2,0]`, two copies of old household 2 get new IDs 0 and 1, keeping their likelihoods separate. Concatenating with `ignore_index=True` gives clean row labels. Resampling individual weeks would break persistent household preferences and within-household dependence.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 823–828:

```
823 def resample_households(frame, rng):
824     """Keep whole histories; repeated households get distinct bootstrap IDs.
825 
826     Stocks/Xi stay attached to the original histories. Rebuilding stocks after
827     dropping incomplete price weeks would incorrectly erase observed purchases.
828     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `rng`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 829–831:

```
829 groups = [group for _, group in frame.groupby("household_code", sort=True)]
830 selected = rng.integers(0, len(groups), size=len(groups))
831 samples = []
```

Read this block:

- Line 829, `groups`: Set this name to a list computed by the displayed for-expression.
- Line 830, `selected`: Set this name to the result of calling `rng.integers(...)`; the arguments and preceding method chain are shown above.
- Line 831, `samples`: Set this name to the list `[]`.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.integers`: Draw integer positions with replacement by default; the high endpoint is excluded.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 832–835:

```
832 for new_id, old_id in enumerate(selected):
833     group = groups[old_id].copy()
834     group["household_code"] = new_id
835     samples.append(group)
```

Read this block:

- Line 832: for each entry yielded by `enumerate(selected)`, bind/unpack it into `(new_id, old_id)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 833, `group`: Set this name to the result of calling `groups[old_id].copy(...)`; the arguments and preceding method chain are shown above.
- Line 834, `group['household_code']`: Store the object currently named `new_id`; plain assignment does not copy it in the selected entry/column.
- Line 835: call `samples.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 836–836:

```
836 return pd.concat(samples, ignore_index=True)
```

Read this block:

- Line 836: return the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above. This ends the call.

bootstrap standard errors

Original function: `bootstrap_standard_errors`. Source begins at line 842.

The `estimate` argument is a callable, not an estimate array. Each repetition resamples households, refits enabled price controls, calls `estimate(sample, initial)`, and records its result. `record.update` adds named fields to an existing dictionary. Accepted vectors go to `values`; all attempts go to `records`, so failure does not disappear from the audit trail.

The `try/except` catches expected data/linear-algebra problems. Every attempt checkpoints the records as CSV. After the loop, the threshold requires at least 20 usable fits and at least 80% success. Returning `(None, report)` explicitly signals inference was withheld. Otherwise, `std(ddof=1)` uses sample rather than population SD, `quantile` forms empirical percentile intervals, and `np.cov(..., rowvar=False)` treats columns as parameters. Analytic

Hessian SEs would be cheaper but would require handling the estimated first-stage residuals and household dependence; this implementation uses a two-stage cluster bootstrap.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 842–848:

```
842 def bootstrap_standard_errors(frame, settings, estimate, initial, names):
843     """Refit price controls AND demand in every household-cluster replication.
844 
845     estimate(frame, initial) returns a scipy optimizer result. All replications
846     use the same integration draws as the main fit. Retail instruments stay
847     fixed: this is inference conditional on the observed retail price universe.
848     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `settings`, `estimate`, `initial`, `names`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 849–851:

```
849 rng = np.random.default_rng(settings.seed + 10000)
850 records = []
851 values = []
```

Read this block:

- Line 849, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 850, `records`: Set this name to the list `[]`.
- Line 851, `values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 852–871:

```
852 for replication in range(settings.bootstrap_reps):
853     record = {"replication": replication + 1, "converged": False}
854     try:
855         sample = resample_households(frame, rng)
856         if not settings.no_price_control:
857             sample, _ = add_price_controls(sample)
858         fit = estimate(sample, initial)
859         record.update(converged=bool(fit.success), objective=float(fit.fun),
860                       message=str(fit.message))
861         record.update(dict(zip(names, fit.x)))
862         if fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun):
863             values.append(fit.x)
864     except (ValueError, np.linalg.LinAlgError) as error:
865         # An unidentifiable resample is recorded, never silently accepted.
866         record["message"] = str(error)
867     records.append(record)
868     # Checkpoint after each replication in case a long cluster job ends.
869     pd.DataFrame(records).to_csv(settings.output / "bootstrap_draws.csv", index=False)
870     print(f"Bootstrap {replication+1}/{settings.bootstrap_reps}: "
871           f"{len(values)} usable fits", flush=True)
```

Read this block:

- Line 852: for each entry yielded by `range(settings.bootstrap_reps)`, bind/unpack it into `replication` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 853, `record`: Set this name to a dictionary with named entries `replication`, `converged`.
- Line 854: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 855, `sample`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 856: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 857, `(sample, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 858, `fit`: Set this name to the result of calling `estimate(...)`; the arguments and preceding method chain are shown above.
- Line 859: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 861: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 862: test `fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 863: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 864: catch `(ValueError, np.linalg.LinAlgError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 866, `record['message']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 867: call `records.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 869: call `pd.DataFrame(records).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 870: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `estimate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `str`: Convert to text for paths, messages, or serialization.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 872–876:

```
valid = len(values)
report = dict(method=("household bootstrap" if settings.no_price_control
                      else "household bootstrap, refitting both stages"),
              requested=settings.bootstrap_reps, usable=valid,
              conditional_on="retail instruments and fixed integration draws")
```

Read this block:

- Line 872, `valid`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 873, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 878–880:

```
if valid < max(20, int(np.ceil(.8 * settings.bootstrap_reps))):
    report["status"] = "insufficient converged replications; no SEs reported"
    return None, report
```

Read this block:

- Line 878: test `valid < max(20, int(np.ceil(0.8 * settings.bootstrap_reps)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 879, `report['status']`: Store the literal text `'insufficient converged replications; no SEs reported'` in the selected entry/column.
- Line 880: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.ceil`: Round upward, so the minimum usable-bootstrap threshold meets the requested success fraction.

Source lines 881–888:

```
estimates = np.asarray(values)
table = pd.DataFrame({"parameter": names,
                      "std_error": estimates.std(axis=0, ddof=1),
                      "ci_lower": np.quantile(estimates, .025, axis=0),
                      "ci_upper": np.quantile(estimates, .975, axis=0)})
covariance = pd.DataFrame(np.cov(estimates, rowvar=False), index=names, columns=names)
covariance.to_csv(settings.output / "covariance.csv")
report["status"] = "computed; percentile 95% intervals"
```

Read this block:

- Line 881, `estimates`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 882, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 886, `covariance`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 887: call `covariance.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 888, `report['status']`: Store the literal text `'computed; percentile 95% intervals'` in the selected entry/column.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `.cov`: Compute covariance; rowvar=False means columns represent parameters rather than treating rows as variables.

Source lines 889–889:

```
889 return table, report
```

Read this block:

- Line 889: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

observed covariance

Original function: `observed_covariance`. Source begins at line 893.

Central-differences the analytic mean-household NLL gradient to form observed information, checks bounds and eigenvalues, and divides its inverse by the household count. Missing household scaling would give incorrect standard errors. Nonpositive or ill-conditioned information is rejected instead of inverted blindly.

Definition and stated intent:

Source lines 893–894:

```
893 def observed_covariance(theta, gradient, households, bounds):
894     """Differentiate the mean negative-log-likelihood gradient, then scale by I."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 895–897:

```
895 theta = np.asarray(theta, dtype=float)
896 steps = np.cbrt(np.finfo(float).eps) * np.maximum(1., np.abs(theta))
897 lower, upper = np.asarray(bounds, dtype=float).T
```

Read this block:

- Line 895, `theta`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 896, `steps`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 897, `(lower, upper)`: Unpack the attribute `np.asarray(bounds, dtype=float).T` of the object before the dot into these targets in order.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 898–899:

```
898 if np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 1e-4)):
899     raise ValueError("parameter at or near bounds; normal inference withheld")
```

Read this block:

- Line 898: test `np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 0.0001))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 899: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 900–900:

```
900 hessian = np.empty((len(theta), len(theta)))
```

Read this block:

- Line 900, `hessian`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 901–904:

```
901 for j, step in enumerate(steps):
902     delta = np.zeros(len(theta))
903     delta[j] = step
904     hessian[:, j] = (gradient(theta + delta) - gradient(theta - delta)) / (2 * step)
```

Read this block:

- Line 901: for each entry yielded by `enumerate(steps)`, bind/unpack it into `(j, step)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 902, `delta`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 903, `delta[j]`: Store the object currently named `step`; plain assignment does not copy it in the selected entry/column.
- Line 904, `hessian[:, j]`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 905–905:

```
905 hessian = (hessian + hessian.T) / 2
```

Read this block:

- Line 905, `hessian`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 906–907:

```
906 if not np.isfinite(hessian).all():
907     raise ValueError("nonfinite observed information; SEs withheld")
```

Read this block:

- Line 906: test `not np.isfinite(hessian).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 907: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 908–908:

```
908 eigenvalues = np.linalg.eigvalsh(hessian)
```

Read this block:

- Line 908, `eigenvalues`: Set this name to the result of calling `np.linalg.eigvalsh(...)`; the arguments and preceding method chain are shown above.

Source lines 909–910:

```
909 if eigenvalues[0] <= max(0., eigenvalues[-1] * 1e-10):
910     raise ValueError("observed information not positive definite or ill-conditioned; SEs withheld")
```

Read this block:

- Line 909: test `eigenvalues[0] <= max(0.0, eigenvalues[-1] * 1e-10)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 910: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 911–911:

```
911 return np.linalg.solve(hessian, np.eye(len(theta))) / households
```

Read this block:

- Line 911: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

hessian standard errors

Original function: `hessian_standard_errors`. Source begins at line 916.

Converts an accepted observed-information covariance into parameter SEs and normal intervals, writes the covariance, and returns inference metadata. These are conditional model-based SEs; they do not include price-first-stage uncertainty or provide a robust sandwich covariance.

Definition and stated intent:

Source lines 916–916:

```
916 def hessian_standard_errors(theta, gradient, households, bounds, names, output):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`, `names`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 917–919:

```
917 report = dict(method="observed-information Hessian",
918               conditional_on="estimated price controls, retail instruments, and fixed integration draws",
919               first_stage_uncertainty="not included", robust=False)
```

Read this block:

- Line 917, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 920–924:

```
920 try:
921     covariance = observed_covariance(theta, gradient, households, bounds)
922 except ValueError as error:
923     report["status"] = str(error)
924     return None, report
```

Read this block:

- Line 920: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 921, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 922: catch `ValueError` from the preceding try; the optional as-name holds the error object for logging.
- Line 923, `report['status']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 924: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 925–930:

```
925 pd.DataFrame(covariance, index=names, columns=names).to_csv(output / "covariance.csv")
926 se = np.sqrt(np.diag(covariance))
927 critical = norm.ppf(.975)
928 table = pd.DataFrame(dict(parameter=names, std_error=se,
929                            ci_lower=theta - critical * se, ci_upper=theta + critical * se))
930 report["status"] = "computed; normal 95% intervals; first-stage uncertainty omitted"
```

Read this block:

- Line 925: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 926, `se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 927, `critical`: Set this name to the result of calling `norm.ppf(...)`; the arguments and preceding method chain are shown above.
- Line 928, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 930, `report['status']`: Store the literal text `'computed; normal 95% intervals; first-stage uncertainty omitted'` in the selected entry/column.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 931–931:

```
931 return table, report
```

Read this block:

- Line 931: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

implied wtp

Original function: `implied_wtp`. Source begins at line 941.

Computes population moments of coefficient/positive price sensitivity. Independence permits multiplying the coefficient moments by inverse-lognormal moments. Averaging these household ratios differs from dividing a mean coefficient by mean price sensitivity. Split history rows use their own coefficients without lambda weights.

Definition and stated intent:

Source lines 941–942:

```
941 def implied_wtp(estimates, habit_weight=.7):
942     """Return population moments, not sampling standard errors, in price units."""
```

`def` creates this callable; the parentheses name its inputs (`estimates`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 943–947:

```
943 p = dict(zip(estimates.parameter, estimates.estimate))
944 mu, sigma = p['mu_log_minus_alpha'], p.get('sd_log_minus_alpha', 0.)
945 inverse_mean = np.exp(-mu + sigma**2 / 2)
946 inverse_second = np.exp(-2 * mu + 2 * sigma**2)
947 rows = []
```

Read this block:

- Line 943, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 944, `(mu, sigma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 945, `inverse_mean`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 946, `inverse_second`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 947, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 949–953:

```
949 def add(term, mean, second, interpretation):
950     wtp_mean = mean * inverse_mean
951     variance = max(0., second * inverse_second - wtp_mean**2)
952     rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
953                      interpretation=interpretation))
```

Read this block:

- Line 949: define nested callable `add`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 955–959:

```
for name in ['inside_constant'] + ['beta_' + f for f in FLAVORS[1:]]:
    if name in p:
        interpretation = ('Other flavor vs outside, at zero history and control residual'
                          if name == 'inside_constant' else 'Flavor premium relative to other')
        add(name, p[name], p[name]**2, interpretation)
```

Read this block:

- Line 955: for each entry yielded by `['inside_constant'] + ['beta_' + f for f in FLAVORS[1:]]`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 956: test `name in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 957, `interpretation`: Set this name to the true-branch value if `name == 'inside_constant'` holds, otherwise the else-branch value.
- Line 959: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 960–974:

```
if 'mean_gamma' in p:
    mean, sd = p['mean_gamma'], p.get('sd_gamma', 0.)
    second = mean**2 + sd**2
    if 'mixture_weight' in p:
        weight = p['mixture_weight']
        mean2 = mean + p['gamma_mean_gap']
        second2 = mean2**2 + p['sd_gamma_2']**2
        add('gamma_component_1', mean, second, 'Per unit Xi, conditional on component 1')
        add('gamma_component_2', mean2, second2, 'Per unit Xi, conditional on component 2')
        mean, second = weight * mean + (1 - weight) * mean2, weight * second + (1 - weight) * second2
    add('gamma', mean, second, 'Per unit increase in history index Xi')
    add('gamma_novelty', (1 - habit_weight) * mean, (1 - habit_weight)**2 * second,
        'Per unit novelty N, holding own-flavor stock fixed')
    add('gamma_own_stock', -habit_weight * mean, habit_weight**2 * second,
        'Per unit own-flavor stock S, holding novelty fixed')
```

Read this block:

- Line 960: test `'mean_gamma' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 961, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 962, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 963: test `'mixture_weight' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 964, `weight`: Set this name to the entry/slice selected by `p['mixture_weight']`; brackets select from the object to their left.
- Line 965, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 966, `second2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 967: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 968: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 969, `(mean, second)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 970: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 971: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 973: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 975–979:

```
975 for term in ["novelty", "own_stock"]:
976     if "mean_" + term in p:
977         mean, sd = p["mean_" + term], p.get("sd_" + term, 0.)
978         add(term, mean, mean**2 + sd**2,
979             "Per unit " + term + ", holding the other history measure fixed")
```

Read this block:

- Line 975: for each entry yielded by `['novelty', 'own_stock']`, bind/unpack it into `term` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 976: test `'mean_' + term in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 977, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 978: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 980–980:

```
980 return pd.DataFrame(rows)
```

Read this block:

- Line 980: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

implied wtp.add

Original function: `implied_wtp.add`. Source begins at line 949.

This nested helper adds one named WTP row from coefficient first and second moments. It uses variables captured from the enclosing WTP call, converts coefficient moments into ratio moments, and appends to the enclosing output list. Its side effect is a new row; it does not refit coefficients.

Definition and stated intent:

Source lines 949–949:

```
949 def add(term, mean, second, interpretation):
```

`def` creates this callable; the parentheses name its inputs (`term`, `mean`, `second`, `interpretation`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 950–953:

```
950 wtp_mean = mean * inverse_mean
951 variance = max(0., second * inverse_second - wtp_mean**2)
952 rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
953                 interpretation=interpretation))
```

Read this block:

- Line 950, `wtp_mean`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 951, `variance`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 952: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot gamma distribution

Original function: `plot_gamma_distribution`. Source begins at line 982.

Input is an estimate table, output directory, and convergence flag. This is a side-effect function: it saves a figure and implicitly returns `None`. `set_index("parameter")["estimate"]` creates a name-indexed Series for coefficient lookup. The early bare `return` skips naive models with no gamma.

For positive SD, `linspace` makes a smooth horizontal grid, `norm.pdf` computes density, and `fill_between(..., where=x < 0)` shades the negative region. For zero SD a continuous density would divide by zero or misrepresent a point mass, so the `else` branch draws a vertical line. `norm.cdf(0, ...)` computes the implied population habit share. `Agg` supports file rendering without a GUI; `plt.close(fig)` releases the figure. This is a distribution of latent types, not an uncertainty interval for the mean.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 982–983:

```
982 def plot_gamma_distribution(table, output, converged=True):
983     """Plot the fitted population distribution, not uncertainty in its mean."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 984–984:

```
984 estimates = table.set_index("parameter")["estimate"]
```

Read this block:

- Line 984, `estimates`: Set this name to the entry/slice selected by `table.set_index('parameter')['estimate']`; brackets select from the object to their left.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 985–986:

```
985 if "mean_gamma" not in estimates:
986     return  # Naive models have no gamma parameter.
```

Read this block:

- Line 985: test `'mean_gamma' not in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 986: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 987–993:

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.stats import norm

mean = float(estimates["mean_gamma"])
sd = float(estimates.get("sd_gamma", 0.))
```

Read this block:

- Line 987: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 988: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 989: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 990: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 992, `mean`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 993, `sd`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 994–994:

```
fig, ax = plt.subplots(figsize=(7, 5))
```

Read this block:

- Line 994, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 995–1030:

```
if "mixture_weight" in estimates:
    weight = float(estimates["mixture_weight"])
    mean2 = mean + float(estimates["gamma_mean_gap"])
    sd2 = float(estimates["sd_gamma_2"])
    x = np.linspace(min(mean - 4 * sd, mean2 - 4 * sd2) - .01,
                    max(mean + 4 * sd, mean2 + 4 * sd2) + .01, 1001)
    density = np.zeros_like(x)
    negative = 0.
    for mu, sigma, share in [(mean, sd, weight), (mean2, sd2, 1 - weight)]:
        if sigma > 0:
            component = share * norm.pdf(x, loc=mu, scale=sigma)
            density += component
            ax.plot(x, component, linestyle=":", label=f"Component (weight {share:.1%})")
            negative += share * norm.cdf(0, loc=mu, scale=sigma)
```

```
        else:
            ax.axvline(mu, linestyle=":", label=f"Point mass (weight {share:.1%})")
            negative += share * (mu < 0)
    ax.plot(x, density, color="steelblue", label="Mixture continuous density")
    ax.set_ylabel("Population density")
    ax.legend()
    description = f"Component means = {mean:.4g}, {mean2:.4g}; P(gamma < 0) = {negative:.1%}"
elif sd > 0:
    x = np.linspace(mean - 4 * sd, mean + 4 * sd, 801)
    density = norm.pdf(x, loc=mean, scale=sd)
    ax.plot(x, density, color="steelblue")
    ax.fill_between(x, density, where=x < 0, color="steelblue", alpha=.25)
    ax.set_ylabel("Population density")
    negative = norm.cdf(0, loc=mean, scale=sd)
    description = f"Mean = {mean:.4g}; SD = {sd:.4g}; P(gamma < 0) = {negative:.1%}"
else:
    ax.axvline(mean, color="steelblue", linewidth=2, label="Point mass (SD = 0)")
    ax.set_xlim(mean - 1, mean + 1)
    ax.set_yticks([])
    ax.set_ylabel("Point mass")
    ax.legend()
    description = f"All households: gamma = {mean:.4g} (SD = 0)"
```

Read this block:

- Line 995: test `'mixture_weight' in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 996, `weight`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 997, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 998, `sd2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 999, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 1001, `density`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.
- Line 1002, `negative`: Set this name to the literal value `0.0`.
- Line 1003: for each entry yielded by `[(mean, sd, weight), (mean2, sd2, 1 - weight)]`, bind/unpack it into `(mu, sigma, share)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1004: test `sigma > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1005, `component`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 1006: update `density` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1007: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1008: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1010: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1011: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1012: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1013: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 1014: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1015, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 1016: test `sd > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1017, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 1018, `density`: Set this name to the result of calling `norm.pdf(...)`; the arguments and preceding method chain are shown above.
- Line 1019: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1020: call `ax.fill_between` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1021: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1022, `negative`: Set this name to the result of calling `norm.cdf(...)`; the arguments and preceding method chain are shown above.
- Line 1023, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 1025: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1026: call `ax.set_xlim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1027: call `ax.set_yticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1028: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1029: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1030, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.pdf`: Evaluate a probability density (here a normal distribution), not a PDF document file.
- `.fill_between`: Shade the area between curves/zero, optionally only where a Boolean condition is true.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set_xlim`: Set the visible x-axis limits; this changes presentation rather than the calculated results.
- `.set_yticks`: Choose which vertical tick marks to show, including an empty list for a point-mass display.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 1031–1032:

```
1031 ax.axvline(0, color="gray", linestyle="--", linewidth=1)
1032 title = "Fitted population gamma distribution"
```

Read this block:

- Line 1031: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1032, `title`: Set this name to the literal text `'Fitted population gamma distribution'`.

Source lines 1033–1034:

```
1033 if not converged or table["at_bound"].any():
1034     title += " (provisional fit)"
```

Read this block:

- Line 1033: test `not converged or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1034: update `title` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 1035–1039:

```
1035 ax.set(xlabel="Gamma (common across flavors)", title=title)
1036 fig.text(.5, .02, description, ha="center")
1037 fig.tight_layout(rect=(0, .06, 1, 1))
1038 fig.savefig(output / "gamma_distribution.pdf")
1039 plt.close(fig)
```

Read this block:

- Line 1035: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1036: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1037: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1038: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1039: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

export history distributions

Original function: `export_history_distributions`. Source begins at line 1041.

Inputs and outputs. The coefficient table supplies mean_novelty, mean_own_stock and their SDs; output is a directory. For each marginal, the function writes a summary of mean, SD, sign share and quantiles, a grid of densities/CDFs, and PDF/PNG plots. These describe population heterogeneity, not confidence intervals or household posterior means.

`NormalDist(mean, sd)` represents a fitted normal distribution; `.cdf(0)` gives its share below zero and `.inv_cdf(.25)` gives its lower quartile. A zero SD is handled as a point mass instead of dividing by zero. A negative own-stock coefficient means satiation, while a positive one means reinforcement. Convergence/boundary status marks

provisional exports. This calculation recovers the already estimated distribution without optimizing demand again.

Definition and stated intent:

Source lines 1041–1041:

```
1041 def export_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1042–1042:

```
1042 p = table.set_index('parameter').estimate
```

Read this block:

- Line 1042, `p`: Set this name to the attribute `table.set_index('parameter').estimate` of the object before the dot.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 1043–1044:

```
1043 if 'mean_novelty' not in p:
1044     return
```

Read this block:

- Line 1043: test `'mean_novelty' not in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1044: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 1045–1050:

```
1045 output = Path(output)
1046 output.mkdir(parents=True, exist_ok=True)
1047 provisional = not converged or table.get('at_bound', pd.Series(False)).astype(str).str.lower().eq('true').any()
1048 summaries, curves = [], []
1049 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
1050 import matplotlib
```

Read this block:

- Line 1045, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 1046: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1047, `provisional`: Set this name to the Boolean condition `not converged or table.get('at_bound', pd.Series(False)).astyp...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 1048, `(summaries, curves)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 1049: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1050: import `matplotlib`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 1051–1053:

```
1051 matplotlib.use('Agg')
1052 import matplotlib.pyplot as plt
1053 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

Read this block:

- Line 1051: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1052: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 1053, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 1054–1074:

```
1054 for ax, term, label in zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'Own-stock habit']):
1055     mean, sd = float(p['mean_'+term]), float(p.get('sd_'+term, 0.))
1056     if not np.isfinite([mean, sd]).all() or sd < 0:
1057         raise ValueError('Invalid distribution parameters')
1058     normal = NormalDist(mean, sd) if sd > 0 else None
1059     row = dict(coefficient=term, mean=mean, sd=sd, point_mass=sd == 0,
1060                share_below_zero=normal.cdf(0) if normal else float(mean < 0), provisional=bool(provisional))
1061     for q in [.01, .05, .25, .5, .75, .95, .99]:
1062         row[f'q{int(100*q):02d}'] = normal.inv_cdf(q) if normal else mean
1063     summaries.append(row)
1064     if normal:
1065         x = np.linspace(mean-4*sd, mean+4*sd, 801)
1066         density = np.exp(-.5*((x-mean)/sd)**2)/(sd*np.sqrt(2*np.pi))
1067         ax.plot(x, density)
1068         curves.extend(dict(coefficient=term, value=float(v), density=float(d), cdf=normal.cdf(float(v)),
     ↪ point_mass=False)
1069                       for v, d in zip(x, density))
1070     else:
1071         ax.axvline(mean, label='Point mass')
1072         curves.append(dict(coefficient=term, value=mean, density=np.nan, cdf=1., point_mass=True))
1073     ax.axvline(0, color='gray', ls='--')
1074     ax.set(title=f'{label}\nMean={mean:.4g}; SD={sd:.4g}', xlabel='Utility coefficient', ylabel='Population
     ↪ density')
```

Read this block:

- Line 1054: for each entry yielded by `zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'O...`, bind/unpack it into `(ax, term, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1055, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 1056: test `not np.isfinite([mean, sd]).all() or sd < 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 1057: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 1058, `normal`: Set this name to the true-branch value if `sd > 0` holds, otherwise the else-branch value.
- Line 1059, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 1061: for each entry yielded by `[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]`, bind/unpack it into `q` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1062, `row[f'q{int(100 * q):02d}']`: Store the true-branch value if `normal` holds, otherwise the else-branch value in the selected entry/column.
- Line 1063: call `summaries.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1064: test `normal`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1065, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 1066, `density`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 1067: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1068: call `curves.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1071: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1072: call `curves.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1073: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1074: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `NormalDist`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored

coefficients.

Source lines 1075–1078:

```
pd.DataFrame(summaries).to_csv(output/'history_distribution_summary.csv', index=False)
pd.DataFrame(curves).to_csv(output/'history_distribution_grid.csv', index=False)
fig.suptitle('Fitted population history distributions'+(' (provisional)' if provisional else ''))
fig.tight_layout()
```

Read this block:

- Line 1075: call `pd.DataFrame(summaries).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1076: call `pd.DataFrame(curves).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1077: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1078: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 1079–1080:

```
for extension in ['pdf', 'png']:
    fig.savefig(output/f'history_distributions.{extension}')
```

Read this block:

- Line 1079: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1080: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 1081–1081:

```
plt.close(fig)
```

Read this block:

- Line 1081: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

plot history distributions

Original function: `plot_history_distributions`. Source begins at line 1084.

Delegates to the population-distribution export routine immediately after a split fit. Its side effects are tables and plots, not new coefficient estimates. The readable estimator keeps the full export calculation in the same file; the original estimator imports that helper.

Definition and stated intent:

Source lines 1084–1084:

```
1084 def plot_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1085–1085:

```
1085 export_history_distributions(table, output, converged)
```

Read this block:

- Line 1085: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

write run notes

Original function: `write_run_notes`. Source begins at line 1092.

Writes a readable text rendering of the same automatic run information saved as JSON. pformat lays out nested records for reading; this is an output record, not an alternative settings file.

Definition and stated intent:

Source lines 1092–1093:

```
1092 def write_run_notes(output, information):
1093     """Write the same run information in plain text; you never edit a JSON file."""
```

`def` creates this callable; the parentheses name its inputs (`output`, `information`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1094–1099:

```
1094 output.mkdir(parents=True, exist_ok=True)
1095 text = "RUN INFORMATION\n\n"
1096 text += "This is an output record, not a settings file.\n"
1097 text += "Change the Python parameters at the top of the script to change the run.\n\n"
1098 text += pformat(information, sort_dicts=False, width=90)
1099 (output / "run_notes.txt").write_text(text + "\n")
```

Read this block:

- Line 1094: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1095, `text`: Set this name to the literal text `'RUN INFORMATION\n\n'`.
- Line 1096: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1097: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1098: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1099: call `(output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

fit model

Original function: `fit_model`. Source begins at line 1105.

This coordinates point fitting, reporting, optional inference, and saved files. It turns `best.x` into a labeled table. The boundary list comprehension pairs each coefficient with its bounds and tests distance below `1e-4`; flags do not automatically change the estimate. Saving point estimates before bootstrap protects the useful main result if a long inference job is interrupted.

The nested `estimate` callback supplies demand fitting to the generic bootstrap. The bootstrap runs only after a converged main fit and with the original weekly frame available. Missing inference columns are explicitly filled with NaN so an absent SE cannot be mistaken for zero. `vars(args).items()` enumerates settings; Path objects become strings so JSON serialization can encode them. The returned `best` is still a SciPy result object, while CSV/JSON provide portable outputs.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Distribution outputs. After the main split fit, both history marginals are exported as parameter/quantile summaries, density/CDF grids and PDF/PNG plots. Their SDs are population variation; SEs in estimates.csv describe sampling uncertainty. The saved fit can regenerate these plots without optimization.

Definition and stated intent:

Source lines 1105–1106:

```
1105 def fit_model(data, settings, frame=None):
1106     """Estimate demand, compute requested inference, and save readable outputs."""
```

`def` creates this callable; the parentheses name its inputs (`data`, `settings`, `frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1107–1114:

```
1107 best, results, bounds = optimize_demand(data, settings)
1108 names = parameter_names(settings.model, data["outside"], data["control"])
1109 table = pd.DataFrame({"parameter": names, "estimate": best.x})
1110 table["at_bound"] = [
1111     min(abs(v - lo), abs(v - hi)) < 1e-4 for v, (lo, hi) in zip(best.x, bounds)
1112 ]
1113 settings.output.mkdir(parents=True, exist_ok=True)
1114 table.to_csv(settings.output / "point_estimates.csv", index=False)
```

Read this block:

- Line 1107, `(best, results, bounds)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1108, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 1109, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 1110, `table['at_bound']`: Store a list computed by the displayed for-expression in the selected entry/column.

- Line 1113: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1114: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 1115–1120:

```
1115 plot_gamma_distribution(table, settings.output, converged=best.success)
1116 plot_history_distributions(table, settings.output, converged=best.success)
1117 wtp = implied_wtp(table, getattr(settings, "habit_weight", .7))
1118 wtp["provisional_fit"] = not best.success or bool(table["at_bound"].any())
1119 wtp.to_csv(settings.output / "implied_wtp.csv", index=False)
1120 inference = {"method": settings.se_method, "status": "disabled"}
```

Read this block:

- Line 1115: call `plot_gamma_distribution` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1116: call `plot_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1117, `wtp`: Set this name to the result of calling `implied_wtp(...)`; the arguments and preceding method chain are shown above.
- Line 1118, `wtp['provisional_fit']`: Store the Boolean condition `not best.success or bool(table['at_bound'].any())`; it can be a scalar or a row/array mask depending on its inputs in the selected entry/column.
- Line 1119: call `wtp.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1120, `inference`: Set this name to a dictionary with named entries `method`, `status`.

Syntax and calls used here:

- `plot_gamma_distribution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `plot_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 1121–1151:

```
1121 if settings.se_method == "bootstrap":
1122     if not best.success:
1123         inference["status"] = "main fit not converged; bootstrap not run"
1124     elif frame is None:
```

```
1125         raise ValueError("Bootstrap requires the weekly input frame.")
1126     else:
1127
1128         def estimate(sample, initial):
1129             bootstrap_data = make_arrays(sample, settings.capacity)
1130             fitted, _, _ = optimize_demand(
1131                 bootstrap_data, settings, initial, settings.bootstrap_starts, quiet=True
1132             )
1133             return fitted
1134
1135         standard_errors, inference = bootstrap_standard_errors(
1136             frame, settings, estimate, best.x, names
1137         )
1138         if standard_errors is not None:
1139             table = table.merge(
1140                 standard_errors, on="parameter", validate="one_to_one"
1141             )
1142 elif settings.se_method == "hessian":
1143     if not best.success:
1144         inference["status"] = "main fit not converged; Hessian inference not run"
1145     else:
1146         draws = make_draws(settings.draws, settings.seed, settings.model)
1147         gradient = lambda theta: demand_objective_gradient(theta, data, draws, settings.model)[1]
1148         standard_errors, inference = hessian_standard_errors(
1149             best.x, gradient, data["n_households"], bounds, names, settings.output)
1150         if standard_errors is not None:
1151             table = table.merge(standard_errors, on="parameter", validate="one_to_one")
```

Read this block:

- Line 1121: test `settings.se_method == 'bootstrap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1122: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1123, `inference['status']`: Store the literal text `'main fit not converged; bootstrap not run'` in the selected entry/column.
- Line 1124: test `frame is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1125: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 1128: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 1135, `(standard_errors, inference)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1138: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1139, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.
- Line 1142: test `settings.se_method == 'hessian'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1143: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1144, `inference['status']`: Store the literal text `'main fit not converged; Hessian inference not run'` in the selected entry/column.
- Line 1146, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 1147, `gradient`: Set this name to a one-expression callable, with input `theta` and returned expression `demand_objective_gradient(theta, data, draws, settin...`.

- Line 1148, `(standard_errors, inference)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1150: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1151, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 1153–1155:

```
1153 for column in ["std_error", "ci_lower", "ci_upper"]:
1154     if column not in table:
1155         table[column] = np.nan
```

Read this block:

- Line 1153: for each entry yielded by `['std_error', 'ci_lower', 'ci_upper']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1154: test `column not in table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1155, `table[column]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Source lines 1156–1156:

```
1156 table.to_csv(settings.output / "estimates.csv", index=False)
```

Read this block:

- Line 1156: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1157–1186:

```
1157 summary = dict(
1158     model=settings.model,
1159     price_control=data["control"],
1160     flavor_spec=getattr(settings, "flavor_spec", "three"),
1161     flavor_categories=list(FLAVORS),
1162     novelty_measure=getattr(settings, "novelty_measure", "previous_share"),
1163     history_specification="separate" if settings.model in ("separate_history", "heterogeneous_history") else
      ↪ "combined_or_none",
1164     sample_restriction="household_size == 1 on each purchase record",
1165     households=data["n_households"],
1166     weeks=len(data["hh"]),
```

```
    capacity=settings.capacity,
    inference=inference,
    converged=bool(best.success),
    message=str(best.message),
    nll=float(best.fun * data["n_households"]),
    parameters=len(names),
    aic=float(2 * best.fun * data["n_households"] + 2 * len(names)),
    bic=float(2 * best.fun * data["n_households"] + np.log(data["n_households"]) * len(names)),
    settings={
        k: str(v) if isinstance(v, Path) else v for k, v in vars(settings).items()
    },
    starts=[
        dict(
            nll=float(r.fun * data["n_households"]),
            converged=bool(r.success),
            theta=r.x.tolist(),
        )
        for r in results
    ],
)
```

Read this block:

- Line 1157, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `list`: Materialize iterable entries as an ordered Python list.
- `len`: Count elements/rows rather than summing their numerical values.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 1187–1190:

```
(settings.output / "fit.json").write_text(json.dumps(summary, indent=2))
write_run_notes(settings.output, summary)
print("Inference: " + inference["status"], flush=True)
print(table.to_string(index=False))
```

Read this block:

- Line 1187: call `(settings.output / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1188: call `write_run_notes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1189: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1190: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `write_run_notes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 1191–1192:

```
1191 if not best.success or table["at_bound"].any():
1192     print("Provisional fit: inspect convergence/bounds before interpretation.")
```

Read this block:

- Line 1191: test `not best.success or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1192: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1193–1193:

```
1193 return best
```

Read this block:

- Line 1193: return the object currently named `best`; plain assignment does not copy it. This ends the call.

fit model.estimate

Original function: `fit_model.estimate`. Source begins at line 1128.

This nested fake estimator returns an `OptimizeResult` with coefficients equal to resampled residual SDs. It deliberately depends on the recomputed controls, so frozen controls would change the bootstrap distribution. It declares success and a harmless objective because the test targets bootstrap orchestration, not numerical optimization. Mocking the demand fit makes this check faster and isolates whether first stages are rerun and empirical SEs calculated correctly.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 1128–1128:

```
1128 def estimate(sample, initial):
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1129–1132:

```
1129 bootstrap_data = make_arrays(sample, settings.capacity)
1130 fitted, _, _ = optimize_demand(
1131     bootstrap_data, settings, initial, settings.bootstrap_starts, quiet=True
1132 )
```

Read this block:

- Line 1129, `bootstrap_data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 1130, `(fitted, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 1133–1133:

```
1133 return fitted
```

Read this block:

- Line 1133: return the object currently named `fitted`; plain assignment does not copy it. This ends the call.

read command line

Original function: `read_command_line`. Source begins at line 1201.

Parses server settings including flavor_spec, model, capacity, novelty_measure, controls, draws, starts and inference. Ordinary editable settings supply defaults; current pipeline flags explicitly select uncontrolled split demand and original previous-share novelty.

Definition and stated intent:

Source lines 1201–1202:

```
1201 def read_command_line(settings):
1202     """Optional server flags. Normal use: edit the parameters at the top."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1203–1216:

```
1203 parser = argparse.ArgumentParser(description=__doc__)
1204 parser.add_argument("--flavor-spec", choices=["three", "five"], default=flavor_spec)
1205 parser.add_argument(
1206     "--sample",
1207     type=Path,
1208     default=DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet",
1209 )
1210 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
1211 parser.add_argument(
1212     "--products-extra",
1213     type=Path,
1214     help="Annual flavor metadata; defaults to RMS/combined/products_extra.parquet beside the RMS data directory.",
1215 )
1216 parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results/estimation_split"))
```

Read this block:

- Line 1203, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 1204: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1205: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1210: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1211: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 1216: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 1217–1230:

```
1217 parser.add_argument(
1218     "--model",
1219     choices=["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
     ↪ "heterogeneous_history"],
1220     default=model,
1221 )
1222 parser.add_argument("--first-year", type=int, default=2006)
1223 parser.add_argument("--last-year", type=int, default=2015)
1224 parser.add_argument("--max-households", type=int)
1225 parser.add_argument(
1226     "--capacity",
1227     type=int,
1228     default=0,
1229     help="0: conditional flavor demand; 7: outside=max(7-Q,0)",
1230 )
```

Read this block:

- Line 1217: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1222: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1223: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1224: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1225: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1231–1244:

```
1231 parser.add_argument(
1232     "--no-price-control",
1233     action="store_true",
1234     default=no_price_control,
1235     help="Disable the price control using the overall other-DMA mean-price IV",
1236 )
1237 parser.add_argument("--household-dma-column", default="dma_cd")
1238 parser.add_argument(
1239     "--se-method", choices=["hessian", "bootstrap", "none"], default="hessian"
1240 )
1241 parser.add_argument("--bootstrap-reps", type=int, default=49)
1242 parser.add_argument("--bootstrap-starts", type=int, default=2)
1243 parser.add_argument("--novelty-measure", choices=["previous_share", "lagged_switch_away"],
1244                     default="previous_share", help="Legacy share or lagged binary recovery event")
```

Read this block:

- Line 1231: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1237: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 1238: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1241: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1242: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1243: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1245–1250:

```
1245 parser.add_argument("--habit-weight", type=float, default=0.7)
1246 parser.add_argument("--draws", type=int, default=128)
1247 parser.add_argument("--starts", type=int, default=3)
1248 parser.add_argument("--maxiter", type=int, default=600)
1249 parser.add_argument("--seed", type=int, default=306)
1250 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 1245: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1246: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1247: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1248: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1249: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1250: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 1251–1251:

```
1251 return parser.parse_args()
```

Read this block:

- Line 1251: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run estimation

Original function: `run_estimation`. Source begins at line 1261.

Runs the preparation, optional price first stages, exact weekly-table export, and demand fit/inference in that order. Current simple defaults select heterogeneous_history and capacity 7. Reconstructing a deleted weekly table with regeneration is a separate path that skips optimization.

Definition and stated intent:

Source lines 1261–1262:

```
1261 def run_estimation(settings):
1262     """The actual research steps, in the order they run."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1263–1264:

```
1263 if not 0 <= settings.habit_weight <= 1 or settings.capacity < 0 or settings.starts < 1:
1264     raise ValueError("Check habit weight, capacity, and starts.")
```

Read this block:

- Line 1263: test `not 0 <= settings.habit_weight <= 1 or settings.capacity < 0 or settings.star...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1264: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 1265–1268:

```
1265 if settings.first_year > settings.last_year or (
1266     settings.max_households is not None and settings.max_households < 1
1267 ):
1268     raise ValueError("Check years and household limit.")
```

Read this block:

- Line 1265: test `settings.first_year > settings.last_year or (settings.max_households is not N...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1268: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 1269–1270:

```
1269 if settings.draws < 2 or settings.draws % 2 or settings.maxiter < 1:
1270     raise ValueError("Use an even draw count >= 2 and maxiter >= 1.")
```

Read this block:

- Line 1269: test `settings.draws < 2 or settings.draws % 2 or settings.maxiter < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1270: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 1271–1274:

```
1271 if settings.se_method == "bootstrap" and (
1272     settings.bootstrap_reps < 20 or settings.bootstrap_starts < 1
1273 ):
1274     raise ValueError("Bootstrap requires at least 20 replications and one start.")
```

Read this block:

- Line 1271: test `settings.se_method == 'bootstrap' and (settings.bootstrap_reps < 20 or settin...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1274: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 1276–1276:

```
1276 frame = prepare_data(settings)
```

Read this block:

- Line 1276, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 1279–1283:

```
1279 if not settings.no_price_control:
1280     frame, first_stage = add_price_controls(frame)
1281     settings.output.mkdir(parents=True, exist_ok=True)
1282     first_stage.to_csv(settings.output / "first_stage.csv", index=False)
1283     print(first_stage.to_string(index=False), flush=True)
```

Read this block:

- Line 1279: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1280, `(frame, first_stage)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1281: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1282: call `first_stage.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1283: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 1284–1290:

```
1284 settings.output.mkdir(parents=True, exist_ok=True)
1285 # Step 3: save the exact table that will enter the likelihood.
1286 frame.to_csv(settings.output / "weekly_inputs.csv", index=False)
1287
1288 # Step 4: form numeric arrays, estimate demand, compute inference, save results.
1289 data = make_arrays(frame, settings.capacity)
1290 fit_model(data, settings, frame)
```

Read this block:

- Line 1284: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1286: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1289, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

- Line 1290: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/demand/gamma_calibration.py

Auxiliary binary-theory calibration to aggregate observed price moments and estimated novelty-type support. The uninformed firm chooses offers with fixed beliefs; both policies are evaluated under persistent types. Numerical validity and status diagnostics are part of the result.

Module setup and direct-execution blocks

Source lines 1–11:

```
"""Server-only supply calibration to observed prices and fitted gamma types.

Same binary utility and fixed-belief timing as theory/gamma_uncertainty.py.
No coupon prices or counterfactual demand forecasts enter calibration targets.
"""
from pathlib import Path
import sys
_policy_root = next(p for p in Path(__file__).resolve().parents
                    if (p / 'Code/privacy/server_data_policy.py').is_file())
sys.path.insert(0, str(_policy_root / 'Code/privacy'))
from server_data_policy import server_only
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: import `sys`. An as-name is a convenient local alias, not another library.
- Line 8, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 10: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 11: from `server_data_policy`, import `server_only` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 12–18:

```
import json
import numpy as np
```

```
import pandas as pd
from scipy.optimize import least_squares, minimize_scalar
from scipy.special import expit, lambertw

GROUPS = ['low', 'middle', 'high']
```

Read this block:

- Line 12: import `json`. An as-name is a convenient local alias, not another library.
- Line 13: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 14: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 15: from `scipy.optimize`, import `least_squares, minimize_scalar` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `scipy.special`, import `expit, lambertw` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 18, `GROUPS`: Set this name to the list `['low', 'middle', 'high']`.

Source lines 19–20:

```
PRIORS = [('Equal types', [1/3]*3), ('Low type more likely', [.5,.25,.25]),
          ('Medium type more likely', [.25,.5,.25]), ('High type more likely', [.25,.25,.5])]
```

Read this block:

- Line 19, `PRIORS`: Set this name to a list containing 4 displayed entries.

Source lines 71–71:

```
class CalibrationError(ValueError):
```

Read this block:

- Line 71: define class `CalibrationError` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 72–72:

```
"""Observed price moments do not support an admissible binary calibration."""
```

Read this block:

- Line 72: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

known prices

Original function: `known_prices`. Source begins at line 23.

Solves the binary symmetric switching theory for a known gamma and initial history using Lambert W. Alpha is signed negative. This auxiliary theory differs from the full empirical split-history demand; its prices are calibrated to aggregate observed-price moments.

Definition and stated intent:

Source lines 23–24:

```
def known_prices(gamma, h0, alpha, beta, delta):
    """Theory's known-type analytic prices; cost=0, alpha signed negative."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `h0`, `alpha`, `beta`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 25–30:

```
terminal_z = lambertw(np.exp(beta+np.array([-gamma, gamma])-1)).real
terminal_price, terminal_value = -(1+terminal_z)/alpha, -terminal_z/alpha
effective = -delta*(terminal_value[0]-terminal_value[1])
intercept = beta+(-gamma if h0 == 'j' else gamma)
first_z = float(lambertw(np.exp(intercept+alpha*effective-1)).real)
p1 = effective-(1+first_z)/alpha
```

Read this block:

- Line 25, `terminal_z`: Set this name to the attribute `lambertw(np.exp(beta + np.array([-gamma, gamma]) - 1)).real` of the object before the dot.
- Line 26, `(terminal_price, terminal_value)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 27, `effective`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `intercept`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 29, `first_z`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 30, `p1`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `lambertw`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

Source lines 31–32:

```
s1 = float(expit(intercept+alpha*p1))
profit = s1*p1+delta*(s1*terminal_value[0]+(1-s1)*terminal_value[1])
```

Read this block:

- Line 31, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 32, `profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 33–34:

```
return dict(profit=float(profit), expected_p1=float(p1),
            expected_p2=float(s1*terminal_price[0]+(1-s1)*terminal_price[1]))
```

Read this block:

- Line 33: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

maximize

Original function: `maximize`. Source begins at line 37.

Uses a coarse grid to find candidate local peaks, refines them with scalar optimization, and includes endpoints. It rejects an optimum at the search boundary rather than assuming an arbitrary bracket contains the solution.

Definition and stated intent:

Source lines 37–37:

```
37 def maximize(fun, bounds, points=161):
```

`def` creates this callable; the parentheses name its inputs (`fun`, `bounds`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 38–40:

```
38 grid = np.linspace(*bounds, points)
39 values = np.array([fun(p) for p in grid])
40 candidates = [(values[0], grid[0]), (values[-1], grid[-1])]
```

Read this block:

- Line 38, `grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 39, `values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 40, `candidates`: Set this name to a list containing 2 displayed entries.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `fun`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 41–45:

```
41 for i in range(1, points-1):
42     if values[i] >= values[i-1] and values[i] >= values[i+1]:
43         r = minimize_scalar(lambda p: -fun(p), bounds=(grid[i-1], grid[i+1]),
44                             method='bounded', options={'xatol': 1e-10})
45         candidates.append((-r.fun, r.x))
```

Read this block:

- Line 41: for each entry yielded by `range(1, points - 1)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 42: test `values[i] >= values[i - 1] and values[i] >= values[i + 1]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 43, `r`: Set this name to the result of calling `minimize_scalar(...)`; the arguments and preceding method chain are shown above.
- Line 45: call `candidates.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `minimize_scalar`: Numerically minimize a one-variable objective, used as an independent check of price optimality.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 46–46:

```
46 value, price = max(candidates)
```

Read this block:

- Line 46, `(value, price)`: Unpack the result of calling `max(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 47–48:

```
47 if min(price-bounds[0], bounds[1]-price) < 1e-6:
48     raise RuntimeError('Supply price optimum at numerical boundary')
```

Read this block:

- Line 47: test `min(price - bounds[0], bounds[1] - price) < 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 49–49:

```
49 return float(price), float(value)
```

Read this block:

- Line 49: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

uncertainty

Original function: `uncertainty`. Source begins at line 52.

Chooses terminal and initial offers using unchanged prior weights in every history. It checks the first-price optimum against a wider grid and returns this fixed-belief policy and its objective. Actual persistent-type revenue is evaluated separately in run_calibration.

Definition and stated intent:

Source lines 52–52:

```
52 def uncertainty(gamma, prior, h0, alpha, beta, delta):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `prior`, `h0`, `alpha`, `beta`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 53–54:

```
53 prior = np.asarray(prior, float)
54 scale = (30+abs(beta)+np.max(np.abs(gamma)))/abs(alpha)
```

Read this block:

- Line 53, `prior`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 54, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 55–56:

```
55 def shares(p, h):
56     return expit(beta+alpha*p+(-gamma if h == 'j' else gamma))
```

Read this block:

- Line 55: define nested callable `shares`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 57–58:

```
57 p2j, wj = maximize(lambda p: float(prior @ shares(p, 'j'))*p, (0., scale))
58 p20, w0 = maximize(lambda p: float(prior @ shares(p, '0'))*p, (0., scale))
```

Read this block:

- Line 57, `(p2j, wj)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 58, `(p20, w0)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `maximize`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `shares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 59–61:

```
59 def objective(p):
60     s = float(prior @ shares(p, h0))
61     return s*p+delta*(s*wj+(1-s)*w0)
```

Read this block:

- Line 59: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 62–63:

```
62 p1, value = maximize(objective, (-scale, scale))
63 _, wide_value = maximize(objective, (-2*scale, 2*scale), 321)
```

Read this block:

- Line 62, `(p1, value)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 63, `(_, wide_value)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 64–65:

```
64 if not np.isclose(value, wide_value, atol=1e-7, rtol=1e-8):
65     raise AssertionError('Supply price wider-grid audit failed')
```

Read this block:

- Line 64: test `not np.isclose(value, wide_value, atol=1e-07, rtol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 65: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 66–66:

```
66 s1 = float(prior @ shares(p1, h0))
```

Read this block:

- Line 66, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Source lines 67–68:

```
67 return dict(profit=value, expected_p1=p1, expected_p2=s1*p2j+(1-s1)*p20,
68            p2j=p2j, p20=p20, belief_j=prior.tolist(), belief_0=prior.tolist())
```

Read this block:

- Line 67: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.

uncertainty.shares

Original function: `uncertainty.shares`. Source begins at line 55.

Returns the vector of type-specific binary purchase probabilities for a price/history pair. A closure supplies gamma, beta and alpha from uncertainty; the caller averages the vector using fixed prior weights.

Definition and stated intent:

Source lines 55–55:

```
55 def shares(p, h):
```

`def` creates this callable; the parentheses name its inputs (`p`, `h`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 56–56:

```
56 return expit(beta+alpha*p+(-gamma if h == 'j' else gamma))
```

Read this block:

- Line 56: return the result of calling `expit(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

uncertainty.objective

Original function: `uncertainty.objective`. Source begins at line 59.

Computes first-period expected margin plus discounted branch values using fixed beliefs. The same prior is used after either branch; this is the decision objective, not automatically the persistent-type policy evaluation.

Definition and stated intent:

Source lines 59–59:

```
59 def objective(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 60–60:

```
60 s = float(prior @ shares(p, h0))
```

Read this block:

- Line 60, `s`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `shares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 61–61:

```
61 return s*p+delta*(s*wj+(1-s)*w0)
```

Read this block:

- Line 61: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

CalibrationError. init

Original function: `CalibrationError.__init__`. Source begins at line 73.

Attaches an aggregate diagnostics dictionary to a named exception and sets a readable message pointing to the status file. Catching this specific exception permits coupon pricing to continue without treating a rejected calibration as valid.

Definition and stated intent:

Source lines 73–73:

```
73 def __init__(self, diagnostics):
```

`def` creates this callable; the parentheses name its inputs (`self`, `diagnostics`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 74–75:

```
74 self.diagnostics = diagnostics
75 super().__init__('Binary supply calibration rejected; see information_calibration_status.json')
```

Read this block:

- Line 74, `self.diagnostics`: Set this name to the object currently named `diagnostics`; plain assignment does not copy it.
- Line 75: call `super().__init__` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `super`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

calibration rank

Original function: `calibration_rank`. Source begins at line 78.

Uses Jacobian singular values and a relative tolerance to detect nearly flat parameter directions. Numerical rank is an identification check for the aggregate calibration; convergence alone is insufficient.

Definition and stated intent:

Source lines 78–79:

```
78 def calibration_rank(jacobian):
79     """Reject numerically flat directions, not merely exact floating-point zeros."""
```

`def` creates this callable; the parentheses name its inputs (`jacobian`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 80–80:

```
80 singular = np.linalg.svd(jacobian, compute_uv=False)
```

Read this block:

- Line 80, `singular`: Set this name to the result of calling `np.linalg.svd(...)`; the arguments and preceding method chain are shown above.

Source lines 81–81:

```
81 return int(np.sum(singular > max(1e-12, singular[0]*1e-7)))
```

Read this block:

- Line 81: return the result of calling `int(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

calibrate

Original function: `calibrate`. Source begins at line 84.

Fits common binary-theory alpha and beta to group/history price moments with household-count weights and relative residuals. It uses multiple starts and a larger evaluation budget, filters out failed/bound/rank-deficient results, then chooses the best admissible objective. No admissible solution raises CalibrationError with optimizer diagnostics.

Definition and stated intent:

Source lines 84–85:

```
84 def calibrate(moments, gamma, delta):
85     """Weighted relative price-moment fit, common alpha/beta across type/h0 cells."""
```

`def` creates this callable; the parentheses name its inputs (`moments`, `gamma`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–87:

```
86 target = moments[['observed_p1', 'observed_p2']].to_numpy()
87 weight = np.sqrt(moments.households.to_numpy()/moments.households.sum())[:, None]
```

Read this block:

- Line 86, `target`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 87, `weight`: Set this name to the entry/slice selected by `np.sqrt(moments.households.to_numpy() / moments.household...`; brackets select from the object to their left.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 88–92:

```
88 def predictions(x):
89     alpha, beta = -np.exp(x[0]), x[1]
90     return np.array([[r['expected_p1'], r['expected_p2']] for r in [
91         known_prices(gamma[GROUPS.index(row.group)], row.h0, alpha, beta, delta)
92         for row in moments.itertuples()]])
```

Read this block:

- Line 88: define nested callable `predictions`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 93–94:

```
93 def residual(x):
94     return ((predictions(x)-target)/target*weight).ravel()
```

Read this block:

- Line 93: define nested callable `residual`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 95–107:

```
start = -np.log(target.mean())
fits = [least_squares(residual, [np.clip(start, -7., 4.), b],
                      bounds=([-8., -15.], [5., 15.]), x_scale='jac', max_nfev=4000,
                      xtol=1e-11, ftol=1e-11, gtol=1e-11)
        for b in [-1., 2., 5.]]
# Choose among admissible solutions, not an inadmissible lower objective.
attempts = [dict(success=bool(f.success), active_bounds=f.active_mask.tolist(),
                 jacobian_rank=calibration_rank(f.jac),
                 singular_values=np.linalg.svd(f.jac, compute_uv=False).tolist(),
                 parameters=f.x.tolist(), objective=float(np.sum(f.fun**2)),
                 message=str(f.message)) for f in fits]
valid = [f for f in fits if f.success and not np.any(f.active_mask)
         and calibration_rank(f.jac) == 2 and np.isfinite(f.fun).all()]
```

Read this block:

- Line 95, `start`: Set this name to the negated or sign-adjusted expression `-np.log(target.mean())`.
- Line 96, `fits`: Set this name to a list computed by the displayed for-expression.
- Line 101, `attempts`: Set this name to a list computed by the displayed for-expression.
- Line 106, `valid`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `least_squares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `calibration_rank`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `str`: Convert to text for paths, messages, or serialization.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 108–110:

```
if not valid:
    raise CalibrationError(dict(status='failed', attempts=attempts,
                                bounds=[[-8., -15.], [5., 15.]]))
```

Read this block:

- Line 108: test `not valid`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 109: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `CalibrationError`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 111–115:

```
111 fit = min(valid, key=lambda r: np.sum(r.fun**2))
112 result = moments.copy()
113 result[['fitted_p1', 'fitted_p2']] = predictions(fit.x)
114 result['relative_error_p1'] = (result.fitted_p1-result.observed_p1)/result.observed_p1
115 result['relative_error_p2'] = (result.fitted_p2-result.observed_p2)/result.observed_p2
```

Read this block:

- Line 111, `fit`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.
- Line 112, `result`: Set this name to the result of calling `moments.copy(...)`; the arguments and preceding method chain are shown above.
- Line 113, `result[['fitted_p1', 'fitted_p2']]`: Store the result of calling `predictions(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 114, `result['relative_error_p1']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 115, `result['relative_error_p2']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `predictions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 116–116:

```
116 return -float(np.exp(fit.x[0])), float(fit.x[1]), result, float(np.sqrt(np.mean(fit.fun**2)))
```

Read this block:

- Line 116: return a tuple of 4 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

calibrate.predictions

Original function: `calibrate.predictions`. Source begins at line 88.

Transforms log price sensitivity to a negative alpha and solves known-type theoretical prices for every aggregate group/history cell. The output aligns with the two observed price columns; a wrong cell order would fit the wrong moments.

Definition and stated intent:

Source lines 88–88:

```
88 def predictions(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 89–89:

```
89 alpha, beta = -np.exp(x[0]), x[1]
```

Read this block:

- Line 89, `(alpha, beta)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 90–92:

```
90 return np.array([[r['expected_p1'], r['expected_p2']] for r in [
91     known_prices(gamma[GROUPS.index(row.group)], row.h0, alpha, beta, delta)
92     for row in moments.itertuples()]])
```

Read this block:

- Line 90: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `known_prices`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

calibrate.residual

Original function: `calibrate.residual`. Source begins at line 93.

Subtracts observed aggregate prices from predictions, divides by observed prices and applies square-root cell weights. Flattening gives least_squares a one-dimensional residual vector while preserving the weighted objective.

Definition and stated intent:

Source lines 93–93:

```
93 def residual(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–94:

```
94 return ((predictions(x)-target)/target*weight).ravel()
```

Read this block:

- Line 94: return the result of calling `.ravel(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `predictions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

run calibration

Original function: `run_calibration`. Source begins at line 120.

Forms actual consecutive-week observed-price targets and fitted novelty support, calibrates binary supply, and solves known-type/fixed-belief offers. It evaluates both policies under persistent types before averaging. Valid runs save common-evaluation revenues; rejected runs save aggregate moments and explicit status without fabricated charts.

Definition and stated intent:

Source lines 120–122:

```
120 def run_calibration(frame, households, delta, output):
121     # frame is already permuted so berry is the focal flavor. Require actual
122     # consecutive-week observations; never carry one price into future periods.
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`, `delta`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 123–126:

```
123 first = frame.sort_values(['household_code', 'week']).groupby('household_code').head(1)
124 first = first.set_index('household_code').reindex(households.household_code)
125 weekly = frame.set_index(['household_code', 'week'])
126 observed = []
```

Read this block:

- Line 123, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 124, `first`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 125, `weekly`: Set this name to the result of calling `frame.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 126, `observed`: Set this name to the list `[]`.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 127–130:

```
127 for lag in [7, 14]:
128     index = pd.MultiIndex.from_arrays([first.index, first.week+pd.Timedelta(days=lag)],
129                                       names=['household_code', 'week'])
130     observed.append(weekly.reindex(index).price_berry.to_numpy(float))
```

Read this block:

- Line 127: for each entry yielded by `[7, 14]`, bind/unpack it into `lag` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `index`: Set this name to the result of calling `pd.MultiIndex.from_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 130: call `observed.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 131–134:

```
131 targets = pd.DataFrame(dict(group=households.group.to_numpy(),
132     h0=np.where(first.berry.to_numpy() > 0, 'j', '0'), p1=observed[0], p2=observed[1]))
133 valid = np.isfinite(targets[['p1', 'p2']]).all(axis=1) & (targets[['p1', 'p2']] > 0).all(axis=1)
134 targets = targets.loc[valid]
```

Read this block:

- Line 131, `targets`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 133, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 134, `targets`: Set this name to the entry/slice selected by `targets.loc[valid]`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 135–139:

```
135 if targets.empty:
136     status = dict(status='failed', reason='No observed focal prices at both t0+7 and t0+14',
137                   coupon_pricing_affected=False)
138     (output/'information_calibration_status.json').write_text(json.dumps(status, indent=2)+'\n')
139     raise CalibrationError(status)
```

Read this block:

- Line 135: test `targets.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 136, `status`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 138: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `CalibrationError`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 140–142:

```
140 moments = targets.groupby(['group', 'h0']).agg(
141     households=('p1', 'size'), observed_p1=('p1', 'mean'), observed_p2=('p2', 'mean')).reset_index()
142 gamma = households.groupby('group').switching_mean.mean().reindex(GROUPS).to_numpy(float)
```

Read this block:

- Line 140, `moments`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 142, `gamma`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 143–144:

```
143 if not np.isfinite(gamma).all():
144     raise ValueError('Missing fitted low/medium/high gamma types')
```

Read this block:

- Line 143: test `not np.isfinite(gamma).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 144: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 145–153:

```
145 try:
146     alpha, beta, fitted_moments, rms = calibrate(moments, gamma, delta)
147 except CalibrationError as error:
148     moments.to_csv(output / 'information_calibration_moments.csv', index=False)
149     status = dict(error.diagnostics, estimated_gamma=gamma.tolist(),
150                   price_moment_cells=len(moments), delta=delta,
151                   coupon_pricing_affected=False)
152     (output/'information_calibration_status.json').write_text(json.dumps(status, indent=2)+'\n')
153     raise
```

Read this block:

- Line 145: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 146, `(alpha, beta, fitted_moments, rms)`: Unpack the result of calling `calibrate(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 147: catch `CalibrationError` from the preceding try; the optional as-name holds the error object for logging.
- Line 148: call `moments.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 149, `status`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 152: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 153: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `calibrate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 154–155:

```
154 fitted_moments.to_csv(output / 'information_calibration_moments.csv', index=False)
155 rows, type_rows = [], []
```

Read this block:

- Line 154: call `fitted_moments.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 155, `(rows, type_rows)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Source lines 156–182:

```
156 for scenario, prior in PRIORS:
157     for h0 in ['j', '0']:
158         known = [known_prices(g, h0, alpha, beta, delta) for g in gamma]
159         full = {key: float(np.dot(prior, [r[key] for r in known])) for key in known[0]}
160         unknown = uncertainty(gamma, prior, h0, alpha, beta, delta)
161         persistent_unknown = []
162         for k, label in enumerate(['low', 'medium', 'high']):
163             s1 = float(expit(beta+alpha*unknown['expected_p1']+(-gamma[k] if h0 == 'j' else gamma[k])))
164             s2j = float(expit(beta+alpha*unknown['p2j']-gamma[k]))
165             s20 = float(expit(beta+alpha*unknown['p20']+gamma[k]))
166             true_profit = s1*unknown['expected_p1']+delta*(
167                 s1*unknown['p2j']*s2j+(1-s1)*unknown['p20']*s20)
168             if known[k]['profit']+1e-7 < true_profit:
169                 raise AssertionError('Known-type bound on persistent-type revenue failed')
170             persistent_unknown.append(true_profit)
171             type_rows.append(dict(scenario=scenario, h0=h0, type=label,
172                 gamma=float(gamma[k]), probability=prior[k],
173                 known_profit=known[k]['profit'], known_p1=known[k]['expected_p1'],
174                 known_expected_p2=known[k]['expected_p2'],
175                 unknown_policy_persistent_profit=true_profit))
176         persistent_unknown_revenue = float(np.dot(prior, persistent_unknown))
177         information_gain = full['profit'] - persistent_unknown_revenue
178         for case, result in [('known', full), ('unknown', unknown)]:
179             rows.append(dict(scenario=scenario, h0=h0, case=case,
180                 **{key: result[key] for key in full},
181                 persistent_type_revenue=(full['profit'] if case == 'known' else persistent_unknown_revenue),
182                 value_of_information=information_gain))
```

Read this block:

- Line 156: for each entry yielded by `PRIORS`, bind/unpack it into `(scenario, prior)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 157: for each entry yielded by `['j', '0']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 158, `known`: Set this name to a list computed by the displayed for-expression.
- Line 159, `full`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 160, `unknown`: Set this name to the result of calling `uncertainty(...)`; the arguments and preceding method chain are shown above.
- Line 161, `persistent_unknown`: Set this name to the list `[]`.
- Line 162: for each entry yielded by `enumerate(['low', 'medium', 'high'])`, bind/unpack it into `(k, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 163, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 164, `s2j`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

- Line 165, `s20`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 166, `true_profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 168: test `known[k]['profit'] + 1e-07 < true_profit`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 169: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 170: call `persistent_unknown.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 171: call `type_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 176, `persistent_unknown_revenue`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 177, `information_gain`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 178: for each entry yielded by `[('known', full), ('unknown', unknown)]`, bind/unpack it into `(case, result)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 179: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `uncertainty`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `known_prices`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 183–197:

```
pd.DataFrame(type_rows).to_csv(output / 'information_friction_by_group.csv', index=False)
table = pd.DataFrame(rows)
table.to_csv(output / 'information_friction_aggregate.csv', index=False)
settings = dict(alpha=alpha, beta=beta, delta=delta, cost=0.,
    types=['low', 'medium', 'high'], estimated_gamma=gamma.tolist(), priors=PRIORS,
    beliefs='fixed in every state', baseline='known type supply pricing',
    calibrated_parameters=['alpha', 'beta'], calibration='household-count-weighted relative price moments',
    weighted_residual_rms=rms, households_in_price_targets=int(valid.sum()),
    households_without_complete_price_targets=int((~valid).sum()),
    observed_prices='actual weekly prices at t0+7 and t0+14 days',
    outcome_units='per model consumer, not population revenue totals',
    revenue_evaluation='same persistent consumer types under both policies',
    profit_column='fixed-belief optimization objective; use persistent_type_revenue for policy comparisons',
    assumption='observed mean offered prices target known-type optimal supply prices',
    gamma_mapping='fitted novelty group means used as symmetric binary theory gamma')
```

Read this block:

- Line 183: call `pd.DataFrame(type_rows).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 184, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 185: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186, `settings`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 198–205:

```
198 (output / 'information_friction_settings.json').write_text(json.dumps(settings, indent=2)+'\n')
199 # State probabilities and estimated support are aggregate results only.
200 pd.DataFrame([dict(scenario=name, type=label, gamma=float(gamma[k]), probability=prior[k],
201                    belief_j=prior[k], belief_0=prior[k])
202                for name, prior in PRIORS for k, label in enumerate(['low','medium','high'])]
203               ).to_csv(output / 'information_friction_states.csv', index=False)
204 (output/'information_calibration_status.json').write_text(json.dumps(
205     dict(status='complete', alpha=alpha, beta=beta, weighted_residual_rms=rms), indent=2)+'\n')
```

Read this block:

- Line 198: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 200: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 204: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 206–206:

```
206 return table, settings
```

Read this block:

- Line 206: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

../final_simple/demand/history_distributions.py

Small saved-fit distribution entry point using the export calculation in the readable estimator. Edit fit_dir or use –fit-dir. It exports population density/CDF grids, quantiles/sign shares and plots without reading a weekly household table.

Module setup and direct-execution blocks

Source lines 1–5:

```
1 """Export distributions from an existing split fit; edit Parameters or use flags."""
2 from pathlib import Path
3 import argparse
4 import json
5 import pandas as pd
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 4: import `json`. An as-name is a convenient local alias, not another library.
- Line 5: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 6–9:

```
6 try:
7     from .estimation import export_history_distributions
8 except ImportError:
9     from estimation import export_history_distributions
```

Read this block:

- Line 6: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 7: from `estimation`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: catch `ImportError` from the preceding try; the optional as-name holds the error object for logging.
- Line 9: from `estimation`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 12–13:

```
12 fit_dir = None
13 output = None  # None writes alongside the saved fit.
```

Read this block:

- Line 12, `fit_dir`: Set this name to the literal value `None`.
- Line 13, `output`: Set this name to the literal value `None`.

Source lines 29–30:

```
29 if __name__ == '__main__':
30     main()
```

Read this block:

- Line 29: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 30: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 16.

Reads saved fit metadata and coefficient estimates, verifies a split model, and exports population marginals. It does not read household histories or re-estimate demand.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 16–16:

```
16 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–20:

```
17 parser = argparse.ArgumentParser(description=__doc__)
18 parser.add_argument('--fit-dir', type=Path, default=fit_dir)
19 parser.add_argument('--output', type=Path, default=output)
20 args = parser.parse_args()
```

Read this block:

- Line 17, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 18: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 19: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 21–22:

```
21 if args.fit_dir is None:
22     parser.error('Set fit_dir above or supply --fit-dir.')
```

Read this block:

- Line 21: test `args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 23–23:

```
23 fit = json.loads((args.fit_dir/'fit.json').read_text())
```

Read this block:

- Line 23, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 24–25:

```
24 if fit['model'] not in ['separate_history', 'heterogeneous_history']:
25     parser.error('Requires a split-history fit')
```

Read this block:

- Line 24: test `fit['model'] not in ['separate_history', 'heterogeneous_history']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 25: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 26–27:

```
26 export_history_distributions(pd.read_csv(args.fit_dir/'point_estimates.csv'),
27                              args.output or args.fit_dir, fit.get('converged', False))
```

Read this block:

- Line 26: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/demand/plain_berry_pricing.py

Main two-focal comparison using an uncontrolled original-share split fit. Only the seller list is limited to plain/berry; the saved three/five demand category set remains intact. This wrapper does not estimate demand.

Module setup and direct-execution blocks

Source lines 1–11:

```
1  """Compare split-model pricing for plain and berry, retaining all demand categories.
2
3  Edit Parameters below or supply server flags. Uses the same household solver as
4  split_pricing.py. Other remains a demand alternative and history state, while
5  only plain and berry are considered as focal sellers in separate counterfactuals.
6  """
7
8  # Empirical I/O is permitted only on the research server, never on this computer.
9  from pathlib import Path as _PolicyPath
10 import sys as _policy_sys
11 _policy_root = next((p for p in _PolicyPath(__file__).resolve().parents if (p /
    ↪  "Code/privacy/server_data_policy.py").is_file()), None)
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 9: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 11, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 12–13:

```
12 if _policy_root is not None:
13     _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
```

Read this block:

- Line 12: test `_policy_root is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 13: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.

Source lines 14–19:

```
14 from server_data_policy import server_only, require_server, require_data_path
15 from pathlib import Path
16 from types import SimpleNamespace
17 from pprint import pformat
18 import argparse
19 import json
```

Read this block:

- Line 14: from `server_data_policy`, import `server_only, require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 17: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 18: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 19: import `json`. An as-name is a convenient local alias, not another library.

Source lines 20–23:

```
20 import sys
21
22 import numpy as np
23 import pandas as pd
```

Read this block:

- Line 20: import `sys`. An as-name is a convenient local alias, not another library.
- Line 22: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 23: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 25–28:

```
25 if __package__:
26     from . import split_pricing as pricing
27 else:
28     import split_pricing as pricing
```

Read this block:

- Line 25: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 26: from ‘`, importsplit_pricing as pricing‘ into this module’s namespace. Importing can execute the imported module’s top-level setup.
- Line 28: import `split_pricing as pricing`. An as-name is a convenient local alias, not another library.

Source lines 32–37:

```
household_input = None   # Path to regenerated weekly_inputs.csv.
fit_dir = None           # Matching uncontrolled heterogeneous_history fitted-results folder.
output = Path("/scratch/dtm63837/Kilts_Panel/results") / "plain_berry_pricing"
discounts = np.linspace(0., 1., 101)
demand_delta = .92
type_treatment = "posterior-mean"   # Or posterior-integrated.
```

Read this block:

- Line 32, `household_input`: Set this name to the literal value `None`.
- Line 33, `fit_dir`: Set this name to the literal value `None`.
- Line 34, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 35, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 36, `demand_delta`: Set this name to the literal value `0.92`.
- Line 37, `type_treatment`: Set this name to the literal text `'posterior-mean'`.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 38–42:

```
batch_size = 24
save_state_policies = False

# This selects focal sellers, not the categories retained in demand estimation.
FOCAL_FLAVORS = ("plain", "berry")
```

Read this block:

- Line 38, `batch_size`: Set this name to the literal value `24`.
- Line 39, `save_state_policies`: Set this name to the literal value `False`.
- Line 42, `FOCAL_FLAVORS`: Set this name to the tuple `('plain', 'berry')`.

Source lines 105–112:

```
if __name__ == '__main__':
    settings = SimpleNamespace(household_input=household_input, fit_dir=fit_dir,
        output=output, discounts=discounts, demand_delta=demand_delta,
        type_treatment=type_treatment, batch_size=batch_size,
        save_state_policies=save_state_policies)
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_comparison(settings)
```

Read this block:

- Line 105: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 106, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 110: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 111, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 112: call `run_comparison` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_comparison`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

read command line

Original function: `read_command_line`. Source begins at line 46.

Parses explicit server weekly-input and fit paths plus pricing settings. No focal-selection flag is needed because the main comparison always chooses plain and berry. A five-group fit still retains all five demand alternatives.

Definition and stated intent:

Source lines 46–46:

```
46 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–52:

```
47 parser = argparse.ArgumentParser(description=__doc__)
48 parser.add_argument('--household-input', type=Path)
49 parser.add_argument('--fit-dir', type=Path)
50 parser.add_argument('--output', type=Path)
51 parser.add_argument('--discounts', type=float, nargs='+')
52 parser.add_argument('--demand-delta', type=float)
```

Read this block:

- Line 47, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 48: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 50: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 53–56:

```
parser.add_argument('--type-treatment', choices=['posterior-mean', 'posterior-integrated'])
parser.add_argument('--batch-size', type=int)
parser.add_argument('--save-state-policies', action='store_true')
parser.set_defaults(**vars(settings))
```

Read this block:

- Line 53: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 55: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 57–57:

```
return parser.parse_args()
```

Read this block:

- Line 57: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run comparison

Original function: `run_comparison`. Source begins at line 62.

Rejects controlled or experimental-novelty fits, then runs the shared solver for plain and berry separately. FOCAL_FLAVORS selects sellers while saved metadata selects all three/five demand alternatives. Results and hashes are recorded for a later allowlisted export.

Definition and stated intent:

Source lines 62–62:

```
def run_comparison(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 63–64:

```
if settings.household_input is None or settings.fit_dir is None:
    raise ValueError('Set household_input and fit_dir, or supply their server flags.')
```

Read this block:

- Line 63: test `settings.household_input is None or settings.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 64: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 65–66:

```
require_data_path(settings.fit_dir)
fit = json.loads((settings.fit_dir / 'fit.json').read_text())
```

Read this block:

- Line 65: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–68:

```
if fit['settings'].get('no_price_control') is not True or fit.get('price_control', False):
    raise ValueError('Plain/berry pricing requires the uncontrolled demand fit.')
```

Read this block:

- Line 67: test `fit['settings'].get('no_price_control') is not True or fit.get('price_control...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 68: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 69–70:

```
if fit['settings'].get('novelty_measure', 'previous_share') != 'previous_share':
    raise ValueError('Pricing requires the original previous-share novelty measure.')
```

Read this block:

- Line 69: test `fit['settings'].get('novelty_measure', 'previous_share') != 'previous_share'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 70: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 71–71:

```
grid = np.unique(np.asarray(settings.discounts, dtype=float))
```

Read this block:

- Line 71, `grid`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 72–74:

```
if (grid.size == 0 or not np.isfinite(grid).all() or grid[0] != 0. or grid[-1] > 1.
        or not 0 <= settings.demand_delta <= 1 or settings.batch_size < 1):
    raise ValueError('Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.')
```

Read this block:

- Line 72: test `grid.size == 0 or not np.isfinite(grid).all() or grid[0] != 0.0 or (grid[-1] ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 74: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 75–76:

```
75 settings.output.mkdir(parents=True, exist_ok=True)
76 reports, comparison = {}, []
```

Read this block:

- Line 75: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76, `(reports, comparison)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 77–87:

```
77 for focal in FOCAL_FLAVORS:
78     # Each case uses the same fit and its full three/five-category server table.
79     # The shared solver moves this focal flavor into its priced coordinate.
80     case = SimpleNamespace(**vars(settings))
81     case.focal_flavor = focal
82     case.discounts = grid
83     case.output = settings.output / focal
84     case.output.mkdir(parents=True, exist_ok=True)
85     reports[focal] = pricing.run_households(case)
86     for row in reports[focal]['aggregate']:
87         comparison.append(dict(focal_flavor=focal, **row))
```

Read this block:

- Line 77: for each entry yielded by `FOCAL_FLAVORS`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 80, `case`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 81, `case.focal_flavor`: Set this name to the object currently named `focal`; plain assignment does not copy it.
- Line 82, `case.discounts`: Set this name to the object currently named `grid`; plain assignment does not copy it.
- Line 83, `case.output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84: call `case.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85, `reports[focal]`: Store the result of calling `pricing.run_households(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 86: for each entry yielded by `reports[focal]['aggregate']`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 87: call `comparison.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 89–100:

```
89 pd.DataFrame(comparison).to_csv(settings.output / 'focal_flavor_comparison.csv', index=False)
90 manifest = dict(
91     focal_flavors=list(FOCAL_FLAVORS), demand_categories=list(pricing.FLAVORS),
92     household_identifiers_exported=False,
93     settings={key: str(value) if isinstance(value, Path) else value.tolist()
94               if isinstance(value, np.ndarray) else value for key, value in vars(settings).items()},
95     results=reports, script_sha256=pricing.digest(Path(__file__).resolve()),
96     solver_sha256=pricing.digest(Path(pricing.__file__).resolve()),
97     versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
98 (settings.output / 'run_manifest.json').write_text(json.dumps(manifest, indent=2)+'\n')
99 (settings.output / 'run_notes.txt').write_text(pformat(manifest, sort_dicts=False, width=90)+'\n')
100 print(f'Plain and berry comparison completed: {settings.output}')
```

Read this block:

- Line 89: call `pd.DataFrame(comparison).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 98: call `(settings.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `(settings.output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 100: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `list`: Materialize iterable entries as an ordered Python list.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 101–101:

```
101 return manifest
```

Read this block:

- Line 101: return the object currently named `manifest`; plain assignment does not copy it. This ends the call.

../final_simple/demand/pricing.py

Legacy three-group combined-gamma berry exercise, with .95 discount default. It explicitly rejects five-group fits. Current main code uses separate novelty/stock effects and plain_berry_pricing instead.

Module setup and direct-execution blocks

Source lines 3–15:

```
3  """Empirical berry pricing. Edit Parameters, then run python3 demand/pricing.py.
4
5  The first observed choice becomes t=0 and establishes h0. The firm then chooses
6  p1(h0), observes a basket h1, and chooses terminal p2(h1). Only dynamic pricing
7  is optimized. No coupons is retained solely as a revenue benchmark.
8  """
9
10 # Empirical I/O is permitted only on the research server, never on this computer.
11 from pathlib import Path as _PolicyPath
12 import sys as _policy_sys
13 _policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
   ↪  "Code/privacy/server_data_policy.py").is_file())
14 _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
15 from server_data_policy import server_only, require_server, require_data_path
```

Read this block:

- Line 3: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 11: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 13, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 14: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 15: from `server_data_policy`, import `server_only, require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 16–21:

```
16 from pathlib import Path
17 from types import SimpleNamespace
18 import argparse
19 import json
```

```
20 import os
21 import sys
```

Read this block:

- Line 16: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 17: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 18: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 19: import `json`. An as-name is a convenient local alias, not another library.
- Line 20: import `os`. An as-name is a convenient local alias, not another library.
- Line 21: import `sys`. An as-name is a convenient local alias, not another library.

Source lines 22–31:

```
22 import tempfile
23 from pprint import pformat
24
25 import numpy as np
26 import pandas as pd
27
28 # Use a writable plotting cache, including on a shared research server.
29 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "final_simple_matplotlib"))
30
31 import math
```

Read this block:

- Line 22: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 23: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 25: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 26: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 29: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 31: import `math`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 32–32:

```
32 import hashlib
```

Read this block:

- Line 32: import `hashlib`. An as-name is a convenient local alias, not another library.

Source lines 40–45:

```
40 household_input = None              # Example: Path("/your/results/weekly_inputs.csv")
41 fit_dir = None                      # Example: Path("/your/results")
42 output = Path("/scratch/dtm63837/Kilts_Panel/results") / "empirical_pricing"
43 discounts = np.linspace(0.0, 1.0, 101) # 0%, 1%, ..., 100% berry coupons.
44 demand_delta = 0.95                  # Revenue discount factor; independent of history weight.
45 batch_size = 24                     # Progress interval; each household is solved separately.
```

Read this block:

- Line 40, `household_input`: Set this name to the literal value `None`.
- Line 41, `fit_dir`: Set this name to the literal value `None`.

- Line 42, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 43, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 44, `demand_delta`: Set this name to the literal value `0.95`.
- Line 45, `batch_size`: Set this name to the literal value `24`.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 46–49:

```
46 type_treatment = "posterior-mean"    # Or posterior-integrated as a sensitivity.
47 save_state_policies = False          # True saves every conditional terminal coupon.
48 run_self_tests = False                    # See checks/run_checks.py for checks without CSV access.
49 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 46, `type_treatment`: Set this name to the literal text `'posterior-mean'`.
- Line 47, `save_state_policies`: Set this name to the literal value `False`.
- Line 48, `run_self_tests`: Set this name to the literal value `False`.
- Line 49, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 740–756:

```
740 if __name__ == "__main__":
741     # Collect the ordinary Python variables above into one settings record.
742     # settings.discounts means exactly the variable discounts from the Parameters section.
743     settings = SimpleNamespace(
744         household_input=household_input,
745         fit_dir=fit_dir,
746         output=output,
747         discounts=discounts,
748         demand_delta=demand_delta,
749         batch_size=batch_size,
750         type_treatment=type_treatment,
751         save_state_policies=save_state_policies,
752         self_test=run_self_tests,
753     )
754     if len(sys.argv) > 1:
755         settings = read_command_line(settings)
756     run_pricing(settings)
```

Read this block:

- Line 740: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 743, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 754: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 755, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 756: call `run_pricing` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.

- `run_pricing`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 58.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 58–58:

```
58 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–60:

```
59 x = np.asarray(x)
60 small = np.exp(-np.abs(x))
```

Read this block:

- Line 59, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 60, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 61–61:

```
61 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 61: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 63.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 63–63:

```
63 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 64–66:

```
64 x = np.asarray(x)
65 maximum = np.max(x, axis=axis, keepdims=True)
66 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 64, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 65, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 66, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 67–67:

```
67 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 67: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

persistent draws

Original function: `persistent_draws`. Source begins at line 69.

Generates seeded antithetic integration draws for the legacy combined-gamma model: price sensitivity and gamma. These two dimensions differ from the current split model's three dimensions.

Definition and stated intent:

Source lines 69–69:

```
69 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 70–71:

```
70 if number < 2 or number % 2:
71     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 70: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 71: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 72–72:

```
72 half = np.random.default_rng(seed).standard_normal((number//2, 2))
```

Read this block:

- Line 72, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 73–73:

```
73 return np.vstack([half, -half])
```

Read this block:

- Line 73: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 77.

Recovers persistent price/gamma draw weights from the complete observed history under the legacy combined Xi model. It retains three flavor alternatives and ranks posterior gamma means for reporting. It does not accept a split-history fit.

Definition and stated intent:

Source lines 77–78:

```
77 def posterior(frame, parameters, draws, control=True):
78     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 79–84:

```
79 labels = np.sort(frame.household_code.unique())
80 hh = pd.Categorical(frame.household_code, categories=labels).codes
81 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
```

```
available = np.isfinite(prices) & (prices > 0)
counts = frame[FLAVORS+["outside"]].to_numpy(float)
xi = frame[["xi_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 79, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 80, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 81, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 82, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 84, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 85–85:

```
cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 85, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 86–89:

```
if (not np.isfinite(counts).all() or (counts < 0).any()
        or not np.isfinite(xi).all() or not np.isfinite(cf).all()
        or ((counts[:, :3] > 0) & ~available).any()):
    raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 86: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(xi).a...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 89: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 90–93:

```
90 beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
91 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
92 gamma = parameters["mean_gamma"] + parameters["sd_gamma"]*draws[:, 1]
93 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 90, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 91, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 92, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 93, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 94–99:

```
94 for d, (a, g) in enumerate(zip(alpha, gamma)):
95     u = beta-a*np.nan_to_num(prices)+g*xi+parameters.get("price_control", 0.)*cf
96     u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
97     logp = u-logsumexp(u, axis=1, keepdims=True)
98     ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
99     loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 94: for each entry yielded by `enumerate(zip(alpha, gamma))`, bind/unpack it into `(d, (a, g))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 95, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 96, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 97, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 98, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 99, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.

- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 100–106:

```
100 weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
101 nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
102 means = weights @ gamma
103 # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
104 order = np.argsort(means, kind="stable")
105 groups = np.full(len(labels), "middle", dtype=object)
106 n = len(labels)//4
```

Read this block:

- Line 100, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 101, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 102, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 104, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 105, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 106, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 107–108:

```
107 if n < 1:
108     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 107: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 108: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 109–115:

```
109 groups[order[:n]], groups[order[-n:]] = "low", "high"
110 population_q = np.quantile(gamma, [.25, .75])
111 table = pd.DataFrame(dict(household_code=labels, group=groups, gamma_mean=means,
112                           gamma_sd=np.sqrt(np.maximum(weights @ gamma**2-means**2, 0)),
113                           alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
114                           probability_gamma_below_draw_q25=weights @ (gamma <= population_q[0]),
```

```
115                         probability_gamma_above_draw_q75=weights @ (gamma >= population_q[1])))
```

Read this block:

- Line 109, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 110, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 111, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 116–116:

```
116 return table, weights, alpha, gamma, beta, nll
```

Read this block:

- Line 116: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 126.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 126–127:

```
126 def compositions(capacity):
127     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 128–129:

```
128 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
129                  for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 128: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 133.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 133–138:

```
133 def forecast_context(frame, households):
134     """First observed choice establishes h0; price the next two consecutive weeks.
135 
136     Carry the initial observation's menu, posted prices and controls forward.
137     Type estimation still uses the complete observed sample.
138     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 139–145:

```
139 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
140 first = first.set_index("household_code").loc[households.household_code]
141 counts = first[FLAVORS].to_numpy(float)
142 totals = counts.sum(axis=1, keepdims=True)
143 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
144 # Saved stocks precede the observed choice; add its basket exactly once.
145 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 139, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 140, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 141, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 142, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 143, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 145, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 146–151:

```
146 novelty = shares.sum(axis=1, keepdims=True)-shares
147 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
148 available = np.isfinite(prices) & (prices > 0)
149 eligible = available[:, 1]
150 households["berry_eligible"] = eligible
151 initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 146, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 147, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 148, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 149, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 150, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 151, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 152–156:

```
152 households["initial_observed_week"] = initial_weeks
153 households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
154 households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
155 cf_cols = ["cf_"+f for f in FLAVORS]
156 cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 152, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 153, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 154, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 155, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 156, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 157–158:

```
157 if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
158     raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 157: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 158: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 159–160:

```
159 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
160             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 159: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 171.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 171–180:

```
171 def targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
172                      delta=1., capacity=7, habit_weight=.7, control_coefficient=0.,
173                      batch_size=24, type_treatment="posterior-mean", state_output=None):
174     """Choose household-specific p1(h0) and terminal p2(h1) by backward induction.
175 
176     Work through ONE household at a time. This makes type-specific pricing
177     explicit and avoids the old extra household dimension in every array.
178     For this household, arrays run over possible types, baskets, and coupons.
179     batch_size only controls how frequently progress is reported.
180     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `gamma`, `beta`, `discounts`, `delta`, `capacity`, `habit_weight`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 181–188:

```
181 baskets = compositions(capacity)
182 number_states = len(baskets)
183 number_coupons = len(discounts)
184 state_numbers = np.arange(number_states)
185
186 # A basket records [other, berry, plain, outside]. History uses inside shares.
187 inside_counts = baskets[:, :3]
188 inside_total = inside_counts.sum(axis=1, keepdims=True)
```

Read this block:

- Line 181, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 182, `number_states`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 183, `number_coupons`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 184, `state_numbers`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 187, `inside_counts`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 188, `inside_total`: Set this name to the result of calling `inside_counts.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 189–199:

```
189 basket_shares = np.divide(
190     inside_counts, inside_total,
191     out=np.zeros_like(inside_counts, dtype=float), where=inside_total > 0)
192 basket_novelty = basket_shares.sum(axis=1, keepdims=True) - basket_shares
193 log_multinomial_coefficient = (
194     math.lgamma(capacity + 1)
195     - np.vectorize(math.lgamma)(baskets + 1).sum(axis=1))
196
197 eligible_households = np.flatnonzero(context["eligible"])
198 targeted = households.group.isin(["low", "high"]).to_numpy()
199 rows = []
```

Read this block:

- Line 189, `basket_shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 192, `basket_novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 193, `log_multinomial_coefficient`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 197, `eligible_households`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 198, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 199, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 200–201:

```
maximum_mass_error = 0.
wrote_states = False
```

Read this block:

- Line 200, `maximum_mass_error`: Set this name to the literal value `0.0`.
- Line 201, `wrote_states`: Set this name to the literal value `False`.

Source lines 203–244:

```
for number_done, household in enumerate(eligible_households, start=1):
    prices = context["prices"][household]
    available = context["available"][household]
    stock = context["stocks"][household]
    novelty = context["novelty"][household]
    control = context["cf"][household]

    # Default: the firm knows the household's estimated (alpha, gamma).
    # Sensitivity: retain the full probability distribution of possible types.
    if type_treatment == "posterior-mean":
        price_sensitivity = np.array([weights[household] @ alpha])
        history_coefficient = np.array([weights[household] @ gamma])
        type_weights = np.ones(1)
    else:
        price_sensitivity = alpha
        history_coefficient = gamma
        type_weights = weights[household]

    xi1 = (1 - habit_weight) * novelty - habit_weight * stock
    xi2 = ((1 - habit_weight) * basket_novelty
           - habit_weight * (stock + basket_shares))

    # u1: possible type x flavor. u2: possible type x basket h1 x flavor.
    utility1 = (beta - price_sensitivity[:, None] * prices
                + history_coefficient[:, None] * xi1
                + control_coefficient * control)
    utility2 = (beta - price_sensitivity[:, None, None] * prices
                + history_coefficient[:, None, None] * xi2
                + control_coefficient * control)
    utility1 = np.where(available, utility1, -np.inf)
    utility2 = np.where(available, utility2, -np.inf)

    # STEP A: terminal outcomes for every candidate coupon and every h1.
    # Metrics are [revenue, berry units, coupon expenditure].
    terminal = []
    for discount in discounts:
        utility = utility2.copy()
        utility[:, :, 1] += price_sensitivity[:, None] * prices[1] * discount
        outside = np.zeros((*utility.shape[:-1], 1))
```

```
        all_utilities = np.concatenate([utility, outside], axis=-1)
        log_denominator = logsumexp(all_utilities, axis=-1)
        berry_quantity = capacity * np.exp(utility[:, :, 1] - log_denominator)
```

Source lines 245–286:

```
    offered_price = prices[1] * (1 - discount)
    terminal.append(np.stack([
        berry_quantity * offered_price,
        berry_quantity,
        berry_quantity * prices[1] * discount], axis=-1))
terminal = np.asarray(terminal)

first = np.zeros((number_coupons, 3))
second = np.zeros((number_coupons, 3))
paths = []

# STEP B: try each p1(h0), calculate its state probabilities, and then
# select the best p2 for each state. Optimize BEFORE averaging revenue.
for action, discount in enumerate(discounts):
    utility = utility1.copy()
    utility[:, 1] += price_sensitivity * prices[1] * discount
    all_utilities = np.column_stack([utility, np.zeros(len(type_weights))])
    log_probability = all_utilities - logsumexp(all_utilities, axis=1, keepdims=True)
    terms = np.zeros((len(type_weights), number_states, 4))
    np.multiply(baskets[None, :, :], log_probability[:, None, :],
                out=terms, where=baskets[None, :, :] > 0)
    basket_probability = np.exp(log_multinomial_coefficient + terms.sum(axis=-1))
    error = float(np.abs(basket_probability.sum(axis=1) - 1).max())
    maximum_mass_error = max(maximum_mass_error, error)

    # Joint(type,h1) = initial type weight x Pr(h1 | type,p1).
    joint = type_weights[:, None] * basket_probability
    state_probability = joint.sum(axis=0)
    berry_quantity = capacity * (type_weights * np.exp(log_probability[:, 1])).sum()
    first[action] = [berry_quantity * prices[1] * (1 - discount),
                     berry_quantity,
                     berry_quantity * prices[1] * discount]

    # In the uncertainty sensitivity, integrate possible types before
    # choosing a terminal coupon: the firm cannot observe the true draw.
    # Result dimensions: terminal coupon x basket h1 x metric.
    weighted_terminal = (terminal * joint[None, :, :, None]).sum(axis=1)
    best_terminal_action = weighted_terminal[:, :, 0].argmax(axis=0)
    selected = weighted_terminal[best_terminal_action, state_numbers]
    second[action] = selected.sum(axis=0)
    if action == 0:
        no_coupon_second = weighted_terminal[0].sum(axis=0)
```

Source lines 287–328:

```
    paths.append((best_terminal_action, state_probability))

# STEP C: choose the first coupon by total expected discounted revenue.
dynamic_action = int((first[:, 0] + delta * second[:, 0]).argmax())
if not targeted[household]:
    dynamic_action = 0

# Keep posted prices as a benchmark. Only dynamic pricing is optimized.
for policy, action in [("no_coupon", 0), ("personalized_dynamic", dynamic_action)]:
    use_coupons = targeted[household] and policy == "personalized_dynamic"
    initial = first[action]
```

```
    best_terminal_action, state_probability = paths[action]
    if use_coupons:
        terminal_metrics = second[action]
        terminal_coupon = discounts[best_terminal_action]
    else:
        terminal_metrics = no_coupon_second
        terminal_coupon = np.zeros(number_states)

    p1 = prices[1] * (1 - discounts[action])
    p2 = prices[1] * (1 - terminal_coupon)
    mean_p2 = float(state_probability @ p2)
    ratio = np.divide(p1, p2, out=np.full(number_states, np.nan), where=p2 > 0)
    positive_mass = state_probability > 1e-14
    expected_ratio = np.nan
    if np.isfinite(ratio[positive_mass]).all():
        expected_ratio = float(state_probability[positive_mass] @ ratio[positive_mass])

    row = dict(
        household_code=households.iloc[household].household_code,
        group=households.iloc[household].group, policy=policy,
        discount1=discounts[action],
        expected_discount2=float(state_probability @ terminal_coupon),
        p1=p1, expected_p2=mean_p2,
        ratio_of_expected_prices=p1 / mean_p2 if mean_p2 > 0 else np.nan,
        expected_price_ratio=expected_ratio,
        revenue1=initial[0], revenue2=terminal_metrics[0],
        discounted_revenue=initial[0] + delta * terminal_metrics[0],
        berry_units1=initial[1], berry_units2=terminal_metrics[1],
        coupon_expenditure=initial[2] + delta * terminal_metrics[2])
    for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
        branch_mass = state_probability[branch].sum()
```

Source lines 329–348:

```
        row["probability_" + label] = branch_mass
        row["mean_p2_given_" + label] = np.nan
        if branch_mass > 0:
            row["mean_p2_given_" + label] = float(
                state_probability[branch] @ p2[branch] / branch_mass)
    rows.append(row)

    if state_output is not None and use_coupons:
        detail = pd.DataFrame(dict(
            household_code=row["household_code"], group=row["group"], policy=policy,
            state=state_numbers, other=baskets[:, 0], berry=baskets[:, 1],
            plain=baskets[:, 2], outside=baskets[:, 3], probability=state_probability,
            discount1=discounts[action], discount2=terminal_coupon, p1=p1, p2=p2,
            price_ratio=ratio))
        detail.drop(columns=["household_code"]).to_csv(state_output, mode="a" if wrote_states else "w",
                      header=not wrote_states, index=False, compression="gzip")
        wrote_states = True

if number_done % (batch_size * 40) == 0 or number_done == len(eligible_households):
    print(f"Targeted pricing: integrated {number_done}/{len(eligible_households)} eligible households", flush=True)
```

Read this block:

- Line 203: for each entry yielded by `enumerate(eligible_households, start=1)`, bind/unpack it into `(number_done, household)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 204, `prices`: Set this name to the entry/slice selected by `context['prices'][household]`; brackets select from the object to their left.

- Line 205, `available`: Set this name to the entry/slice selected by `context['available'][household]`; brackets select from the object to their left.
- Line 206, `stock`: Set this name to the entry/slice selected by `context['stocks'][household]`; brackets select from the object to their left.
- Line 207, `novelty`: Set this name to the entry/slice selected by `context['novelty'][household]`; brackets select from the object to their left.
- Line 208, `control`: Set this name to the entry/slice selected by `context['cf'][household]`; brackets select from the object to their left.
- Line 212: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 213, `price_sensitivity`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 214, `history_coefficient`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 215, `type_weights`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 217, `price_sensitivity`: Set this name to the object currently named `alpha`; plain assignment does not copy it.
- Line 218, `history_coefficient`: Set this name to the object currently named `gamma`; plain assignment does not copy it.
- Line 219, `type_weights`: Set this name to the entry/slice selected by `weights[household]`; brackets select from the object to their left.
- Line 221, `xi1`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 222, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 226, `utility1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 229, `utility2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 232, `utility1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 233, `utility2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 237, `terminal`: Set this name to the list `[]`.
- Line 238: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 239, `utility`: Set this name to the result of calling `utility2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 240: update `utility[:, :, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 241, `outside`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 242, `all_utilities`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 243, `log_denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 244, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 245, `offered_price`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 246: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 250, `terminal`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 252, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method

chain are shown above.
- Line 253, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 254, `paths`: Set this name to the list `[]`.
- Line 258: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(action, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 259, `utility`: Set this name to the result of calling `utility1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 260: update `utility[:, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 261, `all_utilities`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 262, `log_probability`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 263, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 264: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 266, `basket_probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 267, `error`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 268, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 271, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 272, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 273, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 274, `first[action]`: Store a list containing 3 displayed entries in the selected entry/column.
- Line 281, `weighted_terminal`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 282, `best_terminal_action`: Set this name to the result of calling `weighted_terminal[:, :, 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 283, `selected`: Set this name to the entry/slice selected by `weighted_terminal[best_terminal_action, state_numbers]`; brackets select from the object to their left.
- Line 284, `second[action]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 285: test `action == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 286, `no_coupon_second`: Set this name to the result of calling `weighted_terminal[0].sum(...)`; the arguments and preceding method chain are shown above.
- Line 287: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 290, `dynamic_action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 291: test `not targeted[household]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 292, `dynamic_action`: Set this name to the literal value `0`.
- Line 295: for each entry yielded by `[('no_coupon', 0), ('personalized_dynamic', dynamic_action)]`, bind/unpack it into `(policy, action)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 296, `use_coupons`: Set this name to the Boolean condition `targeted[household] and policy == 'personalized_dynamic'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 297, `initial`: Set this name to the entry/slice selected by `first[action]`; brackets select from the

object to their left.
- Line 298, `(best_terminal_action, state_probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 299: test `use_coupons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 300, `terminal_metrics`: Set this name to the entry/slice selected by `second[action]`; brackets select from the object to their left.
- Line 301, `terminal_coupon`: Set this name to the entry/slice selected by `discounts[best_terminal_action]`; brackets select from the object to their left.
- Line 303, `terminal_metrics`: Set this name to the object currently named `no_coupon_second`; plain assignment does not copy it.
- Line 304, `terminal_coupon`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 306, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 307, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 308, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 309, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 310, `positive_mass`: Set this name to the Boolean condition `state_probability > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 311, `expected_ratio`: Set this name to the attribute `np.nan` of the object before the dot.
- Line 312: test `np.isfinite(ratio[positive_mass]).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 313, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 315, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 327: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 328, `branch_mass`: Set this name to the result of calling `state_probability[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 329, `row['probability_' + label]`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 330, `row['mean_p2_given_' + label]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 331: test `branch_mass > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 332, `row['mean_p2_given_' + label]`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 334: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 336: test `state_output is not None and use_coupons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 337, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 343: call `detail.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 345, `wrote_states`: Set this name to the literal value `True`.
- Line 347: test `number_done % (batch_size * 40) == 0 or number_done == len(eligible_households)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 348: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 350–351:

```
350 if maximum_mass_error > 1e-9:
351     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 350: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 351: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 352–357:

```
352 table = pd.DataFrame(rows)
353 baseline = table.loc[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
```

```
354 table["baseline_revenue"] = table.household_code.map(baseline)
355 table["revenue_change"] = table.discounted_revenue - table.baseline_revenue
356 table["revenue_change_pct"] = 100 * table.revenue_change / table.baseline_revenue
357 comparison = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 352, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 353, `baseline`: Set this name to the attribute `table.loc[table.policy == 'no_coupon'].set_index('househo...` of the object before the dot.
- Line 354, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 355, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 356, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 357, `comparison`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 358–359:

```
358 if (comparison.personalized_dynamic + 1e-8 < comparison.no_coupon).any():
359     raise AssertionError("Dynamic pricing does not dominate the feasible no-coupon benchmark.")
```

Read this block:

- Line 358: test `(comparison.personalized_dynamic + 1e-08 < comparison.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 359: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 360–360:

```
360 return table, maximum_mass_error
```

Read this block:

- Line 360: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

digest

Original function: `digest`. Source begins at line 368.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 368–368:

```
368 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 369–369:

```
369 h = hashlib.sha256()
```

Read this block:

- Line 369, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 370–372:

```
370 with path.open("rb") as handle:
371     for chunk in iter(lambda: handle.read(1024*1024), b""):
372         h.update(chunk)
```

Read this block:

- Line 370: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 371: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 372: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 373–373:

```
373 return h.hexdigest()
```

Read this block:

- Line 373: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

run households

Original function: `run_households`. Source begins at line 378.

Validates a converged three-group combined-gamma fit, preserves the caller settings separately from saved fit settings, reconstructs the likelihood and solves berry pricing. Five-group fits are rejected explicitly. The current main pipeline instead uses plain_berry_pricing and the split solver.

Definition and stated intent:

Source lines 378–380:

```
378 def run_households(settings):
379     # This branch is entered only when the person running the script supplies
380     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 381–382:

```
require_data_path(settings.fit_dir)
fit = json.loads((settings.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 381: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 382, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 383–384:

```
if fit["model"] != "heterogeneous" or not fit.get("converged", False):
    raise ValueError("This exercise requires a converged single-normal heterogeneous-gamma fit.")
```

Read this block:

- Line 383: test `fit['model'] != 'heterogeneous' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 384: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 385–386:

```
if fit['settings'].get('flavor_spec', 'three') != 'three':
    raise ValueError('Legacy combined-gamma pricing supports only three groups; use plain_berry_pricing.py for
    ↪ current three/five fits.')
```

Read this block:

- Line 385: test `fit['settings'].get('flavor_spec', 'three') != 'three'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 386: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 387–387:

```
estimates = pd.read_csv(settings.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 387, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 388–389:

```
if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
    raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 388: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 389: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 390–392:

```
390 parameters = dict(zip(estimates.parameter, estimates.estimate))
391 saved_settings = fit["settings"]
392 capacity = saved_settings["capacity"]
```

Read this block:

- Line 390, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 391, `saved_settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 392, `capacity`: Set this name to the entry/slice selected by `saved_settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 393–394:

```
393 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
394     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 393: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 394: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 395–398:

```
395 frame = pd.read_csv(settings.household_input, dtype={"household_code": str}, parse_dates=["week"])
396 required = ["household_code", "week", "outside", *FLAVORS]
397 required += [prefix+f for prefix in ["price_", "stock_", "novelty_", "xi_"] for f in FLAVORS]
398 control = not saved_settings.get("no_price_control", False)
```

Read this block:

- Line 395, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 396, `required`: Set this name to a list containing 4 displayed entries.
- Line 397: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 398, `control`: Set this name to the negated or sign-adjusted expression `not saved_settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 399–400:

```
399 if control:
400     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 399: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 400: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 401–401:

```
401 missing = set(required)-set(frame)
```

Read this block:

- Line 401, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 402–403:

```
402 if missing:
403     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 402: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 403: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 404–405:

```
404 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪ "week"]).any():
405     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 404: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 405: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 406–407:

```
406 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
407     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 406: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 407: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 408–408:

```
408 weight = saved_settings.get("habit_weight", .7)
```

Read this block:

- Line 408, `weight`: Set this name to the result of calling `saved_settings.get(...)`; the arguments and preceding method chain are shown above.

Source lines 409–412:

```
409 for f in FLAVORS:
410     expected = (1-weight)*frame["novelty_"+f]-weight*frame["stock_"+f]
411     if not np.allclose(expected, frame["xi_"+f], atol=1e-8):
412         raise ValueError("Saved Xi does not match the fit's history definition.")
```

Read this block:

- Line 409: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 410, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 411: test `not np.allclose(expected, frame['xi_' + f], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 412: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 413–414:

```
413 draws = persistent_draws(saved_settings["draws"], saved_settings["seed"])
414 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 413, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 414, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 416–417:

```
416 if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
417     raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 416: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 417: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 418–419:

```
418 context = forecast_context(frame, households)
419 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 418, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 419, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 420–421:

```
420 if any(n == 0 for n in counts):
421     raise ValueError("Each gamma segment must have berry-eligible forecast households.")
```

Read this block:

- Line 420: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 421: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 422–433:

```
422 table, mass_error = targeted_coupons(
423     context, households, weights, alpha, gamma, beta, settings.discounts,
424     settings.demand_delta, capacity, weight,
425     parameters.get("price_control", 0.) if control else 0., settings.batch_size,
426     settings.type_treatment,
```

```
427     settings.output / "targeted_state_policies.csv.gz" if settings.save_state_policies else None)
428 households.drop(columns=["household_code"]).to_csv(settings.output / "household_posterior_summary.csv",
    ↪  index=False)
429 table.drop(columns=["household_code"]).to_csv(settings.output / "targeted_household_results.csv", index=False)
430 totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
431           "berry_units1", "berry_units2", "coupon_expenditure"]
432 aggregate = table.groupby("policy", sort=False)[totals].sum()
433 aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
```

Read this block:

- Line 422, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 428: call `households.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 429: call `table.drop(columns=['household_code']).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 430, `totals`: Set this name to a list containing 8 displayed entries.
- Line 432, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 433, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 434–440:

```
434 aggregate.to_csv(settings.output / "targeted_revenue_aggregate.csv")
435 group = table.groupby(["policy", "group"], sort=False)[totals].sum()
436 group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
437 means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
438                                              "ratio_of_expected_prices", "expected_price_ratio"]].mean()
439 group = group.join(means)
440 group["eligible_households"] = table.groupby(["policy", "group"]).size()
```

Read this block:

- Line 434: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 435, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 436, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 437, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.
- Line 439, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 440, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 441–450:

```
group.to_csv(settings.output / "targeted_revenue_by_gamma_group.csv")
households.groupby("group").agg(households=("household_code", "size"),
                                berry_eligible=("berry_eligible", "sum"),
                                gamma_mean=("gamma_mean", "mean"),
                                gamma_mean_min=("gamma_mean", "min"),
                                gamma_mean_max=("gamma_mean", "max"),
                                mean_posterior_sd=("gamma_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(settings.output /
 ↪  "household_group_summary.csv")
make_targeting_figures(households, table, aggregate, settings.output, delta=settings.demand_delta)
```

Read this block:

- Line 441: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 442: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 450: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 451–461:

```
return dict(households=len(households), eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
            reconstructed_nll=nll, basket_mass_error=mass_error,
            input_sha256=digest(settings.household_input), fit_sha256=digest(settings.fit_dir / "fit.json"),
            parameter_sha256=digest(settings.fit_dir / "point_estimates.csv"),
            posterior_draws=len(draws), posterior_seed=saved_settings["seed"],
            capacity=capacity, habit_weight=weight, price_control=control,
            initial_history="first_observed_basket", pricing_periods=2,
            baskets=len(compositions(capacity)),
            discount_grid=settings.discounts.tolist(), demand_delta=settings.demand_delta,
            type_treatment=settings.type_treatment,
            aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 451: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

plotting

Original function: `plotting`. Source begins at line 469.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 469–469:

```
469 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 470–473:

```
470 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
471 import matplotlib
472 matplotlib.use("Agg")
473 import matplotlib.pyplot as plt
```

Read this block:

- Line 470: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 471: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 472: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 473: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 474–474:

```
474 return plt
```

Read this block:

- Line 474: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 476.

Draws legacy posterior gamma, berry revenue and discount comparisons. These figures are outputs of the earlier three-group combined-gamma exercise, not automatically updated results for the new split-history pipeline.

Definition and stated intent:

Source lines 476–477:

```
476 def make_targeting_figures(households, table, aggregate, output, delta=.95):
477     """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 478–483:

```
478 plt = plotting()
479 groups = ["low", "middle", "high"]
480 labels = ["Low gamma\n(bottom 25%)", "Middle gamma\n(middle 50%)", "High gamma\n(top 25%)"]
481 colors = ["#3575AD", "#999999", "#D17925"]
482 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
483 bins = np.histogram_bin_edges(households.gamma_mean, bins=30)
```

Read this block:

- Line 478, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 479, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 480, `labels`: Set this name to the list `['Low gamma\n(bottom 25%)', 'Middle gamma\n(middle 50%)',...`.
- Line 481, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 482, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 483, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 484–486:

```
484 for g, label, color in zip(groups, labels, colors):
485     axes[0].hist(households.loc[households.group == g, "gamma_mean"],
486                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 484: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 485: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 487–494:

```
487 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $\gamma$)",
488             ylabel="Number of households")
489 axes[0].legend(fontsize=9)
490 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
491 x = np.arange(len(groups))
492 # Average over state-contingent terminal prices first, discount, then total.
493 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
494 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 487: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 489: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 490, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 491, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 493, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 494: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 495–501:

```
495 axes[1].set_xticks(x, labels)
496 axes[1].set(title="Total Expected Revenue",
497             xlabel="Consumer type group", ylabel="Total Expected Revenue")
498 axes[1].set_ylim(bottom=0)
499 axes[1].grid(axis="y", alpha=.2)
500 axes[1].set_axisbelow(True)
501 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 495: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 496: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 498: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 499: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 500: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 501: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 502–506:

```
fig.text(.5, .015,
         f"First-period revenue + {delta:g} × expected terminal revenue. "
         "Low/high types receive personalized berry offers; middle types receive no coupons.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 502: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 506: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 507–508:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 507: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 508: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 509–516:

```
plt.close(fig)

fig, ax = plt.subplots(figsize=(9, 5))
x = np.arange(3)
means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
 ↪  "expected_discount2"]].mean().reindex(groups)
ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
ax.bar(x+.18, 100*means.expected_discount2, width=.36,
       label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 509: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 511, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 512, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 513, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 514: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 515: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 517–523:

```
517 ax.set_xticks(x, labels)
518 ax.set(xlabel="Consumer type group", ylim=(0, 105),
519        ylabel="Mean optimal berry discount (% of posted price)")
520 ax.grid(axis="y", alpha=.2)
521 ax.set_axisbelow(True)
522 ax.legend(fontsize=8, loc="upper left")
523 fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 517: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 518: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 520: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 521: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 522: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 523: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 524–526:

```
524 fig.text(.5, .015, "Household-weighted means among berry-eligible households. Middle group receives no coupons by
    ↪  policy design.",
525          ha="center", fontsize=9)
526 fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 524: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 526: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 527–528:

```
527 for extension in ["png", "pdf"]:
528     fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 527: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 528: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 529–533:

```
529 plt.close(fig)
530
531 dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
532     households[["household_code", "gamma_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
533 fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 529: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 533, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 534–546:

```
534 for column, (variable, xlabel) in enumerate([
535         ("gamma_mean", r"Estimated household type (posterior mean $\gamma$)"),
536         ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
537     for row, (discount, title) in enumerate([
538             ("discount1", r"Optimal discount at $p_1(h_0)$"),
539             ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
540         ax = axes[row, column]
541         for g, label, color in zip(groups, labels, colors):
542             subset = dynamic.loc[dynamic.group == g]
543             ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
544                        color=color, label=label.replace("\n", " "))
545         ax.set(xlabel=xlabel, ylabel="Berry discount (% of posted price)", title=title, ylim=(-2, 102))
546         ax.grid(alpha=.2)
```

Read this block:

- Line 534: for each entry yielded by `enumerate([('gamma_mean', 'Estimated household type (posterior...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 537: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 540, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 541: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 542, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 543: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 545: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 546: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 547–551:

```
547 axes[0, 0].legend(fontsize=8)
548 fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
549 fig.text(.5, .015, "Each point is one berry-eligible household. Terminal offers average over the household's
    ↪ realized h1 histories.",
550         ha="center", fontsize=9)
```

```
551 fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 547: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 548: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 549: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 551: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 552–553:

```
552 for extension in ["png", "pdf"]:
553     fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 552: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 553: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 554–554:

```
554 plt.close(fig)
```

Read this block:

- Line 554: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 562.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 562–563:

```
562 def self_test():
563     """Generated synthetic households only; no user-supplied input is accessed."""
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 564–568:

```
564 rng = np.random.default_rng(41)
565 parameters = dict(inside_constant=.8, beta_berry=-.2, beta_plain=-.1,
566                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
567                   mean_gamma=-.1, sd_gamma=.2)
568 rows = []
```

Read this block:

- Line 564, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 565, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 568, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 569–582:

```
569 for household in range(12):
570     stock, previous = np.zeros(3), np.zeros(3)
571     for t in range(3):
572         counts = rng.multinomial(2, [.25, .3, .2, .25])
573         novelty = previous.sum()-previous
574         row = dict(household_code=f"synthetic_{household:02}",
575                    week=pd.Timestamp("2020-01-04")+pd.Timedelta(weeks=t), outside=counts[3])
576         for j, flavor in enumerate(FLAVORS):
577             row.update({flavor: counts[j], "price_"+flavor: 1.+.1*j,
578                         "stock_"+flavor: stock[j], "novelty_"+flavor: novelty[j],
579                         "xi_"+flavor: .3*novelty[j]-.7*stock[j]})
580         rows.append(row)
581         previous = counts[:3]/max(counts[:3].sum(), 1)
582         stock += previous
```

Read this block:

- Line 569: for each entry yielded by `range(12)`, bind/unpack it into `household` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 570, `(stock, previous)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 571: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 572, `counts`: Set this name to the result of calling `rng.multinomial(...)`; the arguments and preceding method chain are shown above.
- Line 573, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 574, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 576: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 577: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 580: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 581, `previous`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 582: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 583–585:

```
583 frame = pd.DataFrame(rows)
584 draws = persistent_draws(8, 19)
585 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, False)
```

Read this block:

- Line 583, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 584, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 585, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 586–586:

```
586 assert np.allclose(weights.sum(axis=1), 1.)
```

Read this block:

- Line 586: require `np.allclose(weights.sum(axis=1), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 587–587:

```
587 scalar_ll = []
```

Read this block:

- Line 587, `scalar_ll`: Set this name to the list `[]`.

Source lines 588–593:

```
588 for a, g in zip(alpha, gamma):
589     value = 0.
590     for row in rows[:3]:
591         u = [beta[j]-a*row["price_"+f]+g*row["xi_"+f] for j, f in enumerate(FLAVORS)]+[0.]
592         value += np.array([row[f] for f in FLAVORS]+[row["outside"]]) @ (np.array(u)-logsumexp(u))
593     scalar_ll.append(value)
```

Read this block:

- Line 588: for each entry yielded by `zip(alpha, gamma)`, bind/unpack it into `(a, g)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 589, `value`: Set this name to the literal value `0.0`.
- Line 590: for each entry yielded by `rows[:3]`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 591, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 592: update `value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 593: call `scalar_ll.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 594–594:

```
594 assert np.allclose(weights[0], np.exp(scalar_ll-logsumexp(scalar_ll)))
```

Read this block:

- Line 594: require `np.allclose(weights[0], np.exp(scalar_ll - logsumexp(scalar_ll)))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 595–596:

```
595 context = forecast_context(frame, households)
596 discounts = np.array([0., .25, .5])
```

Read this block:

- Line 595, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 596, `discounts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 597–604:

```
597 for treatment in ["posterior-mean", "posterior-integrated"]:
598     table, error = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
599                                     capacity=2, batch_size=4, type_treatment=treatment)
600     assert error < 1e-12 and len(table) == 24
601     # Accounting and treating the middle segment as a holdout.
602     assert np.allclose(table.discounted_revenue+table.coupon_expenditure,
603                        1.1*(table.berry_units1+table.berry_units2))
604     assert np.allclose(table.loc[table.group == "middle", "revenue_change"], 0.)
```

Read this block:

- Line 597: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 598, `(table, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 600: require `error < 1e-12 and len(table) == 24` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 602: require `np.allclose(table.discounted_revenue + table.coupon_expenditure, 1.1 * (table.berr...` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 604: require `np.allclose(table.loc[table.group == 'middle', 'revenue_change'], 0.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 605–606:

```
605 no_history, _ = targeted_coupons(context, households, weights, np.ones_like(alpha),
606                                  np.zeros_like(gamma), beta, discounts, capacity=2)
```

Read this block:

- Line 605, `(no_history, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 607–607:

```
607 assert np.allclose(no_history.revenue1, no_history.revenue2)
```

Read this block:

- Line 607: require `np.allclose(no_history.revenue1, no_history.revenue2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 608–608:

```
608 assert np.allclose(no_history.discount1, no_history.expected_discount2)
```

Read this block:

- Line 608: require `np.allclose(no_history.discount1, no_history.expected_discount2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 610–613:

```
610 swapped = households.copy()
611 swapped["group"] = swapped.group.replace({"low": "high", "high": "low"})
612 changed, _ = targeted_coupons(context, swapped, weights, alpha, gamma, beta, discounts, capacity=2)
613 original, _ = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts, capacity=2)
```

Read this block:

- Line 610, `swapped`: Set this name to the result of calling `households.copy(...)`; the arguments and preceding method chain are shown above.
- Line 611, `swapped['group']`: Store the result of calling `swapped.group.replace(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 612, `(changed, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 613, `(original, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 614–614:

```
614 assert np.allclose(changed.discount1, original.discount1)
```

Read this block:

- Line 614: require `np.allclose(changed.discount1, original.discount1)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 615–615:

```
615 assert np.allclose(changed.discounted_revenue, original.discounted_revenue)
```

Read this block:

- Line 615: require `np.allclose(changed.discounted_revenue, original.discounted_revenue)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 616–616:

```
616 assert len(compositions(7)) == 120
```

Read this block:

- Line 616: require `len(compositions(7)) == 120` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 617–621:

```
617 restricted = {k: v.copy() for k,v in context.items()}
618 restricted["available"][:, 2] = False
619 restricted["prices"][:, 2] = 0.
620 _, error = targeted_coupons(restricted, households, weights, alpha, gamma, beta,
621                              discounts[:1], capacity=7, batch_size=4)
```

Read this block:

- Line 617, `restricted`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 618, `restricted['available'][:, 2]`: Store the literal value `False` in the selected entry/column.
- Line 619, `restricted['prices'][:, 2]`: Store the literal value `0.0` in the selected entry/column.

- Line 620, `(_, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 622–622:

```
622 assert error < 1e-12
```

Read this block:

- Line 622: require `error < 1e-12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 623–664:

```
623 with tempfile.TemporaryDirectory(prefix="targeting_synthetic_") as directory:
624     root = Path(directory)
625     synthetic_input = root / "synthetic_records.csv"
626     frame.to_csv(synthetic_input, index=False)
627     pd.DataFrame({"parameter": list(parameters), "estimate": list(parameters.values()), "at_bound":
    ↪  False}).to_csv(root / "point_estimates.csv", index=False)
628     fit = dict(model="heterogeneous", converged=True, nll=nll, weeks=len(frame), households=12,
629                settings=dict(capacity=2, draws=8, seed=19, no_price_control=True, habit_weight=.7))
630     (root / "fit.json").write_text(json.dumps(fit))
631     settings = argparse.Namespace(fit_dir=root, household_input=synthetic_input, output=root,
632                                   discounts=discounts, demand_delta=1., batch_size=4,
633                                   type_treatment="posterior-mean", save_state_policies=True)
634     report = run_households(settings)
635     assert report["households"] == 12
636     states = pd.read_csv(root / "targeted_state_policies.csv.gz")
637     assert "household_code" not in states.columns
638     expected_households = int(households.group.isin(["low", "high"]).sum())
639     assert np.allclose(states.groupby("policy").probability.sum(), expected_households)
640     # Independent exhaustive two-period calculation for the first targeted
641     # household: all first actions and all terminal actions at each basket.
642     household = int(np.flatnonzero(households.group.eq("low").to_numpy())[0])
643     a, g = weights[household] @ alpha, weights[household] @ gamma
644     basket_rows = compositions(2)
645     values = []
646     for d1 in discounts:
647         prices = context["prices"][household].copy()
648         prices[1] *= 1-d1
649         xi = .3*context["novelty"][household]-.7*context["stocks"][household]
650         u = np.r_[beta-a*prices+g*xi, 0.]
651         prob = np.exp(u-logsumexp(u))
652         value = 2*prob[1]*prices[1]
653         continuation = 0.
654         for basket in basket_rows:
655             mass = math.factorial(2)/math.prod(math.factorial(int(n)) for n in basket)*np.prod(prob**basket)
656             x = basket[:3]/max(basket[:3].sum(), 1)
657             xi2 = .3*(x.sum()-x)-.7*(context["stocks"][household]+x)
658             returns = []
659             for d2 in discounts:
660                 second_prices = context["prices"][household].copy()
661                 second_prices[1] *= 1-d2
662                 u2 = np.r_[beta-a*second_prices+g*xi2, 0.]
663                 prob2 = np.exp(u2-logsumexp(u2))
664                 returns.append(2*prob2[1]*second_prices[1])
```

Source lines 665–670:

```
        continuation += mass*max(returns)
    values.append(value+continuation)
actual = pd.read_csv(root / "targeted_household_results.csv")
assert "household_code" not in actual.columns
predicted = actual.loc[actual.policy.eq("personalized_dynamic"), "discounted_revenue"]
assert np.isclose(predicted, max(values)).any()
```

Read this block:

- Line 623: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 624, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 625, `synthetic_input`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 626: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 627: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 628, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 630: call `(root / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631, `settings`: Set this name to the result of calling `argparse.Namespace(...)`; the arguments and preceding method chain are shown above.
- Line 634, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 635: require `report['households'] == 12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 636, `states`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 637: require `'household_code' not in states.columns` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 638, `expected_households`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 639: require `np.allclose(states.groupby('policy').probability.sum(), expected_households)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 642, `household`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 643, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 644, `basket_rows`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 645, `values`: Set this name to the list `[]`.
- Line 646: for each entry yielded by `discounts`, bind/unpack it into `d1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 647, `prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 648: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 649, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 650, `u`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets

select from the object to their left.
- Line 651, `prob`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 652, `value`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 653, `continuation`: Set this name to the literal value `0.0`.
- Line 654: for each entry yielded by `basket_rows`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 655, `mass`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 656, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 657, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 658, `returns`: Set this name to the list `[]`.
- Line 659: for each entry yielded by `discounts`, bind/unpack it into `d2` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 660, `second_prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 661: update `second_prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 662, `u2`: Set this name to the entry/slice selected by `np.r_[beta - a * second_prices + g * xi2, 0.0]`; brackets select from the object to their left.
- Line 663, `prob2`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 664: call `returns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 665: update `continuation` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 666: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 667, `actual`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 668: require `'household_code' not in actual.columns` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 669, `predicted`: Set this name to the entry/slice selected by `actual.loc[actual.policy.eq('personali zed_dynamic'), 'dis...`; brackets select from the object to their left.
- Line 670: require `np.isclose(predicted, max(values)).any()` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `list`: Materialize iterable entries as an ordered Python list.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 671–671:

```
671 print("Synthetic targeting checks passed: posterior likelihood, per-household optimization, full-basket backward
    ↪  induction, reporting-group invariance, policy dominance, missing menus, exact basket mass, exports.")
```

Read this block:

- Line 671: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

write run notes

Original function: `write_run_notes`. Source begins at line 679.

Writes a readable text rendering of the same automatic run information saved as JSON. pformat lays out nested records for reading; this is an output record, not an alternative settings file.

Definition and stated intent:

Source lines 679–680:

```
679 def write_run_notes(output, information):
680     """Write the same run information in plain text; you never edit a JSON file."""
```

`def` creates this callable; the parentheses name its inputs (`output`, `information`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 681–686:

```
681 output.mkdir(parents=True, exist_ok=True)
682 text = "RUN INFORMATION\n\n"
683 text += "This is an output record, not a settings file.\n"
684 text += "Change the Python parameters at the top of the script to change the run.\n\n"
685 text += pformat(information, sort_dicts=False, width=90)
686 (output / "run_notes.txt").write_text(text + "\n")
```

Read this block:

- Line 681: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 682, `text`: Set this name to the literal text `'RUN INFORMATION\n\n'`.
- Line 683: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 684: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 685: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 686: call `(output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 695.

Parses settings for the legacy three-group combined-gamma berry counterfactual. Its default discount factor is .95; use the main plain_berry entry point and after-data job for the current split-history three/five analysis.

Definition and stated intent:

Source lines 695–696:

```
695 def read_command_line(settings):
696     """Optional server flags. Normal use: edit the parameters at the top."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 697–703:

```
697 parser = argparse.ArgumentParser(description=__doc__)
698 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
699 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
700 parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results") /
    ↪  "empirical_outputs")
701 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
702                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
703 parser.add_argument("--demand-delta", type=float, default=.95)
```

Read this block:

- Line 697, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 698: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 699: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 700: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 701: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 703: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 704–709:

```
704 parser.add_argument("--batch-size", type=int, default=24)
705 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪  default="posterior-mean",
706                     help="Default treats estimated household types as known, matching base supply; optional
                        ↪  posterior uncertainty sensitivity.")
707 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
    ↪  terminal coupon to compressed CSV.")
708 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
709 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 704: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 705: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 707: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 708: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 709: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 710–710:

```
710 return parser.parse_args()
```

Read this block:

- Line 710: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run pricing

Original function: `run_pricing`. Source begins at line 720.

Validates explicit legacy input/settings, runs the single berry combined-gamma pricing case, and writes run records and figures. Its .95 discount default and single-normal combined history model differ from the current .92 split-history main workflow.

Definition and stated intent:

Source lines 720–721:

```
720 def run_pricing(settings):
721     """The actual research steps, in the order they run."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 722–724:

```
722 if settings.self_test:
723     self_test()
724     return
```

Read this block:

- Line 722: test `settings.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 723: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 724: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 725–726:

```
725 if settings.household_input is None or settings.fit_dir is None:
726     raise ValueError("Set household_input and fit_dir in the Parameters section, or supply --household-input and
      ↪  --fit-dir.")
```

Read this block:

- Line 725: test `settings.household_input is None or settings.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 726: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 727–727:

```
727 settings.discounts = np.unique(np.asarray(settings.discounts, dtype=float))
```

Read this block:

- Line 727, `settings.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 728–730:

```
728 if (not np.isfinite(settings.discounts).all() or settings.discounts[0] != 0. or settings.discounts[-1] > 1.
729         or not 0 <= settings.demand_delta <= 1 or settings.batch_size < 1):
730     raise ValueError("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 728: test `not np.isfinite(settings.discounts).all() or settings.discounts[0] != 0.0 or ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 730: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 731–738:

```
731 settings.output.mkdir(parents=True, exist_ok=True)
732 report = run_households(settings)
733 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(settings).items()},
734                 results=report, script_sha256=digest(Path(__file__).resolve()),
735                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
736 (settings.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
737 write_run_notes(settings.output, manifest)
738 print(f"Standalone empirical targeting completed: {settings.output}")
```

Read this block:

- Line 731: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 732, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 733, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 736: call `(settings.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 737: call `write_run_notes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 738: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `write_run_notes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

../final_simple/demand/regenerate_weekly_inputs.py

Readable server reconstruction entry point. It imports preparation/likelihood functions from the local estimation module, avoiding a second demand implementation. Edit paths only on the server. Consistency failures stop before replacing the export; successful regeneration leaves fitted demand coefficients unchanged.

Module setup and direct-execution blocks

Source lines 1–8:

```
"""Regenerate weekly inputs on the server without refitting saved demand parameters."""

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next(p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file())
_policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
from server_data_policy import server_only, require_server, require_data_path
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 4: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 6, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: from `server_data_policy`, import `server_only, require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 9–15:

```
import argparse
import json
from pathlib import Path
import tempfile
from types import SimpleNamespace

import numpy as np
```

Read this block:

- Line 9: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: import `tempfile`. An as-name is a convenient local alias, not another library.

- Line 13: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 16–16:

```
16 import pandas as pd
```

Read this block:

- Line 16: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 18–21:

```
18 try:
19     from .estimation import prepare_data, make_arrays, make_draws, parameter_names, household_loglike,
       ↪  add_price_controls
20 except ImportError:
21     from estimation import prepare_data, make_arrays, make_draws, parameter_names, household_loglike,
       ↪  add_price_controls
```

Read this block:

- Line 18: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 19: from `estimation`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 20: catch `ImportError` from the preceding try; the optional as-name holds the error object for logging.
- Line 21: from `estimation`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 24–26:

```
24 fit_dir = None
25 sample = None     # Optional override if the saved prepared-sample path moved.
26 flavors = None    # Optional override if the saved flavor-reference path moved.
```

Read this block:

- Line 24, `fit_dir`: Set this name to the literal value `None`.
- Line 25, `sample`: Set this name to the literal value `None`.
- Line 26, `flavors`: Set this name to the literal value `None`.

Source lines 105–106:

```
105 if __name__ == '__main__':
106     main()
```

Read this block:

- Line 105: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 106: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

regenerate

Original function: `regenerate`. Source begins at line 32.

Restores saved sample/model/category/history settings, defaulting older records to three groups and original-share novelty. It reconstructs the server table, checks household/week totals and the saved fixed-parameter likelihood, then atomically publishes the weekly file. It does not refit demand or change fitted parameters.

Definition and stated intent:

Source lines 32–32:

```
32 def regenerate(fit_dir, sample=None, flavors=None, weekly_output=None):
```

`def` creates this callable; the parentheses name its inputs (`fit_dir`, `sample`, `flavors`, `weekly_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 33–39:

```
33 fit_dir = Path(fit_dir)
34 fit = json.loads((fit_dir / 'fit.json').read_text())
35 settings = dict(fit['settings'])
36 settings.setdefault('flavor_spec', 'three')
37 settings.setdefault('novelty_measure', 'previous_share')
38 required = ['sample', 'first_year', 'last_year', 'max_households', 'capacity',
39             'no_price_control', 'household_dma_column', 'habit_weight', 'draws', 'seed']
```

Read this block:

- Line 33, `fit_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 34, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 35, `settings`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 36: call `settings.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `settings.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `required`: Set this name to a list containing 10 displayed entries.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 40–40:

```
40 missing = set(required)-settings.keys()
```

Read this block:

- Line 40, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 41–42:

```
41 if missing:
42     raise ValueError(f'Saved fit lacks reconstruction settings: {sorted(missing)}')
```

Read this block:

- Line 41: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 43–44:

```
43 settings['model'] = fit['model']
44 settings['output'] = Path(weekly_output).parent if weekly_output is not None else
    ↪  Path('/scratch/dtm63837/Kilts_Panel/Data/temporary') / fit_dir.name
```

Read this block:

- Line 43, `settings['model']`: Store the entry/slice selected by `fit['model']`; brackets select from the object to their left in the selected entry/column.
- Line 44, `settings['output']`: Store the true-branch value if `weekly_output is not None` holds, otherwise the else-branch value in the selected entry/column.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 45–49:

```
45 for key in ['sample', 'flavors', 'products_extra']:
46     if settings.get(key) is not None:
47         settings[key] = Path(settings[key])
48     else:
49         settings[key] = None
```

Read this block:

- Line 45: for each entry yielded by `['sample', 'flavors', 'products_extra']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: test `settings.get(key) is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 47, `settings[key]`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 49, `settings[key]`: Store the literal value `None` in the selected entry/column.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 50–51:

```
50 if sample is not None:
51     settings['sample'] = Path(sample)
```

Read this block:

- Line 50: test `sample is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51, `settings['sample']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 52–53:

```
52 if flavors is not None:
53     settings['flavors'] = Path(flavors)
```

Read this block:

- Line 52: test `flavors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 53, `settings['flavors']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 54–55:

```
54 args = SimpleNamespace(**settings)
55 frame = prepare_data(args)
```

Read this block:

- Line 54, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 55, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 56–58:

```
56 if not args.no_price_control:
57     # Reconstruct the original first stage; preserve saved inference artifacts.
58     frame, _ = add_price_controls(frame)
```

Read this block:

- Line 56: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 58, `(frame, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 59–59:

```
59 data = make_arrays(frame, args.capacity)
```

Read this block:

- Line 59, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 60–61:

```
if len(frame) != fit['weeks'] or data['n_households'] != fit['households']:
    raise ValueError('Regenerated household/week totals do not match the saved fit; no CSV written.')
```

Read this block:

- Line 60: test `len(frame) != fit['weeks'] or data['n_households'] != fit['households']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 61: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 62–67:

```
estimates = pd.read_csv(fit_dir / 'point_estimates.csv')
parameters = estimates.set_index('parameter').estimate
names = parameter_names(args.model, data['outside'], data['control'])
theta = np.array([parameters[name] for name in names])
draws = make_draws(args.draws, args.seed, args.model)
nll = -float(household_loglike(theta, data, draws, args.model).sum())
```

Read this block:

- Line 62, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 63, `parameters`: Set this name to the attribute `estimates.set_index('parameter').estimate` of the object before the dot.
- Line 64, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 65, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 66, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 67, `nll`: Set this name to the negated or sign-adjusted expression `-float(household_loglike(theta, data, draws, args.model)....`.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 68–69:

```
if not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-9, atol=1e-4):
    raise ValueError(f'Regenerated likelihood {nll} differs from saved {fit["nll"]}; no CSV written.')
```

Read this block:

- Line 68: test `not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 70–72:

```
70 target = Path(weekly_output) if weekly_output is not None else Path('/scratch/dtm63837/Kilts_Panel/Data/temporary')
   ↪  / fit_dir.name / 'weekly_inputs.csv'
71 require_data_path(target)
72 target.parent.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 70, `target`: Set this name to the true-branch value if `weekly_output is not None` holds, otherwise the else-branch value.
- Line 71: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `target.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 74–75:

```
74 with tempfile.NamedTemporaryFile(dir=target.parent, prefix='.weekly_inputs_', suffix='.csv', delete=False) as
   ↪  handle:
75     temporary = Path(handle.name)
```

Read this block:

- Line 74: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 75, `temporary`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Source lines 76–80:

```
76 try:
77     frame.to_csv(temporary, index=False)
78     temporary.replace(target)
79 finally:
80     temporary.unlink(missing_ok=True)
```

Read this block:

- Line 76: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 77: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `temporary.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: call `temporary.unlink` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 81–86:

```
report = dict(households=data['n_households'], weeks=len(frame), reconstructed_nll=nll,
              saved_nll=fit['nll'], model=args.model, flavor_spec=args.flavor_spec,
              novelty_measure=args.novelty_measure, sample=str(args.sample),
              demand_refitted=False, price_control_refitted=not args.no_price_control)
(fit_dir / 'weekly_inputs_regeneration.json').write_text(json.dumps(report, indent=2)+'\n')
print(f'Regenerated {target}; household/week totals and saved likelihood verified.', flush=True)
```

Read this block:

- Line 81, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 85: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 87–87:

```
return report
```

Read this block:

- Line 87: return the object currently named `report`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 93.

Parses a fit directory and optional moved input paths, then calls server-side reconstruction. Missing required paths stop before processing purchases. No optimizer is invoked.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 93–93:

```
def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–99:

```
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('--fit-dir', type=Path, default=fit_dir)
parser.add_argument('--sample', type=Path, default=sample, help='Override saved prepared-sample path if moved.')
parser.add_argument('--flavors', type=Path, default=flavors, help='Override saved flavor-reference path if moved.')
```

```
98 parser.add_argument('--weekly-output', type=Path, help='Temporary weekly CSV outside the code checkout.')
99 args = parser.parse_args()
```

Read this block:

- Line 94, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 95: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 100–101:

```
100 if args.fit_dir is None:
101     parser.error('Set fit_dir above or supply --fit-dir on the server.')
```

Read this block:

- Line 100: test `args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 101: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 102–102:

```
102 regenerate(args.fit_dir, args.sample, args.flavors, args.weekly_output)
```

Read this block:

- Line 102: call `regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `regenerate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/demand/split_pricing.py

Shared exact-basket solver for three/five alternatives. Main plain/berry comparison uses this implementation; optional all-category sensitivity selects categories from saved metadata. It retains separate rival histories and

reports auxiliary calibration failures without discarding coupon outputs.

Module setup and direct-execution blocks

Source lines 2–12:

```
"""Split-history pricing for each focal flavor, separate from demand estimation.

User-supplied paths only. No estimation is performed. --self-test uses only
generated synthetic inputs. See README.md for definitions.
"""
from __future__ import annotations

# Empirical I/O is permitted only on the research server, never on this computer.
from pathlib import Path as _PolicyPath
import sys as _policy_sys
_policy_root = next((p for p in _PolicyPath(__file__).resolve().parents if (p /
    "Code/privacy/server_data_policy.py").is_file()), None)
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pathlib`, import `Path as _PolicyPath` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: import `sys as _policy_sys`. An as-name is a convenient local alias, not another library.
- Line 12, `_policy_root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `_PolicyPath`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 13–14:

```
if _policy_root is not None:
    _policy_sys.path.insert(0, str(_policy_root / "Code/privacy"))
```

Read this block:

- Line 13: test `_policy_root is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 14: call `_policy_sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.

Source lines 15–20:

```
from server_data_policy import server_only, require_server, require_data_path
import argparse
import hashlib
import json
import math
import os
```

Read this block:

- Line 15: from `server_data_policy`, import `server_only, require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 17: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 18: import `json`. An as-name is a convenient local alias, not another library.
- Line 19: import `math`. An as-name is a convenient local alias, not another library.
- Line 20: import `os`. An as-name is a convenient local alias, not another library.

Source lines 21–26:

```
from pathlib import Path
import sys
from types import SimpleNamespace
from pprint import pformat
import tempfile
import numpy as np
```

Read this block:

- Line 21: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 22: import `sys`. An as-name is a convenient local alias, not another library.
- Line 23: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 24: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 25: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 26: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 27–27:

```
import pandas as pd
```

Read this block:

- Line 27: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 32–37:

```
household_input = None
fit_dir = None
output = Path("/scratch/dtm63837/Kilts_Panel/results") / "split_pricing"
focal_flavor = "all"                  # all fitted groups, or one named fitted group
discounts = np.linspace(0., 1., 101)  # Candidate focal-product coupons.
demand_delta = .92                    # Same default as theoretical supply.
```

Read this block:

- Line 32, `household_input`: Set this name to the literal value `None`.
- Line 33, `fit_dir`: Set this name to the literal value `None`.
- Line 34, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 35, `focal_flavor`: Set this name to the literal text `'all'`.
- Line 36, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 37, `demand_delta`: Set this name to the literal value `0.92`.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 38–41:

```
batch_size = 24                      # Progress interval in the household loop.
type_treatment = "posterior-mean"    # Or posterior-integrated.
save_state_policies = False
run_self_tests = False
```

Read this block:

- Line 38, `batch_size`: Set this name to the literal value `24`.
- Line 39, `type_treatment`: Set this name to the literal text `'posterior-mean'`.
- Line 40, `save_state_policies`: Set this name to the literal value `False`.
- Line 41, `run_self_tests`: Set this name to the literal value `False`.

Source lines 47–50:

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from flavor_specs import categories

FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 47: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: from `flavor_specs`, import `categories` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 50, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 382–385:

```
if __package__:
    from .gamma_calibration import run_calibration, PRIORS, CalibrationError
else:
    from gamma_calibration import run_calibration, PRIORS, CalibrationError
```

Read this block:

- Line 382: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383: from `gamma_calibration`, import `run_calibration, PRIORS, CalibrationError` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 385: from `gamma_calibration`, import `run_calibration, PRIORS, CalibrationError` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 747–754:

```
if __name__ == "__main__":
    settings = SimpleNamespace(household_input=household_input, fit_dir=fit_dir,
        output=output, focal_flavor=focal_flavor, discounts=discounts,
        demand_delta=demand_delta, batch_size=batch_size, type_treatment=type_treatment,
        save_state_policies=save_state_policies, self_test=run_self_tests)
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_pricing(settings)
```

Read this block:

- Line 747: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 748, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.

- Line 752: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 753, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 754: call `run_pricing` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_pricing`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 52.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 52–52:

```
52 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 53–54:

```
53 x = np.asarray(x)
54 small = np.exp(-np.abs(x))
```

Read this block:

- Line 53, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 54, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 55–55:

```
55 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 55: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 57.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 57–57:

```
57 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 58–60:

```
58 x = np.asarray(x)
59 maximum = np.max(x, axis=axis, keepdims=True)
60 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 58, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 59, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 60, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 61–61:

```
61 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 61: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 63.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 63–63:

```
63 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 64–67:

```
64 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
65 import matplotlib
66 matplotlib.use("Agg")
67 import matplotlib.pyplot as plt
```

Read this block:

- Line 64: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 66: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 67: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 68–68:

```
68 return plt
```

Read this block:

- Line 68: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 70.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 70–70:

```
70 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 71–72:

```
71 if number < 2 or number % 2:
72     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 71: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 72: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 73–73:

```
73 half = np.random.default_rng(seed).standard_normal((number//2, 3))
```

Read this block:

- Line 73, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 74–74:

```
74 return np.vstack([half, -half])
```

Read this block:

- Line 74: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 76.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 76–77:

```
76 def posterior(frame, parameters, draws, control=True):
77     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–83:

```
78 labels = np.sort(frame.household_code.unique())
79 hh = pd.Categorical(frame.household_code, categories=labels).codes
80 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
81 available = np.isfinite(prices) & (prices > 0)
82 counts = frame[FLAVORS+["outside"]].to_numpy(float)
83 novelty = frame[["novelty_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 78, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 79, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 80, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 81, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 83, `novelty`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 84–85:

```
84 stock = frame[["stock_"+f for f in FLAVORS]].to_numpy(float)
85 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 84, `stock`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 85, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 86–89:

```
86 if (not np.isfinite(counts).all() or (counts < 0).any()
87         or not np.isfinite(novelty).all() or not np.isfinite(stock).all() or not np.isfinite(cf).all()
88         or ((counts[:, :len(FLAVORS)] > 0) & ~available).any()):
89     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 86: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(novel...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 89: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `len`: Count elements/rows rather than summing their numerical values.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 90–94:

```
90 beta = parameters["inside_constant"] + np.array([0.] + [parameters["beta_"+f] for f in FLAVORS[1:]])
91 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
92 switching = parameters["mean_novelty"] + parameters["sd_novelty"]*draws[:, 1]
93 habit = parameters["mean_own_stock"] + parameters["sd_own_stock"]*draws[:, 2]
94 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 90, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 91, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 92, `switching`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 93, `habit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 94, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 95–100:

```
95 for d, (a, g, b) in enumerate(zip(alpha, switching, habit)):
96     u = beta-a*np.nan_to_num(prices)+g*novelty+b*stock+parameters.get("price_control", 0.)*cf
97     u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
98     logp = u-logsumexp(u, axis=1, keepdims=True)
99     ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
100    loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 95: for each entry yielded by `enumerate(zip(alpha, switching, habit))`, bind/unpack it into `(d, (a, g, b))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 96, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 98, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 99, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

- Line 100, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 101–107:

```
101 weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
102 nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
103 means = weights @ switching
104 # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
105 order = np.argsort(means, kind="stable")
106 groups = np.full(len(labels), "middle", dtype=object)
107 n = len(labels)//4
```

Read this block:

- Line 101, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 102, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 103, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 105, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 106, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 107, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 108–109:

```
108 if n < 1:
109     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 108: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 109: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 110–118:

```
groups[order[:n]], groups[order[-n:]] = "low", "high"
population_q = np.quantile(switching, [.25, .75])
table = pd.DataFrame(dict(household_code=labels, group=groups, switching_mean=means,
                          switching_sd=np.sqrt(np.maximum(weights @ switching**2-means**2, 0)),
                          alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
                          probability_switching_below_draw_q25=weights @ (switching <= population_q[0]),
                          probability_switching_above_draw_q75=weights @ (switching >= population_q[1])))
table["habit_mean"] = weights @ habit
table["habit_sd"] = np.sqrt(np.maximum(weights @ habit**2 - table.habit_mean.to_numpy()**2, 0))
```

Read this block:

- Line 110, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 111, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 112, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 117, `table['habit_mean']`: Store matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 118, `table['habit_sd']`: Store the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 119–119:

```
return table, weights, alpha, np.column_stack([switching, habit]), beta, nll
```

Read this block:

- Line 119: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 123.

Enumerates every nonnegative basket for the selected inside categories plus nonpurchase. With capacity K and J inside groups, there are choose(K+J,J) states: 120 for J=3/K=7 and 792 for J=5/K=7. Exact states preserve separate rival histories.

Definition and stated intent:

Source lines 123–124:

```
def compositions(capacity, inside=None):
    """All baskets for the selected inside alternatives plus outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`, `inside`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 125–125:

```
125 inside = len(FLAVORS) if inside is None else inside
```

Read this block:

- Line 125, `inside`: Set this name to the true-branch value if `inside is None` holds, otherwise the else-branch value.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 126–132:

```
126 def generate(total, slots):
127     if slots == 1:
128         yield (total,)
129     else:
130         for count in range(total+1):
131             for rest in generate(total-count, slots-1):
132                 yield (count,)+rest
```

Read this block:

- Line 126: define nested callable `generate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 133–133:

```
133 return np.array(list(generate(capacity, inside+1)), dtype=int)
```

Read this block:

- Line 133: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `list`: Materialize iterable entries as an ordered Python list.
- `generate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

compositions.generate

Original function: `compositions.generate`. Source begins at line 126.

A recursive generator assigns one category count at a time, then allocates the remaining total to the remaining slots. The base case puts all remaining opportunities into the last slot. Each nonnegative basket appears once.

Definition and stated intent:

Source lines 126–126:

```
126 def generate(total, slots):
```

`def` creates this callable; the parentheses name its inputs (`total`, `slots`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 127–132:

```
127 if slots == 1:
128     yield (total,)
129 else:
130     for count in range(total+1):
131         for rest in generate(total-count, slots-1):
132             yield (count,)+rest
```

Read this block:

- Line 127: test `slots == 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128: evaluate this expression without assigning its returned value.
- Line 130: for each entry yielded by `range(total + 1)`, bind/unpack it into `count` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 131: for each entry yielded by `generate(total - count, slots - 1)`, bind/unpack it into `rest` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 132: evaluate this expression without assigning its returned value.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `generate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

forecast context

Original function: `forecast_context`. Source begins at line 136.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 136–141:

```
136 def forecast_context(frame, households):
137     """First observed choice establishes h0; price the next two consecutive weeks.
138 
139     Carry the initial observation's menu, posted prices and controls forward.
140     Type estimation still uses the complete observed sample.
141     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 142–148:

```
142 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
143 first = first.set_index("household_code").loc[households.household_code]
144 counts = first[FLAVORS].to_numpy(float)
145 totals = counts.sum(axis=1, keepdims=True)
146 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
147 # Saved stocks precede the observed choice; add its basket exactly once.
148 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 142, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 143, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 144, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 145, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 146, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 148, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 149–154:

```
149 novelty = shares.sum(axis=1, keepdims=True)-shares
150 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
151 available = np.isfinite(prices) & (prices > 0)
152 eligible = available[:, 1]
153 households["berry_eligible"] = eligible
154 initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 149, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 150, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 151, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 152, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 153, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 154, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 155–159:

```
155 households["initial_observed_week"] = initial_weeks
156 households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
157 households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
158 cf_cols = ["cf_"+f for f in FLAVORS]
```

```
159 cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 155, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 156, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 157, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 158, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 159, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 160–161:

```
160 if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
161     raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 160: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 161: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 162–163:

```
162 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
163             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 162: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 165.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 165–165:

```
def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 166–166:

```
h = hashlib.sha256()
```

Read this block:

- Line 166, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 167–169:

```
with path.open("rb") as handle:
    for chunk in iter(lambda: handle.read(1024*1024), b""):
        h.update(chunk)
```

Read this block:

- Line 167: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 168: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 169: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 170–170:

```
return h.hexdigest()
```

Read this block:

- Line 170: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 176.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 176–185:

```
176 def targeted_coupons(context, households, weights, alpha, switching, beta, discounts,
177                      delta=1., capacity=7, control_coefficient=0.,
178                      batch_size=24, type_treatment="posterior-mean", state_output=None, benchmark_output=None):
179     """Choose household-specific p1(h0) and terminal p2(h1) by backward induction.
180 
181     Work through ONE household at a time. This makes type-specific pricing
182     explicit and avoids the old extra household dimension in every array.
183     For this household, arrays run over possible types, baskets, and coupons.
184     batch_size only controls how frequently progress is reported.
185     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `switching`, `beta`, `discounts`, `delta`, `capacity`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`, `benchmark_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 186–187:

```
186 if state_output is not None:
187     raise ValueError("Only aggregate comparisons may be saved; individual state export is disabled.")
```

Read this block:

- Line 186: test `state_output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 187: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 188–196:

```
188 baskets = compositions(capacity)
189 number_states = len(baskets)
190 number_coupons = len(discounts)
191 state_numbers = np.arange(number_states)
192 
193 # A basket records the selected inside flavors followed by outside.
194 # Coordinate one is the focal seller after permutation; histories retain all flavors.
195 inside_counts = baskets[:, :len(FLAVORS)]
196 inside_total = inside_counts.sum(axis=1, keepdims=True)
```

Read this block:

- Line 188, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 189, `number_states`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 190, `number_coupons`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 191, `state_numbers`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.

- Line 195, `inside_counts`: Set this name to the entry/slice selected by `baskets[:, :len(FLAVORS)]`; brackets select from the object to their left.
- Line 196, `inside_total`: Set this name to the result of calling `inside_counts.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 197–207:

```
197 basket_shares = np.divide(
198     inside_counts, inside_total,
199     out=np.zeros_like(inside_counts, dtype=float), where=inside_total > 0)
200 basket_novelty = basket_shares.sum(axis=1, keepdims=True) - basket_shares
201 log_multinomial_coefficient = (
202     math.lgamma(capacity + 1)
203     - np.vectorize(math.lgamma)(baskets + 1).sum(axis=1))
204
205 eligible_households = np.flatnonzero(context["eligible"])
206 targeted = households.group.isin(["low", "high"]).to_numpy()
207 rows = []
```

Read this block:

- Line 197, `basket_shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 200, `basket_novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 201, `log_multinomial_coefficient`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 205, `eligible_households`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 206, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 207, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 208–210:

```
208 maximum_mass_error = 0.
209 wrote_states = False
210 common_values = np.zeros((number_coupons, number_coupons))
```

Read this block:

- Line 208, `maximum_mass_error`: Set this name to the literal value `0.0`.
- Line 209, `wrote_states`: Set this name to the literal value `False`.
- Line 210, `common_values`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 212–253:

```
for number_done, household in enumerate(eligible_households, start=1):
    prices = context["prices"][household]
    available = context["available"][household]
    stock = context["stocks"][household]
    novelty = context["novelty"][household]
    control = context["cf"][household]

    # Default: the firm knows estimated price, switching, and habit coefficients.
    # Sensitivity: retain the full probability distribution of possible types.
    if type_treatment == "posterior-mean":
        price_sensitivity = np.array([weights[household] @ alpha])
        history_coefficient = (weights[household] @ switching)[None, :]
        type_weights = np.ones(1)
    else:
        price_sensitivity = alpha
        history_coefficient = switching
        type_weights = weights[household]

    # Columns of history_coefficient are b_N (switching) and b_S (habit).
    b_n = history_coefficient[:, 0]
    b_s = history_coefficient[:, 1]
    history1 = b_n[:, None]*novelty + b_s[:, None]*stock
    history2 = (b_n[:, None, None]*basket_novelty
                + b_s[:, None, None]*(stock + basket_shares))
    utility1 = (beta - price_sensitivity[:, None]*prices
                + history1 + control_coefficient*control)
    utility2 = (beta - price_sensitivity[:, None, None]*prices
                + history2 + control_coefficient*control)
    utility1 = np.where(available, utility1, -np.inf)
    utility2 = np.where(available, utility2, -np.inf)

    # STEP A: terminal outcomes for every candidate coupon and every h1.
    # Metrics are [revenue, berry units, coupon expenditure].
    terminal = []
    for discount in discounts:
        utility = utility2.copy()
        utility[:, :, 1] += price_sensitivity[:, None] * prices[1] * discount
        outside = np.zeros((*utility.shape[:-1], 1))
        all_utilities = np.concatenate([utility, outside], axis=-1)
        log_denominator = logsumexp(all_utilities, axis=-1)
        berry_quantity = capacity * np.exp(utility[:, :, 1] - log_denominator)
        offered_price = prices[1] * (1 - discount)
```

Source lines 254–295:

```
    terminal.append(np.stack([
        berry_quantity * offered_price,
        berry_quantity,
        berry_quantity * prices[1] * discount], axis=-1))
terminal = np.asarray(terminal)

first = np.zeros((number_coupons, 3))
second = np.zeros((number_coupons, 3))
paths = []

# STEP B: try each p1(h0), calculate its state probabilities, and then
# select the best p2 for each state. Optimize BEFORE averaging revenue.
for action, discount in enumerate(discounts):
    utility = utility1.copy()
```

```
    utility[:, 1] += price_sensitivity * prices[1] * discount
    all_utilities = np.column_stack([utility, np.zeros(len(type_weights))])
    log_probability = all_utilities - logsumexp(all_utilities, axis=1, keepdims=True)
    terms = np.zeros((len(type_weights), number_states, len(FLAVORS)+1))
    np.multiply(baskets[None, :, :], log_probability[:, None, :],
                out=terms, where=baskets[None, :, :] > 0)
    basket_probability = np.exp(log_multinomial_coefficient + terms.sum(axis=-1))
    error = float(np.abs(basket_probability.sum(axis=1) - 1).max())
    maximum_mass_error = max(maximum_mass_error, error)

    # Joint(type,h1) = initial type weight x Pr(h1 | type,p1).
    joint = type_weights[:, None] * basket_probability
    state_probability = joint.sum(axis=0)
    berry_quantity = capacity * (type_weights * np.exp(log_probability[:, 1])).sum()
    first[action] = [berry_quantity * prices[1] * (1 - discount),
                     berry_quantity,
                     berry_quantity * prices[1] * discount]

    # In the uncertainty sensitivity, integrate possible types before
    # choosing a terminal coupon: the firm cannot observe the true draw.
    # Result dimensions: terminal coupon x basket h1 x metric.
    weighted_terminal = (terminal * joint[None, :, :, None]).sum(axis=1)
    if targeted[household]:
        common_values[action] += first[action, 0] + delta * weighted_terminal[:, :, 0].sum(axis=1)
    elif action == 0:
        common_values += first[0, 0] + delta * weighted_terminal[0, :, 0].sum()
    best_terminal_action = weighted_terminal[:, :, 0].argmax(axis=0)
    selected = weighted_terminal[best_terminal_action, state_numbers]
```

Source lines 296–337:

```
    second[action] = selected.sum(axis=0)
    if action == 0:
        no_coupon_second = weighted_terminal[0].sum(axis=0)
    paths.append((best_terminal_action, state_probability))

# STEP C: choose the first coupon by total expected discounted revenue.
dynamic_action = int((first[:, 0] + delta * second[:, 0]).argmax())
if not targeted[household]:
    dynamic_action = 0

# Keep posted prices as a benchmark. Only dynamic pricing is optimized.
for policy, action in [("no_coupon", 0), ("personalized_dynamic", dynamic_action)]:
    use_coupons = targeted[household] and policy == "personalized_dynamic"
    initial = first[action]
    best_terminal_action, state_probability = paths[action]
    if use_coupons:
        terminal_metrics = second[action]
        terminal_coupon = discounts[best_terminal_action]
    else:
        terminal_metrics = no_coupon_second
        terminal_coupon = np.zeros(number_states)

    p1 = prices[1] * (1 - discounts[action])
    p2 = prices[1] * (1 - terminal_coupon)
    mean_p2 = float(state_probability @ p2)
    ratio = np.divide(p1, p2, out=np.full(number_states, np.nan), where=p2 > 0)
    positive_mass = state_probability > 1e-14
    expected_ratio = np.nan
    if np.isfinite(ratio[positive_mass]).all():
        expected_ratio = float(state_probability[positive_mass] @ ratio[positive_mass])

```

```
    row = dict(
        household_code=households.iloc[household].household_code,
        group=households.iloc[household].group, policy=policy,
        discount1=discounts[action],
        expected_discount2=float(state_probability @ terminal_coupon),
        p1=p1, expected_p2=mean_p2,
        ratio_of_expected_prices=p1 / mean_p2 if mean_p2 > 0 else np.nan,
        expected_price_ratio=expected_ratio,
        revenue1=initial[0], revenue2=terminal_metrics[0],
        discounted_revenue=initial[0] + delta * terminal_metrics[0],
        berry_units1=initial[1], berry_units2=terminal_metrics[1],
```

Source lines 338–349:

```
        coupon_expenditure=initial[2] + delta * terminal_metrics[2])
    for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
        branch_mass = state_probability[branch].sum()
        row["probability_" + label] = branch_mass
        row["mean_p2_given_" + label] = np.nan
        if branch_mass > 0:
            row["mean_p2_given_" + label] = float(
                state_probability[branch] @ p2[branch] / branch_mass)
    rows.append(row)

if number_done % (batch_size * 40) == 0 or number_done == len(eligible_households):
    print(f"Targeted pricing: integrated {number_done}/{len(eligible_households)} eligible households", flush=True)
```

Read this block:

- Line 212: for each entry yielded by `enumerate(eligible_households, start=1)`, bind/unpack it into `(number_done, household)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 213, `prices`: Set this name to the entry/slice selected by `context['prices'][household]`; brackets select from the object to their left.
- Line 214, `available`: Set this name to the entry/slice selected by `context['available'][household]`; brackets select from the object to their left.
- Line 215, `stock`: Set this name to the entry/slice selected by `context['stocks'][household]`; brackets select from the object to their left.
- Line 216, `novelty`: Set this name to the entry/slice selected by `context['novelty'][household]`; brackets select from the object to their left.
- Line 217, `control`: Set this name to the entry/slice selected by `context['cf'][household]`; brackets select from the object to their left.
- Line 221: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 222, `price_sensitivity`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 223, `history_coefficient`: Set this name to the entry/slice selected by `(weights[household] @ switching)[None, :]`; brackets select from the object to their left.
- Line 224, `type_weights`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 226, `price_sensitivity`: Set this name to the object currently named `alpha`; plain assignment does not copy it.
- Line 227, `history_coefficient`: Set this name to the object currently named `switching`; plain assignment does not copy it.
- Line 228, `type_weights`: Set this name to the entry/slice selected by `weights[household]`; brackets select from the object to their left.
- Line 231, `b_n`: Set this name to the entry/slice selected by `history_coefficient[:, 0]`; brackets select from the object to their left.
- Line 232, `b_s`: Set this name to the entry/slice selected by `history_coefficient[:, 1]`; brackets select from the object to their left.

- Line 233, `history1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 234, `history2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 236, `utility1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 238, `utility2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 240, `utility1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 241, `utility2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 245, `terminal`: Set this name to the list `[]`.
- Line 246: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 247, `utility`: Set this name to the result of calling `utility2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 248: update `utility[:, :, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 249, `outside`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 250, `all_utilities`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 251, `log_denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 252, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 253, `offered_price`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 254: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 258, `terminal`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 260, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 261, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 262, `paths`: Set this name to the list `[]`.
- Line 266: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(action, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 267, `utility`: Set this name to the result of calling `utility1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 268: update `utility[:, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 269, `all_utilities`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 270, `log_probability`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 271, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 272: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 274, `basket_probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 275, `error`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

- Line 276, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 279, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 280, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 281, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 282, `first[action]`: Store a list containing 3 displayed entries in the selected entry/column.
- Line 289, `weighted_terminal`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 290: test `targeted[household]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 291: update `common_values[action]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 292: test `action == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 293: update `common_values` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 294, `best_terminal_action`: Set this name to the result of calling `weighted_terminal[:, :, 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 295, `selected`: Set this name to the entry/slice selected by `weighted_terminal[best_terminal_action, state_numbers]`; brackets select from the object to their left.
- Line 296, `second[action]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 297: test `action == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 298, `no_coupon_second`: Set this name to the result of calling `weighted_terminal[0].sum(...)`; the arguments and preceding method chain are shown above.
- Line 299: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 302, `dynamic_action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 303: test `not targeted[household]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 304, `dynamic_action`: Set this name to the literal value `0`.
- Line 307: for each entry yielded by `[('no_coupon', 0), ('personalized_dynamic', dynamic_action)]`, bind/unpack it into `(policy, action)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 308, `use_coupons`: Set this name to the Boolean condition `targeted[household] and policy == 'personalized_dynamic'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 309, `initial`: Set this name to the entry/slice selected by `first[action]`; brackets select from the object to their left.
- Line 310, `(best_terminal_action, state_probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 311: test `use_coupons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 312, `terminal_metrics`: Set this name to the entry/slice selected by `second[action]`; brackets select from the object to their left.
- Line 313, `terminal_coupon`: Set this name to the entry/slice selected by `discounts[best_terminal_action]`; brackets select from the object to their left.
- Line 315, `terminal_metrics`: Set this name to the object currently named `no_coupon_second`; plain assignment does not copy it.
- Line 316, `terminal_coupon`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 318, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subex-

pressions are evaluated together.
- Line 319, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 320, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 321, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 322, `positive_mass`: Set this name to the Boolean condition `state_probability > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 323, `expected_ratio`: Set this name to the attribute `np.nan` of the object before the dot.
- Line 324: test `np.isfinite(ratio[positive_mass]).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 325, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 327, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 339: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 340, `branch_mass`: Set this name to the result of calling `state_probability[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 341, `row['probability_' + label]`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 342, `row['mean_p2_given_' + label]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 343: test `branch_mass > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 344, `row['mean_p2_given_' + label]`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 346: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 348: test `number_done % (batch_size * 40) == 0 or number_done == len(eligible_households)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 349: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress

promptly.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 351–352:

```
351 if maximum_mass_error > 1e-9:
352     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 351: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 352: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 353–358:

```
353 table = pd.DataFrame(rows)
354 baseline = table.loc[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
355 table["baseline_revenue"] = table.household_code.map(baseline)
356 table["revenue_change"] = table.discounted_revenue - table.baseline_revenue
357 table["revenue_change_pct"] = 100 * table.revenue_change / table.baseline_revenue
358 comparison = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 353, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 354, `baseline`: Set this name to the attribute `table.loc[table.policy == 'no_coupon'].set_index('househo...` of the object before the dot.
- Line 355, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 356, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 357, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 358, `comparison`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 359–360:

```
359 if (comparison.personalized_dynamic + 1e-8 < comparison.no_coupon).any():
360     raise AssertionError("Dynamic pricing does not dominate the feasible no-coupon benchmark.")
```

Read this block:

- Line 359: test `(comparison.personalized_dynamic + 1e-08 < comparison.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 360: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 361–373:

```
361 if benchmark_output is not None:
362     a1, a2 = np.unravel_index(common_values.argmax(), common_values.shape)
363     totals = table.groupby("policy").discounted_revenue.sum()
364     common = float(common_values[a1, a2])
365     if common + 1e-8 < totals["no_coupon"] or common > totals["personalized_dynamic"] + 1e-8:
366         raise AssertionError("Expected no coupon <= common coupons <= personalized dynamic.")
367     for policy, value, d1, d2 in [
368             ("no_coupon", totals["no_coupon"], 0., 0.),
369             ("optimized_common_coupons", common, discounts[a1], discounts[a2]),
370             ("personalized_dynamic", totals["personalized_dynamic"], np.nan, np.nan)]:
371         benchmark_output.append(dict(policy=policy, discounted_revenue=value,
372             discount1=d1, discount2=d2,
373             revenue_change_pct=100*(value/totals["no_coupon"]-1)))
```

Read this block:

- Line 361: test `benchmark_output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 362, `(a1, a2)`: Unpack the result of calling `np.unravel_index(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 363, `totals`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 364, `common`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 365: test `common + 1e-08 < totals['no_coupon'] or common > totals['personalized_dynamic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 366: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 367: for each entry yielded by `[('no_coupon', totals['no_coupon'], 0.0, 0.0), ('optimized_com...`, bind/unpack it into `(policy, value, d1, d2)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 371: call `benchmark_output.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 374–374:

```
374 return table, maximum_mass_error
```

Read this block:

- Line 374: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

information frictions

Original function: `information_frictions`. Source begins at line 389.

Calls the auxiliary binary calibration and creates information figures using persistent_type_revenue. CalibrationError is handled explicitly: diagnostics are returned and coupon pricing continues, while unsupported information figures are omitted. Other programming exceptions remain failures.

Definition and stated intent:

Source lines 389–389:

```
389 def information_frictions(frame, households, delta, output):
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`, `delta`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 390–397:

```
390 try:
391     table, settings = run_calibration(frame, households, delta, output)
392 except CalibrationError:
393     # A rejected auxiliary binary calibration must not discard coupon results.
394     status = json.loads((output/'information_calibration_status.json').read_text())
395     print('Auxiliary type-information calibration rejected; coupon pricing will finish. '
396           'See information_calibration_status.json.', flush=True)
397     return status
```

Read this block:

- Line 390: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 391, `(table, settings)`: Unpack the result of calling `run_calibration(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 392: catch `CalibrationError` from the preceding try; the optional as-name holds the error object for logging.
- Line 394, `status`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 395: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 397: return the object currently named `status`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `run_calibration`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 398–400:

```
398 summary = table.set_index(["scenario", "h0", "case"])
399 lines = [r"\begin{tabular}{llrrr}\hline",
400          r"Initial state & Prior & Known profit & Fixed-belief profit & Difference\\\hline"]
```

Read this block:

- Line 398, `summary`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 399, `lines`: Set this name to the list `['\\begin{tabular}{llrrr}\\hline', 'Initial state & Prior...`.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 401–406:

```
401 for scenario, _ in PRIORS:
402     for h0 in ["j", "0"]:
403         k = summary.loc[(scenario, h0, "known")]
404         u = summary.loc[(scenario, h0, "unknown")]
405         lines.append(f"{h0} & {scenario} & {k.persistent_type_revenue:.4f} & {u.persistent_type_revenue:.4f} & "
406                      f"{k.value_of_information:.4f} " + r"\\")
```

Read this block:

- Line 401: for each entry yielded by `PRIORS`, bind/unpack it into `(scenario, _)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 402: for each entry yielded by `['j', '0']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 403, `k`: Set this name to the entry/slice selected by `summary.loc[scenario, h0, 'known']`; brackets select from the object to their left.
- Line 404, `u`: Set this name to the entry/slice selected by `summary.loc[scenario, h0, 'unknown']`; brackets select from the object to their left.
- Line 405: call `lines.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 407–411:

```
407 lines.append(r"\hline\end{tabular}")
408 (output / "information_friction_results.tex").write_text("\n".join(lines)+"\n")
409 plt = plotting()
410 scenarios = [name for name, _ in PRIORS]
411 labels = ["Equal", "Low-heavy", "Medium-heavy", "High-heavy"]
```

Read this block:

- Line 407: call `lines.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 408: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 409, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 410, `scenarios`: Set this name to a list computed by the displayed for-expression.
- Line 411, `labels`: Set this name to the list `['Equal', 'Low-heavy', 'Medium-heavy', 'High-heavy']`.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 412–438:

```
412 for metric, stem, ylabel in [("persistent_type_revenue", "profits", "Expected discounted revenue"),
413                              ("expected_p1", "first_prices", "Mean first-period offered price"),
414                              ("expected_p2", "terminal_prices", "Mean terminal offered price"),
```

```
                          ("value_of_information", "information_value", "Revenue gain from knowing novelty type")]:
    fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
    x = np.arange(4)
    for ax, h0 in zip(axes, ["j", "0"]):
        for case, offset, label in [("known", -.18, "Known type"), ("unknown", .18, "Uncertain type; fixed
        ↪ beliefs")]:
            if metric == "value_of_information" and case == "unknown":
                continue
            try:
                values = summary.xs((h0, case), level=("h0", "case")).reindex(scenarios)[metric]
            except KeyError:
                values = np.full(4, np.nan)
            ax.bar(x+(0 if metric == "value_of_information" else offset), values,
                   width=.6 if metric == "value_of_information" else .36, label=label)
        ax.set_xticks(x, labels, rotation=20)
        ax.set_title("Initially "+("inside" if h0 == "j" else "outside"))
        ax.grid(axis="y", alpha=.2)
        ax.set_axisbelow(True)
    axes[0].set_ylabel(ylabel)
    if metric != "value_of_information":
        axes[0].legend(fontsize=8)
    fig.tight_layout()
    for extension in ["pdf", "png"]:
        fig.savefig(output / f"information_friction_{stem}.{extension}", dpi=180)
    plt.close(fig)
```

Read this block:

- Line 412: for each entry yielded by `[('persistent_type_revenue', 'profits', 'Expected discounted r...`, bind/unpack it into `(metric, stem, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 416, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 417, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 418: for each entry yielded by `zip(axes, ['j', '0'])`, bind/unpack it into `(ax, h0)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 419: for each entry yielded by `[('known', -0.18, 'Known type'), ('unknown', 0.18, 'Uncertain ...`, bind/unpack it into `(case, offset, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 420: test `metric == 'value_of_information' and case == 'unknown'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 421: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 422: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 423, `values`: Set this name to the entry/slice selected by `summary.xs((h0, case), level=('h0', 'case')).reindex(scen...`; brackets select from the object to their left.
- Line 424: catch `KeyError` from the preceding try; the optional as-name holds the error object for logging.
- Line 425, `values`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 426: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 428: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 429: call `ax.set_title` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 430: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 431: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 432: call `axes[0].set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 433: test `metric != 'value_of_information'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 434: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 435: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 436: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 437: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 438: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 441–441:

```
441 return dict(settings, status='complete')
```

Read this block:

- Line 441: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

run households

Original function: `run_households`. Source begins at line 445.

Validates a converged split fit, configures its three/five categories, reconstructs the saved likelihood and permutes the focal category into solver coordinate one. It solves coupon benchmarks and records auxiliary calibration status. Rivals remain separate alternatives, and failed auxiliary calibration does not erase valid coupon outputs.

Definition and stated intent:

Source lines 445–445:

```
445 def run_households(args):
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 446–446:

```
446 require_data_path(args.fit_dir)
```

Read this block:

- Line 446: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 447–448:

```
447 if getattr(args, "save_state_policies", False):
448     raise ValueError("This exercise saves aggregate comparisons only; state-policy export is disabled.")
```

Read this block:

- Line 447: test `getattr(args, 'save_state_policies', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 448: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 451–451:

```
451 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 451, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 452–453:

```
452 if fit["model"] != "heterogeneous_history" or not fit.get("converged", False):
453     raise ValueError("This exercise requires a converged split heterogeneous-history fit.")
```

Read this block:

- Line 452: test `fit['model'] != 'heterogeneous_history' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 453: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 454–455:

```
454 FLAVORS[:] = categories(fit["settings"].get("flavor_spec", "three"))
455 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 454, `FLAVORS[:]`: Store the result of calling `categories(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 455, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 456–457:

```
456 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
457     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 456: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 457: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 458–460:

```
458 parameters = dict(zip(estimates.parameter, estimates.estimate))
459 settings = fit["settings"]
460 capacity = settings["capacity"]
```

Read this block:

- Line 458, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 459, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 460, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 461–462:

```
461 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
462     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 461: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 462: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 463–466:

```
frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
required = ["household_code", "week", "outside", *FLAVORS]
required += [prefix+f for prefix in ["price_", "stock_", "novelty_"] for f in FLAVORS]
control = not settings.get("no_price_control", False)
```

Read this block:

- Line 463, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 464, `required`: Set this name to a list containing 4 displayed entries.
- Line 465: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 466, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 467–468:

```
if control:
    required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 467: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 468: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 469–469:

```
missing = set(required)-set(frame)
```

Read this block:

- Line 469, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 470–471:

```
if missing:
    raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 470: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 471: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 472–473:

```
472 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪ "week"]).any():
473     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 472: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 473: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 474–475:

```
474 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
475     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 474: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 475: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 476–477:

```
476 draws = persistent_draws(settings["draws"], settings["seed"])
477 households, weights, alpha, switching, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 476, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 477, `(households, weights, alpha, switching, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 479–480:

```
479 if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
480     raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 479: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 480: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 481–481:

```
481 focal = args.focal_flavor
```

Read this block:

- Line 481, `focal`: Set this name to the attribute `args.focal_flavor` of the object before the dot.

Source lines 482–490:

```
482 if focal != "berry":
483     mapping = {"berry": focal, focal: "berry"}
484     original = frame.copy()
485     original_beta = beta.copy()
486     for destination, source in mapping.items():
487         for prefix in ["", "price_", "stock_", "novelty_", "cf_"]:
488             if prefix+source in original:
489                 frame[prefix+destination] = original[prefix+source]
490         beta[FLAVORS.index(destination)] = original_beta[FLAVORS.index(source)]
```

Read this block:

- Line 482: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 483, `mapping`: Set this name to a dictionary with named entries `berry`.
- Line 484, `original`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 485, `original_beta`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.
- Line 486: for each entry yielded by `mapping.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 487: for each entry yielded by `['', 'price_', 'stock_', 'novelty_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 488: test `prefix + source in original`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 489, `frame[prefix + destination]`: Store the entry/slice selected by `original[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 490, `beta[FLAVORS.index(destination)]`: Store the entry/slice selected by `original_beta[FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 491–492:

```
491 context = forecast_context(frame, households)
492 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 491, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 492, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 493–494:

```
493 if any(n == 0 for n in counts):
494     raise ValueError("Each switching segment must have focal-eligible forecast households.")
```

Read this block:

- Line 493: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 494: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 495–506:

```
495 benchmarks = []
496 table, mass_error = targeted_coupons(
497     context, households, weights, alpha, switching, beta, args.discounts,
498     args.demand_delta, capacity,
499     parameters.get("price_control", 0.) if control else 0., args.batch_size,
500     args.type_treatment,
501     None,
502     benchmark_output=benchmarks)
503 benchmark_table = pd.DataFrame(benchmarks)
504 benchmark_table.to_csv(args.output / "pricing_policy_benchmarks.csv", index=False)
505 plt = plotting()
506 fig, ax = plt.subplots(figsize=(9, 5))
```

Read this block:

- Line 495, `benchmarks`: Set this name to the list `[]`.
- Line 496, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 503, `benchmark_table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 504: call `benchmark_table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 505, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 506, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 507–514:

```
507 ax.bar(["No coupons", "Optimized common\ncoupons", "Personalized\ndynamic coupons"],
508        benchmark_table.revenue_change_pct, color=["#999999", "#3575AD", "#D17925"])
509 ax.set(ylabel="Revenue gain relative to no coupons (%)", title=f"{focal.title()}: pricing policy benchmarks")
510 ax.grid(axis="y", alpha=.2)
511 ax.set_axisbelow(True)
512 fig.text(.5, .015, "Same eligible households; middle group receives no coupons. Common offers vary by period, not
    ↪  household or history.",
513          ha="center", fontsize=8)
514 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 507: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 509: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 510: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 511: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 512: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 514: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 515–516:

```
515 for extension in ["pdf", "png"]:
516     fig.savefig(args.output / f"pricing_policy_benchmarks.{extension}", dpi=180)
```

Read this block:

- Line 515: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 516: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 517–523:

```
517 plt.close(fig)
518 supply_calibration = information_frictions(frame, households, args.demand_delta, args.output)
```

```
households.rename(columns={"berry_eligible": "focal_eligible"}, inplace=True)
table.rename(columns={c: c.replace("berry", "focal") for c in table if "berry" in c}, inplace=True)
table["focal_flavor"] = focal
totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
          "focal_units1", "focal_units2", "coupon_expenditure"]
```

Read this block:

- Line 517: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 518, `supply_calibration`: Set this name to the result of calling `information_frictions(...)`; the arguments and preceding method chain are shown above.
- Line 519: call `households.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 520: call `table.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 521, `table['focal_flavor']`: Store the object currently named `focal`; plain assignment does not copy it in the selected entry/column.
- Line 522, `totals`: Set this name to a list containing 8 displayed entries.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `information_frictions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 524–530:

```
aggregate = table.groupby("policy", sort=False)[totals].sum()
aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
group = table.groupby(["policy", "group"], sort=False)[totals].sum()
group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
                                            "ratio_of_expected_prices", "expected_price_ratio"]].mean()
```

Read this block:

- Line 524, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 525, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 526: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 527, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 528, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 529, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 531–537:

```
group = group.join(means)
group["eligible_households"] = table.groupby(["policy", "group"]).size()
group.to_csv(args.output / "targeted_revenue_by_switching_group.csv")
# Aggregate terminal offers conditional on first-period focal purchase.
# Weight by branch probability, rather than averaging household conditional means.
branch_rows = []
dynamic = table.loc[table.policy == "personalized_dynamic"]
```

Read this block:

- Line 531, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 532, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 533: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 536, `branch_rows`: Set this name to the list `[]`.
- Line 537, `dynamic`: Set this name to the entry/slice selected by `table.loc[table.policy == 'personalized_dynamic']`; brackets select from the object to their left.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 538–545:

```
for group_name, segment in dynamic.groupby("group"):
    for branch in ["focal", "no_focal"]:
        mass = segment["probability_" + branch]
        branch_price = segment["mean_p2_given_" + branch]
        probability_sum = mass.sum()
        branch_rows.append(dict(group=group_name, branch=branch,
            probability=probability_sum/len(segment),
            mean_terminal_price=(mass*branch_price).sum()/probability_sum if probability_sum else np.nan))
```

Read this block:

- Line 538: for each entry yielded by `dynamic.groupby('group')`, bind/unpack it into `(group_name, segment)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 539: for each entry yielded by `['focal', 'no_focal']`, bind/unpack it into `branch` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 540, `mass`: Set this name to the entry/slice selected by `segment['probability_' + branch]`; brackets select from the object to their left.
- Line 541, `branch_price`: Set this name to the entry/slice selected by `segment['mean_p2_given_' + branch]`; brackets select from the object to their left.
- Line 542, `probability_sum`: Set this name to the result of calling `mass.sum(...)`; the arguments and preceding method chain are shown above.
- Line 543: call `branch_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 546–554:

```
branches = pd.DataFrame(branch_rows)
paired = dynamic.assign(terminal_price_gap=(dynamic.mean_p2_given_focal - dynamic.mean_p2_given_no_focal))
paired.groupby("group").agg(
    mean_within_household_price_gap=("terminal_price_gap", "mean"),
    households_with_both_branches=("terminal_price_gap", "count")
```

```
).to_csv(args.output / "terminal_offer_within_household_gap.csv")
branches.to_csv(args.output / "terminal_offers_by_purchase.csv", index=False)
plt = plotting()
fig, ax = plt.subplots(figsize=(9, 4))
```

Read this block:

- Line 546, `branches`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 547, `paired`: Set this name to the result of calling `dynamic.assign(...)`; the arguments and preceding method chain are shown above.
- Line 548: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 552: call `branches.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 553, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 554, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

Source lines 555–555:

```
x = np.arange(3)
```

Read this block:

- Line 555, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.

Source lines 556–559:

```
for branch, offset, label in [("no_focal", -.18, "After no focal purchase"),
                              ("focal", .18, "After focal purchase")]:
    values = branches.loc[branches.branch == branch].set_index("group").reindex(["low", "middle", "high"])
    ax.bar(x+offset, values.mean_terminal_price, width=.36, label=label)
```

Read this block:

- Line 556: for each entry yielded by `[('no_focal', -0.18, 'After no focal purchase'), ('focal', 0.1...`, bind/unpack it into `(branch, offset, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 558, `values`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 559: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 560–564:

```
ax.set(xticks=x, xticklabels=["Low switching", "Middle switching", "High switching"],
       ylabel="Mean terminal price ($)", title=f"{focal.title()}: terminal offers conditional on purchase")
ax.legend()
fig.text(.5, .015, "Branch-probability-weighted means under personalized dynamic pricing; middle receives no coupons.\nConditional comparisons can reflect household composition as well as history effects.", ha="center", fontsize=8)
fig.tight_layout(rect=(0, .10, 1, 1))
```

Read this block:

- Line 560: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 562: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 563: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 564: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.legend`: Render labels previously attached to plotted series, identifying curves.

Source lines 565–566:

```
for extension in ["pdf", "png"]:
    fig.savefig(args.output / f"terminal_offers_by_purchase.{extension}", dpi=180)
```

Read this block:

- Line 565: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 566: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 567–580:

```
plt.close(fig)

households.groupby("group").agg(households=("household_code", "size"),
                                focal_eligible=("focal_eligible", "sum"),
                                switching_mean=("switching_mean", "mean"),
                                switching_mean_min=("switching_mean", "min"),
                                switching_mean_max=("switching_mean", "max"),
                                mean_posterior_sd=("switching_sd", "mean"),
                                habit_mean=("habit_mean", "mean"),
                                mean_habit_posterior_sd=("habit_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(args.output / "household_group_summary.csv")
make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta,
                       focal_flavor=focal)
```

Read this block:

- Line 567: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 569: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 579: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 581–596:

```
581 return dict(focal_flavor=focal, state_coordinates={"focal_units": focal,
582             "rival1_units": "berry" if focal == "other" else "other",
583             "rival2_units": "berry" if focal == "plain" else "plain",
584             "inside_flavors": list(FLAVORS)}, households=len(households), eligible_by_group=dict(zip(["low",
                 ↪  "middle", "high"], counts)),
585             reconstructed_nll=nll, basket_mass_error=mass_error,
586             input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
587             parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
588             posterior_draws=len(draws), posterior_seed=settings["seed"],
589             flavor_spec=settings.get("flavor_spec", "three"), flavor_categories=list(FLAVORS),
590             capacity=capacity, history_model="b_N*N + b_S*S", price_control=control,
591             initial_history="first_observed_basket", pricing_periods=2,
592             baskets=len(compositions(capacity)),
593             discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
594             type_treatment=args.type_treatment,
595             gamma_information=supply_calibration,
596             aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 581: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `list`: Materialize iterable entries as an ordered Python list.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 601.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 601–602:

```
601 def make_targeting_figures(households, table, aggregate, output, delta=.95, focal_flavor="berry"):
602     """Plot household types and individually optimized focal-flavor discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`, `focal_flavor`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 603–608:

```
plt = plotting()
groups = ["low", "middle", "high"]
labels = ["Low switching\n(bottom 25%)", "Middle switching\n(middle 50%)", "High switching\n(top 25%)"]
colors = ["#3575AD", "#999999", "#D17925"]
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
bins = np.histogram_bin_edges(households.switching_mean, bins=30)
```

Read this block:

- Line 603, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 604, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 605, `labels`: Set this name to the list `['Low switching\n(bottom 25%)', 'Middle switching\n(middl...`.
- Line 606, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 607, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 608, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 609–611:

```
for g, label, color in zip(groups, labels, colors):
    axes[0].hist(households.loc[households.group == g, "switching_mean"],
                 bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 609: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 610: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 612–619:

```
axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $b_N$)",
            ylabel="Number of households")
axes[0].legend(fontsize=9)
revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
 ↪  "revenue2"]].sum().reindex(groups)
x = np.arange(len(groups))
# Average over state-contingent terminal prices first, discount, then total.
total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 612: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 614: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 615, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 616, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 618, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 619: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 620–626:

```
axes[1].set_xticks(x, labels)
axes[1].set(title="Total Expected Revenue",
            xlabel="Consumer type group", ylabel="Total Expected Revenue")
axes[1].set_ylim(bottom=0)
axes[1].grid(axis="y", alpha=.2)
axes[1].set_axisbelow(True)
axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 620: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 621: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 623: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 624: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 625: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 626: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 627–631:

```
fig.text(.5, .015,
         f"First-period revenue + {delta:g} × expected terminal revenue. "
         f"Low/high types receive personalized {focal_flavor} offers; middle types receive no coupons.",
         ha="center", fontsize=9)
```

```
631 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 627: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 632–633:

```
632 for extension in ["png", "pdf"]:
633     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 632: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 633: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 634–641:

```
634 plt.close(fig)
635
636 fig, ax = plt.subplots(figsize=(9, 5))
637 x = np.arange(3)
638 means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    ↪  "expected_discount2"]].mean().reindex(groups)
639 ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
640 ax.bar(x+.18, 100*means.expected_discount2, width=.36,
641        label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 634: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 636, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 637, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 638, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 639: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 640: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 642–648:

```
642 ax.set_xticks(x, labels)
643 ax.set(xlabel="Consumer type group", ylim=(0, 105),
644        ylabel=f"Mean optimal {focal_flavor} discount (% of posted price)")
645 ax.grid(axis="y", alpha=.2)
646 ax.set_axisbelow(True)
647 ax.legend(fontsize=8, loc="upper left")
648 fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 642: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 643: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 645: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 646: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 647: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 648: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 649–651:

```
649 fig.text(.5, .015, f"Household-weighted means among {focal_flavor}-eligible households. Middle group receives no
    ↪  coupons by policy design.",
650          ha="center", fontsize=9)
651 fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 649: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 651: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 652–653:

```
652 for extension in ["png", "pdf"]:
653     fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 652: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 653: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 654–654:

```
654 plt.close(fig)
```

Read this block:

- Line 654: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 659.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 659–660:

```
659 def self_test():
660     """Server-only theory calibration recovery and limiting-case checks."""
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 661–664:

```
661 if __package__:
662     from .gamma_calibration import calibrate, known_prices, uncertainty
663 else:
664     from gamma_calibration import calibrate, known_prices, uncertainty
```

Read this block:

- Line 661: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 662: from `gamma_calibration`, import `calibrate, known_prices, uncertainty` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 664: from `gamma_calibration`, import `calibrate, known_prices, uncertainty` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 665–666:

```
665 gamma = np.array([-.7, .2, 1.1])
666 rows = []
```

Read this block:

- Line 665, `gamma`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 666, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.

Source lines 667–671:

```
667 for group, g in zip(["low", "middle", "high"], gamma):
668     for h0 in ["j", "0"]:
669         r = known_prices(g, h0, -1.1, 1.8, .92)
670         rows.append(dict(group=group, h0=h0, households=10,
671                          observed_p1=r["expected_p1"], observed_p2=r["expected_p2"]))
```

Read this block:

- Line 667: for each entry yielded by `zip(['low', 'middle', 'high'], gamma)`, bind/unpack it into `(group, g)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 668: for each entry yielded by `['j', '0']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 669, `r`: Set this name to the result of calling `known_prices(...)`; the arguments and preceding method chain are shown above.

- Line 670: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `known_prices`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 672–672:

```
672 a, b, fitted, rms = calibrate(pd.DataFrame(rows), gamma, .92)
```

Read this block:

- Line 672, `(a, b, fitted, rms)`: Unpack the result of calling `calibrate(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `calibrate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 673–673:

```
673 assert np.isclose(a, -1.1, atol=1e-6) and np.isclose(b, 1.8, atol=1e-6)
```

Read this block:

- Line 673: require `np.isclose(a, -1.1, atol=1e-06) and np.isclose(b, 1.8, atol=1e-06)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 674–674:

```
674 assert rms < 1e-8
```

Read this block:

- Line 674: require `rms < 1e-08` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 675–682:

```
675 for k in range(3):
676     for h0 in ["j", "0"]:
677         prior = np.eye(3)[k]
678         known = known_prices(gamma[k], h0, a, b, .92)
679         unknown = uncertainty(gamma, prior, h0, a, b, .92)
680         for key in known:
681             assert np.isclose(known[key], unknown[key], atol=1e-6)
682         assert unknown["belief_j"] == unknown["belief_0"] == prior.tolist()
```

Read this block:

- Line 675: for each entry yielded by `range(3)`, bind/unpack it into `k` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 676: for each entry yielded by `['j', '0']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 677, `prior`: Set this name to the entry/slice selected by `np.eye(3)[k]`; brackets select from the object to their left.
- Line 678, `known`: Set this name to the result of calling `known_prices(...)`; the arguments and preceding method chain are shown above.
- Line 679, `unknown`: Set this name to the result of calling `uncertainty(...)`; the arguments and preceding method chain are shown above.
- Line 680: for each entry yielded by `known`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 681: require `np.isclose(known[key], unknown[key], atol=1e-06)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 682: require `unknown['belief_j'] == unknown['belief_0'] == prior.tolist()` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `uncertainty`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.

Source lines 683–683:

```
683 print("Server supply checks passed: recovery from observed-price targets, degenerate types, fixed beliefs, wider
    ↪  price scans.")
```

Read this block:

- Line 683: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 688.

Creates optional command-line overrides, then sets parser defaults from the ordinary Python settings record. With no flags, the top-of-file parameters govern the run. Parsing arguments itself does not estimate demand or rebuild household inputs.

Definition and stated intent:

Source lines 688–688:

```
688 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 689–695:

```
689 parser = argparse.ArgumentParser(description=__doc__)
690 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
691 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
```

```
parser.add_argument("--output", type=Path, default=Path("/scratch/dtm63837/Kilts_Panel/results") / "split_empirical_outputs")
parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
                    help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
parser.add_argument("--demand-delta", type=float, default=.92)
```

Read this block:

- Line 689, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 690: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 691: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 692: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 693: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 695: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 696–702:

```
parser.add_argument("--batch-size", type=int, default=24)
parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"], default="posterior-mean",
                    help="Default treats estimated household types as known, matching base supply; optional posterior uncertainty sensitivity.")
parser.add_argument("--save-state-policies", action="store_true", help="Unsupported: this exercise saves aggregate comparisons only.")
parser.add_argument("--focal-flavor", choices=["all", *categories("five")], default="all")
parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
parser.set_defaults(**vars(settings))
```

Read this block:

- Line 696: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 697: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 699: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 700: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 701: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 702: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 703–703:

```
703 return parser.parse_args()
```

Read this block:

- Line 703: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run pricing

Original function: `run_pricing`. Source begins at line 709.

Reads the saved fit category metadata before selecting focal cases; all means all three or five fitted categories. Main jobs instead use plain_berry_pricing for two focal sellers. It validates choices and writes separate aggregate counterfactual outputs, not an equilibrium.

Definition and stated intent:

Source lines 709–709:

```
709 def run_pricing(args):
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 710–712:

```
710 if args.self_test:
711     self_test()
712     return
```

Read this block:

- Line 710: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 711: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 712: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 713–714:

```
713 if args.household_input is None or args.fit_dir is None:
714     raise ValueError("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 713: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 714: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 715–715:

```
715 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 715, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 716–718:

```
716 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
717         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
718     raise ValueError("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 716: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 718: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 719–721:

```
719 require_data_path(args.fit_dir)
720 fit = json.loads((args.fit_dir/'fit.json').read_text())
721 FLAVORS[:] = categories(fit['settings'].get('flavor_spec', 'three'))
```

Read this block:

- Line 719: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 720, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 721, `FLAVORS[:]`: Store the result of calling `categories(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 722–723:

```
722 if args.focal_flavor != 'all' and args.focal_flavor not in FLAVORS:
723     raise ValueError('Focal flavor is absent from the selected fit specification.')
```

Read this block:

- Line 722: test `args.focal_flavor != 'all' and args.focal_flavor not in FLAVORS`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 723: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 724–727:

```
724 args.output.mkdir(parents=True, exist_ok=True)
725 output_root = args.output
726 selected = list(FLAVORS) if args.focal_flavor == "all" else [args.focal_flavor]
727 reports, comparisons = {}, []
```

Read this block:

- Line 724: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 725, `output_root`: Set this name to the attribute `args.output` of the object before the dot.
- Line 726, `selected`: Set this name to the true-branch value if `args.focal_flavor == 'all'` holds, otherwise the else-branch value.
- Line 727, `(reports, comparisons)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `list`: Materialize iterable entries as an ordered Python list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 728–734:

```
728 for focal in selected:
729     args.focal_flavor = focal
730     args.output = output_root / focal
731     args.output.mkdir(parents=True, exist_ok=True)
732     reports[focal] = run_households(args)
733     for row in reports[focal]["aggregate"]:
734         comparisons.append(dict(focal_flavor=focal, **row))
```

Read this block:

- Line 728: for each entry yielded by `selected`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 729, `args.focal_flavor`: Set this name to the object currently named `focal`; plain assignment does not copy it.
- Line 730, `args.output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 731: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 732, `reports[focal]`: Store the result of calling `run_households(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 733: for each entry yielded by `reports[focal]['aggregate']`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 734: call `comparisons.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 735–742:

```
735 args.output = output_root
736 args.focal_flavor = "all" if len(selected) == len(FLAVORS) else selected[0]
737 pd.DataFrame(comparisons).to_csv(output_root / "focal_flavor_comparison.csv", index=False)
738 report = reports
739 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
740                 results=report, script_sha256=digest(Path(__file__).resolve()),
741                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
742 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 735, `args.output`: Set this name to the object currently named `output_root`; plain assignment does not copy it.
- Line 736, `args.focal_flavor`: Set this name to the true-branch value if `len(selected) == len(FLAVORS)` holds, otherwise the else-branch value.
- Line 737: call `pd.DataFrame(comparisons).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 738, `report`: Set this name to the object currently named `reports`; plain assignment does not copy it.
- Line 739, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 742: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 743–744:

```
743 (args.output / "run_notes.txt").write_text(pformat(manifest, sort_dicts=False, width=90)+"\n")
744 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 743: call `(args.output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 744: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

../final_simple/flavor_specs.py

Shared, data-free three/five category definitions and descriptor rules. Category order aligns counts, prices, histories and fixed effects. The baseline rule is preserved; five prioritizes plain, desserts, berries, other fruits, then residual other.

Module setup and direct-execution blocks

Source lines 1–11:

```
"""Flavor specifications and descriptor rules. No empirical I/O."""
import re

SPECIFICATIONS = {
    'three': ['other', 'berry', 'plain'],
    'five': ['other', 'berry', 'plain', 'other_fruits', 'desserts'],
}
PLAIN = r'plain|unflavored'
# Preserve the original three-group berry rule exactly.
BERRY = r'berry|straw|blue|rasp|black|cran|cherry|wildberry'
EXPANDED_BERRY = r'berr(?:y|ies)|straw|blue|rasp|cran'
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `re`. An as-name is a convenient local alias, not another library.
- Line 4, `SPECIFICATIONS`: Set this name to a dictionary with named entries `three`, `five`.
- Line 8, `PLAIN`: Set this name to the literal text `'plain|unflavored'`.
- Line 10, `BERRY`: Set this name to the literal text `'berry|straw|blue|rasp|black|cran|cherry|wildberry'`.
- Line 11, `EXPANDED_BERRY`: Set this name to the literal text `'berr(?:y|ies)|straw|blue|rasp|cran'`.

Source lines 12–22:

```
DESSERT = (r'vanilla|chocolat|\bpie\b|cheesecake|cake|cookie|cookies|brownie|'
           r'pudding|custard|caramel|toffee|butterscotch|fudge|tiramisu|'
           r'cobbler|\btart\b|dessert|sundae|ice\s*cream|creme\s*brulee|'
           r'cr[eè]me\s*br[uû]l[eé]e|dulce|marshmallow|praline|'
           r'cinnamon\s*roll|boston\s*cream|banana\s*cream')
OTHER_FRUIT = (r'cherry|cherries|peach|apricot|nectarine|mango|pineapple|banana|\bapple\b|'
               r'\bpear\b|orange|lemon|lime|citrus|grapefruit|tangerine|'
               r'mandarin|clementine|plum|prune|grape|raisin|fig|date|'
               r'coconut|passion\s*fruit|guava|papaya|kiwi|melon|'
               r'watermelon|cantaloupe|honeydew|pomegranate|lychee|'
               r'tropical|mixed\s*fruit|fruit\s*medley')
```

Read this block:

- Line 12, `DESSERT`: Set this name to a multiline text template; its contents are data, not Python instructions.
- Line 17, `OTHER_FRUIT`: Set this name to a multiline text template; its contents are data, not Python instructions.

categories

Original function: `categories`. Source begins at line 25.

Returns a fresh ordered category list for three or five and rejects an unknown specification. Fresh lists avoid accidental mutation of the dictionary; order defines the alignment of counts, prices, histories and fixed effects.

Definition and stated intent:

Source lines 25–25:

```
25 def categories(specification='three'):
```

`def` creates this callable; the parentheses name its inputs (`specification`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–29:

```
26 try:
27     return list(SPECIFICATIONS[specification])
28 except KeyError:
29     raise ValueError('Unknown flavor specification: '+str(specification)) from None
```

Read this block:

- Line 26: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 27: return the result of calling `list(...)`; the arguments and preceding method chain are shown above. This ends the call.
- Line 28: catch `KeyError` from the preceding try; the optional as-name holds the error object for logging.
- Line 29: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `list`: Materialize iterable entries as an ordered Python list.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `str`: Convert to text for paths, messages, or serialization.

classify descriptor

Original function: `classify_descriptor`. Source begins at line 32.

Applies explicit descriptor precedence without empirical I/O. The baseline keeps the original plain/berry/other rule. Five uses plain, then dessert, then berry, then nonberry fruit, then residual other; cherry is nonberry fruit only in the expanded rule.

Definition and stated intent:

Source lines 32–33:

```
32 def classify_descriptor(text, specification='three'):
33     """Classify a descriptor; dessert beats fruit, plain retains first priority."""
```

`def` creates this callable; the parentheses name its inputs (`text`, `specification`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–35:

```
34 categories(specification)
35 text = str(text or '').lower()
```

Read this block:

- Line 34: call `categories` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35, `text`: Set this name to the result of calling `.lower(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 36–37:

```
36 if re.search(PLAIN, text):
37     return 'plain'
```

Read this block:

- Line 36: test `re.search(PLAIN, text)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 37: return the literal text `'plain'`. This ends the call.

Source lines 38–39:

```
38 if specification == 'five' and re.search(DESSERT, text):
39     return 'desserts'
```

Read this block:

- Line 38: test `specification == 'five' and re.search(DESSERT, text)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 39: return the literal text `'desserts'`. This ends the call.

Source lines 40–41:

```
40 if re.search(BERRY if specification == 'three' else EXPANDED_BERRY, text):
41     return 'berry'
```

Read this block:

- Line 40: test `re.search(BERRY if specification == 'three' else EXPANDED_BERRY, text)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 41: return the literal text `'berry'`. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 42–43:

```
42 if specification == 'five' and re.search(OTHER_FRUIT, text):
43     return 'other_fruits'
```

Read this block:

- Line 42: test `specification == 'five' and re.search(OTHER_FRUIT, text)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 43: return the literal text `'other_fruits'`. This ends the call.

Source lines 44–44:

```
44 return 'other'
```

Read this block:

- Line 44: return the literal text `'other'`. This ends the call.

classification expression

Original function: `classification_expression`. Source begins at line 47.

Constructs the equivalent chained Polars expression for server-side classification. It returns a computation plan, not collected data. Tests compare its outcomes against the Python string classifier so the two implementations share precedence.

Definition and stated intent:

Source lines 47–48:

```
47 def classification_expression(text, specification='three'):
48     """Equivalent Polars expression for server-side reference classification."""
```

`def` creates this callable; the parentheses name its inputs (`text`, `specification`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 49–51:

```
49 import polars as pl
50 categories(specification)
51 expression = pl.when(text.str.contains(PLAIN)).then(pl.lit('plain'))
```

Read this block:

- Line 49: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 50: call `categories` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51, `expression`: Set this name to the result of calling `.then(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.

Source lines 52–53:

```
52 if specification == 'five':
53     expression = expression.when(text.str.contains(DESSERT)).then(pl.lit('desserts'))
```

Read this block:

- Line 52: test `specification == 'five'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 53, `expression`: Set this name to the result of calling `.then(...)`; the arguments and preceding method chain are shown above.

Source lines 54–54:

```
54 expression = expression.when(text.str.contains(BERRY if specification == 'three' else
    ↪  EXPANDED_BERRY)).then(pl.lit('berry'))
```

Read this block:

- Line 54, `expression`: Set this name to the result of calling `.then(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 55–56:

```
if specification == 'five':
    expression = expression.when(text.str.contains(OTHER_FRUIT)).then(pl.lit('other_fruits'))
```

Read this block:

- Line 55: test `specification == 'five'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56, `expression`: Set this name to the result of calling `.then(...)`; the arguments and preceding method chain are shown above.

Source lines 57–57:

```
return expression.otherwise(pl.lit('other'))
```

Read this block:

- Line 57: return the result of calling `expression.otherwise(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.

../final_simple/server/export_uncontrolled_pricing.py

Server-only results boundary. Explicit filenames and coefficient hashes restrict export to aggregate results and sanitized uncontrolled-fit provenance. Failed auxiliary calibrations export status/aggregate diagnostics and omit information figures.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Server-only allowlisted export with uncontrolled-fit provenance."""
from pathlib import Path
import argparse
import json
import shutil
import sys
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 4: import `json`. An as-name is a convenient local alias, not another library.
- Line 5: import `shutil`. An as-name is a convenient local alias, not another library.
- Line 6: import `sys`. An as-name is a convenient local alias, not another library.

Source lines 8–12:

```
root = next(p for p in Path(__file__).resolve().parents
            if (p/'Code/privacy/server_data_policy.py').is_file())
sys.path.insert(0, str(root/'Code/privacy'))
from server_data_policy import require_server, require_data_path
from export_rebuild_results import FIT_RESULTS, PRICING_RESULTS
```

Read this block:

- Line 8, `root`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 10: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 11: from `server_data_policy`, import `require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `export_rebuild_results`, import `FIT_RESULTS, PRICING_RESULTS` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 93–99:

```
93 if __name__ == '__main__':
94     require_server()
95     parser = argparse.ArgumentParser(description=__doc__)
96     for flag in ['fit-dir', 'pricing-dir', 'output']:
97         parser.add_argument('--'+flag, type=Path, required=True)
98     args = parser.parse_args()
99     export_results(args.fit_dir, args.pricing_dir, args.output)
```

Read this block:

- Line 93: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 94: call `require_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 96: for each entry yielded by `['fit-dir', 'pricing-dir', 'output']`, bind/unpack it into `flag` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 97: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 99: call `export_results` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `export_results`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

export results

Original function: `export_results`. Source begins at line 15.

Validates server paths and uncontrolled original-share fit metadata, checks the pricing reports against parameter hashes, and copies only explicit allowed aggregate results. It preserves calibration status and copies presentation information figures only for valid calibrations. Household tables and full internal manifests are not exported.

Definition and stated intent:

Source lines 15–15:

```
15 def export_results(fit_dir, pricing_dir, output):
```

`def` creates this callable; the parentheses name its inputs (`fit_dir`, `pricing_dir`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 16–16:

```
16 require_server()
```

Read this block:

- Line 16: call `require_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 17–18:

```
17 for path in [fit_dir, pricing_dir, output]:
18     require_data_path(path)
```

Read this block:

- Line 17: for each entry yielded by `[fit_dir, pricing_dir, output]`, bind/unpack it into `path` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 18: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–20:

```
19 fit = json.loads((fit_dir/'fit.json').read_text())
20 manifest = json.loads((pricing_dir/'run_manifest.json').read_text())
```

Read this block:

- Line 19, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 20, `manifest`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 21–22:

```
21 if fit['settings'].get('no_price_control') is not True or fit.get('price_control', False):
22     raise ValueError('Results export requires an uncontrolled fit.')
```

Read this block:

- Line 21: test `fit['settings'].get('no_price_control') is not True or fit.get('price_control...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 23–24:

```
23 if fit['settings'].get('novelty_measure', 'previous_share') != 'previous_share':
24     raise ValueError('Results export requires original previous-share novelty.')
```

Read this block:

- Line 23: test `fit['settings'].get('novelty_measure', 'previous_share') != 'previous_share'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 24: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 26–26:

```
26 import hashlib
```

Read this block:

- Line 26: import `hashlib`. An as-name is a convenient local alias, not another library.

Source lines 27–28:

```
27 def digest(path):
28     return hashlib.sha256(path.read_bytes()).hexdigest()
```

Read this block:

- Line 27: define nested callable `digest`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 29–31:

```
29 expected_fit = digest(fit_dir/'fit.json')
30 expected_parameters = digest(fit_dir/'point_estimates.csv')
31 cases = {}
```

Read this block:

- Line 29, `expected_fit`: Set this name to the result of calling `digest(...)`; the arguments and preceding method chain are shown above.
- Line 30, `expected_parameters`: Set this name to the result of calling `digest(...)`; the arguments and preceding method chain are shown above.
- Line 31, `cases`: Set this name to a dictionary from the displayed keys and values.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 32–43:

```
32 for flavor in ['plain', 'berry']:
33     report = manifest['results'][flavor]
34     if report['price_control'] is not False:
35         raise ValueError(flavor+': pricing used controls; refusing export.')
36     if (report['fit_sha256'] != expected_fit or
37             report['parameter_sha256'] != expected_parameters):
38         raise ValueError(flavor+': pricing provenance does not match requested fit.')
39     keys = ['price_control', 'fit_sha256', 'parameter_sha256', 'reconstructed_nll',
40             'households', 'capacity', 'history_model', 'demand_delta', 'type_treatment',
41             'flavor_spec', 'flavor_categories']
```

```
42      cases[flavor] = {key: report[key] for key in keys}
43      cases[flavor]['information_calibration_status'] = report['gamma_information'].get('status', 'complete')
```

Read this block:

- Line 32: for each entry yielded by `['plain', 'berry']`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 33, `report`: Set this name to the entry/slice selected by `manifest['results'][flavor]`; brackets select from the object to their left.
- Line 34: test `report['price_control'] is not False`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 35: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 36: test `report['fit_sha256'] != expected_fit or report['parameter_sha256'] != expecte...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 38: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 39, `keys`: Set this name to a list containing 11 displayed entries.
- Line 42, `cases[flavor]`: Store a dictionary computed one key/value pair at a time by the comprehension in the selected entry/column.
- Line 43, `cases[flavor]['information_calibration_status']`: Store the result of calling `report['gamma_information'].get(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 44–45:

```
44 if output.exists():
45      raise FileExistsError('Refusing to overwrite an existing results export.')
```

Read this block:

- Line 44: test `output.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileExistsError`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 46–46:

```
46 output.mkdir(parents=True)
```

Read this block:

- Line 46: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 48–55:

```
48 for source, destination, names in [
49         (fit_dir, output/'no_control_fit', FIT_RESULTS),
```

```
        (pricing_dir, output/'pricing', {'focal_flavor_comparison.csv'})]:
    destination.mkdir()
    for name in sorted(names):
        item = source/name
        if item.is_file() and not item.is_symlink():
            shutil.copyfile(item, destination/name)
```

Read this block:

- Line 48: for each entry yielded by `[(fit_dir, output / 'no_control_fit', FIT_RESULTS), (pricing_d...`, bind/unpack it into `(source, destination, names)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 51: call `destination.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: for each entry yielded by `sorted(names)`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53, `item`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 54: test `item.is_file() and (not item.is_symlink())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 55: call `shutil.copyfile` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 56–63:

```
for flavor in ['plain', 'berry']:
    destination = output/'pricing'/flavor
    destination.mkdir()
    for name in sorted(PRICING_RESULTS | {'information_calibration_status.json',
                                          'information_calibration_moments.csv'}):
        item = pricing_dir/flavor/name
        if item.is_file() and not item.is_symlink():
            shutil.copyfile(item, destination/name)
```

Read this block:

- Line 56: for each entry yielded by `['plain', 'berry']`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 57, `destination`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58: call `destination.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: for each entry yielded by `sorted(PRICING_RESULTS | {'information_calibration_status.json..`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 61, `item`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 62: test `item.is_file() and (not item.is_symlink())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 63: call `shutil.copyfile` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 65–77:

```
for flavor in ['plain', 'berry']:
    if cases[flavor]['information_calibration_status'] != 'complete':
        continue
    target = output/'empirical_information'/flavor
    target.mkdir(parents=True)
    for source_name, target_name in [('profits', 'revenues'),
                                     ('information_value', 'information_value'),
                                     ('first_prices', 'first_prices'),
                                     ('terminal_prices', 'terminal_prices')]:
        source = pricing_dir/flavor/('information_friction_'+source_name+'.pdf')
        if not source.is_file() or source.is_symlink():
            raise ValueError('Missing presentation figure: '+str(source))
        shutil.copyfile(source, target/(target_name+'.pdf'))
```

Read this block:

- Line 65: for each entry yielded by `['plain', 'berry']`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 66: test `cases[flavor]['information_calibration_status'] != 'complete'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 67: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 68, `target`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69: call `target.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 70: for each entry yielded by `[('profits', 'revenues'), ('information_value', 'information_v...`, bind/unpack it into `(source_name, target_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 74, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 75: test `not source.is_file() or source.is_symlink()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 76: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 77: call `shutil.copyfile` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.

Source lines 78–90:

```
provenance = dict(price_control=False, novelty_measure='previous_share',
                  flavor_spec=fit['settings'].get('flavor_spec', 'three'),
                  cases=cases, script_sha256=manifest['script_sha256'],
                  solver_sha256=manifest['solver_sha256'])
(output/'pricing_provenance.json').write_text(json.dumps(provenance, indent=2)+'\n')
(output/'README.txt').write_text(
    'Uncontrolled demand fit; original previous-share novelty.\n'
    'Plain/berry coupons and policy benchmarks use the same uncontrolled fit.\n'
    'Check pricing_provenance.json for auxiliary calibration status.\n'
    'Rejected calibrations have diagnostics, not information-value figures.\n'
    'Copy only this RESULTS_ONLY directory. No household records, weekly\n'
    'inputs, full internal manifests, or research logs are included.\n'
    'Slide sources and existing local assets have not been overwritten.\n')
```

Read this block:

- Line 78, `provenance`: Set this name to the result of calling `dict(...)`; the arguments and preceding method

chain are shown above.
- Line 82: call `(output / 'pricing_provenance.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `(output / 'README.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

export results.digest

Original function: `export_results.digest`. Source begins at line 27.

Computes SHA-256 of a saved fit or parameter result file for provenance comparison. These hashes verify which coefficients the pricing run used; the function is not a research-record reader.

Definition and stated intent:

Source lines 27–27:

```
27 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 28–28:

```
28 return hashlib.sha256(path.read_bytes()).hexdigest()
```

Read this block:

- Line 28: return the result of calling `.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

../final_simple/server/run_after_data.py

Current server pipeline after the expensive build. It runs summaries and uncontrolled estimates for both category specifications, then pricing and allowlisted exports, with separate outputs and cross-spec comparisons.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Server-only two-specification pipeline starting from a completed data build."""
2 from pathlib import Path
3 import argparse
4 import json
5 import os
6 import shutil
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 4: import `json`. An as-name is a convenient local alias, not another library.
- Line 5: import `os`. An as-name is a convenient local alias, not another library.

- Line 6: import `shutil`. An as-name is a convenient local alias, not another library.

Source lines 7–13:

```
import subprocess
import sys

project = next(p for p in Path(__file__).resolve().parents
               if (p/'Code/privacy/server_data_policy.py').is_file())
sys.path.insert(0, str(project/'Code/privacy'))
from server_data_policy import require_server, require_data_path
```

Read this block:

- Line 7: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 8: import `sys`. An as-name is a convenient local alias, not another library.
- Line 10, `project`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 12: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 13: from `server_data_policy`, import `require_server, require_data_path` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 133–145:

```
if __name__ == '__main__':
    require_server()
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('rebuild_dir', type=Path)
    parser.add_argument('--flavors', type=Path, required=True)
    parser.add_argument('--first-year', type=int, default=2006)
    parser.add_argument('--last-year', type=int, default=2015)
    parser.add_argument('--household-batches', type=int, default=16)
    parser.add_argument('--draws', type=int, default=128)
    parser.add_argument('--starts', type=int, default=3)
    parser.add_argument('--maxiter', type=int, default=600)
    parser.add_argument('--seed', type=int, default=306)
    run_pipeline(parser.parse_args())
```

Read this block:

- Line 133: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 134: call `require_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 136: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 137: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 139: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 141: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 142: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 143: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 144: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 145: call `run_pipeline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `run_pipeline`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run pipeline

Original function: `run_pipeline`. Source begins at line 16.

Requires a completed server data build and a new job directory. It runs separate summaries and uncontrolled fits for three/five, then both plain/berry pricing exercises; it exports aggregate results, statuses and cross-spec comparisons. Raw conversion/matching is never invoked, and existing outputs remain intact.

Definition and stated intent:

Source lines 16–16:

```
16 def run_pipeline(args):
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–19:

```
17 require_server()
18 require_data_path(args.rebuild_dir)
19 require_data_path(args.flavors)
```

Read this block:

- Line 17: call `require_server` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 18: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 19: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `require_server`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `require_data_path`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 20–21:

```
20 if not os.environ.get('SLURM_JOB_ID'):
21     raise ValueError('Submit with the accompanying sbatch shell.')
```

Read this block:

- Line 20: test `not os.environ.get('SLURM_JOB_ID')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 21: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 22–22:

```
22 data = args.rebuild_dir/'data'
```

Read this block:

- Line 22, `data`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 23–26:

```
23 for name in ['merged_panel.parquet', 'yogurt_estimation_sample.parquet']:
24     require_data_path(data/name)
25     if not (data/name).is_file():
26         raise FileNotFoundError('Completed data build missing '+str(data/name))
```

Read this block:

- Line 23: for each entry yielded by `['merged_panel.parquet', 'yogurt_estimation_sample.parquet']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 24: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 25: test `not (data / name).is_file()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 27–28:

```
27 if not args.flavors.is_file():
28     raise FileNotFoundError('Missing descriptor reference: '+str(args.flavors))
```

Read this block:

- Line 27: test `not args.flavors.is_file()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 29–30:

```
29 work = args.rebuild_dir/('after_data_'+os.environ['SLURM_JOB_ID'])
30 require_data_path(work)
```

Read this block:

- Line 29, `work`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 30: call `require_data_path` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 31–32:

```
31 if work.exists():
32     raise FileExistsError('Refusing to overwrite an existing after-data run.')
```

Read this block:

- Line 31: test `work.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileExistsError`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 33–36:

```
33 work.mkdir()
34 os.environ['MPLCONFIGDIR'] = str(work/'matplotlib_cache')
35 os.environ['PYTHONPYCACHEPREFIX'] = str(work/'python_cache')
36 source = project/'Code/final_simple'
```

Read this block:

- Line 33: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34, `os.environ['MPLCONFIGDIR']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 35, `os.environ['PYTHONPYCACHEPREFIX']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 36, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 37–39:

```
37 def call(script, *arguments):
38     subprocess.run([sys.executable, str(source/script), *map(str, arguments)],
39                    cwd=work, check=True)
```

Read this block:

- Line 37: define nested callable `call`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 40–40:

```
reports = {}
```

Read this block:

- Line 40, `reports`: Set this name to a dictionary from the displayed keys and values.

Source lines 43–81:

```
for spec in ['three', 'five']:
    folder = work/spec
    summary = folder/'data/summary_outputs'
    fit = folder/'fits/no_control'
    print('\nSUMMARY / ESTIMATION: '+spec, flush=True)
    call('data/summaries.py', '--data-dir', data, '--flavors', args.flavors,
         '--flavor-spec', spec, '--output', summary,
         '--first-year', args.first_year, '--last-year', args.last_year,
         '--household-batches', args.household_batches)
    call('demand/estimation.py', '--sample', summary/'yogurt_estimation_sample.parquet',
         '--flavors', summary/'estimation_flavor_reference.csv',
         '--flavor-spec', spec, '--output', fit, '--no-price-control',
         '--novelty-measure', 'previous_share', '--model', 'heterogeneous_history',
         '--capacity', 7, '--first-year', args.first_year, '--last-year', args.last_year,
         '--draws', args.draws, '--starts', args.starts, '--seed', args.seed,
         '--maxiter', args.maxiter, '--se-method', 'hessian')
    reports[spec] = dict(status='estimated')
    # Stage population estimates immediately, without exporting research inputs.
    from export_uncontrolled_pricing import FIT_RESULTS
    estimate_export = work/'RESULTS_ONLY'/spec/'no_control_fit'
    estimate_export.mkdir(parents=True)
    for name in sorted(FIT_RESULTS):
        item = fit/name
        if item.is_file() and not item.is_symlink():
            shutil.copyfile(item, estimate_export/name)
    fit_info = json.loads((fit/'fit.json').read_text())
    allowed = ['model', 'price_control', 'flavor_spec', 'flavor_categories',
               'novelty_measure', 'households', 'weeks', 'converged', 'inference',
               'capacity', 'nll', 'aic', 'bic', 'message']
    (estimate_export/'fit_summary.json').write_text(json.dumps(
        {k: fit_info[k] for k in allowed}, indent=2)+'\n')
    target = work/'RESULTS_ONLY'/spec/'summary_statistics'
    for item in summary.rglob('*'):
        if (item.is_file() and not item.is_symlink() and
                item.suffix.lower() in {'.tex', '.pdf', '.png'} and
                not any(p.startswith('summary-work-') for p in item.relative_to(summary).parts)):
            destination = target/item.relative_to(summary)
            destination.parent.mkdir(parents=True, exist_ok=True)
            shutil.copyfile(item, destination)
```

Read this block:

- Line 43: for each entry yielded by `['three', 'five']`, bind/unpack it into `spec` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 44, `folder`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45, `summary`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 46, `fit`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `call` with the displayed arguments. No assignment stores its return here; its documented side

effect or assertion is what matters.
- Line 52: call `call` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59, `reports[spec]`: Store the result of calling `dict(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 61: from `export_uncontrolled_pricing`, import `FIT_RESULTS` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 62, `estimate_export`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 63: call `estimate_export.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: for each entry yielded by `sorted(FIT_RESULTS)`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 65, `item`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66: test `item.is_file() and (not item.is_symlink())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 67: call `shutil.copyfile` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68, `fit_info`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 69, `allowed`: Set this name to a list containing 14 displayed entries.
- Line 72: call `(estimate_export / 'fit_summary.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74, `target`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 75: for each entry yielded by `summary.rglob('*')`, bind/unpack it into `item` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 76: test `item.is_file() and (not item.is_symlink()) and (item.suffix.lower() in {'.tex...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 79, `destination`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80: call `destination.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 81: call `shutil.copyfile` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `call`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 82–82:

```
82 failed = False
```

Read this block:

- Line 82, `failed`: Set this name to the literal value `False`.

Source lines 83–108:

```
for spec in ['three', 'five']:
    folder = work/spec
    fit = folder/'fits/no_control'
    pricing = folder/'pricing/plain_berry'
    print('\nPRICING: '+spec, flush=True)
    try:
        call('demand/plain_berry_pricing.py', '--household-input', fit/'weekly_inputs.csv',
             '--fit-dir', fit, '--demand-delta', .92, '--output', pricing)
        subprocess.run([sys.executable, str(project/'Code/final_draft/seminar_pres/make_policy_figures.py'),
                        '--output', str(pricing)], cwd=work, check=True)
        # The exporter requires a new directory; merge only its allowlisted output.
        staged = folder/'RESULTS_ONLY'
        call('server/export_uncontrolled_pricing.py', '--fit-dir', fit,
             '--pricing-dir', pricing, '--output', staged)
        shutil.copytree(staged, work/'RESULTS_ONLY'/spec, dirs_exist_ok=True)
        provenance = json.loads((staged/'pricing_provenance.json').read_text())
        reports[spec] = dict(status='complete', information_calibration={
            flavor: case['information_calibration_status']
            for flavor, case in provenance['cases'].items()})
    except subprocess.CalledProcessError as error:
        failed = True
        reports[spec] = dict(status='pricing_failed', returncode=error.returncode)
        print('Pricing failed for '+spec+'; continuing the other specification.', flush=True)
    finally:
        # Only the exact weekly file created by this job is removed.
        (fit/'weekly_inputs.csv').unlink(missing_ok=True)
```

Read this block:

- Line 83: for each entry yielded by `['three', 'five']`, bind/unpack it into `spec` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 84, `folder`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `fit`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 86, `pricing`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 89: call `call` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94, `staged`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 95: call `call` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `shutil.copytree` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98, `provenance`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 99, `reports[spec]`: Store the result of calling `dict(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 102: catch `subprocess.CalledProcessError` from the preceding try; the optional as-name holds the

error object for logging.
- Line 103, `failed`: Set this name to the literal value `True`.
- Line 104, `reports[spec]`: Store the result of calling `dict(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 108: call `(fit / 'weekly_inputs.csv').unlink` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 110–111:

```
110 import pandas as pd
111 comparisons, parameters = [], []
```

Read this block:

- Line 110: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 111, `(comparisons, parameters)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Source lines 112–119:

```
112 for spec in ['three', 'five']:
113     estimate_file = work/'RESULTS_ONLY'/spec/'no_control_fit/estimates.csv'
114     if estimate_file.is_file():
115         parameters.append(pd.read_csv(estimate_file).assign(flavor_spec=spec))
116     for flavor in ['plain', 'berry']:
117         file = work/'RESULTS_ONLY'/spec/'pricing'/flavor/'pricing_policy_benchmarks.csv'
118         if file.is_file():
119             comparisons.append(pd.read_csv(file).assign(flavor_spec=spec, focal_flavor=flavor))
```

Read this block:

- Line 112: for each entry yielded by `['three', 'five']`, bind/unpack it into `spec` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 113, `estimate_file`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 114: test `estimate_file.is_file()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 115: call `parameters.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116: for each entry yielded by `['plain', 'berry']`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 117, `file`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 118: test `file.is_file()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 119: call `comparisons.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 120–122:

```
120 if comparisons:
121     pd.concat(comparisons, ignore_index=True).to_csv(
122         work/'RESULTS_ONLY/pricing_specification_comparison.csv', index=False)
```

Read this block:

- Line 120: test `comparisons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 121: call `pd.concat(comparisons, ignore_index=True).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 123–125:

```
123 if parameters:
124     pd.concat(parameters, ignore_index=True).to_csv(
125         work/'RESULTS_ONLY/estimate_specification_comparison.csv', index=False)
```

Read this block:

- Line 123: test `parameters`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 124: call `pd.concat(parameters, ignore_index=True).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 126–128:

```
126 (work/'RESULTS_ONLY/run_status.json').write_text(json.dumps(reports, indent=2)+'\n')
127 print('\nRecover ONLY: '+str(work/'RESULTS_ONLY'), flush=True)
128 print(json.dumps(reports, indent=2), flush=True)
```

Read this block:

- Line 126: call `(work / 'RESULTS_ONLY/run_status.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 129–130:

```
129 if failed:
130     raise SystemExit('One or more pricing stages failed; exported estimates and summaries are retained.')
```

Read this block:

- Line 129: test `failed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 130: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `SystemExit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run pipeline.call

Original function: `run_pipeline.call`. Source begins at line 37.

Builds a subprocess argument list using the current Python interpreter, a source script path and explicit settings. check=True raises on failure so the driver cannot label a failed child as completed; the working directory is server-side.

Definition and stated intent:

Source lines 37–37:

```
37 def call(script, *arguments):
```

`def` creates this callable; the parentheses name its inputs (`script`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 38–39:

```
38 subprocess.run([sys.executable, str(source/script), *map(str, arguments)],
39                cwd=work, check=True)
```

Read this block:

- Line 38: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `str`: Convert to text for paths, messages, or serialization.
- `map`: Apply a callable to each iterable entry lazily, such as converting result scalars to Python float.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/supply/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 """Supply research scripts."""
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

../final_simple/supply/pricing_model.py

Reusable pure binary-theory numerical solver and outcome helpers. It is separate from empirical category grouping; changes to three/five yogurt categories do not rewrite these theoretical utilities.

Module setup and direct-execution blocks

Source lines 1–10:

```
"""Shared binary supply calculations for the separate information exercises.

These preserve the baseline equations in supply_simulation.py. Only NumPy and
pandas are needed for calculations; matplotlib is loaded when drawing figures.
"""
from pathlib import Path
import math
import os
import tempfile
import numpy as np
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: import `math`. An as-name is a convenient local alias, not another library.
- Line 8: import `os`. An as-name is a convenient local alias, not another library.
- Line 9: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 10: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 11–14:

```
import pandas as pd

CASES = ["known_state", "hidden_initial_state", "ex_ante_commitment"]
LABELS = dict(zip(CASES, ["Known state (base)", "Hidden initial state", "Ex ante commitment"]))
```

Read this block:

- Line 11: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 13, `CASES`: Set this name to the list `['known_state', 'hidden_initial_state', 'ex_ante_commitme...`.
- Line 14, `LABELS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

expit

Original function: `expit`. Source begins at line 17.

Evaluates the logistic probability with numerically stable exponential expressions. Its input is a utility difference; the output is a probability, not a price or a simulated purchase.

Definition and stated intent:

Source lines 17–17:

```
def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 18–19:

```
18 x = np.asarray(x)
19 small = np.exp(-np.abs(x))
```

Read this block:

- Line 18, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 19, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 20–20:

```
20 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 20: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

golden maximum

Original function: `golden_maximum`. Source begins at line 22.

Refines a bracketed scalar maximizer by golden-section interval reduction, reusing previous function values. This is a numerical search, not a proof that a multimodal objective has only one peak.

Definition and stated intent:

Source lines 22–22:

```
22 def golden_maximum(function, lo, hi):
```

`def` creates this callable; the parentheses name its inputs (`function`, `lo`, `hi`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 23–25:

```
23 ratio = (math.sqrt(5)-1)/2
24 left, right = hi-ratio*(hi-lo), lo+ratio*(hi-lo)
25 fl, fr = function(left), function(right)
```

Read this block:

- Line 23, `ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 24, `(left, right)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 25, `(fl, fr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 26–36:

```
for _ in range(100):
    if hi-lo <= 1e-10*(1+abs(left)+abs(right)):
        break
    if fl > fr:
        hi, right, fr = right, left, fl
        left = hi-ratio*(hi-lo)
        fl = function(left)
    else:
        lo, left, fl = left, right, fr
        right = lo+ratio*(hi-lo)
        fr = function(right)
```

Read this block:

- Line 26: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27: test `hi - lo <= 1e-10 * (1 + abs(left) + abs(right))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 29: test `fl > fr`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 30, `(hi, right, fr)`: Unpack the tuple `(right, left, fl)` into these targets in order.
- Line 31, `left`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 32, `fl`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.
- Line 34, `(lo, left, fl)`: Unpack the tuple `(left, right, fr)` into these targets in order.
- Line 35, `right`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 36, `fr`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 37–37:

```
return (left, fl) if fl > fr else (right, fr)
```

Read this block:

- Line 37: return the true-branch value if `fl > fr` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

scalar maximum

Original function: `scalar_maximum`. Source begins at line 39.

Evaluates a scalar objective on a grid, refines every detected local peak and includes endpoints, then keeps the best candidate. Hidden-state mixtures can be multimodal, so assuming a single global peak would be unsafe.

Definition and stated intent:

Source lines 39–40:

```
39 def scalar_maximum(function, bounds, points=161):
40     """Search every grid-local maximum, including boundaries (mixtures may multimode)."""
```

`def` creates this callable; the parentheses name its inputs (`function`, `bounds`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 41–41:

```
41 grid = np.linspace(*bounds, points)
```

Read this block:

- Line 41, `grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 43–48:

```
43 try:
44     values = np.asarray(function(grid))
45     if values.shape != grid.shape:
46         raise ValueError("Scalar objective")
47 except (ValueError, TypeError):
48     values = np.array([function(x) for x in grid])
```

Read this block:

- Line 43: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 44, `values`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 45: test `values.shape != grid.shape`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 46: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 47: catch `(ValueError, TypeError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 48, `values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 49–49:

```
49 candidates = [(values[0], grid[0]), (values[-1], grid[-1])]
```

Read this block:

- Line 49, `candidates`: Set this name to a list containing 2 displayed entries.

Source lines 50–53:

```
50 for i in range(1, points - 1):
51     if values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i - 1], values[i + 1]):
52         price, value = golden_maximum(function, grid[i - 1], grid[i + 1])
53         candidates.append((value, price))
```

Read this block:

- Line 50: for each entry yielded by `range(1, points - 1)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 51: test `values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i -...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 52, `(price, value)`: Unpack the result of calling `golden_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 53: call `candidates.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `golden_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

Source lines 54–54:

```
54 value, price = max(candidates)
```

Read this block:

- Line 54, `(value, price)`: Unpack the result of calling `max(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 55–55:

```
55 return float(price), float(value)
```

Read this block:

- Line 55: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

static price

Original function: `static_price`. Source begins at line 57.

Solves the transformed binary-logit price first-order condition by bracketing and bisection for signed alpha<0. The effective cost can incorporate continuation value, allowing introductory subsidies; do not impose the coupon floor from empirical pricing on this theoretical solver.

Definition and stated intent:

Source lines 57–58:

```
57 def static_price(intercept, alpha, effective_cost=0.):
58     """Signed alpha<0; monotone transformed FOC, including introductory subsidies."""
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–60:

```
59 def foc(p):
60     return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 59: define nested callable `foc`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 61–61:

```
61 lo, hi = effective_cost, effective_cost+max(2., 2/abs(alpha))
```

Read this block:

- Line 61, `(lo, hi)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 62–63:

```
62 while foc(hi) > 0:
63     hi = lo+2*(hi-lo)
```

Read this block:

- Line 62: repeat while `foc(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 63, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `foc`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 64–69:

```
64 for _ in range(100):
65     mid = (lo+hi)/2
66     if foc(mid) > 0:
67         lo = mid
68     else:
69         hi = mid
```

Read this block:

- Line 64: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 65, `mid`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66: test `foc(mid) > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 67, `lo`: Set this name to the object currently named `mid`; plain assignment does not copy it.
- Line 69, `hi`: Set this name to the object currently named `mid`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 70–70:

```
70 return (lo+hi)/2
```

Read this block:

- Line 70: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

static price.foc

Original function: `static_price.foc`. Source begins at line 59.

This nested function returns the transformed price first-order-condition residual for a trial scalar. Captured slope, intercept and effective cost remain fixed during the root search. A zero is a candidate optimal price under the maintained monotonicity argument.

Definition and stated intent:

Source lines 59–59:

```
59 def foc(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 60–60:

```
60 return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 60: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.

supply solution

Original function: `supply_solution`. Source begins at line 72.

Solves the known-state baseline, hidden-initial-state case, and commitment case for one fixed type and prior. Both terminal histories enter expected profits. The hidden-state case pools initial histories within a known type; it does not pool different gamma types. The commitment case chooses two numbers before the first purchase and cannot adapt p2 to h1.

Definition and stated intent:

Source lines 72–73:

```
72 def supply_solution(gamma, prior=.5, beta=2., alpha=-1., delta=.92, cost=0.):
73     """Known type; hidden h0, then either observe h1 or commit to p1,p2."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `prior`, `beta`, `alpha`, `delta`, `cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 74–76:

```
74 if not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all()
75         and alpha < 0 and 0 <= prior <= 1 and 0 <= delta <= 1):
76     raise ValueError("Need finite inputs, alpha<0, prior/delta in [0,1].")
```

Read this block:

- Line 74: test `not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all() and alpha < ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 76: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 78–83:

```
78 intercepts = beta + np.array([-gamma, gamma])
79 weights = np.array([prior, 1 - prior])
80 p2 = np.array([static_price(x, alpha, cost) for x in intercepts])
81 s2 = expit(intercepts + alpha * p2)
82 W2 = (p2 - cost) * s2
83 adjustment = delta * (W2[0] - W2[1])
```

Read this block:

- Line 78, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 79, `weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 80, `p2`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 81, `s2`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 82, `W2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `adjustment`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `static_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 85–86:

```
85 def share1(p):
86     return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 85: define nested callable `share1`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 88–90:

```
88 def adaptive_value(p):
89     s = share1(p)
90     return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 88: define nested callable `adaptive_value`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 92–96:

```
92 scale = (20 + abs(beta) + abs(gamma) + abs(alpha * cost)) / abs(alpha)
93 bounds = (cost - scale, cost + scale)
94 pooled_p1, adaptive = scalar_maximum(adaptive_value, bounds)
95 # Rechecking a wider domain catches a binding numerical search boundary.
96 wide_p1, wide_value = scalar_maximum(adaptive_value, (cost - 2*scale, cost + 2*scale), 321)
```

Read this block:

- Line 92, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 93, `bounds`: Set this name to a tuple containing 2 displayed entries.
- Line 94, `(pooled_p1, adaptive)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 96, `(wide_p1, wide_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 97–98:

```
97 if wide_value > adaptive + 1e-8:
98     pooled_p1, adaptive = wide_p1, wide_value
```

Read this block:

- Line 97: test `wide_value > adaptive + 1e-08`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 98, `(pooled_p1, adaptive)`: Unpack the tuple `(wide_p1, wide_value)` into these targets in order.

Source lines 99–100:

```
99 known_p1 = np.array([static_price(x, alpha, cost - adjustment) for x in intercepts])
100 known_values = []
```

Read this block:

- Line 99, `known_p1`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 100, `known_values`: Set this name to the list `[]`.

Source lines 101–103:

```
101 for x, price in zip(intercepts, known_p1):
102     s = expit(x + alpha * price)
103     known_values.append(s*(price-cost) + delta*(s*W2[0]+(1-s)*W2[1]))
```

Read this block:

- Line 101: for each entry yielded by `zip(intercepts, known_p1)`, bind/unpack it into `(x, price)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 102, `s`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 103: call `known_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 105–112:

```
105 def committed_profile(second, return_price=False):
106     terminal = expit(intercepts + alpha * second)
107     Wj, W0 = terminal*(second-cost)
108     def objective(first):
109         s = share1(first)
110         return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
111     price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
112     return (price, value) if return_price else value
```

Read this block:

- Line 105: define nested callable `committed_profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 116–127:

```
116 if delta == 0:
117     cp1, cp2, committed = pooled_p1, static_price(beta, alpha, cost), adaptive
118 else:
119     cp2, committed = scalar_maximum(committed_profile, (cost, cost+scale))
120     cp1, committed = committed_profile(cp2, True)
121     # Independent wider and denser profile scan.
122     check_p2, check_value = scalar_maximum(committed_profile, (cost, cost+2*scale), 321)
123     if check_value > committed+1e-9:
124         cp2 = check_p2
125         cp1, committed = committed_profile(cp2, True)
126     if min(cp2-cost, cost+2*scale-cp2) < scale*1e-5:
127         raise RuntimeError("Committed optimum is near a search bound; inspect calibration.")
```

Read this block:

- Line 116: test `delta == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 117, `(cp1, cp2, committed)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 119, `(cp2, committed)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 120, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `(check_p2, check_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 123: test `check_value > committed + 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 124, `cp2`: Set this name to the object currently named `check_p2`; plain assignment does not copy it.
- Line 125, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 126: test `min(cp2 - cost, cost + 2 * scale - cp2) < scale * 1e-05`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 127: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `committed_profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 128–128:

```
128 known = float(weights @ known_values)
```

Read this block:

- Line 128, `known`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 129–130:

```
129 if not known + 1e-7 >= adaptive >= committed - 1e-7:
130     raise AssertionError("Information-value ordering failed.")
```

Read this block:

- Line 129: test `not known + 1e-07 >= adaptive >= committed - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 130: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 131–137:

```
131 s = share1(pooled_p1)
132 derivative = float(weights @ (alpha*expit(intercepts+alpha*pooled_p1)
133                              * expit(-intercepts-alpha*pooled_p1)))
134 committed_first_share = share1(cp1)
135 committed_first_derivative = float(weights @ (alpha*expit(intercepts+alpha*cp1)*expit(-intercepts-alpha*cp1)))
136 terminal = expit(intercepts+alpha*cp2)
137 terminal_weights = np.array([committed_first_share, 1-committed_first_share])
```

Read this block:

- Line 131, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 132, `derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 134, `committed_first_share`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 135, `committed_first_derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 136, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 137, `terminal_weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 138–139:

```
committed_foc1 =
    committed_first_share+committed_first_derivative*(cp1-cost+delta*(cp2-cost)*(terminal[0]-terminal[1]))
committed_foc2 = float(terminal_weights @ (terminal+alpha*(cp2-cost)*terminal*(1-terminal))) if delta > 0 else 0.
```

Read this block:

- Line 138, `committed_foc1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 139, `committed_foc2`: Set this name to the true-branch value if `delta > 0` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 140–148:

```
return dict(gamma=gamma, prior_inside=prior, beta=beta, alpha=alpha, delta=delta, cost=cost,
            known_p1_inside=known_p1[0], known_p1_outside=known_p1[1],
            known_value=known, hidden_p1=pooled_p1, adaptive_p2_inside=p2[0],
            adaptive_p2_outside=p2[1], hidden_value=adaptive,
            committed_p1=cp1, committed_p2=cp2, committed_value=committed,
            initial_information_value=known-adaptive,
            second_period_flexibility_value=adaptive-committed,
            committed_foc1=committed_foc1, committed_foc2=committed_foc2,
            hidden_foc=s+derivative*(pooled_p1-cost+adjustment))
```

Read this block:

- Line 140: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

supply solution.share1

Original function: `supply_solution.share1`. Source begins at line 85.

Returns first-period binary-logit demand, or its initial-state mixture, at a trial price. The function closes over the current theoretical type and prior. Initial history is integrated as probabilities, not averaged as utilities.

Definition and stated intent:

Source lines 85–85:

```
def share1(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–86:

```
return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 86: return matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

supply solution.adaptive value

Original function: `supply_solution.adaptive_value`. Source begins at line 88.

Evaluates current profit plus discounted, state-contingent optimized terminal values. The first choice changes the probabilities of both continuation histories, which is why its derivative contains their value difference.

Definition and stated intent:

Source lines 88–88:

```
88 def adaptive_value(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 89–89:

```
89 s = share1(p)
```

Read this block:

- Line 89, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 90–90:

```
90 return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 90: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

supply solution.committed profile

Original function: `supply_solution.committed_profile`. Source begins at line 105.

For a trial first price, solves the best single committed terminal price and returns its value/profile. This inner optimization must respect the information restriction: p2 is common across realized h1.

Definition and stated intent:

Source lines 105–105:

```
105 def committed_profile(second, return_price=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `return_price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 106–107:

```
106 terminal = expit(intercepts + alpha * second)
107 Wj, W0 = terminal*(second-cost)
```

Read this block:

- Line 106, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 107, `(Wj, W0)`: Unpack multiplication of the displayed operands; parentheses control which subexpressions are evaluated together into these targets in order.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 108–110:

```
108 def objective(first):
109     s = share1(first)
110     return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 108: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 111–111:

```
111 price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
```

Read this block:

- Line 111, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 112–112:

```
112 return (price, value) if return_price else value
```

Read this block:

- Line 112: return the true-branch value if `return_price` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

supply solution.committed profile.objective

Original function: `supply_solution.committed_profile.objective`. Source begins at line 108.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 108–108:

```
108 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 109–109:

```
109 s = share1(first)
```

Read this block:

- Line 109, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 110–110:

```
110 return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 110: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

plotting

Original function: `plotting`. Source begins at line 150.

Creates a noninteractive Matplotlib plotting interface and writable cache for theoretical figures. Importing this helper does not read purchase data.

Definition and stated intent:

Source lines 150–150:

```
150 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 151–154:

```
151 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
152 import matplotlib
153 matplotlib.use("Agg")
154 import matplotlib.pyplot as plt
```

Read this block:

- Line 151: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 152: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 153: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 154: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 155–155:

```
155 return plt
```

Read this block:

- Line 155: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

outcome tables

Original function: `outcome_tables`. Source begins at line 157.

Exports history-specific prices, quantities and profits plus properly probability-weighted ex ante outcomes. Conditional realized-history values and expected profits answer different questions; averaging conditional profits without their path probabilities would be wrong.

Definition and stated intent:

Source lines 157–158:

```
157 def outcome_tables(result):
158     """All four realized histories per case, plus correctly weighted ex ante objects."""
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 159–160:

```
159 g, beta, alpha, delta, cost, pi = [result[k] for k in ["gamma", "beta", "alpha", "delta", "cost", "prior_inside"]]
160 paths, summary = [], []
```

Read this block:

- Line 159, `(g, beta, alpha, delta, cost, pi)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 160, `(paths, summary)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 161–199:

```
161 for case in CASES:
162     case_paths = []
163     for h0, weight in [("j", pi), ("0", 1-pi)]:
164         if case == "known_state":
165             p1 = result["known_p1_inside" if h0 == "j" else "known_p1_outside"]
166         else:
167             p1 = result["hidden_p1" if case == "hidden_initial_state" else "committed_p1"]
168         s1 = float(expit(beta+(-1 if h0 == "j" else 1)*g+alpha*p1))
169         conditional_value = 0.
170         local = []
171         for h1, transition in [("j", s1), ("0", 1-s1)]:
172             p2 = result["committed_p2"] if case == "ex_ante_commitment" else result["adaptive_p2_inside" if h1 ==
    ↪ "j" else "adaptive_p2_outside"]
173             s2 = float(expit(beta+(-1 if h1 == "j" else 1)*g+alpha*p2))
174             # Realize the first choice; integrate the terminal taste shock.
175             conditional_profit = (p1-cost)*(h1 == "j")+delta*(p2-cost)*s2
176             conditional_value += transition*conditional_profit
177             row = dict(gamma=g, prior_inside=pi, case=case, h0=h0, h1=h1,
178                        initial_probability=weight, transition_probability=transition,
179                        path_probability=weight*transition, p1=p1, p2=p2,
180                        price_ratio=p1/p2 if abs(p2) > 1e-12 else np.nan,
181                        first_purchase_probability=s1, second_purchase_probability=s2,
182                        conditional_profit=conditional_profit)
183             local.append(row)
184         for row in local:
185             row["optimal_profit_given_h0"] = conditional_value
```

```
186         case_paths.extend(local)
187     probability = np.array([r["path_probability"] for r in case_paths])
188     if not np.isclose(probability.sum(), 1.):
189         raise AssertionError("Path probabilities do not sum to one.")
190     mean_p1, mean_p2, profit = [float(probability @ np.array([r[k] for r in case_paths]))
191                                 for k in ["p1", "p2", "conditional_profit"]]
192     expected_ratio = float(probability @ np.array([r["price_ratio"] for r in case_paths]))
193     expected_key = {"known_state": "known_value", "hidden_initial_state": "hidden_value", "ex_ante_commitment":
    ↪ "committed_value"}[case]
194     if not np.isclose(profit, result[expected_key], atol=1e-8):
195         raise AssertionError("Path profits do not reproduce optimized objective.")
196     summary.append(dict(gamma=g, prior_inside=pi, case=case, expected_p1=mean_p1,
197                         expected_p2=mean_p2, ratio_of_expected_prices=mean_p1/mean_p2 if abs(mean_p2) > 1e-12 else
    ↪ np.nan,
198                         expected_price_ratio=expected_ratio, optimal_profit=profit))
199     paths.extend(case_paths)
```

Read this block:

- Line 161: for each entry yielded by `CASES`, bind/unpack it into `case` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 162, `case_paths`: Set this name to the list `[]`.
- Line 163: for each entry yielded by `[('j', pi), ('0', 1 - pi)]`, bind/unpack it into `(h0, weight)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 164: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 165, `p1`: Set this name to the entry/slice selected by `result['known_p1_inside' if h0 == 'j' else 'known_p1_outs...`; brackets select from the object to their left.
- Line 167, `p1`: Set this name to the entry/slice selected by `result['hidden_p1' if case == 'hidden_initial_state' else...`; brackets select from the object to their left.
- Line 168, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 169, `conditional_value`: Set this name to the literal value `0.0`.
- Line 170, `local`: Set this name to the list `[]`.
- Line 171: for each entry yielded by `[('j', s1), ('0', 1 - s1)]`, bind/unpack it into `(h1, transition)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 172, `p2`: Set this name to the true-branch value if `case == 'ex_ante_commitment'` holds, otherwise the else-branch value.
- Line 173, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 175, `conditional_profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 176: update `conditional_value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 177, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 183: call `local.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 184: for each entry yielded by `local`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 185, `row['optimal_profit_given_h0']`: Store the object currently named `conditional_value`; plain assignment does not copy it in the selected entry/column.
- Line 186: call `case_paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 187, `probability`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 188: test `not np.isclose(probability.sum(), 1.0)`. Execute the indented branch only when it is true;

an else/elif branch handles its alternative.
- Line 189: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 190, `(mean_p1, mean_p2, profit)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 192, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 193, `expected_key`: Set this name to the entry/slice selected by `{'known_state': 'known_value', 'hidden_initial_state': 'h...`; brackets select from the object to their left.
- Line 194: test `not np.isclose(profit, result[expected_key], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 195: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 196: call `summary.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 199: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 200–200:

```
200 return paths, summary
```

Read this block:

- Line 200: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

self test

Original function: `self_test`. Source begins at line 202.

Checks the pure binary-theory solver against special cases and independent numerical conditions. It exercises mathematical parameters only and does not validate an empirical fit.

Definition and stated intent:

Source lines 202–202:

```
202 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 203–214:

```
for gamma, prior, delta in [(0., .5, .92), (-2., .25, .92), (1.5, .75, .92),
                            (-1., 0., .92), (1., 1., .92), (1., .5, 0.)]:
    solution = supply_solution(gamma, prior, delta=delta)
    paths, summary = outcome_tables(solution)
    assert len(paths) == 12 and len(summary) == 3
    for key in ["hidden_foc", "committed_foc1", "committed_foc2"]:
        assert abs(solution[key]) < 1e-6
    if gamma == 0:
        assert np.ptp([row["optimal_profit"] for row in summary]) < 1e-7
        assert all(np.isclose(row["price_ratio"], 1., atol=1e-6) for row in paths)
    if prior in (0., 1.):
        assert np.isclose(solution["known_value"], solution["hidden_value"], atol=1e-7)
```

Read this block:

- Line 203: for each entry yielded by `[(0.0, 0.5, 0.92), (-2.0, 0.25, 0.92), (1.5, 0.75, 0.92), (-1....`, bind/unpack it into `(gamma, prior, delta)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 205, `solution`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 206, `(paths, summary)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 207: require `len(paths) == 12 and len(summary) == 3` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 208: for each entry yielded by `['hidden_foc', 'committed_foc1', 'committed_foc2']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 209: require `abs(solution[key]) < 1e-06` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 210: test `gamma == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 211: require `np.ptp([row['optimal_profit'] for row in summary]) < 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 212: require `all((np.isclose(row['price_ratio'], 1.0, atol=1e-06) for row in paths))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 213: test `prior in (0.0, 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 214: require `np.isclose(solution['known_value'], solution['hidden_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `len`: Count elements/rows rather than summing their numerical values.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 215–215:

```
215 print("Supply checks passed: all cases, four histories, probabilities, profit reconstruction, FOCs, degenerate
    ↪  priors, gamma=0, delta=0.")
```

Read this block:

- Line 215: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/supply/supply_information.py

Theoretical comparison of revealed gamma, hidden gamma with adaptive Bayesian learning, and committed common prices. Do not confuse this learning experiment with fixed beliefs in the empirically calibrated auxiliary exercise.

Module setup and direct-execution blocks

Source lines 2–7:

```
2 """Recreate supply_information for unknown gamma and observed initial history."""
3 from pathlib import Path
4 from types import SimpleNamespace
5 import argparse
6 import hashlib
7 import json
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 6: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 7: import `json`. An as-name is a convenient local alias, not another library.

Source lines 8–11:

```
8 import sys
9
10 import numpy as np
11 import pandas as pd
```

Read this block:

- Line 8: import `sys`. An as-name is a convenient local alias, not another library.
- Line 10: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 11: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 13–16:

```
if __package__:
    from .unknown_type import GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting
else:
    from unknown_type import GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting
```

Read this block:

- Line 13: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 14: from `unknown_type`, import `GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `unknown_type`, import `GAMMAS, solve, expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–24:

```
output = Path(__file__).resolve().parents[1] / "results" / "unknown_type"
probability_points = 41
beta = 2.0
alpha = -1.0
delta = 0.92
cost = 0.0
```

Read this block:

- Line 19, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 20, `probability_points`: Set this name to the literal value `41`.
- Line 21, `beta`: Set this name to the literal value `2.0`.
- Line 22, `alpha`: Set this name to the negated or sign-adjusted expression `-1.0`.
- Line 23, `delta`: Set this name to the literal value `0.92`.
- Line 24, `cost`: Set this name to the literal value `0.0`.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 155–160:

```
if __name__ == "__main__":
    settings = SimpleNamespace(output=output, probability_points=probability_points,
                               beta=beta, alpha=alpha, delta=delta, cost=cost)
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_information(settings)
```

Read this block:

- Line 155: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 158: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 159, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 160: call `run_information` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_information`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

committed policy

Original function: `committed_policy`. Source begins at line 27.

Chooses two common prices before outcomes when initial history is known but gamma is hidden. Both prices must apply across possible types; the method profiles over the terminal price and verifies first-order conditions. This is a commitment/Bayesian-theory comparison, distinct from empirical fixed beliefs.

Definition and stated intent:

Source lines 27–28:

```
27 def committed_policy(h0, weights, beta, alpha, delta, cost, points=81):
28     """Two common prices chosen before purchases; h0 is observed, gamma is not."""
```

`def` creates this callable; the parentheses name its inputs (`h0`, `weights`, `beta`, `alpha`, `delta`, `cost`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–30:

```
29 intercepts = beta+(-1 if h0 == "j" else 1)*GAMMAS
30 scale = (20+abs(beta)+max(abs(GAMMAS))+abs(alpha*cost))/abs(alpha)
```

Read this block:

- Line 29, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 30, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 32–42:

```
32 def profile(second, details=False):
33     sj = expit(beta-GAMMAS+alpha*second)
34     s0 = expit(beta+GAMMAS+alpha*second)
35 
36     def objective(first):
37         shares = expit(intercepts+alpha*np.asarray(first)[..., None])
38         return ((np.asarray(first)-cost)*(shares @ weights)
39                 + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))
40 
41     first, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale), points)
42     return (first, value) if details else value
```

Read this block:

- Line 32: define nested callable `profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 44–51:

```
if delta:
    second, value = scalar_maximum(profile, (cost, cost+2*scale), points)
    first, value = profile(second, True)
    if min(second-cost, cost+2*scale-second) < 1e-6*scale:
        raise RuntimeError("Committed terminal price is near a search bound.")
else:
    second = cost+1/abs(alpha)  # Unidentified by the objective at delta=0.
    first, value = profile(second, True)
```

Read this block:

- Line 44: test `delta`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45, `(second, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 46, `(first, value)`: Unpack the result of calling `profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 47: test `min(second - cost, cost + 2 * scale - second) < 1e-06 * scale`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 50, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `(first, value)`: Unpack the result of calling `profile(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 52–58:

```
shares = expit(intercepts+alpha*first)
sj = expit(beta-GAMMAS+alpha*second)
s0 = expit(beta+GAMMAS+alpha*second)
ds = alpha*shares*(1-shares)
foc1 = float(weights @ (shares+ds*(first-cost+delta*(second-cost)*(sj-s0))))
foc2 = float(weights @ (shares*(sj+alpha*(second-cost)*sj*(1-sj))
                        +(1-shares)*(s0+alpha*(second-cost)*s0*(1-s0)))) if delta else 0.
```

Read this block:

- Line 52, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 53, `sj`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 54, `s0`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 55, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 56, `foc1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 57, `foc2`: Set this name to the true-branch value if `delta` holds, otherwise the else-branch value.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 59–60:

```
59 if max(abs(foc1), abs(foc2)) > 1e-6:
60     raise AssertionError("Committed policy FOC failed.")
```

Read this block:

- Line 59: test `max(abs(foc1), abs(foc2)) > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 60: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 61–61:

```
61 return first, second, value
```

Read this block:

- Line 61: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

committed policy.profile

Original function: `committed_policy.profile`. Source begins at line 32.

For a candidate committed second price, optimizes the first price and returns either its value or detailed price/value pair. The details flag lets the outer optimizer use a scalar while the caller later recovers the selected policy.

Definition and stated intent:

Source lines 32–32:

```
32 def profile(second, details=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `details`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 33–34:

```
33 sj = expit(beta-GAMMAS+alpha*second)
34 s0 = expit(beta+GAMMAS+alpha*second)
```

Read this block:

- Line 33, `sj`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 34, `s0`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 36–39:

```
36 def objective(first):
37     shares = expit(intercepts+alpha*np.asarray(first)[..., None])
38     return ((np.asarray(first)-cost)*(shares @ weights)
39             + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))
```

Read this block:

- Line 36: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 41–41:

```
41 first, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale), points)
```

Read this block:

- Line 41, `(first, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 42–42:

```
42 return (first, value) if details else value
```

Read this block:

- Line 42: return the true-branch value if `details` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

committed policy.profile.objective

Original function: `committed_policy.profile.objective`. Source begins at line 36.

Weights each persistent type's first purchase and second-period branch revenues under a single committed policy. Broadcasting permits evaluating a price grid without changing which consumers keep their type.

Definition and stated intent:

Source lines 36–36:

```
36 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–37:

```
37 shares = expit(intercepts+alpha*np.asarray(first)[..., None])
```

Read this block:

- Line 37, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 38–39:

```
return ((np.asarray(first)-cost)*(shares @ weights)
        + delta*(second-cost)*((shares*sj+(1-shares)*s0) @ weights))
```

Read this block:

- Line 38: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

draw figures

Original function: `draw_figures`. Source begins at line 64.

Draws prices, expected profits and the value of observing first-period history as type probabilities vary. Grouping selects each focal gamma and initial state. These are theoretical curves, not newly estimated empirical revenues.

Definition and stated intent:

Source lines 64–65:

```
def draw_figures(results, output):
    """Use the original figure's horizontal three-panel line-plot layout."""
```

`def` creates this callable; the parentheses name its inputs (`results`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 66–66:

```
plt = plotting()
```

Read this block:

- Line 66, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–96:

```
for (focal_gamma, h0), data in results.groupby(["focal_gamma", "h0"], sort=False):
    state = "inside" if h0 == "j" else "outside"
    tag = {-1.25: "negative", 0.: "zero", 1.25: "positive"}[focal_gamma]
    stem = ("supply_information" if focal_gamma == 1.25 and h0 == "0"
            else f"supply_information_{tag}_h0_{state}")
    x = data.focal_probability
    with plt.rc_context({"font.size": 11, "axes.titlesize": 13}):
        fig, axes = plt.subplots(1, 3, figsize=(17, 4.6), layout="constrained")
        for key, label in [("p1", "Unknown gamma: adaptive p1"),
                           ("committed_p1", "Unknown gamma: committed p1"),
                           ("committed_p2", "Unknown gamma: committed p2")]:
            axes[0].plot(x, data[key], label=label, linewidth=2)
        axes[0].set(title="Optimal prices", ylabel="Price")
        for key, label in [("known_gamma_profit", "Gamma revealed before pricing"),
                           ("optimal_profit", "Gamma unknown: observe h1"),
                           ("committed_profit", "Gamma unknown: commit to both prices")]:
            axes[1].plot(x, data[key], label=label, linewidth=2)
```

```
84        axes[1].set(title="Expected two-period profit", ylabel="Discounted expected profit")
85        axes[2].plot(x, data.value_observing_h1, linewidth=2,
86                     label="Adaptive profit - committed profit")
87        axes[2].set(title="Value of observing h1", ylabel="Expected profit gain")
88        axes[2].set_ylim(bottom=0)
89        for ax in axes:
90            ax.set(xlabel=f"Pr(gamma = {focal_gamma:g})", xlim=(0, 1))
91            ax.legend(fontsize=8, frameon=True)
92        fig.suptitle(f"Known h0: {state} | Gamma unobserved in {{-1.25, 0, +1.25}} | Remaining probability split
    ↪ equally",
93                     fontsize=13)
94        for extension in ["png", "pdf"]:
95            fig.savefig(output / f"{stem}.{extension}", dpi=180)
96        plt.close(fig)
```

Read this block:

- Line 67: for each entry yielded by `results.groupby(['focal_gamma', 'h0'], sort=False)`, bind/unpack it into `((focal_gamma, h0), data)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 68, `state`: Set this name to the true-branch value if `h0 == 'j'` holds, otherwise the else-branch value.
- Line 69, `tag`: Set this name to the entry/slice selected by `{-1.25: 'negative', 0.0: 'zero', 1.25: 'positive'}[focal_...]`; brackets select from the object to their left.
- Line 70, `stem`: Set this name to the true-branch value if `focal_gamma == 1.25 and h0 == '0'` holds, otherwise the else-branch value.
- Line 72, `x`: Set this name to the attribute `data.focal_probability` of the object before the dot.
- Line 73: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 74, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 75: for each entry yielded by `[('p1', 'Unknown gamma: adaptive p1'), ('committed_p1', 'Unkno...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 78: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: for each entry yielded by `[('known_gamma_profit', 'Gamma revealed before pricing'), ('op...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 83: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `axes[2].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `axes[2].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `axes[2].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: for each entry yielded by `axes`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 90: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.
- Line 94: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 95: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run information

Original function: `run_information`. Source begins at line 99.

Loops over known initial states and probability grids, compares known gamma, adaptive hidden gamma and committed prices, checks value bounds, then saves theoretical figures and provenance. Its adaptive hidden-type model learns via outcomes; it is not the auxiliary fixed-belief specification.

Definition and stated intent:

Source lines 99–99:

```
99 def run_information(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 100–101:

```
100 if settings.probability_points < 3:
101     raise ValueError("Need at least three probability points.")
```

Read this block:

- Line 100: test `settings.probability_points < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 101: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 102–103:

```
grid = np.unique(np.r_[np.linspace(0, 1, settings.probability_points), 1/3, .5])
rows = []
```

Read this block:

- Line 102, `grid`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.
- Line 103, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 104–131:

```
for h0 in ["0", "j"]:
    # A benchmark only: reveal gamma before pricing, while retaining known h0.
    known_values = np.array([supply_solution(g, float(h0 == "j"), settings.beta,
                             settings.alpha, settings.delta, settings.cost)["known_value"] for g in GAMMAS])
    for focal_index, gamma in enumerate(GAMMAS):
        for probability in grid:
            weights = np.full(3, (1-probability)/2)
            weights[focal_index] = probability
            _, policy, _ = solve(h0, weights, settings.beta, settings.alpha, settings.delta, settings.cost)
            first, second, committed_value = committed_policy(h0, weights, settings.beta,
                                             settings.alpha, settings.delta, settings.cost)
            adaptive_value = policy[0]["optimal_profit"]
            known_value = float(weights @ known_values)
            if not known_value+1e-7 >= adaptive_value >= committed_value-1e-7:
                raise AssertionError("Known gamma >= adaptive unknown gamma >= committed unknown gamma failed.")
            if any(np.isclose(probability, p, atol=1e-12) for p in [0., 1/3, .5, 1.]):
                _, dense_policy, _ = solve(h0, weights, settings.beta, settings.alpha,
                                           settings.delta, settings.cost, points=161)
                _, _, dense_committed = committed_policy(h0, weights, settings.beta, settings.alpha,
                                                         settings.delta, settings.cost, points=161)
                assert np.isclose(dense_policy[0]["optimal_profit"], adaptive_value, atol=1e-7, rtol=0)
                assert np.isclose(dense_committed, committed_value, atol=1e-7, rtol=0)
            row = dict(policy[0], focal_gamma=gamma, focal_probability=probability,
                       committed_p1=first, committed_p2=second, committed_profit=committed_value,
                       known_gamma_profit=known_value,
                       value_observing_h1=max(0., adaptive_value-committed_value))
            rows.append(row)
        print(f"Information curves: known h0={h0}, varying Pr(gamma={gamma:g})", flush=True)
```

Read this block:

- Line 104: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 106, `known_values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 108: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(focal_index, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 109: for each entry yielded by `grid`, bind/unpack it into `probability` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 110, `weights`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 111, `weights[focal_index]`: Store the object currently named `probability`; plain assignment does not copy it in the selected entry/column.
- Line 112, `(_, policy, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 113, `(first, second, committed_value)`: Unpack the result of calling `committed_policy(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 115, `adaptive_value`: Set this name to the entry/slice selected by `policy[0]['optimal_profit']`; brackets select from the object to their left.
- Line 116, `known_value`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 117: test `not known_value + 1e-07 >= adaptive_value >= committed_value - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 118: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 119: test `any((np.isclose(probability, p, atol=1e-12) for p in [0.0, 1 / 3, 0.5, 1.0]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 120, `(_, dense_policy, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `(_, _, dense_committed)`: Unpack the result of calling `committed_policy(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 124: require `np.isclose(dense_policy[0]['optimal_profit'], adaptive_value, atol=1e-07, rtol=0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 125: require `np.isclose(dense_committed, committed_value, atol=1e-07, rtol=0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 126, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 130: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `committed_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 132–141:

```
settings.output.mkdir(parents=True, exist_ok=True)
results = pd.DataFrame(rows)
results.to_csv(settings.output / "supply_information_results.csv", index=False)
draw_figures(results, settings.output)
manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k, v in vars(settings).items()},
                information="h0 known in every case; gamma uncertain except in revealed-type benchmark",
                varying="Probability of one gamma; other two types split remaining probability equally",
                script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
               solver_sha256=hashlib.sha256(Path(__file__).with_name("unknown_type.py").read_bytes()).hexdigest())
(settings.output / "supply_information_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 132: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133, `results`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 134: call `results.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `draw_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 141: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `draw_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 142–142:

```
print(f"Saved six additional supply-information PNG/PDF figures to {settings.output}")
```

Read this block:

- Line 142: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 145.

Parses theoretical output/grid and beta/alpha/delta/cost overrides. These ordinary numerical parameters do not select empirical flavor groups.

Definition and stated intent:

Source lines 145–145:

```
145 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 146–148:

```
146 parser = argparse.ArgumentParser(description=__doc__)
147 parser.add_argument("--output", type=Path)
148 parser.add_argument("--probability-points", type=int)
```

Read this block:

- Line 146, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 147: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 149–150:

```
149 for name in ["beta", "alpha", "delta", "cost"]:
150     parser.add_argument(f"--{name}", type=float)
```

Read this block:

- Line 149: for each entry yielded by `['beta', 'alpha', 'delta', 'cost']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 150: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 151–151:

```
151 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 151: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 152–152:

```
152 return parser.parse_args()
```

Read this block:

- Line 152: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

../final_simple/supply/supply_simulation.py

Readable theory simulation and comparative-statics entry point using supply_sympy expressions. It uses theoretical parameters and writes theory tables/figures, independently of the server data pipeline.

Module setup and direct-execution blocks

Source lines 1–10:

```
"""Theoretical supply. Edit Parameters, then run python3 supply/supply_simulation.py.

One product versus outside; fixed known type. Solve p2(h1) first, then p1(h0).
Includes known state, hidden initial state, and ex ante price commitment.
"""
from pathlib import Path
from types import SimpleNamespace
import argparse
import json
import os
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `json`. An as-name is a convenient local alias, not another library.
- Line 10: import `os`. An as-name is a convenient local alias, not another library.

Source lines 11–19:

```
import sys
import tempfile
from pprint import pformat

import numpy as np
import pandas as pd

# Use a writable plotting cache, including on a shared research server.
os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "final_simple_matplotlib"))
```

Read this block:

- Line 11: import `sys`. An as-name is a convenient local alias, not another library.
- Line 12: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 13: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 16: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 19: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 21–26:

```
21 import math
22 import hashlib
23 import csv
24 from scipy.special import expit as scipy_expit
25 from scipy.optimize import brentq
26 import sympy as sp
```

Read this block:

- Line 21: import `math`. An as-name is a convenient local alias, not another library.
- Line 22: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 23: import `csv`. An as-name is a convenient local alias, not another library.
- Line 24: from `scipy.special`, import `expit as scipy_expit` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 25: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 26: import `sympy as sp`. An as-name is a convenient local alias, not another library.

Source lines 28–31:

```
28 if __package__:
29     from . import supply_sympy as theory
30 else:
31     import supply_sympy as theory
```

Read this block:

- Line 28: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 29: from ‘`, import`supply_sympy as theory‘ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 31: import `supply_sympy as theory`. An as-name is a convenient local alias, not another library.

Source lines 37–42:

```
37 output = Path(__file__).resolve().parents[1] / "results" / "supply"
38 beta = 2.0
39 alpha = -1.0                         # Signed price coefficient: must be negative.
40 delta = 0.92                         # Preserve the theory's existing calibration.
41 cost = 0.0
42 gamma_min = -3.0
```

Read this block:

- Line 37, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 38, `beta`: Set this name to the literal value `2.0`.
- Line 39, `alpha`: Set this name to the negated or sign-adjusted expression `-1.0`.
- Line 40, `delta`: Set this name to the literal value `0.92`.
- Line 41, `cost`: Set this name to the literal value `0.0`.
- Line 42, `gamma_min`: Set this name to the negated or sign-adjusted expression `-3.0`.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 43–49:

```
gamma_max = 3.0
gamma_points = 61
priors = [0.25, 0.5, 0.75]           # Probability the observed initial choice was inside.
# Additional figures from the symbolic derivatives (the original 601-point grid).
derivative_gamma_min = -3.0
derivative_gamma_max = 3.0
derivative_gamma_points = 601
```

Read this block:

- Line 43, `gamma_max`: Set this name to the literal value `3.0`.
- Line 44, `gamma_points`: Set this name to the literal value `61`.
- Line 45, `priors`: Set this name to the list `[0.25, 0.5, 0.75]`.
- Line 47, `derivative_gamma_min`: Set this name to the negated or sign-adjusted expression `-3.0`.
- Line 48, `derivative_gamma_max`: Set this name to the literal value `3.0`.
- Line 49, `derivative_gamma_points`: Set this name to the literal value `601`.

Source lines 50–55:

```
skip_derivatives = False             # True runs only the three supply cases.
from_solutions = False               # True reads your saved derivative solutions.
no_appendix = False                 # True skips the benchmark-only TeX appendix.
run_self_tests = False                    # True runs theoretical checks, not the full grid.
CASES = ["known_state", "hidden_initial_state", "ex_ante_commitment"]
CASE_LABELS = dict(zip(CASES, ["Known state (base)", "Hidden initial state", "Ex ante commitment"]))
```

Read this block:

- Line 50, `skip_derivatives`: Set this name to the literal value `False`.
- Line 51, `from_solutions`: Set this name to the literal value `False`.
- Line 52, `no_appendix`: Set this name to the literal value `False`.
- Line 53, `run_self_tests`: Set this name to the literal value `False`.
- Line 54, `CASES`: Set this name to the list `['known_state', 'hidden_initial_state', 'ex_ante_commitme...`.
- Line 55, `CASE_LABELS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 56–60:

```
STATE_LABELS = {"j": r"Initially inside ($x_0=j$)", "0": r"Initially outside ($x_0=0$)"}
STATES = [("0", "0"), ("0", "j"), ("j", "j"), ("j", "0")]
PARAMETERS = dict(beta=beta, alpha=alpha, delta=delta, c=cost)
GAMMAS = np.linspace(derivative_gamma_min, derivative_gamma_max, derivative_gamma_points)
OUT = output / "derivatives" / "figures"
```

Read this block:

- Line 56, `STATE_LABELS`: Set this name to a dictionary with named entries `j`, `0`.
- Line 57, `STATES`: Set this name to a list containing 4 displayed entries.
- Line 58, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 59, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 60, `OUT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 355–362:

```
_KEYS = ("dp1", "d2p1", "dprofit", "matched_j", "matched_0", "induce_harvest", "dp2j", "dp20")

_ARGS = (the-
    ory.alpha,theory.beta,theory.gamma,theory.delta,theory.c,theory.q,theory.sj1,theory.u,theory.v,theory.p1,theory.p2j,theory.p2

_EXPRESSIONS = (the-
    ory.dp1_dgamma,theory.d2p1_dgamma2,theory.dPi_dgamma,theory.matched_j,theory.matched_0,theory.induce_harvest,
                theory.price2_derivative(theory.gamma,theory.u,-1
    ,theory.p2j),theory.price2_derivative(theory.gamma,theory.v,1,theory.p20))

_EVALUATE = sp.lambdify(_ARGS,_EXPRESSIONS,"numpy")
```

Read this block:

- Line 355, `_KEYS`: Set this name to a tuple containing 8 displayed entries.
- Line 357, `_ARGS`: Set this name to a tuple containing 14 displayed entries.
- Line 359, `_EXPRESSIONS`: Set this name to a tuple containing 8 displayed entries.
- Line 362, `_EVALUATE`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.

Source lines 676–699:

```
if __name__ == "__main__":
    # Collect the ordinary Python variables above into one settings record.
    # settings.beta means exactly the variable beta from the Parameters section.
    settings = SimpleNamespace(
        output=output,
        gamma_min=gamma_min,
        gamma_max=gamma_max,
        gamma_points=gamma_points,
        priors=priors,
        beta=beta,
        alpha=alpha,
        delta=delta,
        cost=cost,
        derivative_gamma_min=derivative_gamma_min,
        derivative_gamma_max=derivative_gamma_max,
        derivative_gamma_points=derivative_gamma_points,
        skip_derivatives=skip_derivatives,
        from_solutions=from_solutions,
        no_appendix=no_appendix,
        self_test=run_self_tests,
    )
    if len(sys.argv) > 1:
        settings = read_command_line(settings)
    run_supply(settings)
```

Read this block:

- Line 676: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 679, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 697: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 698, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 699: call `run_supply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_supply`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 70.

Stable logistic transformation for theoretical choice probabilities. It prevents exponential overflow at large positive or negative utility differences.

Definition and stated intent:

Source lines 70–70:

```
70 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 71–72:

```
71 x = np.asarray(x)
72 small = np.exp(-np.abs(x))
```

Read this block:

- Line 71, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 72, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 73–73:

```
73 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 73: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

golden maximum

Original function: `golden_maximum`. Source begins at line 75.

Refines a bracketed scalar maximizer by golden-section interval reduction, reusing previous function values. This is a numerical search, not a proof that a multimodal objective has only one peak.

Definition and stated intent:

Source lines 75–75:

```
75 def golden_maximum(function, lo, hi):
```

`def` creates this callable; the parentheses name its inputs (`function`, `lo`, `hi`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 76–78:

```
76 ratio = (math.sqrt(5)-1)/2
77 left, right = hi-ratio*(hi-lo), lo+ratio*(hi-lo)
78 fl, fr = function(left), function(right)
```

Read this block:

- Line 76, `ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 77, `(left, right)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 78, `(fl, fr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 79–89:

```
79 for _ in range(100):
80     if hi-lo <= 1e-10*(1+abs(left)+abs(right)):
81         break
82     if fl > fr:
83         hi, right, fr = right, left, fl
84         left = hi-ratio*(hi-lo)
85         fl = function(left)
86     else:
87         lo, left, fl = left, right, fr
88         right = lo+ratio*(hi-lo)
89         fr = function(right)
```

Read this block:

- Line 79: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 80: test `hi - lo <= 1e-10 * (1 + abs(left) + abs(right))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 81: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 82: test `fl > fr`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 83, `(hi, right, fr)`: Unpack the tuple `(right, left, fl)` into these targets in order.
- Line 84, `left`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `fl`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.
- Line 87, `(lo, left, fl)`: Unpack the tuple `(left, right, fr)` into these targets in order.

- Line 88, `right`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 89, `fr`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 90–90:

```
90 return (left, fl) if fl > fr else (right, fr)
```

Read this block:

- Line 90: return the true-branch value if `fl > fr` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

scalar maximum

Original function: `scalar_maximum`. Source begins at line 92.

Evaluates a scalar objective on a grid, refines every detected local peak and includes endpoints, then keeps the best candidate. Hidden-state mixtures can be multimodal, so assuming a single global peak would be unsafe.

Definition and stated intent:

Source lines 92–93:

```
92 def scalar_maximum(function, bounds, points=161):
93     """Search every grid-local maximum, including boundaries (mixtures may multimode)."""
```

`def` creates this callable; the parentheses name its inputs (`function`, `bounds`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–94:

```
94 grid = np.linspace(*bounds, points)
```

Read this block:

- Line 94, `grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 96–101:

```
96 try:
97     values = np.asarray(function(grid))
98     if values.shape != grid.shape:
99         raise ValueError("Scalar objective")
100 except (ValueError, TypeError):
101     values = np.array([function(x) for x in grid])
```

Read this block:

- Line 96: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 97, `values`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 98: test `values.shape != grid.shape`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 99: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 100: catch `(ValueError, TypeError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 101, `values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 102–102:

```
102 candidates = [(values[0], grid[0]), (values[-1], grid[-1])]
```

Read this block:

- Line 102, `candidates`: Set this name to a list containing 2 displayed entries.

Source lines 103–106:

```
103 for i in range(1, points - 1):
104     if values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i - 1], values[i + 1]):
105         price, value = golden_maximum(function, grid[i - 1], grid[i + 1])
106         candidates.append((value, price))
```

Read this block:

- Line 103: for each entry yielded by `range(1, points - 1)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104: test `values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i -...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 105, `(price, value)`: Unpack the result of calling `golden_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 106: call `candidates.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `golden_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

Source lines 107–107:

```
107 value, price = max(candidates)
```

Read this block:

- Line 107, `(value, price)`: Unpack the result of calling `max(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 108–108:

```
108 return float(price), float(value)
```

Read this block:

- Line 108: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

static price

Original function: `static_price`. Source begins at line 112.

Solves the transformed binary-logit price first-order condition by bracketing and bisection for signed alpha<0. The effective cost can incorporate continuation value, allowing introductory subsidies; do not impose the coupon floor from empirical pricing on this theoretical solver.

Definition and stated intent:

Source lines 112–113:

```
112 def static_price(intercept, alpha, effective_cost=0.):
113     """Signed alpha<0; monotone transformed FOC, including introductory subsidies."""
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 114–115:

```
114 def foc(p):
115     return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 114: define nested callable `foc`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 116–116:

```
116 lo, hi = effective_cost, effective_cost+max(2., 2/abs(alpha))
```

Read this block:

- Line 116, `(lo, hi)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 117–118:

```
117 while foc(hi) > 0:
118     hi = lo+2*(hi-lo)
```

Read this block:

- Line 117: repeat while `foc(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.

- Line 118, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `foc`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 119–124:

```
119 for _ in range(100):
120     mid = (lo+hi)/2
121     if foc(mid) > 0:
122         lo = mid
123     else:
124         hi = mid
```

Read this block:

- Line 119: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 120, `mid`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 121: test `foc(mid) > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 122, `lo`: Set this name to the object currently named `mid`; plain assignment does not copy it.
- Line 124, `hi`: Set this name to the object currently named `mid`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 125–125:

```
125 return (lo+hi)/2
```

Read this block:

- Line 125: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

static price.foc

Original function: `static_price.foc`. Source begins at line 114.

This nested function returns the transformed price first-order-condition residual for a trial scalar. Captured slope, intercept and effective cost remain fixed during the root search. A zero is a candidate optimal price under the maintained monotonicity argument.

Definition and stated intent:

Source lines 114–114:

```
114 def foc(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 115–115:

```
115 return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 115: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.

supply solution

Original function: `supply_solution`. Source begins at line 135.

Solves the known-state baseline, hidden-initial-state case, and commitment case for one fixed type and prior. Both terminal histories enter expected profits. The hidden-state case pools initial histories within a known type; it does not pool different gamma types. The commitment case chooses two numbers before the first purchase and cannot adapt p2 to h1.

Definition and stated intent:

Source lines 135–136:

```
135 def supply_solution(gamma, prior=.5, beta=2., alpha=-1., delta=.92, cost=0.):
136     """Known type; hidden h0, then either observe h1 or commit to p1,p2."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `prior`, `beta`, `alpha`, `delta`, `cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 137–139:

```
137 if not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all()
138         and alpha < 0 and 0 <= prior <= 1 and 0 <= delta <= 1):
139     raise ValueError("Need finite inputs, alpha<0, prior/delta in [0,1].")
```

Read this block:

- Line 137: test `not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all() and alpha < ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 139: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 141–146:

```
141 intercepts = beta + np.array([-gamma, gamma])
142 weights = np.array([prior, 1 - prior])
143 p2 = np.array([static_price(x, alpha, cost) for x in intercepts])
144 s2 = expit(intercepts + alpha * p2)
145 W2 = (p2 - cost) * s2
146 adjustment = delta * (W2[0] - W2[1])
```

Read this block:

- Line 141, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 142, `weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

- Line 143, `p2`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 144, `s2`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 145, `W2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 146, `adjustment`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `static_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 148–149:

```
148 def share1(p):
149     return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 148: define nested callable `share1`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 151–153:

```
151 def adaptive_value(p):
152     s = share1(p)
153     return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 151: define nested callable `adaptive_value`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 155–159:

```
155 scale = (20 + abs(beta) + abs(gamma) + abs(alpha * cost)) / abs(alpha)
156 bounds = (cost - scale, cost + scale)
157 pooled_p1, adaptive = scalar_maximum(adaptive_value, bounds)
158 # Rechecking a wider domain catches a binding numerical search boundary.
159 wide_p1, wide_value = scalar_maximum(adaptive_value, (cost - 2*scale, cost + 2*scale), 321)
```

Read this block:

- Line 155, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 156, `bounds`: Set this name to a tuple containing 2 displayed entries.
- Line 157, `(pooled_p1, adaptive)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 159, `(wide_p1, wide_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 160–161:

```
160 if wide_value > adaptive + 1e-8:
161     pooled_p1, adaptive = wide_p1, wide_value
```

Read this block:

- Line 160: test `wide_value > adaptive + 1e-08`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 161, `(pooled_p1, adaptive)`: Unpack the tuple `(wide_p1, wide_value)` into these targets in order.

Source lines 162–163:

```
162 known_p1 = np.array([static_price(x, alpha, cost - adjustment) for x in intercepts])
163 known_values = []
```

Read this block:

- Line 162, `known_p1`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 163, `known_values`: Set this name to the list `[]`.

Source lines 164–166:

```
164 for x, price in zip(intercepts, known_p1):
165     s = expit(x + alpha * price)
166     known_values.append(s*(price-cost) + delta*(s*W2[0]+(1-s)*W2[1]))
```

Read this block:

- Line 164: for each entry yielded by `zip(intercepts, known_p1)`, bind/unpack it into `(x, price)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 165, `s`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 166: call `known_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 168–175:

```
168 def committed_profile(second, return_price=False):
169     terminal = expit(intercepts + alpha * second)
170     Wj, W0 = terminal*(second-cost)
171     def objective(first):
172         s = share1(first)
173         return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
174     price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
175     return (price, value) if return_price else value
```

Read this block:

- Line 168: define nested callable `committed_profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 179–190:

```
179 if delta == 0:
180     cp1, cp2, committed = pooled_p1, static_price(beta, alpha, cost), adaptive
181 else:
182     cp2, committed = scalar_maximum(committed_profile, (cost, cost+scale))
183     cp1, committed = committed_profile(cp2, True)
184     # Independent wider and denser profile scan.
```

```
    check_p2, check_value = scalar_maximum(committed_profile, (cost, cost+2*scale), 321)
    if check_value > committed+1e-9:
        cp2 = check_p2
        cp1, committed = committed_profile(cp2, True)
    if min(cp2-cost, cost+2*scale-cp2) < scale*1e-5:
        raise RuntimeError("Committed optimum is near a search bound; inspect calibration.")
```

Read this block:

- Line 179: test `delta == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 180, `(cp1, cp2, committed)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 182, `(cp2, committed)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 183, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 185, `(check_p2, check_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 186: test `check_value > committed + 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 187, `cp2`: Set this name to the object currently named `check_p2`; plain assignment does not copy it.
- Line 188, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 189: test `min(cp2 - cost, cost + 2 * scale - cp2) < scale * 1e-05`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 190: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `committed_profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 191–191:

```
known = float(weights @ known_values)
```

Read this block:

- Line 191, `known`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 192–193:

```
if not known + 1e-7 >= adaptive >= committed - 1e-7:
    raise AssertionError("Information-value ordering failed.")
```

Read this block:

- Line 192: test `not known + 1e-07 >= adaptive >= committed - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 193: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 194–200:

```
s = share1(pooled_p1)
derivative = float(weights @ (alpha*expit(intercepts+alpha*pooled_p1)
                              * expit(-intercepts-alpha*pooled_p1)))
committed_first_share = share1(cp1)
committed_first_derivative = float(weights @ (alpha*expit(intercepts+alpha*cp1)*expit(-intercepts-alpha*cp1)))
terminal = expit(intercepts+alpha*cp2)
terminal_weights = np.array([committed_first_share, 1-committed_first_share])
```

Read this block:

- Line 194, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 195, `derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 197, `committed_first_share`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 198, `committed_first_derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 199, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 200, `terminal_weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 201–202:

```
committed_foc1 =
    committed_first_share+committed_first_derivative*(cp1-cost+delta*(cp2-cost)*(terminal[0]-terminal[1]))
committed_foc2 = float(terminal_weights @ (terminal+alpha*(cp2-cost)*terminal*(1-terminal))) if delta > 0 else 0.
```

Read this block:

- Line 201, `committed_foc1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 202, `committed_foc2`: Set this name to the true-branch value if `delta > 0` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 203–211:

```
return dict(gamma=gamma, prior_inside=prior, beta=beta, alpha=alpha, delta=delta, cost=cost,
            known_p1_inside=known_p1[0], known_p1_outside=known_p1[1],
            known_value=known, hidden_p1=pooled_p1, adaptive_p2_inside=p2[0],
            adaptive_p2_outside=p2[1], hidden_value=adaptive,
            committed_p1=cp1, committed_p2=cp2, committed_value=committed,
            initial_information_value=known-adaptive,
            second_period_flexibility_value=adaptive-committed,
            committed_foc1=committed_foc1, committed_foc2=committed_foc2,
            hidden_foc=s+derivative*(pooled_p1-cost+adjustment))
```

Read this block:

- Line 203: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

supply solution.share1

Original function: `supply_solution.share1`. Source begins at line 148.

Returns first-period binary-logit demand, or its initial-state mixture, at a trial price. The function closes over the current theoretical type and prior. Initial history is integrated as probabilities, not averaged as utilities.

Definition and stated intent:

Source lines 148–148:

```
148 def share1(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 149–149:

```
149 return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 149: return matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

supply solution.adaptive value

Original function: `supply_solution.adaptive_value`. Source begins at line 151.

Evaluates current profit plus discounted, state-contingent optimized terminal values. The first choice changes the probabilities of both continuation histories, which is why its derivative contains their value difference.

Definition and stated intent:

Source lines 151–151:

```
151 def adaptive_value(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 152–152:

```
152 s = share1(p)
```

Read this block:

- Line 152, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 153–153:

```
153 return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 153: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

supply solution.committed profile

Original function: `supply_solution.committed_profile`. Source begins at line 168.

For a trial first price, solves the best single committed terminal price and returns its value/profile. This inner optimization must respect the information restriction: p2 is common across realized h1.

Definition and stated intent:

Source lines 168–168:

```
168 def committed_profile(second, return_price=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `return_price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 169–170:

```
169 terminal = expit(intercepts + alpha * second)
170 Wj, W0 = terminal*(second-cost)
```

Read this block:

- Line 169, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 170, `(Wj, W0)`: Unpack multiplication of the displayed operands; parentheses control which subexpressions are evaluated together into these targets in order.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 171–173:

```
171 def objective(first):
172     s = share1(first)
173     return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 171: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 174–174:

```
174 price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
```

Read this block:

- Line 174, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 175–175:

```
175 return (price, value) if return_price else value
```

Read this block:

- Line 175: return the true-branch value if `return_price` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

supply solution.committed profile.objective

Original function: `supply_solution.committed_profile.objective`. Source begins at line 171.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 171–171:

```
171 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 172–172:

```
172 s = share1(first)
```

Read this block:

- Line 172, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 173–173:

```
173 return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 173: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

outcome tables

Original function: `outcome_tables`. Source begins at line 221.

Exports history-specific prices, quantities and profits plus properly probability-weighted ex ante outcomes. Conditional realized-history values and expected profits answer different questions; averaging conditional profits without their path probabilities would be wrong.

Definition and stated intent:

Source lines 221–222:

```
def outcome_tables(result):
    """All four realized histories per case, plus correctly weighted ex ante objects."""
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 223–224:

```
g, beta, alpha, delta, cost, pi = [result[k] for k in ["gamma", "beta", "alpha", "delta", "cost", "prior_inside"]]
paths, summary = [], []
```

Read this block:

- Line 223, `(g, beta, alpha, delta, cost, pi)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 224, `(paths, summary)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 225–263:

```
for case in CASES:
    case_paths = []
    for h0, weight in [("j", pi), ("0", 1-pi)]:
        if case == "known_state":
            p1 = result["known_p1_inside" if h0 == "j" else "known_p1_outside"]
        else:
            p1 = result["hidden_p1" if case == "hidden_initial_state" else "committed_p1"]
        s1 = float(expit(beta+(-1 if h0 == "j" else 1)*g+alpha*p1))
        conditional_value = 0.
        local = []
        for h1, transition in [("j", s1), ("0", 1-s1)]:
            p2 = result["committed_p2"] if case == "ex_ante_commitment" else result["adaptive_p2_inside" if h1 ==
↪ "j" else "adaptive_p2_outside"]
            s2 = float(expit(beta+(-1 if h1 == "j" else 1)*g+alpha*p2))
            # Realize the first choice; integrate the terminal taste shock.
            conditional_profit = (p1-cost)*(h1 == "j")+delta*(p2-cost)*s2
            conditional_value += transition*conditional_profit
            row = dict(gamma=g, prior_inside=pi, case=case, h0=h0, h1=h1,
                       initial_probability=weight, transition_probability=transition,
                       path_probability=weight*transition, p1=p1, p2=p2,
                       price_ratio=p1/p2 if abs(p2) > 1e-12 else np.nan,
                       first_purchase_probability=s1, second_purchase_probability=s2,
                       conditional_profit=conditional_profit)
            local.append(row)
        for row in local:
            row["optimal_profit_given_h0"] = conditional_value
        case_paths.extend(local)
    probability = np.array([r["path_probability"] for r in case_paths])
    if not np.isclose(probability.sum(), 1.):
        raise AssertionError("Path probabilities do not sum to one.")
    mean_p1, mean_p2, profit = [float(probability @ np.array([r[k] for r in case_paths]))
                                for k in ["p1", "p2", "conditional_profit"]]
    expected_ratio = float(probability @ np.array([r["price_ratio"] for r in case_paths]))
    expected_key = {"known_state": "known_value", "hidden_initial_state": "hidden_value", "ex_ante_commitment":
↪ "committed_value"}[case]
    if not np.isclose(profit, result[expected_key], atol=1e-8):
        raise AssertionError("Path profits do not reproduce optimized objective.")
    summary.append(dict(gamma=g, prior_inside=pi, case=case, expected_p1=mean_p1,
```

```
261                         expected_p2=mean_p2, ratio_of_expected_prices=mean_p1/mean_p2 if abs(mean_p2) > 1e-12 else
    ↪ np.nan,
262                         expected_price_ratio=expected_ratio, optimal_profit=profit))
263     paths.extend(case_paths)
```

Read this block:

- Line 225: for each entry yielded by `CASES`, bind/unpack it into `case` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 226, `case_paths`: Set this name to the list `[]`.
- Line 227: for each entry yielded by `[('j', pi), ('0', 1 - pi)]`, bind/unpack it into `(h0, weight)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 228: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 229, `p1`: Set this name to the entry/slice selected by `result['known_p1_inside' if h0 == 'j' else 'known_p1_outs...`; brackets select from the object to their left.
- Line 231, `p1`: Set this name to the entry/slice selected by `result['hidden_p1' if case == 'hidden_initial_state' else...`; brackets select from the object to their left.
- Line 232, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 233, `conditional_value`: Set this name to the literal value `0.0`.
- Line 234, `local`: Set this name to the list `[]`.
- Line 235: for each entry yielded by `[('j', s1), ('0', 1 - s1)]`, bind/unpack it into `(h1, transition)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 236, `p2`: Set this name to the true-branch value if `case == 'ex_ante_commitment'` holds, otherwise the else-branch value.
- Line 237, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 239, `conditional_profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 240: update `conditional_value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 241, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 247: call `local.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 248: for each entry yielded by `local`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 249, `row['optimal_profit_given_h0']`: Store the object currently named `conditional_value`; plain assignment does not copy it in the selected entry/column.
- Line 250: call `case_paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 251, `probability`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 252: test `not np.isclose(probability.sum(), 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 253: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 254, `(mean_p1, mean_p2, profit)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 256, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 257, `expected_key`: Set this name to the entry/slice selected by `{'known_state': 'known_value', 'hidden_initial_state': 'h...`; brackets select from the object to their left.
- Line 258: test `not np.isclose(profit, result[expected_key], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 259: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 260: call `summary.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 263: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 264–264:

```
264 return paths, summary
```

Read this block:

- Line 264: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

plotting

Original function: `plotting`. Source begins at line 272.

Returns the noninteractive plotting interface with an appropriate cache. It is shared by theoretical result and derivative figures; no empirical input is needed.

Definition and stated intent:

Source lines 272–272:

```
272 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 273–276:

```
273 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
274 import matplotlib
275 matplotlib.use("Agg")
276 import matplotlib.pyplot as plt
```

Read this block:

- Line 273: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 274: import `matplotlib`. An as-name is a convenient local alias, not another library.

- Line 275: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 276: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 277–277:

```
277 return plt
```

Read this block:

- Line 277: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

make case figures

Original function: `make_case_figures`. Source begins at line 279.

Plots policy and outcome comparisons for the theoretical initial-history information cases. The values come from solved parameter grids, not household price records.

Definition and stated intent:

Source lines 279–279:

```
279 def make_case_figures(paths, summary, output):
```

`def` creates this callable; the parentheses name its inputs (`paths`, `summary`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 280–280:

```
280 plt = plotting()
```

Read this block:

- Line 280, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 281–322:

```
281 for prior in summary.prior_inside.unique():
282     stem = f"prior_{prior:g}".replace(".", "p")
283     subset = paths[paths.prior_inside == prior]
284     fig, axes = plt.subplots(3, 3, figsize=(16, 12))
285     for column, case in enumerate(CASES):
286         data = subset[subset.case == case]
287         if case == "known_state":
288             for h0 in ["0", "j"]:
289                 initial = data[(data.h0 == h0) & (data.h1 == "j")]
290                 axes[0, column].plot(initial.gamma, initial.p1, label=f"p1(h0={h0})")
291         else:
292             initial = data[(data.h0 == "0") & (data.h1 == "j")]
293             axes[0, column].plot(initial.gamma, initial.p1, label="p1")
294         if case == "ex_ante_commitment":
295             axes[0, column].plot(initial.gamma, initial.p2, label="p2")
296         else:
```

```
            for h1 in ["0", "j"]:
                terminal = data[(data.h0 == "0") & (data.h1 == h1)]
                axes[0, column].plot(terminal.gamma, terminal.p2, "--", label=f"p2(h1={h1})")
        for (h0, h1), history in data.groupby(["h0", "h1"]):
            # Duplicate ratios in restricted cases need only one curve.
            if case != "known_state" and h0 != "0":
                continue
            if case == "ex_ante_commitment" and h1 != "j":
                continue
            axes[1, column].plot(history.gamma, history.price_ratio, label=f"h0={h0}, h1={h1}")
        for h0 in ["0", "j"]:
            initial = data[(data.h0 == h0) & (data.h1 == "j")]
            axes[2, column].plot(initial.gamma, initial.optimal_profit_given_h0, label=f"E[profit|h0={h0}]")
        expected = summary[(summary.prior_inside == prior) & (summary.case == case)]
        axes[2, column].plot(expected.gamma, expected.optimal_profit, "k--", label="Ex ante E[profit]")
        axes[0, column].set_title(CASE_LABELS[case])
    for row, ylabel in enumerate(["Optimal prices", "Path-specific p1 / p2", "Optimal discounted profit"]):
        for ax in axes[row]:
            ax.set(xlabel="gamma", ylabel=ylabel)
            ax.axhline(0, color="gray", linewidth=.5)
            ax.legend(fontsize=7)
    fig.suptitle(f"Supply cases: Pr(h0=j)={prior:g}")
    fig.tight_layout()
    fig.savefig(output / f"supply_cases_{stem}.png", dpi=180)
    fig.savefig(output / f"supply_cases_{stem}.pdf")
    plt.close(fig)
```

Source lines 323–347:

```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for case, group in summary[summary.prior_inside == prior].groupby("case", sort=False):
    axes[0].plot(group.gamma, group.optimal_profit, label=CASE_LABELS[case])
    axes[1].plot(group.gamma, group.ratio_of_expected_prices, label=CASE_LABELS[case])
    axes[2].plot(group.gamma, group.expected_price_ratio, label=CASE_LABELS[case])
for ax, title in zip(axes, ["Optimal expected profit", "E[p1] / E[p2]", "E[p1 / p2]"]):
    ax.set(xlabel="gamma", title=title)
    ax.legend(fontsize=7)
fig.tight_layout()
fig.savefig(output / f"supply_comparison_{stem}.png", dpi=180)
fig.savefig(output / f"supply_comparison_{stem}.pdf")
plt.close(fig)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, case in zip(axes, CASES):
    data = subset[subset.case == case]
    for (h0, h1), history in data.groupby(["h0", "h1"]):
        ax.plot(history.gamma, history.conditional_profit, label=f"h0={h0}, h1={h1}")
    ax.set(xlabel="gamma", ylabel="Path-conditional discounted profit", title=CASE_LABELS[case])
    ax.legend(fontsize=7)
fig.tight_layout()
fig.savefig(output / f"supply_history_profit_{stem}.png", dpi=180)
fig.savefig(output / f"supply_history_profit_{stem}.pdf")
plt.close(fig)
```

Read this block:

- Line 281: for each entry yielded by `summary.prior_inside.unique()`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 282, `stem`: Set this name to the result of calling `f'prior_{prior:g}'.replace(...)`; the arguments and preceding method chain are shown above.

- Line 283, `subset`: Set this name to the entry/slice selected by `paths[paths.prior_inside == prior]`; brackets select from the object to their left.
- Line 284, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 285: for each entry yielded by `enumerate(CASES)`, bind/unpack it into `(column, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 286, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 287: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 288: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 289, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 290: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 292, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 293: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 294: test `case == 'ex_ante_commitment'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 297: for each entry yielded by `['0', 'j']`, bind/unpack it into `h1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 298, `terminal`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == h1)]`; brackets select from the object to their left.
- Line 299: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 300: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 302: test `case != 'known_state' and h0 != '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 303: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 304: test `case == 'ex_ante_commitment' and h1 != 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 305: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 306: call `axes[1, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 307: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 308, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 309: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 310, `expected`: Set this name to the entry/slice selected by `summary[(summary.prior_inside == prior) & (summary.case =...`; brackets select from the object to their left.
- Line 311: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 312: call `axes[0, column].set_title` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 313: for each entry yielded by `enumerate(['Optimal prices', 'Path-specific p1 / p2', 'Optimal...`, bind/unpack it into `(row, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 314: for each entry yielded by `axes[row]`, bind/unpack it into `ax` and run the indented body. Changes

inside an iteration can feed subsequent iterations.
- Line 315: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 316: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 317: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 318: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 319: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 320: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 321: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 322: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 324, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 325: for each entry yielded by `summary[summary.prior_inside == prior].groupby('case', sort=Fa...`, bind/unpack it into `(case, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 326: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 327: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 328: call `axes[2].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 329: for each entry yielded by `zip(axes, ['Optimal expected profit', 'E[p1] / E[p2]', 'E[p1 /...`, bind/unpack it into `(ax, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 330: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 331: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 332: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 333: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 334: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 335: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 337, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 338: for each entry yielded by `zip(axes, CASES)`, bind/unpack it into `(ax, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 339, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 340: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 341: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 342: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 343: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.
- Line 344: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 345: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 346: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 347: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

inside share

Original function: `inside_share`. Source begins at line 364.

Evaluates the binary inside purchase probability for a given price, gamma, signed initial state and common tastes. It is the numerical counterpart of the symbolic logit expression.

Definition and stated intent:

Source lines 364–364:

```
364 def inside_share(price,gamma,state,beta=2.,alpha=-1.):
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 365–365:

```
365 return float(scipy_expit(beta+theory.signed_state(state)*gamma+alpha*price))
```

Read this block:

- Line 365: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `scipy_expit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimal price

Original function: `optimal_price`. Source begins at line 367.

This finds one optimal price for a utility intercept, negative alpha, and an effective cost. `f = lambda p: ...` defines the transformed scalar FOC. `clip` limits the exponential's argument to a representable range; it is a numerical safeguard rather than a theoretical cap on utility.

The initial lower endpoint is effective cost, where f is positive. `max` sets a reasonable initial width, and `while f(hi) > 0` doubles it until the bracket changes sign. `brentq` can then locate the zero to `xtol=1e-12`. A fixed interval is simpler but can miss a root under another calibration. Unconstrained generic minimization ignores the monotonic scalar structure and can need more evaluations. The maintained alpha<0 assumption establishes a unique root; the caller validates it.

Definition and stated intent:

Source lines 367–368:

```
367 def optimal_price(intercept,alpha,effective_cost):
368     # Multiplying the FOC by alpha gives a strictly decreasing function.
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 369–372:

```
369 f = lambda p: alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p,-745,700))
370 lo = effective_cost
371 width = max(2.,2/abs(alpha))
372 hi = lo+width
```

Read this block:

- Line 369, `f`: Set this name to a one-expression callable, with input `p` and returned expression `alpha * (p - effective_cost) + 1 + np.exp(np.clip(in...`.
- Line 370, `lo`: Set this name to the object currently named `effective_cost`; plain assignment does not copy it.
- Line 371, `width`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 372, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 373–375:

```
373 while f(hi) > 0:
374     width *= 2
375     hi = lo+width
```

Read this block:

- Line 373: repeat while `f(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 374: update `width` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 375, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `f`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 376–376:

```
376 return float(brentq(f,lo,hi,xtol=1e-12))
```

Read this block:

- Line 376: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

evaluate

Original function: `evaluate`. Source begins at line 380.

Solves the known-state two-period pricing problem and evaluates named prices, profits and derivatives at one gamma/state. Parameter signs and both terminal branches must match the symbolic model.

Definition and stated intent:

Source lines 380–380:

```
380 def evaluate(gamma,initial_state,beta=2.,alpha=-1.,delta=.92,c=0.):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`, `beta`, `alpha`, `delta`, `c`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 381–381:

```
381 q = theory.signed_state(initial_state)
```

Read this block:

- Line 381, `q`: Set this name to the result of calling `theory.signed_state(...)`; the arguments and preceding method chain are shown above.

Source lines 382–383:

```
382 if not (np.isfinite([gamma,beta,alpha,delta,c]).all() and alpha<0 and 0<=delta<1):
383     raise ValueError("Require finite parameters, alpha<0, and 0<=delta<1 (delta=0 is the static counterfactual).")
```

Read this block:

- Line 382: test `not (np.isfinite([gamma, beta, alpha, delta, c]).all() and alpha < 0 and (0 <...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 384–389:

```
p2j,p20 = [optimal_price(beta+theory.signed_state(state)*gamma,alpha,c) for state in ["j","0"]]
u,v = inside_share(p2j,gamma,"j",beta,alpha),inside_share(p20,gamma,"0",beta,alpha)
Vj,V0 = (p2j-c)*u,(p20-c)*v
DV = Vj-V0
p1 = optimal_price(beta+q*gamma,alpha,c-delta*DV)
s = inside_share(p1,gamma,initial_state,beta,alpha)
```

Read this block:

- Line 384, `(p2j, p20)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 385, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 386, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 387, `DV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 388, `p1`: Set this name to the result of calling `optimal_price(...)`; the arguments and preceding method chain are shown above.
- Line 389, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `optimal_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 390–397:

```
profit = s*(p1-c)+delta*(s*Vj+(1-s)*V0)
values = _EVALUATE(alpha,beta,gamma,delta,c,q,s,u,v,p1,p2j,p20,Vj,V0)
result = dict(gamma=float(gamma),initial_state=initial_state,p1=p1,p2j=p2j,p20=p20,s=s,u=u,v=v,
              V2j=Vj,V20=V0,DeltaV=DV,profit=profit,
              static_component=c-1/(alpha*(1-s)),continuation_adjustment=-delta*DV,
              static_counterfactual=optimal_price(beta+q*gamma,alpha,c))
result.update(dict(zip(_KEYS,map(float,values))))
result['matched_j' if initial_state=='0' else 'matched_0'] = float('nan')
```

Read this block:

- Line 390, `profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 391, `values`: Set this name to the result of calling `_EVALUATE(...)`; the arguments and preceding method chain are shown above.
- Line 392, `result`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 396: call `result.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 397, `result['matched_j' if initial_state == '0' else 'matched_0']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `_EVALUATE`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `map`: Apply a callable to each iterable entry lazily, such as converting result scalars to Python float.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 398–399:

```
398 if initial_state=='j':
399     result['induce_harvest'] = float('nan')
```

Read this block:

- Line 398: test `initial_state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 399, `result['induce_harvest']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 400–404:

```
400 result['d_matched_ratio'] = result['matched_j'] if initial_state=='j' else result['matched_0']
401 result['d_induce_ratio'] = result['induce_harvest'] if initial_state=='0' else float('nan')
402 result['foc1'] = s+alpha*s*(1-s)*(p1-c+delta*DV)
403 result['foc2j'] = u+alpha*u*(1-u)*(p2j-c)
404 result['foc20'] = v+alpha*v*(1-v)*(p20-c)
```

Read this block:

- Line 400, `result['d_matched_ratio']`: Store the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value in the selected entry/column.
- Line 401, `result['d_induce_ratio']`: Store the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value in the selected entry/column.
- Line 402, `result['foc1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 403, `result['foc2j']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 404, `result['foc20']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 405–405:

```
405 return result
```

Read this block:

- Line 405: return the object currently named `result`; plain assignment does not copy it. This ends the call.

solve

Original function: `solve`. Source begins at line 407.

This is a compact convenience interface. The bare `*` makes `initial_state` keyword-only; `**parameters` collects other keywords. Membership tests reject obsolete z1/z2 inputs. `parameters.pop("gamma", 0.)` retrieves and removes gamma, allowing the rest to be expanded into `evaluate` without passing gamma twice. The function returns seven values as a tuple. It is convenient for unpacking, while `evaluate` is preferable when you need named derivative or diagnostic fields.

Definition and stated intent:

Source lines 407–407:

```
407 def solve(*, initial_state="0", **parameters):
```

`def` creates this callable; the parentheses name its inputs (`initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**parameters` collects additional keyword arguments in a dictionary.

Source lines 408–409:

```
408 if 'z1' in parameters or 'z2' in parameters:
409     raise ValueError("Old 0/1 histories are superseded: use initial_state='j' or '0'; both continuation branches
      ↪  are solved.")
```

Read this block:

- Line 408: test `'z1' in parameters or 'z2' in parameters`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 409: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 410–411:

```
410 gamma = parameters.pop('gamma',0.)
411 r = evaluate(gamma,initial_state,**parameters)
```

Read this block:

- Line 410, `gamma`: Set this name to the result of calling `parameters.pop(...)`; the arguments and preceding method chain are shown above.
- Line 411, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.pop`: Retrieve and remove a dictionary entry, using the supplied default if it is absent.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 412–412:

```
412 return r['p1'],r['p2j'],r['p20'],r['s'],r['u'],r['v'],r['profit']
```

Read this block:

- Line 412: return a tuple of 7 results. The caller can unpack them; return ends the call, unlike print.

state outcomes

Original function: `state_outcomes`. Source begins at line 418.

Conditions on each initial/first-choice history, retains terminal demand expectation, and produces histories with their path weights. This separates conditional economic mechanisms from ex ante comparisons.

Definition and stated intent:

Source lines 418–419:

```
418 def state_outcomes(curves):
419     """Condition on (x0, x1); terminal demand remains integrated out."""
```

`def` creates this callable; the parentheses name its inputs (`curves`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 420–420:

```
420 rows=[]
```

Read this block:

- Line 420, `rows`: Set this name to the list `[]`.

Source lines 421–435:

```
421 for initial, realized in STATES:
422     for r in curves[initial]:
423         p2=r['p2j' if realized=='j' else 'p20']
424         dp2=r['dp2j' if realized=='j' else 'dp20']
425         value=r['V2j' if realized=='j' else 'V20']
426         rows.append(dict(
427             gamma=r['gamma'], x0=initial, x1=realized,
428             probability=r['s'] if realized=='j' else 1-r['s'],
429             share1=r['s'], outside_share1=1-r['s'],
430             share2=r['u' if realized=='j' else 'v'],
431             outside_share2=1-r['u' if realized=='j' else 'v'],
432             price_ratio=r['p1']/p2,
433             price_ratio_derivative=(r['dp1']*p2-r['p1']*dp2)/p2**2,
434             realized_profit=(r['p1']-PARAMETERS['c'])*(realized=='j')
435                             +PARAMETERS['delta']*value))
```

Read this block:

- Line 421: for each entry yielded by `STATES`, bind/unpack it into `(initial, realized)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 422: for each entry yielded by `curves[initial]`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 423, `p2`: Set this name to the entry/slice selected by `r['p2j' if realized == 'j' else 'p20']`; brackets select from the object to their left.
- Line 424, `dp2`: Set this name to the entry/slice selected by `r['dp2j' if realized == 'j' else 'dp20']`; brackets select from the object to their left.
- Line 425, `value`: Set this name to the entry/slice selected by `r['V2j' if realized == 'j' else 'V20']`; brackets select from the object to their left.
- Line 426: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 436–436:

```
436 return rows
```

Read this block:

- Line 436: return the object currently named `rows`; plain assignment does not copy it. This ends the call.

plot state outcomes

Original function: `plot_state_outcomes`. Source begins at line 439.

Plots conditional prices, ratios or profits across gamma for the history-specific outcomes. Histories that are rare under a policy should not be confused with an equally weighted population average.

Definition and stated intent:

Source lines 439–439:

```
439 def plot_state_outcomes(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 440–445:

```
440 specs=[('all_state_price_ratios','price_ratio',r'$p_1^*(x_0)/p_2^*(x_1)$',
441         'Price ratios by realized history'),
442        ('all_state_price_ratio_derivatives','price_ratio_derivative',r'$dR_{x_0,x_1}/d\gamma$',
443         'Price-ratio derivatives by realized history'),
444        ('all_state_realized_profit','realized_profit',r'$\Pi^*(x_0,x_1)$',
445         'Profit conditional on realized first-period choice')]
```

Read this block:

- Line 440, `specs`: Set this name to a list containing 3 displayed entries.

Source lines 446–460:

```
446 for name,key,ylabel,title in specs:
447     fig,ax=plt.subplots(figsize=(8,5))
448     for (initial,realized),style in zip(STATES,['-','--','-.',':']):
449         group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
450         ax.plot([r['gamma'] for r in group],[r[key] for r in group],
451                 ls=style,lw=2,label=rf'$(x_0,x_1)=({initial},{realized})$')
452     ax.axhline(0,color='gray',lw=.7)
453     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
454     ax.legend()
455     ax.grid(alpha=.2)
456     if key=='realized_profit':
457         fig.text(.5,.01,'First-period margin if purchased + discounted expected terminal profit',
458                  ha='center',fontsize=9)
459         fig.tight_layout(rect=(0,.04,1,1))
460     save(fig,name)
```

Read this block:

- Line 446: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 447, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 448: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 449, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 450: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 452: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 453: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 454: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 455: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 456: test `key == 'realized_profit'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 457: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 459: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 460: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot state shares

Original function: `plot_state_shares`. Source begins at line 463.

Plots how policy-induced probabilities allocate households across realized histories. These probability weights explain why an ex ante profit comparison can differ from comparing conditional histories alone.

Definition and stated intent:

Source lines 463–463:

```
463 def plot_state_shares(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 464–464:

```
464 fig,axes=plt.subplots(2,2,figsize=(11,8),sharex=True,sharey=True)
```

Read this block:

- Line 464, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 465–475:

```
for ax,key,title in zip(axes.flat,
        ['share1','share2','outside_share1','outside_share2'],
        ['Period 1: inside share','Period 2: inside share',
         'Period 1: outside share','Period 2: outside share']):
    for (initial,realized),style in zip(STATES,['-','--','-.',':']):
        group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
        ax.plot([r['gamma'] for r in group],[r[key] for r in group],
                ls=style,lw=2,label=rf'$({initial},{realized})$')
    ax.set(xlabel=r'$\gamma$',ylabel='Share',title=title,ylim=(0,1))
    ax.grid(alpha=.2)
    ax.legend(title=r'$(x_0,x_1)$',fontsize=8,ncol=2)
```

Read this block:

- Line 465: for each entry yielded by `zip(axes.flat, ['share1', 'share2', 'outside_share1', 'outside...`, bind/unpack it into `(ax, key, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 469: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 470, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 471: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 473: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 474: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 475: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 476–476:

```
save(fig,'all_state_shares')
```

Read this block:

- Line 476: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

save

Original function: `save`. Source begins at line 479.

This formatting/output helper takes a Matplotlib Figure and filename stem. `tight_layout` adjusts spacing; a two-element suffix loop saves both PDF and PNG; `plt.close` frees the Figure. `OUT / f"{name}.{suffix}"` constructs each Path. Shared export code prevents one figure accidentally lacking its vector PDF version. DPI matters chiefly for raster PNG content; it does not make an ordinary vector line in a PDF more mathematically precise.

Definition and stated intent:

Source lines 479–479:

```
479 def save(fig,name):
```

`def` creates this callable; the parentheses name its inputs (`fig`, `name`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 480–480:

```
480 fig.tight_layout()
```

Read this block:

- Line 480: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 481–482:

```
481 for suffix in ['pdf','png']:
482     fig.savefig(OUT/f'{name}.{suffix}',dpi=160)
```

Read this block:

- Line 481: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `suffix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 482: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 483–483:

```
483 plt.close(fig)
```

Read this block:

- Line 483: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

make derivative figures

Original function: `make_derivative_figures`. Source begins at line 486.

Runs the configured theoretical parameter grid and plots solved comparative statics. It reuses the symbolic/numerical model rather than estimating demand from data.

Definition and stated intent:

Source lines 486–486:

```
486 def make_derivative_figures():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 487–487:

```
487 global plt
```

Read this block:

- Line 487: Global statement; read its literal source and the enclosing function's contract together.

Source lines 488–489:

```
488 plt = plotting()
489 OUT.mkdir(parents=True,exist_ok=True)
```

Read this block:

- Line 488, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 489: call `OUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 490–498:

```
490 if from_solutions:
491     with (OUT/'solutions.csv').open(newline='') as file:
492         saved=[{k:v if k=='initial_state' else float(v) for k,v in row.items()}
493                for row in csv.DictReader(file)]
494     curves={state:[r for r in saved if r['initial_state']==state] for state in ['j','0']}
495     for group in curves.values():
496         np.testing.assert_allclose([r['gamma'] for r in group],GAMMAS,atol=1e-12)
497 else:
498     curves={state:[evaluate(float(g),state,**PARAMETERS) for g in GAMMAS] for state in ['j','0']}
```

Read this block:

- Line 490: test `from_solutions`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 491: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 492, `saved`: Set this name to a list computed by the displayed for-expression.
- Line 494, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 495: for each entry yielded by `curves.values()`, bind/unpack it into `group` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 496: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 498, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 499–499:

```
499 outcomes=state_outcomes(curves)
```

Read this block:

- Line 499, `outcomes`: Set this name to the result of calling `state_outcomes(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 500–503:

```
500 with (OUT/'all_state_outcomes.csv').open('w',newline='') as file:
501     writer=csv.DictWriter(file,fieldnames=list(outcomes[0]))
502     writer.writeheader()
503     writer.writerows(outcomes)
```

Read this block:

- Line 500: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 501, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 502: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 503: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.
- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 505–509:

```
505 for initial in ['0','j']:
506     groups={a:[r for r in outcomes if r['x0']==initial and r['x1']==a] for a in ['0','j']}
507     expected=[sum(groups[a][i]['probability']*groups[a][i]['realized_profit']
```

```
                for a in ['0','j']) for i in range(len(GAMMAS))]
    np.testing.assert_allclose(expected,[r['profit'] for r in curves[initial]],atol=1e-12)
```

Read this block:

- Line 505: for each entry yielded by `['0', 'j']`, bind/unpack it into `initial` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 506, `groups`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 507, `expected`: Set this name to a list computed by the displayed for-expression.
- Line 509: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `len`: Count elements/rows rather than summing their numerical values.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 510–510:

```
plot_state_outcomes(outcomes)
```

Read this block:

- Line 510: call `plot_state_outcomes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 511–514:

```
for r in outcomes:
    for period in [1,2]:
        assert 0<=r[f'share{period}']<=1
        np.testing.assert_allclose(r[f'share{period}']+r[f'outside_share{period}'],1)
```

Read this block:

- Line 511: for each entry yielded by `outcomes`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 512: for each entry yielded by `[1, 2]`, bind/unpack it into `period` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 513: require `0 <= r[f'share{period}'] <= 1` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 514: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 515–515:

```
plot_state_shares(outcomes)
```

Read this block:

- Line 515: call `plot_state_shares` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_shares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 516–520:

```
516 with (OUT/'solutions.csv').open('w',newline='') as file:
517     writer=csv.DictWriter(file,fieldnames=list(curves['j'][0]))
518     writer.writeheader()
519     for state in ['j','0']:
520         writer.writerows(curves[state])
```

Read this block:

- Line 516: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 517, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 518: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 519: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 520: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 521–524:

```
521 specs=[('profit_by_gamma','profit',r'$\Pi^*$','Optimized two-period profit'),
522        ('gamma_profit_by_gamma','dprofit',r'$d\Pi^*/d\gamma$','Optimized profit derivative'),
523        ('gamma_p1_by_gamma','dp1',r'$dp_1^*/d\gamma$','Optimal period-1 price derivative'),
524        ('matched_price_ratio_derivative','d_matched_ratio',r'$dR_a/d\gamma$','State-matched price-ratio
    ↪ derivatives')]
```

Read this block:

- Line 521, `specs`: Set this name to a list containing 4 displayed entries.

Source lines 525–533:

```
525 for name,key,ylabel,title in specs:
526     fig,ax=plt.subplots(figsize=(7.2,4.5))
527     for state in ['j','0']:
528         ax.plot(GAMMAS,[r[key] for r in curves[state]],label=STATE_LABELS[state],lw=2)
529     ax.axhline(0,color='gray',lw=.7)
530     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
531     ax.legend()
532     ax.grid(alpha=.2)
533     save(fig,name)
```

Read this block:

- Line 525: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 526, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 527: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 528: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 529: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 530: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 532: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 533: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

Source lines 534–540:

```
534 fig,ax=plt.subplots(figsize=(7.2,4.5))
535 ax.plot(GAMMAS,[r['d_induce_ratio'] for r in curves['0']],lw=2,
536         label=r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$')
537 ax.axhline(0,color='gray',lw=.7)
538 ax.set(xlabel=r'$\gamma$',ylabel=r'$dR_{0\to j}/d\gamma$',title='Induce-then-harvest price-ratio derivative')
539 ax.legend()
540 ax.grid(alpha=.2)
```

Read this block:

- Line 534, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 535: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 537: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 538: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 539: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 540: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 541–542:

```
541 save(fig,'induce_harvest_price_ratio_derivative')
542 fig,axes=plt.subplots(1,2,figsize=(10,4.2))
```

Read this block:

- Line 541: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 542, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 543–544:

```
543 for state in ['j','0']:
544     axes[0].plot(GAMMAS,[r['p1'] for r in curves[state]],label=STATE_LABELS[state],lw=2)
```

Read this block:

- Line 543: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 544: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 545–546:

```
545 for state,key,label in [('j','p2j',r'Previous inside ($x_1=j$)'),('0','p20',r'Previous outside ($x_1=0$)')]:
546     axes[1].plot(GAMMAS,[r[key] for r in curves['j']],label=label,lw=2)
```

Read this block:

- Line 545: for each entry yielded by `[('j', 'p2j', 'Previous inside ($x_1=j$)'), ('0', 'p20', 'Prev...`, bind/unpack it into `(state, key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 546: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 547–551:

```
547 for ax,title,ylabel in zip(axes,['Optimal period-1 prices','Optimal period-2 prices'],[r'$p_1^*$',r'$p_2^*$']):
548     ax.axhline(0,color='gray',lw=.7)
549     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
550     ax.legend(fontsize=8)
551     ax.grid(alpha=.2)
```

Read this block:

- Line 547: for each entry yielded by `zip(axes, ['Optimal period-1 prices', 'Optimal period-2 prices...`, bind/unpack it into `(ax, title, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 548: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 549: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 550: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 551: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 552–553:

```
552 save(fig,'optimal_prices_by_gamma')
553 maximum=max(abs(r[k]) for state in curves for r in curves[state] for k in ['foc1','foc2j','foc20'])
```

Read this block:

- Line 552: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 553, `maximum`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 554–555:

```
554 if maximum>1e-9:
555     raise AssertionError(f'FOC residual {maximum}')
```

Read this block:

- Line 554: test `maximum > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 555: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 556–556:

```
556 print(f'Saved ten figures, {2*len(GAMMAS):,} solved observations, and {len(outcomes):,} conditional outcomes; max
    ↪  FOC residual={maximum:.3g}')
```

Read this block:

- Line 556: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 566.

Checks the theory solver's special cases, analytic results and numerical optimality. It is a pure-theory check; passing it is not evidence that a server calibration is identified.

Definition and stated intent:

Source lines 566–566:

```
566 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 567–578:

```
567 for gamma, prior, delta in [(0., .5, .92), (-2., .25, .92), (1.5, .75, .92),
568                            (-1., 0., .92), (1., 1., .92), (1., .5, 0.)]:
569     solution = supply_solution(gamma, prior, delta=delta)
570     paths, summary = outcome_tables(solution)
571     assert len(paths) == 12 and len(summary) == 3
572     for key in ["hidden_foc", "committed_foc1", "committed_foc2"]:
573         assert abs(solution[key]) < 1e-6
574     if gamma == 0:
575         assert np.ptp([row["optimal_profit"] for row in summary]) < 1e-7
576         assert all(np.isclose(row["price_ratio"], 1., atol=1e-6) for row in paths)
577     if prior in (0., 1.):
```

```
578         assert np.isclose(solution["known_value"], solution["hidden_value"], atol=1e-7)
```

Read this block:

- Line 567: for each entry yielded by `[(0.0, 0.5, 0.92), (-2.0, 0.25, 0.92), (1.5, 0.75, 0.92), (-1....`, bind/unpack it into `(gamma, prior, delta)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 569, `solution`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 570, `(paths, summary)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 571: require `len(paths) == 12 and len(summary) == 3` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 572: for each entry yielded by `['hidden_foc', 'committed_foc1', 'committed_foc2']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 573: require `abs(solution[key]) < 1e-06` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 574: test `gamma == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 575: require `np.ptp([row['optimal_profit'] for row in summary]) < 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 576: require `all((np.isclose(row['price_ratio'], 1.0, atol=1e-06) for row in paths))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 577: test `prior in (0.0, 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 578: require `np.isclose(solution['known_value'], solution['hidden_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `len`: Count elements/rows rather than summing their numerical values.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 579–579:

```
579 print("Supply checks passed: all cases, four histories, probabilities, profit reconstruction, FOCs, degenerate
    ↪  priors, gamma=0, delta=0.")
```

Read this block:

- Line 579: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

write run notes

Original function: `write_run_notes`. Source begins at line 581.

Writes a readable text rendering of the same automatic run information saved as JSON. pformat lays out nested records for reading; this is an output record, not an alternative settings file.

Definition and stated intent:

Source lines 581–582:

```
581 def write_run_notes(output, information):
582     """Write the same run information in plain text; you never edit a JSON file."""
```

`def` creates this callable; the parentheses name its inputs (`output`, `information`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 583–588:

```
583 output.mkdir(parents=True, exist_ok=True)
584 text = "RUN INFORMATION\n\n"
585 text += "This is an output record, not a settings file.\n"
586 text += "Change the Python parameters at the top of the script to change the run.\n\n"
587 text += pformat(information, sort_dicts=False, width=90)
588 (output / "run_notes.txt").write_text(text + "\n")
```

Read this block:

- Line 583: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 584, `text`: Set this name to the literal text `'RUN INFORMATION\n\n'`.
- Line 585: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 586: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 587: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 588: call `(output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 597.

Parses theoretical grids and output options, including skipping derivative plots and benchmark appendices. Changing flavor_spec is not part of this binary simulation.

Definition and stated intent:

Source lines 597–598:

```
597 def read_command_line(settings):
598     """Optional server flags. Normal use: edit the parameters at the top."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 599–604:

```
599 parser = argparse.ArgumentParser(description=__doc__)
600 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "supply_outputs")
601 parser.add_argument("--gamma-min", type=float, default=-3.)
602 parser.add_argument("--gamma-max", type=float, default=3.)
603 parser.add_argument("--gamma-points", type=int, default=61)
604 parser.add_argument("--priors", type=float, nargs="+", default=[.25, .5, .75])
```

Read this block:

- Line 599, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 600: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 601: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 602: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 603: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 604: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 605–610:

```
605 parser.add_argument("--beta", type=float, default=2.)
606 parser.add_argument("--alpha", type=float, default=-1., help="Signed price coefficient, negative.")
607 parser.add_argument("--delta", type=float, default=.92)
608 parser.add_argument("--cost", type=float, default=0.)
609 parser.add_argument("--self-test", action="store_true")
610 parser.add_argument("--derivative-gamma-min", type=float)
```

Read this block:

- Line 605: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 606: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 607: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 608: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 609: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 610: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 611–616:

```
611 parser.add_argument("--derivative-gamma-max", type=float)
612 parser.add_argument("--derivative-gamma-points", type=int)
613 parser.add_argument("--skip-derivatives", action="store_true")
614 parser.add_argument("--from-solutions", action="store_true")
615 parser.add_argument("--no-appendix", action="store_true")
616 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 611: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 612: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 613: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 614: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 615: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 616: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 617–617:

```
617 return parser.parse_args()
```

Read this block:

- Line 617: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run supply

Original function: `run_supply`. Source begins at line 626.

Validates settings, solves the configured theory cases, saves tables/figures and optional derivative/appendix outputs. This entry point is independent of the empirical after-data pipeline.

Definition and stated intent:

Source lines 626–627:

```
626 def run_supply(settings):
627     """The actual research steps, in the order they run."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 628–628:

```
628 global PARAMETERS, GAMMAS, OUT, from_solutions
```

Read this block:

- Line 628: Global statement; read its literal source and the enclosing function's contract together.

Source lines 629–632:

```
PARAMETERS = dict(beta=settings.beta, alpha=settings.alpha, delta=settings.delta, c=settings.cost)
GAMMAS = np.linspace(settings.derivative_gamma_min, settings.derivative_gamma_max,
    settings.derivative_gamma_points)
OUT = settings.output / "derivatives" / "figures"
from_solutions = settings.from_solutions
```

Read this block:

- Line 629, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 630, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 631, `OUT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 632, `from_solutions`: Set this name to the attribute `settings.from_solutions` of the object before the dot.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 633–635:

```
if settings.self_test:
    self_test()
    return
```

Read this block:

- Line 633: test `settings.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 634: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 635: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 636–637:

```
if settings.gamma_points < 2 or settings.gamma_max <= settings.gamma_min:
    raise ValueError("Need >=2 gamma points and gamma-max > gamma-min.")
```

Read this block:

- Line 636: test `settings.gamma_points < 2 or settings.gamma_max <= settings.gamma_min`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 637: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 638–645:

```
638 if not settings.skip_derivatives:
639     if settings.derivative_gamma_points < 2 or settings.derivative_gamma_max <= settings.derivative_gamma_min:
640         raise ValueError("The derivative grid needs at least two points and increasing endpoints.")
641     if not settings.no_appendix:
642         benchmark_parameters = dict(beta=2.0, alpha=-1.0, delta=0.92, c=0.0)
643         benchmark_grid = np.linspace(-3.0, 3.0, 601)
644         if PARAMETERS != benchmark_parameters or not np.array_equal(GAMMAS, benchmark_grid):
645             raise ValueError("The written appendix describes the original benchmark. Use no_appendix=True for a
    ↪  custom calibration/grid.")
```

Read this block:

- Line 638: test `not settings.skip_derivatives`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 639: test `settings.derivative_gamma_points < 2 or settings.derivative_gamma_max <= sett...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 640: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 641: test `not settings.no_appendix`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 642, `benchmark_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 643, `benchmark_grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 644: test `PARAMETERS != benchmark_parameters or not np.array_equal(GAMMAS, benchmark_grid)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 645: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 646–647:

```
646 settings.output.mkdir(parents=True, exist_ok=True)
647 solutions, paths, summaries = [], [], []
```

Read this block:

- Line 646: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 647, `(solutions, paths, summaries)`: Unpack a tuple containing 3 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 648–656:

```
648 for prior in settings.priors:
649     for i, gamma in enumerate(np.linspace(settings.gamma_min, settings.gamma_max, settings.gamma_points)):
650         result = supply_solution(gamma, prior, settings.beta, settings.alpha, settings.delta, settings.cost)
651         history_rows, summary_rows = outcome_tables(result)
652         solutions.append(result)
653         paths.extend(history_rows)
654         summaries.extend(summary_rows)
655         if i % 10 == 0 or i == settings.gamma_points-1:
656             print(f"Prior {prior:g}: solved {i+1}/{settings.gamma_points} gamma values", flush=True)
```

Read this block:

- Line 648: for each entry yielded by `settings.priors`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 649: for each entry yielded by `enumerate(np.linspace(settings.gamma_min, settings.gamma_max, ...`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 650, `result`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 651, `(history_rows, summary_rows)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 652: call `solutions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 653: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 654: call `summaries.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 655: test `i % 10 == 0 or i == settings.gamma_points - 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 656: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 657–661:

```
657 raw, histories, expected = pd.DataFrame(solutions), pd.DataFrame(paths), pd.DataFrame(summaries)
658 raw.to_csv(settings.output / "supply_solutions.csv", index=False)
659 histories.to_csv(settings.output / "supply_history_outcomes.csv", index=False)
660 expected.to_csv(settings.output / "supply_expected_outcomes.csv", index=False)
661 residual = float(raw[["hidden_foc", "committed_foc1", "committed_foc2"]].abs().to_numpy().max())
```

Read this block:

- Line 657, `(raw, histories, expected)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 658: call `raw.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659: call `histories.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 660: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 661, `residual`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 662–663:

```
662 if residual > 1e-6:
663     raise AssertionError(f"FOC residual too large: {residual}")
```

Read this block:

- Line 662: test `residual > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 663: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 664–669:

```
664 make_case_figures(histories, expected, settings.output)
665 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k,v in vars(settings).items()},
666                 maximum_foc_residual=residual, calibrations=len(raw), cases=CASES,
667                 script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest())
668 (settings.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
669 write_run_notes(settings.output, manifest)
```

Read this block:

- Line 664: call `make_case_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 665, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 668: call `(settings.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 669: call `write_run_notes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_case_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `write_run_notes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 670–673:

```
670 if not settings.skip_derivatives:
671     make_derivative_figures()
672     if not settings.no_appendix:
673         theory.make_latex_appendix(OUT.parent, PARAMETERS, GAMMAS, evaluate)
```

Read this block:

- Line 670: test `not settings.skip_derivatives`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 671: call `make_derivative_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 672: test `not settings.no_appendix`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 673: call `theory.make_latex_appendix` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_derivative_figures`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.

Source lines 674–674:

```
674 print(f"All three supply cases completed. CSVs and PNG/PDF figures: {settings.output}")
```

Read this block:

- Line 674: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

../final_simple/supply/supply_sympy.py

Symbolic binary-theory equations, envelope/implicit derivatives and benchmark appendix generation. This module defines algebra rather than loading household histories.

Module setup and direct-execution blocks

Source lines 1–9:

```
1 """Supply mathematics: symbols, utilities, FOCs, and comparative statics.
2
3 This is the single source of the theoretical derivatives. supply_simulation.py
4 imports these formulas when it evaluates and plots them. This file never reads
5 empirical data. Running it directly prints equations; it does not run a simulation.
6 """
7 from pathlib import Path
8 import numpy as np
9 import sympy as sp
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `sympy as sp`. An as-name is a convenient local alias, not another library.

Source lines 20–30:

```
20 alpha, beta, gamma, delta, c = sp.symbols("alpha beta gamma delta c", real=True)
21
22 p1, p2, p2j, p20 = sp.symbols("p_1 p_2 p_{2j} p_{20}", real=True)
23
24 q, q2 = sp.symbols("q q_2", real=True)
```

```
25
26 sj1, sj2, u, v = sp.symbols("s s_2 u v", positive=True)
27
28 s01, s02 = 1-sj1, 1-sj2
29
30 Vj, V0 = sp.symbols("V_{2j} V_{20}", real=True)
```

Read this block:

- Line 20, `(alpha, beta, gamma, delta, c)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 22, `(p1, p2, p2j, p20)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 24, `(q, q2)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 26, `(sj1, sj2, u, v)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 28, `(s01, s02)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 30, `(Vj, V0)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.symbols`: Create SymPy symbols, which stand for algebraic variables rather than observed numbers.

Source lines 32–36:

```
32 h0, h1 = sp.symbols("h_0 h_1")
33
34 p1_star_sym, p2_star_sym = sp.symbols("p_1^* p_2^*")
35
36 z1, z2 = q, q2
```

Read this block:

- Line 32, `(h0, h1)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 34, `(p1_star_sym, p2_star_sym)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 36, `(z1, z2)`: Unpack the tuple `(q, q2)` into these targets in order.

Source lines 49–59:

```
49 raw_inside1, raw_outside1 = raw_utilities(p1, q)
50
51 raw_inside2, raw_outside2 = raw_utilities(p2, q2)
52
53 U1 = sp.expand(raw_inside1 - raw_outside1)
54
55 U2 = sp.expand(raw_inside2 - raw_outside2)
56
57 sj1_expr = 1/(1+sp.exp(-U1))
58
59 sj2_expr = 1/(1+sp.exp(-U2))
```

Read this block:

- Line 49, `(raw_inside1, raw_outside1)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 51, `(raw_inside2, raw_outside2)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 53, `U1`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.
- Line 55, `U2`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.
- Line 57, `sj1_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 59, `sj2_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.expand`: Distribute products and combine terms in a symbolic expression without solving its unknowns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 61–71:

```
61 s01_expr, s02_expr = 1-sj1_expr, 1-sj2_expr
62
63 dsj1_dp1, dsj2_dp2 = alpha*sj1*s01, alpha*sj2*s02
64
65 ds01_dp1, ds02_dp2 = -dsj1_dp1, -dsj2_dp2
66
67 dsj1_dgamma = q*sj1*s01
68
69 dsj2_dgamma_direct = q2*sj2*s02
70
71 dsj2_dutility = sj2*s02
```

Read this block:

- Line 61, `(s01_expr, s02_expr)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 63, `(dsj1_dp1, dsj2_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 65, `(ds01_dp1, ds02_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 67, `dsj1_dgamma`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `dsj2_dgamma_direct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 71, `dsj2_dutility`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 79–89:

```
79 pi1 = sj1*(p1-c)
80
81 profit2 = sj2*(p2-c)
82
83 pi2 = sj1*Vj + (1-sj1)*V0
84
85 Pi = pi1 + delta*pi2
86
87 DeltaV = Vj-V0
88
89 foc1_raw = sj1 + alpha*sj1*s01*(p1-c) + delta*alpha*sj1*s01*DeltaV
```

Read this block:

- Line 79, `pi1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 81, `profit2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `pi2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `Pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 87, `DeltaV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 89, `foc1_raw`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 91–101:

```
foc1_collected = sj1 + alpha*sj1*s01*(p1-c+delta*DeltaV)

foc1_eq = sp.Eq(foc1_collected, 0)

foc2 = sj2 + alpha*sj2*s02*(p2-c)

foc2_eq = sp.Eq(foc2, 0)

p1_star = c-1/(alpha*s01)-delta*(Vj-V0)

p1_star_eq = sp.Eq(p1_star_sym, p1_star)
```

Read this block:

- Line 91, `foc1_collected`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 93, `foc1_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.
- Line 95, `foc2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97, `foc2_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.
- Line 99, `p1_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 101, `p1_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

Source lines 103–109:

```
p2_star = c-1/(alpha*s02)

p2_star_eq = sp.Eq(p2_star_sym, p2_star)

assumptions = {"alpha": "<0", "delta": "0<delta<1", "prices": "unconstrained real",
               "cost": "fixed", "shares": "strictly interior", "type": "fixed and observed",
               "history": "realized j or 0; both continuation states integrated"}
```

Read this block:

- Line 103, `p2_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 105, `p2_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.
- Line 107, `assumptions`: Set this name to a dictionary with named entries `alpha`, `delta`, `prices`, `cost`, `shares`, `type`, `history`.

Source lines 194–204:

```
dp1_dgamma = price1_derivative(gamma)

dp2_dgamma = price2_derivative(gamma)

dVj_dgamma, dV0_dgamma = value2_derivative(gamma,u,-1,p2j), value2_derivative(gamma,v,1,p20)

ds_dgamma = sp.factor(sj1*s01*(q+alpha*dp1_dgamma))

du_dgamma, dv_dgamma = -u*(1-u)**2, v*(1-v)**2

d2p1_dgamma2 = total_derivative(dp1_dgamma,gamma)
```

Read this block:

- Line 194, `dp1_dgamma`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 196, `dp2_dgamma`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 198, `(dVj_dgamma, dV0_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 200, `ds_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 202, `(du_dgamma, dv_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 204, `d2p1_dgamma2`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 232–242:

```
d2p2_dgamma2 = terminal_total_derivative(dp2_dgamma,gamma)

ds2_dgamma = sp.factor(sj2*s02*(q2+alpha*dp2_dgamma))

dp2dbeta_dgamma = terminal_total_derivative(price2_derivative(beta),gamma)

dPi_dgamma = profit_derivative(gamma)

dPi_dbeta = profit_derivative(beta)

ratio_derivative = (p2*dp1_dgamma-p1*dp2_dgamma)/p2**2
```

Read this block:

- Line 232, `d2p2_dgamma2`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 234, `ds2_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.

- Line 236, `dp2dbeta_dgamma`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 238, `dPi_dgamma`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 240, `dPi_dbeta`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 242, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `terminal_total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `profit_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 244–248:

```
244 matched_j = ratio_derivative.subs({q:-1,p2:p2j,sj2:u,q2:-1})
245
246 matched_0 = ratio_derivative.subs({q:1,p2:p20,sj2:v,q2:1})
247
248 induce_harvest = ratio_derivative.subs({q:1,p2:p2j,sj2:u,q2:-1})
```

Read this block:

- Line 244, `matched_j`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 246, `matched_0`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 248, `induce_harvest`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

Source lines 529–540:

```
529 if __name__ == "__main__":
530     show("Inside utility in period 1", U1)
531     show("Inside utility in period 2", U2)
532     show("Expected discounted profit", Pi)
533     show("Period 1 first-order condition", foc1_collected)
534     show("Terminal first-order condition", foc2)
535     show("Period 1 optimal price", p1_star)
536     show("Terminal optimal price", p2_star)
537     show("dp1/dgamma", dp1_dgamma)
538     show("d2p1/dgamma2", d2p1_dgamma2)
539     show("dprofit/dgamma", dPi_dgamma)
540     show("Price-ratio derivative", ratio_derivative)
```

Read this block:

- Line 529: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 530: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 532: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 533: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 534: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 535: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 536: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 537: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 538: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 539: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 540: call `show` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `show`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

signed state

Original function: `signed_state`. Source begins at line 38.

This converts the two accepted string state names into integer signs. Two independent `if` blocks are fine because each successful branch returns at once. Any other value reaches `raise ValueError`. `signed_state("j")` returns -1; `signed_state("0")` returns +1; numeric `0` is not string `"0"` and is rejected. A dictionary lookup could shorten this to a mapping, but these explicit branches make both accepted values and the custom error easy for a novice to see.

Definition and stated intent:

Source lines 38–38:

```
38 def signed_state(state):
```

`def` creates this callable; the parentheses name its inputs (`state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 39–40:

```
39 if state == "j":
40     return -1
```

Read this block:

- Line 39: test `state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 40: return the negated or sign-adjusted expression `-1`. This ends the call.

Source lines 41–42:

```
41 if state == "0":
42     return 1
```

Read this block:

- Line 41: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42: return the literal value `1`. This ends the call.

Source lines 43–43:

```
43 raise ValueError("state must be 'j' or '0'")
```

Read this block:

- Line 43: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

raw utilities

Original function: `raw_utilities`. Source begins at line 45.

Input `price` may be a SymPy symbol and `signed_history` a sign or symbolic q. The return statement contains two comma-separated expressions, so it returns a tuple `(inside_utility, outside_utility)`. The combinations `(1+q)/2` and `(1-q)/2` act as complementary indicators when q is ±1. At q=-1 the first is zero and second one; at q=+1 they reverse.

Arithmetic with SymPy objects builds algebraic expressions rather than computing a numerical utility immediately. Returning both raw utilities preserves the outside switching payoff before normalization. Writing only an inside bonus would encode a different model. Module-level unpacking later names each result and subtracts the outside utility explicitly.

Definition and stated intent:

Source lines 45–45:

```
45 def raw_utilities(price, signed_history):
```

`def` creates this callable; the parentheses name its inputs (`price`, `signed_history`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–47:

```
46 return (beta + alpha*price + gamma*(1+signed_history)/2,
47         gamma*(1-signed_history)/2)
```

Read this block:

- Line 46: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

price2 derivative

Original function: `price2_derivative`. Source begins at line 119.

This returns a **symbolic expression** for the terminal optimal-price derivative with respect to symbolic parameter theta. Default arguments `sj2`, `q2`, and `p2` are shared symbols; callers substitute a particular branch by passing u/-1/p2j or v/+1/p20. `sp.diff(beta, theta)` is one when theta is beta and zero for an independent parameter, so one generic expression covers several comparative statics.

`direct` is the fixed-price derivative of the utility index. The returned formula already incorporates implicit share/price feedback; `simplify` reduces algebra. Differentiating the markup while holding its share fixed would omit that feedback. Unlike numerical finite differences, symbolic derivatives do not need a step size, though they still depend on a correct symbolic model.

Definition and stated intent:

Source lines 119–120:

```
119 def price2_derivative(theta, share=sj2, history=q2, price=p2):
120     """Implicit derivative of the terminal optimal price with respect to theta."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–128:

```
122 intercept_effect = sp.diff(beta, theta)
123 price_coefficient_effect = price * sp.diff(alpha, theta)
124 history_effect = history * sp.diff(gamma, theta)
125 direct_utility_effect = intercept_effect + price_coefficient_effect + history_effect
126
127 cost_effect = (1 - share) * sp.diff(c, theta)
128 alpha_effect = sp.diff(alpha, theta) / alpha**2
```

Read this block:

- Line 122, `intercept_effect`: Set this name to the result of calling `sp.diff(...)`; the arguments and preceding method chain are shown above.
- Line 123, `price_coefficient_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 124, `history_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 125, `direct_utility_effect`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 127, `cost_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 128, `alpha_effect`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 129–129:

```
129 demand_effect = -share * direct_utility_effect / alpha
```

Read this block:

- Line 129, `demand_effect`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 130–130:

```
130 return sp.simplify(cost_effect + alpha_effect + demand_effect)
```

Read this block:

- Line 130: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.

value2 derivative

Original function: `value2_derivative`. Source begins at line 132.

This computes the symbolic envelope derivative of a reoptimized continuation value. It uses the same fixed-price index derivative as terminal pricing, then returns `-share*direct/alpha - share*diff(c,theta)`. There is no explicit derivative of optimal price because the terminal FOC cancels that first-order effect. Omitting price

responses here is justified by the envelope theorem, not a general Python or SymPy rule. Substituting fully expanded optimal prices would be harder algebra with the same result under the maintained optimum conditions.

Definition and stated intent:

Source lines 132–133:

```
132 def value2_derivative(theta, share, history, price):
133     """Terminal value derivative from the envelope theorem."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 134–138:

```
134 direct_utility_effect = (sp.diff(beta, theta)
135                          + price * sp.diff(alpha, theta)
136                          + history * sp.diff(gamma, theta))
137 demand_effect = -share * direct_utility_effect / alpha
138 cost_effect = -share * sp.diff(c, theta)
```

Read this block:

- Line 134, `direct_utility_effect`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 137, `demand_effect`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 138, `cost_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 139–139:

```
139 return demand_effect + cost_effect
```

Read this block:

- Line 139: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

price1 derivative

Original function: `price1_derivative`. Source begins at line 143.

This forms the symbolic first-period price derivative. It builds its direct utility effect, obtains the two terminal envelope derivatives, subtracts them, and inserts the result into the implicit first-price formula. `sp.diff(delta, theta)` includes a direct discount-factor change when theta is delta; for gamma that term is zero but the continuation values still change.

The two helper calls use the correct opposite signed terminal histories. Maintaining a single generic function avoids separate alpha/beta/delta/gamma formulas disagreeing about those branches. `sp.simplify` does algebraic cleanup; it does not decide whether a term is positive under the economic domain.

Definition and stated intent:

Source lines 143–144:

```
143 def price1_derivative(theta):
144     """Implicit derivative of the initial price, including both future branches."""
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 145–153:

```
145 direct_utility_effect = (sp.diff(beta, theta)
146                          + p1 * sp.diff(alpha, theta)
147                          + q * sp.diff(gamma, theta))
148 inside_value_effect = value2_derivative(theta, u, -1, p2j)
149 outside_value_effect = value2_derivative(theta, v, 1, p20)
150 value_difference_effect = inside_value_effect - outside_value_effect
151
152 cost_effect = s01 * sp.diff(c, theta)
153 alpha_effect = sp.diff(alpha, theta) / alpha**2
```

Read this block:

- Line 145, `direct_utility_effect`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 148, `inside_value_effect`: Set this name to the result of calling `value2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 149, `outside_value_effect`: Set this name to the result of calling `value2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 150, `value_difference_effect`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 152, `cost_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 153, `alpha_effect`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 154–156:

```
154 current_demand_effect = -sj1 * direct_utility_effect / alpha
155 continuation_effect = -s01 * (sp.diff(delta, theta) * (Vj - V0)
156                              + delta * value_difference_effect)
```

Read this block:

- Line 154, `current_demand_effect`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 155, `continuation_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 157–158:

```
157 return sp.simplify(cost_effect + alpha_effect + current_demand_effect
158                    + continuation_effect)
```

Read this block:

- Line 157: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.

total derivative

Original function: `total_derivative`. Source begins at line 161.

Input is a symbolic expression and differentiation parameter. Output is its total derivative along the solved equilibrium. The first assignments compute price derivatives, then logistic chain rules compute derivatives of the three shares. A dictionary maps each endogenous symbol to its derivative.

The return is explicit partial differentiation plus `sum(diff(expr,k)*val for k,val in substitutions.items())`. This is the chain rule over every mapped endogenous object. The dictionary is a derivative map, not values actually substituted with `.subs` in this function. `sp.factor` collects common algebraic factors. Plain `sp.diff(expr,theta)` would treat shares, prices, and values as independent symbols and miss their indirect changes.

Definition and stated intent:

Source lines 161–162:

```
161 def total_derivative(expr, theta):
162     """Apply the chain rule to an expression evaluated at optimal prices."""
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 163–176:

```
163 initial_price_effect = price1_derivative(theta)
164 terminal_inside_price_effect = price2_derivative(theta, u, -1, p2j)
165 terminal_outside_price_effect = price2_derivative(theta, v, 1, p20)
166
167 # logit share derivative = share*(1-share)*utility derivative.
168 initial_share_effect = sj1 * s01 * (
169     sp.diff(beta, theta) + p1 * sp.diff(alpha, theta)
170     + q * sp.diff(gamma, theta) + alpha * initial_price_effect)
171 terminal_inside_share_effect = u * (1 - u) * (
172     sp.diff(beta, theta) + p2j * sp.diff(alpha, theta)
173     - sp.diff(gamma, theta) + alpha * terminal_inside_price_effect)
174 terminal_outside_share_effect = v * (1 - v) * (
175     sp.diff(beta, theta) + p20 * sp.diff(alpha, theta)
176     + sp.diff(gamma, theta) + alpha * terminal_outside_price_effect)
```

Read this block:

- Line 163, `initial_price_effect`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 164, `terminal_inside_price_effect`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 165, `terminal_outside_price_effect`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 168, `initial_share_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 171, `terminal_inside_share_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 174, `terminal_outside_share_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 179–189:

```
179 changes = {
180     p1: initial_price_effect,
181     p2j: terminal_inside_price_effect,
182     p20: terminal_outside_price_effect,
183     sj1: initial_share_effect,
184     u: terminal_inside_share_effect,
185     v: terminal_outside_share_effect,
186     Vj: value2_derivative(theta, u, -1, p2j),
187     V0: value2_derivative(theta, v, 1, p20),
188 }
189 derivative = sp.diff(expr, theta)
```

Read this block:

- Line 179, `changes`: Set this name to a dictionary from the displayed keys and values.
- Line 189, `derivative`: Set this name to the result of calling `sp.diff(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 190–191:

```
190 for variable, variable_change in changes.items():
191     derivative += sp.diff(expr, variable) * variable_change
```

Read this block:

- Line 190: for each entry yielded by `changes.items()`, bind/unpack it into `(variable, variable_change)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 191: update `derivative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 192–192:

```
192 return sp.factor(derivative)
```

Read this block:

- Line 192: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.

terminal total derivative

Original function: `terminal_total_derivative`. Source begins at line 206.

This is the smaller one-branch version of the total derivative. It calculates terminal price response, computes terminal share response including repricing, and adds explicit theta, price, and share chain-rule terms. Because

there is only one terminal branch, a short direct expression is clearer than building the full period-1 derivative dictionary. It is used for terminal curvature and mixed derivatives; a second plain symbolic partial derivative would hold the share fixed and give the wrong comparative static.

Definition and stated intent:

Source lines 206–207:

```
206 def terminal_total_derivative(expr, theta):
207     """The same chain rule, restricted to a single terminal history."""
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 208–214:

```
208 price_effect = price2_derivative(theta)
209 utility_effect = (sp.diff(beta, theta) + p2 * sp.diff(alpha, theta)
210                   + q2 * sp.diff(gamma, theta) + alpha * price_effect)
211 share_effect = sj2 * s02 * utility_effect
212 derivative = (sp.diff(expr, theta)
213               + sp.diff(expr, p2) * price_effect
214               + sp.diff(expr, sj2) * share_effect)
```

Read this block:

- Line 208, `price_effect`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 209, `utility_effect`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 211, `share_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 212, `derivative`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 215–215:

```
215 return sp.factor(derivative)
```

Read this block:

- Line 215: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.

profit derivative

Original function: `profit_derivative`. Source begins at line 217.

This returns the optimized two-period profit derivative with respect to theta. `partial` combines direct current demand/cost effects, direct discounting, and both branch envelope effects. `.subs(p1, markup_expression)` inserts the first-period optimal pricing equation, then `simplify` reduces the result.

The code does not explicitly multiply by dp1/dtheta because the first-period FOC makes that contribution zero at the optimum. Replacing the entire objective with current revenue before differentiating would lose continuation effects. This helper is reusable for gamma and beta, while relying on the same fixed-type, two-branch, interior optimum as the rest of the model.

Definition and stated intent:

Source lines 217–218:

```
217 def profit_derivative(theta):
218     """Envelope derivative of optimized profit, including both terminal values."""
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 219–227:

```
219 direct_utility_effect = (sp.diff(beta, theta) + p1 * sp.diff(alpha, theta)
220                          + q * sp.diff(gamma, theta))
221 current_demand_effect = direct_utility_effect * sj1 * s01 * (p1 - c + delta * (Vj - V0))
222 current_cost_effect = -sj1 * sp.diff(c, theta)
223 discount_effect = sp.diff(delta, theta) * (sj1 * Vj + s01 * V0)
224 future_value_effect = delta * (
225     sj1 * value2_derivative(theta, u, -1, p2j)
226     + s01 * value2_derivative(theta, v, 1, p20))
227 derivative = current_demand_effect + current_cost_effect + discount_effect + future_value_effect
```

Read this block:

- Line 219, `direct_utility_effect`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 221, `current_demand_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 222, `current_cost_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 223, `discount_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 224, `future_value_effect`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 227, `derivative`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 229–229:

```
229 optimal_initial_price = c - 1 / (alpha * s01) - delta * (Vj - V0)
```

Read this block:

- Line 229, `optimal_initial_price`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 230–230:

```
230 return sp.simplify(derivative.subs(p1, optimal_initial_price))
```

Read this block:

- Line 230: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

display equation

Original function: `display_equation`. Source begins at line 256.

This returns a LaTeX display-equation string. `lhs` is already a LaTeX label; `sp.latex(expression)` serializes the actual symbolic right-hand side. `'\\[\n'` contains a backslash followed by a bracket and a newline; Python string escapes are why source contains doubled backslashes. Concatenation places the equation between display delimiters. Deriving the right side from SymPy avoids manually copying a formula that later changes in the code. The function returns text and does not itself invoke a TeX compiler.

Definition and stated intent:

Source lines 256–257:

```
256 def display_equation(lhs, expression):
257     """Keep the mathematical right-hand side sourced directly from SymPy."""
```

`def` creates this callable; the parentheses name its inputs (`lhs`, `expression`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 258–258:

```
258 return '\\[\n' + lhs + ' = ' + sp.latex(expression) + '\n\\]\n'
```

Read this block:

- Line 258: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.latex`: Serialize symbolic algebra as LaTeX source rather than recomputing the mathematical formula manually.

show

Original function: `show`. Source begins at line 260.

This is a terminal-output helper returning None. Blank `print()` creates a new line; `"=" * 70` repeats a string seventy times, whereas multiplication of numeric values has a different meaning. `sp.pprint` renders an expression in readable mathematical form in the terminal. It is useful for inspection, while `sp.latex` is the appropriate alternative for document source.

Definition and stated intent:

Source lines 260–260:

```
260 def show(title, expr):
```

`def` creates this callable; the parentheses name its inputs (`title`, `expr`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 261–265:

```
print()
print("=" * 70)
print(title)
print("=" * 70)
sp.pprint(expr)
```

Read this block:

- Line 261: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 262: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 263: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 264: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 265: call `sp.pprint` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.pprint`: Pretty-print symbolic mathematics to the terminal for human inspection.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

make latex appendix

Original function: `make_latex_appendix`. Source begins at line 267.

Writes a theoretical benchmark appendix from symbolic formulas, numerical parameter choices and a supplied evaluator. It generates documentation rather than editing presentation slides or accessing research records.

Definition and stated intent:

Source lines 267–267:

```
def make_latex_appendix(output, parameters, gammas, evaluate_model):
```

`def` creates this callable; the parentheses name its inputs (`output`, `parameters`, `gammas`, `evaluate_model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 268–268:

```
TEX_OUTPUT = Path(output)
```

Read this block:

- Line 268, `TEX_OUTPUT`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 269–270:

```
if parameters != dict(beta=2.0, alpha=-1.0, delta=0.92, c=0.0) or not np.array_equal(gammas, np.linspace(-3, 3,
    601)):
    raise ValueError("The written appendix describes the original benchmark. Set make_appendix=False for custom
        calibrations.")
```

Read this block:

- Line 269: test `parameters != dict(beta=2.0, alpha=-1.0, delta=0.92, c=0.0) or not np.array_e...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 270: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 271–273:

```
271 TEX_OUTPUT.mkdir(parents=True, exist_ok=True)
272 equations='\\section{Symbolically derived equations}\n'
273 equations += r'Here $q_2=q(x_1)$, $s_2=s_2(x_1)$ and $p_2=p_2(x_1)$ describe a fixed terminal history. Price
    ↪  equations and comparative-static right-hand sides use equilibrium shares; the profit and FOCs use shares at the
    ↪  candidate prices.'+'\n'
```

Read this block:

- Line 271: call `TEX_OUTPUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 272, `equations`: Set this name to the literal text `'\\section{Symbolically derived equations}\n'`.
- Line 273: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 274–280:

```
274 for lhs,expression in [(r'v_1',U1),(r'v_2',U2),(r'\Pi',Pi),
275                        (r'F_1',foc1_collected),(r'F_2',foc2),
276                        (r'p_1^*',p1_star),(r'p_2^*',p2_star),
277                        (r'\frac{dp_1^*}{d\gamma}',dp1_dgamma),
278                        (r'\frac{dp_2^*}{d\gamma}',dp2_dgamma),
279                        (r'\frac{d\Pi^*}{d\gamma}',dPi_dgamma)]:
280     equations+=display_equation(lhs,expression)
```

Read this block:

- Line 274: for each entry yielded by `[('v_1', U1), ('v_2', U2), ('\\Pi', Pi), ('F_1', foc1_collecte...`, bind/unpack it into `(lhs, expression)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 280: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `display_equation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 281–281:

```
281 (TEX_OUTPUT/'sympy_equations.tex').write_text(equations)
```

Read this block:

- Line 281: call `(TEX_OUTPUT / 'sympy_equations.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Source lines 282–323:

```
    document=r'''\documentclass[11pt]{article}
\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm,graphicx,booktabs,array}
\usepackage[hidelinks]{hyperref}
\newtheorem{proposition}{Proposition}
\newcommand{\dd}{\mathrm{d}}
\setlength{\parindent}{0pt}
\setlength{\parskip}{5pt}
\title{Two-period pricing with symmetric switching preferences}
\author{}\date{October 30, 2026}
\begin{document}\maketitle
\section{Model, histories, and normalization}
One firm sells one inside product $j$ against outside option $0$ in periods
$t=1,2$. Type $r$ is fixed and observed by the firm, which prices separately for
that type. All formulas below are conditional on $r$; its subscript and the
fixed product index are suppressed. There is no pooling of heterogeneous types.
The initial realized choice is $x_0\in\{j,0\}$. The second-period price can depend
on the realized first-period choice $x_1$. Independent Type-I extreme-value
shocks enter both alternatives in each period.

Before normalization the deterministic utilities are
\[
 V_{jt}^{\rm raw}=\beta+\alpha p_t+\gamma\mathbf1\{x_{t-1}=0\},
 \qquad V_{0t}^{\rm raw}=\gamma\mathbf1\{x_{t-1}=j\}.
\]
The payoff to switching applies in either direction. Subtracting the raw outside
utility from both alternatives gives normalized outside utility zero and
\[
 v_t(h)=\beta+\alpha p_t(h)+q(h)\gamma,\qquad
 q(j)=-1,\quad q(0)=+1.
\]
Normalization retains the outside switching payoff through the signed inside
utility difference; it does not delete that payoff. Negative $\gamma$ denotes
switching aversion and positive $\gamma$ denotes novelty preference.
\[
 s_t(h)=\frac{e^{v_t(h)}}{1+e^{v_t(h)}},\quad s_{0t}(h)=1-s_t(h),\quad
 \partial_p s_t=\alpha s_t(1-s_t),\quad
 (\partial_\gamma s_t)_p=q(h)s_t(1-s_t).
\]
A realized state $x_t$, a hypothetical switching indicator, and the probability
$s_t$ are distinct objects. Every derivative below holds the discrete history fixed.
```

Source lines 324–365:

```
\textbf{Assumptions.} $\alpha<0$, $0<\delta<1$, fixed marginal cost $c$, finite
parameters, and interior binary-logit shares. Prices are unrestricted real
numbers: introductory subsidies are allowed. Continuation values are conditional
on realized states and have no direct dependence on $p_1$. Type does not evolve.
The case $\delta=0$ is used only as a static counterfactual.

\begin{center}\begin{tabular}{ll}\toprule
Notation & Meaning\\\midrule
$q=q(x_0)$ & Signed initial-state coefficient\\
$s=s_1^*(x_0),\quad r_0=1-s$ & First-period equilibrium shares\\
$u=s_2^*(j),\quad v=s_2^*(0)$ & Inside shares in the two continuation states\\
$V_j=V_2(j),\quad V_0=V_2(0)$ & Optimized continuation profits\\
$\Delta V=V_j-V_0,\quad B=u+v$ & Continuation difference and sum of terminal shares\\
```

```
$\Pi^*$ & Expected discounted profit per initial consumer\\\bottomrule
\end{tabular}\end{center}

\section{Backward induction and optimality}
For each realized first-period choice $a\in\{j,0\}$, solve
\[
 V_2(a)=\max_{p_2(a)}s_2(a)[p_2(a)-c].
\]
Its first-order condition is
\[
 0=s_2+\alpha s_2(1-s_2)(p_2-c),\qquad
 p_2^*(a)=c-\frac1{\alpha[1-s_2^*(a)]}.
\]
This is an implicit equation: the share depends on the same price being solved.
Both $V_j$ and $V_0$ must be computed before period 1.

First-period profit integrates over both possible realized histories:
\begin{align*}
 \Pi(p_1;x_0)&=s_1(p_1-c)+\delta[s_1V_j+(1-s_1)V_0]\\
 &=\delta V_0+s_1[p_1-c+\delta\Delta V].
\end{align*}
Because the branch values do not depend directly on $p_1$,
\[
 \partial_{p_1}\{\delta[s_1V_j+(1-s_1)V_0]\}
 =\delta\alpha s_1(1-s_1)(V_j-V_0).
\]
Consequently,
\[
 0=s_1+\alpha s_1(1-s_1)[p_1-c+\delta\Delta V],\qquad
```

Source lines 366–407:

```
 p_1^*=c-\frac1{\alpha(1-s)}-\delta\Delta V.
\]
The static FOC component $c-1/[\alpha(1-s)]$ uses the share at the dynamic
optimum. It is not the counterfactual static optimum, which must be solved
separately with $\delta=0$.

\begin{proposition}[Unique optimal prices]
Under the stated assumptions each continuation price and the first-period price
has a unique finite global optimum, with no lower bound imposed on $p_1$.
\end{proposition}
\begin{proof}
Both price problems reduce, up to a constant, to maximizing
$s(p)(p-k)$ with $s(p)=\operatorname{logit}^{-1}(b+\alpha p)$.
Use $k=c$ in period 2 and $k=c-\delta\Delta V$ in period 1.
The sign of the profit derivative is the sign of
\[
 H(p)=1+e^{b+\alpha p}+\alpha(p-k),\quad
 H'(p)=\alpha(1+e^{b+\alpha p})<0.
\]
$H$ ranges from positive infinity to negative infinity, so its root is unique
and the objective increases before it and decreases afterward. The price may
lie below physical cost $c$ in period 1 when continuation incentives justify it.
\end{proof}

\section{Envelope derivatives and conditional comparative statics}
Differentiating each continuation value using the envelope theorem gives
\[
 V_2'(a)=q(a)s_2(1-s_2)(p_2^*-c)=-\frac{q(a)s_2}{\alpha}.
\]
Implicit differentiation of the terminal FOC gives the same expression for the
```

```
optimal price derivative:
\[
 p_2^{*\prime}(a)=-\frac{q(a)s_2}{\alpha},\quad
 V_j'=u/\alpha,\quad V_0'=-v/\alpha,\quad
 \Delta V'=(u+v)/\alpha.
\]
In particular, $p_2^{*\prime}(j)<0$ and $p_2^{*\prime}(0)>0$.
The total equilibrium terminal-share derivative is
$s_2'=q(a)s_2(1-s_2)^2$, including the optimal-price response.

For the first-period implicit pricing equation let
$F=p_1-c+1/[\alpha(1-s_1)]+\delta\Delta V$. At fixed history,
```

Source lines 408–449:

```
\[
 F_{p_1}=\frac1{r_0},\qquad
 F_\gamma=\frac{qs}{\alpha r_0}+\frac{\delta B}{\alpha},\qquad
 \boxed{p_1^{*\prime}=-\frac{qs+\delta r_0 B}{\alpha}}.
\]
The share therefore changes according to
\[
 s'=sr_0(q+\alpha p_1^{*\prime})=sr_0^2(q-\delta B).
\]
The first-period envelope theorem retains both continuation branches:
\begin{align*}
 \Pi^{*\prime}
 &=qsr_0[p_1^*-c+\delta\Delta V]+\delta[sV_j'+r_0V_0']\\
 &=\boxed{\frac{-qs+\delta(su-r_0v)}{\alpha}}.
\end{align*}
The term $\delta V_0'$ must not be dropped. The envelope partial derivative holds
the optimizing price fixed inside the partial derivative; it does not hold the
continuation value fixed as $\gamma$ varies.

For the analytical second price derivative define
\[
 u'=-u(1-u)^2,\quad v'=v(1-v)^2,\quad B'=u'+v'.
\]
Differentiating the first derivative and substituting $s'$ gives
\[
 \boxed{p_1^{*\prime\prime}
 =-\frac{sr_0^2(q-\delta B)^2+\delta r_0B'}{\alpha}}.
\]
This expression is used analytically in checks, rather than obtained from noisy
twice-differenced optimized prices. No second-derivative figure is included.

\begin{proposition}[Conditional signs]
For an initially outside consumer, $p_1^{*\prime}>0$ and $\Pi^{*\prime}>0$.
For an initially inside consumer,
\[
 p_1^{*\prime}>0\ \Longleftrightarrow\ \delta r_0(u+v)>s,\qquad
 \Pi^{*\prime}>0\ \Longleftrightarrow\ \delta r_0v>s(1+\delta u).
\]
Equalities give zero derivatives; reversed inequalities give negative derivatives.
\end{proposition}
\begin{proof}
For $q=1$, the price numerator $s+\delta r_0B$ is positive and $-1/\alpha>0$.
```

Source lines 450–491:

```
The profit numerator is $-s(1-\delta u)-\delta r_0v<0$ because $\delta u<1$;
dividing by $\alpha<0$ makes the derivative positive. Substituting $q=-1$
```

```
gives the two stated thresholds. These statements do not establish a common
monotonicity sign for both histories or a global sign for the price ratios.
\end{proof}

\section{Price ratios: three distinct conditioning statements}
The state-matched ratios are descriptive comparisons:
\[
 R_j=\frac{p_1^*(x_0=j)}{p_2^*(x_1=j)},\qquad
 R_0=\frac{p_1^*(x_0=0)}{p_2^*(x_1=0)}.
\]
They do not necessarily describe one realized consumer path. The acquisition
comparison is instead
\[
 R_{0\to j}=\frac{p_1^*(x_0=0)}{p_2^*(x_1=j)},
\]
conditional on acquisition in period 1. It is not an expectation over histories.
For each definition use its own numerator, denominator and state derivatives:
\[
 \boxed{R'=\frac{p_2^*p_1^{*\prime}-p_1^*p_2^{*\prime}}{(p_2^*)^2}}.
\]
Terminal prices exceed $c$; the benchmark $c=0$ therefore has strictly positive
ratio denominators. For other calibrations a ratio is only defined when its
denominator is nonzero. No expected ratio or ratio of expectations is substituted.

\section{Numerical benchmark and verification}
Every plotted point reoptimizes both terminal branches and period 1:
\[
 \beta=2,\qquad\alpha=-1,\qquad\delta=0.92,\qquad c=0,\qquad
 \gamma\in[-3,3]\ \text{on a 601-point grid}.
\]
The calibration is illustrative, not estimated. SymPy verifies raw-utility
normalization, both branch weights, the profit-derived FOCs, implicit price
derivatives and the envelope identities. Numerical checks compare all 1,202
benchmark state/grid solutions with the handoff's independent reference.
Central differences reoptimize the full model at nearby parameters; an
independent scalar profit optimizer checks optimality. Additional calibrations
check derivatives and an introductory-subsidy case. Numerical evidence is not
a proof of unqualified global comparative-static signs.

\subsection*{Selected optimized values}
```

Source lines 492–494:

```
\begin{center}\small\begin{tabular}{crrrrrr}\toprule
$x_0$ & $\gamma$ & $p_1^*$ & $p_2^*(j)$ & $p_2^*(0)$ & $\Pi^*$ & $\Pi^{*\prime}$\\\midrule
'''
```

Read this block:

- Line 282, `document`: Set this name to a multiline text template; its contents are data, not Python instructions.

Source lines 495–498:

```
for state in ['j','0']:
    for g in [-3,-1,0,1,3]:
        r=evaluate_model(g,state)
        document+=f"${state}$ & {g} & {r['p1']:.3f} & {r['p2j']:.3f} & {r['p20']:.3f} & {r['profit']:.3f} &
 ↪  {r['dprofit']:.3f}"+r'\\'+'\n'
```

Read this block:

- Line 495: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 496: for each entry yielded by `[-3, -1, 0, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 497, `r`: Set this name to the result of calling `evaluate_model(...)`; the arguments and preceding method chain are shown above.
- Line 498: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `evaluate_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 499–502:

```
    document+=r'''\bottomrule\end{tabular}\end{center}
\clearpage
\input{sympy_equations.tex}
'''
```

Read this block:

- Line 499: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 503–516:

```
specs=[
    ('matched_price_ratio_derivative','State-matched price-ratio derivatives',
     r'$R_j=p_1^*(j)/p_2^*(j)$ and $R_0=p_1^*(0)/p_2^*(0)$. Each curve holds its stated histories fixed and uses
      ↪  the analytical quotient rule.'),
    ('induce_harvest_price_ratio_derivative','Induce-then-harvest price-ratio derivative',
     r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$, conditional on a first-period acquisition. This is a path-specific
      ↪  comparison, not an ex-ante expectation.'),
    ('profit_by_gamma','Optimized profit levels',
     r'Expected discounted profit includes both realized continuation histories. The two curves condition on the
      ↪  initial realized state.'),
    ('gamma_profit_by_gamma','Optimized profit derivative',
     r'Total derivatives use the envelope theorem and both continuation-value derivatives. Their signs depend on
      ↪  the initial history and the stated thresholds.'),
    ('optimal_prices_by_gamma','Optimal prices in both periods',
     r'Left: period-1 prices for both initial states. Right: period-2 prices for both realized first-period states.
      ↪  Prices are unconstrained; all shares adjust.'),
    ('gamma_p1_by_gamma','Optimal period-1 price derivative',
     r'Analytical total derivatives, with both continuation branches reoptimized. Initial-state signs are
      ↪  interpreted using the proved conditional statements.'),
]
```

Read this block:

- Line 503, `specs`: Set this name to a list containing 6 displayed entries.

Source lines 517–522:

```
for name,title,caption in specs:
    if not (TEX_OUTPUT/'figures'/f'{name}.pdf').exists():
        raise FileNotFoundError(name)
    document+='\\clearpage\n\\section*{'+title+'}\n'
    document+='\\begin{figure}[htbp]\\centering\n\\includegraphics[width=\\linewidth]{figures/'+name+'.pdf}\n'
    document+='\\caption{'+caption+r' Benchmark: $\beta=2$, $\alpha=-1$, $\delta=0.92$, $c=0$;
 ↪  $\gamma\in[-3,3]$.}'+'\n\\end{figure}\n'
```

Read this block:

- Line 517: for each entry yielded by `specs`, bind/unpack it into `(name, title, caption)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 518: test `not (TEX_OUTPUT / 'figures' / f'{name}.pdf').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 519: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 520: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 521: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 522: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 523–525:

```
523 document+='\\end{document}\n'
524 (TEX_OUTPUT/'comparative_statics_figures.tex').write_text(document)
525 print('Generated verified symbolic equations and appendix with six requested figures.')
```

Read this block:

- Line 523: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 524: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 525: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/supply/unknown_type.py

Earlier theoretical hidden-gamma problem with Bayesian posterior updating after first-period outcomes. This remains available as a separate theory exercise; the new fixed-belief calibration intentionally does not learn.

Module setup and direct-execution blocks

Source lines 2–7:

```
2 """Observed initial history, hidden fixed gamma, Bayesian learning from choices."""
3 from pathlib import Path
4 import argparse
5 from types import SimpleNamespace
6 import hashlib
7 import json
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.

- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 5: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 7: import `json`. An as-name is a convenient local alias, not another library.

Source lines 8–11:

```
8 import sys
9
10 import numpy as np
11 import pandas as pd
```

Read this block:

- Line 8: import `sys`. An as-name is a convenient local alias, not another library.
- Line 10: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 11: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 13–16:

```
13 if __package__:
14     from .pricing_model import expit, scalar_maximum, supply_solution, plotting
15 else:
16     from pricing_model import expit, scalar_maximum, supply_solution, plotting
```

Read this block:

- Line 13: test `__package__`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 14: from `pricing_model`, import `expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `pricing_model`, import `expit, scalar_maximum, supply_solution, plotting` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–24:

```
19 output = Path(__file__).resolve().parents[1] / "results" / "unknown_type"
20 beta = 2.0
21 alpha = -1.0
22 delta = 0.92
23 cost = 0.0
24 run_self_tests = False
```

Read this block:

- Line 19, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 20, `beta`: Set this name to the literal value `2.0`.
- Line 21, `alpha`: Set this name to the negated or sign-adjusted expression `-1.0`.
- Line 22, `delta`: Set this name to the literal value `0.92`.
- Line 23, `cost`: Set this name to the literal value `0.0`.
- Line 24, `run_self_tests`: Set this name to the literal value `False`.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 26–32:

```
26 GAMMAS = np.array([-1.25, 0., 1.25])
27 PRIORS = {
28     "favor_negative": np.array([.5, .25, .25]),
29     "favor_zero": np.array([.25, .5, .25]),
30     "favor_positive": np.array([.25, .25, .5]),
31     "equal": np.full(3, 1/3),
32 }
```

Read this block:

- Line 26, `GAMMAS`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 27, `PRIORS`: Set this name to a dictionary with named entries `favor_negative`, `favor_zero`, `favor_positive`, `equal`.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.

Source lines 247–252:

```
247 if __name__ == "__main__":
248     settings = SimpleNamespace(output=output, beta=beta, alpha=alpha, delta=delta,
249                                cost=cost, self_test=run_self_tests)
250     if len(sys.argv) > 1:
251         settings = read_command_line(settings)
252     run_unknown_type(settings)
```

Read this block:

- Line 247: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 248, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 250: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 251, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 252: call `run_unknown_type` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_unknown_type`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

solve

Original function: `solve`. Source begins at line 35.

Solves a two-period hidden-gamma problem for a known initial history and prior over persistent types. Terminal prices use Bayesian type posteriors after each first-period outcome; the function checks probability mass, value reconstruction and first-order conditions, then returns path rows, policy summaries and residual. This is a learning theory, not fixed beliefs.

Definition and stated intent:

Source lines 35–36:

```
def solve(h0, weights, beta=2., alpha=-1., delta=.92, cost=0., points=81):
    """One policy for a known h0 and a distribution over unobserved gamma."""
```

`def` creates this callable; the parentheses name its inputs (`h0`, `weights`, `beta`, `alpha`, `delta`, `cost`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–37:

```
weights = np.asarray(weights, dtype=float)
```

Read this block:

- Line 37, `weights`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 38–42:

```
if (h0 not in ["j", "0"] or weights.shape != (3,) or not np.isfinite(weights).all()
        or (weights < 0).any() or not np.isclose(weights.sum(), 1.)
        or not np.isfinite([beta, alpha, delta, cost]).all() or alpha >= 0
        or not 0 <= delta <= 1):
    raise ValueError("Need known h0, three normalized type probabilities, alpha<0, delta in [0,1].")
```

Read this block:

- Line 38: test `h0 not in ['j', '0'] or weights.shape != (3,) or (not np.isfinite(weights).al...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 43–44:

```
initial_intercepts = beta + (-1 if h0 == "j" else 1)*GAMMAS
scale = (20+abs(beta)+max(abs(GAMMAS))+abs(alpha*cost))/abs(alpha)
```

Read this block:

- Line 43, `initial_intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 44, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 46–52:

```
def maximize(function, terminal=False):
    lower = cost if terminal else cost-2*scale
    upper = cost+2*scale
    price, value = scalar_maximum(function, (lower, upper), points)
    if min(price-lower, upper-price) < 1e-6*scale:
        raise RuntimeError("Optimum near numerical bound; enlarge search domain.")
    return price, value
```

Read this block:

- Line 46: define nested callable `maximize`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 54–74:

```
def terminal_policy(first):
    # The firm observes h1, not gamma. Bayes' rule weights each possible type.
    inside = expit(initial_intercepts+alpha*first)
    outside = expit(-initial_intercepts-alpha*first)
    policies = []
    for h1, likelihood in [("j", inside), ("0", outside)]:
        joint = weights*likelihood
        probability = float(joint.sum())
        posterior = joint/probability
        intercepts = beta+(-1 if h1 == "j" else 1)*GAMMAS

        def expected_terminal_profit(price):
            shares = expit(intercepts+alpha*np.asarray(price)[..., None])
            return (np.asarray(price)-cost)*(shares @ posterior)

        price, value = maximize(expected_terminal_profit, terminal=True)
        share = expit(intercepts+alpha*price)
        foc = float(posterior @ (share+alpha*(price-cost)*share*(1-share)))
        policies.append(dict(h1=h1, p2=price, value=value,
                             probability=probability, posterior=posterior, foc=foc))
    return policies
```

Read this block:

- Line 54: define nested callable `terminal_policy`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 76–80:

```
def expected_profit(first):
    # Optimize the expectation over gamma, rather than prices for each gamma.
    purchase_probability = float(weights @ expit(initial_intercepts+alpha*first))
    continuation = sum(p["probability"]*p["value"] for p in terminal_policy(first))
    return (first-cost)*purchase_probability+delta*continuation
```

Read this block:

- Line 76: define nested callable `expected_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 82–84:

```
p1, optimal_profit = maximize(expected_profit)
terminal = terminal_policy(p1)
rows = []
```

Read this block:

- Line 82, `(p1, optimal_profit)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 83, `terminal`: Set this name to the result of calling `terminal_policy(...)`; the arguments and preceding method chain are shown above.
- Line 84, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `maximize`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `terminal_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 85–98:

```
85 for i, gamma in enumerate(GAMMAS):
86     s1 = float(expit(initial_intercepts[i]+alpha*p1))
87     for branch in terminal:
88         h1, p2 = branch["h1"], branch["p2"]
89         probability = s1 if h1 == "j" else float(expit(-initial_intercepts[i]-alpha*p1))
90         s2 = float(expit(beta+(-1 if h1 == "j" else 1)*gamma+alpha*p2))
91         row = dict(case="unknown_type", h0=h0, gamma=gamma, h1=h1,
92                    prior_type=weights[i], transition_probability=probability,
93                    path_probability=weights[i]*probability, p1=p1, p2=p2,
94                    second_purchase_probability=s2,
95                    conditional_profit=(p1-cost)*(h1 == "j")+delta*(p2-cost)*s2)
96         for k, name in enumerate(["negative", "zero", "positive"]):
97             row[f"posterior_{name}"] = branch["posterior"][k]
98         rows.append(row)
```

Read this block:

- Line 85: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 86, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 87: for each entry yielded by `terminal`, bind/unpack it into `branch` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 88, `(h1, p2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 89, `probability`: Set this name to the true-branch value if `h1 == 'j'` holds, otherwise the else-branch value.
- Line 90, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 91, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 96: for each entry yielded by `enumerate(['negative', 'zero', 'positive'])`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 97, `row[f'posterior_{name}']`: Store the entry/slice selected by `branch['posterior'][k]`; brackets select from the object to their left in the selected entry/column.
- Line 98: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 100–101:

```
probabilities = np.array([r["path_probability"] for r in rows])
reconstructed = float(probabilities @ [r["conditional_profit"] for r in rows])
```

Read this block:

- Line 100, `probabilities`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 101, `reconstructed`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 102–102:

```
assert np.isclose(probabilities.sum(), 1.)
```

Read this block:

- Line 102: require `np.isclose(probabilities.sum(), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 103–103:

```
assert np.isclose(reconstructed, optimal_profit, atol=1e-8)
```

Read this block:

- Line 103: require `np.isclose(reconstructed, optimal_profit, atol=1e-08)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 104–104:

```
assert np.allclose(sum(p["probability"]*p["posterior"] for p in terminal), weights)
```

Read this block:

- Line 104: require `np.allclose(sum((p['probability'] * p['posterior'] for p in terminal)), weights)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 105–107:

```
for branch in terminal:
    joint = [r["path_probability"] for r in rows if r["h1"] == branch["h1"]]
    assert np.allclose(branch["posterior"]*branch["probability"], joint, atol=1e-10)
```

Read this block:

- Line 105: for each entry yielded by `terminal`, bind/unpack it into `branch` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 106, `joint`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 107: require `np.allclose(branch['posterior'] * branch['probability'], joint, atol=1e-10)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 110–115:

```
shares = expit(initial_intercepts+alpha*p1)
derivative = alpha*shares*(1-shares)
future = [expit(beta+q*GAMMAS+alpha*p["p2"])*(p["p2"]-cost)
          for q, p in zip([-1, 1], terminal)]
foc1 = float(weights @ (shares+derivative*(p1-cost+delta*(future[0]-future[1]))))
residual = max(abs(foc1), *(abs(p["foc"]) for p in terminal))
```

Read this block:

- Line 110, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 111, `derivative`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 112, `future`: Set this name to a list computed by the displayed for-expression.
- Line 114, `foc1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 115, `residual`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 116–117:

```
if residual > 1e-6:
    raise AssertionError(f"FOC residual {residual}")
```

Read this block:

- Line 116: test `residual > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 117: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 118–121:

```
summary = dict(case="unknown_type", h0=h0, p1=p1,
               p2_after_inside=terminal[0]["p2"], p2_after_outside=terminal[1]["p2"],
               optimal_profit=optimal_profit, first_purchase_probability=float(weights @ shares),
               prior_negative=weights[0], prior_zero=weights[1], prior_positive=weights[2])
```

Read this block:

- Line 118, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 122–122:

```
return rows, [summary], residual
```

Read this block:

- Line 122: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

solve.maximize

Original function: `solve.maximize`. Source begins at line 46.

Chooses bounds appropriate to an initial or terminal price and delegates grid/refinement maximization. A boundary optimum raises rather than being silently accepted.

Definition and stated intent:

Source lines 46–46:

```
46 def maximize(function, terminal=False):
```

`def` creates this callable; the parentheses name its inputs (`function`, `terminal`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–49:

```
47 lower = cost if terminal else cost-2*scale
48 upper = cost+2*scale
49 price, value = scalar_maximum(function, (lower, upper), points)
```

Read this block:

- Line 47, `lower`: Set this name to the true-branch value if `terminal` holds, otherwise the else-branch value.
- Line 48, `upper`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 49, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 50–51:

```
50 if min(price-lower, upper-price) < 1e-6*scale:
51     raise RuntimeError("Optimum near numerical bound; enlarge search domain.")
```

Read this block:

- Line 50: test `min(price - lower, upper - price) < 1e-06 * scale`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 52–52:

```
52 return price, value
```

Read this block:

- Line 52: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

solve.terminal policy

Original function: `solve.terminal_policy`. Source begins at line 54.

Updates hidden-type weights by Bayes' rule after each possible first-period purchase outcome and optimizes one terminal price per history. Consumers retain type; this older theory learns, unlike the current fixed-belief auxiliary exercise.

Definition and stated intent:

Source lines 54–55:

```
54 def terminal_policy(first):
55     # The firm observes h1, not gamma. Bayes' rule weights each possible type.
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 56–58:

```
56 inside = expit(initial_intercepts+alpha*first)
57 outside = expit(-initial_intercepts-alpha*first)
58 policies = []
```

Read this block:

- Line 56, `inside`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 57, `outside`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 58, `policies`: Set this name to the list `[]`.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 59–73:

```
59 for h1, likelihood in [("j", inside), ("0", outside)]:
60     joint = weights*likelihood
61     probability = float(joint.sum())
62     posterior = joint/probability
63     intercepts = beta+(-1 if h1 == "j" else 1)*GAMMAS
64 
65     def expected_terminal_profit(price):
66         shares = expit(intercepts+alpha*np.asarray(price)[..., None])
67         return (np.asarray(price)-cost)*(shares @ posterior)
68 
69     price, value = maximize(expected_terminal_profit, terminal=True)
70     share = expit(intercepts+alpha*price)
71     foc = float(posterior @ (share+alpha*(price-cost)*share*(1-share)))
72     policies.append(dict(h1=h1, p2=price, value=value,
73                          probability=probability, posterior=posterior, foc=foc))
```

Read this block:

- Line 59: for each entry yielded by `[('j', inside), ('0', outside)]`, bind/unpack it into `(h1, likelihood)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 61, `probability`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 62, `posterior`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 63, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 65: define nested callable `expected_terminal_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 69, `(price, value)`: Unpack the result of calling `maximize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 70, `share`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 71, `foc`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 72: call `policies.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `maximize`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 74–74:

```
74 return policies
```

Read this block:

- Line 74: return the object currently named `policies`; plain assignment does not copy it. This ends the call.

solve.terminal policy.expected terminal profit

Original function: `solve.terminal_policy.expected_terminal_profit`. Source begins at line 65.

Computes terminal expected margin using the branch posterior over types. One price applies to all types in that observed history; optimizing separately by true gamma would give the firm information it lacks.

Definition and stated intent:

Source lines 65–65:

```
65 def expected_terminal_profit(price):
```

`def` creates this callable; the parentheses name its inputs (`price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 66–66:

```
66 shares = expit(intercepts+alpha*np.asarray(price)[..., None])
```

Read this block:

- Line 66, `shares`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 67–67:

```
67 return (np.asarray(price)-cost)*(shares @ posterior)
```

Read this block:

- Line 67: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

solve.expected_profit

Original function: `solve.expected_profit`. Source begins at line 76.

Combines initial expected margin and continuation values from the optimized history-specific policies. The first price affects both state probabilities and inferred type composition.

Definition and stated intent:

Source lines 76–77:

```
76 def expected_profit(first):
77     # Optimize the expectation over gamma, rather than prices for each gamma.
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–79:

```
78 purchase_probability = float(weights @ expit(initial_intercepts+alpha*first))
79 continuation = sum(p["probability"]*p["value"] for p in terminal_policy(first))
```

Read this block:

- Line 78, `purchase_probability`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 79, `continuation`: Set this name to the result of calling `sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `terminal_policy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 80–80:

```
80 return (first-cost)*purchase_probability+delta*continuation
```

Read this block:

- Line 80: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

figures

Original function: `figures`. Source begins at line 127.

Draws one theoretical page per prior showing hidden gamma weights, optimal history-dependent prices and expected profit. All values are generated from model parameters rather than purchase data.

Definition and stated intent:

Source lines 127–128:

```
127 def figures(paths, summary, output):
128     """One readable page per prior: distribution, policy, and expected profit."""
```

`def` creates this callable; the parentheses name its inputs (`paths`, `summary`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 129–134:

```
129 plt = plotting()
130 colors = ["#3366a6", "#78909c", "#d48435"]
131 titles = {"favor_negative": "Pr(gamma = -1.25) = 0.50",
132           "favor_zero": "Pr(gamma = 0) = 0.50",
133           "favor_positive": "Pr(gamma = +1.25) = 0.50",
134           "equal": "Equal probability on all three gamma values"}
```

Read this block:

- Line 129, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 130, `colors`: Set this name to the list `['#3366a6', '#78909c', '#d48435']`.
- Line 131, `titles`: Set this name to a dictionary with named entries `favor_negative`, `favor_zero`, `favor_positive`, `equal`.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 135–169:

```
135 for scenario, weights in PRIORS.items():
136     data = summary[summary.scenario == scenario].set_index("h0").loc[["0", "j"]]
137     with plt.rc_context({"font.size": 13, "axes.titlesize": 15, "axes.labelsize": 13}):
138         fig = plt.figure(figsize=(13, 7), layout="constrained")
139         grid = fig.add_gridspec(2, 2, width_ratios=[1, 1.7])
140         prior_ax = fig.add_subplot(grid[:, 0])
141         price_ax = fig.add_subplot(grid[0, 1])
142         profit_ax = fig.add_subplot(grid[1, 1])
143         bars = prior_ax.bar(["-1.25", "0", "+1.25"], weights, color="#8267a0", width=.6)
144         prior_ax.bar_label(bars, labels=[f"{p:.0%}" if scenario != "equal" else "1/3" for p in weights], padding=6)
145         prior_ax.set(xlabel="Possible gamma (unobserved)", ylabel="Firm's prior probability",
146                      title="Uncertainty faced by the firm", ylim=(0, .65))
147         x = np.arange(2)
148         width = .23
149         for offset, key, label, color in zip([-1, 0, 1],
150                 ["p1", "p2_after_inside", "p2_after_outside"],
151                 ["Period 1", "Period 2: bought in period 1", "Period 2: did not buy in period 1"], colors):
152             bars = price_ax.bar(x+offset*width, data[key], width, color=color, label=label)
153             price_ax.bar_label(bars, fmt="%.3f", padding=3, fontsize=11)
154         price_ax.set(title="Optimal prices under gamma uncertainty", ylabel="Price")
155         price_ax.set_ylim(min(0, float(data[["p1", "p2_after_inside", "p2_after_outside"]].min().min())*1.2),
156                           float(data[["p1", "p2_after_inside", "p2_after_outside"]].max().max())*1.6)
157         price_ax.legend(loc="upper left", bbox_to_anchor=(0, 1.01), fontsize=9, frameon=False)
158         bars = profit_ax.bar(x, data.optimal_profit, color="#467d5d", width=.5)
```

```
159         profit_ax.bar_label(bars, fmt="%.3f", padding=4)
160         profit_ax.set(title="Maximized expected two-period profit", ylabel="Discounted expected profit",
161                       ylim=(min(0, data.optimal_profit.min()*1.2), data.optimal_profit.max()*1.2))
162         for ax in [price_ax, profit_ax]:
163             ax.set_xticks(x, ["Known h0: outside", "Known h0: inside"])
164             ax.spines[["top", "right"]].set_visible(False)
165         prior_ax.spines[["top", "right"]].set_visible(False)
166         fig.suptitle(titles[scenario], fontsize=18)
167         for extension in ["png", "pdf"]:
168             fig.savefig(output / f"optimal_policy_{scenario}.{extension}", dpi=180)
169         plt.close(fig)
```

Read this block:

- Line 135: for each entry yielded by `PRIORS.items()`, bind/unpack it into `(scenario, weights)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 136, `data`: Set this name to the entry/slice selected by `summary[summary.scenario == scenario].set_index('h0').loc...`; brackets select from the object to their left.
- Line 137: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 138, `fig`: Set this name to the result of calling `plt.figure(...)`; the arguments and preceding method chain are shown above.
- Line 139, `grid`: Set this name to the result of calling `fig.add_gridspec(...)`; the arguments and preceding method chain are shown above.
- Line 140, `prior_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 141, `price_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 142, `profit_ax`: Set this name to the result of calling `fig.add_subplot(...)`; the arguments and preceding method chain are shown above.
- Line 143, `bars`: Set this name to the result of calling `prior_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 144: call `prior_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 145: call `prior_ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 147, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 148, `width`: Set this name to the literal value `0.23`.
- Line 149: for each entry yielded by `zip([-1, 0, 1], ['p1', 'p2_after_inside', 'p2_after_outside'],...`, bind/unpack it into `(offset, key, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 152, `bars`: Set this name to the result of calling `price_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 153: call `price_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 154: call `price_ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 155: call `price_ax.set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 157: call `price_ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 158, `bars`: Set this name to the result of calling `profit_ax.bar(...)`; the arguments and preceding method chain are shown above.
- Line 159: call `profit_ax.bar_label` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 160: call `profit_ax.set` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.
- Line 162: for each entry yielded by `[price_ax, profit_ax]`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 163: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 164: call `ax.spines[['top', 'right']].set_visible` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `prior_ax.spines[['top', 'right']].set_visible` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 166: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 167: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 168: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 169: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 174.

Verifies degenerate-type nesting, common offers across hidden types, Bayesian belief consistency, probability mass, revenue reconstruction and first-order conditions. These checks distinguish this learning theory from fixed-belief pricing.

Definition and stated intent:

Source lines 174–174:

```
174 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 175–190:

```
for h0 in ["0", "j"]:
    for i, gamma in enumerate(GAMMAS):
        _, summary, _ = solve(h0, np.eye(3)[i])
        reference = supply_solution(gamma, float(h0 == "j"))
        actual = summary[0]
        assert np.isclose(actual["optimal_profit"], reference["known_value"], atol=1e-7)
        assert np.isclose(actual["p1"], reference["known_p1_inside" if h0 == "j" else "known_p1_outside"],
         ↪  atol=1e-6)
    for weights in PRIORS.values():
        rows, summary, _ = solve(h0, weights)
        # All true types face exactly the same p1 and same p2 at a given h1.
        assert np.ptp([r["p1"] for r in rows]) == 0
        for h1 in ["0", "j"]:
            assert np.ptp([r["p2"] for r in rows if r["h1"] == h1]) == 0
        # Revealing gamma can only increase the optimized expected profit.
        known = sum(w*supply_solution(g, float(h0 == "j"))["known_value"] for w, g in zip(weights, GAMMAS))
        assert known >= summary[0]["optimal_profit"]-1e-7
```

Read this block:

- Line 175: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 176: for each entry yielded by `enumerate(GAMMAS)`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 177, `(_, summary, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 178, `reference`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 179, `actual`: Set this name to the entry/slice selected by `summary[0]`; brackets select from the object to their left.
- Line 180: require `np.isclose(actual['optimal_profit'], reference['known_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 181: require `np.isclose(actual['p1'], reference['known_p1_inside' if h0 == 'j' else 'known_p1_o...` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 182: for each entry yielded by `PRIORS.values()`, bind/unpack it into `weights` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 183, `(rows, summary, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 185: require `np.ptp([r['p1'] for r in rows]) == 0` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 186: for each entry yielded by `['0', 'j']`, bind/unpack it into `h1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 187: require `np.ptp([r['p2'] for r in rows if r['h1'] == h1]) == 0` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 189, `known`: Set this name to the result of calling `sum(...)`; the arguments and preceding method chain are shown above.
- Line 190: require `known >= summary[0]['optimal_profit'] - 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 191–191:

```
191 print("Passed: common policy across gamma, degenerate-type benchmark, Bayesian beliefs, probability mass, profit
    ↪  reconstruction and FOCs.")
```

Read this block:

- Line 191: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 196.

Parses theoretical parameters, priors, output and self-test options. It does not recover household types or rerun demand estimation.

Definition and stated intent:

Source lines 196–196:

```
196 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 197–202:

```
197 parser = argparse.ArgumentParser(description=__doc__)
198 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "outputs")
199 parser.add_argument("--beta", type=float, default=2.)
200 parser.add_argument("--alpha", type=float, default=-1.)
201 parser.add_argument("--delta", type=float, default=.92)
202 parser.add_argument("--cost", type=float, default=0.)
```

Read this block:

- Line 197, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.

- Line 198: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 199: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 200: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 201: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 202: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 203–204:

```
203 parser.add_argument("--self-test", action="store_true")
204 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 203: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 204: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 205–205:

```
205 return parser.parse_args()
```

Read this block:

- Line 205: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run unknown type

Original function: `run_unknown_type`. Source begins at line 210.

Solves the configured theoretical hidden-gamma scenarios and initial states, records results and draws the policy figures. The main empirical pipeline does not call this Bayesian-learning entry point.

Definition and stated intent:

Source lines 210–210:

```
210 def run_unknown_type(args):
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 211–213:

```
211 if args.self_test:
212     self_test()
213     return
```

Read this block:

- Line 211: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 212: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 213: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 214–214:

```
214 paths, summaries, residuals = [], [], []
```

Read this block:

- Line 214, `(paths, summaries, residuals)`: Unpack a tuple containing 3 displayed entries into these targets in order.

Source lines 215–227:

```
215 for scenario, weights in PRIORS.items():
216     for h0 in ["0", "j"]:
217         rows, summary, residual = solve(h0, weights, args.beta, args.alpha, args.delta, args.cost)
218         # Independently repeat with a denser price search.
219         _, dense, _ = solve(h0, weights, args.beta, args.alpha, args.delta, args.cost, points=161)
220         if not np.allclose([r["optimal_profit"] for r in summary], [r["optimal_profit"] for r in dense], atol=1e-7,
            ↪ rtol=0):
221             raise AssertionError("Denser search disagrees.")
222         for row in rows+summary:
223             row["scenario"] = scenario
224         paths.extend(rows)
225         summaries.extend(summary)
226         residuals.append(residual)
227         print(f"Solved {scenario}, observed h0={h0}", flush=True)
```

Read this block:

- Line 215: for each entry yielded by `PRIORS.items()`, bind/unpack it into `(scenario, weights)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 216: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 217, `(rows, summary, residual)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 219, `(_, dense, _)`: Unpack the result of calling `solve(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 220: test `not np.allclose([r['optimal_profit'] for r in summary], [r['optimal_profit'] ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 221: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 222: for each entry yielded by `rows + summary`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 223, `row['scenario']`: Store the object currently named `scenario`; plain assignment does not copy it in the selected entry/column.

- Line 224: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 225: call `summaries.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 226: call `residuals.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 227: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `solve`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 228–233:

```
228 args.output.mkdir(parents=True, exist_ok=True)
229 histories, expected = pd.DataFrame(paths), pd.DataFrame(summaries)
230 histories.to_csv(args.output / "supply_history_outcomes.csv", index=False)
231 expected.to_csv(args.output / "supply_expected_outcomes.csv", index=False)
232 expected.to_csv(args.output / "optimal_policies.csv", index=False)
233 figures(histories, expected, args.output)
```

Read this block:

- Line 228: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 229, `(histories, expected)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 230: call `histories.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 231: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 232: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 233: call `figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 234–244:

```
234 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()},
235                 information="h0 observed, gamma unobserved and fixed; h1 observed before p2",
236                 gamma_values=GAMMAS.tolist(), priors={k: v.tolist() for k, v in PRIORS.items()},
237                 maximum_foc_residual=max(residuals), denser_search_verified=True,
```

```
238                script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
239               solver_sha256=hashlib.sha256(Path(__file__).with_name("pricing_model.py").read_bytes()).hexdigest())
240 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
241 (args.output / "run_notes.txt").write_text(
242     "Known h0, unobserved gamma. One price policy per prior and observed history.\n"
243     + "\n".join(f"{key}: {value}" for key, value in manifest.items()) + "\n")
244 print(f"Saved figures and tables to {args.output}")
```

Read this block:

- Line 234, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 240: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 241: call `(args.output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 244: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Tests and verification: worked calling examples

data_code/test_summ_stats_memory.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Synthetic checks for sparse summaries and exact results across batch sizes."""
import contextlib
import io
from pathlib import Path
import tempfile
from types import SimpleNamespace
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
import unittest

import pandas as pd
import polars as pl

import summ_stats as summary
```

Read this block:

- Line 7: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12: import `summ_stats as summary`. An as-name is a convenient local alias, not another library.

Source lines 15–15:

```
15 class MemorySummaryChecks(unittest.TestCase):
```

Read this block:

- Line 15: define class `MemorySummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 141–142:

```
141 if __name__ == "__main__":
142     unittest.main()
```

Read this block:

- Line 141: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 142: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

MemorySummaryChecks.test single person records precede repeat week rule

Original function: `MemorySummaryChecks.test_single_person_records_precede_repeat_week_rule`. Source begins at line 16.

Checks record-level size-one filtering occurs before requiring repeated weeks; otherwise a household can qualify using excluded purchases. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 16–16:

```
16 def test_single_person_records_precede_repeat_week_rule(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–25:

```
17 source = pl.DataFrame({
18     "household_code": [1, 1, 2, 2, 3, 3, 4, 4],
19     "purchase_date": ["2010-01-02", "2010-01-09"] * 4,
20     "household_size": ["1", "2", "1", "1", None, "1", "bad", "2"],
21 }).lazy().with_columns(pl.col("purchase_date").str.to_date())
22 restricted = summary.single_person_purchases(source)
23 self.assertEqual(restricted.collect().height, 4)
24 repeated = summary.repeated_yogurt_households(restricted).collect()
25 self.assertEqual(repeated["household_code"].unique().to_list(), [2])
```

Read this block:

- Line 17, `source`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 22, `restricted`: Set this name to the result of calling `summary.single_person_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `repeated`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 25: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 26–27:

```
26 with self.assertRaisesRegex(ValueError, "require household_size"):
27     summary.single_person_purchases(source.drop("household_size"))
```

Read this block:

- Line 26: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 27: call `summary.single_person_purchases` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test batches preserve global statistics

Original function: `MemorySummaryChecks.test_batches_preserve_global_statistics`. Source begins at line 29.

What this test establishes. Runs a small complete summary pipeline with one and three partitions, sorts outputs, and compares their actual contents. It verifies partitioning changes execution/storage rather than statistical results and preserves original metadata in the prepared sample.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 29–29:

```
29 def test_batches_preserve_global_statistics(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 30–71:

```
30 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
31     root = Path(directory)
32     source = pl.DataFrame({
33         "household_code": [1, 1, 1, 1, 2, 2, 3], "panel_year": [2010] * 7,
34         "purchase_date": ["2010-01-02", "2010-01-09", "2010-01-09", "2010-01-23", "2010-01-02", "2010-01-09",
            ↪  "2010-01-02"],
35         "trip_code_uc": [1, 2, 2, 3, 4, 5, 6], "upc": [1, 1, 2, 3, 2, 3, 1],
```

```
        "quantity": [2, 1, 1, 3, 1, 2, 1], "race": [1, 1, 1, 1, 2, 2, 3],
        "hispanic_origin": [2, 2, 2, 2, 1, 1, 2], "household_income": [5, 5, 5, 5, 8, 8, 9],
        "female_head_age": [3, 3, 3, 3, 0, 0, 4], "male_head_age": [4, 4, 4, 4, 2, 2, 5],
        "household_size": [1] * 7, "store_code_uc": [100] * 7, "price": [1.] * 7,
        "original_metadata": ["preserved"] * 7,
    })
    for name in ["merged_panel", "yogurt_estimation_sample"]:
        source.write_parquet(root / f"{name}.parquet")
    reference = root / "reference.csv"
    reference.write_text("upc,flavor_descr,module\n001,vanilla,3612\n002,strawberry,3612\n003,plain,3612\n
↪   004,unused,3612\n")
    for batches in [1, 3]:
        out, work = root / f"out{batches}", root / f"work{batches}"
        out.mkdir()
        work.mkdir()
        args = SimpleNamespace(data_dir=root, flavors=reference, products_extra=root / "absent",
                               first_year=2006, last_year=2015, household_batches=batches, no_figures=True)
        summary.run(args, out, work)
    for name, keys in [
        ("full_panel_households", ["household_code"]), ("matched_yogurt_households", ["household_code"]),
        ("yogurt_household_flavors", ["household_code"]), ("yogurt_dynamics", ["metric"]),
        ("full_panel_overview", ["metric"]), ("matched_yogurt_overview", ["metric"]),
        ("yogurt_flavor_shares", ["flavor"]), ("full_panel_year_coverage", ["panel_year"]),
    ]:
        left = pd.read_csv(root / "out1" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        right = pd.read_csv(root / "out3" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        pd.testing.assert_frame_equal(left, right)
    prepared = pl.read_parquet(root / "out3/yogurt_estimation_sample.parquet")
    self.assertEqual(prepared.height, 6)
    self.assertTrue(set(source.columns).issubset(prepared.columns))
    self.assertEqual(set(prepared["household_code"].to_list()), {1, 2})
    self.assertEqual(prepared["original_metadata"].unique().to_list(), ["preserved"])
    self.assertEqual(set(prepared["upc"].to_list()), {"1", "2", "3"})
    self.assertEqual(prepared["flavor"].null_count(), 0)
    hh = pd.read_csv(root / "out3/yogurt_household_flavors.csv").set_index("household_code")
    self.assertEqual(set(hh.index), {1, 2})
    self.assertAlmostEqual(hh.loc[1, "flavor_HHI"], 19 / 49)
```

Source lines 72–97:

```
self.assertAlmostEqual(hh.loc[2, "flavor_HHI"], 5 / 9)
metrics = pd.read_csv(root / "out3/yogurt_dynamics.csv").set_index("metric")["value"]
self.assertEqual(metrics["Eligible adjacent single-flavor week pairs"], 1)
self.assertEqual(metrics["Switch rate in eligible pairs (%)"], 100)
self.assertEqual(metrics["Median gap between purchase weeks"], 1)
# Price filtering precedes the two-week household rule.
unpriced = source.with_columns(pl.when(
    ((pl.col("household_code") == 2) & (pl.col("purchase_date") == "2010-01-09"))
    | ((pl.col("household_code") == 1) & (pl.col("purchase_date") == "2010-01-09") & (pl.col("upc") == 1))
    ).then(None).otherwise(pl.col("price")).alias("price"))
unpriced.write_parquet(root / "yogurt_estimation_sample.parquet")
out, work = root / "filtered", root / "filtered_work"
out.mkdir()
work.mkdir()
args.household_batches = 2
summary.run(args, out, work)
filtered = pl.read_parquet(out / "yogurt_estimation_sample.parquet")
self.assertEqual(filtered.height, 3)
self.assertEqual(filtered["household_code"].unique().to_list(), [1])
self.assertTrue(filtered["price"].is_not_null().all())
self.assertEqual(pl.read_csv(out / "yogurt_price_coverage.csv")["purchase_lines_without_price_coverage"][0], 2)
```

```
# Sparse quantity output has one row per observed flavor/week, not
# one column for every flavor in the full reference.
quantities = pl.read_csv(root / "out3/yogurt_weekly_flavor_quantities.csv")
self.assertEqual(quantities.height, 6)
self.assertEqual(set(quantities.columns), {"household_code", "week", "flavor", "quantity"})
```

Read this block:

- Line 30: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 32, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 42: for each entry yielded by `['merged_panel', 'yogurt_estimation_sample']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44, `reference`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45: call `reference.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: for each entry yielded by `[1, 3]`, bind/unpack it into `batches` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 48: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 50, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 52: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: for each entry yielded by `[('full_panel_households', ['household_code']), ('matched_yogu...`, bind/unpack it into `(name, keys)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 59, `left`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 60, `right`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 61: call `pd.testing.assert_frame_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 62, `prepared`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 63: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 67: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 69, `hh`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 70: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 71: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73, `metrics`: Set this name to the entry/slice selected by `pd.read_csv(root / 'out3/yogurt_dynamics.csv').set_index(...`; brackets select from the object to their left.
- Line 74: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 75: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78, `unpriced`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 82: call `unpriced.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 84: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86, `args.household_batches`: Set this name to the literal value `2`.
- Line 87: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88, `filtered`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95, `quantities`: Set this name to the result of calling `pl.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 96: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.read_parquet`: Read Parquet into an eager table, rather than retaining only a deferred plan.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.assert_frame_equal`: Compare pandas tables including values, labels, and dtype conventions rather than only row counts.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test demographics match previous fieldwise sort

Original function: `MemorySummaryChecks.test_demographics_match_previous_fieldwise_sort`. Source begins at line 99.

What this test establishes. Compares the optimized shared demographic-history sort to separate field-by-field selection on a synthetic example. This checks that a memory/runtime improvement preserves earliest-nonmissing semantics.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 99–101:

```
99 def test_demographics_match_previous_fieldwise_sort(self):
100     # Unsorted histories, missing fields, conflicting same-date values and
101     # repeated purchase lines must preserve the earliest nonmissing values.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 102–116:

```
source = pl.DataFrame({
    "household_code": [1, 2, 1, 1, 1, 2],
    "panel_year": [2011, 2010, 2010, 2010, 2010, 2011],
    "purchase_date": ["2011-01-01", "2010-01-02", "2010-01-02", "2010-01-02", "2010-01-02", "2011-01-01"],
    "trip_code_uc": [1, 2, 3, 3, 3, 4], "upc": [1, 2, 3, 3, 3, 4],
    "quantity": [1.] * 6,
}).with_columns(pl.col("purchase_date").str.to_date())
fields = ["race", "income", "hispanic_origin", "household_size", "head_age",
          "female_head_age", "male_head_age", "dma_cd"]
source = source.with_columns([pl.Series(c, ["9", None, None, "2", "3", "7"]) for c in fields])
expected = source.lazy().group_by("household_code").agg([
    pl.col(c).sort_by(["panel_year", "purchase_date"]).drop_nulls().first().alias(c)
    for c in fields
]).collect().sort("household_code")
actual = summary.household_summary(source.lazy()).collect().sort("household_code")
```

Read this block:

- Line 102, `source`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 109, `fields`: Set this name to a list containing 8 displayed entries.
- Line 111, `source`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 112, `expected`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 116, `actual`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 117–118:

```
self.assertEqual(actual.select(["household_code"] + fields).to_dicts(), expected.to_dicts())
self.assertEqual(actual["purchase_lines"].to_list(), [4, 2])
```

Read this block:

- Line 117: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 119–126:

```
with tempfile.TemporaryDirectory() as directory:
    parts = summary.partition_purchases(source.lazy(), Path(directory), "check", 16)
    restored = pl.concat(parts).collect()
    self.assertEqual(restored.height, source.height)
    counts = restored.group_by("household_code").len().sort("household_code")
    self.assertEqual(counts["len"].to_list(), [4, 2])
    batched = pl.concat([summary.household_summary(part) for part in parts]).collect().sort("household_code")
    self.assertEqual(batched.to_dicts(), actual.to_dicts())
```

Read this block:

- Line 119: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 120, `parts`: Set this name to the result of calling `summary.partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 121, `restored`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 122: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `counts`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125, `batched`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 126: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test recovery preserves existing labels and uses earliest year

Original function: `MemorySummaryChecks.test_recovery_preserves_existing_labels_and_uses_earliest_year`. Source begins at line 128.

What this test establishes. Constructs reference and annual metadata cases to check that existing labels are retained and absent UPCs take earliest usable metadata. It verifies a substantive recovery precedence rule.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 128–128:

```
128 def test_recovery_preserves_existing_labels_and_uses_earliest_year(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 129–138:

```
129 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
130     root = Path(directory)
131     ref, extra, output = root / "ref.csv", root / "extra.parquet", root / "result.csv"
132     ref.write_text("upc,flavor_descr\n001,vanilla\n002,NA\n")
133     pl.DataFrame({"upc": [1, 2, 3, 3, 4], "flavor_descr": ["plain", "berry", "cherry", "strawberry", None],
134                   "panel_year": [2006, 2006, 2007, 2006, 2006]}).write_parquet(extra)
135     summary.recover_flavor_reference(ref, extra, output)
136     result = summary.flavor_reference(output, detailed=True).sort("upc")
137     self.assertEqual(result.to_dict(as_series=False),
138                      {"upc": ["1", "2", "3"], "flavor": ["vanilla", "unknown", "strawberry"]})
```

Read this block:

- Line 129: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 130, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 131, `(ref, extra, output)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 132: call `ref.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `summary.recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136, `result`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 137: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Small checks that need no Nielsen data: python3 -m unittest -v test_est.py."""
import unittest
import tempfile
from pathlib import Path
from types import SimpleNamespace
import polars as pl
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 7–10:

```
import numpy as np
import pandas as pd
from scipy.special import logsumexp
from est import read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws,
    parameter_names, demand_objective_gradient
```

Read this block:

- Line 7: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 8: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 9: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `est`, import `read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws, parameter_names, demand_objective_gradient` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–13:

```
class EstimationChecks(unittest.TestCase):
```

Read this block:

- Line 13: define class `EstimationChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 270–271:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 270: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 271: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EstimationChecks.frame

Original function: `EstimationChecks.frame`. Source begins at line 14.

This fixture method returns a tiny three-week pandas table. It includes a two-week gap, mixed counts, all three prices, and outside counts so history, menu, and likelihood tests can reuse the same explicit example. `self.frame()` returns a fresh table each time, letting one test modify it without contaminating another. A shared global mutable table would make results depend on test order.

Definition and stated intent:

Source lines 14–14:

```
14 def frame(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–20:

```
15 return pd.DataFrame({
16     "household_code": [10, 10, 10],
17     "week": pd.to_datetime(["2010-01-02", "2010-01-09", "2010-01-23"]),
18     "other": [2., 0., 1.], "berry": [0., 2., 1.], "plain": [0., 0., 0.],
19     "price_other": [1., 1.2, 1.1], "price_berry": [1.2, 1., 1.3],
20     "price_plain": [1.5, 1.4, 1.3], "outside": [5., 5., 5.]})
```

Read this block:

- Line 15: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.

EstimationChecks.test history excludes current counts and accumulates all occasions

Original function: `EstimationChecks.test_history_excludes_current_counts_and_accumulates_all_occasions`. Source begins at line 22.

What this test establishes. Checks pre-choice timing: changing a current basket must not rewrite the stock already available for that choice. It compares explicit stock vectors, not just output dimensions.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 22–22:

```
22 def test_history_excludes_current_counts_and_accumulates_all_occasions(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 23–28:

```
23 frame = add_stocks(self.frame())
24 stock = frame[["stock_other", "stock_berry", "stock_plain"]]
25 np.testing.assert_allclose(stock.iloc[0], [0., 0., 0.])
```

```
26 np.testing.assert_allclose(stock.iloc[1], [1., 0., 0.])
27 np.testing.assert_allclose(stock.iloc[2], [1., 1., 0.])
28 changed = self.frame()
```

Read this block:

- Line 23, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 24, `stock`: Set this name to the entry/slice selected by `frame[['stock_other', 'stock_berry', 'stock_plain']]`; brackets select from the object to their left.
- Line 25: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 26: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 27: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28, `changed`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 29–31:

```
29 changed.loc[2, "plain"] = 100
30 np.testing.assert_allclose(add_stocks(changed).filter(like="stock_"),
31                            frame.filter(like="stock_"))
```

Read this block:

- Line 29, `changed.loc[2, 'plain']`: Store the literal value `100` in the selected entry/column.
- Line 30: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test xi sign and unlimited history

Original function: `EstimationChecks.test_xi_sign_and_unlimited_history`. Source begins at line 33.

What this test establishes. Checks numerical Xi signs and persistence across long gaps. The explicit -.7/.3 expectations distinguish own-stock penalties from recent-other rewards.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 33–33:

```
def test_xi_sign_and_unlimited_history(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–40:

```
frame = add_stocks(self.frame(), .7)
self.assertAlmostEqual(frame.loc[1, "xi_other"], -.7)
self.assertAlmostEqual(frame.loc[1, "xi_berry"], .3)
# Time gaps do not change occasion-time histories when delta=1.
old = self.frame()
old.loc[2, "week"] = pd.Timestamp("2011-04-03")
old = add_stocks(old)
```

Read this block:

- Line 34, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 35: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `old`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 39, `old.loc[2, 'week']`: Store the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 40, `old`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 41–43:

```
np.testing.assert_allclose(old.filter(like="xi_"), frame.filter(like="xi_"))
self.assertEqual(frame.loc[2, "novelty_other"], 0.)
self.assertAlmostEqual(frame.loc[2, "xi_other"], -.7)
```

Read this block:

- Line 41: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test parquet pipeline and fit

Original function: `EstimationChecks.test_parquet_pipeline_and_fit`. Source begins at line 45.

What this test establishes. Builds a tiny real Parquet input and runs preparation plus a deliberately short fit. It checks all-store counts, exposure-store menus, missing alternatives, exported files, and own-DMA exclusion. Its low iteration limit is a smoke check, not proof of statistical convergence.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 45–45:

```
45 def test_parquet_pipeline_and_fit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–87:

```
46 with tempfile.TemporaryDirectory() as temp:
47     root = Path(temp)
48     rows = []
49     def append(household, week, upc, store, dma, price, quantity=1):
50         rows.append(dict(household_code=household, household_size=1 if household in [10, 40] else 2,
    ↪  panel_year=2010,
51                          purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
52                          trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
53                          quantity=quantity, price=float(price), dma_cd=dma,
54                          flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
55     append(10, 20100102, 1, 100, 1, 3, 2)
56     append(10, 20100102, 2, 200, 1, 4)
57     append(10, 20100109, 2, 100, 1, 4, 2)
58     for week in [20100102, 20100109]:
59         for upc in [1, 2, 3]:
60             append(20, week, upc, 200, 2, 2 + upc)
61             append(30, week, upc, 300, 3, 6 + upc)
62             append(40, week, upc, 100, 1, 2 + upc)
63     # Repeat a cheap quote: it must not overweight the overall IV.
64     append(20, 20100102, 1, 200, 2, 3)
65     sample = root / "prepared.parquet"
66     original = pl.DataFrame(rows)
67     original.write_parquet(sample)
68     args = SimpleNamespace(
69         sample=sample, flavors=root / "not_needed.csv", rms=root / "does_not_exist.parquet",
70         first_year=2010, last_year=2010, max_households=1, habit_weight=.7,
71         capacity=0, no_price_control=True, household_dma_column="dma_cd", se_method="none",
72         model="heterogeneous", draws=8, starts=1, maxiter=5, seed=306, output=root / "output")
73     # The limit is applied after restricting demand to single-person records.
74     mixed = original.with_columns(pl.when(pl.col("household_code") == 10)
75         .then(2).otherwise(pl.col("household_size")).alias("household_size"))
76     mixed.write_parquet(sample)
77     self.assertEqual(prepare_data(args).household_code.unique().tolist(), [40])
78     original.drop("household_size").write_parquet(sample)
79     with self.assertRaisesRegex(ValueError, "requires household_size"):
80         prepare_data(args)
81     original.with_columns(pl.lit(None).alias("household_size")).write_parquet(sample)
82     with self.assertRaisesRegex(ValueError, "No purchase lines"):
83         prepare_data(args)
```

```
    original.write_parquet(sample)
    frame = prepare_data(args)
    self.assertEqual(len(frame), 2)
    self.assertEqual(frame.household_code.unique().tolist(), [10])
```

Source lines 88–119:

```
self.assertEqual(frame.loc[0, "other"], 2)
self.assertEqual(frame.loc[0, "berry"], 1)
self.assertEqual(frame.loc[0, "store_code_uc"], 100)
self.assertEqual(frame.loc[0, "price_other"], 3)
np.testing.assert_allclose(frame.filter(like="stock_").iloc[0], 0)
fit = fit_model(make_arrays(frame, 0), args)
self.assertTrue(np.isfinite(fit.fun))
self.assertTrue((args.output / "fit.json").exists())
self.assertTrue((args.output / "gamma_distribution.pdf").exists())
incomplete = original.filter(~((pl.col("purchase_date") == "2010-01-02")
                               & (pl.col("store_code_uc") == 100) & (pl.col("upc") == "3")))
incomplete.write_parquet(sample)
partial = prepare_data(args)
self.assertEqual(len(partial), 2)
self.assertFalse(partial.loc[0, "available_plain"])
self.assertTrue(np.isnan(partial.loc[0, "price_plain"]))
original.write_parquet(sample)
args.no_price_control = False
controlled = prepare_data(args)
# Pool DMA 2's quotes 3,4,5 and DMA 3's 7,8,9; mean = 6.
for name in ["iv_overall", "iv_other", "iv_berry", "iv_plain"]:
    np.testing.assert_allclose(controlled[name], 6.)
# Changing every own-DMA quote cannot change the instrument.
changed = original.with_columns(pl.when(pl.col("dma_cd") == 1)
                                .then(pl.col("price") + 100).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
# Store 200 is shared across DMAs in this fixture. Update both copies
# of its quoted price to keep local store/week/UPC prices consistent.
changed = changed.with_columns(pl.when(pl.col("store_code_uc") == 200)
                               .then(pl.col("upc").cast(pl.Float64) + 2).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
np.testing.assert_allclose(prepare_data(args)["iv_overall"], 6.)
```

Read this block:

- Line 46: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 47, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 48, `rows`: Set this name to the list `[]`.
- Line 49: define nested callable `append`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 55: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: for each entry yielded by `[20100102, 20100109]`, bind/unpack it into `week` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 59: for each entry yielded by `[1, 2, 3]`, bind/unpack it into `upc` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: call `append` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.
- Line 61: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 62: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65, `sample`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `original`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 67: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 74, `mixed`: Set this name to the result of calling `original.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 76: call `mixed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `original.drop('household_size').write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 80: call `prepare_data` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 81: call `.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 83: call `prepare_data` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 86: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93, `fit`: Set this name to the result of calling `fit_model(...)`; the arguments and preceding method chain are shown above.
- Line 94: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.
- Line 96: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97, `incomplete`: Set this name to the result of calling `original.filter(...)`; the arguments and preceding method chain are shown above.
- Line 99: call `incomplete.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 100, `partial`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 101: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 102: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 103: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105, `args.no_price_control`: Set this name to the literal value `False`.
- Line 106, `controlled`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 108: for each entry yielded by `['iv_overall', 'iv_other', 'iv_berry', 'iv_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 109: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 111, `changed`: Set this name to the result of calling `original.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 113: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116, `changed`: Set this name to the result of calling `changed.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 118: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `append`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `len`: Count elements/rows rather than summing their numerical values.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test parquet pipeline and fit.append

Original function: `EstimationChecks.test_parquet_pipeline_and_fit.append`. Source begins at line 49.

This nested fixture builder appends one purchase dictionary to an enclosing `rows` list. Positional inputs describe household/week/UPC/store/DMA/price; quantity defaults to one. A dictionary lookup turns UPC 1/2/3 into the three flavor names. `len(rows)+1` assigns a fresh trip ID at append time. The helper reduces repetitive fixture text while keeping test prices and households explicit. It mutates the captured list and returns None; callers need not assign its result.

Definition and stated intent:

Source lines 49–49:

```
49 def append(household, week, upc, store, dma, price, quantity=1):
```

`def` creates this callable; the parentheses name its inputs (`household`, `week`, `upc`, `store`, `dma`, `price`, `quantity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–54:

```
50 rows.append(dict(household_code=household, household_size=1 if household in [10, 40] else 2, panel_year=2010,
51                  purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
52                  trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
53                  quantity=quantity, price=float(price), dma_cd=dma,
54                  flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
```

Read this block:

- Line 50: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `len`: Count elements/rows rather than summing their numerical values.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test unavailable flavor is excluded from logit denominator

Original function: `EstimationChecks.test_unavailable_flavor_is_excluded_from_logit_denominator`. Source begins at line 121.

What this test establishes. Removes plain prices and independently computes the remaining two-flavor likelihood. It also requires a ValueError if an unavailable flavor is actually purchased. This catches treating a missing-price placeholder as an available free good.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 121–121:

```
121 def test_unavailable_flavor_is_excluded_from_logit_denominator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–127:

```
122 frame = self.frame()
123 frame["price_plain"] = np.nan
124 data = make_arrays(add_stocks(frame), 0)
125 theta = np.array([.1, -.2, -.4])
126 actual = household_loglike(theta, data, make_draws(8, 306), "naive")[0]
127 utility = np.array([0., .1]) - np.exp(-.4) * frame[["price_other", "price_berry"]].to_numpy()
```

Read this block:

- Line 122, `frame`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 123, `frame['price_plain']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 124, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

- Line 125, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 126, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, make_draws(8, 306), 'naive...`; brackets select from the object to their left.
- Line 127, `utility`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 128–131:

```
expected = (frame[["other", "berry"]].to_numpy() * (utility - logsumexp(utility, axis=1, keepdims=True))).sum()
self.assertAlmostEqual(actual, expected)
self.assertFalse(data["available"][:, 2].any())
frame.loc[0, "plain"] = 1
```

Read this block:

- Line 128, `expected`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 129: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 130: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131, `frame.loc[0, 'plain']`: Store the literal value `1` in the selected entry/column.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 132–133:

```
with self.assertRaisesRegex(ValueError, "Purchased flavors"):
    make_arrays(add_stocks(frame), 0)
```

Read this block:

- Line 132: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 133: call `make_arrays` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test persistent household integration

Original function: `EstimationChecks.test_persistent_household_integration`. Source begins at line 135.

What this test establishes. Independently computes a draw's full household history before averaging draws. This catches a subtle wrong model that integrates each week separately.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 135–135:

```
135 def test_persistent_household_integration(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 136–142:

```
136 data = make_arrays(add_stocks(self.frame()), 7)
137 draws = make_draws(8, 306)
138 names = parameter_names("heterogeneous", True)
139 theta = np.array([.4 if n.startswith("sd_") else .2 for n in names])
140 actual = household_loglike(theta, data, draws, "heterogeneous")[0]
141 # Independently calculate each fixed type's full household history.
142 by_draw = []
```

Read this block:

- Line 136, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 137, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 138, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 139, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 140, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, draws, 'heterogeneous')[0]`; brackets select from the object to their left.
- Line 142, `by_draw`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 143–149:

```
143 for z in draws:
144     alpha = -np.exp(.2 + .4*z[0])
145     gamma = .2 + .4*z[1]
146     utility = np.array([.2, .4, .4]) + alpha*data["prices"] + gamma*data["xi"]
147     utility = np.column_stack([utility, np.zeros(3)])
148     logp = utility - logsumexp(utility, axis=1, keepdims=True)
149     by_draw.append(np.sum(data["counts"]*logp))
```

Read this block:

- Line 143: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(0.2 + 0.4 * z[0])`.
- Line 145, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 146, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 147, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 148, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 149: call `by_draw.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 150–150:

```
150 self.assertAlmostEqual(actual, logsumexp(by_draw)-np.log(8))
```

Read this block:

- Line 150: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test separate history nests combined index

Original function: `EstimationChecks.test_separate_history_nests_combined_index`. Source begins at line 152.

Sets b_N=(1-lambda)*gamma and b_S=-lambda*gamma and verifies the split likelihood nests the old combined index under that restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 152–152:

```
152 def test_separate_history_nests_combined_index(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 153–153:

```
153 data = make_arrays(add_stocks(self.frame(), .7), 7)
```

Read this block:

- Line 153, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 154–167:

```
154 for control in [False, True]:
155     data["control"] = control
156     data["cf"] = np.full_like(data["prices"], .1)
157     old_names = parameter_names("heterogeneous", True, control)
158     values = dict.fromkeys(old_names, .2)
159     values["sd_gamma"] = 0.
160     values["mean_gamma"] = -.4
161     separate = dict(values, mean_novelty=.3*values["mean_gamma"],
162                     mean_own_stock=-.7*values["mean_gamma"])
163     draws = make_draws(8, 306)
164     old = household_loglike([values[n] for n in old_names], data, draws, "heterogeneous")
165     new = household_loglike([separate[n] for n in parameter_names("separate_history", True, control)],
166                             data, draws, "separate_history")
167     np.testing.assert_allclose(old, new, atol=1e-12)
```

Read this block:

- Line 154: for each entry yielded by `[False, True]`, bind/unpack it into `control` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 155, `data['control']`: Store the object currently named `control`; plain assignment does not copy it in the selected entry/column.
- Line 156, `data['cf']`: Store the result of calling `np.full_like(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 157, `old_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.

- Line 158, `values`: Set this name to the result of calling `dict.fromkeys(...)`; the arguments and preceding method chain are shown above.
- Line 159, `values['sd_gamma']`: Store the literal value `0.0` in the selected entry/column.
- Line 160, `values['mean_gamma']`: Store the negated or sign-adjusted expression `-0.4` in the selected entry/column.
- Line 161, `separate`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 163, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 164, `old`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 165, `new`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 167: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test no control ignores residuals but keeps price

Original function: `EstimationChecks.test_no_control_ignores_residuals_but_keeps_price`. Source begins at line 169.

Checks disabling the control removes residual utility while retaining price sensitivity. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 169–169:

```
169 def test_no_control_ignores_residuals_but_keeps_price(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 170–170:

```
170 data = make_arrays(add_stocks(self.frame()), 7)
```

Read this block:

- Line 170, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 171–179:

```
171 for model in ["heterogeneous", "separate_history", "heterogeneous_history"]:
172     names = parameter_names(model, True, False)
173     self.assertNotIn("price_control", names)
174     self.assertIn("mu_log_minus_alpha", names)
175     theta = np.full(len(names), .2)
176     draws = make_draws(8, 306, model)
177     before = household_loglike(theta, data, draws, model)
178     np.testing.assert_allclose(before, household_loglike(theta, dict(data, cf=data["cf"]+100), draws, model))
179     self.assertFalse(np.allclose(before, household_loglike(theta, dict(data, prices=data["prices"]+1), draws,
        ↪  model)))
```

Read this block:

- Line 171: for each entry yielded by `['heterogeneous', 'separate_history', 'heterogeneous_history']`, bind/unpack it into `model` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 172, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 173: call `self.assertNotIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 174: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 176, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 177, `before`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 178: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 179: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `len`: Count elements/rows rather than summing their numerical values.
- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test analytic gradient

Original function: `EstimationChecks.test_analytic_gradient`. Source begins at line 181.

What this test establishes. Perturbs each coefficient and compares centered finite-difference objective slopes with the supplied analytic gradient. This checks derivative values rather than just whether optimization returns a result.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 181–181:

```
181 def test_analytic_gradient(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 182–182:

```
182 from scipy.optimize._numdiff import approx_derivative
```

Read this block:

- Line 182: from `scipy.optimize._numdiff`, import `approx_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 183–193:

```
183 for capacity in [0, 7]:
184     frame = add_stocks(self.frame())
185     frame["cf_other"], frame["cf_berry"], frame["cf_plain"] = .1, -.2, .3
186     data = make_arrays(frame, capacity)
187     for model in ["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
        ↪ "heterogeneous_history"]:
188         theta = np.full(len(parameter_names(model, bool(capacity), True)), .2)
189         draws = make_draws(8, 306, model)
190         value, gradient = demand_objective_gradient(theta, data, draws, model)
191         objective = lambda t: -household_loglike(t, data, draws, model).mean()
192         self.assertAlmostEqual(value, objective(theta))
193         np.testing.assert_allclose(gradient, approx_derivative(objective, theta).ravel(), rtol=1e-5, atol=1e-7)
```

Read this block:

- Line 183: for each entry yielded by `[0, 7]`, bind/unpack it into `capacity` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 184, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 185, `(frame['cf_other'], frame['cf_berry'], frame['cf_plain'])`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 186, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 187: for each entry yielded by `['naive', 'homogeneous', 'mixed_naive', 'heterogeneous', 'mixt...`, bind/unpack it into `model` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 188, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.

- Line 189, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 190, `(value, gradient)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 191, `objective`: Set this name to a one-expression callable, with input `t` and returned expression `-household_loglike(t, data, draws, model).mean()`.
- Line 192: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `len`: Count elements/rows rather than summing their numerical values.
- `objective`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `approx_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixture combines household likelihoods and nests normal

Original function: `EstimationChecks.test_mixture_combines_household_likelihoods_and_nests_normal`. Source begins at line 195.

Verifies component mixing happens after whole-household likelihood construction and coincident components recover the single-normal model. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 195–195:

```
195 def test_mixture_combines_household_likelihoods_and_nests_normal(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 196–201:

```
from est import mixture_components
frame = add_stocks(self.frame())
other = frame.copy()
other["household_code"] = 20
other["xi_other"] += .5
data = make_arrays(pd.concat([frame, other], ignore_index=True), 7)
```

Read this block:

- Line 196: from `est`, import `mixture_components` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 197, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 198, `other`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 199, `other['household_code']`: Store the literal value `20` in the selected entry/column.
- Line 200: update `other['xi_other']` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 201, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 202–207:

```
draws = make_draws(16, 306)
names = parameter_names("mixture", True)
theta = np.full(len(names), .2)
theta[names.index("mixture_weight")] = .35
_, _, first, second, weight = mixture_components(theta, data)
l1 = household_loglike(first, data, draws, "heterogeneous")
```

Read this block:

- Line 202, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 203, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 204, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 205, `theta[names.index('mixture_weight')]`: Store the literal value `0.35` in the selected entry/column.
- Line 206, `(_, _, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 207, `l1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `len`: Count elements/rows rather than summing their numerical values.
- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 208–212:

```
l2 = household_loglike(second, data, draws, "heterogeneous")
expected = np.logaddexp(np.log(weight) + l1, np.log1p(-weight) + l2)
np.testing.assert_allclose(household_loglike(theta, data, draws, "mixture"), expected)
theta[names.index("gamma_mean_gap")] = 0.
np.testing.assert_allclose(household_loglike(theta, data, draws, "mixture"), l1)
```

Read this block:

- Line 208, `l2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 209, `expected`: Set this name to the result of calling `np.logaddexp(...)`; the arguments and preceding method chain are shown above.
- Line 210: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 211, `theta[names.index('gamma_mean_gap')]`: Store the literal value `0.0` in the selected entry/column.
- Line 212: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixture fit outputs

Original function: `EstimationChecks.test_mixture_fit_outputs`. Source begins at line 214.

Checks a tiny fitted mixture writes consistent parameter and inference metadata; its empirical content is entirely generated. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 214–214:

```
def test_mixture_fit_outputs(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 215–216:

```
import json
data = make_arrays(add_stocks(self.frame()), 7)
```

Read this block:

- Line 215: import `json`. An as-name is a convenient local alias, not another library.
- Line 216, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 217–226:

```
217 with tempfile.TemporaryDirectory() as directory:
218     args = SimpleNamespace(model="mixture", draws=8, seed=306, starts=2,
219                            maxiter=3, output=Path(directory), se_method="none", capacity=7)
220     fit_model(data, args)
221     estimates = pd.read_csv(args.output / "estimates.csv")
222     self.assertIn("mixture_weight", estimates.parameter.tolist())
223     self.assertTrue((args.output / "gamma_distribution.pdf").exists())
224     report = json.loads((args.output / "fit.json").read_text())
225     self.assertEqual(report["parameters"], len(estimates))
226     self.assertAlmostEqual(report["aic"], 2 * report["nll"] + 2 * len(estimates))
```

Read this block:

- Line 217: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 218, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 220: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 221, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 222: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 223: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 224, `report`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 225: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 226: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing,

or another documented side effect.

EstimationChecks.test four model specifications

Original function: `EstimationChecks.test_four_model_specifications`. Source begins at line 228.

What this test establishes. Checks parameter layouts/likelihood behavior for the four supported specifications. The labels define vector meaning; silently changed ordering would corrupt estimates even when vector lengths still match.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 228–228:

```
228 def test_four_model_specifications(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 229–240:

```
229 for model, gamma, alpha_sd, gamma_sd in [
230     ("naive", False, False, False),
231     ("homogeneous", True, False, False),
232     ("mixed_naive", False, True, False),
233     ("heterogeneous", True, True, True),
234 ]:
235     with self.subTest(model=model):
236         names = parameter_names(model, False)
237         self.assertEqual("mean_gamma" in names, gamma)
238         self.assertEqual("sd_log_minus_alpha" in names, alpha_sd)
239         self.assertEqual("sd_gamma" in names, gamma_sd)
240         self.assertFalse(any("sd_beta" in n for n in names))
```

Read this block:

- Line 229: for each entry yielded by `[('naive', False, False, False), ('homogeneous', True, False, ...`, bind/unpack it into `(model, gamma, alpha_sd, gamma_sd)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 235: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 236, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 237: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 238: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 239: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 240: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixed naive integrates only alpha

Original function: `EstimationChecks.test_mixed_naive_integrates_only_alpha`. Source begins at line 242.

What this test establishes. Checks that mixed-naive choice probabilities vary with alpha draws while excluding a gamma effect. It independently constructs the corresponding integrated likelihood.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 242–242:

```
242 def test_mixed_naive_integrates_only_alpha(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 243–247:

```
243 data = make_arrays(add_stocks(self.frame()), 0)
244 draws = make_draws(8, 306)
245 theta = np.array([.1, -.2, -.4, .6])
246 actual = household_loglike(theta, data, draws, "mixed_naive")
247 independent = []
```

Read this block:

- Line 243, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 244, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 245, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 246, `actual`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 247, `independent`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 248–252:

```
248 for z in draws:
249     alpha = -np.exp(-.4 + .6*z[0])
250     utility = np.array([0., .1, -.2]) + alpha*data["prices"]
251     logp = utility - logsumexp(utility, axis=1, keepdims=True)
252     independent.append(np.sum(data["counts"]*logp))
```

Read this block:

- Line 248: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 249, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(-0.4 + 0.6 * z[0])`.
- Line 250, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 251, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 252: call `independent.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 253–258:

```
253 np.testing.assert_allclose(actual, [logsumexp(independent)-np.log(len(draws))])
254 changed = dict(data, xi=data["xi"] + 100.)
255 np.testing.assert_allclose(actual, household_loglike(theta, changed, draws, "mixed_naive"))
256 np.testing.assert_allclose(
257     household_loglike(theta[:3], data, draws, "naive"),
258     household_loglike(np.r_[theta[:3], 0.], data, draws, "mixed_naive"))
```

Read this block:

- Line 253: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 254, `changed`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 255: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 256: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `len`: Count elements/rows rather than summing their numerical values.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test zero heterogeneity nests homogeneous model

Original function: `EstimationChecks.test_zero_heterogeneity_nests_homogeneous_model`. Source begins at line 260.

What this test establishes. Sets random-coefficient SDs to zero and compares against the common-coefficient model. A nesting identity is a strong check on parameter transforms and integration.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 260–260:

```
260 def test_zero_heterogeneity_nests_homogeneous_model(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 261–267:

```
261 data = make_arrays(add_stocks(self.frame()), 0)
262 fixed = np.array([.1, -.2, -.4, .3])
263 random = np.r_[fixed, np.zeros(2)]
264 draws = make_draws(8, 306)
265 np.testing.assert_allclose(
266     household_loglike(fixed, data, draws, "homogeneous"),
267     household_loglike(random, data, draws, "heterogeneous"))
```

Read this block:

- Line 261, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 262, `fixed`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 263, `random`: Set this name to the entry/slice selected by `np.r_[fixed, np.zeros(2)]`; brackets select from the object to their left.
- Line 264, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 265: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__hessian__se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Check information scaling, correlation, and invalid-inference handling."""
2 import tempfile
3 import unittest
4 from pathlib import Path
5 import numpy as np
6 from hessian_se import observed_covariance, hessian_standard_errors
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: from `hessian_se`, import `observed_covariance, hessian_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 9–9:

```
9 class HessianChecks(unittest.TestCase):
```

Read this block:

- Line 9: define class `HessianChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

HessianChecks.test correlated information and household scaling

Original function: `HessianChecks.test_correlated_information_and_household_scaling`. Source begins at line 10.

Uses known quadratic curvature to check inversion and the number-of-households scaling in Hessian covariance. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 10–10:

```
10 def test_correlated_information_and_household_scaling(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 11–15:

```
information = np.array([[4., 1.], [1., 2.]])
theta = np.array([.2, -.1])
gradient = lambda x: information @ (x - theta)
covariance = observed_covariance(theta, gradient, 100, [(-5, 5)] * 2)
np.testing.assert_allclose(covariance, np.linalg.inv(100 * information), rtol=1e-9)
```

Read this block:

- Line 11, `information`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 12, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 13, `gradient`: Set this name to a one-expression callable, with input `x` and returned expression `information @ (x - theta)`.
- Line 14, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 15: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 16–23:

```
with tempfile.TemporaryDirectory() as tmp:
    table, report = hessian_standard_errors(theta, gradient, 100,
        [(-5, 5)] * 2, ['a', 'b'], Path(tmp))
    np.testing.assert_allclose(table.std_error, np.sqrt(np.diag(covariance)))
    self.assertTrue((table.ci_lower < theta).all())
    self.assertTrue((table.ci_upper > theta).all())
    self.assertTrue((Path(tmp) / 'covariance.csv').exists())
    self.assertEqual(report['first_stage_uncertainty'], 'not included')
```

Read this block:

- Line 16: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 17, `(table, report)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 19: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 21: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

HessianChecks.test boundary and indefinite information withheld

Original function: `HessianChecks.test_boundary_and_indefinite_information_withheld`. Source begins at line 25.

Confirms boundary or invalid-information cases withhold inference instead of reporting misleading SEs. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 25–25:

```
25 def test_boundary_and_indefinite_information_withheld(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–27:

```
26 with self.assertRaisesRegex(ValueError, 'bounds'):
27     observed_covariance(np.array([0.]), lambda x: x, 10, [(0, 3)])
```

Read this block:

- Line 26: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 27: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 28–29:

```
28 with self.assertRaisesRegex(ValueError, 'positive definite'):
29     observed_covariance(np.array([1.]), lambda x: -x, 10, [(-3, 3)])
```

Read this block:

- Line 28: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.

- Line 29: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 30–32:

```
with self.assertRaisesRegex(ValueError, 'ill-conditioned'):
    observed_covariance(np.array([1., 1.]), lambda x: np.array([x[0], 0.]),
                        10, [(-3, 3)] * 2)
```

Read this block:

- Line 30: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/test_price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Checks for DMA exclusion, first-stage OLS, and two-stage inference."""
import contextlib
import io
import tempfile
import unittest
from pathlib import Path
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
from types import SimpleNamespace

import numpy as np
import pandas as pd
from scipy.optimize import OptimizeResult
from unittest.mock import patch

from price_control import other_dma_instruments, absorb_effects, add_price_controls
```

Read this block:

- Line 7: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 9: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 10: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 11: from `scipy.optimize`, import `OptimizeResult` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: from `price_control`, import `other_dma_instruments, absorb_effects, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 15–16:

```
15 from bootstrap_se import resample_households, bootstrap_standard_errors
16 from est import FLAVORS, make_arrays, optimize_demand
```

Read this block:

- Line 15: from `bootstrap_se`, import `resample_households, bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `est`, import `FLAVORS, make_arrays, optimize_demand` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 43–43:

```
43 class PriceControlChecks(unittest.TestCase):
```

Read this block:

- Line 43: define class `PriceControlChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 136–137:

```
136 if __name__ == "__main__":
137     unittest.main()
```

Read this block:

- Line 136: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 137: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

synthetic panel

Original function: `synthetic_panel`. Source begins at line 19.

This constructs a reproducible simulated panel with 60 households and eight periods, first-stage prices, residuals, utilities, and multinomial package counts. `repeat` holds each household ID for eight rows; `tile` repeats the eight-date sequence for all households. Stable softmax subtracts each row's largest utility before exponentiating. `rng.multinomial(8,p)` samples an eight-package vector whose entries sum to eight. This gives the tests known structure without private data; it is a test fixture, not estimated reality.

Definition and stated intent:

Source lines 19–19:

```
19 def synthetic_panel():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–25:

```
rng = np.random.default_rng(120)
n_households, periods = 60, 8
size = n_households * periods
frame = pd.DataFrame({"household_code": np.repeat(np.arange(n_households), periods),
                      "week": np.tile(pd.date_range("2010-01-02", periods=periods, freq="7D"), n_households),
                      "dma_cd": np.repeat(np.arange(n_households) % 4 + 1, periods)})
```

Read this block:

- Line 20, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 21, `(n_households, periods)`: Unpack the tuple `(60, 8)` into these targets in order.
- Line 22, `size`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 23, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.repeat`: Repeat each element consecutively, e.g. each household ID across all its periods.
- `.tile`: Repeat a complete sequence, e.g. the period-date sequence for every household.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 26–30:

```
for flavor in FLAVORS:
    frame["xi_" + flavor] = rng.normal(size=size)
    frame["iv_" + flavor] = rng.uniform(.5, 2.5, size=size)
    frame["price_" + flavor] = (1 + .8*frame["iv_" + flavor]
                                + .1*frame["xi_" + flavor] + rng.normal(0, .2, size))
```

Read this block:

- Line 26: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27, `frame['xi_' + flavor]`: Store the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `frame['iv_' + flavor]`: Store the result of calling `rng.uniform(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 29, `frame['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.uniform`: Draw values from a continuous uniform interval for controlled synthetic inputs.

Source lines 31–37:

```
controlled, _ = add_price_controls(frame)
utility = (np.array([0., .4, -.3])
           - controlled[["price_"+f for f in FLAVORS]].to_numpy()
           + .8*controlled[["cf_"+f for f in FLAVORS]].to_numpy())
probs = np.exp(utility - utility.max(axis=1, keepdims=True))
probs /= probs.sum(axis=1, keepdims=True)
counts = np.array([rng.multinomial(8, p) for p in probs])
```

Read this block:

- Line 31, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 32, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 35, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 36: update `probs` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 37, `counts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.

Source lines 38–39:

```
38 for j, flavor in enumerate(FLAVORS):
39     frame[flavor] = counts[:, j]
```

Read this block:

- Line 38: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 39, `frame[flavor]`: Store the entry/slice selected by `counts[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 40–40:

```
40 return frame
```

Read this block:

- Line 40: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

PriceControlChecks.test excludes consumer dma even when exposure store elsewhere

Original function: `PriceControlChecks.test_excludes_consumer_dma_even_when_exposure_store_elsewhere`. Source begins at line 44.

What this test establishes. Exercises the retained UPC-matched IV helper with explicit tiny prices. It excludes the consumer DMA rather than assuming exposure store and consumer are in the same market, and checks incomplete baskets become missing.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results,

and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 44–44:

```
44 def test_excludes_consumer_dma_even_when_exposure_store_elsewhere(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–53:

```
45 week = pd.Timestamp("2010-01-02")
46 weekly = pd.DataFrame({"store_code_uc": [99, 99, 99], "week": [week]*3,
47                        "dma_cd": [1, 2, 4]})
48 exposure = pd.DataFrame({"store_code_uc": [99, 99], "week": [week]*2,
49                          "upc": ["a", "b"], "flavor": ["berry", "berry"]})
50 prices = pd.DataFrame({"upc": ["a"]*3+["b"]*3, "week": [week]*6,
51                        "dma_code": [1, 2, 3]*2, "dma_price": [1., 3., 5., 2., 4., 6.]})
52 result = other_dma_instruments(weekly, exposure, prices)
53 np.testing.assert_allclose(result["iv_berry"], [4.5, 3.5, 3.5])
```

Read this block:

- Line 45, `week`: Set this name to the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above.
- Line 46, `weekly`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 48, `exposure`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 50, `prices`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 52, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 53: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `other_dma_instruments`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 55–57:

```
55 missing = prices.loc[~((prices["upc"] == "b") & (prices["dma_code"] != 1))]
56 result = other_dma_instruments(weekly, exposure, missing)
57 self.assertTrue(np.isnan(result.loc[0, "iv_berry"]))
```

Read this block:

- Line 55, `missing`: Set this name to the entry/slice selected by `prices.loc[~((prices['upc'] == 'b') & (prices['dma_code']...`; brackets select from the object to their left.
- Line 56, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test absorption matches explicit dummy ols

Original function: `PriceControlChecks.test_absorption_matches_explicit_dummy_ols`. Source begins at line 59.

What this test establishes. Compares repeated demeaning to a small regression on explicit week/DMA dummy columns. The dummy matrix is feasible in a fixture and supplies a transparent independent alternative to the production memory-saving algorithm.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 59–59:

```
59 def test_absorption_matches_explicit_dummy_ols(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 60–66:

```
60 rng = np.random.default_rng(20)
61 weeks = pd.Series([1, 1, 2, 2, 2, 3, 3, 4, 4, 4])
62 dma = pd.Series([1, 2, 1, 2, 3, 1, 3, 1, 2, 3])
63 x = rng.normal(size=(len(weeks), 3))
64 dummies = pd.concat([pd.get_dummies(weeks, dtype=float),
65                      pd.get_dummies(dma, dtype=float)], axis=1).to_numpy()
66 expected = x - dummies @ np.linalg.lstsq(dummies, x, rcond=None)[0]
```

Read this block:

- Line 60, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 61, `weeks`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 62, `dma`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 63, `x`: Set this name to the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above.
- Line 64, `dummies`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 66, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `len`: Count elements/rows rather than summing their numerical values.

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.get_dummies`: Construct indicator columns for categories, used as a small explicit fixed-effect benchmark in tests.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 67–67:

```
67 np.testing.assert_allclose(absorb_effects(x, [weeks, dma]), expected, atol=1e-8)
```

Read this block:

- Line 67: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test first stage residuals orthogonal and in utility

Original function: `PriceControlChecks.test_first_stage_residuals_orthogonal_and_in_utility`. Source begins at line 69.

What this test establishes. Checks positive first-stage relevance in controlled data, residual orthogonality, zero group means, and that the demand arrays actually contain the computed controls.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 69–69:

```
69 def test_first_stage_residuals_orthogonal_and_in_utility(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 70–73:

```
70 frame = synthetic_panel()
71 controlled, report = add_price_controls(frame)
72 self.assertTrue((report["iv_coefficient"] > .7).all())
73 self.assertTrue((report["iv_cluster_F"] > 10).all())
```

Read this block:

- Line 70, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 71, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 72: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 74–77:

```
74 for flavor in FLAVORS:
75     residual = controlled["cf_" + flavor]
76     self.assertAlmostEqual(float(residual @ frame["iv_" + flavor]), 0., places=7)
77     np.testing.assert_allclose(residual.groupby(frame["week"]).mean(), 0., atol=1e-8)
```

Read this block:

- Line 74: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 75, `residual`: Set this name to the entry/slice selected by `controlled['cf_' + flavor]`; brackets select from the object to their left.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 78–80:

```
78 data = make_arrays(controlled, 0)
79 self.assertTrue(data["control"])
80 np.testing.assert_allclose(data["cf"], controlled[["cf_"+f for f in FLAVORS]])
```

Read this block:

- Line 78, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 79: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test first stage uses only covered flavor prices

Original function: `PriceControlChecks.test_first_stage_uses_only_covered_flavor_prices`. Source begins at line 82.

What this test establishes. Moves purchases away from unavailable alternatives, removes their prices, and checks each first stage uses only its covered rows. It also verifies zero residual placeholders remain finite and masked.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 82–82:

```
82 def test_first_stage_uses_only_covered_flavor_prices(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 83–83:

```
83 frame = synthetic_panel()
```

Read this block:

- Line 83, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 84–89:

```
84 for j, flavor in enumerate(FLAVORS):
85     missing = np.arange(len(frame)) % 4 == j
86     next_flavor = FLAVORS[(j + 1) % 3]
87     frame.loc[missing, next_flavor] += frame.loc[missing, flavor]
88     frame.loc[missing, flavor] = 0
89     frame.loc[missing, "price_" + flavor] = np.nan
```

Read this block:

- Line 84: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 85, `missing`: Set this name to the Boolean condition `np.arange(len(frame)) % 4 == j`; it can be a scalar or a row/array mask depending on its inputs.
- Line 86, `next_flavor`: Set this name to the entry/slice selected by `FLAVORS[(j + 1) % 3]`; brackets select from the object to their left.

- Line 87: update `frame.loc[missing, next_flavor]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 88, `frame.loc[missing, flavor]`: Store the literal value `0` in the selected entry/column.
- Line 89, `frame.loc[missing, 'price_' + flavor]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 90–90:

```
90 controlled, report = add_price_controls(frame)
```

Read this block:

- Line 90, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 91–96:

```
91 for flavor in FLAVORS:
92     available = frame["price_" + flavor].notna()
93     self.assertEqual(report.set_index("flavor").loc[flavor, "observations"], available.sum())
94     self.assertTrue((controlled.loc[~available, "cf_" + flavor] == 0).all())
95     self.assertAlmostEqual(float(controlled.loc[available, "cf_" + flavor]
96                                  @ frame.loc[available, "iv_" + flavor]), 0., places=7)
```

Read this block:

- Line 91: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 92, `available`: Set this name to the result of calling `frame['price_' + flavor].notna(...)`; the arguments and preceding method chain are shown above.
- Line 93: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 97–99:

```
97 arrays = make_arrays(controlled, 0)
98 self.assertFalse(arrays["available"].all())
99 self.assertTrue(np.isfinite(arrays["cf"]).all())
```

Read this block:

- Line 97, `arrays`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 98: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test resampling preserves histories and relabels duplicates

Original function: `PriceControlChecks.test_resampling_preserves_histories_and_relabels_duplicates`. Source begins at line 101.

What this test establishes. Resamples a 60-household/eight-period fixture and verifies there are 60 newly labeled histories, each with eight rows and no repeated new-household/week key. Sampling whole groups is compared with the behavior a row bootstrap would break.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 101–101:

```
101 def test_resampling_preserves_histories_and_relabels_duplicates(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 102–106:

```
102 frame = synthetic_panel()
103 sampled = resample_households(frame, np.random.default_rng(1))
104 self.assertEqual(sampled["household_code"].nunique(), 60)
105 np.testing.assert_array_equal(sampled.groupby("household_code").size(), 8)
106 self.assertFalse(sampled.duplicated(["household_code", "week"]).any())
```

Read this block:

- Line 102, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 103, `sampled`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 104: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test bootstrap refits first stage and reports empirical sd

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd`. Source begins at line 108.

What this test establishes. Wraps the first-stage callable with a mock to count 20 refits, supplies a cheap residual-dependent fake estimator, and compares output SEs with SDs calculated from saved bootstrap draws. This isolates two-stage orchestration from optimization.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 108–108:

```
108 def test_bootstrap_refits_first_stage_and_reports_empirical_sd(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 109–109:

```
109 frame = synthetic_panel()
```

Read this block:

- Line 109, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 110–125:

```
110 with tempfile.TemporaryDirectory() as directory:
111     args = SimpleNamespace(seed=306, bootstrap_reps=20, no_price_control=False,
112                            output=Path(directory))
113     def estimate(sample, initial):
114         # Explicitly depend on the recomputed residual, so frozen residuals
115         # would produce a different bootstrap distribution.
116         theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
117         return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
118     with patch("bootstrap_se.add_price_controls", wraps=add_price_controls) as first_stage:
119         with contextlib.redirect_stdout(io.StringIO()):
120             table, report = bootstrap_standard_errors(frame, args, estimate, np.zeros(2), ["a", "b"])
121     self.assertEqual(first_stage.call_count, 20)
122     self.assertEqual(report["usable"], 20)
123     draws = pd.read_csv(Path(directory) / "bootstrap_draws.csv")
124     np.testing.assert_allclose(table["std_error"], draws[["a", "b"]].std(ddof=1))
125     self.assertTrue((table["std_error"] > 0).all())
```

Read this block:

- Line 110: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 111, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 113: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 118: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 119: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 120, `(table, report)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 121: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 122: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `draws`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.
- `patch`: Temporarily replace/wrap a callable in a test so its invocations can be observed.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd.estimate`. Source begins at line 113.

This nested fake estimator returns an `OptimizeResult` with coefficients equal to resampled residual SDs. It deliberately depends on the recomputed controls, so frozen controls would change the bootstrap distribution. It declares success and a harmless objective because the test targets bootstrap orchestration, not numerical optimization. Mocking the demand fit makes this check faster and isolates whether first stages are rerun and empirical SEs calculated correctly.

Definition and stated intent:

Source lines 113–115:

```
113 def estimate(sample, initial):
114     # Explicitly depend on the recomputed residual, so frozen residuals
115     # would produce a different bootstrap distribution.
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 116–116:

```
116 theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
```

Read this block:

- Line 116, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.

Source lines 117–117:

```
117 return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
```

Read this block:

- Line 117: return the result of calling `OptimizeResult(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.

PriceControlChecks.test controlled demand optimizer runs

Original function: `PriceControlChecks.test_controlled_demand_optimizer_runs`. Source begins at line 127.

What this test establishes. Fits a small controlled naive demand model and checks convergence, a four-coefficient vector, and finite objective. This is a controlled optimizer smoke test, not an assertion about private-data fit quality.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 127–127:

```
127 def test_controlled_demand_optimizer_runs(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 128–133:

```
128 controlled, _ = add_price_controls(synthetic_panel())
129 args = SimpleNamespace(model="naive", draws=8, seed=306, starts=2, maxiter=150)
130 fit, _, _ = optimize_demand(make_arrays(controlled, 0), args, quiet=True)
131 self.assertTrue(fit.success)
132 self.assertEqual(len(fit.x), 4)  # Two betas, mean log(-alpha), price control.
133 self.assertTrue(np.isfinite(fit.fun))
```

Read this block:

- Line 128, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 129, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 130, `(fit, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 131: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `len`: Count elements/rows rather than summing their numerical values.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/test_regenerate_weekly_inputs.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Recovery control flow tests with generated fixtures only."""
2 import json
3 from pathlib import Path
4 import tempfile
5 import unittest
6 from unittest.mock import patch
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `json`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 import regenerate_weekly_inputs as recovery
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `regenerate_weekly_inputs as recovery`. An as-name is a convenient local alias, not another library.

Source lines 13–13:

```
13 class RecoveryTests(unittest.TestCase):
```

Read this block:

- Line 13: define class `RecoveryTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 53–54:

```
53 if __name__ == '__main__':
54     unittest.main()
```

Read this block:

- Line 53: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

RecoveryTests.check recovery

Original function: `RecoveryTests.check_recovery`. Source begins at line 14.

Builds temporary synthetic fit records and mocks data preparation/CSV loading. It checks successful publication or a likelihood mismatch, and verifies saved fitted parameters are preserved. The mock is a controlled substitute, not evidence that an actual server dataset has already matched.

Definition and stated intent:

Source lines 14–14:

```
14 def check_recovery(self, mismatch):
```

`def` creates this callable; the parentheses name its inputs (`self`, `mismatch`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–44:

```
15 with tempfile.TemporaryDirectory() as temporary:
16     directory = Path(temporary)
17     fit = dict(model='heterogeneous_history', households=1, weeks=2, nll=3., settings=dict(
18         sample='/server/synthetic.parquet', first_year=2006, last_year=2015,
19         max_households=None, capacity=7, no_price_control=False,
20         household_dma_column='dma_cd', habit_weight=.7, draws=4, seed=306))
21     (directory/'fit.json').write_text(json.dumps(fit))
22     names = recovery.parameter_names('heterogeneous_history', True, True)
23     pd.DataFrame(dict(parameter=names, estimate=np.ones(len(names)))).to_csv(directory/'point_estimates.csv',
    ↪  index=False)
24     before = {p.name: p.read_bytes() for p in directory.iterdir()}
25     frame = pd.DataFrame({'synthetic_row': [1, 2]})
26     data = dict(n_households=1, outside=True, control=True)
27     with patch.object(recovery, 'prepare_data', return_value=frame) as prepare, \
28             patch.object(recovery, 'add_price_controls', return_value=(frame, None)) as controls, \
29             patch.object(recovery, 'make_arrays', return_value=data), \
30             patch.object(recovery, 'household_loglike', return_value=np.array([-4. if mismatch else -3.])):
31         if mismatch:
32             (directory/'weekly_inputs.csv').write_text('existing export')
33             with self.assertRaisesRegex(ValueError, 'likelihood'):
34                 recovery.regenerate(directory)
35             self.assertEqual((directory/'weekly_inputs.csv').read_text(), 'existing export')
36             self.assertFalse((directory/'weekly_inputs_regeneration.json').exists())
37         else:
38             report = recovery.regenerate(directory)
39             self.assertFalse(report['demand_refitted'])
40             self.assertEqual(len(pd.read_csv(directory/'weekly_inputs.csv')), 2)
41         controls.assert_called_once()
42         self.assertEqual(prepare.call_args.args[0].sample, Path('/server/synthetic.parquet'))
43     for name, content in before.items():
```

```
44          self.assertEqual((directory/name).read_bytes(), content)
```

Read this block:

- Line 15: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 16, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 21: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22, `names`: Set this name to the result of calling `recovery.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `before`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 25, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 26, `data`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 27: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31: test `mismatch`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: call `(directory / 'weekly_inputs.csv').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 34: call `recovery.regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `report`: Set this name to the result of calling `recovery.regenerate(...)`; the arguments and preceding method chain are shown above.
- Line 39: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `controls.assert_called_once` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: for each entry yielded by `before.items()`, bind/unpack it into `(name, content)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 44: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test recovery preserves parameters

Original function: `RecoveryTests.test_recovery_preserves_parameters`. Source begins at line 46.

Exercises successful weekly-export regeneration with mocks and confirms fit/parameter records remain byte-for-byte unchanged. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 46–46:

```
46 def test_recovery_preserves_parameters(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–47:

```
47 self.check_recovery(False)
```

Read this block:

- Line 47: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test mismatch preserves existing export

Original function: `RecoveryTests.test_mismatch_preserves_existing_export`. Source begins at line 49.

Forces a reconstructed-likelihood mismatch and checks no replacement weekly export or success record is published. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 49–49:

```
49 def test_mismatch_preserves_existing_export(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–50:

```
50 self.check_recovery(True)
```

Read this block:

- Line 50: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/test_summary_stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Checks for the sibling summary script, using synthetic purchases only."""
2 import importlib.util
3 import os
4 from pathlib import Path
5 import tempfile
6 import unittest
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `importlib.util`. An as-name is a convenient local alias, not another library.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: import `unittest`. An as-name is a convenient local alias, not another library.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 import polars as pl
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 13–13:

```
13 class SummaryChecks(unittest.TestCase):
```

Read this block:

- Line 13: define class `SummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 61–62:

```
61 if __name__ == "__main__":
62     unittest.main()
```

Read this block:

- Line 61: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

SummaryChecks.test demographics and mixed baskets

Original function: `SummaryChecks.test_demographics_and_mixed_baskets`. Source begins at line 14.

What this test establishes. Uses tiny tables to check household demographic selection, missing categories, and mixed-basket summary behavior. Explicit source values make it possible to verify denominators by hand.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 14–15:

```
14 def test_demographics_and_mixed_baskets(self):
15     # The override lets a prepared replacement be tested before installation.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 16–19:

```
16 script = Path(os.environ.get("SUMMARY_SCRIPT", Path(__file__).parent / "../data_code/summ_stats.py"))
17 spec = importlib.util.spec_from_file_location("summary", script)
18 summary = importlib.util.module_from_spec(spec)
19 spec.loader.exec_module(summary)
```

Read this block:

- Line 16, `script`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `spec`: Set this name to the result of calling `importlib.util.spec_from_file_location(...)`; the arguments and preceding method chain are shown above.
- Line 18, `summary`: Set this name to the result of calling `importlib.util.module_from_spec(...)`; the arguments and preceding method chain are shown above.
- Line 19: call `spec.loader.exec_module` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 20–58:

```
20 with tempfile.TemporaryDirectory() as directory:
21     root = Path(directory)
22     source = pl.DataFrame({
23         "household_code": [1, 1, 1, 1, 2], "panel_year": [2010]*5,
```

```
        "purchase_date": ["2010-01-02", "2010-01-02", "2010-01-09", "2010-01-23", "2010-01-03"],
        "trip_code_uc": [1, 1, 2, 3, 4], "upc": [1, 2, 2, 3, 1], "quantity": [2, 1, 1, 3, 1],
        "race": [1, 1, 1, 1, 2], "hispanic_origin": [2, 2, 2, 2, 1],
        "household_income": [5, 5, 5, 5, 8], "female_head_age": [3, 3, 3, 3, 0],
        "male_head_age": [4, 4, 4, 4, 2], "household_size": [2]*5})
    source.write_parquet(root / "sample.parquet")
    scan = summary.scan_purchases(root / "sample.parquet", 2006, 2015)
    households =
↪   summary.household_summary(scan).collect(engine="streaming").to_pandas().set_index("household_code")
    self.assertEqual(len(households), 2)
    self.assertEqual(households.loc[1, "trips"], 3)
    self.assertEqual(households.loc[2, "race"], "Hispanic")
    self.assertEqual(households.loc[2, "race_original"], "2")
    summary.demographic_tables(households.reset_index(), root, "test")
    race = pd.read_csv(root / "test_race.csv")
    np.testing.assert_allclose(race["percent_all_households"], 50.)
    reference = pl.DataFrame({"upc": ["1", "2", "3"], "flavor": ["other", "berry", "plain"]})
    weekly, _ = summary.yogurt_weeks(scan, reference)
    self.assertEqual(pd.Timestamp(weekly.filter(pl.col("household_code") ==
↪    2).select("week").collect(engine="streaming").item()), pd.Timestamp("2010-01-09"))
    repeated = summary.repeated_yogurt_households(scan)
    self.assertEqual(summary.household_summary(repeated).collect(engine="streaming")["household_code"].to_list(),
↪    [1])
    # Several purchases in a single week still do not qualify.
    one_week = scan.filter(pl.col("purchase_date") <= pl.date(2010, 1, 3))
    self.assertEqual(summary.repeated_yogurt_households(one_week).collect().height, 0)
    shares, transitions = summary.flavor_summaries(weekly, root)
    summary.make_hhi_figures(root)
    metrics = pd.read_csv(root / "yogurt_household_flavors.csv").set_index("household_code")
    self.assertAlmostEqual(metrics.loc[1, "flavor_HHI"], 17 / 49)
    self.assertEqual(metrics.loc[2, "flavor_HHI"], 1.)
    for kind in ["histogram", "cdf"]:
        self.assertTrue((root / f"flavor_hhi_{kind}.pdf").exists())
    baskets = pd.read_csv(root / "yogurt_weekly_baskets.csv")
    self.assertEqual(baskets.loc[(baskets.household_code == 1) & (baskets.week == "2010-01-02"),
↪    "flavors_in_basket"].iloc[0], 2)
    self.assertTrue(transitions.isna().all().all())
    self.assertAlmostEqual(shares.percent_packages.sum(), 100.)
    self.assertTrue((root / "test_income.tex").exists())
```

Read this block:

- Line 20: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 21, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 22, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 29: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 30, `scan`: Set this name to the result of calling `summary.scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 31, `households`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 32: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 35: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `summary.demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37, `race`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 38: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `reference`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 40, `(weekly, _)`: Unpack the result of calling `summary.yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 41: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42, `repeated`: Set this name to the result of calling `summary.repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45, `one_week`: Set this name to the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above.
- Line 46: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47, `(shares, transitions)`: Unpack the result of calling `summary.flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48: call `summary.make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `metrics`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54, `baskets`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 55: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `len`: Count elements/rows rather than summing their numerical values.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__wtp.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import unittest
2 import numpy as np
3 import pandas as pd
4 from wtp import implied_wtp
```

Read this block:

- Line 1: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 4: from `wtp`, import `implied_wtp` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–7:

```
7 class WTPChecks(unittest.TestCase):
```

Read this block:

- Line 7: define class `WTPChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 50–51:

```
50 if __name__ == '__main__':
51     unittest.main()
```

Read this block:

- Line 50: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

WTPChecks.table

Original function: `WTPChecks.table`. Source begins at line 8.

Builds a small generated coefficient table with the parameter/estimate schema expected by the WTP routines; it performs no file read.

Definition and stated intent:

Source lines 8–8:

```
8 def table(self, **values):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**values` collects additional keyword arguments in a dictionary.

Source lines 9–9:

```
9 return pd.DataFrame(dict(parameter=list(values), estimate=list(values.values())))
```

Read this block:

- Line 9: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `list`: Materialize iterable entries as an ordered Python list.

WTPChecks.test fixed alpha and signed history

Original function: `WTPChecks.test_fixed_alpha_and_signed_history`. Source begins at line 11.

Checks WTP conversion against an elementary fixed-price-sensitivity calculation, preserving the signs of history effects. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 11–11:

```
11 def test_fixed_alpha_and_signed_history(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–18:

```
12 result = implied_wtp(self.table(mu_log_minus_alpha=np.log(2), beta_berry=-4.,
13                                  mean_gamma=-2.), .7).set_index('term')
14 self.assertAlmostEqual(result.loc['beta_berry', 'mean_wtp'], -2.)
15 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'], -1.)
```

```
16 self.assertAlmostEqual(result.loc['gamma_novelty', 'mean_wtp'], -.3)
17 self.assertAlmostEqual(result.loc['gamma_own_stock', 'mean_wtp'], .7)
18 np.testing.assert_allclose(result.population_sd, 0., atol=1e-7)
```

Read this block:

- Line 12, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 14: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 15: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 17: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 18: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test mixture moments include between component variation

Original function: `WTPChecks.test_mixture_moments_include_between_component_variation`. Source begins at line 20.

Checks mixture WTP variance includes differences between component means as well as within-component variances. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 20–20:

```
20 def test_mixture_moments_include_between_component_variation(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 21–29:

```
21 result = implied_wtp(self.table(mu_log_minus_alpha=0., sd_log_minus_alpha=.4,
22     mean_gamma=-1., sd_gamma=.2, gamma_mean_gap=3., sd_gamma_2=.3,
23     mixture_weight=.25)).set_index('term')
24 mean = .25 * -1 + .75 * 2
25 second = .25 * (1 + .2**2) + .75 * (4 + .3**2)
26 expected = mean * np.exp(.4**2/2)
27 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'], expected)
28 self.assertAlmostEqual(result.loc['gamma', 'population_sd'],
29                        np.sqrt(second * np.exp(2*.4**2) - expected**2))
```

Read this block:

- Line 21, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 24, `mean`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 25, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 26, `expected`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 30–32:

```
30 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'],
31     .25 * result.loc['gamma_component_1', 'mean_wtp'] +
32     .75 * result.loc['gamma_component_2', 'mean_wtp'])
```

Read this block:

- Line 30: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test separate history wtp has no fixed weight

Original function: `WTPChecks.test_separate_history_wtp_has_no_fixed_weight`. Source begins at line 34.

Verifies independently estimated novelty and stock coefficients are converted to WTP without applying the old lambda weights. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 34–34:

```
34 def test_separate_history_wtp_has_no_fixed_weight(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–41:

```
35 table = self.table(mu_log_minus_alpha=np.log(2), mean_novelty=.6,
36                    mean_own_stock=-.8, sd_own_stock=.4)
37 result = implied_wtp(table, .1).set_index("term")
38 self.assertAlmostEqual(result.loc["novelty", "mean_wtp"], .3)
39 self.assertAlmostEqual(result.loc["own_stock", "mean_wtp"], -.4)
```

```
40 self.assertAlmostEqual(result.loc["own_stock", "population_sd"], .2)
41 pd.testing.assert_frame_equal(implied_wtp(table, .1), implied_wtp(table, .9))
```

Read this block:

- Line 35, `table`: Set this name to the result of calling `self.table(...)`; the arguments and preceding method chain are shown above.
- Line 37, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 38: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `pd.testing.assert_frame_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_frame_equal`: Compare pandas tables including values, labels, and dtype conventions rather than only row counts.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test beta averages ratios

Original function: `WTPChecks.test_beta_averages_ratios`. Source begins at line 43.

Checks beta WTP averages inverse price sensitivities rather than using the inverse of their mean. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 43–43:

```
43 def test_beta_averages_ratios(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 44–47:

```
44 result = implied_wtp(self.table(mu_log_minus_alpha=.2, sd_log_minus_alpha=.5,
45                                  beta_plain=2.)).iloc[0]
46 self.assertAlmostEqual(result.mean_wtp, 2 * np.exp(-.2 + .5**2/2))
47 self.assertGreater(result.population_sd, 0.)
```

Read this block:

- Line 44, `result`: Set this name to the entry/slice selected by `implied_wtp(self.table(mu_log_minus_alpha=0.2, sd_log_min...`; brackets select from the object to their left.
- Line 46: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 47: call `self.assertGreater` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/checks/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/checks/check_alpha.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__beta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__delta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__focs.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__gamma.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.

- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__numerical.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import NumericalChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `NumericalChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__pricing__derivatives.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply_code/checks/test_symmetric.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbolic identities and independent reoptimized finite differences."""
import unittest
import numpy as np
import sympy as sp
from scipy.optimize import minimize_scalar
from model.symbols import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 4: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.optimize`, import `minimize_scalar` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
from model.utility import raw_utilities, U1, U2
from model.shares import sj1_expr, sj2_expr
from model.profit import Pi
from pricing.period1_foc import foc1_collected
from pricing.period2_foc import foc2
```

```
12 from comparative_statics import canonical as C
```

Read this block:

- Line 7: from `model.utility`, import `raw_utilities, U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `model.shares`, import `sj1_expr, sj2_expr` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–14:

```
13 from model.numerical import evaluate, inside_share
14 import structural_switching_reference as reference
```

Read this block:

- Line 13: from `model.numerical`, import `evaluate, inside_share` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `structural_switching_reference as reference`. An as-name is a convenient local alias, not another library.

Source lines 17–17:

```
17 class SymbolicChecks(unittest.TestCase):
```

Read this block:

- Line 17: define class `SymbolicChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 57–57:

```
57 class NumericalChecks(unittest.TestCase):
```

Read this block:

- Line 57: define class `NumericalChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 134–135:

```
134 if __name__=='__main__':
135     unittest.main()
```

Read this block:

- Line 134: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 135: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

SymbolicChecks.equal

Original function: `SymbolicChecks.equal`. Source begins at line 18.

This helper asserts symbolic equality by simplifying the difference to zero. `self` supplies the current unittest instance, so `self.assertEqual` records a normal test failure instead of only printing a warning. Structural `left == right` may fail for algebraically equivalent expressions written in different forms. Simplifying their difference is a stronger algebraic comparison for these expressions, though symbolic simplification is not a universal theorem prover.

Definition and stated intent:

Source lines 18–18:

```
18 def equal(self,left,right):
```

`def` creates this callable; the parentheses name its inputs (`self`, `left`, `right`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 self.assertEqual(sp.simplify(left-right),0)
```

Read this block:

- Line 19: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test raw normalization and both states

Original function: `SymbolicChecks.test_raw_normalization_and_both_states`. Source begins at line 21.

What this test establishes. Evaluates both signed histories and symbolically verifies subtracting raw outside utility yields beta+alpha*price*+*q*gamma. This catches accidentally deleting outside switching utility.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 21–21:

```
21 def test_raw_normalization_and_both_states(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–24:

```
22 for history in [-1,1]:
23     a,b = raw_utilities(p1,history)
24     self.equal(a-b,beta+alpha*p1+history*gamma)
```

Read this block:

- Line 22: for each entry yielded by `[-1, 1]`, bind/unpack it into `history` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 23, `(a, b)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 24: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 25–25:

```
25 self.equal(U1,beta+alpha*p1+q*gamma)
```

Read this block:

- Line 25: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test focs from actual profit

Original function: `SymbolicChecks.test_focs_from_actual_profit`. Source begins at line 27.

What this test establishes. Differentiates literal symbolic profits and compares them to stored FOCs, including both continuation weights. It checks the objective behind the equations rather than merely rearranging a copied FOC.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 27–27:

```
27 def test_focs_from_actual_profit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 28–33:

```
28 literal = Pi.subs(sj1,sj1_expr)
29 expected = foc1_collected.subs(sj1,sj1_expr)
30 self.equal(sp.diff(literal,p1),expected)
31 self.equal(sp.diff((p2-c)*sj2_expr,p2),foc2.subs(sj2,sj2_expr))
32 self.equal(sp.diff(literal,Vj),delta*sj1_expr)
33 self.equal(sp.diff(literal,V0),delta*(1-sj1_expr))
```

Read this block:

- Line 28, `literal`: Set this name to the result of calling `Pi.subs(...)`; the arguments and preceding method chain are shown above.
- Line 29, `expected`: Set this name to the result of calling `foc1_collected.subs(...)`; the arguments and preceding method chain are shown above.
- Line 30: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 31: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 32: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 34–34:

```
34 self.equal(sp.diff(Vj,p1),0)
```

Read this block:

- Line 34: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test envelope and implicit gamma derivatives

Original function: `SymbolicChecks.test_envelope_and_implicit_gamma_derivatives`. Source begins at line 36.

What this test establishes. Verifies explicit gamma price/profit/curvature identities and total derivatives of the implicit FOC. Algebraic comparison catches missing chain-rule and continuation terms.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 36–36:

```
36 def test_envelope_and_implicit_gamma_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–43:

```
37 self.equal(C.dp1_dgamma,-(q*sj1+delta*(1-sj1)*(u+v))/alpha)
38 self.equal(C.dPi_dgamma,(-q*sj1+delta*(sj1*u-(1-sj1)*v))/alpha)
39 expected2 = -(sj1*(1-sj1)**2*(q-delta*(u+v))**2
40              +delta*(1-sj1)*(-u*(1-u)**2+v*(1-v)**2))/alpha
41 self.equal(C.d2p1_dgamma2,expected2)
42 self.equal(C.ds_dgamma,sj1*(1-sj1)**2*(q-delta*(u+v)))
43 self.equal(C.dVj_dgamma,u/alpha)
```

Read this block:

- Line 37: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `expected2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 41: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 44–49:

```
44 self.equal(C.dV0_dgamma,-v/alpha)
45 # Differentiate implicit pricing equations with endogenous shares.
46 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
47 self.equal(C.total_derivative(F,gamma),0)
48 self.equal(sp.diff(p2-c+1/(alpha*(1-sj2)),p2)
49           +sp.diff(p2-c+1/(alpha*(1-sj2)),sj2)*alpha*sj2*(1-sj2),1/(1-sj2))
```

Read this block:

- Line 44: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test all parameter implicit derivatives

Original function: `SymbolicChecks.test_all_parameter_implicit_derivatives`. Source begins at line 51.

What this test establishes. Loops through alpha, beta, delta, and cost and verifies that differentiating the equilibrium pricing identity along each solution gives zero. This extends checks beyond the headline gamma plots.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 51–51:

```
51 def test_all_parameter_implicit_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 52–52:

```
52 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
```

Read this block:

- Line 52, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 53–54:

```
53 for theta in [alpha,beta,delta,c]:
54     self.equal(C.total_derivative(F,theta),0)
```

Read this block:

- Line 53: for each entry yielded by `[alpha, beta, delta, c]`, bind/unpack it into `theta` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 54: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

NumericalChecks.test reference on entire benchmark grid

Original function: `NumericalChecks.test_reference_on_entire_benchmark_grid`. Source begins at line 58.

What this test establishes. Solves both histories at all 601 gamma grid points and compares named numerical outputs against the independently coded reference. It also checks original FOC residual magnitudes.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 58–58:

```
58 def test_reference_on_entire_benchmark_grid(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–64:

```
59 for g in np.linspace(-3,3,601):
60     for state in ['j','0']:
61         ours, original = evaluate(float(g),state),reference.evaluate(float(g),state)
62         for key in ['p1','p2j','p20','profit','dp1','d2p1','dprofit','d_matched_ratio']:
63             self.assertAlmostEqual(ours[key],original[key],places=9)
64         self.assertLess(max(abs(ours[k]) for k in ['foc1','foc2j','foc20']),1e-10)
```

Read this block:

- Line 59: for each entry yielded by `np.linspace(-3, 3, 601)`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 61, `(ours, original)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 62: for each entry yielded by `['p1', 'p2j', 'p20', 'profit', 'dp1', 'd2p1', 'dprofit', 'd_ma...`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 63: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

NumericalChecks.test independent reoptimization and derivatives

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives`. Source begins at line 66.

What this test establishes. At several calibrations compares analytic slopes and curvature with finite differences of fully reoptimized solutions, ratio derivatives with recomputed ratios, and p1 with a separate direct profit optimizer. Different computational routes can catch shared-expression mistakes.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 66–66:

```
66 def test_independent_reoptimization_and_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 67–67:

```
67 h=1e-4
```

Read this block:

- Line 67, `h`: Set this name to the literal value `0.0001`.

Source lines 68–91:

```
68 for parameters in [dict(beta=2,alpha=-1,delta=.92,c=0),
69                    dict(beta=-1,alpha=-2,delta=.7,c=1),
70                    dict(beta=1.5,alpha=-.7,delta=.4,c=.3)]:
71     for g in [-2.4,-1.2,0,.7,2.6]:
72         for state in ['j','0']:
73             r=evaluate(g,state,**parameters)
74             plus,minus=evaluate(g+h,state,**parameters),evaluate(g-h,state,**parameters)
75             for derivative,level in [('dp1','p1'),('dprofit','profit'),('dp2j','p2j'),('dp20','p20')]:
76                 self.assertAlmostEqual(r[derivative],(plus[level]-minus[level])/(2*h),delta=3e-7)
```

```
77            self.assertAlmostEqual(r['d2p1'],(plus['dp1']-minus['dp1'])/(2*h),delta=3e-7)
78            self.assertAlmostEqual(r['d2p1'],(plus['p1']-2*r['p1']+minus['p1'])/h**2,delta=3e-5)
79            denominator='p2j' if state=='j' else 'p20'
80            dr=(plus['p1']/plus[denominator]-minus['p1']/minus[denominator])/(2*h)
81            self.assertAlmostEqual(r['d_matched_ratio'],dr,delta=3e-7)
82            if state=='0':
83                self.assertAlmostEqual(r['d_induce_ratio'],(plus['p1']/plus['p2j']-minus['p1']/minus['p2j'])/(2*
                  ↪ h),delta=3e-7)
84            self.assertAlmostEqual(r['p1'],r['static_component']+r['continuation_adjustment'],places=10)
85            # Independent optimizer of the complete two-branch profit.
86            def negative_profit(price):
87                share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
88                return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
89            optimum=minimize_scalar(negative_profit,bounds=(r['p1']-10,r['p1']+10),method='bounded')
90            self.assertAlmostEqual(optimum.x,r['p1'],delta=1e-5)
91            self.assertLess(negative_profit(r['p1']),(negative_profit(r['p1']+h)+negative_profit(r['p1']-h))/2)
```

Read this block:

- Line 68: for each entry yielded by `[dict(beta=2, alpha=-1, delta=0.92, c=0), dict(beta=-1, alpha=...`, bind/unpack it into `parameters` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 74, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 75: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit'), ('dp2j', 'p2j'), ('dp20...`, bind/unpack it into `(derivative, level)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79, `denominator`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.
- Line 80, `dr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 83: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: define nested callable `negative_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 89, `optimum`: Set this name to the result of calling `minimize_scalar(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `minimize_scalar`: Numerically minimize a one-variable objective, used as an independent check of price optimality.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `negative_profit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

NumericalChecks.test independent reoptimization and derivatives.negative profit

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives.negative_profit`. Source begins at line 86.

This nested objective accepts a candidate first-period price, computes its logit share, and returns the negative of complete two-branch discounted profit. It captures current gamma, state, calibration, and solved continuation values from the enclosing test loop. `minimize_scalar` minimizes, so negating profit turns maximization into minimization. Using a fresh direct objective here checks the production root against another computational route.

Definition and stated intent:

Source lines 86–86:

```
86 def negative_profit(price):
```

`def` creates this callable; the parentheses name its inputs (`price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 87–87:

```
87 share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
```

Read this block:

- Line 87, `share`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 88–88:

```
88 return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
```

Read this block:

- Line 88: return the negated or sign-adjusted expression `-(share * (price - parameters['c']) + parameters['delta']...`. This ends the call.

NumericalChecks.test all parameter and mixed derivatives numerically

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically`. Source begins at line 93.

What this test establishes. Lambdifies general parameter derivatives, perturbs alpha/beta/delta/c separately, and compares finite differences. It also checks a beta-gamma mixed derivative using reoptimized results.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 93–93:

```
93 def test_all_parameter_and_mixed_derivatives_numerically(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–95:

```
94 parameters=dict(beta=2.,alpha=-1.,delta=.92,c=0.)
95 arguments=(alpha,beta,gamma,delta,c,q,sj1,u,v,p1,p2j,p20,Vj,V0)
```

Read this block:

- Line 94, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 95, `arguments`: Set this name to a tuple containing 14 displayed entries.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 96–98:

```
96 def arguments_at(result):
97     return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98
 ↪  result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

Read this block:

- Line 96: define nested callable `arguments_at`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 99–112:

```
99 for theta,name in [(alpha,'alpha'),(beta,'beta'),(delta,'delta'),(c,'c')]:
100     expression=C.price1_derivative(theta)
101     terminal=C.price2_derivative(theta,u,-1,p2j)
102     evaluator=sp.lambdify(arguments,[expression,terminal],'numpy')
103     for state in ['j','0']:
104         for g in [-1.,1.]:
105             result=evaluate(g,state,**parameters)
106             h=1e-5
107             plus_parameters=dict(parameters);plus_parameters[name]+=h
108             minus_parameters=dict(parameters);minus_parameters[name]-=h
109             plus=evaluate(g,state,**plus_parameters);minus=evaluate(g,state,**minus_parameters)
110             analytical=evaluator(*arguments_at(result))
111             self.assertAlmostEqual(float(analytical[0]),(plus['p1']-minus['p1'])/(2*h),delta=2e-6)
112             self.assertAlmostEqual(float(analytical[1]),(plus['p2j']-minus['p2j'])/(2*h),delta=2e-6)
```

Read this block:

- Line 99: for each entry yielded by `[(alpha, 'alpha'), (beta, 'beta'), (delta, 'delta'), (c, 'c')]`, bind/unpack it into `(theta, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 100, `expression`: Set this name to the result of calling `C.price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 101, `terminal`: Set this name to the result of calling `C.price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 102, `evaluator`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 103: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104: for each entry yielded by `[-1.0, 1.0]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 105, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 106, `h`: Set this name to the literal value `1e-05`.
- Line 107, `plus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: update `plus_parameters[name]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 108, `minus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 108: update `minus_parameters[name]` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 109, `plus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 109, `minus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 110, `analytical`: Set this name to the result of calling `evaluator(...)`; the arguments and preceding method chain are shown above.
- Line 111: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `evaluator`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `arguments_at`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 113–114:

```
113 mixed=sp.lambdify(arguments,C.total_derivative(C.price1_derivative(beta),gamma),'numpy')
114 slope=sp.lambdify(arguments,C.price1_derivative(beta),'numpy')
```

Read this block:

- Line 113, `mixed`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 114, `slope`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Source lines 115–119:

```
115 for state in ['j','0']:
116     g=.7;h=1e-4
117     actual=mixed(*arguments_at(evaluate(g,state)))
118     expected=(slope(*arguments_at(evaluate(g+h,state)))-slope(*arguments_at(evaluate(g-h,state))))/(2*h)
119     self.assertAlmostEqual(float(actual),float(expected),delta=2e-6)
```

Read this block:

- Line 115: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 116, `g`: Set this name to the literal value `0.7`.
- Line 116, `h`: Set this name to the literal value `0.0001`.
- Line 117, `actual`: Set this name to the result of calling `mixed(...)`; the arguments and preceding method chain are shown above.
- Line 118, `expected`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 119: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `mixed`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `slope`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

NumericalChecks.test all parameter and mixed derivatives numerically.arguments at

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically.arguments_at`. Source begins at line 96.

This nested adapter turns a result dictionary into the exact tuple expected by lambdified symbolic derivatives. The q sign uses initial history, and branch prices/shares/values retain their order. Its benchmark constants deliberately match the enclosing test calibration. A tuple is appropriate because the numerical evaluator uses positional arguments; a dictionary would require explicit ordered extraction anyway. Swapping u/v could make a wrong formula look correct at the symmetric gamma=0 point, so tests also use nonzero gamma.

Definition and stated intent:

Source lines 96–96:

```
96 def arguments_at(result):
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 97–98:

```
97 return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98         result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

Read this block:

- Line 97: return a tuple of 14 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

NumericalChecks.test zero switching static case and symmetry

Original function: `NumericalChecks.test_zero_switching_static_case_and_symmetry`. Source begins at line 121.

What this test establishes. Checks gamma=0 branch equality, delta=0 equality with the true static counterfactual, terminal history/sign symmetry, and an admissible negative introductory price. These limit cases catch state and price-floor regressions.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 121–121:

```
121 def test_zero_switching_static_case_and_symmetry(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–127:

```
122 for state in ['j','0']:
123     r=evaluate(0,state)
124     self.assertAlmostEqual(r['p1'],r['p2j'])
125     self.assertAlmostEqual(r['p2j'],r['p20'])
126     static=evaluate(2,state,delta=0)
127     self.assertAlmostEqual(static['p1'],static['static_counterfactual'])
```

Read this block:

- Line 122: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 123, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126, `static`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 127: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 128–129:

```
128 for g in [-3,-1,1,3]:
129     self.assertAlmostEqual(evaluate(g,'j')['p2j'],evaluate(-g,'0')['p20'])
```

Read this block:

- Line 128: for each entry yielded by `[-3, -1, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 129: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 131–131:

```
131 self.assertLess(evaluate(-8,'0')['p1'],0)
```

Read this block:

- Line 131: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises/test_empirical_timing.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 """Synthetic timing and backward-induction checks; no CSV input or output."""
2 import math
3 import unittest
4
5 import numpy as np
6 import pandas as pd
7
8 from pricing_exercises import run_empirical_targeting as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: from `pricing_exercises`, import `run_empirical_targeting as pricing` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 11–11:

```
11 class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 95–96:

```
95 if __name__ == "__main__":
96     unittest.main()
```

Read this block:

- Line 95: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 96: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
12 def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
28 frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
29 households = pd.DataFrame({"household_code": ["b", "a"]})
30 context = pricing.forecast_context(frame, households)
31 np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
32 np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
33 np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
34 np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
35 np.testing.assert_array_equal(context["eligible"], [False, True])
36 self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
37 self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
```

```
38 self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
39 later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
40 changed = frame.copy()
41 changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
42 for prefix in ["price_", "cf_"]:
43     changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
44 repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
45 for key in context:
46     np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–61:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 beta = np.ones(3)*.5
59 table, error = pricing.targeted_coupons(
60     context, households, np.ones((1, 1)), np.array([a]), np.array([g]),
61     beta, discounts, capacity=capacity, delta=delta)
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 59, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 62–64:

```
62 self.assertLess(error, 1e-12)
63 baskets = pricing.compositions(capacity)
64 terminal_values = []
```

Read this block:

- Line 62: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 63, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 64, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 65–75:

```
65 for basket in baskets:
66     x = basket[:3]/max(basket[:3].sum(), 1)
67     xi = .3*(x.sum()-x)-.7*(context["stocks"][0]+x)
68     values = []
69     for discount in discounts:
70         prices = context["prices"][0].copy()
71         prices[1] *= 1-discount
72         utilities = np.r_[beta-a*prices+g*xi, 0.]
73         probability = np.exp(utilities-pricing.logsumexp(utilities))
74         values.append(capacity*prices[1]*probability[1])
75     terminal_values.append(max(values))
```

Read this block:

- Line 65: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 66, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 67, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `values`: Set this name to the list `[]`.
- Line 69: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 70, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 71: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 72, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.

- Line 73, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 74: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 75: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 76–76:

```
76 total_values = []
```

Read this block:

- Line 76, `total_values`: Set this name to the list `[]`.

Source lines 77–87:

```
77 for discount in discounts:
78     prices = context["prices"][0].copy()
79     prices[1] *= 1-discount
80     xi = .3*context["novelty"][0]-.7*context["stocks"][0]
81     utilities = np.r_[beta-a*prices+g*xi, 0.]
82     probability = np.exp(utilities-pricing.logsumexp(utilities))
83     first = capacity*prices[1]*probability[1]
84     mass = np.array([
85         math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
86         *np.prod(probability**basket) for basket in baskets])
87     total_values.append(first+delta*(mass @ terminal_values))
```

Read this block:

- Line 77: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 78, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 79: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 80, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.
- Line 82, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 83, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 87: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 88–92:

```
88 policies = table.set_index("policy")
89 self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
90 dynamic = policies.loc["personalized_dynamic"]
91 self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
92 self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 88, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`; brackets select from the object to their left.
- Line 91: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises/test_split_targeting.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 """Synthetic timing and backward-induction checks; no CSV input or output."""
2 import math
3 import unittest
4
5 import numpy as np
6 import pandas as pd
7
8 import run_split_targeting as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: import `run_split_targeting as pricing`. An as-name is a convenient local alias, not another library.

Source lines 11–11:

```
11 class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 97–97:

```
97 class SplitPosteriorTests(unittest.TestCase):
```

Read this block:

- Line 97: define class `SplitPosteriorTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 203–204:

```
203 if __name__ == "__main__":
204     unittest.main()
```

Read this block:

- Line 203: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 204: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
12 def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
28 frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
29 households = pd.DataFrame({"household_code": ["b", "a"]})
30 context = pricing.forecast_context(frame, households)
31 np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
32 np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
33 np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
np.testing.assert_array_equal(context["eligible"], [False, True])
self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
changed = frame.copy()
changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.

- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
42 for prefix in ["price_", "cf_"]:
43     changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
44 repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
45 for key in context:
46     np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–59:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 b_n, b_s = -.6, .35
59 beta = np.ones(3)*.5
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `(b_n, b_s)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 59, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 60–65:

```
60 table, error = pricing.targeted_coupons(
61     context, households, np.ones((1, 1)), np.array([a]), np.array([[b_n, b_s]]),
62     beta, discounts, capacity=capacity, delta=delta)
63 self.assertLess(error, 1e-12)
64 baskets = pricing.compositions(capacity)
65 terminal_values = []
```

Read this block:

- Line 60, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 63: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 65, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 66–76:

```
66 for basket in baskets:
67     x = basket[:3]/max(basket[:3].sum(), 1)
```

```
    history = b_n*(x.sum()-x)+b_s*(context["stocks"][0]+x)
    values = []
    for discount in discounts:
        prices = context["prices"][0].copy()
        prices[1] *= 1-discount
        utilities = np.r_[beta-a*prices+history, 0.]
        probability = np.exp(utilities-pricing.logsumexp(utilities))
        values.append(capacity*prices[1]*probability[1])
    terminal_values.append(max(values))
```

Read this block:

- Line 66: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 67, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `values`: Set this name to the list `[]`.
- Line 70: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 72: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 73, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 74, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 75: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 77–77:

```
total_values = []
```

Read this block:

- Line 77, `total_values`: Set this name to the list `[]`.

Source lines 78–88:

```
for discount in discounts:
    prices = context["prices"][0].copy()
    prices[1] *= 1-discount
    history = b_n*context["novelty"][0]+b_s*context["stocks"][0]
    utilities = np.r_[beta-a*prices+history, 0.]
    probability = np.exp(utilities-pricing.logsumexp(utilities))
```

```
    first = capacity*prices[1]*probability[1]
    mass = np.array([
        math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
        *np.prod(probability**basket) for basket in baskets])
    total_values.append(first+delta*(mass @ terminal_values))
```

Read this block:

- Line 78: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 80: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 81, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 83, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 84, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 88: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 89–93:

```
policies = table.set_index("policy")
self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
dynamic = policies.loc["personalized_dynamic"]
self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 89, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`; brackets select from the object to their left.
- Line 92: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.fixture

Original function: `SplitPosteriorTests.fixture`. Source begins at line 98.

Constructs tiny, explicitly generated household histories and coefficient dictionaries for checks. These values are inputs chosen to expose errors, not estimates from the private sample.

Definition and stated intent:

Source lines 98–98:

```
98 def fixture(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 99–99:

```
99 rows = []
```

Read this block:

- Line 99, `rows`: Set this name to the list `[]`.

Source lines 100–112:

```
100 for i in range(8):
101     stock = np.zeros(3)
102     previous = np.zeros(3)
103     for t in range(3):
104         counts = np.eye(3)[(i+t) % 3]
105         novelty = previous.sum()-previous
106         row = dict(household_code=str(i), week=pd.Timestamp('2020-01-04')+pd.Timedelta(weeks=t), outside=0.)
107         for j, f in enumerate(pricing.FLAVORS):
108             row.update({f: counts[j], 'price_'+f: 1.+j/2,
109                         'novelty_'+f: novelty[j], 'stock_'+f: stock[j]})
110         rows.append(row)
111         stock += counts
112         previous = counts
```

Read this block:

- Line 100: for each entry yielded by `range(8)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 101, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 102, `previous`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 103: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104, `counts`: Set this name to the entry/slice selected by `np.eye(3)[(i + t) % 3]`; brackets select from the object to their left.
- Line 105, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 106, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 108: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 110: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 111: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 112, `previous`: Set this name to the object currently named `counts`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `str`: Convert to text for paths, messages, or serialization.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 113–115:

```
113 parameters = dict(inside_constant=.5, beta_berry=.2, beta_plain=-.1,
114                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
115                   mean_novelty=-.2, sd_novelty=.7, mean_own_stock=.1, sd_own_stock=.3)
```

Read this block:

- Line 113, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 116–116:

```
116 return pd.DataFrame(rows), parameters
```

Read this block:

- Line 116: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

SplitPosteriorTests.test posterior matches scalar and estimator

Original function: `SplitPosteriorTests.test_posterior_matches_scalar_and_estimator`. Source begins at line 118.

Compares persistent split-draw likelihoods and posterior weights against a scalar calculation and the demand estimator, and verifies exact binary aggregation by an outside inclusive value. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 118–118:

```
118 def test_posterior_matches_scalar_and_estimator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 119–122:

```
frame, p = self.fixture()
draws = pricing.persistent_draws(8, 306)
households, weights, alpha, history, beta, nll = pricing.posterior(frame, p, draws, False)
scalar = []
```

Read this block:

- Line 119, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 120, `draws`: Set this name to the result of calling `pricing.persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 121, `(households, weights, alpha, history, beta, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `scalar`: Set this name to the list `[]`.

Source lines 123–133:

```
for _, block in frame.groupby('household_code', sort=True):
    logs = []
    for a, (b_n, b_s) in zip(alpha, history):
        ll = 0.
        for _, row in block.iterrows():
            u = [beta[j]-a*row['price_'+f]+b_n*row['novelty_'+f]+b_s*row['stock_'+f]
                 for j, f in enumerate(pricing.FLAVORS)]+[0.]
            probs = np.exp(u-pricing.logsumexp(u))
            ll += sum(row[f]*np.log(probs[j]) for j, f in enumerate(pricing.FLAVORS+['outside']))
        logs.append(ll)
    scalar.append(logs)
```

Read this block:

- Line 123: for each entry yielded by `frame.groupby('household_code', sort=True)`, bind/unpack it into `(_, block)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `logs`: Set this name to the list `[]`.
- Line 125: for each entry yielded by `zip(alpha, history)`, bind/unpack it into `(a, (b_n, b_s))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 126, `ll`: Set this name to the literal value `0.0`.
- Line 127: for each entry yielded by `block.iterrows()`, bind/unpack it into `(_, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 130, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 131: update `ll` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 132: call `logs.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `scalar.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 134–139:

```
scalar = np.array(scalar)
expected = np.exp(scalar-pricing.logsumexp(scalar, axis=1, keepdims=True))
np.testing.assert_allclose(weights, expected, atol=1e-13)
self.assertAlmostEqual(nll, -sum(pricing.logsumexp(row)-np.log(len(draws)) for row in scalar), places=12)
np.testing.assert_allclose(households.habit_mean, weights @ history[:, 1])
import sys
```

Read this block:

- Line 134, `scalar`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 135, `expected`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 136: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 137: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: import `sys`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `len`: Count elements/rows rather than summing their numerical values.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 140–142:

```
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]/"est_code"))
import est
```

Read this block:

- Line 140: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 141: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 142: import `est`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 143–144:

```
for f in pricing.FLAVORS:
    frame["xi_"+f] = .3*frame["novelty_"+f]-.7*frame["stock_"+f]
```

Read this block:

- Line 143: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `frame['xi_' + f]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 145–152:

```
data = est.make_arrays(frame, 1)
names = est.parameter_names("heterogeneous_history", True, False)
estimator_draws = est.make_draws(8, 306, "heterogeneous_history")
np.testing.assert_array_equal(draws, estimator_draws)
expected_ll = est.household_loglike(np.array([p[n] for n in names]), data,
                                    estimator_draws, "heterogeneous_history")
np.testing.assert_allclose(pricing.logsumexp(scalar, axis=1)-np.log(len(draws)),
                           expected_ll, atol=1e-12)
```

Read this block:

- Line 145, `data`: Set this name to the result of calling `est.make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 146, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 147, `estimator_draws`: Set this name to the result of calling `est.make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 148: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 149, `expected_ll`: Set this name to the result of calling `est.household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 151: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.

Source lines 155–156:

```
u = np.array([.3, -.2, .7, 0.])
probs = np.exp(u-pricing.logsumexp(u))
```

Read this block:

- Line 155, `u`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 156, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Source lines 157–159:

```
for focal in range(3):
    inclusive = pricing.logsumexp(np.delete(u, focal))
    self.assertAlmostEqual(pricing.expit(u[focal]-inclusive), probs[focal])
```

Read this block:

- Line 157: for each entry yielded by `range(3)`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 158, `inclusive`: Set this name to the result of calling `pricing.logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 159: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.test all focal exports and integrated types

Original function: `SplitPosteriorTests.test_all_focal_exports_and_integrated_types`. Source begins at line 161.

Runs all focal flavors under both type treatments with generated temporary inputs and verifies policy dominance, likelihood guards, outputs and the all-flavor entry point. This complete export check reads only its own synthetic CSVs. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 161–161:

```
161 def test_all_focal_exports_and_integrated_types(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 162–167:

```
162 import tempfile
163 import json
164 from pathlib import Path
165 from types import SimpleNamespace
166 frame, p = self.fixture()
167 _, _, _, _, _, nll = pricing.posterior(frame, p, pricing.persistent_draws(8, 306), False)
```

Read this block:

- Line 162: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 163: import `json`. An as-name is a convenient local alias, not another library.
- Line 164: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 165: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 166, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 167, `(_, _, _, _, _, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 168–199:

```
168 with tempfile.TemporaryDirectory() as directory:
169     directory = Path(directory)
170     frame.to_csv(directory/'weekly_inputs.csv', index=False)
171     pd.DataFrame(dict(parameter=list(p), estimate=list(p.values()),
    ↪   at_bound=False)).to_csv(directory/'point_estimates.csv', index=False)
172     (directory/'fit.json').write_text(json.dumps(dict(model='heterogeneous_history', converged=True,
173         nll=nll, weeks=len(frame), households=8,
174         settings=dict(capacity=1, no_price_control=True, draws=8, seed=306))))
```

```
    for focal in pricing.FLAVORS:
        for treatment in ['posterior-mean', 'posterior-integrated']:
            output = directory/focal/treatment
            output.mkdir(parents=True)
            args = SimpleNamespace(fit_dir=directory, household_input=directory/'weekly_inputs.csv',
                output=output, focal_flavor=focal, discounts=np.array([0., .5, 1.]),
                demand_delta=.95, batch_size=3, type_treatment=treatment, save_state_policies=True)
            report = pricing.run_households(args)
            self.assertLess(report['basket_mass_error'], 1e-12)
            result = pd.read_csv(output/'targeted_household_results.csv')
            self.assertTrue((result.revenue_change >= -1e-10).all())
            self.assertEqual(set(result.focal_flavor), {focal})
            self.assertIn('focal_units1', result)
    from unittest.mock import patch
    import sys
    output = directory/'all_focal_cli'
    with patch.object(sys, 'argv', ['run_split_targeting.py', '--household-input',
            str(directory/'weekly_inputs.csv'), '--fit-dir', str(directory),
            '--output', str(output), '--discounts', '0', '.5', '1']), \
            patch.object(pricing, 'make_targeting_figures'):
        pricing.main()
    comparison = pd.read_csv(output/'focal_flavor_comparison.csv')
    self.assertEqual(set(comparison.focal_flavor), set(pricing.FLAVORS))
    manifest = json.loads((output/'run_manifest.json').read_text())
    self.assertEqual(set(manifest['results']), set(pricing.FLAVORS))
```

Read this block:

- Line 168: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 169, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 170: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 171: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 172: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 176: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 177, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 178: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 179, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 182, `report`: Set this name to the result of calling `pricing.run_households(...)`; the arguments and preceding method chain are shown above.
- Line 183: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 184, `result`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 185: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 187: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 188: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 189: import `sys`. An as-name is a convenient local alias, not another library.
- Line 190, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 191: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 195: call `pricing.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 196, `comparison`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 197: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 198, `manifest`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 199: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `str`: Convert to text for paths, messages, or serialization.
- `len`: Count elements/rows rather than summing their numerical values.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `list`: Materialize iterable entries as an ordered Python list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/checks/test_split_pricing.py

Generated-array checks of split posterior likelihood, exact binary aggregation, timing, independent-coefficient backward induction, and all focal cases under both type treatments. The final_simple check runner blocks CSV readers.

Module setup and direct-execution blocks

Source lines 1–8:

```
"""Synthetic timing and backward-induction checks; no CSV input or output."""
import math
import unittest

import numpy as np
import pandas as pd

from demand import split_pricing as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: from `demand`, import `split_pricing as pricing` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 11–11:

```
class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 115–115:

```
class SplitPosteriorTests(unittest.TestCase):
```

Read this block:

- Line 115: define class `SplitPosteriorTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 196–196:

```
class PlainBerryComparisonTests(unittest.TestCase):
```

Read this block:

- Line 196: define class `PlainBerryComparisonTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 226–227:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 226: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 227: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
12 def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
households = pd.DataFrame({"household_code": ["b", "a"]})
context = pricing.forecast_context(frame, households)
np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
np.testing.assert_array_equal(context["eligible"], [False, True])
self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
40 changed = frame.copy()
41 changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
42 for prefix in ["price_", "cf_"]:
43     changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
44 repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
45 for key in context:
46     np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–59:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 b_n, b_s = -.6, .35
59 beta = np.ones(3)*.5
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `(b_n, b_s)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 59, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 60–66:

```
60 benchmarks = []
61 table, error = pricing.targeted_coupons(
62     context, households, np.ones((1, 1)), np.array([a]), np.array([[b_n, b_s]]),
63     beta, discounts, capacity=capacity, delta=delta, benchmark_output=benchmarks)
64 self.assertLess(error, 1e-12)
```

```
baskets = pricing.compositions(capacity)
terminal_values = []
```

Read this block:

- Line 60, `benchmarks`: Set this name to the list `[]`.
- Line 61, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 64: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 66, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 67–77:

```
for basket in baskets:
    x = basket[:3]/max(basket[:3].sum(), 1)
    history = b_n*(x.sum()-x)+b_s*(context["stocks"][0]+x)
    values = []
    for discount in discounts:
        prices = context["prices"][0].copy()
        prices[1] *= 1-discount
        utilities = np.r_[beta-a*prices+history, 0.]
        probability = np.exp(utilities-pricing.logsumexp(utilities))
        values.append(capacity*prices[1]*probability[1])
    terminal_values.append(max(values))
```

Read this block:

- Line 67: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 68, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 70, `values`: Set this name to the list `[]`.
- Line 71: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 73: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 74, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 75, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 76: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 78–79:

```
78 total_values = []
79 common_values = []
```

Read this block:

- Line 78, `total_values`: Set this name to the list `[]`.
- Line 79, `common_values`: Set this name to the list `[]`.

Source lines 80–103:

```
80 for discount in discounts:
81     prices = context["prices"][0].copy()
82     prices[1] *= 1-discount
83     history = b_n*context["novelty"][0]+b_s*context["stocks"][0]
84     utilities = np.r_[beta-a*prices+history, 0.]
85     probability = np.exp(utilities-pricing.logsumexp(utilities))
86     first = capacity*prices[1]*probability[1]
87     mass = np.array([
88         math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
89         *np.prod(probability**basket) for basket in baskets])
90     total_values.append(first+delta*(mass @ terminal_values))
91     fixed_terminal = []
92     for terminal_discount in discounts:
93         state_revenues = []
94         for basket in baskets:
95             x = basket[:3]/max(basket[:3].sum(), 1)
96             history2 = b_n*(x.sum()-x)+b_s*(context["stocks"][0]+x)
97             prices2 = context["prices"][0].copy()
98             prices2[1] *= 1-terminal_discount
99             utilities2 = np.r_[beta-a*prices2+history2, 0.]
100            probability2 = np.exp(utilities2-pricing.logsumexp(utilities2))
101            state_revenues.append(capacity*prices2[1]*probability2[1])
102        fixed_terminal.append(first+delta*(mass @ state_revenues))
103    common_values.append(fixed_terminal)
```

Read this block:

- Line 80: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 82: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 83, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 85, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 86, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 87, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method

chain are shown above.
- Line 90: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91, `fixed_terminal`: Set this name to the list `[]`.
- Line 92: for each entry yielded by `discounts`, bind/unpack it into `terminal_discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 93, `state_revenues`: Set this name to the list `[]`.
- Line 94: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 95, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 96, `history2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97, `prices2`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 98: update `prices2[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 99, `utilities2`: Set this name to the entry/slice selected by `np.r_[beta - a * prices2 + history2, 0.0]`; brackets select from the object to their left.
- Line 100, `probability2`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 101: call `state_revenues.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 102: call `fixed_terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 103: call `common_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 104–110:

```
benchmark = pd.DataFrame(benchmarks).set_index("policy")
self.assertAlmostEqual(benchmark.loc["optimized_common_coupons", "discounted_revenue"],
                       np.max(common_values), places=12)
policies = table.set_index("policy")
self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
dynamic = policies.loc["personalized_dynamic"]
self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
```

Read this block:

- Line 104, `benchmark`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 105: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 107, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 108: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`;

brackets select from the object to their left.
- Line 110: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 111–111:

```
111 self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 111: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.fixture

Original function: `SplitPosteriorTests.fixture`. Source begins at line 116.

Constructs tiny, explicitly generated household histories and coefficient dictionaries for checks. These values are inputs chosen to expose errors, not estimates from the private sample.

Definition and stated intent:

Source lines 116–116:

```
116 def fixture(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 117–117:

```
117 rows = []
```

Read this block:

- Line 117, `rows`: Set this name to the list `[]`.

Source lines 118–130:

```
118 for i in range(8):
119     stock = np.zeros(3)
120     previous = np.zeros(3)
121     for t in range(3):
122         counts = np.eye(3)[(i+t) % 3]
123         novelty = previous.sum()-previous
124         row = dict(household_code=str(i), week=pd.Timestamp('2020-01-04')+pd.Timedelta(weeks=t), outside=0.)
125         for j, f in enumerate(pricing.FLAVORS):
126             row.update({f: counts[j], 'price_'+f: 1.+j/2,
127                         'novelty_'+f: novelty[j], 'stock_'+f: stock[j]})
128         rows.append(row)
129         stock += counts
130         previous = counts
```

Read this block:

- Line 118: for each entry yielded by `range(8)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 119, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 120, `previous`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 121: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `counts`: Set this name to the entry/slice selected by `np.eye(3)[(i + t) % 3]`; brackets select from the object to their left.
- Line 123, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 124, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 125: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 126: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 128: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 129: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 130, `previous`: Set this name to the object currently named `counts`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `str`: Convert to text for paths, messages, or serialization.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 131–133:

```
131 parameters = dict(inside_constant=.5, beta_berry=.2, beta_plain=-.1,
132                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
133                   mean_novelty=-.2, sd_novelty=.7, mean_own_stock=.1, sd_own_stock=.3)
```

Read this block:

- Line 131, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 134–134:

```
134 return pd.DataFrame(rows), parameters
```

Read this block:

- Line 134: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

SplitPosteriorTests.test posterior matches scalar and estimator

Original function: `SplitPosteriorTests.test_posterior_matches_scalar_and_estimator`. Source begins at line 136.

Compares persistent split-draw likelihoods and posterior weights against a scalar calculation and the demand estimator, and verifies exact binary aggregation by an outside inclusive value. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 136–136:

```
136 def test_posterior_matches_scalar_and_estimator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 137–140:

```
137 frame, p = self.fixture()
138 draws = pricing.persistent_draws(8, 306)
139 households, weights, alpha, history, beta, nll = pricing.posterior(frame, p, draws, False)
140 scalar = []
```

Read this block:

- Line 137, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 138, `draws`: Set this name to the result of calling `pricing.persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 139, `(households, weights, alpha, history, beta, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 140, `scalar`: Set this name to the list `[]`.

Source lines 141–151:

```
141 for _, block in frame.groupby('household_code', sort=True):
142     logs = []
143     for a, (b_n, b_s) in zip(alpha, history):
144         ll = 0.
145         for _, row in block.iterrows():
146             u = [beta[j]-a*row['price_'+f]+b_n*row['novelty_'+f]+b_s*row['stock_'+f]
147                  for j, f in enumerate(pricing.FLAVORS)]+[0.]
148             probs = np.exp(u-pricing.logsumexp(u))
149             ll += sum(row[f]*np.log(probs[j]) for j, f in enumerate(pricing.FLAVORS+['outside']))
150         logs.append(ll)
151     scalar.append(logs)
```

Read this block:

- Line 141: for each entry yielded by `frame.groupby('household_code', sort=True)`, bind/unpack it into `(_, block)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 142, `logs`: Set this name to the list `[]`.
- Line 143: for each entry yielded by `zip(alpha, history)`, bind/unpack it into `(a, (b_n, b_s))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `ll`: Set this name to the literal value `0.0`.
- Line 145: for each entry yielded by `block.iterrows()`, bind/unpack it into `(_, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 146, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 148, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 149: update `ll` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 150: call `logs.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 151: call `scalar.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 152–157:

```
152 scalar = np.array(scalar)
153 expected = np.exp(scalar-pricing.logsumexp(scalar, axis=1, keepdims=True))
154 np.testing.assert_allclose(weights, expected, atol=1e-13)
155 self.assertAlmostEqual(nll, -sum(pricing.logsumexp(row)-np.log(len(draws)) for row in scalar), places=12)
156 np.testing.assert_allclose(households.habit_mean, weights @ history[:, 1])
157 from demand import estimation as est
```

Read this block:

- Line 152, `scalar`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 153, `expected`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 154: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 155: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 156: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 157: from `demand`, import `estimation as est` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `len`: Count elements/rows rather than summing their numerical values.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 158–159:

```
158 for f in pricing.FLAVORS:
159     frame["xi_"+f] = .3*frame["novelty_"+f]-.7*frame["stock_"+f]
```

Read this block:

- Line 158: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 159, `frame['xi_' + f]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 160–167:

```
160 data = est.make_arrays(frame, 1)
161 names = est.parameter_names("heterogeneous_history", True, False)
162 estimator_draws = est.make_draws(8, 306, "heterogeneous_history")
163 np.testing.assert_array_equal(draws, estimator_draws)
164 expected_ll = est.household_loglike(np.array([p[n] for n in names]), data,
165                                     estimator_draws, "heterogeneous_history")
166 np.testing.assert_allclose(pricing.logsumexp(scalar, axis=1)-np.log(len(draws)),
167                            expected_ll, atol=1e-12)
```

Read this block:

- Line 160, `data`: Set this name to the result of calling `est.make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 161, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 162, `estimator_draws`: Set this name to the result of calling `est.make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 163: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 164, `expected_ll`: Set this name to the result of calling `est.household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 166: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.

Source lines 169–170:

```
169 u = np.array([.3, -.2, .7, 0.])
170 probs = np.exp(u-pricing.logsumexp(u))
```

Read this block:

- Line 169, `u`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 170, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Source lines 171–173:

```
171 for focal in range(3):
172     inclusive = pricing.logsumexp(np.delete(u, focal))
173     self.assertAlmostEqual(pricing.expit(u[focal]-inclusive), probs[focal])
```

Read this block:

- Line 171: for each entry yielded by `range(3)`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 172, `inclusive`: Set this name to the result of calling `pricing.logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 173: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.test three focal cases both type treatments

Original function: `SplitPosteriorTests.test_three_focal_cases_both_type_treatments`. Source begins at line 175.

Checks all focal permutations under known posterior means and persistent posterior uncertainty. Generated arrays preserve probability mass and dynamic revenue weakly dominates no coupons; the simple runner blocks all CSV reads. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 175–175:

```
175 def test_three_focal_cases_both_type_treatments(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 176–178:

```
176 frame, parameters = self.fixture()
177 households, weights, alpha, history, beta, _ = pricing.posterior(
178     frame, parameters, pricing.persistent_draws(8, 306), False)
```

Read this block:

- Line 176, `(frame, parameters)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 177, `(households, weights, alpha, history, beta, _)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 179–193:

```
179 for focal in pricing.FLAVORS:
180     forecast = frame.copy()
181     intercepts = beta.copy()
182     if focal != 'berry':
183         for destination, source in {'berry': focal, focal: 'berry'}.items():
184             for prefix in ['', 'price_', 'stock_', 'novelty_']:
185                 forecast[prefix+destination] = frame[prefix+source]
186             intercepts[pricing.FLAVORS.index(destination)] = beta[pricing.FLAVORS.index(source)]
187     context = pricing.forecast_context(forecast, households.copy())
188     for treatment in ['posterior-mean', 'posterior-integrated']:
189         table, error = pricing.targeted_coupons(context, households, weights, alpha,
190             history, intercepts, np.array([0., .5, 1.]), capacity=2,
191             delta=.92, type_treatment=treatment)
192         self.assertLess(error, 1e-12)
193         self.assertTrue((table.revenue_change >= -1e-10).all())
```

Read this block:

- Line 179: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 180, `forecast`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 181, `intercepts`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.
- Line 182: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 183: for each entry yielded by `{'berry': focal, focal: 'berry'}.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 184: for each entry yielded by `['', 'price_', 'stock_', 'novelty_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 185, `forecast[prefix + destination]`: Store the entry/slice selected by `frame[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 186, `intercepts[pricing.FLAVORS.index(destination)]`: Store the entry/slice selected by `beta[pricing.FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.
- Line 187, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 188: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 189, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 192: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PlainBerryComparisonTests.test only focal sellers change and full demand is retained

Original function: `PlainBerryComparisonTests.test_only_focal_sellers_change_and_full_demand_is_retained`. Source begins at line 197.

Mocks the household solver and captures the comparison table in memory. It checks there are exactly two calls, plain and berry, with the original fitted-results and household-input paths, no other focal folder, all three demand categories in the run record, and an unchanged caller output path. No empirical inputs or CSV files are read.

Definition and stated intent:

Source lines 197–197:

```
197 def test_only_focal_sellers_change_and_full_demand_is_retained(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 198–202:

```
from demand import plain_berry_pricing as comparison
from types import SimpleNamespace
from pathlib import Path
from unittest.mock import patch
import tempfile
```

Read this block:

- Line 198: from `demand`, import `plain_berry_pricing as comparison` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 199: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 200: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 201: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 202: import `tempfile`. An as-name is a convenient local alias, not another library.

Source lines 203–223:

```
with tempfile.TemporaryDirectory() as directory:
    settings = SimpleNamespace(household_input=Path('/server/weekly_inputs.csv'),
        fit_dir=Path('/server/split_fit'), output=Path(directory),
        discounts=np.array([0., .5, 1.]), demand_delta=.92,
        type_treatment='posterior-mean', batch_size=24, save_state_policies=False)
    def solve(case):
        self.assertEqual(case.household_input, settings.household_input)
        self.assertEqual(case.fit_dir, settings.fit_dir)
        return dict(aggregate=[dict(policy='personalized_dynamic', discounted_revenue=1.)])
    captured = []
    def capture(frame, path, **kwargs):
        captured.append(frame.copy())
    with patch.object(comparison.pricing, 'run_households', side_effect=solve) as solver, \
            patch.object(pd.DataFrame, 'to_csv', capture):
        manifest = comparison.run_comparison(settings)
    self.assertEqual([call.args[0].focal_flavor for call in solver.call_args_list], ['plain', 'berry'])
    self.assertEqual(set(captured[0].focal_flavor), {'plain', 'berry'})
    self.assertEqual(manifest['demand_categories'], ['other', 'berry', 'plain'])
    self.assertEqual(set(manifest['results']), {'plain', 'berry'})
    self.assertFalse((settings.output/'other').exists())
    self.assertEqual(settings.output, Path(directory))
```

Read this block:

- Line 203: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 204, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 208: define nested callable `solve`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 212, `captured`: Set this name to the list `[]`.
- Line 213: define nested callable `capture`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 215: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 217, `manifest`: Set this name to the result of calling `comparison.run_comparison(...)`; the arguments and preceding method chain are shown above.
- Line 218: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 219: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 220: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 221: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 222: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 223: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

PlainBerryComparisonTests.test only focal sellers change and full demand is retained.solve

Original function: `PlainBerryComparisonTests.test_only_focal_sellers_change_and_full_demand_is_retained.solve`. Source begins at line 208.

This nested mock supplies a tiny aggregate result instead of reading household data. Its assertions ensure both focal cases receive the same full fitted-results and weekly-input paths. The surrounding test checks which focal products were requested.

Definition and stated intent:

Source lines 208–208:

```
208 def solve(case):
```

`def` creates this callable; the parentheses name its inputs (`case`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 209–210:

```
209 self.assertEqual(case.household_input, settings.household_input)
210 self.assertEqual(case.fit_dir, settings.fit_dir)
```

Read this block:

- Line 209: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 210: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

Source lines 211–211:

```
211 return dict(aggregate=[dict(policy='personalized_dynamic', discounted_revenue=1.)])
```

Read this block:

- Line 211: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

PlainBerryComparisonTests.test only focal sellers change and full demand is retained.capture

Original function: `PlainBerryComparisonTests.test_only_focal_sellers_change_and_full_demand_is_retained.capture`. Source begins at line 213.

Captures the DataFrame passed to to_csv in a Python list rather than creating or reopening a CSV. This makes it possible to verify the two comparison labels while the test runner blocks CSV readers. The helper is a test substitute, not a production export routine.

Definition and stated intent:

Source lines 213–213:

```
213 def capture(frame, path, **kwargs):
```

`def` creates this callable; the parentheses name its inputs (`frame`, `path`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**kwargs` collects additional keyword arguments in a dictionary.

Source lines 214–214:

```
214 captured.append(frame.copy())
```

Read this block:

- Line 214: call `captured.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/checks/test_regenerate_weekly_inputs.py

Mocked regeneration checks. Synthetic records supply saved settings and parameters; mocked preparation/CSV loading allows the checks to run while CSV readers are blocked. Success preserves fitted artifacts and a likelihood mismatch preserves any old export.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Recovery control flow tests with generated fixtures only."""
import json
from pathlib import Path
import tempfile
import unittest
from unittest.mock import patch
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `json`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–10:

```
import numpy as np
import pandas as pd
from demand import regenerate_weekly_inputs as recovery
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: from `demand`, import `regenerate_weekly_inputs as recovery` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–13:

```
class RecoveryTests(unittest.TestCase):
```

Read this block:

- Line 13: define class `RecoveryTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 54–55:

```
if __name__ == '__main__':
    unittest.main()
```

Read this block:

- Line 54: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 55: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

RecoveryTests.check recovery

Original function: `RecoveryTests.check_recovery`. Source begins at line 14.

Builds temporary synthetic fit records and mocks data preparation/CSV loading. It checks successful publication or a likelihood mismatch, and verifies saved fitted parameters are preserved. The mock is a controlled substitute, not evidence that an actual server dataset has already matched.

Definition and stated intent:

Source lines 14–14:

```
def check_recovery(self, mismatch):
```

`def` creates this callable; the parentheses name its inputs (`self`, `mismatch`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–45:

```
with tempfile.TemporaryDirectory() as temporary:
    directory = Path(temporary)
    fit = dict(model='heterogeneous_history', households=1, weeks=2, nll=3., settings=dict(
        sample='/server/synthetic.parquet', first_year=2006, last_year=2015,
        max_households=None, capacity=7, no_price_control=False,
        household_dma_column='dma_cd', habit_weight=.7, draws=4, seed=306))
    (directory/'fit.json').write_text(json.dumps(fit))
    names = recovery.parameter_names('heterogeneous_history', True, True)
    pd.DataFrame(dict(parameter=names, estimate=np.ones(len(names)))).to_csv(directory/'point_estimates.csv',
    index=False)
    before = {p.name: p.read_bytes() for p in directory.iterdir()}
    frame = pd.DataFrame({'synthetic_row': [1, 2]})
    data = dict(n_households=1, outside=True, control=True)
    with patch.object(recovery, 'prepare_data', return_value=frame) as prepare, \
            patch.object(recovery, 'add_price_controls', return_value=(frame, None)) as controls, \
            patch.object(recovery, 'make_arrays', return_value=data), \
            patch.object(recovery, 'household_loglike', return_value=np.array([-4. if mismatch else -3.])), \
            patch.object(recovery.pd, 'read_csv', return_value=pd.DataFrame(dict(parameter=names,
    estimate=np.ones(len(names))))):
        if mismatch:
            (directory/'weekly_inputs.csv').write_text('existing export')
            with self.assertRaisesRegex(ValueError, 'likelihood'):
                recovery.regenerate(directory)
            self.assertEqual((directory/'weekly_inputs.csv').read_text(), 'existing export')
            self.assertFalse((directory/'weekly_inputs_regeneration.json').exists())
        else:
            report = recovery.regenerate(directory)
            self.assertFalse(report['demand_refitted'])
            self.assertTrue((directory/'weekly_inputs.csv').exists())
        controls.assert_called_once()
        self.assertEqual(prepare.call_args.args[0].sample, Path('/server/synthetic.parquet'))
    for name, content in before.items():
        self.assertEqual((directory/name).read_bytes(), content)
```

Read this block:

- Line 15: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 16, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 21: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22, `names`: Set this name to the result of calling `recovery.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `before`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 25, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

- Line 26, `data`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 27: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 32: test `mismatch`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 33: call `(directory / 'weekly_inputs.csv').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 35: call `recovery.regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `report`: Set this name to the result of calling `recovery.regenerate(...)`; the arguments and preceding method chain are shown above.
- Line 40: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `controls.assert_called_once` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: for each entry yielded by `before.items()`, bind/unpack it into `(name, content)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 45: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test recovery preserves parameters

Original function: `RecoveryTests.test_recovery_preserves_parameters`. Source begins at line 47.

Exercises successful weekly-export regeneration with mocks and confirms fit/parameter records remain byte-for-byte unchanged. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 47–47:

```
47 def test_recovery_preserves_parameters(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 48–48:

```
48 self.check_recovery(False)
```

Read this block:

- Line 48: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test mismatch preserves existing export

Original function: `RecoveryTests.test_mismatch_preserves_existing_export`. Source begins at line 50.

Forces a reconstructed-likelihood mismatch and checks no replacement weekly export or success record is published. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 50–50:

```
50 def test_mismatch_preserves_existing_export(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–51:

```
51 self.check_recovery(True)
```

Read this block:

- Line 51: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/checks/check_flavor_specifications.py

Pure numerical and descriptor-string verification. It checks original likelihood nesting, gradients with five active choices, all basket states/mass, classification agreement, calibration recovery and identification rejection, without empirical I/O or household panel fixtures.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Numerical kernel checks only. No empirical I/O or household record fixtures."""
from pathlib import Path
import importlib.util
import sys
import math
import numpy as np
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `importlib.util`. An as-name is a convenient local alias, not another library.
- Line 4: import `sys`. An as-name is a convenient local alias, not another library.
- Line 5: import `math`. An as-name is a convenient local alias, not another library.
- Line 6: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 7–12:

```
import polars as pl
from scipy.optimize import approx_fprime

base=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(base))
sys.path.insert(0,str(base/'demand'))
```

Read this block:

- Line 7: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 8: from `scipy.optimize`, import `approx_fprime` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10, `base`: Set this name to the entry/slice selected by `Path(__file__).resolve().parents[1]`; brackets select from the object to their left.
- Line 11: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 12: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 20–25:

```
from flavor_specs import categories, classify_descriptor, classification_expression
cases=[('plain','plain'),('strawberry','berry'),('peach','other_fruits'),
       ('vanilla','desserts'),('chocolate','desserts'),('strawberry cheesecake','desserts'),
       ('banana cream pie','desserts'),('mango pineapple','other_fruits'),
       ('cherry','other_fruits'),('black cherry','other_fruits'),('blackberries','berry'),
       ('coffee','other'),('lemon tart','desserts'),('mixed berry','berry')]
```

Read this block:

- Line 20: from `flavor_specs`, import `categories, classify_descriptor, classification_expression` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 21, `cases`: Set this name to a list containing 14 displayed entries.

Source lines 26–29:

```
for text,expected in cases:
    assert classify_descriptor(text,'five')==expected,(text,expected)
    observed=pl.select(classification_expression(pl.lit(text),'five')).item()
    assert observed==expected,(text,observed)
```

Read this block:

- Line 26: for each entry yielded by `cases`, bind/unpack it into `(text, expected)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27: require `classify_descriptor(text, 'five') == expected` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 28, `observed`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 29: require `observed == expected` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `classify_descriptor`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `classification_expression`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

Source lines 30–30:

```
assert classify_descriptor('vanilla','three')=='other'
```

Read this block:

- Line 30: require `classify_descriptor('vanilla', 'three') == 'other'` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 31–31:

```
assert classify_descriptor('strawberry cheesecake','three')=='berry'
```

Read this block:

- Line 31: require `classify_descriptor('strawberry cheesecake', 'three') == 'berry'` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 32–32:

```
assert classify_descriptor('peach','three')=='other'
```

Read this block:

- Line 32: require `classify_descriptor('peach', 'three') == 'other'` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 34–35:

```
34 est=module('expanded_estimation',base/'demand/estimation.py')
35 price=module('expanded_split_pricing',base/'demand/split_pricing.py')
```

Read this block:

- Line 34, `est`: Set this name to the result of calling `module(...)`; the arguments and preceding method chain are shown above.
- Line 35, `price`: Set this name to the result of calling `module(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `module`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 54–54:

```
54 results=[]
```

Read this block:

- Line 54, `results`: Set this name to the list `[]`.

Source lines 55–81:

```
55 for spec in ['three','five']:
56     flavors=categories(spec)
57     est.FLAVORS[:]=flavors
58     price.FLAVORS[:]=flavors
59     data=mathematical_data(len(flavors))
60     names=est.parameter_names('heterogeneous_history',True,False)
61     parameter={'inside_constant':1.,'beta_berry':-.4,'beta_plain':-.2,
62                'mu_log_minus_alpha':-.1,'sd_log_minus_alpha':.25,
63                'mean_novelty':-.6,'sd_novelty':.3,'mean_own_stock':.05,'sd_own_stock':.1}
64     theta=np.array([parameter.get(n,0.) for n in names])
65     draws=est.make_draws(8,306,'heterogeneous_history')
66     value,gradient=est.demand_objective_gradient(theta,data,draws,'heterogeneous_history')
67     numerical=approx_fprime(theta,lambda
   ↪ t:est.demand_objective_gradient(t,data,draws,'heterogeneous_history')[0],1e-6)
68     np.testing.assert_allclose(gradient,numerical,atol=2e-6,rtol=2e-5)
69     np.testing.assert_allclose(value,-est.household_loglike(theta,data,draws,'heterogeneous_history').mean())
70     results.append((value,{name:gradient[k] for k,name in enumerate(names)}))
71     # Check basket coverage, uniqueness, and multinomial probability normalization.
72     for capacity in [1,3,7]:
73         baskets=price.compositions(capacity)
74         assert len(baskets)==math.comb(capacity+len(flavors),len(flavors))
75         assert len(np.unique(baskets,axis=0))==len(baskets)
76         assert np.all(baskets>=0) and np.all(baskets.sum(axis=1)==capacity)
77         probabilities=np.arange(1,len(flavors)+2,dtype=float)
78         probabilities/=probabilities.sum()
79         logcoef=math.lgamma(capacity+1)-np.vectorize(math.lgamma)(baskets+1).sum(axis=1)
80         mass=np.exp(logcoef+(baskets*np.log(probabilities)).sum(axis=1)).sum()
81         np.testing.assert_allclose(mass,1.,atol=1e-12)
```

Read this block:

- Line 55: for each entry yielded by `['three', 'five']`, bind/unpack it into `spec` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 56, `flavors`: Set this name to the result of calling `categories(...)`; the arguments and preceding method chain are shown above.
- Line 57, `est.FLAVORS[:]`: Store the object currently named `flavors`; plain assignment does not copy it in the selected entry/column.

- Line 58, `price.FLAVORS[:]`: Store the object currently named `flavors`; plain assignment does not copy it in the selected entry/column.
- Line 59, `data`: Set this name to the result of calling `mathematical_data(...)`; the arguments and preceding method chain are shown above.
- Line 60, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 61, `parameter`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `mean_novelty`, `sd_novelty`, `mean_own_stock`.
- Line 64, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 65, `draws`: Set this name to the result of calling `est.make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 66, `(value, gradient)`: Unpack the result of calling `est.demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 67, `numerical`: Set this name to the result of calling `approx_fprime(...)`; the arguments and preceding method chain are shown above.
- Line 68: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 69: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 70: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: for each entry yielded by `[1, 3, 7]`, bind/unpack it into `capacity` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73, `baskets`: Set this name to the result of calling `price.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 74: require `len(baskets) == math.comb(capacity + len(flavors), len(flavors))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 75: require `len(np.unique(baskets, axis=0)) == len(baskets)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 76: require `np.all(baskets >= 0) and np.all(baskets.sum(axis=1) == capacity)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 77, `probabilities`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 78: update `probabilities` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 79, `logcoef`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80, `mass`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `categories`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `mathematical_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `approx_fprime`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `len`: Count elements/rows rather than summing their numerical values.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 83–88:

```
83 active=mathematical_data(5)
84 active['available'][:]=True
85 active['prices'][:,3:]=[[1.7,1.8],[1.6,1.9]]
86 active['counts'][:,3:5]=[[1.,2.],[2.,1.]]
87 active['stock'][:,3:]=[[.2,.3],[.8,.4]]
88 active['novelty'][:,3:]=[[.9,.8],[.7,.6]]
```

Read this block:

- Line 83, `active`: Set this name to the result of calling `mathematical_data(...)`; the arguments and preceding method chain are shown above.
- Line 84, `active['available'][:]`: Store the literal value `True` in the selected entry/column.
- Line 85, `active['prices'][:, 3:]`: Store a list containing 2 displayed entries in the selected entry/column.
- Line 86, `active['counts'][:, 3:5]`: Store a list containing 2 displayed entries in the selected entry/column.
- Line 87, `active['stock'][:, 3:]`: Store a list containing 2 displayed entries in the selected entry/column.
- Line 88, `active['novelty'][:, 3:]`: Store a list containing 2 displayed entries in the selected entry/column.

Source lines 89–93:

```
89 names=est.parameter_names('heterogeneous_history',True,False)
90 theta=np.array([parameter.get(n,-.15 if n.startswith('beta_') else 0.) for n in names])
91 value,gradient=est.demand_objective_gradient(theta,active,draws,'heterogeneous_history')
92 numerical=approx_fprime(theta,lambda
   ↪  t:est.demand_objective_gradient(t,active,draws,'heterogeneous_history')[0],1e-6)
93 np.testing.assert_allclose(gradient,numerical,atol=3e-6,rtol=3e-5)
```

Read this block:

- Line 89, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 90, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 91, `(value, gradient)`: Unpack the result of calling `est.demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 92, `numerical`: Set this name to the result of calling `approx_fprime(...)`; the arguments and preceding method chain are shown above.
- Line 93: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 94–94:

```
94 assert abs(gradient[names.index('beta_other_fruits')])>1e-5
```

Read this block:

- Line 94: require `abs(gradient[names.index('beta_other_fruits')]) > 1e-05` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 95–95:

```
95 assert abs(gradient[names.index('beta_desserts')])>1e-5
```

Read this block:

- Line 95: require `abs(gradient[names.index('beta_desserts')]) > 1e-05` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 96–96:

```
96 np.testing.assert_allclose(results[0][0],results[1][0],atol=1e-12)
```

Read this block:

- Line 96: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 97–98:

```
97 for name,value in results[0][1].items():
98     np.testing.assert_allclose(value,results[1][1][name],atol=1e-12)
```

Read this block:

- Line 97: for each entry yielded by `results[0][1].items()`, bind/unpack it into `(name, value)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 98: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 100–104:

```
100 cal=module('expanded_gamma_calibration',base/'demand/gamma_calibration.py')
101 # Aggregate theoretical price moments identify a known interior calibration.
102 import pandas as pd
103 rows=[]
104 gamma=np.array([-1.,0.,1.])
```

Read this block:

- Line 100, `cal`: Set this name to the result of calling `module(...)`; the arguments and preceding method chain are shown above.
- Line 102: import `pandas as pd`. An as-name is a convenient local alias, not another library.

- Line 103, `rows`: Set this name to the list `[]`.
- Line 104, `gamma`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Source lines 105–108:

```
for group,g in zip(cal.GROUPS,gamma):
    for h in ['j','0']:
        p=cal.known_prices(g,h,-1.2,2.,.92)

    rows.append(dict(group=group,h0=h,households=100,observed_p1=p['expected_p1'],observed_p2=p['expected_p2']))
```

Read this block:

- Line 105: for each entry yielded by `zip(cal.GROUPS, gamma)`, bind/unpack it into `(group, g)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 106: for each entry yielded by `['j', '0']`, bind/unpack it into `h` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 107, `p`: Set this name to the result of calling `cal.known_prices(...)`; the arguments and preceding method chain are shown above.
- Line 108: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 109–110:

```
a,b,_,rms=cal.calibrate(pd.DataFrame(rows),gamma,.92)
np.testing.assert_allclose([a,b],[-1.2,2.],atol=1e-6)
```

Read this block:

- Line 109, `(a, b, _, rms)`: Unpack the result of calling `cal.calibrate(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 110: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 111–111:

```
assert rms<1e-8
```

Read this block:

- Line 111: require `rms < 1e-08` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 114–115:

```
flat=pd.DataFrame([dict(group=g,h0=h,households=100,observed_p1=2.,observed_p2=2.)
                   for g in cal.GROUPS for h in ['j','0']])
```

Read this block:

- Line 114, `flat`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Source lines 116–122:

```
try:
    cal.calibrate(flat,np.zeros(3),.92)
except cal.CalibrationError as error:
```

```
119     assert error.diagnostics['status']=='failed'
120     assert len(error.diagnostics['attempts'])==3
121 else:
122     raise AssertionError('Unidentified calibration was accepted')
```

Read this block:

- Line 116: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 117: call `cal.calibrate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 122: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 118: catch `cal.CalibrationError` from the preceding try; the optional as-name holds the error object for logging.
- Line 119: require `error.diagnostics['status'] == 'failed'` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 120: require `len(error.diagnostics['attempts']) == 3` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 123–123:

```
123 print('PASS: descriptor precedence; Polars/Python agreement; three/five gradients; legacy likelihood equivalence;
    ↪  six basket-mass checks; interior calibration recovery and unidentified-calibration rejection.')
```

Read this block:

- Line 123: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

module

Original function: `module`. Source begins at line 14.

Loads a source module for pure mathematical checks using an import specification. This registers functions without invoking guarded empirical run steps; only numerical kernels are called by this check.

Definition and stated intent:

Source lines 14–14:

```
14 def module(name, path):
```

`def` creates this callable; the parentheses name its inputs (`name`, `path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–17:

```
15 specification=importlib.util.spec_from_file_location(name,path)
16 result=importlib.util.module_from_spec(specification)
17 specification.loader.exec_module(result)
```

Read this block:

- Line 15, `specification`: Set this name to the result of calling `importlib.util.spec_from_file_location(...)`; the arguments and preceding method chain are shown above.
- Line 16, `result`: Set this name to the result of calling `importlib.util.module_from_spec(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `specification.loader.exec_module` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 18–18:

```
18 return result
```

Read this block:

- Line 18: return the object currently named `result`; plain assignment does not copy it. This ends the call.

mathematical data

Original function: `mathematical_data`. Source begins at line 38.

Constructs aggregate multinomial count and utility arrays for an abstract likelihood calculation. The five-column version appends unavailable alternatives to check nesting; no household identifiers, purchase histories, or research records are constructed or persisted.

Definition and stated intent:

Source lines 38–38:

```
38 def mathematical_data(size):
```

`def` creates this callable; the parentheses name its inputs (`size`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 39–43:

```
39 counts=np.array([[2.,1.,1.,3.],[1.,2.,0.,4.]])
40 prices=np.array([[1.2,1.5,1.1],[1.3,1.4,1.2]])
41 stock=np.array([[.4,1.1,.5],[1.2,.3,.5]])
42 novelty=np.array([[.8,.3,.9],[.4,.9,.7]])
43 available=np.ones((2,3),dtype=bool)
```

Read this block:

- Line 39, `counts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 40, `prices`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 41, `stock`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 42, `novelty`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 43, `available`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.

Source lines 44–49:

```
44 if size==5:
45     counts=np.column_stack([counts[:,:3],np.zeros((2,2)),counts[:,-1]])
46     prices=np.column_stack([prices,np.zeros((2,2))])
```

```
47      stock=np.column_stack([stock,np.zeros((2,2))])
48      novelty=np.column_stack([novelty,np.zeros((2,2))])
49      available=np.column_stack([available,np.zeros((2,2),dtype=bool)])
```

Read this block:

- Line 44: test `size == 5`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45, `counts`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 46, `prices`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 47, `stock`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 48, `novelty`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 49, `available`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 50–52:

```
50  return dict(counts=counts,prices=prices,stock=stock,novelty=novelty,
51              xi=.3*novelty-.7*stock,available=available,cf=np.zeros_like(prices),
52              control=False,hh=np.array([0,0]),n_households=1,outside=True)
```

Read this block:

- Line 50: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Coverage, rebuilding, and next steps

This edition includes 113 Python files, 409 named functions/methods (including nested helpers), and 13,969 source lines. Every AST statement start is accounted for in the annotated listings. Empty files and top-level scripts are included. Lambda expressions are explained at their use sites rather than counted as named functions. Large raw LaTeX strings are reproduced as data; their mathematical content is discussed in the separate economic guide.

The build reads Python files as text and parses their syntax; it never imports or executes the private-data processing scripts. The companion JSON manifest records source hashes, function names/lines, and statement coverage. If source changes, rebuild the book so printed line references match the new snapshot. A newly added function requires a teaching note before the build succeeds.

Rebuild from the repository's Code directory:

```
python3 final_draft/build_python101.py
```

Use `--markdown-only` to regenerate the readable source/coverage manifest without invoking Pandoc/XeLaTeX. The source lessons are in `final_draft/python101/`. The generated Markdown is `final_draft/PYTHON101_WALKTHROUGH.md`.

Keep this book beside the separate `complete_code_guides.pdf`: use that guide for model derivations and this one for what the corresponding Python actually says. Try changing tiny test fixtures or tracing values on paper before editing the large server pipelines.